

WET2VO Web Technologies

Vorlesungsunterlagen

Wolfgang Hochleitner
wolfgang.hochleitner@fh-hagenberg.at

SS 2026

Inhaltsverzeichnis

1 Grundlagen der Kommunikation im Web	1-1
1.1 Zur Web Technologies Vorlesung	1-1
1.1.1 Organisatorisches	1-1
1.1.2 Inhalte der Vorlesung	1-2
1.2 Der Webrequest	1-2
1.3 Der URL als Adressierungsschema	1-3
1.3.1 Aufbau eines HTTP-URLs	1-4
1.4 Das Client-Server-Modell	1-5
1.4.1 Proxies, Reverse Proxies und Firewalls	1-6
1.5 Schichtenmodelle: OSI und TCP/IP	1-7
1.5.1 Das OSI-Referenzmodell	1-7
1.5.2 Das TCP/IP-Modell in der Praxis	1-8
1.5.3 Encapsulation: Wie Schichten zusammenspielen	1-8
1.6 Vom Namen zur Adresse: DNS	1-9
1.6.1 Hostnamen, Domains und FQDN	1-9
1.6.2 Ablauf der Namensauflösung	1-10
1.7 Die Netzwerkschicht: IP und lokale Zustellung	1-10
1.7.1 IP-Adressen	1-11
1.7.2 Unicast, Broadcast und Multicast	1-12
1.7.3 Private Adressen und NAT	1-12
1.7.4 DHCP: Automatische Netzwerkkonfiguration	1-13
1.7.5 ARP: Vom IP zum MAC	1-13
1.7.6 Ethernet-Frames und lokale Zustellung	1-14
1.7.7 Netzwerkkomponenten: Hub, Switch, Router	1-14
1.7.8 Diagnose: ICMP, ping und traceroute	1-15
1.8 Die Transportschicht: TCP	1-15
1.8.1 TCP und UDP	1-16
1.8.2 Der Three-Way-Handshake	1-16
1.9 Sichere Übertragung: HTTPS und TLS	1-17
1.9.1 HTTPS ist HTTP über TLS	1-17
1.9.2 Der TLS-Handshake	1-18
1.9.3 Zertifikate und Zertifizierungsstellen	1-18
1.10 HTTP: Das Protokoll des Web	1-19
1.10.1 Nachrichtenformat	1-19
1.10.2 HTTP-Methoden	1-20

1.10.3	Statuscodes	1-20
1.10.4	Wichtige Headerfelder	1-21
1.10.5	HTTP-Versionen im Überblick	1-21
1.10.6	HTTP ist zustandslos	1-22
1.11	Der Gesamtablauf im Überblick	1-22
1.11.1	Brücke zur serverseitigen Entwicklung	1-23
1.12	Weiterführende Literatur	1-24
2	PHP-Grundlagen Teil 1	2-1
2.1	Allgemeines zu PHP	2-1
2.1.1	Geschichte	2-1
2.1.2	Alternativen	2-3
2.1.3	Verbreitung und Beispiele	2-5
2.2	Serverseitige Webentwicklung	2-5
2.2.1	Wozu überhaupt?	2-5
2.2.2	Funktionsweise	2-6
2.2.3	Lokaler Webserver mit Docker	2-6
2.3	PHP-Sprachgrundlagen	2-8
2.3.1	Einbettung in HTML	2-8
2.3.2	Ausgabe von Strings	2-10
2.3.3	Lexikalische Struktur	2-11
2.3.4	Datentypen	2-12
2.3.5	Variablen	2-14
2.3.6	Operatoren	2-16
2.3.7	Kontrollstrukturen	2-20
2.3.8	Advanced Escaping	2-23
2.3.9	Strings und Stringfunktionen	2-23
2.4	Weiterführende Literatur	2-30
3	PHP-Grundlagen Teil 2 und Regular Expressions	3-1
3.1	Funktionen	3-1
3.1.1	Funktionen definieren (benannte Funktionen)	3-1
3.1.2	Funktionsparameter	3-2
3.1.3	Funktionen aufrufen	3-3
3.1.4	Default-Parameter	3-3
3.1.5	Anonyme Funktionen und Arrow Functions	3-4
3.1.6	Der Pipe-Operator	3-5
3.1.7	Typ- und Rückgabewertsdeklarationen	3-5
3.1.8	Gültigkeitsbereich von Variablen	3-8
3.2	Konstanten	3-9
3.2.1	Benannte Konstanten	3-10
3.2.2	Vordefinierte Konstanten	3-11
3.3	Arrays	3-12
3.3.1	Erzeugen von Arrays	3-13
3.3.2	Zugriff auf Array-Elemente	3-13
3.3.3	Erweitern von Arrays	3-14

3.3.4	Arrays durchlaufen	3-14
3.4	Funktionen der PHP-Bibliothek	3-15
3.4.1	Funktionen für Variablen	3-15
3.4.2	Funktionen für Arrays	3-17
3.5	Formulardaten verarbeiten	3-19
3.5.1	Vordefinierte Variablen	3-19
3.5.2	Übergabe von Werten	3-20
3.5.3	Übergabe von Mehrfachauswahl	3-22
3.6	Regular Expressions	3-23
3.6.1	Regex-Funktionen	3-24
3.6.2	Muster definieren	3-25
3.6.3	Muster-Übersicht	3-29
3.7	Weiterführende Literatur	3-29
4	Objektorientiertes PHP	4-1
4.1	Modularisierung	4-1
4.1.1	Dateien inkludieren	4-1
4.2	Objektorientierung in PHP	4-3
4.2.1	Allgemeines und Terminologie	4-3
4.2.2	Objekte erzeugen	4-4
4.2.3	Zugriff auf Objekteigenschaften und -methoden	4-5
4.2.4	Klassen deklarieren	4-5
4.2.5	Interfaces	4-14
4.2.6	Vererbung	4-16
4.2.7	Abstrakte Klassen	4-18
4.2.8	Traits	4-19
4.2.9	Enums (Aufzählungen)	4-21
4.2.10	Exceptions	4-23
4.2.11	Namespaces	4-24
4.3	Autoloading	4-26
4.3.1	Autoloading an einem Beispiel erklärt	4-27
4.4	Weiterführende Literatur	4-28
5	Komponenten und Standards	5-1
5.1	Allgemeines zu Komponenten	5-1
5.1.1	Von DIY zu Frameworks	5-1
5.1.2	PHP-Komponenten	5-2
5.2	Komponenten verwalten: Composer & Packagist	5-3
5.2.1	Composer	5-3
5.2.2	Packagist Repository	5-4
5.2.3	Packages updaten	5-8
5.2.4	Autoloading	5-8
5.2.5	Beispielprojekt: Custom Chuck Norris Fact	5-9
5.2.6	Eigene Komponenten definieren	5-11
5.3	Standards in PHP: PHP-FIG	5-12
5.3.1	Allgemeines zu Standards	5-12

5.3.2	PSR-1	5-13
5.3.3	PER Coding Style	5-14
5.3.4	PSR-4 und PSR-0	5-16
5.3.5	Weitere PSRs	5-16
5.3.6	Standards einhalten	5-17
5.4	Weiterführende Literatur	5-21
6	Templates und Routing	6-1
6.1	Template-Systeme	6-1
6.1.1	Allgemeines	6-1
6.1.2	Bekannte Template-Engines	6-2
6.1.3	Latte Template Engine	6-2
6.2	Routing	6-8
6.2.1	Allgemeines	6-8
6.2.2	Funktionsweise	6-9
6.2.3	Einfaches Routing mit <code>foooe/router</code>	6-11
6.2.4	Komplexere Router	6-19
6.3	Weiterführende Literatur	6-21
7	Formularverarbeitung	7-1
7.1	Wiederholung: Formulardatenverarbeitung	7-1
7.1.1	Das HTML-Formular	7-1
7.1.2	Übertragungsmethode und Zugriff	7-1
7.2	Gefahren für Webanwendungen	7-5
7.2.1	Cross-Site-Scripting	7-5
7.2.2	SQL-Injection	7-6
7.2.3	Cross-Site Request Forgery (CSRF)	7-7
7.2.4	OWASP Top Ten	7-8
7.2.5	PHP Filter-Funktionen	7-9
7.2.6	Symfony HTML Sanitizer: Komponente zur Filterung von Inhalten	7-11
7.3	Dateiuploads	7-12
7.3.1	Voraussetzungen und Ablauf	7-12
7.3.2	Verarbeitung im superglobalen Array	7-13
7.3.3	Fehlercodes	7-14
7.3.4	Verschieben der Datei und Sicherheitsaspekte	7-15
7.3.5	Hochladen mehrerer Dateien	7-16
7.3.6	Hochladen ganzer Verzeichnisse	7-17
7.3.7	bulletproof: Dateiuploads mit einer fertigen Komponente	7-18
7.4	Weiterführende Literatur	7-19
8	Sessions und Authentifizierung	8-1
8.1	HTTP-Response Header	8-1
8.1.1	Unterschiedliche Content-Types	8-1
8.1.2	Weiterleitungen	8-2
8.2	Cookies	8-2
8.2.1	Allgemeines	8-2
8.2.2	Cookie-Eigenschaften	8-3

8.2.3	Cookies setzen	8-3
8.2.4	Cookies auslesen	8-4
8.2.5	Cookies löschen	8-4
8.3	Sessions	8-5
8.3.1	Allgemeines	8-5
8.3.2	PHP-Sessions	8-6
8.3.3	Klassen und Sessions	8-9
8.3.4	Session-Sicherheit und weitere Funktionen	8-10
8.4	Authentifizierung mit PHP	8-11
8.4.1	Funktionsweise und Ablauf	8-11
8.4.2	Erzeugen der Login-Daten	8-12
8.4.3	Der Login-Vorgang	8-13
8.5	Weiterführende Literatur	8-19
9	Datum, Zeit und Internationalisierung	9-1
9.1	Datum und Zeit	9-1
9.1.1	Datum und Zeit ausgeben	9-1
9.1.2	Zeitzone setzen	9-2
9.1.3	Mit Datum und Zeit rechnen	9-4
9.1.4	Unveränderbare Zeitstempel	9-6
9.1.5	Erweiterte Funktionalitäten mit Carbon	9-6
9.2	Internationalisierung	9-7
9.2.1	Allgemeines zu Internationalisierung	9-7
9.2.2	Internationalisierung und Lokalisierung mit PHP	9-9
9.2.3	Übersetzung mit PHP-Arrays	9-10
9.2.4	Übersetzung mit Gettext	9-13
9.2.5	Übersetzung mit symfony/translation	9-18
9.3	Weiterführende Literatur	9-26
10	Medienverarbeitung	10-1
10.1	XML-Verarbeitung	10-1
10.1.1	Der Parser	10-1
10.1.2	XML einlesen in PHP	10-3
10.1.3	XML erzeugen in PHP	10-10
10.1.4	Diskussion der einzelnen Erweiterungen	10-15
10.1.5	XML-Komponenten	10-16
10.2	JSON lesen und schreiben	10-16
10.2.1	Lesen von JSON-Dateien	10-16
10.2.2	Schreiben von JSON-Dateien	10-19
10.2.3	JSON Fehlerbehandlung	10-20
10.2.4	JSON validieren	10-20
10.3	Grafiken mit der GD Graphics Library	10-21
10.3.1	Allgemeines zur GDlib	10-21
10.3.2	Neue Bilder erstellen	10-23
10.3.3	Bestehende Bilder öffnen	10-23
10.3.4	Bilder ausgeben	10-25

10.3.5	Zeichenfunktionen	10-26
10.3.6	Bilder kopieren	10-27
10.3.7	Bildgröße ändern	10-28
10.3.8	Bilder rotieren	10-30
10.4	PDFs erzeugen	10-30
10.4.1	Dompdf initialisieren und Grundlagen	10-31
10.4.2	PDFs mit einer Template-Engine erstellen	10-33
10.5	Weiterführende Literatur	10-37
11	REST APIs & AJAX	11-1
11.1	REST APIs	11-1
11.1.1	Allgemeines zu REST	11-1
11.1.2	Aufbau einer REST API	11-3
11.1.3	Eine REST API in PHP implementieren	11-6
11.1.4	Weitere Funktionalitäten von REST APIs	11-12
11.2	AJAX	11-12
11.2.1	Allgemein	11-12
11.2.2	Die Fetch API	11-14
11.2.3	Beispiel: Textdaten verarbeiten	11-16
11.2.4	Beispiel: JSON-Daten verarbeiten	11-19
11.2.5	Beispiel: Formulardaten mit POST übertragen	11-23
11.2.6	Historischer Kontext: XMLHttpRequest	11-27
11.2.7	Ausblick: Promises	11-28
11.3	Serverseitige Konsumierung von REST APIs	11-28
11.3.1	Das Client-Objekt erstellen	11-29
11.3.2	Requests senden	11-29
11.3.3	Responses verarbeiten	11-29
11.3.4	Fehlerbehandlung	11-30
11.4	Weiterführende Literatur	11-31
12	Microframeworks und Testing	12-1
12.1	Einführung in Microframeworks	12-1
12.1.1	Definition	12-1
12.1.2	Vertreter	12-2
12.2	Slim Framework	12-3
12.2.1	Technische Voraussetzungen	12-4
12.2.2	Hello World-Beispiel	12-4
12.2.3	Dependencies	12-7
12.2.4	Middleware	12-13
12.2.5	Routing	12-15
12.3	Einführung in Testing	12-20
12.3.1	Zeitpunkt für Testing	12-20
12.3.2	Was wird wie getestet?	12-21
12.4	PHPUnit	12-21
12.4.1	PHPUnit vorbereiten	12-22
12.4.2	Die Code-Klasse <code>FavoriteMTDSubject</code>	12-22

12.4.3 Die Test-Klasse <code>FavoriteMTDSubjectTest</code>	12-23
12.4.4 Tests aufrufen	12-25
12.4.5 Weitere Unit Tests	12-25
12.4.6 Funktionale Tests	12-26
12.4.7 Test-Alternative: Pest	12-27
12.5 Weiterführende Literatur	12-28

Quellenverzeichnis**12-29**

Vorlesung 1

Grundlagen der Kommunikation im Web

Bevor in den kommenden Vorlesungen die serverseitige Programmierung mit PHP behandelt wird, ist es wichtig zu verstehen, was genau passiert, wenn in einem Webbrowser eine Adresse wie <https://www.fh-ooe.at/> eingetippt und die Eingabetaste gedrückt wird. Das Ergebnis ist bekannt: Nach kurzer Zeit erscheint eine fertig gerenderte Webseite. Zwischen Tastendruck und fertig geladener Seite läuft jedoch eine erstaunlich lange Kette an Vorgängen ab, an der eine Vielzahl von Protokollen, Diensten und Rechnern beteiligt ist. Die Kenntnis dieser Vorgänge ist für die Arbeit von Webdeveloper*innen wesentlich, da sich viele Eigenheiten von Dingen wie HTTP, Sessions, Cookies, Formularen oder APIs (mehr dazu in späteren Vorlesungen) direkt daraus ableiten lassen.

Dieses Kapitel nimmt einen konkreten Webrequest als Leitmotiv und arbeitet entlang dieses Beispiels die wichtigsten Netzwerk- und Protokoll-Grundlagen auf. Ziel ist ein solides Verständnis des *Weges*, den ein Request vom Browser zum Webserver und wieder zurück nimmt. Dieses Verständnis bildet die Grundlage für alle weiteren Kapitel, in denen es um das geht, was am Webserver selbst geschieht: das Verarbeiten des Requests mit PHP und das Erzeugen einer Antwort.

1.1 Zur Web Technologies Vorlesung

Die Vorlesung Web Technologies – kurz WET2VO – baut auf der Vorlesung Web Fundamentals (WEF1VO) direkt auf. Die Modalitäten sind daher weitgehend gleich. Im Folgenden sind die wichtigsten organisatorischen und inhaltlichen Dinge jedoch der Vollständigkeit halber angeführt.

1.1.1 Organisatorisches

Web Technologies behandelt serverseitige Webentwicklung und ist auf *zwölf Termine zu zwei Einheiten* aufgeteilt. Die Beurteilung erfolgt im Rahmen einer *elektronischen Klausur*, die am Ende des Semesters stattfindet. Die Klausur dauert 90 Minuten und findet ohne Zuhilfenahme der Unterlagen statt.

Die dazugehörigen Übungen sind auf je *zwölf Termine zu zwei Einheiten* aufgeteilt. Dabei werden Übungsaufgaben in Teams durchgeführt, von den zwölf Terminen beinhalten sechs eine Abgabe, die anderen sechs sind code-along Übungen.

Die Unterlagen zum Kurs befinden sich auf der eLearning-Plattform der FH Oberösterreich in den zugehörigen Kursen für Vorlesung¹ und Übung².

Zum Modul Web Technologies (WEB2) gehört weiters noch die Lehrveranstaltung Relational Databases (RDB2VO und RDB2UE). Diese beschäftigt sich mit der Persistierung, d. h. Datenbanken. Diese Inhalte werden in der zweiten Hälfte dieser Lehrveranstaltung integriert, wenn etwa eine Datenbank zur Persistierung von Inhalten benötigt wird.

1.1.2 Inhalte der Vorlesung

In diesem Semester liegt der Schwerpunkt auf serverseitiger Webentwicklung, vornehmlich mit PHP. Dabei werden folgende Themengebiete behandelt:

- Grundlagen der Kommunikation im Web (dieses Kapitel),
- PHP-Sprachgrundlagen (Syntax, Datentypen, Bibliotheksfunktionen),
- Regular Expressions,
- objektorientiertes PHP,
- Komponenten und Standards,
- Routing und Template-Systeme,
- Formularverarbeitung,
- Sessions und Authentifizierung,
- Datum, Zeit und Internationalisierung,
- Medienverarbeitung,
- REST APIs und AJAX, sowie
- Microframeworks und Testing.

1.2 Der Webrequest

Als Leitmotiv für die Vorlesung dient folgendes Szenario: Ein Laptop wird mit dem WLAN verbunden, im Browser wird die Adresse <https://www.fh-ooe.at/> eingetippt und die Eingabetaste gedrückt. Kurz darauf erscheint die Startseite der FH Oberösterreich. Was in dieser kurzen Zeitspanne im Hintergrund geschieht, lässt sich in eine Reihe wohldefinierter Schritte zerlegen.

Abbildung 1.1 zeigt eine stark vereinfachte Sicht auf die beteiligten Komponenten. Tatsächlich sind mindestens folgende Schritte nötig, damit die Seite im Browser erscheint – sie bilden den roten Faden dieser Vorlesung:

1. Der Browser muss verstehen, *was* angefordert wurde. Dazu wird die eingetippte Adresse (der *URL*) zerlegt (Abschnitt 1.3).
2. Der Laptop braucht eine eigene IP-Adresse im lokalen Netz. Diese bekommt er automatisch über *DHCP* (Abschnitt 1.7.4).
3. Der Name *www.fh-ooe.at* muss in eine IP-Adresse übersetzt werden. Dafür wird das *DNS* befragt (Abschnitt 1.6).

¹<https://elearning.fh-ooe.at/course/view.php?id=43705>

²<https://elearning.fh-ooe.at/course/view.php?id=43703>

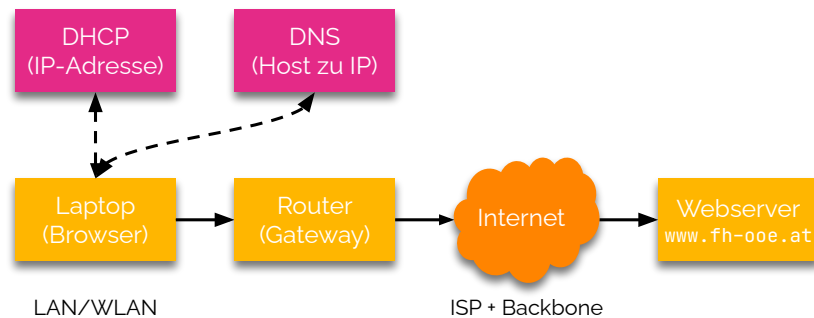


Abbildung 1.1: Vereinfachte Sicht auf die beteiligten Komponenten eines Webrequests. Neben Client und Server sind Hilfsdienste wie DHCP (für die lokale Netzwerkkonfiguration) und DNS (für die Namensauflösung) erforderlich, ebenso der lokale Router als Übergang ins Internet.

4. Für die lokale Zustellung an den Router muss die zugehörige Hardware-Adresse (*MAC*) ermittelt werden – das geschieht mit *ARP* (Abschnitt 1.7.5).
5. Der Browser baut zu Port 443 (*HTTPS*) des Webserver eine *TCP*-Verbindung auf (Abschnitt 1.8) und
6. verhandelt darüber per *TLS* einen verschlüsselten Kanal (Abschnitt 1.9).
7. Über diesen Kanal wird schließlich der eigentliche *HTTP-Request* verschickt (Abschnitt 1.10).
8. Der Webserver verarbeitet den Request und liefert eine *HTTP-Response* zurück, die *HTML*-Markup enthält – oder diesen Markup-Code dynamisch durch eine Skriptsprache wie *PHP* erzeugen lässt.
9. Der Browser parst das *HTML*, lädt eingebettete Ressourcen nach und rendert die Seite. Dieser letzte Schritt war bereits Thema von Web Fundamentals im ersten Semester.

Bei den Schritten 5 bis 8 läuft im Hintergrund jeweils noch *Encapsulation* ab: Die Daten werden vom sendenden Rechner Schicht für Schicht in *TCP*-, *IP*- und *Ethernet*-Header verpackt und vom empfangenden Rechner in umgekehrter Reihenfolge wieder ausgepackt (siehe Abschnitt 1.5.3). Dieser Vorgang wiederholt sich für jedes einzelne Paket – bei einem typischen Webrequest also viele Hundert Male.

All diese Schritte geschehen in Bruchteilen einer Sekunde. Jede einzelne Stufe setzt auf mehreren Protokollen auf, die wiederum in einem Schichtenmodell organisiert sind. Die nächsten Abschnitte arbeiten diese Schritte der Reihe nach ab.

1.3 Der URL als Adressierungsschema

Ein *URL* (Uniform Resource Locator) ist eine standardisierte Zeichenkette, mit dem eine Ressource im Web eindeutig adressiert werden kann. Der URL wurde historisch in RFC 1738 [6] definiert und später durch die allgemeinere URI-Spezifikation in RFC 3986 [5] abgelöst bzw. erweitert. Parallel dazu pflegt die WHATWG einen URL Living Standard [47], der die IETF-Dokumente vereinheitlicht und laufend an das tatsächliche

Tabelle 1.1: Einige gebräuchliche URL-Schemata.

Schema	Bedeutung
http	Hypertext Transfer Protocol (unverschlüsselt)
https	Hypertext Transfer Protocol Secure (verschlüsselt)
file	Lokale Datei im Dateisystem
mailto	E-Mail-Adresse
ftp	File Transfer Protocol
ws/wss	WebSocket (unverschlüsselt/verschlüsselt)

Verhalten moderner Browser anpasst. Im allgemeinsten Fall sieht ein URL so aus:

`<schema>:<schemaspezifischer Teil>`

Der Schemaname am Anfang legt fest, wie der restliche Teil zu interpretieren ist. Tabelle 1.1 zeigt einige gebräuchliche Schemata.

Nicht jedes Schema hat zwangsläufig eine Kommunikation über ein Netzwerk zur Folge. Ein URL vom Typ `file://` etwa öffnet eine Datei direkt im lokalen Dateisystem, *ohne* dass ein Webserver und ein Request beteiligt sind. Wird auf diese Weise eine PHP-Datei geöffnet, so sieht man im Browser lediglich den reinen PHP-Quelltext – es findet keine Verarbeitung statt, weil keine PHP-Engine involviert ist. Mehr dazu in Vorlesung 2. Ab jetzt sind daher primär `http`- und `https`-URLs interessant.

1.3.1 Aufbau eines HTTP-URLs

Ein HTTP-URL hat viele mögliche Elemente. Das folgende (etwas konstruierte) Beispiel zeigt alle möglichen Bestandteile:

`https://user:pass@www.example.org:8080/shop/item.php?id=42&lang=de#top`

Die Einzelteile haben dabei folgende Bedeutung:

https:// Das Schema, hier HTTP über TLS (verschlüsselt).

user:pass Optionale Zugangsdaten für HTTP-Basic-Auth (`.htaccess`). In der Praxis kaum mehr verwendet und aus Sicherheitsgründen abzulehnen, da Passwörter im Klartext im URL stehen.

www.example.org Der *Host*. Das kann ein *Fully Qualified Domain Name* (FQDN, siehe Abschnitt 1.6.1), oder eine IPv4- oder IPv6-Adresse sein (Abschnitt 1.7.1).

:8080 Der *Port*, auf dem der Webserver auf Verbindungen hört. Wird er nicht angegeben, verwendet der Browser den jeweiligen Standardport (80 für HTTP, 443 für HTTPS).

/shop/item.php Der *Pfad* (*url-path*) der angeforderten Ressource. Aus Server-Sicht ist das zunächst einmal einfach ein Name, der intern auf eine Datei, ein Skript oder einen Routing-Eintrag abgebildet wird.

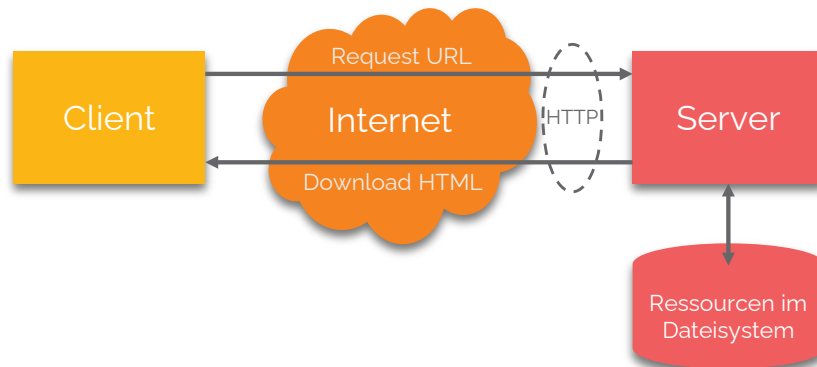


Abbildung 1.2: Client-Server-Kommunikation über das Internet.

?id=42&lang=de Der *Query-String*. Er beginnt mit ? und enthält ein oder mehrere **name=wert**-Paare, die in weiterer Folge durch & voneinander getrennt sind. Über den Query-String werden Parameter an den Server übergeben.

#top Das *Fragment*. Es beginnt mit # und adressiert einen bestimmten Abschnitt innerhalb der Ressource, etwa einen HTML-Anker. Wichtig: Das Fragment wird *nicht* mit an den Server geschickt, sondern ausschließlich clientseitig ausgewertet.

Für das am Anfang des Abschnitts vorgestellte Beispiel ist der URL wesentlich einfacher: `https://www.fh-ooe.at/` besteht aus Schema `https`, Host `www.fh-ooe.at`, implizitem Port 443 und als Pfad das Wurzel-Verzeichnis `/`. Query und Fragment sind nicht vorhanden.

All diese Informationen muss der Browser als Erstes aus der eingetippten Zeichenkette extrahieren, bevor er irgendetwas anderes tun kann.

1.4 Das Client-Server-Modell

Das Web ist, wie sehr viele andere Netzwerkanwendungen auch, nach dem *Client-Server-Modell* aufgebaut. Ein Rechner übernimmt die Rolle des *Servers* und stellt einen bestimmten Dienst bereit. Andere Rechner – die *Clients* – nutzen diesen Dienst, indem sie dem Server Anfragen schicken. Der Server beantwortet diese Anfragen und wartet ansonsten passiv auf weitere Clients. Abbildung 1.2 – bekannt aus dem 1. Semester – stellt diesen Zusammenhang dar.

Im Web ist der Client typischerweise ein Webbrowser (Chrome, Firefox, Safari etc.), der Server ein sogenannter *Webserver*. Der Begriff Webserver ist dabei zweideutig: Er bezeichnet sowohl den physischen oder virtuellen Rechner als auch die Software, die auf diesem Rechner läuft und eingehende HTTP-Requests verarbeitet. Gängige Webserver-Softwareprodukte sind *nginx*³, der *Apache HTTP Server*⁴ und *LiteSpeed*⁵. Erster hält Anfang April 2026 dabei bei einem Marktanteil von 32,8 % (2025: 33,8 %, 2024: 34,1 %, 2023: 34,3 %), zweiter bei 24,0 % (2025: 26,8 %, 2024: 30,2 %, 2023: 32,3 %), und

³<https://nginx.org/>

⁴<https://httpd.apache.org/>

⁵<https://www.litespeedtech.com/>

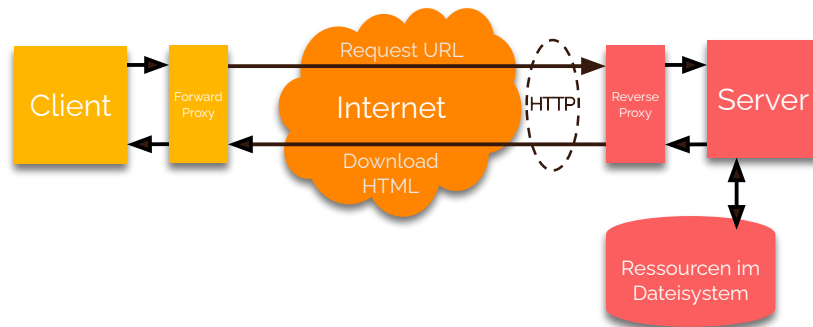


Abbildung 1.3: Client-Server-Kommunikation mit Forward und Reverse Proxies im Kommunikationsablauf.

dritter bei 15,2 % (2025: 14,5 %, 2024: 13,1 %, 2023: 11,6 %) [45]. Ein Sonderfall sind Anbieter wie Cloudflare, die als Reverse-Proxy vor der eigentlichen Infrastruktur stehen (siehe Abschnitt 1.4.1). In der verlinkten Statistik werden sie als eigene Einträge geführt, verbergen aber den dahinterliegenden tatsächlichen Webserver.

Der Webserver alleine kann zunächst nur statische Dateien ausliefern – also HTML-Dokumente, CSS-Dateien, Bilder, JavaScript-Dateien und Ähnliches. Für dynamisch erzeugte Inhalte wird eine zusätzliche Komponente benötigt, die den Request annimmt, mit Hilfe einer Skriptsprache wie PHP verarbeitet und das Ergebnis (z. B. HTML) an den Webserver zurückgibt, damit dieser es als Response an den Client zurückschicken kann. Vorlesung 2 wird darauf genauer eingehen.

1.4.1 Proxies, Reverse Proxies und Firewalls

In der Praxis ist die Darstellung „ein Client spricht direkt mit einem Webserver“ oft zu einfach. Gerade bei größeren Webanwendungen befinden sich zwischen Client und eigentlichem Anwendungsserver noch weitere Zwischenstationen, die bestimmte Aufgaben übernehmen. Zwei Begriffe sind hier besonders wichtig:

Forward Proxy Ein *Forward Proxy* (kurz: Proxy) steht *im Namen des Clients* zwischen diesem und dem Internet. Der Browser schickt seine Requests nicht direkt an den Zielservers, sondern an den Proxy, der die Anfrage stellvertretend weiterleitet. Typisch ist dieser Aufbau in Firmennetzen, wo ein Proxy zentral alle ausgehenden Verbindungen zwischenspeichert (*Caching*), protokolliert oder nach Inhalten filtert.

Reverse Proxy Ein *Reverse Proxy* steht *im Namen des Servers* zwischen dem Internet und dem eigentlichen Anwendungsserver. Der Client merkt davon nichts – aus seiner Sicht spricht er ganz normal mit „dem Webserver“. Tatsächlich nimmt der Reverse Proxy den Request entgegen und reicht ihn intern an einen oder mehrere dahinterliegende Anwendungsserver weiter.

Abbildung 1.3 stellt dar, wo die Proxies im Kommunikationsablauf positioniert sind.

Reverse Proxies sind aus modernen Webdeployments kaum mehr wegzudenken, weil sie eine Reihe wichtiger Aufgaben übernehmen: Sie terminieren die TLS-Verschlüsselung (siehe Abschnitt 1.9), verteilen die Last auf mehrere Backend-Server (*Load Balancing*),

cachen häufig angeforderte Ressourcen, komprimieren Antworten und schützen die eigentlichen Anwendungsserver, die dadurch gar nicht erst direkt aus dem Internet erreichbar sein müssen. Ein typisches Beispiel: Eine Node.js-Anwendung hört auf Port 3000, weil Ports unterhalb 1024 unter Linux nur mit Root-Rechten geöffnet werden dürfen und eine Anwendung aus Sicherheitsgründen nicht als Root laufen soll. Browser erwarten HTTPS-Verbindungen aber auf Port 443. Die Lösung: Ein vorgelagerter nginx-Webserver als Reverse Proxy nimmt eingehende Verbindungen auf Port 443 entgegen, terminiert TLS und leitet die entschlüsselten Requests intern an die Node-Anwendung auf Port 3000 weiter. Für den Client sieht alles wie eine ganz normale HTTPS-Verbindung aus, die interne Port-Verschiebung bleibt unsichtbar. Dasselbe Architektur-Muster findet sich auch bei PHP-Anwendungen in Produktion wieder: Dort trennt man die Auslieferung statischer Dateien (HTML, CSS, JavaScript, Bilder) von der Ausführung des PHP-Codes. Ein schlanker Webserver wie nginx oder Apache im event-basierten Modus nimmt alle Anfragen entgegen, liefert statische Ressourcen selbst aus und reicht PHP-Requests per FastCGI-Protokoll an einen separaten PHP-FPM-Prozess-Pool weiter, der den eigentlichen PHP-Code ausführt. Diese Trennung skaliert deutlich besser als ältere Ansätze, bei denen ein vollständiger PHP-Interpreter in jeden Webserver-Worker eingebettet war – schon weil sich Worker für statische und dynamische Inhalte unabhängig dimensionieren lassen.

Noch eine Stufe weiter außen steht oft eine *Firewall*. Sie filtert den ein- und ausgehenden Netzwerkverkehr anhand von Regeln – typischerweise auf Basis von Quell- und Ziel-IP-Adressen sowie Ports. Für einen Webserver ist eine gängige Regel etwa: „Erlaube eingehende Verbindungen auf Port 443, blockiere alles andere.“ Firewalls sind für unsere Betrachtung der Kommunikationswege primär ein zusätzlicher Kontrollpunkt, den ein Paket auf seinem Weg passieren muss; sie ändern am eigentlichen HTTP-Ablauf nichts.

1.5 Schichtenmodelle: OSI und TCP/IP

Die Kommunikation über ein Netzwerk ist komplex. Damit sie überhaupt handhabbar wird, wird sie gedanklich in mehrere *Schichten* zerlegt. Jede Schicht ist für einen klar abgegrenzten Aufgabenbereich zuständig und bietet der jeweils darüberliegenden Schicht eine definierte Schnittstelle an. Die unteren Schichten kümmern sich um physikalische und lokale Aspekte (wie werden Bits über ein Kabel übertragen?), die oberen Schichten um anwendungsbezogene Dinge (wie sieht ein HTTP-Request inhaltlich aus?).

1.5.1 Das OSI-Referenzmodell

Der bekannteste theoretische Entwurf ist das OSI-Referenzmodell (Open Systems Interconnection), das 1984 erstmals von der ISO veröffentlicht und 1994 in seiner heute gültigen Form als ISO/IEC,7498-1 standardisiert wurde [18].

1. **Bitübertragungsschicht** (Physical Layer): legt fest, wie Nullen und Einsen elektrisch, optisch oder per Funk auf das Übertragungsmedium (Kupferkabel, Glasfaser, Luft) aufgebracht werden.
2. **Sicherungsschicht** (Data Link Layer): kümmert sich um die physikalische Adressierung (MAC-Adressen) und stellt mittels Prüfsummen sicher, dass die Daten korrekt übertragen wurden. Formal teilt sie sich in zwei Unterschichten: *LLC* (Logical

Tabelle 1.2: Vergleich von OSI- und TCP/IP-Modell. Für das Szenario eines Webrequests sind vor allem die drei mittleren Ebenen relevant: HTTP auf der Anwendungsschicht, TCP auf der Transportschicht und IP auf der Vermittlungsschicht.

OSI	TCP/IP	Protokolle (Auswahl)	Dateneinheit
7 Anwendung	Anwendung	HTTP(S), SMTP, DNS, SSH	Nachricht / Daten
6 Darstellung			
5 Sitzung			
4 Transport	Transport	TCP, UDP	Segment / Datagramm
3 Vermittlung	Internet	IP, ICMP	Paket
2 Sicherung	Netzzugang	Ethernet, WLAN, ARP	Frame
1 Bitübertragung		Kupfer, Glasfaser, Funk	Bit

Link Control) für Fluss- und Fehlerkontrolle und *MAC* (Media Access Control) für die Hardware-Adressierung und den Zugriff auf das Übertragungsmedium.

3. **Vermittlungsschicht** (Network Layer): logische Adressierung, Routenfindung (Routing), Datenflusskontrolle zwischen Netzen.
4. **Transportschicht** (Transport Layer): ordnet Daten einer bestimmten Anwendung auf dem Zielrechner zu und kümmert sich um Zuverlässigkeit und Reihenfolge der Übertragung.
5. **Sitzungsschicht** (Session Layer): Auf- und Abbau sowie Verwaltung von Sitzungen auf Anwendungsebene.
6. **Darstellungsschicht** (Presentation Layer): Umwandlung zwischen systemabhängigen und systemunabhängigen Datenformaten, Zeichenkodierung.
7. **Anwendungsschicht** (Application Layer): Funktionen für konkrete Anwendungen wie Webbrowser, E-Mail-Programme oder Dateitransfer-Tools.

1.5.2 Das TCP/IP-Modell in der Praxis

In der Praxis hat sich ein etwas schlankeres Modell durchgesetzt, das *TCP/IP-Modell*. Es fasst einige der OSI-Schichten zusammen und ist näher an den tatsächlich eingesetzten Protokollen. Tabelle 1.2 stellt die beiden Modelle samt typischer Protokolle gegenüber.

Das OSI-Modell ist ein sauberer theoretischer Entwurf, das TCP/IP-Modell ist jedoch aus der realen Entwicklung des Internets heraus entstanden und deshalb näher an dem, was tatsächlich auf der Leitung liegt. In der Fehlerdiagnose denkt man jedoch meist in OSI-Begriffen: Wenn ein entfernter Rechner nicht erreichbar ist, sich aber anpingen lässt, ist das Problem vermutlich auf der Anwendungsschicht; lässt er sich nicht einmal pingen, liegt das Problem eine oder mehrere Schichten darunter.

1.5.3 Encapsulation: Wie Schichten zusammenspielen

Die Schichten arbeiten zusammen, indem jede von ihnen die Daten der darüberliegenden Schicht als *Nutzdaten* (*Payload*) behandelt und einen eigenen *Header* davorsetzt. Dieses

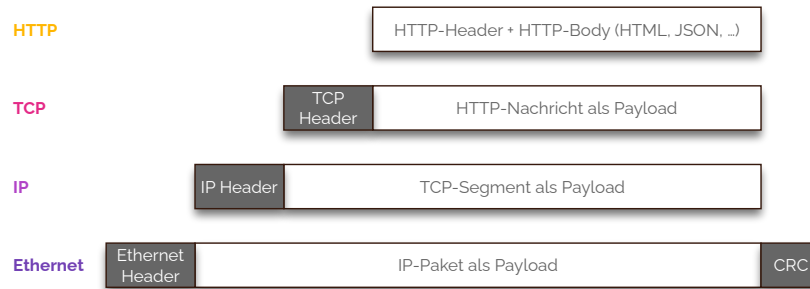


Abbildung 1.4: Encapsulation am Beispiel eines HTTP-Requests: Jede Schicht erweitert die Daten der darüberliegenden Schicht um einen eigenen Header. Ethernet fügt zusätzlich einen Trailer mit einer Prüfsumme (CRC) an, über die sich Übertragungsfehler erkennen lassen.

Prinzip heißt *Encapsulation*. Es bedeutet konkret: Wenn der Browser einen HTTP-Request losschickt, wird dieser Request zunächst als HTTP-Nachricht formuliert, dann mit einem TCP-Header versehen, dann mit einem IP-Header, dann mit einem Ethernet-Header (und einem Ethernet-Trailer mit Prüfsumme). Abbildung 1.4 zeigt das Prinzip.

Am Zielrechner läuft der Prozess in umgekehrter Richtung ab: Der Ethernet-Header wird entfernt und auf Basis seiner Informationen entschieden, dass die Nutzdaten an die IP-Schicht weitergereicht werden. Diese entfernt den IP-Header und übergibt an die TCP-Schicht, die wiederum ihren Header entfernt und die HTTP-Nachricht an den Webserver-Prozess weiterreicht. So weiß jede Schicht ausschließlich, wie ihr eigener Header aussieht und welche Schicht darüberliegt – alles andere ist für sie undurchsichtiger Payload.

Eine konkrete Kombination aufeinanderfolgender Protokolle nennt man *Protokoll-stack*. Für das Szenario eines Webrequests ist der relevante Stack also HTTP → TCP → IP → Ethernet (oder WLAN), für eine DNS-Anfrage (siehe Abschnitt 1.6) wäre es DNS → UDP → IP → Ethernet.

1.6 Vom Namen zur Adresse: DNS

Zurück zum Webrequest-Beispiel: Der Browser hat den URL zerlegt und weiß, dass er mit einem Host namens `www.fh-ooe.at` sprechen will. Rechner im Internet werden aber nicht über Namen adressiert, sondern über *IP-Adressen* – also über Nummern wie 162.55.28.238 (IPv4) oder 2001:62a:4:417::5 (IPv6). Der Name muss also zuerst in eine IP-Adresse übersetzt werden. Diese Aufgabe erledigt das *Domain Name System* (DNS).

1.6.1 Hostnamen, Domains und FQDN

Ein Hostname wie `www.fh-ooe.at` ist hierarchisch aufgebaut und wird *von rechts nach links* gelesen:

- `at` ist die *Top-Level-Domain* (TLD). Für sie ist eine eigene Verwaltungsorganisation zuständig (im Fall von `.at` die nic.at).

- **fh-ooe** ist die *Second-Level-Domain*. Sie wurde bei der zuständigen Organisation registriert und gehört einem bestimmten Inhaber (hier der FH Oberösterreich).
- **www** ist der *Hostname* innerhalb dieser Domain. Er bezeichnet einen konkreten Rechner (oder Dienst) innerhalb der Domain.

Die komplette Kette bis zur TLD nennt man *Fully Qualified Domain Name* (FQDN).

1.6.2 Ablauf der Namensauflösung

Der Browser selbst führt die Namensauflösung nicht direkt durch. Stattdessen fragt das Betriebssystem einen *DNS-Resolver*, der in der Regel beim Internet-Provider steht und dessen IP-Adresse der Rechner üblicherweise per DHCP mitgeteilt bekommen hat (mehr dazu in Abschnitt 1.7.4). Dieser Resolver fragt dann bei Bedarf rekursiv weitere DNS-Server an, bis er eine Antwort hat:

1. Zunächst wird ein *Root-Nameserver* befragt – einer von 13 weltweit verteilten Servern, die den obersten Einstiegspunkt ins DNS bilden und deren Adressen in jeder Resolver-Software fest eingebaut sind. Die Frage lautet: „Wer ist für **.at** zuständig?“
2. Die Antwort verweist auf einen Nameserver der **.at**-Zone. Dieser wird gefragt: „Wer ist für **fh-ooe.at** zuständig?“
3. Die Antwort verweist auf den autoritativen Nameserver der FH Oberösterreich. Dieser wird schließlich gefragt: „Wie lautet die IP-Adresse von **www.fh-ooe.at**?“
4. Die liefert die eigentliche IP-Adresse zurück.

In der Praxis ist dieser Prozess durch aggressives *Caching* auf mehreren Ebenen stark beschleunigt. Der Resolver speichert bereits aufgelöste Namen für eine bestimmte Zeit (die *Time-to-Live*, TTL), und auch das lokale Betriebssystem und der Browser haben eigene DNS-Caches. Für eine populäre Seite wie **www.fh-ooe.at** fällt in den allermeisten Fällen gar keine externe Anfrage an – die Antwort kommt direkt aus dem Cache des Resolvers.

DNS verwendet überwiegend *UDP* als Transportprotokoll (Port 53), weil einzelne kurze Anfragen ohne Verbindungsaufbau effizienter sind als mit TCP. Das passt zu der Faustregel: UDP dort, wo kleine Einzeldatagramme und niedrige Latenz gefragt sind; TCP dort, wo eine verlässliche, geordnete Datenübertragung über längere Zeit notwendig ist (siehe dazu Abschnitt 1.8).

Am Ende dieses Schritts kennt der Browser die IP-Adresse des Webservers, etwa **162.55.28.238**. Offen bleibt nun die Frage, *wie* die Anfrage tatsächlich dorthin gelangt. Abbildung 1.5 stellt den Ablauf dar.

1.7 Die Netzwerkschicht: IP und lokale Zustellung

Mit der aufgelösten IP-Adresse des Webservers kann die eigentliche Zustellung beginnen. Die dafür zuständige Schicht ist die Netzwerkschicht, auf der das *Internet Protocol* (IP) arbeitet. Die folgenden Unterabschnitte behandeln die wichtigsten Bausteine, die für diese Zustellung erforderlich sind: die IP-Adressierung selbst, die automatische Netzwerkkonfiguration per DHCP, die Übersetzung von IP- auf Hardware-Adressen per

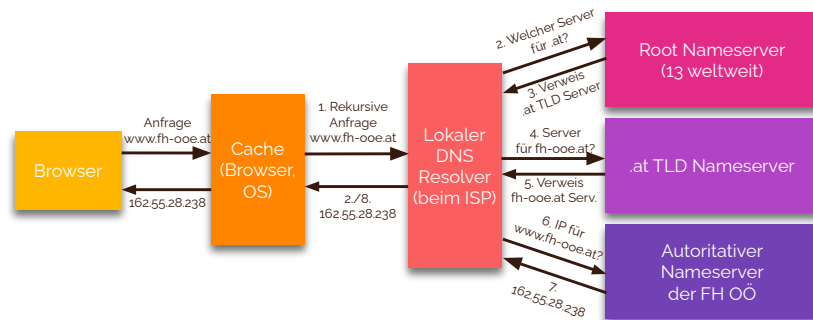


Abbildung 1.5: Der Ablauf vom Namen `www.fh-ooe.at` bis zur IP-Adresse, inklusive möglichem Caching.

ARP, den Aufbau von Ethernet-Frames und schließlich die Netzwerkkomponenten, die die Pakete tatsächlich weiterleiten.

1.7.1 IP-Adressen

IP-Adressen sind die logischen Adressen, unter denen Rechner im Internet erreichbar sind. *IPv4*-Adressen sind 32 ($4 \cdot 8$) Bit lang und werden in der gewohnten Punktschreibweise notiert (162.55.28.238). Da der IPv4-Adressraum viel zu klein für das heutige Internet ist, gibt es parallel den Nachfolger *IPv6* mit 128 Bit langen Adressen, die in Hexadezimal-Blöcken notiert werden (2a01:04f8:2190:2103:d64a:3471:c57f:8d4c). Für unsere Betrachtung ist der Unterschied unerheblich.

Jede IP-Adresse zerfällt in zwei Teile: den *Netzanteil* (links) und den *Hostanteil* (rechts). Der Netzanteil identifiziert das Netzwerk, in dem sich der Rechner befindet; der Hostanteil identifiziert den konkreten Rechner innerhalb dieses Netzes. Wie lang der Netzanteil ist, bestimmt die *Subnetzmaske*. Für ein Netz, in dem die ersten 24 Bit zum Netzanteil gehören, lautet die Subnetzmaske dezimal 255.255.255.0. Statt der ausführlichen Maske hat sich heute die kürzere *CIDR-Schreibweise* (Classless Inter-Domain Routing) durchgesetzt: Man hängt an die Netzadresse einen Schrägstrich und die Anzahl der Netzanteil-Bits an, also beispielsweise 192.168.0.0/24. Diese Notation begegnet in der Praxis ständig – in Router-Konfigurationen, in Cloud-Dashboards und im Output von Kommandos wie `ip route`.

Ein paar konkrete Beispiele machen den Zusammenhang anschaulich:

Adresse	Subnetzmaske	CIDR	Bedeutung
192.168.0.17	255.255.255.0	/24	Host im kleinen Heimnetz
10.42.7.200	255.255.0.0	/16	Host in großem Firmennetz
172.16.5.3	255.255.255.240	/28	Host in kleinem Subnetz

Im ersten Beispiel gehören die ersten drei Oktette (192.168.0) zum Netzanteil, das letzte Oktett (17) ist der Hostanteil. Im /24-Netz sind dadurch 254 Host-Adressen nutzbar. Zunächst wird von den gesamten 32 Bit einer IPv4-Adresse der Hostanteil abgezogen, also $32 - 24 = 8$, es bleiben also 8 Bit für die Hosts, $2^8 = 256$, somit wären theoretisch 256 Hostadressen möglich. Für die nutzbaren Hosts müssen jedoch immer

2 abgezogen werden: die erste Adresse (hier 192.168.0.0), genannt *Netz-ID* oder *Netz-Adresse* und die letzte Adresse (hier 192.168.0.255), genannt *Broadcast-Adresse*. Somit verbleiben 254 nutzbare Hosts, von 192.168.0.1 bis 192.168.0.254.

Im zweiten Beispiel umfasst der Netzanteil nur die ersten beiden Oktette (10.42), und es bleiben 16 Bit für Hosts – also $65.534 (2^{16} - 2)$ mögliche Adressen.

Das dritte Beispiel zeigt ein bewusst kleines Subnetz: Bei /28 werden 28 Bit für das Netz verwendet und nur 4 Bit bleiben für Hosts übrig, was lediglich 14 nutzbare Adressen ($2^4 - 2$) ergibt.

Rechner, die im selben Subnetz sind, können sich direkt auf Ethernet-Ebene miteinander unterhalten. Rechner in unterschiedlichen Subnetzen müssen über einen *Router* kommunizieren. Der Router des eigenen Subnetzes wird als *Standard-Gateway* (*Default Gateway*) bezeichnet. Ob ein Ziel im gleichen Subnetz liegt oder nicht, lässt sich anhand der Subnetzmaske durch einen einfachen Bit-Vergleich ermitteln – Details dazu sind aber vertiefender Stoff der Netzwerktechnik und werden hier nicht weiter thematisiert.

1.7.2 Unicast, Broadcast und Multicast

Pakete auf IP-Ebene werden in drei unterschiedlichen Modi verschickt:

Unicast Ein Paket hat genau einen Absender und genau einen Empfänger. Das ist der Normalfall und betrifft praktisch die gesamte Web-Kommunikation: Ein HTTP-Request geht von genau einem Browser an genau einen Webserver.

Broadcast Ein Paket geht an *alle* Rechner im lokalen Subnetz. Dafür gibt es eine spezielle Broadcast-Adresse (in einem 192.168.0.0/24-Netz etwa 192.168.0.255). Broadcast wird unter anderem von DHCP und ARP verwendet, weil ein neu hinzukommender Rechner zunächst noch gar nicht weiß, wen er gezielt ansprechen könnte.

Multicast Ein Paket geht an eine bestimmte *Gruppe* von Rechnern, die sich zuvor für einen gemeinsamen Kanal angemeldet haben. Für diese Betrachtung spielt Multicast keine Rolle, er wird vor allem für Medienstreams in lokalen Netzen eingesetzt.

Broadcasts werden von Routern bewusst *nicht* weitergeleitet – sonst würde ein einzelner DHCP-Broadcast das gesamte Internet überschwemmen. Sie bleiben immer auf das lokale Subnetz beschränkt.

1.7.3 Private Adressen und NAT

An dieser Stelle stellt sich die Frage, warum der Laptop im Beispiel eine Adresse wie 192.168.0.17 erhält – solche Adressen können ja nicht beliebig „im Internet“ vergeben werden, ohne sich mit anderen Nutzer*innen zu überschneiden. Die Antwort: Bestimmte IPv4-Bereiche sind explizit als *private Adressen* reserviert und dürfen innerhalb beliebiger lokaler Netze verwendet werden. Die drei Bereiche sind:

CIDR	Adressbereich
10.0.0.0/8	10.0.0.0–10.255.255.255
172.16.0.0/12	172.16.0.0–172.31.255.255
192.168.0.0/16	192.168.0.0–192.168.255.255

Pakete mit solchen Quell- oder Zieladressen werden im öffentlichen Internet nicht geroutet. Damit ein Rechner aus einem privaten Netz trotzdem Webseiten abrufen kann, kommt *Network Address Translation* (NAT) zum Einsatz: Der Router am Übergang zwischen privatem Netz und Internet ersetzt in ausgehenden Paketen die private Quell-IP durch seine eigene öffentliche IP (und merkt sich die Zuordnung anhand der Portnummern). Kommen Antwortpakete zurück, macht er die Übersetzung rückgängig und leitet sie an den richtigen internen Rechner weiter. Aus Sicht des Webservers kommuniziert er dabei immer nur mit dem NAT-Router; die dahinterliegenden Rechner sind für ihn nicht sichtbar. NAT ist ein zentraler Grund dafür, dass IPv4 trotz seines viel zu kleinen Adressraums bis heute funktioniert.

Für die serverseitige Entwicklung ist NAT unter anderem dann relevant, wenn PHP-Skripte die IP-Adresse des aufrufenden Clients auswerten wollen (z. B. für Geolocation oder Logging): Die „gesehene“ IP-Adresse ist dann in vielen Fällen nicht die tatsächliche IP des Clients, sondern die eines zwischengeschalteten NAT-Routers oder Reverse Proxys (vgl. Abschnitt 1.4.1).

1.7.4 DHCP: Automatische Netzwerkkonfiguration

Woher bezieht der Laptop im Beispiel eigentlich seine eigene IP-Adresse? In modernen Netzen wird diese dynamisch vergeben. Verantwortlich dafür ist das *Dynamic Host Configuration Protocol* (DHCP). Sobald sich der Laptop mit dem WLAN verbindet, schickt er einen DHCP-Request als *Broadcast* ins lokale Netz. Ein DHCP-Server (meist der Router selbst) antwortet mit einem Datenpaket, das mindestens folgende Informationen enthält:

- eine freie IP-Adresse für den Laptop,
- die zugehörige Subnetzmaske,
- die IP-Adresse des Standard-Gateways und
- die IP-Adresse(n) eines oder mehrerer DNS-Resolver.

Erst nach erfolgreichem DHCP-Vorgang kennt der Laptop also seine eigene IP-Adresse, weiß, wohin er Pakete für entfernte Rechner schicken soll (zum Gateway), und kann DNS-Anfragen stellen. Unter Windows lassen sich diese Informationen bequem mit dem Kommando `ipconfig /all` anzeigen, unter Linux/macOS mit `ifconfig` bzw. `ip addr` und `ip route`.

1.7.5 ARP: Vom IP zum MAC

Der Laptop weiß jetzt Folgendes: Seine eigene IP ist z. B. 192.168.0.17, die des Gateways 192.168.0.1, und der Webserver liegt unter 162.55.28.238. Der Webserver ist offensichtlich nicht im lokalen Subnetz (anderer Adressbereich), also müssen die Pakete an das Gateway geschickt werden. Aber Ethernet kennt keine IP-Adressen – es kennt nur *MAC-Adressen* (Media Access Control). Eine MAC-Adresse ist 48 Bit lang und weltweit (fast) eindeutig, bestehend aus einer Herstellerkennung und einer fortlaufenden Nummer, beispielsweise 00:0e:7f:27:a8:59.

Um die Ethernet-Frames korrekt adressieren zu können, braucht der Laptop also die MAC-Adresse des Gateways. Diese ermittelt er mit dem *Address Resolution Protocol* (ARP): Er schickt einen ARP-Request als Broadcast ins lokale Netz („Wer hat die

IP-Adresse 192.168.0.1?“), und das Gateway antwortet mit einer unicast-Nachricht zurück („Das bin ich, meine MAC-Adresse ist 00:0e:7f:27:a8:59“). Diese Zuordnung wird in einem lokalen ARP-Cache zwischengespeichert, damit sie nicht für jedes einzelne Paket neu ermittelt werden muss.

1.7.6 Ethernet-Frames und lokale Zustellung

Ethernet ist das mit Abstand am häufigsten eingesetzte Protokoll der Sicherungsschicht (OSI Schicht 2) und regelt die Datenübertragung auf dem lokalen Netzwerksegment, also etwa zwischen einem Laptop und dem Switch oder Router im selben Raum. Die Dateneinheit auf dieser Schicht heißt *Frame*: Ein Ethernet-Frame umschließt das eigentliche IP-Paket mit einem Header (Quell- und Ziel-MAC-Adresse sowie ein Typfeld, das angibt, welches Protokoll der nächsthöheren Schicht eingebettet ist) und einem Trailer mit einer Prüfsumme. Die maximale Größe der Nutzdaten in einem Standard-Frame liegt bei 1500 Byte – dieser Wert wird im TCP-Abschnitt 1.8 noch eine Rolle spielen.

Mit diesem Wissen kann der Laptop nun das erste Frame zusammenstellen. Die Netzwerkkarte schickt es auf das Medium; der Netzwerk-Switch, an dem der Laptop hängt, liest die Ziel-MAC aus, stellt fest, hinter welchem seiner Ports sich der entsprechende Rechner befindet, und leitet das Frame gezielt nur dorthin weiter – in unserem Fall an das Gateway. Alle anderen angeschlossenen Rechner bekommen das Frame gar nicht zu sehen.

Das Gateway schaut sich nun den IP-Header an, sieht die Ziel-IP 162.55.28.238, stellt fest, dass es das nicht selbst ist, und leitet das Paket entsprechend seiner Routing-Tabelle weiter – bei jedem Hop (Router) wird dabei nur der Ethernet-Header getauscht; der IP-Header, der TCP-Header und die HTTP-Nutzdaten bleiben unverändert. So bewegt sich das Paket schließlich über mehrere Router durch das Internet bis zum Webserver.

1.7.7 Netzwerkkomponenten: Hub, Switch, Router

An dieser Stelle lohnt es sich, die Geräte, die auf dem Weg eines Pakets begegnen, nach ihrer Funktion zu sortieren. Die gängige Einteilung erfolgt nach der OSI-Schicht, auf der sie arbeiten:

Hub (Schicht 1) Ein Hub ist im Kern ein dummer Signalverstärker: Was auf einem Port hereinkommt, wird einfach auf alle anderen Ports kopiert. Alle angeschlossenen Rechner sehen den gesamten Datenverkehr, was zu Kollisionen und geringer Effizienz führt. In der Praxis sind Hubs heute praktisch ausgestorben und wurden fast überall durch Switches ersetzt.

Switch (Schicht 2) Ein Switch lernt mit, welche MAC-Adressen hinter welchem seiner Ports liegen, und leitet jedes Frame gezielt nur an den Port weiter, an dem die Ziel-MAC hängt. Dadurch kann jeder Port die volle Bandbreite nutzen, und Rechner sehen nur den Datenverkehr, der sie tatsächlich betrifft. Switches arbeiten mit MAC-Adressen und wissen von IP gar nichts.

Router (Schicht 3) Ein Router verbindet verschiedene Netze miteinander und trifft seine Weiterleitungsentscheidungen auf Basis von IP-Adressen. Er führt eine *Routing-Tabelle*, in der er nachschlägt, über welches seiner Interfaces ein Paket mit

einer gegebenen Ziel-IP weiterverschickt werden muss. Router sind die Stellen im Netz, an denen Pakete das lokale Subnetz verlassen.

In unserem Szenario passiert das Ethernet-Frame des Laptops also zuerst einen Switch im lokalen Netz (der es zum Gateway zustellt), und ab dann eine ganze Reihe von Routern auf dem Weg durchs Internet. Bei jedem Router-Hop wird der Ethernet-Header verworfen und ein neuer für das nächste Teilstück erzeugt, während der IP-Header unverändert bleibt.

1.7.8 Diagnose: ICMP, ping und traceroute

Eng mit IP verwandt ist das *Internet Control Message Protocol* (ICMP). ICMP ist auf derselben Schicht angesiedelt wie IP und dient dazu, Status und Fehlermeldungen im Netz zu transportieren – etwa „Zielrechner nicht erreichbar“ oder „Time-to-Live abgelaufen“. Im Kontext der Webentwicklung ist ICMP vor allem deshalb relevant, weil zwei häufig eingesetzte Diagnose-Tools darauf aufsetzen:

ping schickt ein *Echo Request* an eine angegebene IP-Adresse oder einen Hostnamen. Antwortet die Gegenseite mit einem *Echo Reply*, ist der Zielrechner grundsätzlich erreichbar und es besteht eine funktionierende IP-Verbindung dorthin. Bleibt die Antwort aus, liegt das Problem auf der Netzwerk- oder Vermittlungsschicht – HTTP oder die Webanwendung sind dann gar nicht erst betroffen. Typischer Aufruf: `ping www.fh-ooe.at`

traceroute (unter Windows `tracert`) zeigt den Weg, den ein Paket bis zu seinem Ziel nimmt, und listet alle Zwischen-Router auf. Dabei wird die TTL (*Time to Live*) im IP-Header gezielt manipuliert: Der jeweils erste Router, bei dem sie auf null fällt, meldet dies per ICMP zurück und gibt sich damit zu erkennen. So lässt sich feststellen, an welcher Stelle auf dem Weg ein Paket „hängenbleibt“.

Beide Werkzeuge sind für die Diagnose serverseitiger Probleme unentbehrlich und dienen typischerweise als erster Prüfschritt, wenn eine Webanwendung nicht mehr erreichbar ist. Reagiert der Server auf **ping**, während HTTPS-Verbindungen fehlschlagen, so ist zumindest gesichert, dass die Netzwerkverbindung steht und das Problem auf einer höheren Ebene liegt – etwa beim Webserver, bei TLS oder bei der Anwendung selbst.

Einschränkend sei angemerkt: Aus Sicherheitsgründen blockieren manche Server und Firewalls eingehende ICMP-Pakete. Ein fehlgeschlagener **ping** ist deshalb nicht in jedem Fall ein zwingendes Indiz dafür, dass der Server tatsächlich nicht erreichbar ist.

1.8 Die Transportschicht: TCP

Auf der Netzwerkschicht ist IP ein *verbindungsloses und unzuverlässiges* Protokoll. Das klingt schlimmer, als es ist: IP liefert Pakete so schnell und so gut es geht zu, garantiert aber weder, dass sie ankommen, noch, dass sie in der richtigen Reihenfolge ankommen, noch dass sie nicht dupliziert werden. Für einen Webrequest soll aber garantiert sein, dass *alle* Teile der Anfrage und der Antwort vollständig und in der richtigen Reihenfolge bei dem*der Empfänger*in ankommen. Darum kümmert sich die Transportschicht, konkret das *Transmission Control Protocol* (TCP).

1.8.1 TCP und UDP

Neben TCP gibt es noch das *User Datagram Protocol* (UDP). Die beiden Protokolle ergänzen sich und werden für unterschiedliche Anwendungsfälle eingesetzt. Eine Gegenüberstellung macht auch die Funktionsweise deutlich:

TCP baut eine Verbindung zwischen zwei Endpunkten auf, sorgt für die zuverlässige Übertragung und die korrekte Reihenfolge, fordert verlorene Pakete erneut an und steuert die Datenrate. TCP wird von HTTP(S), SSH, SMTP, FTP und vielen anderen Anwendungen verwendet.

UDP verschickt einzelne Datagramme ohne Verbindungsaufbau und ohne Zustellgarantie. Es ist deutlich schlanker und eignet sich für kurze Anfragen (DNS), für Echtzeitanwendungen wie VoIP und Video-Streaming, bei denen ein paar verlorene Pakete verschmerzbar sind, oder für Broadcasts (DHCP).

Gemeinsam ist beiden, dass sie über *Port-Nummern* adressieren, welcher Prozess auf dem Zielrechner die Daten bekommen soll. Ein Port ist eine 16-Bit-Zahl (also zwischen 0 und 65535). Für bekannte Dienste gibt es fest vergebene *Well-Known Ports*:

Port	Dienst
20, 21	FTP
22	SSH
25	SMTP
53	DNS
67, 68	DHCP
80	HTTP
443	HTTPS

Ports unterhalb 1024 sind reserviert und werden typischerweise nur von Serverdiensten verwendet; Clients wählen sich beim Verbindungsaufbau einen freien Port oberhalb von 1024 aus (*ephemeral port*).

1.8.2 Der Three-Way-Handshake

Bevor Daten über eine TCP-Verbindung fließen können, muss diese aufgebaut werden. Das geschieht mit einem sogenannten *Three-Way-Handshake*, dargestellt in Abbildung 1.6:

1. Der Client (Browser) schickt ein TCP-Segment mit gesetztem SYN-Flag an den Server, Zielport 443. Damit signalisiert er „Ich möchte eine Verbindung öffnen“ und teilt gleichzeitig seine Anfangs-Sequenznummer mit.
2. Der Server antwortet mit einem Segment, das sowohl SYN als auch ACK gesetzt hat: Er bestätigt die Sequenznummer des Clients und sendet seine eigene.
3. Der Client bestätigt seinerseits mit einem reinen ACK.

Ab diesem Moment steht eine bidirektionale TCP-Verbindung. Jede Seite kann nun Daten schicken, die automatisch in der richtigen Reihenfolge wieder zusammengesetzt werden. Fällt ein Paket aus, wird es erneut angefordert; ist die Empfängerseite überlastet, wird die Senderate gedrosselt (*Flow Control*). All das läuft transparent ab – die Anwendungsschicht (also HTTP) bekommt davon nichts mit.

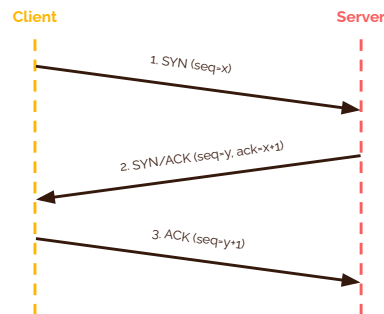


Abbildung 1.6: TCP-Verbindungsaufbau mit dem Three-Way-Handshake. Der Client initiiert die Verbindung mit einem SYN-Paket, der Server bestätigt (SYN/ACK) und der Client quittiert seinerseits (ACK). Erst nach diesen drei Paketen steht die Verbindung.

Damit das Reihenfolge-Versprechen eingelöst werden kann, muss TCP größere Datenmengen zunächst in passende Häppchen zerlegen. Der Grund dafür liegt eine Schicht tiefer: Ein Ethernet-Frame (siehe Abschnitt 1.7.6) kann in der Standardkonfiguration nur bis zu 1.500 Byte Nutzdaten tragen (die *Maximum Transmission Unit*, MTU). Größere HTTP-Nachrichten – etwa ein mehrere Megabyte großes Bild oder ein längeres HTML-Dokument – werden deshalb in mehrere TCP-Segmente unterteilt. Jedes Segment bekommt im TCP-Header eine *Sequenznummer*, die angibt, an welcher Position im Gesamtdatenstrom seine Nutzdaten stehen. Am Empfänger werden die Segmente anhand dieser Sequenznummern wieder in die richtige Reihenfolge gebracht, auch wenn sie unterwegs unterschiedliche Routen genommen haben und in einer anderen Reihenfolge eingetroffen sind. Fehlende Sequenznummern verraten verlorene Pakete und lösen die erneute Übertragung aus.

Der Verbindungsabbau erfolgt analog mit FIN-Paketen, weitere Details können wieder entsprechender Netzwerktechnik-Literatur entnommen werden.

1.9 Sichere Übertragung: HTTPS und TLS

Bisher wurde implizit angenommen, dass die Kommunikation zwischen Browser und Webserver offen erfolgt. Für viele Anwendungen ist das jedoch nicht akzeptabel: Passwörter, Zahlungsdaten, persönliche Informationen oder auch nur der konkret aufgerufene Inhalt sollen nicht für jede Person abhörbar sein, die irgendwo entlang der Leitung Zugriff auf den Netzwerkverkehr erlangen könnte. Deshalb verwendet der Beispiel-URL zur Seite der FH Oberösterreich nicht `http://`, sondern `https://`.

1.9.1 HTTPS ist HTTP über TLS

HTTPS ist kein eigenständiges Protokoll, sondern einfach HTTP, das durch eine zusätzliche Schicht namens *TLS* (Transport Layer Security, früher SSL) geschützt wird. TLS schiebt sich zwischen TCP und HTTP ein und sorgt für drei Dinge:

- **Vertraulichkeit** (*confidentiality*) – die übertragenen Daten werden verschlüsselt, sodass Dritte sie nicht mitlesen können.

- **Integrität** (*integrity*) – Änderungen an den Daten während der Übertragung werden bemerkt.
- **Authentizität** (*authenticity*) – der Client kann sich sicher sein, tatsächlich mit dem Server zu sprechen, dessen Name im URL steht, und nicht mit einem*iner Angreifer*in, der*die sich dazwischengeschaltet hat.

Im Schichtenmodell befindet sich TLS oberhalb der Transportschicht und unterhalb der Anwendungsprotokolle. Im OSI-Modell wird es üblicherweise der *Sitzungsschicht* (Schicht 5) zugeordnet. Im schlankeren TCP/IP-Modell gibt es keine eigene Ebene dafür, und TLS wird dort als transparente Zwischenschicht zwischen Transport- und Anwendungsschicht eingeordnet.

Aus Sicht der HTTP-Nachricht ist TLS transparent: Die HTTP-Nachricht wird vor dem Übergeben an TCP verschlüsselt und bei dem*der Empfänger*in wieder entschlüsselt. Der HTTP-Request selbst sieht genauso aus wie im unverschlüsselten Fall.

1.9.2 Der TLS-Handshake

Bevor Nutzdaten verschlüsselt übertragen werden können, muss zwischen Client und Server ein gemeinsamer Schlüssel ausgehandelt werden, und der Client muss überprüfen können, dass der Server tatsächlich der ist, für den er sich ausgibt. Das geschieht im *TLS-Handshake*, der gleich nach dem TCP-Handshake stattfindet. Stark vereinfacht läuft er so ab:

1. Der Client sendet ein **ClientHello**, in dem er seine unterstützten TLS-Versionen, Cipher Suites (d. h. Verschlüsselungsverfahren) und eine Zufallszahl mitteilt.
2. Der Server antwortet mit einem **ServerHello**, in dem er die Auswahl der TLS-Version und der Cipher Suite trifft und ebenfalls eine Zufallszahl schickt. Zusätzlich übermittelt er sein *Zertifikat*.
3. Der Client überprüft das Zertifikat: Ist es gültig? Wurde es von einer ihm bekannten Zertifizierungsstelle ausgestellt? Passt der im Zertifikat angegebene Name zum Host, mit dem gerade gesprochen wird? Schlägt auch nur eine dieser Prüfungen fehl, bricht der Browser die Verbindung ab und zeigt dem*der Anwender*in eine Warnung an.
4. Anschließend führen beide Seiten durch asymmetrische Kryptografie einen Schlüsselaustausch durch und leiten daraus einen gemeinsamen *symmetrischen* Sitzungsschlüssel ab. Die eigentliche Datenübertragung erfolgt später symmetrisch, was wesentlich schneller ist.
5. Beide Seiten signalisieren mit einer **Finished**-Nachricht, dass ab jetzt verschlüsselt kommuniziert wird.

Das Ergebnis ist ein verschlüsselter Tunnel durch die bestehende TCP-Verbindung, in dem HTTP-Nachrichten vertraulich, unverändert und authentisch übertragen werden können.

1.9.3 Zertifikate und Zertifizierungsstellen

Ein *Zertifikat* im Sinne von TLS ist ein digital signiertes Dokument, das den öffentlichen Schlüssel eines Servers mit einem Domain-Namen verknüpft und von einer *Zer-*

tifizierungsstelle (Certification Authority, CA) ausgestellt wurde. Eine bekannte (und kostenlose) CA ist *Let's Encrypt*⁶, daneben gibt es kommerzielle Anbieter wie *DigiCert*, *Sectigo* oder *GlobalSign*.

Damit Browser Zertifikate überhaupt überprüfen können, liefern sie eine vorinstallierte Liste vertrauenswürdiger CAs mit (den sogenannten *Trust Store*). Wenn das Zertifikat eines Webservers von einer CA aus dieser Liste signiert wurde (oder von einer Zwischen-CA, die wiederum von einer vertrauenswürdigen CA signiert wurde), gilt es als vertrauenswürdig – der Browser zeigt das kleine Schloss-Symbol neben der URL an. Ansonsten erscheint eine Warnung.

Für Webentwickler*innen bedeutet das in der Praxis: Auf Produktivsystemen wird immer TLS eingesetzt, und der Webserver wird so konfiguriert, dass Anfragen auf Port 80 automatisch auf Port 443 umgeleitet werden. In der lokalen Entwicklungsumgebung (etwa mit *fhoee-web-dock*, das ab Vorlesung 2 zum Einsatz kommt) ist der TLS-Einsatz dagegen meist optional.

1.10 HTTP: Das Protokoll des Web

An diesem Punkt ist eine vollständige, verschlüsselte Verbindung zum Webserver aufgebaut. Über sie wird nun die eigentliche Anfrage verschickt. HTTP, das *Hypertext Transfer Protocol*, ist das Protokoll, über das im Web Inhalte ausgetauscht werden. Es ist ein *Anwendungsschicht-Protokoll* und folgt dem *Request-Response-Schema*: Der Client schickt einen Request, der Server antwortet mit einer Response, fertig. Der Server beginnt keine Kommunikation von selbst⁷.

1.10.1 Nachrichtenformat

HTTP-Nachrichten sind – zumindest in den Versionen 1.0 und 1.1 – *textbasiert* und damit bemerkenswert menschenlesbar. Eine HTTP-Nachricht besteht aus drei Teilen:

1. einer *Startzeile*,
2. einer Reihe von *Headerfeldern* (je ein Name-Wert-Paar pro Zeile) und
3. einem optionalen *Message Body*, der mit einer leeren Zeile vom Header-Block getrennt ist.

Ein einfacher HTTP-Request für das Beispielszenario sieht etwa so aus:

```
GET / HTTP/1.1
Host: www.fh-ooe.at
User-Agent: Mozilla/5.0 (...) Firefox/126.0
Accept: text/html,application/xhtml+xml,*/*;q=0.8
Accept-Language: de-AT,de;q=0.9,en;q=0.8
```

Die Startzeile besteht hier aus drei Teilen: der *Methode* (**GET**), dem *Request-URI* (/) und der *HTTP-Version* (HTTP/1.1). Danach folgen mehrere Headerfelder, dann eine

⁶<https://letsencrypt.org/>

⁷Dieser Grundsatz wird in neueren Versionen wie HTTP/2 durch sogenannte *Server-Pushes* etwas aufgeweicht, bleibt aber für unser Verständnis weiterhin eine gute Faustregel.

Leerzeile und im Fall eines GET-Requests kein Body.

Die Antwort des Servers könnte so aussehen:

```
HTTP/1.1 200 OK
Date: Mon, 09 Apr 2026 06:50:00 GMT
Server: Apache/2.4.58 (Ubuntu)
Content-Type: text/html; charset=UTF-8
Content-Length: 11321
```

```
<!DOCTYPE html>
<html lang="de">
...
</html>
```

Auch gibt es Startzeile, Headerblock, Leerzeile und Message Body. In der Startzeile einer Response stehen die HTTP-Version, ein *Statuscode* und eine zugehörige *Reason Phrase*.

1.10.2 HTTP-Methoden

Die *Methode* (auch *Verb*) gibt an, welche Art von Operation der Client auf der adressierten Ressource wünscht. Die wichtigsten Methoden sind:

GET Eine Ressource abrufen, ohne Seiteneffekte. GET-Anfragen dürfen vom Browser gecached und in die History aufgenommen werden. Parameter werden über den Query-String übergeben (siehe Abschnitt 1.3).

POST Daten an den Server senden, typischerweise zum Anlegen einer neuen Ressource oder zum Auslösen einer Aktion. Die Daten stehen im Message Body und sind daher in der URL nicht sichtbar. POST-Anfragen werden nicht gecached und nicht in der History gespeichert.

PUT Eine Ressource vollständig ersetzen oder anlegen.

DELETE Eine Ressource löschen.

PATCH Eine Ressource teilweise ändern.

HEAD Wie GET, aber der Server liefert nur die Header ohne Body. Nützlich, um z. B. die Größe einer Datei zu ermitteln, ohne sie komplett zu laden.

OPTIONS Fragt ab, welche Methoden auf einer bestimmten Ressource erlaubt sind. Spielt eine wichtige Rolle bei CORS (siehe Kapitel 11).

Für diese Lehrveranstaltung sind vor allem GET und POST wichtig. Kapitel 11 widmet sich den HTTP-Methoden im Detail.

1.10.3 Statuscodes

Der Server signalisiert in jeder Response mit einem dreistelligen *Statuscode*, wie es der Anfrage ergangen ist. Die erste Ziffer grenzt die Klasse ein:

1xx – *Informational* Zwischenmeldung, selten direkt sichtbar.

2xx – *Successful* Die Anfrage war erfolgreich, z. B. 200 OK, 201 Created, 204 No Content.

3xx – *Redirection* Der Client muss weiterspringen, z.B. 301 Moved Permanently, 302 Found, 304 Not Modified.

4xx – *Client Error* Der Client hat etwas falsch gemacht, z.B. 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 405 Method Not Allowed.

5xx – *Server Error* Der Server ist schuld, z.B. 500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable.

Diese Einteilung werden ebenfalls in Kapitel 11 zu REST-APIs wieder aufgegriffen, da sie dort zentral benötigt werden.

1.10.4 Wichtige Headerfelder

Header sind das Arbeitstier von HTTP: Praktisch alles, was über den eigentlichen Ressourcen-Inhalt hinaus transportiert werden soll, läuft über Header. Einige wichtige Vertreter, die auch in späteren Vorlesungen vorkommen werden.

Host Pflicht ab HTTP/1.1. Gibt an, an welchen Hostnamen sich der Request richtet. Auf diese Weise kann ein einzelner Webserver mit einer einzelnen IP-Adresse mehrere Websites bedienen (*virtual hosting*).

User-Agent Identifikation des Clients (Browser, Version, Betriebssystem).

Accept, Accept-Language, Accept-Encoding Die Datenformate, Sprachen bzw. Kompressionsverfahren, die der Client verarbeiten kann. Der Server kann daraufhin seine Antwort entsprechend anpassen (*Content Negotiation*).

Content-Type Formatangabe für den Message Body, etwa `text/html; charset=UTF-8` oder `application/json`.

Content-Length Länge des Message Body in Bytes.

Set-Cookie / Cookie Server setzt bzw. Client sendet Cookies. Grundlage für Sessions (siehe Vorlesung 8).

Location Im Zusammenhang mit 3xx-Redirects: Wohin soll der Client als nächstes springen?

Cache-Control Steuert, wie und wie lange Antworten gecached werden dürfen.

1.10.5 HTTP-Versionen im Überblick

HTTP existiert seit Anfang der 1990er-Jahre und es hat sich in mehreren Schritten weiterentwickelt:

HTTP/1.0 (1996) Erste standardisierte Version. Pro Request wird eine eigene TCP-Verbindung geöffnet und nach Erhalt der Response wieder geschlossen. Bei Seiten mit vielen Ressourcen (Bilder, CSS, JS) ist das sehr ineffizient.

HTTP/1.1 (1999) Die Verbindung bleibt standardmäßig geöffnet (*Keep-Alive*), so dass mehrere Requests über dieselbe TCP-Verbindung abgewickelt werden können. Neu sind ebenfalls *Pipelining* (mehrere Requests nacheinander ohne auf die Antwort zu warten) und der verpflichtende **Host**-Header. Das ist die Version, die hier in allen Beispielen verwendet wird.

HTTP/2 (2015) Binäres Nachrichtenformat, Header-Kompression, *Multiplex* (mehrere logische Streams über eine einzige TCP-Verbindung, ohne sich gegenseitig

zu blockieren) und optionaler *Server Push*. Funktional identisch mit 1.1 (gleiche Methoden, gleiche Statuscodes), aber wesentlich effizienter.

HTTP/3 (2022) Verwendet anstelle von TCP das auf UDP basierende Protokoll *QUIC*. Das sorgt insbesondere bei verlustbehafteten Mobilverbindungen für spürbar kürzere Latenzen, weil ein verlorenes Paket nicht alle parallelen Streams ausbremst.

Für diese Einführung – also das Verständnis, wie HTTP funktioniert und wie sich HTTP-Nachrichten auf Serverseite verarbeiten lassen – ist HTTP/1.1 das richtige Abstraktionslevel. Alle höheren Versionen liefern dieselben Methoden, Statuscodes und Header; es ändert sich im Wesentlichen nur, wie diese Nachrichten konkret über die Leitung wandern. Für PHP-Skripte ist dies in der Regel nicht relevant.

1.10.6 HTTP ist zustandslos

Eine fundamentale Eigenschaft von HTTP ist, dass das Protokoll *stateless* ist. Jeder Request wird isoliert betrachtet, der Server „erinnert sich“ zunächst einmal nicht daran, dass derselbe Client vor einer Sekunde schon einmal etwas angefragt hat. Für viele Webanwendungen ist das aber nicht ausreichend – ein Online-Shop muss sich merken, was im Warenkorb liegt, und eine Login-geschützte Seite muss sich merken, wer sich angemeldet hat.

Um diesen Zustand trotzdem über mehrere Requests hinweg zu erhalten, wurden *Cookies* und darauf aufbauend *Sessions* eingeführt. Beide sind Thema von Vorlesung 8. Es benötigt aufgrund der *stateless* Eigenschaft von HTTP also zusätzliche Mechanismen, um Zustand simulieren zu können.

1.11 Der Gesamtablauf im Überblick

Mit der HTTP-Antwort hat der Client also die angeforderte Webseite erhalten, rendert sie und zeigt sie an. Damit ist der Kommunikationsablauf beendet. Was aus der User*innen-Perspektive nach einem simplen Prozess aussieht, involviert eine ganze Reihe an Protokollen und Abläufen. Abbildung 1.7 zeigt die Abfolge aller Schritte, die beim Aufruf von <https://www.fh-ooe.at/> stattfinden – vom Tastendruck bis zur fertig gerenderten Seite.

Bemerkenswert ist die Vielzahl unterschiedlicher Protokolle, die für eine scheinbar einfache Aktion – eine Webseite öffnen – zusammenspielen müssen: DHCP, DNS, ARP, IP, Ethernet, TCP, TLS und schließlich HTTP. Jedes einzelne dieser Protokolle hat eine klar umrissene Aufgabe, und erst aus diesem Zusammenspiel ergibt sich die Robustheit, mit der das Web in den letzten Jahrzehnten skaliert hat.

Ergänzend sei angemerkt: Schritt 9 umfasst tatsächlich nicht nur einen einzelnen Request, sondern eine ganze Serie. Die ursprüngliche HTML-Datei referenziert typischerweise eine Reihe weiterer Ressourcen – Stylesheets, JavaScript-Dateien, Bilder, Schriftarten, Icons. Jede dieser Ressourcen erfordert einen eigenen Request, der (dank Keep-Alive in HTTP/1.1 und Multiplexing in HTTP/2) meist über dieselbe TCP-Verbindung läuft. Der Browser folgt also iterativ den Referenzen, bis die Seite vollständig geladen und gerendert werden kann.

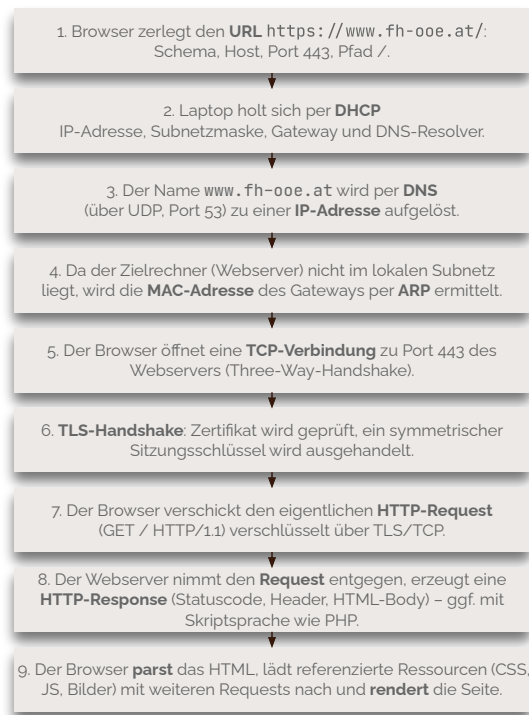


Abbildung 1.7: Der vollständige Ablauf eines Webrequests im Überblick. Jeder einzelne Schritt verwendet eines oder mehrere der in diesem Kapitel besprochenen Protokolle.

1.11.1 Brücke zur serverseitigen Entwicklung

Bis zu diesem Punkt wurde der gesamte Webrequest als Weg von A nach B betrachtet. Im weiteren Verlauf der Vorlesung steht vor allem ein einziger Schritt aus diesem Ablauf im Mittelpunkt: Schritt 8, also das, was auf dem Webserver geschieht, sobald er den Request entgegengenommen hat. In diesem Schritt wird entschieden, *was* an den Client zurückgeht: eine statische HTML-Datei aus dem Dateisystem, eine dynamisch erstellte HTML-Seite mit PHP, eine JSON-Antwort einer API, ein Redirect, eine Fehlermeldung oder auch andere Dinge sind hier möglich.

Serverseitige Programmiersprachen wie PHP setzen genau hier an. Trifft ein Request ein, dessen Pfad auf eine Datei mit der Endung `.php` verweist, übergibt der Webserver diese Datei an die PHP-Engine. Die Engine liest die Datei ein, führt den darin enthaltenen Code aus, sammelt die dabei erzeugte Ausgabe und gibt sie zusammen mit passenden HTTP-Headern als Response an den Webserver zurück. Für den Browser sieht das Ergebnis genauso aus wie bei einer statischen HTML-Datei – er erhält eine ganz normale HTTP-Response mit HTML-Body. Ob dieser HTML-Body aus einer Datei gelesen oder dynamisch erzeugt wurde, ist von außen nicht zu unterscheiden. Mehr dazu in Vorlesung 2.

1.12 Weiterführende Literatur

Für einen umfassenden Einstieg in die Netzwerk-Grundlagen bieten sich zwei Standardwerke an: [23] wählt denselben Zugang wie dieses Kapitel, es beginnt beim Anwendungsfall und arbeitet sich schichtweise nach unten. Deutschsprachige Alternativen sind [40, 53]. Alle drei Werke decken alle Abschnitte dieses Kapitels in deutlich größerer Tiefe ab und eignen sich für alle, die die Grundlagen ausführlicher nachlesen möchten.

Wer einzelne Teile des besprochenen Kommunikationsablaufs mit wunderbaren Katzenillustrationen nachlesen möchte, wird bei [1] fündig.

Für HTTP sind die RFCs die verbindliche Referenz. Zentral sind dabei RFC 9110 (*HTTP Semantics*) [7], das die semantischen Grundlagen von Methoden, Statuscodes und Headern definiert, sowie RFC 9112 (*HTTP/1.1*) [8] für die konkrete Nachrichtensyntax. Eine praxisnäher aufbereitete und laufend aktualisierte Darstellung findet sich bei MDN in [29]. Sie bietet neben Referenzmaterial auch viele kleine Beispiele zu einzelnen Headern, Methoden und Statuscodes.

Vorlesung 2

PHP-Grundlagen Teil 1

Nachdem in Vorlesung 1 die grundlegende Funktionsweise und der Weg eines Webrequests dargestellt wurden, beginnt nun der eigentliche Inhalt dieser Lehrveranstaltung: serverseitige Webentwicklung mit der Skriptsprache PHP.

2.1 Allgemeines zu PHP

PHP – ein rekursives Akronym für *PHP: Hypertext Preprocessor* – ist eine serverseitige Skriptsprache, die vornehmlich zur Erzeugung von HTML-Inhalten verwendet wird. Die Sprache ist (wie JavaScript) interpretiert, es wird also der Code zur Laufzeit eingelesen und verarbeitet.

PHP ist ein Open-Source-Projekt der PHP Group, die zentrale Webseite lautet <https://www.php.net/>.

2.1.1 Geschichte

PHP wurde vom dänischen Programmierer Rasmus Lerdorf entwickelt. Er stellte im Juni 1995 seine *Personal Homepage Tools* (PHP Tools) vor. Sie waren eine Reihe von in der Programmiersprache C geschriebenen Funktionen, mit denen er Zugriffe auf seinen Online-Lebenslauf protokollieren konnte.

Im Zuge seiner Arbeit entwickelte Lerdorf diese Tools weiter und veröffentlichte im April 1996 *PHP/FI*. FI stand hierbei für Form Interpreter. Diese Version kann schon als eigene, wenn auch noch begrenzte Skriptsprache angesehen werden, da bereits ein eigener Parser die verschiedenen Befehle interpretierte.

Im Juni 1998 erschien *PHP3*. Das Projekt war mittlerweile von einer One-Man-Show zu einem kleinen Open-Source-Projekt gewachsen. Der Parser wurde von Grund auf neu geschrieben, zusätzliche Funktionen wie Datenbankverbindung trugen zu einer größeren Verbreitung von PHP bei.

Im Jahr 2000 erschien *PHP4*. Mittlerweile hatten die Entwickler des Parsers (Andi Gutmans und Zeev Suraski) eine eigene Firma – Zend Technologies – gegründet und schufen mit der Zend Engine 1 den Kern der PHP-Implementierung. Durch das enorme Wachstum des World Wide Webs Ende der 1990er Jahre gewann auch PHP stark an Bedeutung. Durch seine einfache Erlernbarkeit etablierte es sich zu dieser Zeit als die dominante Skriptsprache im Web.

2004 erschien dann die lange erwartete *Version 5* von PHP. Kern war die Zend Engine II, die nun erstmals auch objektorientierte Entwicklung in PHP unterstützte. Weitere Konzepte wie Exceptions, Integration weiterer Datenbank Systeme oder verschiedener Erweiterungen wie XML-Parsern machten diese Version zu einer der wichtigsten im PHP Lebenszyklus.

Die *Version 5.1* (vom November 2005) führte vor allem Leistungsverbesserungen ein, in *Version 5.2* (November 2006) wurde die Speicherverwaltung verbessert, der Umgang mit dem Datenformat JSON hinzugefügt sowie neue Methoden für die Zeit- und Datumsverwaltung eingeführt.

Version 5.3 (Ende Juni 2009) brachte die Einführung von Namespaces, verbesserte XML-Unterstützung und noch einige weitere Neuerungen. In *PHP 5.4* vom März 2012 wurden Traits (wiederverwendbare Methodensammlungen), eine Kurzschreibweise für Arrays sowie weitere Sprachverbesserungen und Sicherheitsfeatures eingeführt.

PHP 5.5 (Juni 2013) führte die Unterstützung von Generatoren und eine Verbesserung der Ausnahmebehandlung ein. *PHP Version 5.6* wurde im August 2014 veröffentlicht. Neu waren konstante skalare Ausdrücke, variadische Funktionen oder auch automatisches Argument-Unpacking.

Anfang Dezember 2015 erschien *PHP 7*. Dabei wurde sehr viel der zugrundeliegenden Zend Engine neu geschrieben, was vor allem Performancegewinn brachte. Ebenso wurden viele alte Funktionalitäten, die aus Kompatibilitätsgründen in Version 5 enthalten waren, entfernt. Ebenso gibt es viele neue Sprachfunktionen wie etwa skalare Typdeklarationen, Deklarationen für Rückgabewerte, neue Operatoren, anonyme Klassen und noch einiges mehr¹.

Im Dezember 2016 wurde *PHP 7.1* veröffentlicht. Neue Funktionen umfassen sogenannte „Nullable Types“, den Rückgabetyp `void`, mehrere Sichtbarkeiten für Klassenkonstanten und noch weitere Dinge².

PHP 7.2 wurde Ende November 2017 vorgestellt. Hierbei gab es nur kleine Änderungen, wie etwa die Möglichkeit, `object` als Rückgabetyp oder Typdeklaration anzugeben, die Möglichkeit, abstrakte Methoden zu überschreiben, ein neuer Algorithmus für die Passwort-Hashing API, sowie einige weitere Änderungen³.

PHP 7.3 wurde im Dezember 2018 vorgestellt. Hier kamen Änderungen für die Heredoc- und Nowdoc-Syntax sowie einige Änderungen bei Stringfunktionen, die die Übergabeparameter jetzt strenger prüfen⁴.

PHP 7.4 erschien Ende November 2019. Eine große und wichtige Neuerung waren sogenannte „typed properties“, also die Möglichkeit, Datentypen bei Klasseneigenschaften zu spezifizieren. Ebenso wurden Pfeilfunktionen (arrow functions), der Null coalescing Zuweisungsoperator und noch einige weitere Dinge hinzugefügt⁵.

PHP 8.0 wurde am 20. November 2020 veröffentlicht. Die Version brachte einige Neuigkeiten mit, die zum Teil mit alten Versionen nicht mehr kompatibel sind. Wichtige Änderungen waren unter anderem eine „just-in-time Kompilation“, das Statement `match`, Union-Types, benannte Argumente, Constructor-Property-Promotion oder der

¹<https://www.php.net/manual/de/migration70.new-features.php>

²<https://www.php.net/manual/de/migration71.new-features.php>

³<https://www.php.net/manual/de/migration72.new-features.php>

⁴<https://www.php.net/manual/de/migration73.new-features.php>

⁵<https://www.php.net/manual/de/migration74.new-features.php>

Nullsafe-Operator⁶.

PHP 8.1 erschien am 25. November 2021. Neu waren unter anderem der Enumeration-Datentyp, der **never** Rückgabetypp für Funktionen, die nie zurückkehren, finale Klassenkonstanten oder **readonly** Eigenschaften⁷.

PHP 8.2 wurde am 8. Dezember 2022 vorgestellt. Es wurden **readonly**-Klassen eingeführt, Verbesserungen am Typensystem vorgenommen (Rückgabewerte **null** und **true**), Konstanten in Traits sowie ein neuer, zuverlässiger Zufallszahlengenerator namens **Randomizer** vorgestellt⁸.

PHP 8.3 ist am 23. November 2023 erschienen. Neuerungen waren unter anderem typisierte Klassenkonstanten, anonyme **readonly**-Klassen, eine Funktion zur JSON-Validierung, sowie Verbesserungen am **Randomizer**⁹.

PHP 8.4 wurde am 21. November 2024 veröffentlicht. Neu waren Property Hooks und asymmetrische Sichtbarkeit in Klassen, die Getter- und Setter-Methoden vereinfachen bzw. reduzieren, Updates in den DOM-Klassen und Lazy Objects¹⁰.

Die aktuellste Version ist *PHP 8.5*, welches am 20. November 2025 veröffentlicht wurde. Zentrale Neuerung ist der Pipe-Operator (**|>**), der das Ergebnis des linken Ausdrucks als Argument an den Funktionsaufruf auf der rechten Seite, um verschachtelte Aufrufe in eine linear lesbare Kette zu transformieren. Weiters neue Möglichkeiten zum Clonen von Objekten, Verbesserungen für Closures und Methoden zum URI-Parsing¹¹. Die aktuelle Release-Version ist 8.5.5 vom 9. April 2026.

Eine Version 6 von PHP gab es übrigens nie. Zentraler Kern war die komplette Neuimplementierung unter Berücksichtigung von Unicode, was sich jedoch aus vielfältigen Gründen als problematisch herausstellte und daher verworfen wurde. Einige geplante Neuerungen flossen bereits ab PHP 5.3 ein, der flächendeckende Unicode-Support wurde jedoch nicht implementiert.

2.1.2 Alternativen

PHP ist nicht die einzige Möglichkeit, serverseitig HTML-Code zu erzeugen. Die folgenden Technologien existieren ebenfalls – die Liste ist jedoch bei weitem nicht vollständig.

JavaScript

JavaScript wurde ursprünglich als clientseitige Sprache entworfen (wie sie auch in Web Fundamentals verwendet wurde), durch die Entwicklung von *Node.js*¹² wurde jedoch auch der serverseitige Einsatz populär. Mit seiner nicht-blockierenden Event-Loop-Architektur bietet Node.js eine leistungsstarke Umgebung für skalierbare Netzwerkanwendungen. Ein bekanntes Framework ist etwa *Express.js*¹³. Node.js ist Thema in Web Architectures (WAR3VO).

⁶<https://www.php.net/manual/de/migration80.new-features.php>

⁷<https://www.php.net/manual/de/migration81.new-features.php>

⁸<https://www.php.net/manual/de/migration82.new-features.php>

⁹<https://www.php.net/manual/de/migration83.new-features.php>

¹⁰<https://www.php.net/manual/de/migration84.new-features.php>

¹¹<https://www.php.net/manual/en/migration85.new-features.php>

¹²<https://nodejs.org/>

¹³<https://expressjs.com/>

Python

*Python*¹⁴ ist eine universell einsetzbare, höhere Programmiersprache die objektorientiert oder auch funktional eingesetzt werden kann. Im Web ist sie die Grundlage bekannter Webframeworks wie etwa Django¹⁵ oder Flask¹⁶ und wird zumeist auch mit diesen verwendet. Darüber hinaus ist die Sprache in der wissenschaftlichen Community für mathematische Berechnungen oder Machine-Learning beliebt und wird auch im MTD-Studium unterrichtet.

Java

Die Programmiersprache *Java*¹⁷ ist eine weit verbreitete, objektorientierte Programmiersprache, die für ihre Portabilität, Sicherheit und Robustheit bekannt ist und sehr oft in komplexen Projekten zum Einsatz kommt. Es existieren eine Vielzahl an speziellen *Java Web-Frameworks*, mit denen sich dynamisch Inhalte erzeugen lassen. Beispiele sind etwa Spring¹⁸ oder Struts¹⁹. Diese Themen werden ausführlich im Masterstudium Interactive Media behandelt.

Scala

*Scala*²⁰ ist eine moderne Programmiersprache, die Konzepte der objektorientierten und der funktionalen Programmierung vereint. Sie läuft auf der Java Virtual Machine (JVM) und zeichnet sich durch ein starkes, statisches Typsystem sowie eine prägnante Syntax aus. Im Web-Bereich wird Scala häufig für hochperformante, skalierbare Backends und Systeme mit hoher Nebenläufigkeit eingesetzt. Ein bekanntes Framework zur Erzeugung dynamischer Web-Inhalte ist das *Play Framework*²¹. Durch die volle Interoperabilität mit Java können bestehende Java-Bibliotheken nahtlos eingebunden werden, was Scala besonders für komplexe Enterprise-Anwendungen und im Bereich Big Data attraktiv macht.

Ruby

*Ruby*²² ist eine primär objektorientierte Programmiersprache, die jedoch auch prozedural und funktional eingesetzt werden kann. Im Web wird sie meist zusammen mit Webframeworks oder als domänenspezifische Sprache verwendet. Bekannte Vertreter sind etwa Ruby on Rails²³ oder Sinatra²⁴. Die Sprache hat sich nebenbei auch als Skriptsprache etabliert, die serverseitige Aufgaben (z. B. als Ersatz für Bash-Skripte) ausführt.

¹⁴<https://www.python.org/>

¹⁵<https://www.djangoproject.com/>

¹⁶<https://palletsprojects.com/projects/flask/>

¹⁷<https://www.oracle.com/java/>

¹⁸<https://spring.io/>

¹⁹<https://struts.apache.org/>

²⁰<https://www.scala-lang.org/>

²¹<https://www.playframework.com/>

²²<https://www.ruby-lang.org/>

²³<https://rubyonrails.org/>

²⁴<https://sinatrarb.com/>

Ebenso sind bekannte Softwarepakete, wie etwa der statische Seitengenerator Jekyll²⁵, in Ruby geschrieben.

2.1.3 Verbreitung und Beispiele

Die Seite W3Techs führt eine Analyse der Verbreitung von serverseitigen Programmiersprachen durch [46]. Mitte April 2026 ergibt sich dabei folgende Verteilung:

- PHP – 71,7 % (2025: 74,7 %, 2024: 76,5 %, 2023: 77,6 %),
- Ruby – 6,7 % (2025: 6,2 %, 2024: 5,7 %, 2023: 5,2 %),
- JavaScript – 6,0 % (2025: 4,2 %, 2024: 3,2 %, 2023: 2,3 %),
- Java – 5,4 % (2025: 5,1 %, 2024: 4,7 %, 2023: 4,7 %),
- Scala – 5,0 % (2025: 4,1 %, 2024: 3,0 %, 2023: 2,8 %).

Dabei verwenden derzeit 58,2 % (2025: 41,9 %, 2024: 25,3 %, 2023: 9,6 %) aller PHP-Seiten PHP 8, 33,0 % (2025: 46,1 %, 2024: 58,0 %, 2023: 67,8 %) PHP 7 und 8,7 % (2025: 11,8 %, 2024: 16,6 %, 2023: 22,4 %) PHP 5. Ältere Versionen spielen praktisch keine Rolle mehr.

Folgende bekannte Seiten sind mit PHP erstellt:

- Facebook (<https://www.facebook.com/>, mit Hack²⁶ als „Dialekt“),
- Wikipedia (<https://www.wikipedia.org/>),
- Flickr (<https://flickr.com/>),
- Etsy (<https://www.etsy.com/>),
- Wordpress (<https://wordpress.com/>).

2.2 Serverseitige Webentwicklung

Während sich die Vorlesung zu Web Fundamentals lediglich mit der Erstellung von statischen Webseiten beschäftigte, sind mit PHP nun dynamische Seiten möglich.

2.2.1 Wozu überhaupt?

Statische Seiten eignen sich gut für gleichbleibende Inhalte. Diese werden einmal in HTML verfasst und gegebenenfalls gelegentlich angepasst. Dazu muss jedes Mal das HTML-Markup händisch geändert bzw. neu erzeugt werden. Ein automatisches Reagieren auf neue oder von Anwender*innen bereitgestellte Daten ist nicht möglich.

Dynamische Webseiten zielen genau darauf ab. Mit ihnen können Inhalte programmatisch erstellt werden, ohne diese genau kennen zu müssen. So kann etwa mittels eines Formulars der Benutzer*innenname abgefragt und auf allen Folgeseiten angezeigt werden. Für derartige Dinge reicht eine Markupsprache wie HTML also nicht mehr aus, es muss eine Skriptsprache zum Einsatz kommen. Nun könnte argumentiert werden, dass Client-side-JavaScript für solche Dinge durchaus geeignet sei. Dies stimmt jedoch nur begrenzt, denn JavaScript dient zum Verändern einer bereits bestehenden Seite, bietet aber (clientseitig) bei weitem nicht alle Funktionalitäten für dynamische Webseiten.

²⁵<https://jekyllrb.com/>

²⁶[https://de.wikipedia.org/wiki/Hack_\(Programmiersprache\)](https://de.wikipedia.org/wiki/Hack_(Programmiersprache))

Sollen etwa Daten aus einer Datenbank eingelesen und auf der Seite dargestellt werden, ist JavaScript im Browser allein nicht mehr geeignet.

Hier ist eine serverseitige Skriptsprache wie PHP nötig, die all diese Dinge erledigt, *bevor* die Seite an den Browser geschickt und angezeigt wird. Dies geschieht letztendlich aber wieder in HTML, d. h. serverseitige Skriptsprachen wie PHP generieren HTML-Markup, das an den Browser gesendet wird. Spätere Vorlesungen demonstrieren auch noch andere Anwendungszwecke wie REST APIs, die nicht HTML generieren, sondern Daten (z. B. im JSON-Format) liefern.

2.2.2 Funktionsweise

Während bei einer statischen Seite das HTML-Markup fix in der HTML-Datei festgeschrieben wird, können PHP-Dateien sowohl statische HTML-Inhalte als auch dynamische, mit PHP definierte Abschnitte enthalten. Da der Webbrowser jedoch nur HTML interpretieren und mit PHP-Code nichts anfangen kann (außer diesen vielleicht 1:1 als Text auszugeben), muss es eine Instanz geben, die diesen PHP-Code verarbeitet und ihn in entsprechendes HTML umwandelt. Diese Komponente ist die *PHP-Engine*, auch *PHP-Parser* oder *PHP-Interpreter* genannt. Sie ist Teil (ein sogenanntes Modul) eines Webserver (wie bereits in Abschnitt 1.4 erläutert). Somit können PHP-Seiten nicht einfach lokal aus dem Dateisystem geöffnet werden, sondern müssen immer von einem Webserver angefordert werden, da sonst die PHP-Engine ja nicht zur Verfügung steht.

Wird nun eine PHP-Seite im Browser angefordert (es soll etwa die Datei `file.php` geladen werden), so schickt der Browser einen Request an den Webserver, auf dem diese Seite liegt. Dies kann z. B. über den URL `https://www.myserver.com/file.php` (online im WWW), oder `http://localhost:8080/file.php` (lokal am eigenen Rechner, mehr dazu in Abschnitt 2.2.3) sein. Im Webserver wird nun die Ressource (`file.php`) aus dem Dateisystem angefordert. Aufgrund der Dateiendung `.php` weiß der Webserver, dass er die Datei nun der PHP-Engine übergeben muss (wäre eine HTML-Datei angefordert worden, hätte er diese direkt zurücksenden können).

Die PHP-Engine liest nun den Inhalt der PHP-Datei (PHP-Code, evtl. auch statische HTML-Teile) ein und interpretiert diese. Im Rahmen dieser Verarbeitung erzeugt der enthaltene PHP-Code (in der Regel) HTML-Output, der im Anschluss zusammen mit den ebenfalls in der Datei enthaltenen HTML-Teilen als Response zurück an den Browser geschickt und dort angezeigt wird. Dies entspricht Schritt 8 in Abbildung 1.7.

Da es sich bei PHP um eine Skriptsprache handelt, kann der HTML-Output algorithmisch (mit Hilfe von Schleifen, Bedingungen etc.) erzeugt werden, darum wird auch von dynamischen Webseiten gesprochen.

Abbildung 2.1 stellt in Vorlesung 1, Abschnitt 1.4 gezeigten Kommunikationsablauf, erweitert um die PHP-Engine, grafisch dar.

2.2.3 Lokaler Webserver mit Docker

Es wird also zur Erzeugung von Webseiten mittels PHP ein Computer mit Webserversoftware und installiertem PHP-Modul benötigt. Sollen Datenbanken verwendet werden, muss auch eine solche installiert und konfiguriert sein. Während diese Einzelkomponenten bei der Zusammenstellung eines sogenannten Produktivwebserver, also eines Server, der für den Einsatz im Internet verwendet wird, in der Regel von einem Hoster

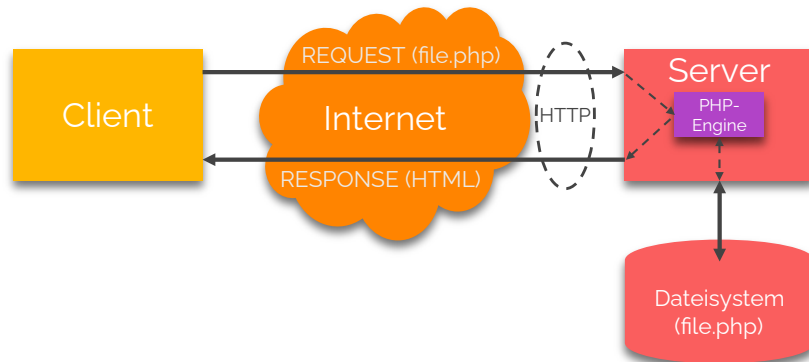


Abbildung 2.1: Client-Server Kommunikation über das Internet. Die PHP-Engine im Webserver kümmert sich dabei um das Verarbeiten des PHP-Codes und das Erzeugen der HTML-Ausgabe.

konfiguriert wird, gibt es für die lokale Webentwicklung spezielle Lösungen.

Bei *Docker* handelt es sich um eine sogenannte Containervirtualisierung. Hier wird nicht ein gesamtes Betriebssystem virtuell zur Verfügung gestellt (wie etwa bei einer virtuellen Maschine), viel mehr laufen Anwendungen eingeschränkt in Containern. Diese erhalten nur die notwendigen Ressourcen, um die darin enthaltenen Programme auszuführen.

Container sind in sogenannten Images definiert. Aus ihnen werden ein oder mehrere Container gestartet. Der Container ist dann die aktive Instanz in der gearbeitet wird. Container verwenden sehr oft Volumes, die das Dateisystem repräsentieren und im umgebenden Host-System gespeichert werden. Somit liegen die Dateien sicher und unabhängig von den Containern im Host-Betriebssystem und bleiben auch beim Zerstören der Container erhalten.

Das Projekt *fhoee-web-dock*²⁷ ist eine Sammlung von Docker-Containern, die für die Webentwicklung an der FH Oberösterreich erstellt wurden. Sie enthalten einen Webserver mit PHP, einen Container mit dem Datenbanksystem MariaDB sowie das Administrationstool phpMyAdmin.

Mit dem Befehl `docker compose up -d` können die Container erstmalig erstellt und gestartet werden. In weiterer Folge können diese mittels `docker compose stop` gestoppt und mit `docker compose start` wieder gestartet werden. `docker compose down` zerstört die Container komplett. Da die *fhoee-web-dock*-Images auf den offiziellen Docker-Images für PHP, MariaDB und phpMyAdmin beruhen, wird beim Erstellen des Containers immer die neueste Version verwendet. Über das `CleanReinstall.bat` bzw. `CleanReinstall.sh` können die Container gestoppt, die lokalen Images mit allen temporären Dateien entfernt, neu heruntergeladen, neu erstellt und wieder gestartet werden.

Laufende Container erstellen einen Webserver, der unter `http://localhost:8080` bzw. `https://localhost:7443` verfügbar ist. Damit die eigenen Webprojekte dort angezeigt werden, müssen sie in dem Ordner `webapp` erstellt werden. Eine Datei `webapp/file.php` ist somit im Browser unter `http://localhost:8080/file.php` verfügbar.

²⁷<https://github.com/Digital-Media/fhoee-web-dock>

Das Datenbank-Tool phpMyAdmin kann unter `http://localhost:8082` bzw. `https://localhost:8443` aufgerufen werden.

Auf die Datenbank kann aus dem PHP-Container über den Hostnamen `db` und den Port `3306` und vom eigenen Betriebssystem über den Host `localhost` und den Port `6033` zugegriffen werden.

2.3 PHP-Sprachgrundlagen

In den folgenden Abschnitten werden die Grundzüge der Skriptsprache PHP vorgestellt. PHP wird primär für die dynamische Generierung von Websites eingesetzt, weshalb mit der Einbettung von PHP-Code begonnen wird. Es folgen Abschnitte über die lexikale Struktur, Datentypen, Variablen, Operatoren und Kontrollstrukturen. Da die Rückgabe von Text (zum Erzeugen des HTML-Outputs) essentiell ist, schließt dieser Abschnitt mit der Ausgabe von Strings und wichtigen Stringfunktionen.

2.3.1 Einbettung in HTML

PHP-Code wird in der Regel in HTML-Markup eingebettet. Statische – also immer gleichbleibende Inhalte (wie z. B. der DOCTYPE oder auch Informationen im `<head>`) werden ganz normal in HTML verfasst. Alle veränderbaren Stellen, deren Ausgabe dynamisch sein soll und etwa von Eingaben der Benutzer*innen oder Abfragen aus einer Datenbank abhängen, werden über PHP generiert. Somit muss PHP-Code in HTML-Dateien (allerdings jetzt mit der Dateierweiterung `.php`) eingebettet werden, und zwar so, dass es für den PHP-Parser erkennbar ist, welche Bereiche er verarbeiten muss (den PHP-Code) und welche Bereiche er unverändert zurückgibt (den HTML-Code).

PHP Tags

Die am weitesten verbreitete Variante, PHP-Code zu kennzeichnen, ist mittels einer XML-konformen *Processing Instruction*, meist einfach *PHP Tags* genannt. Diese beginnt (wie jede Processing Instruction) mit einer spitzen Klammer und einem Fragezeichen, gefolgt vom Schlüsselwort `php`. Geschlossen wird die Instruktion mit einem Fragezeichen und der umgekehrten spitzen Klammer. Die folgende Zeile ist eine korrekte Einbettung von PHP-Code:

```
1 <?php echo "PHP"; ?>
```

Ein solcher „PHP-Block“ kann sich auch über mehrere Zeilen erstrecken:

```
1 <?php
2 echo "Erste Zeile";
3 echo "Zweite Zeile";
4 ?>
```

Das in beiden Beispielen gezeigte Sprachkonstrukt `echo` dient zur Ausgabe von Text. Mehr dazu in Abschnitt 2.3.2 auf Seite 2-10.

Das folgende Beispiel zeigt ein HTML-Grundgerüst, mit dem die Ausgabe im `<body>` mittels PHP erzeugt wird. Dies soll die bereits erwähnte Verzahnung/Einbettung von HTML und PHP verdeutlichen.

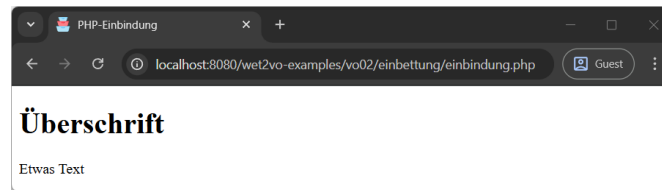


Abbildung 2.2: Die Ausgabe entspricht in diesem Fall der einer statischen HTML-Seite. Inhalte im `<body>` wurden aber vom PHP-Parser interpretiert und ausgegeben. Der URL `http://localhost:8080/wet2vo-examples/vo02/einbettung/einbindung.php` in der Adresszeile zeigt, dass die Datei über den Webserver (in diesem Fall über den *fhoee-web-dock* Docker Container) aufgerufen wurde.

```

1 <!DOCTYPE html>
2 <html lang="de">
3 <head>
4   <meta charset="utf-8">
5   <title>PHP-Einbindung</title>
6 </head>
7 <body>
8 <?php
9 echo "<h1>Überschrift</h1>";
10 echo "<p>Etwas Text</p>";
11 ?>
12 </body>
13 </html>

```

Der dadurch erzeugte Quellcode sieht wie folgt aus. Das Ergebnis im Browser ist in Abbildung 2.2 ersichtlich.

```

1 <!DOCTYPE html>
2 <html lang="de">
3 <head>
4   <meta charset="utf-8">
5   <title>PHP Einbindung</title>
6 </head>
7 <body>
8   <h1>Überschrift</h1>
9   <p>Etwas Text</p>
10 </body>
11 </html>

```

Achtung: Enthält eine Datei ausschließlich PHP-Code, sollte der schließende Tag entfallen um Probleme mit unerwünschten Whitespace-Zeichen am Ende zu verhindern:

```

1 <?php
2 echo "Hallo Welt";
3 // ... mehr Code
4 echo "Letzte Ausgabe";
5 // Das Script endet hier ohne Angabe von ?>

```

Short Open Tags

Neben dieser Variante existiert mit den *Short Open Tags* noch eine weitere. Dabei wird PHP-Code in eine Processing Instruction gesetzt, der Identifikator `php` entfällt

jedoch. Diese Variante ist zwar kürzer, ist aber in den meisten PHP-Konfigurationen aus Sicherheitsgründen (zu große Ähnlichkeit z. B. mit Processing Instructions) über die Einstellung `short_open_tag=false` in `php.ini` deaktiviert. Die Short Tag Syntax sieht wie folgt aus:

```
1 <? echo "PHP"; ?>
```

Short Echo Tags: Ein Spezialfall der Short Open Tags sind die *Short Echo Tags*. Sie sind für einzeilige Ausgabe von Text gedacht. Anstatt `<?php echo "Hallo Welt"; ?>` zu schreiben, kann ein Short Echo Tag wie folgt verwendet werden:

```
1 <?="Hallo Welt" ?>
```

Seit PHP 5.4 sind die Short Echo Tags immer verfügbar, auch wenn die Short Open Tags deaktiviert sind. Sie können daher bedenkenlos verwendet werden.

2.3.2 Ausgabe von Strings

Bevor mit weiteren Details zu PHP fortgefahren wird, ist es notwendig zu wissen, wie in PHP *Output* erzeugt werden kann. Denn schlussendlich sollen am Server mittels PHP ja HTML-Seiten erzeugt werden, die dann im Browser angezeigt werden. Auch die Beispiele in den folgenden Abschnitten geben oft Inhalt aus, um bestimmte Vorgänge im Browser testbar zu machen.

PHP kennt zwei Möglichkeiten, Text auszugeben:

- `echo` und
- `print()`.

Ausgabe mit echo

`echo`²⁸ ist keine Funktion, sondern ein sogenanntes Sprachkonstrukt (wie etwa auch `if`, `for`, `return` und ähnliche). Daher werden die Argumente (der Text), die `echo` zur Ausgabe übergeben werden, nicht geklammert. Die Syntax lautet:

```
1 echo $str;
```

`$str` ist eine Variable (siehe dazu Abschnitt 2.3.5), die Text (einen String) enthält. Folgende Beispiele zeigen die Ausgabe von Text mittels `echo`:

```
1 echo "Hallo Welt"; // Keine Variable, String wird direkt angegeben
2 echo "<h1>Überschrift</h1>"; // Ein HTML-Fragment wird ausgegeben
3 echo $text; // Inhalt der Variable $text wird ausgegeben
```

Mit den in Abschnitt 2.3.1 angesprochenen Short Echo Tags sieht die Syntax wie folgt aus:

```
1 <?=" $str ?>
```

Die drei Beispiele werden wie folgt damit geschrieben:

```
1 <?="Hallo Welt" ?>
2 <?="<h1>Überschrift</h1>" ?>
3 <?=" $text ?>
```

²⁸<https://www.php.net/manual/de/function.echo.php>

Ausgabe mit print()

`print()`²⁹ dient wie `echo` zur Ausgabe von Text. Es handelt sich ebenfalls um ein Sprachkonstrukt, das sich aber wie eine Funktion verhält. Daher wird der auszugebende Text in der Regel auch geklammert (wenngleich die Klammern nicht zwingend notwendig sind):

```
1 print($str);
```

Die drei Beispiele werden mit `print()` folgendermaßen angegeben:

```
1 print("Hallo Welt");
2 print("<h1>Überschrift</h1>");
3 print($text);
```

Da sich `print()` wie eine Funktion verhält, gibt es auch einen Rückgabewert. Dieser ist immer 1.

Wichtige Grundregel bei der Ausgabe

PHP-Skripte können nur Text zurückgeben. Dieser muss im Skript so aufbereitet werden, wie es der Verwendungszweck erfordert.

Dies bedeutet: Soll mit PHP eine Website oder zumindest ein Teil davon (z. B. ein Absatz) dynamisch generiert werden, so genügt es nicht nur, den Text eines Absatzes zurückzugeben, sondern es müssen auch die dazugehörigen HTML-Tags (z. B. `<p></p>`) Teil des zurückgegebenen Texts sein. Selbiges gilt für alle anderen Elemente.

Ein PHP-Skript, das nur aus `echo "Das ist etwas Text."` besteht, kann zwar ohne Probleme im Browser angezeigt werden. Jedoch wird wirklich nur diese eine Zeile Text im Quellcode aufscheinen – weder ein HTML-Grundgerüst, noch HTML-Tags etc. Um eine valide HTML-Seite zu erzeugen, muss entweder das komplette HTML-Grundgerüst mit mehreren `echo`- oder `print()`-Anweisungen ausgegeben werden (sehr umständlich), oder es wird in die PHP-Datei normales HTML-Markup eingefügt, das an den gewünschten Stellen durch PHP-Ausgaben ergänzt wird. Der Beispielcode in Abschnitt 2.3.1 verdeutlicht dies gut.

Der*die Autor*in des PHP-Skripts ist also dafür verantwortlich, dass korrektes HTML-Markup bei der Ausgabe erzeugt wird! Der HTML-Validator³⁰ hilft bei der Überprüfung, ob dies gelungen ist. Selbstverständlich kann dieser aber nur den finalen Output überprüfen, wie er im Browser ankommt. Das PHP-Skript selbst bekommt der Validator nie „zu Gesicht“ (selbst wenn, könnte er mit PHP-Code ohnehin nichts anfangen).

2.3.3 Lexikalische Struktur

Nachdem nun die Ausgabe von Text etwas vorweggenommen wurde, werden im Folgenden die einzelnen Sprachdetails von PHP erläutert. Zunächst die *lexikalische Struktur*. Sie beschreibt die Grundregeln, die für die Sprache gelten.

²⁹ <https://www.php.net/manual/de/function.print.php>

³⁰ <https://validator.w3.org/>

Groß- und Kleinschreibung

Die *Groß- und Kleinschreibung* in PHP wird gemischt gehandhabt. Bei Namen von Klassen und Funktionen, sowie Sprachkonstrukten und Schlüsselwörtern wie `echo`, `while`, `class` etc. spielt die Schreibweise *keine* Rolle. Die folgenden drei Zeilen sind also äquivalent:

- `echo "Hallo Welt";`
- `ECHO "Hallo Welt";`
- `EcHo "Hallo Welt";`

Bei Variablennamen jedoch spielt die Groß- und Kleinschreibung *immer* eine Rolle. `$name`, `$NAME` und `$NaMe` sind drei unterschiedliche Variablen.

Anweisungen und Strichpunkte

Eine *Anweisung* ist ein Stück Code, der etwas (Sinnvolles) tut. Es kann sich um das Zuweisen eines Werts zu einer Variable oder die Ausführung einer Funktion handeln. Anweisungen in PHP müssen immer mit einem Strichpunkt (;) abgeschlossen werden. Einzige Ausnahme ist die letzte Anweisung, bevor der PHP-Block geschlossen wird. Hier kann das Semikolon entfallen, es sei aber auch hier angeraten, es zu setzen, um mögliche Fehlerquellen zu minimieren. Wie auch in anderen Sprachen, können mehrere Anweisungen in einer Zeile geschrieben werden. Einige Beispiele:

```
1 <?php
2 $a = 1; echo "Hallo Welt!";
3 $name = "Igor";
4 ?>
```

Nach der Anweisung in Zeile 3 könnte der Strichpunkt auch entfallen, da der PHP-Block danach zu Ende ist.

Kommentare

PHP kennt die bereits von Java oder JavaScript bekannten ein- und mehrzeiligen *Kommentare*. Diese werden hinter zwei Schrägstriche (einzeilig) bzw. zwischen Schrägstrich und Stern sowie Stern und Schrägstrich (mehrzeilig) gestellt:

```
1 // Einzeiliges Kommentar
2 /* Ein
3    mehrzeiliges
4    Kommentar */
```

Zusätzlich gibt es noch das einzeilige, sogenannte Unix-Shell-Style bzw. Perl-Kommentar. Das hierbei verwendete Spezialzeichen ist die Raute (#), das Verhalten ist identisch mit dem einzeiligen Kommentar:

```
1 # Ein Unix-Shell-Style Kommentar
```

2.3.4 Datentypen

PHP unterstützt neun primitive *Datentypen*. Vier skalare Typen (Ganzzahlen, Gleitkommazahlen, boolesche Werte und Zeichenketten), drei zusammengesetzte Typen (Arrays, Objekte und Callbacks/Callables), sowie die beiden speziellen Typen `null` und

resource. Zunächst wird auf drei der vier skalaren Typen und null eingegangen, Strings folgen in Abschnitt 2.3.9. Arrays werden in Vorlesung 3 und Objekte in Vorlesung 4 behandelt. Details zu den zwei verbleibenden Typen können dem PHP-Handbuch [42] entnommen werden.

Ganzzahlen

Ganzzahlen oder Integer entsprechen dem Datentyp `long` der Programmiersprache Java. Die Größe ist abhängig von der Plattform und PHP-Version. Mit PHP 8 unter einem 64-bit Betriebssystem sind es üblicherweise 8 Bytes (64 Bits) und können daher einen Wertebereich von $-9.223.372.036.854.775.808$ ($-9,23$ Trillionen bzw. $-9,23 \cdot 10^{18}$) bis $9.223.372.036.854.775.807$ darstellen. Da nirgends genau festgelegt ist, wie viele Bytes eine Ganzzahl in PHP belegen muss, kann dieser Wert auch abweichen.

Ganzzahlen können als Dezimal-, Oktal-, Hexadezimal- oder Binärzahlen angegeben werden. Oktalzahlen werden ab PHP 8.1.0 mit führendem „0o“ angegeben, Hexadezimalzahlen wird „0x“ vorangestellt, bei Binärzahlen kommt „0b“ zum Einsatz:

- 735 (dezimal),
- 0o1337 (oktal),
- 0x2DF (hexadezimal),
- 0b001011011111 (binär).

Seit PHP 7.4 können Ganzzahlen durch den *numeric literal separator* besser lesbar geschrieben werden. Der Underscore (`_`) fungiert hierbei als Trenner für Dezimalstellen und kann bei Dezimalstellen etwa nach jeder dritten Zahl und bei Hexadezimal- und Binärzahlen nach jeder Vierergruppe eingefügt werden. Das Zeichen hat keine Auswirkung auf den Wert und wird vom Interpreter bei der Verarbeitung zuvor entfernt. Als einzige Restriktion gilt, dass der Underscore immer zwischen zwei Zahlen stehen muss. Er darf also nicht am Anfang oder Ende eines Zahlenwert vorkommen bzw. dürfen nicht mehrere Underscores aufeinanderfolgen. Eingesetzt kann er z. B. wie folgt werden:

- 1_000_000 (dezimal),
- 0xCAFE_FOOD (hexadezimal),
- 0b0101_1111 (binär).

Gleitkommazahlen

Gleitkommazahlen in PHP entsprechen dem Datentyp `double` in Java, sind also üblicherweise 8 Bytes (64 Bits) lang. Somit lassen sich Zahlen im Bereich von $-1,7 \cdot 10^{-308}$ bis $1,7 \cdot 10^{308}$ darstellen. Wiederum kann der Bereich vom aktuell verwendeten Betriebssystem abhängen.

Gleitkommazahlen können in zwei Schreibweisen angegeben werden. Mit Dezimalpunkt oder in der Exponentialschreibweise. In letzterer wird ein Wert mit einem „e“, gefolgt vom Exponenten notiert, was $\cdot 10^x$ („Mal zehn hoch x “) bedeutet:

- 3.1415,
- 6.02e7 (entspricht $6,02 \cdot 10^7$ bzw. 60.200.000).

Auch hier kann der *numeric literal separator* eingesetzt werden, um etwa lange Kommastellen besser lesbar zu machen:

- 6.674_083e-11 (entspricht $6,674083 \cdot 10^{-11}$ bzw. 0,00000000006674083).

Boolesche Werte

Die *booleschen Werte* **true** und **false** können in PHP genau wie in anderen Sprachen verwendet werden. Die Groß- und Kleinschreibung spielt dabei keine Rolle (auf php.net wird etwa meist **TRUE** und **FALSE** verwendet, da in PHP konstante Werte per Konvention in Großbuchstaben geschrieben werden).

In PHP ergeben auch andere Werte bei der Umwandlung auf den Typ Boolean den Wert **false**, können also mit logischen Operatoren verglichen werden:

- Der Integer 0,
- die Gleitkommazahlen 0.0 und -0.0,
- ein leerer String ("") und der String "0",
- ein Array ohne Elemente und
- der Typ **null**.

Alle anderen Werte ergeben somit **true**, z. B. 5, 3.5, "Hallo" oder auch [3, 5].

null

Der spezielle Datentyp **null** wird verwendet, um auszusagen, dass eine Variable keinen Wert enthält. Auch hier spielt die Groß- und Kleinschreibung keine Rolle, oft wird daher auch **NULL** verwendet. Eine neu erstellte Variable, der noch kein Wert zugewiesen wurde, hat immer den Wert **null**.

2.3.5 Variablen

Das *Variablenkonzept* von PHP unterscheidet sich in einigen Dingen von anderen Sprachen wie Java oder JavaScript. Verantwortlich dafür sind etwa die implizite Deklaration oder auch das schwache Typkonzept. Auch die Namensgebung ist speziell.

Variablennamen

Die Namen von PHP-Variablen beginnen immer mit einem Dollarzeichen (\$). Danach folgt entweder ein ASCII-Zeichen (Groß- oder Kleinbuchstabe von A–Z) oder ein Unterstrich (_). Alle darauffolgenden Zeichen können entweder wiederum ASCII-Zeichen, Unterstriche oder auch Zahlen von 0–9 sein. Vorsicht: PHP unterstützt ausschließlich ASCII-Zeichen bei der Benennung von Variablen. Umlaute oder andere, seltenere Sonderzeichen sind nicht erlaubt und führen zu Fehlern. Die Groß- und Kleinschreibung spielt, wie bereits in Abschnitt 2.3.3 erwähnt, eine Rolle.

Beispiel für gültige Variablennamen sind etwa:

- \$counter,
- \$nr_of_items,
- \$MaxSpeed oder
- \$_temp.

Folgende Variablennamen sind *nicht* gültig:

- `$current score` (Leerzeichen),
- `$überlauf` (Umlaut),
- `$|dash` (non-ASCII-Zeichen),
- `$3fach` (beginnt mit Zahl).

Die folgenden Variablen heißen zwar gleich, verweisen aber aufgrund der Groß- und Kleinschreibung auf unterschiedliche Werte:

- `$old_value`,
- `$Old_value`,
- `$old_Value` und
- `$OLD_VALUE`.

Deklaration

Variablen werden in PHP implizit deklariert. Dies bedeutet, es gibt keine spezielle Syntax oder kein eigenes Schlüsselwort, um eine Variable anzulegen. Wann immer der Wert für eine Variable gesetzt wird, wird diese erzeugt. Die Zuweisung eines Werts fungiert also als Deklaration.

Folgende Zeilen legen unterschiedliche Variablen an:

```
1 $number = 12;
2 $name = "Igor";
3 $sum = 3 + 5;
```

Dynamische und schwache Typbindung

Wie auch JavaScript, hat PHP eine *dynamische Typbindung*. Dies bedeutet, dass eine Variable beliebige Datentypen enthalten kann. Somit ist folgendes erlaubt:

```
1 $x = 42;
2 ...
3 $x = "fourtytwo";
```

PHP hat des Weiteren auch eine *schwache Typbindung*. Dies äußert sich darin, dass der Typ einer Variable zur Laufzeit von PHP umgewandelt werden kann, wenn dies notwendig erscheint. Dies wird auch implizite Typumwandlung genannt. Sie findet etwa statt, wenn eine nicht-boolesche Variable mit einem logischen Operator verglichen wird. Hierbei kommen die Regeln, die in Abschnitt 2.3.4 erwähnt wurden, zur Anwendung. Im folgenden Code wird die Integer-Variable `$a` implizit auf `true` umgewandelt und die `if`-Bedingung somit betreten:

```
1 $a = 1;
2 if ($a == true) {
3     echo "Typumwandlung in Boolean ist erfolgt!"; // Wird betreten
4 }
```

Ein anderes Beispiel für implizite Typumwandlung ist der `+`-Operator (siehe auch Abschnitt 2.3.6). Er wandelt alle Operanden in numerische Typen um. Werden also ein Integer-Wert und ein String, der einen numerischen Wert enthält, damit addiert, so geschieht die implizite Umwandlung. Das folgende Beispiel illustriert dies:

```

1 $b = "2";
2 $b += 4;
3 echo $b; // 6

```

Soll eine Typumwandlung selbst herbeigeführt werden, ist dies wie in Java über sogenanntes Casting möglich. Durch Voranstellen des gewünschten Zieltyps in Klammer wird die Umwandlung (wenn möglich) vorgenommen. Möglich sind die Typen (`bool`), (`int`), (`float`), (`string`), (`array`) und (`object`). Im folgenden Beispiel wird der String-Wert der auf einen Integer-Wert konvertiert und einer neuen Variable zugewiesen:

```

1 $str = "14";
2 $i = (int)$str; // 14

```

Die implizite Typumwandlung bietet jedoch auch großes Fehlerpotential, da sie leicht übersehen werden kann. Um sich davor zu schützen, können bei Vergleichsoperationen die Operatoren `===` bzw. `!==` zum Einsatz kommen. Sie überprüfen nicht nur auf gleichen Wert, sondern auf gleichen Typ. Auf das Beispiel von oben bezogen würde die folgende `if`-Bedingung somit *nicht* betreten werden, obwohl eine implizite Typumwandlung von 1 auf `true` stattfindet. Der Operator erkennt jedoch, dass die ursprünglichen Typen unterschiedlich waren:

```

1 $a = 1;
2 if ($a === true) {
3     echo "Dieser Operator lässt sich nicht überlisten!"; // Wird nicht betreten
4 }

```

Mit PHP 8 kam es hierbei zu einer wichtigen Änderung: werden Zahlen und nicht-numerische Strings mit dem `==`-Operator (Vorsicht, *nicht* `===`) verglichen, so erfolgt nun eine Umwandlung der Zahl in einen String und diese werden verglichen. Zuvor wurde der String in eine Zahl umgewandelt (das Ergebnis war hierbei immer 0) und diese wurden verglichen.

```

1 if (0 == "foo") {
2     echo "true"; // Vor PHP 8
3 } else {
4     echo "false"; // Ab PHP 8
5 }

```

Numerische Strings, also etwa `"42"`, werden nach wie vor in Zahlen umgewandelt und dann mit der Zahl verglichen.

2.3.6 Operatoren

Die verschiedenen *Operatoren* in PHP zur Zuweisung oder zum Vergleich von Werten sind denen von C, Java oder auch JavaScript sehr ähnlich. Die große Ausnahme bildet der String-Konkatenationsoperator, der nicht wie üblich das `+`, sondern der `.` ist. Im Folgenden wird auf die verschiedenen Operatoren eingegangen.

Zuweisungsoperatoren

Der `=`-Operator dient der *Zuweisung* von Werten an Variablen. Er kann mit arithmetischen Operatoren kombiniert werden, um eine Operation und Zuweisung in einem Schritt auszuführen. Folgende Kombinationen sind möglich:

- `=`: Zuweisung.
- `+=`: Addition und Zuweisung.
- `-=`: Subtraktion und Zuweisung.
- `*=`: Multiplikation und Zuweisung.
- `/=`: Division und Zuweisung.
- `%=`: Modulo und Zuweisung.
- `.=`: Stringkonkatenation und Zuweisung.

Einige Beispiele dazu:

```
1 $a = 5;
2 $a += 3; // 8
3 $b = "Hallo";
4 $b .= " WET2"; // Hallo WET2
```

Vergleichsoperatoren

Vergleichsoperatoren vergleichen zwei Werte links und rechts des jeweiligen Operators und geben einen booleschen Ausdruck als Resultat zurück. Folgende, meist aus anderen Sprachen bekannte, Operatoren gehören zu dieser Gruppe:

- `==`: Überprüfung auf gleichen Wert.
- `===`: Überprüfung auf gleichen Wert und gleichen Typ.
- `!=`: Überprüfung auf ungleichen Wert.
- `!==`: Überprüfung auf ungleichen Wert und ungleichen Typ.
- `<`: Kleiner als.
- `>`: Größer als.
- `<=`: Kleiner oder gleich als.
- `>=`: Größer oder gleich als.

Vor allem die Operatoren `==` und `===` sind (wie schon in Abschnitt 2.3.5 gezeigt) interessant. Folgendes Beispiel erläutert nochmals den Unterschied:

```
1 $a = 5;
2 $b = 5.0;
3
4 if ($a == $b) {
5     echo "Werte gleich";
6 }
7 if ($a === $b) {
8     echo "Werte und Typ gleich";
9 }
```

`$a` ist ein Integer mit dem Wert 5, `$b` ist eine Gleitkommazahl mit dem Wert 5,0. Die erste `if`-Bedingung überprüft, ob die Werte gleich sind. Diese Bedingung ist erfüllt, also wird der Text ausgegeben. Die zweite Bedingung testet, ob Wert *und* Typ gleich sind. Sie trifft nicht zu, da die erste Variable eine Ganzzahl, die zweite aber eine Gleitkommazahl ist.

Konditionaler Operator

Ebenfalls eine Art Vergleichsoperator ist der *konditionale Operator*. Er besteht aus den Zeichen `?` und `:`. Vor dem Fragezeichen steht ein boolescher Ausdruck, ergibt dieser `true`, so wird der Wert vor dem Doppelpunkt zurückgegeben, ansonsten der danach. Folgendes Beispiel verdeutlicht dies:

```
1 $test = true;
2 echo $test ? "Hallo" : "Kein Hallo"; // Hallo
```

Null Coalescing Operator

Der *Null Coalescing Operator* `??` verknüpft zwei Operanden miteinander und gibt den ersten zurück, wenn dieser existiert (d. h. nicht `null` ist) oder ansonsten den zweiten.

```
1 $name = $vorname ?? "Nobody";
```

Hierbei wird versucht, der Variablen `$name` der Wert von `$vorname` zuzuweisen. Da diese aber nicht existiert, wird der Wert „Nobody“ zugewiesen. Stünde zuvor die Zuweisung `$vorname = "Jane"`;, würde „Jane“ zugewiesen werden, da die Variable dann existent wäre.

Null Coalescing Assignment Operator

Mit dem *Null Coalescing Assignment Operator* kann ähnlich dem Null Coalescing Operator überprüft werden, ob ein Wert nicht existiert (d. h. `null` ist) und nur dann eine Zuweisung machen.

```
1 $hero ??= "Superman";
```

Existiert die Variable `$hero` *nicht*, dann wird der Wert „Superman“ zugewiesen. Gab es zuvor bereits etwa eine Zuweisung `$hero = "Batman"`;, dann bleibt dieser Wert erhalten und „Superman“ wird nicht zugewiesen.

Andere Schreibweisen hierfür sind etwa `$hero = $hero ?? "Superman"`; (mit dem Null Coalescing Operator) oder mit einer `if`-Bedingung:

```
1 if ($hero === null) {
2     $hero = "Superman";
3 }
```

Spaceship Operator

Der in PHP 7 eingeführte *Spaceship Operator* `<=>` ist ein Sonderfall unter den Vergleichsoperatoren. Er vergleicht zwar zwei Ausdrücke, gibt jedoch keinen booleschen Wert zurück, sondern `-1`, wenn der linke Wert kleiner als der rechte ist, `0`, wenn beide Werte gleich sind oder `1`, wenn der linke Wert größer als der rechte ist. Folgendes Beispiel stellt dieses Verhalten dar:

```
1 echo 1 <=> 1; // 0
2 echo 1.5 <=> 2.5; // -1
3 echo "b" <=> "a"; // 1
```

Arithmetische Operatoren

Arithmetische Operatoren repräsentieren die Grundrechnungsarten sowie die Modulo-, Potenz- und Inkrement-/Dekrement-Operatoren. Folgende Operatoren stehen zur Verfügung:

- `+`: Addition (aber *nicht* Strings zusammenhängen!).
- `-`: Subtraktion.
- `*`: Multiplikation.
- `/`: Division.
- `%`: Modulo (Rest der Division).
- `**`: Potenz.
- `++`: Inkrement (post und prä möglich).
- `--`: Dekrement (post und prä möglich).

Dazu einige Beispiele:

```
1 $a = 5 + 3; // 8
2 $b = 5 % 2; // 1
3 $c = ++$a; // 9 (Prä-Inkrement: $a wird auf 9 erhöht und $c zugewiesen)
4 $d = $a++; // 9 (Post-Inkrement: $a wird mit 9 $d zugewiesen und dann auf 10 erhöht)
```

Stringkonkatenation

Der wohl am häufigsten verwechselte Operator in PHP ist der zur *Stringkonkatenation* (Konkatenation = Verkettung von Zeichen oder Zeichenketten). In PHP wird dazu nämlich der Punkt (`.`) und *nicht* wie in den meisten anderen Sprachen das Plus-Zeichen (`+`) verwendet. Diese „Eigenheit“ wurde von der Skriptsprache Perl übernommen. Im folgenden Beispiel werden zwei Zeichenketten zusammengehängt:

```
1 $str = "Hallo " . "Welt";
2 echo $str; // Hallo Welt
```

Durch das schwache Typkonzept von PHP ist bei der Verwechslung von `.` und `+` jedoch Vorsicht geboten. Denn die folgende Anweisung wird vom PHP-Interpreter in PHP-Versionen vor 8.0 ohne Fehlermeldung ausgeführt:

```
1 $str = "Hallo " + "Welt";
2 echo $str; // 0, aber nur für PHP < 8.0
```

Die Ausgabe ist in diesem Fall die Zahl 0, denn PHP nimmt eine implizite Typumwandlung vor und wandelt beide Strings in Integer um. Da weder in „Hallo“ noch in „Welt“ numerische Werte enthalten sind, ist für beide Teile das Ergebnis 0 und `0 + 0` ergibt schließlich wiederum 0. Ab PHP 8 wird diese Umwandlung nicht mehr durchgeführt und es kommt zu einem schweren Fehler im Skript (*Fatal error: Uncaught TypeError: Unsupported operand types: string + string*).

Auch auf den umgekehrten Fall sei hingewiesen. Gegeben sei folgender Code:

```
1 $first = 100;
2 $second = 50;
3 echo $first . $second; // 10050
4 echo $first + $second; // 150
```

Die beiden Variablen enthalten Integerwerte. Bei der ersten Ausgabe mit `echo` werden diese mit einem Punkt aneinandergehängt. PHP konvertiert aufgrund des Konkatinationsoperators beide Ganzzahlen in Zeichenketten und hängt sie aneinander. Somit wird 10050 ausgegeben. Bei der zweiten Ausgabe werden beide Integerwerte mit dem arithmetischen Operator addiert, somit ist 150 das Ergebnis.

Die Beispiele illustrieren, dass hier durch einfache Verwechslung des Operators völlig unterschiedliche Ergebnisse und somit Fehler entstehen können, die oft nur schwer aufzufinden sind, da die Operationen alle syntaktisch korrekt sind. In PHP 8 wurde aber ein großer Teil dieses Problems, nämlich die (fälschliche) Umwandlung von nicht-numerischen Strings in Zahlen, gelöst.

Logische Operatoren

Zur booleschen Algebra unterstützt PHP die üblichen *logischen Operatoren*:

- `&&`: Logisches UND.
- `||`: Logisches ODER.
- `!`: Logisches NICHT.

Auch hierzu drei Beispiele:

```
1 $a = (4 > 2) && (5 < 3); // false
2 $b = !$a; // true
3 $c = ($a === false) || ($b === false) // true
```

Bitweise Operatoren

PHP unterstützt die üblichen *bitweisen Operatoren* `&` (bitweises UND), `|` (bitweises ODER), `^` (bitweises Exklusiv-ODER), `~` (bitweises NICHT), sowie `<<` (bitweise Verschiebung nach links) und `>>` (bitweise Verschiebung nach rechts). Wie auch schon bei JavaScript sei an dieser Stelle auf die fortführenden Programmierlehrveranstaltungen bzw. die Lehrveranstaltung Data Analysis verwiesen, wo diese Thematik ausführlich behandelt wird.

2.3.7 Kontrollstrukturen

PHP bietet die gängigen *Kontrollstrukturen*, nämlich Verzweigungen und Schleifen, an. Neben der bekannten Schreibweise mit geschwungenen Klammern existiert auch eine Schreibweise mit Doppelpunkten. Diese wird zur Vollständigkeit bei den Verzweigungen demonstriert, gilt aber auch bei den anderen Kontrollstrukturen.

Verzweigungen

Die `if-else`-Bedingung ist, wie auch in anderen Sprachen, die gebräuchliche Art, um Verzweigungen zu realisieren. Die gängige Syntax mit geschwungenen Klammern lautet:

```
1 if (Ausdruck) {
2     Anweisung;
3 } else {
4     Alternative Anweisung;
5 }
```

Wie bereits in der Einleitung zu den Kontrollstrukturen erwähnt, erlaubt PHP auch eine alternative Schreibweise mit Doppelpunkt statt der öffnenden Klammer und abschließendem Statement (hier **endif**) statt der schließenden Klammer. Diese Syntax ist gleichwertig, keinesfalls sollte sie jedoch mit der Klammerschreibweise vermischt eingesetzt werden:

```
1 if (Ausdruck):
2     Anweisung;
3 else:
4     Alternative Anweisung;
5 endif;
```

Wie auch in anderen Sprachen üblich, können im **else**-Zweig wieder neue **if**-Bedingungen begonnen werden. Diese Kaskade kann, wie auch schon bei JavaScript demonstriert, durch eine **switch**-Anweisung ersetzt werden. Folgt in einem **else**-Zweig ein weiteres **if**, soll stattdessen **elseif** verwendet werden. Es wird bevorzugt, da es sich um ein einziges Schlüsselwort handelt.

Es folgen einige verschiedene Beispiele:

```
1 // if-else-Bedingung
2 if ($name !== "") {
3     $username = $name;
4 } else {
5     $username = "John Doe";
6 }
7
8 $n = 1;
9
10 // if-elseif-else-Kaskade
11 if ($n === 1) {
12     echo "Die Zahl ist 1";
13 } elseif ($n === 2) {
14     echo "Die Zahl ist 2";
15 } else {
16     echo "Die Zahl ist weder 1 noch 2";
17 }
18
19 // Switch-Anweisung statt Kaskade
20 switch ($n) {
21     case 1:
22         echo "Die Zahl ist 1";
23         break;
24     case 2:
25         echo "Die Zahl ist 2";
26         break;
27     default:
28         echo "Die Zahl ist weder 1 noch 2";
29         break;
30 }
```

PHP 8: Match Expressions: Statt einer **switch**-Anweisung kann ab PHP 8 auch ein **match**-Ausdruck verwendet werden. Nach dem Schlüsselwort **match** folgt in Klammern der zu evaluierende Ausdruck. Danach folgen in geschweiften Klammern mit Bindestrich getrennte Schlüssel-Wert-Paare, die in der Form **schlüssel => wert** angegeben

werden. Der Schlüssel wird mit dem Ausdruck in der Klammer verglichen, gibt es eine Übereinstimmung, wird der dazugehörige Wert zurückgegeben. Der Vergleich erfolgt dabei intern mit `===` (bei `switch` wird nur `==` verwendet). Die obige `switch`-Anweisung sieht wie folgt als Match Expression aus (`echo` steht hier nur vor `match` um das Ergebnis auszugeben):

```
1 echo match ($n) {  
2   1 => "Die Zahl ist 1",  
3   2 => "Die Zahl ist 2",  
4   default => "Die Zahl ist weder 1 noch 2"  
5 };
```

Schleifen

Auch in PHP finden sich die bekannten Vertreter der *Schleifen* wieder: `while`, `do-while` und `for`.

`while`-Schleifen haben die folgende Syntax:

```
1 while (Ausdruck) {  
2   Anweisung;  
3 }
```

Solange der Ausdruck im Schleifenkopf `true` ergibt, wird die Anweisung in der Schleife ausgeführt. Ist der Ausdruck nie `true`, wird der Rumpf der Schleife auch nie betreten.

Anders bei der `do-while` Schleife. Hier wird der Rumpf auf jeden Fall ein Mal ausgeführt und dann erst überprüft, ob die Anweisung wiederholt werden soll. Die Syntax lautet wie folgt:

```
1 do {  
2   Anweisung;  
3 } while (Ausdruck);
```

Während bei der `while`-Schleife oft „händisch“ eine Zählervariable mit verändert werden muss, um ein Abbruchkriterium zu erhalten, geschieht dies in der `for`-Schleife automatisch. Im Kopf dieser Schleife wird eine Zählervariable initialisiert, auf das Abbruchkriterium überprüft und verändert (z.B. erhöht). Die Syntax lautet hierfür folgendermaßen:

```
1 for (Initialisierung; Test; Veränderung) {  
2   Anweisung;  
3 }
```

Es folgen nun Beispiele für alle drei Schleifentypen. Gerade bei der `for`-Schleife wird auf das `$`-Zeichen bei der Zählervariable im Schleifenkopf vergessen, was zu kryptischen Fehlermeldungen führen kann.

```
1 // while-Schleife  
2 $count = 0;  
3 while ($count < 10) {  
4   // Mach etwas hier...  
5   $count++;  
6 }  
7  
8 // do-while-Schleife  
9 $count = 0;
```

```
10 do {
11     // Mach etwas hier...
12 } while ($count < 10);
13
14 // for-Schleife
15 for ($count = 0; $count < 10; $count++) {
16     // Mach etwas hier...
17 }
```

2.3.8 Advanced Escaping

Dass HTML-Markup und PHP-Code in einer Datei zeilenweise miteinander vermischt werden können, wurde bereits des Öfteren erwähnt. Hier nochmals ein simples Beispiel mit einer Verzweigung:

```
1 <?php
2 if ($wert === "Hallo") {
3     echo "<p>Hallo zurück! :)</p>";
4 } else {
5     echo "<p>Kein Hallo für dich! :(</p>";
6 }
7 ?>
```

Dieses kurze Skript überprüft den Wert einer Variablen und gibt davon abhängig mit `echo` einen Text (als HTML-Absatz ausgezeichnet) zurück. Mit dem sogenannten *Advanced Escaping* kann hier die Verzahnung von HTML- und PHP-Code noch weiter vorangetrieben werden. Denn Inhalte von Blöcken (alles zwischen `{` und `}`) können auch rein in HTML vorliegen. Somit kann statt der `echo`-Aufrufe zunächst der PHP-Block mit `?>` beendet werden. Danach folgt normal der HTML-Code. Wiederum danach wird der PHP-Block vor der schließenden Klammer wieder mit `<?php` begonnen. Dies sieht dann folgendermaßen aus:

```
1 <?php
2 if ($wert === "Hallo") {
3     ?>
4     <p>Hallo zurück! :)</p>
5     <?php
6     } else {
7     ?>
8     <p>Kein Hallo für dich! :(</p>
9     <?php
10    }
11    ?>
```

Diese Syntax macht das Skript in der Regel viel unübersichtlicher, hat aber einen entscheidenden Vorteil bzw. Anwendungsfall: Sollten hier statt einer Zeile Text gleich mehrere ausgegeben werden, können durch Advanced Escaping mehrere `echo`-Aufrufe (und damit verbundenes Verpacken von Text in Anführungszeichen) vermieden werden.

2.3.9 Strings und Stringfunktionen

Strings, auch Zeichenketten genannt, sind ein sehr wichtiger, wenn nicht überhaupt der wichtigste Datentyp in der Webentwicklung. Daten, die Benutzer*innen in Formulare eingeben, kommen als Strings an. Alle Daten, die vom PHP-Programm mittels `echo`

oder `print()` ausgegeben werden, sind ebenfalls Strings. Da PHP mit einigen Besonderheiten in puncto Strings aufwartet, werden diese hier ausführlicher als die restlichen Datentypen (siehe Abschnitt 2.3.4) behandelt.

Strings angeben

Strings können in PHP auf vier unterschiedliche Arten und Weisen angegeben werden:

- In einfachen Anführungszeichen (engl. „single-quoted string“),
- in doppelten Anführungszeichen (engl. „double-quoted string“),
- in der Heredoc-Syntax und
- in der Nowdoc-Syntax.

Strings in einfachen Anführungszeichen: Die einfachste Art, einen String anzugeben, ist, ihn in einfache Anführungszeichen (`' '`) zu setzen. Alle Zeichen innerhalb des Strings werden 1:1 so wiedergegeben, wie sie spezifiziert wurden, es gibt nur zwei Ausnahmen: Wird das einfache Hochkomma innerhalb des Strings benötigt, muss es mit einem Backslash „escaped“ werden (`\'`), wird der Backslash selbst benötigt, muss er mit zwei Backslashes angegeben werden (`\\`). Spezialzeichen wie Zeilenumbrüche (`\n`), Tabulatoren (`\t`) oder auch Variablen (z.B. `$text`) werden nicht ersetzt, sondern genauso ausgegeben. Die folgenden Beispiele illustrieren dies:

```
1 echo 'Hallo'; // Hallo
2 echo 'Hallo\nWelt'; // Hallo\nWelt
3 echo 'Wichtig: $text'; // Wichtig: $text
4 echo 'PHP ist \'toll\''; // PHP ist 'toll'
```

Strings in doppelten Anführungszeichen: Werden Strings in doppelten Anführungszeichen angelegt, kommt es zur Interpretation von Variablen (sog. Variableninterpolation) und ebenfalls zur Ersetzung von diversen Spezial- bzw. Escapezeichen. Werden also Variablennamen (z.B. `$text`) oder Spezialzeichen wie Zeilenumbrüche (`\n`) oder Tabulatoren (`\t`) verwendet, so werden diese durch ihre dahinter liegenden Bedeutungen ersetzt. Doppelte Anführungszeichen müssen durch einen Backslash escaped werden (`\"`), um nicht vorzeitig den String zu beenden. Die folgenden Beispiele sollen dies darstellen:

```
1 echo "Hallo"; // Hallo
2 echo "Hallo\nWelt"; // Hallo
3                      // Welt
4 $text = "Variable wurde interpoliert!";
5 echo "Wichtig: $text"; // Wichtig: Variable wurde interpoliert!
6 echo "PHP ist \"toll\""; // PHP ist "toll"
```

Um dem PHP-Parser exakt mitzuteilen, wo der Bezeichner der Variable endet, kann diese in geschwungene Klammern gesetzt werden (curly syntax). Das ist etwa dann nötig, wenn nach dem Variablennamen danach kein Leerzeichen kommen soll:

```
1 $count = 3;
2 echo "Dauer: {$count}min"; // Dauer: 3min
```

Ohne die Klammern würde PHP versuchen auf eine Variable `$countmin` zuzugreifen, die es nicht gibt. Diese Syntax ist auch beim Zugriff auf zweidimensionale Arrays oder Objekteigenschaften (siehe Vorlesungen 3 und 4) nötig.

Früher galt die Faustregel, dass einfache Anführungszeichen schneller seien, da der Parser nicht nach Variablen suchen muss. Bei modernen PHP-Versionen (OPcache) ist dieser Unterschied jedoch vernachlässigbar. Die Wahl sollte daher primär nach der Lesbarkeit (Interpolation nötig oder nicht?) oder einfach nach persönlicher Präferenz getroffen werden.

Heredoc-Syntax: Die Heredoc-Syntax (kurz für Here Documents) wurde mit PHP4 eingeführt, um zu ermöglichen, längere Texte mit Zeilenumbrüchen in Strings zu speichern, ohne diese mittels `\n` erzeugen zu müssen.

Um einen Heredoc-String zu beginnen, werden drei öffnende spitze Klammern gefolgt von einem möglichst eindeutigen Identifikator angegeben. Danach muss eine neue Zeile für den Text begonnen werden. Abgeschlossen wird der Heredoc-Block vom Identifikator, gefolgt von einem Strichpunkt. Der abschließende Identifikator kann eingerückt werden. In diesem Fall werden genau diese Anzahl Leerzeichen auch aus jeder Zeile des Heredoc entfernt (bessere Lesbarkeit im Code).

Der Heredoc-Block verhält sich identisch mit einem String in doppelten Anführungszeichen. Variablen und Spezialzeichen werden also ersetzt. Einzig doppelte und einfache Anführungszeichen können im Heredoc normal (d. h. ohne Backslash) verwendet werden.

Hier ein Beispiel für die Heredoc-Syntax. Als Identifikator wird „EOT“ verwendet. Es könnte aber genauso „ENDE“ oder jede andere Zeichenkette gewählt werden. Diese sollte nur nicht genauso im Text vorkommen.

```
1 $name = "John Doe";
2 $hdText = <<<EOT
3   <p class="bold">Dies ist
4   ein Text von $name, der
5   genauso durch die
6   Zeilenumbrüche im String
7   gespeichert wird.</p>
8   EOT;
```

Im Text wird `$name` durch „John Doe“ ersetzt und genauso mit den Zeilenumbrüchen im String gespeichert.

Nowdoc-Syntax: Die Nowdoc-Syntax existiert als Gegenstück zu Heredoc. Sie verhält sich zur Heredoc-Syntax so, wie sich ein String unter einfachen Hochkommas zu einem unter doppelten Hochkommas verhält, d. h. Variablen und Spezialzeichen werden nicht ersetzt.

Der Nowdoc-Bereich wird ebenfalls mit drei spitzen Klammern und einem Identifikator begonnen, allerdings steht der Identifikator unter einfachen Anführungszeichen. Beendet wird der Block wieder über die Angabe des Identifikators.

Das folgende Beispiel ist identisch zum vorigen, jedoch als Nowdoc-Block:

```
1 $name = "John Doe";
2 $hdText = <<<'EOT'
3   <p class="bold">Dies ist
4   ein Text von $name,
5   der genauso durch die
6   Zeilenumbrüche im String
7   gespeichert wird.</p>
8   EOT;
```

Die Variable `$name` wird im Text nicht ersetzt, im String steht daher „von `$name`“ und nicht „von John Doe“.

Stringfunktionen

Für die Arbeit mit Zeichenketten bietet PHP eine umfangreiche Sammlung an *Stringfunktionen*, von denen in der Folge einige wichtige exemplarisch vorgestellt werden. Eine komplette Übersicht findet sich in der PHP-Dokumentation³¹. Diese Stringfunktionen stammen jedoch aus frühesten PHP-Tagen und gehen daher von der (naiven) Annahme aus, dass jedes Zeichen ein 8-bit (d. h. 1 Byte) Zeichen ist. Dies ist für alle Zeichen des ASCII-Zeichensatzes der Fall, sobald jedoch Sonderzeichen ins Spiel kommen, wie etwa deutsche Umlaute oder auch so gut wie alle Unicode-Zeichen, führt dies zu Problemen, da diese Zeichen mit mehr als einem Byte repräsentiert werden. Daher wird bei den Stringfunktionen – wo immer möglich – auf die Funktionen der **mbstring**-Erweiterung zurückgegriffen. Diese beinhalten die meisten der nativen PHP-Stringfunktionen, nehmen jedoch auf Zeichen, die mehrere Bytes (Multibytes) benötigen, Rücksicht. Das erste Beispiel zur Länge einer Zeichenkette illustriert die Wichtigkeit dieser Multibyte-Funktionen. Eine volle Auflistung findet sich ebenfalls im PHP-Handbuch³².

Die hier aufgezählten Funktionen werden inklusive ihrer Parameter angegeben. Manche dieser Parameter stehen in eckigen Klammern, dies bedeutet, dass sie optional sind, also nicht angegeben werden müssen.

Länge einer Zeichenkette: Die Funktion `strlen($string)`³³ ermittelt die Länge eines Strings. Zurückgegeben wird eine positive Ganzzahl oder 0, wenn der String leer ist. Das folgende Beispiel stellt dies dar:

```
1 $str = "Hallo";
2 echo strlen($str); // 5
```

In diesem Beispiel kommen nur ASCII-Zeichen mit einem Byte Länge vor, weshalb die Funktion das erwartete Ergebnis liefert. Sobald jedoch Multibyte-Zeichen (wie etwa ein Umlaut) vorkommen, wird die Funktion `mb_strlen($string [, $encoding])`³⁴ benötigt. Das Beispiel demonstriert den Unterschied:

```
1 $str = "Hällo";
2 echo strlen($str); // 6
3 echo mb_strlen($str); // 5
```

Während `strlen()` die Bytes zählt um zum (falschen) Ergebnis 6 kommt, berücksichtigt `mb_strlen()`, dass das Sonderzeichen ein Multibyte-Zeichen ist und gibt die Stringlänge korrekt mit 5 an.

Zugriff auf Einzelzeichen: Der Zugriff auf einzelne Zeichen eines Strings geschieht wie der Zugriff auf Elemente eines Arrays mit Hilfe der eckigen Klammern (`[]`). Das erste Zeichen hat Position 0, das letzte befindet sich an der Position *AnzahlZeichen* – 1. Folgendes Beispiel illustriert dies:

³¹<https://www.php.net/manual/de/ref.strings.php>

³²<https://www.php.net/manual/de/book.mbstring.php>

³³<https://www.php.net/manual/de/function strlen.php>

³⁴<https://www.php.net/manual/de/function.mb-strlen.php>

```
1 $str = "Hallo Welt";
2 echo $str[0]; // H
3 echo $str[3]; // l
```

Extrahieren von Substrings: Soll auf Ausschnitte einer Zeichenkette (sogenannte Substrings) zugegriffen werden, bietet sich dafür die Verwendung der Multibyte-Funktion `mb_substr($string, $start [, $length [, $encoding]])`³⁵ an. `$string` ist dabei die Zeichenkette, aus der ein Teil entnommen werden soll. `$start` gibt den Start-Offset an, also ab welchem Zeichen die Extraktion stattfinden soll. Optional kann mit `$length` angegeben werden, wie viele Zeichen entnommen werden sollen. Entfällt der Wert, wird bis zum Ende des Strings gearbeitet. Beide Werte können auch negativ sein. Bei `$start` wird vom dann Ende des Strings weg zu zählen begonnen, ein negatives `$length` gibt an, wie viele Zeichen vom Ende des Strings abgeschnitten werden sollen. Folgende Beispiele verdeutlichen die Funktion:

```
1 $str = "Hallo Welt";
2 echo mb_substr($str, 2, 3); // llo
3 echo mb_substr($str, -2); // lt
4 echo mb_substr($str, -3, 2); // el
5 echo mb_substr($str, 2, -3); // llo W
```

Entfernen unnötiger Leerzeichen: Oft befinden sich am Anfang oder Ende eines Strings Leerzeichen, die durch Zufall (schlampiges Copy & Paste etc.) hinzugekommen sind. Diese Leerzeichen können mitunter problematisch werden, wenn dadurch ein Benutzer*innenname etwa als „JohnDoe_“ abgespeichert wird.

Die Funktion `mb_trim($string [, $characters, $encoding])`³⁶ schafft hier Abhilfe. Der Parameter `$string` ist die zu überprüfende Zeichenkette. Ohne das zweite Argument werden Leerzeichen, Tabulatoren, Zeilenumbrüche und noch eine Reihe von Spezialzeichen entfernt. Wird zusätzlich `$characters` als zweiter Parameter angegeben, werden die in diesem String enthalten Zeichen ebenfalls entfernt. Zwei Beispiele:

```
1 $str = mb_trim("  Hallo Welt  "); // Hallo Welt
2 $more = mb_trim("!-Hallo Welta", "a-!"); // Hallo Welt
```

Vergleichen von Strings: Die Funktion `strcmp($str1, $str2)`³⁷ überprüft zwei Strings auf Gleichheit. Sind die Werte gleich, wird 0 zurück gegeben, ist `$str1` (lexikografisch) kleiner als `$str2` wird eine negative Zahl zurückgegeben. Im umgekehrten Fall ist eine positive Zahl das Ergebnis des Vergleichs. Die Größe der Zahl richtet sich dabei nach dem Abstand der Zeichen in der ASCII-Tabelle. Wird „A“ mit „D“ verglichen ist das Ergebnis `-3`, denn A ist 3 Stellen vor D in der ASCII-Tabelle positioniert. Groß- und Kleinschreibung wird beachtet. Soll diese ignoriert werden, muss die Funktion `strcasecmp($str1, $str2)`³⁸ verwendet werden. Achtung: Es gibt es kein

³⁵ <https://www.php.net/manual/de/function.mb-substr.php>

³⁶ <https://www.php.net/manual/en/function.mb-trim.php>

³⁷ <https://www.php.net/manual/de/function strcmp.php>

³⁸ <http://php.net/manual/de/function.strcasecmp.php>

`mb_strcmp()`, da der Vergleich etwa von Unicode-Zeichen oder auch Zeichen unterschiedlicher Kodierung ungleich komplexer ist und in der Regel Spezial-Implementierungen benötigt (z. B. mit einem sogenannten Collator-Objekt).

Strings können auch mit den Vergleichsoperatoren `==`, `===`, `<`, `>`, `<=`, `>=`, `>==`, `>==` und `<=>` durchgeführt werden. `strcmp()` ist hier vorzuziehen, da die Funktion beim Vergleichen von Strings und Zahlen auf String konvertiert, während die Operatoren vor PHP 8 noch auf Integer konvertieren. Dazu einige Beispiele:

```

1 $value1 = 3;
2 $value2 = "3";
3 $name1 = "Fred";
4 $name2 = "Wilma";
5
6 /* Vergleich zweier Integer mit strcmp */
7 echo strcmp($value1, $value2); // 0
8 /* Vergleich auf Wert (==) */
9 if ($value1 == $value2) {
10     echo "Wert gleich"; // wird ausgegeben
11 }
12 /* Vergleich auf Wert und Typ (===) */
13 if ($value1 === $value2) {
14     echo "Wert und Typ gleich"; // wird nicht ausgegeben
15 }
16 /* Lexikografischer Vergleich zweier Strings */
17 if ($name1 < $name2) {
18     echo "Fred kommt vor Wilma"; // wird ausgegeben
19 }
20 /* Lexikografischer Vergleich von String mit Integer */
21 if ($name1 < $value1) {
22     echo "Fred < 3"; // // Vor PHP 8 ausgegeben (aber falsch!), mit PHP 8 nicht ausgegeben
23 }
24 /* Vergleich mit strcmp von String mit Integer */
25 echo strcmp($name1, $value1); // 19 (F ist in der ASCII-Tabelle 19 Stellen hinter 3)

```

Das erste Beispiel (Zeile 6) vergleicht eine Zahl und einen String mittels `strcmp()`. Der Integer wird automatisch zu einem String konvertiert und der Vergleich ergibt 0 – somit sind die Strings gleich.

In Beispiel zwei (Zeile 8) werden die Werte mittels `==` auf Gleichheit überprüft. Da die Werte gleich sind, wird die `if`-Bedingung betreten und der Text ausgegeben.

Im dritten Beispiel (Zeile 12) wird mit dem `===`-Operator auf Wert- und Typgleichheit überprüft. Da ein Integer und ein String verglichen werden, ergibt der Vergleich `false` und die Bedingung wird nicht betreten.

In Beispiel vier (Zeile 16) werden zwei Strings lexikografisch mit dem `<` Operator verglichen. Da das „F“ von Fred vor dem „W“ von Wilma kommt, ergibt der Vergleich `true` und der Text wird ausgegeben.

Im fünften Beispiel (Zeile 20) kommt es hingegen bei PHP-Versionen kleiner 8.0 zu Problemen. Es werden ein String und ein Integer verglichen, der String ist jedoch nicht numerisch. Er wird automatisch von PHP auf einen Integer mit dem Wert 0 umgewandelt. 0 ist kleiner als 3, somit wird Fred kleiner als 3 angesehen, obwohl die Buchstaben im lexikografischen Vergleich *nach* den Zahlen kommen. Somit passiert hier ein Fehler. Ab PHP 8 wird dieses Verhalten korrigiert. Die Zahl wird hierbei zu einem String konvertiert, diese ist kleiner (kommt vor den Buchstaben), daher wird nichts

ausgegeben.

In Beispiel sechs (Zeile 24) wird der selbe Vergleich mit `strcmp()` durchgeführt. Nun wird die Zahl 3 in einen String konvertiert und korrekt verglichen. Das Ergebnis ist 19, denn F ist 19 Stellen hinter 3 in der ASCII-Tabelle zu finden.

Suchen von Substrings („Needle in a Haystack“): Mit Hilfe der Multibyte-Funktion `mb_strpos($haystack, $needle [, int $offset = 0 [, $encoding]])`³⁹ kann das *erste* Vorkommen eines Substrings in einem String überprüft werden. `$haystack` ist der String in dem gesucht wird, `$needle` ist der zu suchende Substring. `$offset` ist optional und gibt an, ab welcher Position gesucht werden soll. Dazu zwei Beispiele:

```
1 $text = "testen von strings";
2 $pos1 = mb_strpos($text, "st"); // 2
3 $pos2 = mb_strpos($text, "st", 5); // 11
```

Mit `mb_strrpos($haystack, $needle [, int $offset = 0 [, $encoding]])`⁴⁰ wird nach demselben Schema das *letzte* Vorkommen eines Substrings zurückgegeben:

```
1 $text = "testen von strings";
2 $pos = mb_strrpos($text, "st"); // 11
```

Um nicht den Index, sondern stattdessen den verbleibenden Teil von `$haystack` ab dem ersten Vorkommen von `$needle` zurückzugeben, gibt es die Multibyte-Funktion `mb_strstr($haystack, $needle [, $before_needle = false [, $enc]])`⁴¹. Groß- und Kleinschreibung werden dabei beachtet. Anders handhabt dies die Multibyte-Funktion `mb_stristr($haystack, $needle [, $before_needle = false [, $enc]])`⁴². Ist der dritte, optionale Parameter auf `true` gesetzt, wird der Teil *vor* dem Vorkommen zurückgegeben. Zwei Beispiele dazu:

```
1 $str1 = mb_strstr("Testen von Strings", "St"); // Strings
2 $str2 = mb_stristr("Testen von Strings", "St"); // sten von Strings
```

Umwandeln in Groß- und Kleinbuchstaben: Durch Verwendung der Multibyte-Funktion `mb_convert_case($string, $mode [, $encoding])`⁴³ kann ein String in drei Varianten, abhängig vom Wert des Parameters `$mode` umgewandelt werden. Dieser kann eine von drei Konstanten annehmen:

- `MB_CASE_UPPER`: Wandelt alle Buchstaben in Großbuchstaben um.
- `MB_CASE_LOWER`: Wandelt alle Buchstaben in Kleinbuchstaben um.
- `MB_CASE_TITLE`: Wandelt jeden ersten Buchstaben eines Worts in Großbuchstaben um.

Dazu jeweils ein Beispiel:

```
1 $text = "Hallo kleine Welt";
2 echo mb_convert_case($text, MB_CASE_UPPER); // HALLO KLEINE WELT
3 echo mb_convert_case($text, MB_CASE_LOWER); // hallo kleine welt
4 echo mb_convert_case($text, MB_CASE_TITLE); // Hallo Kleine Welt
```

³⁹<https://www.php.net/manual/de/function.mb-strpos.php>

⁴⁰<https://www.php.net/manual/de/function.mb-strrpos.php>

⁴¹<https://www.php.net/manual/de/function.mb-strstr.php>

⁴²<https://www.php.net/manual/de/function.mb-stristr.php>

⁴³<https://www.php.net/manual/de/function.mb-convert-case.php>

2.4 Weiterführende Literatur

Das Einführungswerk in PHP ist sicherlich [41], da es in seinen ersten drei Auflagen vom Erfinder der Sprache selbst mitverfasst wurde. Das Buch bietet eine komplette Einführung von Grund auf und geht auch auf fortgeschrittenere Themen ein. Seit März 2020 ist die vierte Auflage verfügbar, die alle Features bis inklusive PHP 7.4 enthält und somit wieder einigermaßen aktuell ist. PHP 8 wird allerdings nicht abgedeckt, daher ist es mittlerweile als veraltet zu betrachten, bis eine neue Auflage erscheint.

Eine etwas knappere PHP-Einführung, dafür aber eine ausführliche Einführung in die Welt der MySQL-Datenbanken bietet [44]. Ein derartiges Buch erscheint bei jedem großen PHP- oder MySQL-Release und bietet so immer die letzten Neuerungen, so auch bereits für PHP 8. Inhaltlich richtet es sich zwar durchaus an Anfänger, setzt aber schnell einige Kenntnisse voraus. Wer selbiges in Englisch sucht, wird bei [30] fündig. Dieses Buch beinhaltet auch noch HTML5-, CSS- und JavaScript-Grundlagen und in der neuesten Auflage auch bereits PHP 8.

In dieselbe Kerbe wie die vorigen beiden Empfehlungen schlägt auch [49] – auch hier wird ein Einstieg in PHP 8 und MySQL angeboten, wenngleich das Niveau etwas höher angesetzt ist und sich auch an fortgeschrittene Entwickler*innen richtet.

Lösungen für konkrete PHP-Problemstellungen bietet [27]. Im Buch werden viele alltägliche Probleme, die es beim Programmieren zu lösen gilt heraus, herausgegriffen und effiziente Lösungen inklusive Erklärung präsentiert. Das entsprechende Wissen in PHP wird jedoch bereits vorausgesetzt.

Wer ein Online-Nachschlagewerk sucht, um PHP von Grund auf „richtig“ zu lernen, dem sei das Projekt *PHP: The Right Way* [25] ans Herz gelegt.

Schließlich sei wie beim Erlernen einer jeden Skript- oder Programmiersprache auf die eigene Dokumentation [42] verwiesen. Diese enthält sowohl eine Referenz aller Funktionen und Konstrukte, aber auch Informationen über grundlegende Konzepte bis hin zur Installation.

Vorlesung 3

PHP-Grundlagen Teil 2 und Regular Expressions

Nach der grundlegenden Einführung in die Sprache PHP sowie der Demonstration von Stringfunktionen, soll nun das Konzept der Funktionen im Detail beleuchtet werden. Weiters werden Konstanten und Arrays erklärt, bevor ein grundsätzlicher Formularverarbeitungsablauf besprochen und mit Regular Expressions geschlossen wird.

3.1 Funktionen

Eine *Funktion* ist ein benannter Code-Block, der eine bestimmte Aufgabe erfüllt und beliebig oft aufgerufen werden kann. Der PHP-Interpreter muss eine Funktion dabei nur ein Mal abarbeiten, egal wie oft sie schließlich verwendet wird. Funktionen in PHP können, wie in anderen Sprachen auch, *Parameter* (Übergabewerte) haben oder auch Werte zurückliefern. Ob eine Funktion Parameter hat oder etwas zurückgibt, wird bei ihrer Definition festgelegt.

3.1.1 Funktionen definieren (benannte Funktionen)

Eine Funktion wird mit dem Schlüsselwort `function` definiert. Danach folgen der Name der Funktion (Groß- und Kleinschreibung spielt keine Rolle) und zwei Klammern. Diese können optional Parameter enthalten. Der Code, der durch Aufruf der Funktion ausgeführt werden soll, steht in geschwungenen Klammern. Die Syntax lautet also:

```
1 function name($param1, $param2, ...)  
2 {  
3     Anweisung;  
4 }
```

Damit eine Funktion einen Wert zurückgibt, muss innerhalb der Funktion das Schlüsselwort `return`, gefolgt vom zurückzugebenden Wert gesetzt werden. Folgendes Codebeispiel zeigt eine Funktion, die den übergebenen Wert quadriert und zurück gibt:

```
1 function quadrat($x)  
2 {  
3     return $x * $x;  
4 }
```

Auch bei Funktionen kann *Advanced Escaping* zum Einsatz kommen (da eine Funktion durch die geschweiften Klammern auch einen Block erzeugt). Folgendes Konstrukt ist also möglich:

```
1 <?php
2 function noresult()
3 {
4 ?>
5 <p>Die Abfrage lieferte kein Ergebnis!</p>
6 <?php
7 }
8 ?>
```

3.1.2 Funktionsparameter

PHP kennt zwei Arten der Parameterübergabe: Call by Value und Call by Reference.

Call by Value

Parameter, die an Funktionen übergeben werden, sind, wenn sie wie zuvor gezeigt, übergeben werden, reine Eingangsparameter. D. h., es wird eine Kopie des Werts per *Call by Value* übergeben. Das Beispiel demonstriert dies:

```
1 function cbv($x)
2 {
3     return $x += 5;
4 }
5
6 $test1 = 5;
7 $result1 = cbv($test1);
8 echo $result1; // 10
9 echo $test1; // 5
```

Die Funktion `cbv()` wird mit einem Parameter aufgerufen, zählt zu diesem den Wert 5 hinzu und gibt das Ergebnis zurück. Zu Testzwecken wird die Variable `$test1` mit dem Wert 5 angelegt und der Funktion übergeben. Das Ergebnis der Funktion wird in `$result1` gespeichert. Beide Werte werden ausgegeben. `$result1` hat den Wert 10, `$test1` bleibt jedoch unverändert bei 5, da nur eine Kopie an die Funktion übergeben wurde.

Call by Reference

Anstatt der Kopie eines Werts, kann auch eine Referenz an die Funktion übergeben werden. Dies hat zur Folge, dass sich im Falle einer Änderung des Parameters innerhalb der Funktion diese Änderungen auch in der ursprünglichen Variable widerspiegeln. Um einen sogenannten Übergabeparameter zu erzeugen, wird dem Parameter in der Funktionsdefinition ein kaufmännisches Und (`&`) vorangestellt. Das folgende Beispiel illustriert dies:

```
1 function cbr(&$x)
2 {
3     return $x += 5;
4 }
```



```
5
6 $test2 = 5;
7 $result2 = cbr($test2);
8 echo $result2; // 10
9 echo $test2; // 10
```

Die Funktion heißt nun `cbr()` und akzeptiert wieder einen Parameter. Diesem ist jedoch das `&`-Zeichen vorangestellt, somit handelt es sich um einen Übergabeparameter. Der Wert von `$test2` wird mit 5 festgelegt und der Funktion übergeben. Das Ergebnis des Aufrufs wird in `$result2` gespeichert und ist – wie zu erwarten – 10. Bei der Ausgabe von `$test2` hat die Variable nun aber auch den Wert 10, denn der Übergabeparameter wurde in der Funktion verändert und diese Änderung spiegelt sich in der Variable „außerhalb“ der Funktion wider.

3.1.3 Funktionen aufrufen

Der Aufruf von Funktionen wurde in den soeben besprochenen Beispielen bzw. auch bereits in Abschnitt 2.3.9 demonstriert. Der Name der Funktion, gefolgt von Klammern und ggf. Argumenten genügt. Liefert die Funktion einen Wert zurück, kann dieser einer Variable zugewiesen werden.

Häufige Fehler sind das Weglassen des `$`-Zeichens für Argumente oder auch das Setzen des `&`-Zeichens (weil Call by Reference) gewünscht ist. Dies muss jedoch bei der Definition der Funktion geschehen, nicht bei deren Aufruf. Im folgenden einige Beispiele:

```
1 quadrat(5); // Ruft quadrat() mit dem Wert 5 auf
2 cbv($value); // Ruft cbv() mit der Variable $value auf
3 $result = cbr(5); // Ruft cbv() mit Wert 5 auf und weist Ergebnis an $result zu
4 irgendwas(); // Ruft irgendwas() ohne Parameter auf
```

Häufige Fehler beim Aufruf sind das Vergessen des `$`-Zeichens bei Variablen als Argumenten oder der Versuch, das `&`-Zeichen (für Call by Reference) beim Aufruf mitzugeben. Letzteres darf ausschließlich in der Definition der Funktion stehen.

First-Class Callable Syntax

Seit PHP 8.1 gibt es die *First-Class Callable Syntax*. Mit der Notation `name(...)` kann eine Funktion referenziert werden, ohne sie sofort auszuführen. Dies ist besonders wichtig, wenn Funktionen als Argumente an andere Funktionen übergeben werden oder – wie in Abschnitt 3.1.6 gezeigt – im *Pipe-Operator* zum Einsatz kommen.

```
1 // Referenz auf die Funktion speichern (nicht ausführen!)
2 $funcRef = quadrat(...);
3
4 // Späterer Aufruf über die Referenz
5 echo $funcRef(10); // 100
```

3.1.4 Default-Parameter

PHP erlaubt es, Parameter von Funktionen mit *vorgegebenen Werten* zu belegen. Dies ist dann interessant, wenn Parameter optional sein sollen. Soll z. B. eine Sortierfunktion geschrieben werden, die Wörter von A–Z sortiert, wird als Parameter ein Array mit diesen Wörtern übergeben. Ein optionaler zweiter, boolescher Parameter soll ermöglichen,

dass die Sortierreihenfolge auf Z–A geändert wird. Wird der Parameter mit **false** explizit angegeben, wird die Reihenfolge umgedreht. Wird er weggelassen, wird er einfach mit **true** angenommen.

Dies ist möglich, indem Parametern in der Funktionsdefinition einfach ein Wert zugewiesen wird. Dabei ist zu beachten, dass ab dem ersten vorbelegten Parameter auch alle weiteren vorbelegt werden müssen. Das folgende, einfache Beispiel demonstriert dies:

```
1 function defaultParams($a, $b = "b", $c = "c")
2 {
3     echo "<p>$a$b$c</p>";
4 }
5
6 defaultParams("a"); // abc
7 defaultParams("a", "x"); // axc
8 defaultParams("a", "x", "o"); // axo
```

Der erste Parameter (**\$a**) ist nicht vorbelegt, somit muss er beim Funktionsaufruf immer angegeben werden. Der zweite (**\$b**) und dritte Parameter (**\$c**) sind jeweils vorbelegt und können weggelassen werden. Ist dies der Fall, werden die vorbelegten Werte angenommen. Wird für die Parameter ein Wert mit übergeben, so wird dieser verwendet. Die Werte werden von links nach rechts zugeordnet. Wird also die Funktion wie in Zeile 7 mit zwei Parametern aufgerufen, so wird der zweite Parameter als **\$b** angenommen.

3.1.5 Anonyme Funktionen und Arrow Functions

Normalerweise haben Funktionen einen Namen, um sie später wiederholt aufrufen zu können. Es gibt jedoch Anwendungsfälle, in denen eine Funktion nur an einer einzigen Stelle als kurzes Hilfswerkzeug benötigt wird. In diesem Fall spricht man von *anonymen Funktionen* (auch Closures genannt).

Seit PHP 7.4 existieren zudem *Arrow Functions*. Sie sind eine verkürzte Schreibweise für anonyme Funktionen, die aus nur einem Ausdruck bestehen. Sie beginnen mit dem Schlüsselwort **fn** und geben das Ergebnis nach dem Pfeil (**=>**) automatisch zurück.

Über diese Schreibweise ist es möglich, einer Variable eine Funktion zuzweisen: die Variable „enthält“ also eine Funktion. Über die Angabe des Variablennamens wird somit die Funktion aufgerufen.

```
1 // 1. Klassische anonyme Funktion
2 $callback = function ($name){
3     return "Hallo $name";
4 };
5
6 echo "<p>{$callback("Igor")}</p>"; // Hallo Igor
7
8 // 2. Moderne Arrow Function (kürzer, gleiches Ergebnis)
9 $arrow = fn($name) => "Hallo $name";
10
11 echo "<p>{$arrow("Igor")}</p>"; // Hallo Igor
```

Hier zeigt sich auch ein weiterer Vorteil dieser Syntax: Der Funktionsaufruf kann, da es ja „nur“ der Variablenname ist, in einem String mit doppelten Anführungszeichen in der Complex Curly Syntax stehen. Beim Aufruf einer benannten Funktion muss der String unterbrochen und mit **.** verbunden werden.

3.1.6 Der Pipe-Operator

Mit PHP 8.5 wurde der *Pipe-Operator* (`|>`) eingeführt. Er dient dazu, das Ergebnis einer Operation linear an die nächste Funktion weiterzureichen. Dies verhindert tief verschachtelte Funktionsaufrufe, die „von innen nach außen“ gelesen werden müssten, und macht den Datenfluss von oben nach unten (bzw. links nach rechts) sichtbar.

Der Operator erwartet auf der rechten Seite ein *Callable* (etwas Aufrufbares). Benötigt die Zielfunktion mehr als nur das eine übergebene Argument, muss eine Arrow Function genutzt werden. In dieser wird dann spezifiziert, wo der übergebene Wert verwendet wird.

Im folgenden Beispiel werden bei einem Input-String zunächst mit `mb_trim()` die Leerzeichen am Anfang und Ende entfernt, das Ergebnis in Großbuchstaben umgewandelt und schließlich nur die ersten fünf Buchstaben entnommen:

```

1 $input = "  hällö welt  ";
2
3 // Verschachtelter Funktionsaufruf (ggf. schwer lesbar)
4 $res1 = mb_substr(mb_strtoupper(mb_trim($input)), 0, 5);
5 echo "<p>$res1</p>";
6
7 // Pipe-Operator (linearer Datenfluss)
8 $res2 = $input
9         |> mb_trim(...)
10        |> mb_strtoupper(...)
11        |> (fn($x) => mb_substr($x, 0, 5));
12
13 echo "<p>$res2</p>"; // HÄLLO

```

3.1.7 Typ- und Rückgabewertsdeklarationen

Das dynamische Typkonzept von PHP bietet sehr viel Flexibilität im Zusammenhang mit Funktionen. So können Funktionsparameter jeglichen Datentyp aufnehmen, ebenso kann mit der `return`-Anweisung jeder beliebige Typ zurückgegeben werden. Es gibt jedoch auch die Möglichkeit, diese Freiheiten zugunsten von mehr Kontrolle einzuschränken.

Typdeklarationen

Durch eine *Typdeklaration* wird bei der Erstellung einer Funktion festgelegt, dass diese nur Parameter des angegebenen Typs zur Laufzeit akzeptiert. Um dies zu erreichen, wird der Typ einfach vor den Parameter gestellt. Gültige Angaben sind alle von PHP unterstützten Typen, etwa die skalaren Typen `int`, `float`, `bool` und `string`, `object` sowie Klassennamen (siehe Vorlesung 4), `array` oder auch die Schlüsselwörter `callable`¹ (die in Abschnitt 3.1.5 erwähnten anonymen Arrow-Functions, die auch beim Pipe-Operator in Abschnitt 3.1.6 benötigt werden) und `iterable`² (für Arrays und Klassen, die das Interface `Traversable` implementieren).

Im folgenden Beispiel wird eine Funktion definiert, die einen Parameter vom Typ `int` erwartet:

¹<https://www.php.net/manual/de/language.types.callable.php>

²<https://www.php.net/manual/de/language.types.iterable.php>

```
1 function isInt(int $number)
2 {
3     echo "<p>Eine Zahl: $number</p>";
4 }
```

Diese Funktion soll nun mit drei unterschiedlichen Parametern aufgerufen werden:

```
1 isInt(3);
2 isInt("3");
3 isInt("hallo");
```

Der Aufruf in Zeile 1 funktioniert immer, da eine Ganzzahl übergeben wird. Der Aufruf in Zeile 2 funktioniert ebenso, da durch das lose Typkonzept (erfolgreich) versucht wird, den String in einen Integer zu konvertieren. Der Aufruf in Zeile 3 hingegen schlägt immer fehl und produziert einen `TypeError`-Fehler.

Um ein noch strengeres Vorgehen zu erzwingen und auch den Aufruf von Zeile 2 fehlschlagen zu lassen, kann als erste Anweisung in der PHP-Datei (direkt nach `<?php`) die Deklaration `declare(strict_types=1);` gesetzt werden. Sie verhindert die automatische Umwandlung.

Nullable Types: Seit PHP 7.1 besteht die Möglichkeit, die Typdeklaration „nullable“ zu machen, d. h. dass neben dem angegebenen Datentyp auch `null` übergeben werden kann. Dazu wird vor den Typ ein Fragezeichen (?) gestellt:

```
1 function isIntOrNull(?int $number)
2 {
3     echo "<p>Eine Zahl: $number</p>";
4 }
```

Bis PHP 8.4 war es jedoch (unverständlichlicherweise) möglich, für jeden Parameter `null` als Default-Wert anzugeben, auch wenn kein Fragezeichen vorgestellt war. Mit dieser Version wurde dieses Schlupfloch jedoch geschlossen (deprecated).

Rückgabewertsdeklarationen

Nach demselben Prinzip funktionieren *Rückgabewertsdeklarationen*, die mit PHP 7 eingeführt wurden. Sie erlauben anzugeben, welchen Typ eine Funktion zurückgibt. Der Typ wird dabei durch einen Doppelpunkt getrennt am Ende des Funktionskopfes angegeben. Die folgende Funktion zum Addieren von zwei Werten gibt ihr Ergebnis als Ganzzahl zurück:

```
1 function getSum($a, $b): int
2 {
3     return $a + $b;
4 }
```

Auch hier beeinflusst `declare(strict_types=1);` die Rückgabe. Folgende drei Aufrufe illustrieren dies:

```
1 getSum(3, 4);
2 getSum(3.0, 4.0);
3 getSum(3.0, 2.5);
```

Das Ergebnis des Aufrufs in Zeile 1 ist immer 7, da zwei Ganzzahlen addiert werden und funktioniert immer. Die Zeilen 2 und 3 funktionieren nur, wenn die Angabe von

`declare(strict_types=1);` nicht gesetzt ist, da hier Gleitkommazahlen addiert werden und daher auch das Ergebnis eine Gleitkommazahl ist (allerdings in eine Ganzzahl konvertiert wird). Vorsicht ist daher in Zeile 3 geboten: Hier liefert die Funktion – auch wenn sie funktioniert – ein falsches Ergebnis. Durch die automatische Umwandlung von `float` auf `int` werden die Kommastellen abgeschnitten, weshalb als Ergebnis 5 (anstatt 5,5) zurückgegeben wird. PHP merkt dies mit einer Deprecation Notice an, um Entwickler*innen zu warnen, dass dies unvorhersehbare Konsequenzen haben kann.

Nullable Return-Types: Auch Rückgabedeklarationen können seit PHP 7.1 „nullable“ sein, d.h. es kann der angegebene Typ oder `null` zurückgegeben werden. Wiederum wird das Fragezeichen (?) als Ausdruck vor den Rückgabewert gestellt:

```
1 function getSumOrNull($a, $b): ?int
2 {
3     return ($a + $b) > 0 ? $a + $b : null;
4 }
```

Void Funktionen: Ebenfalls kann für Funktionen explizit der Rückgabotyp `void` angegeben werden. Bei diesen muss das `return`-Statement entweder komplett entfallen oder darf keinen Wert zurückgeben (d.h. muss alleine stehen):

```
1 function getNothing(): void
2 {
3     echo "<p>Hier wird nichts zurückgegeben.</p>";
4     return; // Optional, hier besser gleich weglassen
5 }
```

Never Return-Type: Seit PHP 8.1 kann der Rückgabotyp `never` angegeben werden. Dieser ist dann sinnvoll, wenn eine Funktion nie zurückkehrt. Dies ist dann der Fall, wenn in ihr die Ausführung der PHP-Datei etwas mit `die()` oder `exit()` beendet wird, oder eine Ausnahme (Exception) wirft. Ersteres ist etwa der Fall, wenn eine Weiterleitung auf eine andere Seite durchgeführt wird (mehr dazu in Vorlesung 8):

```
1 function redirect(string $uri): never
2 {
3     header("Location: $uri");
4     exit();
5 }
```

Rückgabewerte true, false und null: Seit PHP 8.2 können `true`, `false` und `null` ebenfalls als Rückgabewerte von Funktionen angegeben werden. Zuvor war dies nur gemeinsam in Union Types (s. unten) oder als Nullable Type möglich. Nun ist eine derartige Deklaration ebenfalls möglich (wenngleich wohl nicht sehr sinnvoll):

```
1 function alwaysFalse(): false
2 {
3     return false;
4 }
```

Union Types

Mit PHP 8.0 wurde es möglich, für Parameter und Rückgabewerte einen sogenannten *Union Type*, also eine Auswahl an zwei oder mehreren Typen, anzugeben. Dies geschieht mittels `|`-Zeichen. Es können dabei beliebig viele Typen aneinandergereiht werden:

```
1 function getSum(int|float $a, int|float $b): int|float
2 {
3     return $a + $b;
4 }
```

Beide Parameter dürfen von Typ `int` oder `float` sein. Sind beide Ganzzahlen, wird auch eine Ganzzahl zurückgegeben, sobald mindestens eine Gleitkommazahl dabei ist, wird auch eine solche zurückgegeben. Andere Datentypen sind nicht möglich.

3.1.8 Gültigkeitsbereich von Variablen

Auch PHP kennt das Konzept von globalen und lokalen Variablen. Mit diesem müssen sich Entwickler*innen meist im Zusammenhang mit Funktionen auseinandersetzen. *Globale Variablen* gelten in der gesamten PHP-Datei, *lokale Variablen* gelten innerhalb des Blocks, in dem sie definiert wurden. PHP verfolgt im Detail jedoch ein etwas anderes Konzept als andere Sprachen (z. B. JavaScript):

Variablen, die *in* einer Funktion definiert wurden, sind *außerhalb* dieser Funktion nicht zugreifbar. Variablen, die *außerhalb* einer Funktion definiert wurden, sind standardmäßig *in* der Funktion ebenfalls nicht zugreifbar.

Für lokale Variablen (die in einer Funktion definiert wurden), ist das Konzept das übliche. Sie gelten nur in der Funktion und nirgendwo sonst. Für globale Variablen jedoch gilt umgekehrt genau das Gleiche. Sie werden außerhalb aller Funktionen definiert, gelten aber dann normalerweise innerhalb der Funktionen *nicht*. Hier unterscheidet sich PHP stark von anderen Sprachen wie Java oder JavaScript, wo globale Variablen auch in Funktionen zugreifbar sind.

Um in PHP nun eine globale Variable auch innerhalb einer Funktion verwenden zu können, muss sie in der Funktion zunächst mit dem Schlüsselwort `global` explizit als globale Variable deklariert werden. Ab diesem Zeitpunkt ist dann die Verwendung wieder dieselbe, wie in anderen Sprachen.

Das folgende Beispiel zeigt die Verwendung einer lokalen Variable:

```
1 function fooLokal(): void
2 {
3     $x = "Lokal";
4 }
5
6 $x = "Global";
7 fooLokal();
8 echo $x; // Global
```

Die Funktion `fooLokal()` legt eine lokale Variable `$x` an und weist ihr den Wert „Lokal“ zu. Vor dem Aufruf wird außerhalb der Funktion eine Variable `$x` angelegt, die den Wert „Global“ erhält. Dann wird `fooLokal()` aufgerufen und `$x` ausgegeben. Die Variable hat nach wie vor den Wert „Global“.

Im Gegensatz dazu nun ein Beispiel, bei dem in der Funktion die globale Variable verändert wird:

```
1 function fooGlobal(): void
2 {
3     global $x;
4     $x = "Jetzt global";
5 }
6
7 $x = "Global";
8 fooGlobal();
9 echo $x; // Jetzt global
```

Die Funktion heißt nun `fooGlobal()`. Zunächst wird innerhalb die Variable `$x` mit dem Schlüsselwort `global` als globale Variable deklariert. Somit ist festgelegt, dass diese Variable auch außerhalb Änderungen hervorrufen kann. Die Variable erhält dann den Wert „Jetzt global“. Zu Testzwecken wird wieder eine globale Variable `$x` mit dem Wert „Global“ angelegt. Danach wird `fooGlobal()` aufgerufen und der Wert von `$x` ausgegeben. Dieser ist nun „Jetzt global“, da die Funktion die globale Variable `$x` verändert hat.

Da in PHP Variablen immer implizit deklariert werden, kann das `global` Schlüsselwort auch mit einer Variable verwendet werden, die außerhalb der Funktion noch gar nicht existiert. Sie wird dann in der Funktion einfach direkt als globale Variable angelegt.

Statische Variablen

PHP kennt (wie C oder Java) auch das Konzept der *statischen Variablen* in Funktionen. Diese werden nur einmal initialisiert und ihr Wert bleibt zwischen allen Funktionsaufrufen erhalten. Um eine Variable als statisch zu definieren, wird ihr das Schlüsselwort `static` vorangestellt. Das folgende Beispiel illustriert dies:

```
1 function counter(): int
2 {
3     static $count = 0;
4     return $count++;
5 }
6
7 for ($i = 0; $i < 5; $i++) {
8     echo counter();
9 }
```

Die Variable `$count` wird hier statisch angelegt und beim ersten Aufruf der Funktion `counter()` mit dem Wert 0 belegt. Dieser Wert wird zurückgegeben und `$count` wird erhöht. Eine normale (lokale) Variable würde hier zerstört werden, die statische „lebt“ aber weiter. Beim nächsten Aufruf der Funktion hat `$count` den Wert 1, welcher wiederum zurückgegeben und dann erhöht wird. Somit gibt die Schleife 01234 aus.

3.2 Konstanten

Eine *Konstante* ist ein Bezeichner für einen einfachen Wert (Boolean, Integer, Double oder String), der einmal definiert wird und danach nicht mehr geändert werden kann.

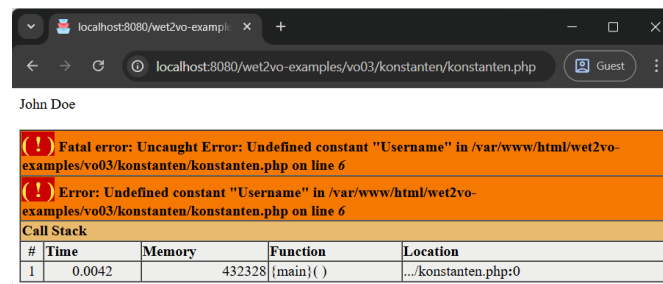


Abbildung 3.1: In der ersten Zeile wird die Konstante `USERNAME` ausgegeben. Danach folgt der Fehler, dass keine Konstante mit dem Namen (bzw. der Schreibweise) `Username` existiert und bricht ab.

Konstanten können selbst definiert werden, PHP stellt jedoch auch eine Reihe von konstanten Werten vorab zur Verfügung. Konstanten sind überall gültig, d. h. sie können auch in Funktionen verwendet werden, ohne vorher speziell deklariert werden zu müssen.

3.2.1 Benannte Konstanten

Konstanten können in PHP auf zwei Arten angelegt werden. Mit der Funktion `define()` oder dem Schlüsselwort `const`.

Konstanten mit `define()`

Die Funktion `define($name, $value)`³ hat zwei Parameter: den Namen oder Identifikator (`$name`) und den Wert (`$value`), der durch die Konstante repräsentiert werden soll. Der Name einer Konstante kann wie der einer Variablen gewählt werden, das `$`-Zeichen entfällt jedoch. Zudem werden Konstantennamen per Konvention in Großbuchstaben gesetzt. Ein Beispiel:

```
1 define("USERNAME", "John Doe");
2 echo USERNAME; // John Doe
```

Zunächst wird die Konstante mit dem Namen `USERNAME` definiert. Sie bekommt den Wert „John Doe“ zugewiesen. Mit `echo` wird dann ihr Wert ausgegeben.

Die Groß- und Kleinschreibung bei Konstanten ist unbedingt einzuhalten. Während vor PHP 8.0 noch eine Warnung bei falscher Schreibweise (z. B. bei `echo Username` anstatt `echo USERNAME`) angezeigt wurde, führt diese nun zu einem Fehler und beendet das Script (siehe Abbildung 3.1). Falsch geschriebene (d. h. in dieser Form so nicht existierende) Variablen erzeugen übrigens nach wie vor nur eine Warnung.

Neben der Groß- und Kleinschreibung sind die Anführungszeichen ein häufiges Problem bei Konstanten. Beim Definieren sind sie anzugeben, bei der Verwendung (z. B. mit `echo`) nicht.

³<https://www.php.net/manual/de/function.define.php>

Konstanten mit `const`

Konstanten können ebenfalls mit dem Schlüsselwort `const` definiert werden. Auch hierbei entfällt das `$`-Zeichen und eine Schreibweise in Großbuchstaben empfiehlt sich.

```
1 const USER = "Jane Doe";  
2 echo USER; // Jane Doe
```

Das Schlüsselwort `const` war ursprünglich für Klassenkonstanten (siehe Vorlesung 4) vorgesehen, kann aber seit PHP 5.3 auch außerhalb von Klassen verwendet werden. Seine Syntax ist klarer und sollte in der Regel bevorzugt werden. Wichtig ist jedoch, dass Konstanten mit `const` nur auf der obersten Ebene in einem Skript verwendet werden können. Eine Definition z. B. innerhalb einer `if`-Bedingung ist nicht möglich. Hierfür muss auf `define()` zurückgegriffen werden:

```
1 if (!defined("FOO")) {  
2     define("FOO", "bar");  
3     // const FOO = "bar"; -> funktioniert hier nicht  
4 }
```

Konstanten überprüfen

Um zu testen, ob eine benannte Konstante existiert, kann (wie gerade zuvor gesehen) `defined($name)`⁴ verwendet werden. Zu beachten ist, dass diese Funktion nur mit Konstanten funktioniert, nicht mit normalen Variablen. Es ist jedoch unerheblich, ob die Konstante mittels `define()` oder `const` angelegt wurde. Der Name der zu überprüfenden Konstanten muss unter Anführungszeichen gesetzt werden. Das Ergebnis der Überprüfung ist `true` oder `false`.

```
1 if (defined("USERNAME")) {  
2     echo "<p>Welcome " . USERNAME . "!\</p>";  
3 }
```

Wie im Beispiel ersichtlich, werden Konstanteninhalte selbst in Strings mit doppelten Anführungszeichen nicht ersetzt. Sie müssen an den String angehängt werden.

3.2.2 Vordefinierte Konstanten

PHP stellt eine Reihe von Konstanten bereit, die immer zur Verfügung stehen und einige praktische Informationen behalten können. Man kann sie grob in drei Kategorien unterteilen:

- magische Konstanten,
- Core-Konstanten und
- Standard-Konstanten.

Magische Konstanten

Magische Konstanten erhalten ihren Namen von der Tatsache, dass sie unterschiedliche Werte haben, abhängig davon, wo sie eingesetzt bzw. aufgerufen werden. Man erkennt

⁴<https://www.php.net/manual/de/function.defined.php>

sie an der Schreibweise: Sie beginnen und enden (bis auf eine Konstante) mit einem doppelten Unterstrich (`__`). Es gibt neun magische Konstanten:

- `__LINE__`: Die aktuelle Zeilennummer in einer Datei.
- `__FILE__`: Der vollständige Pfad- und Dateiname einer Datei.
- `__DIR__`: Der Name des Verzeichnisses, in dem sich die aktuelle Datei befindet.
- `__FUNCTION__`: Der Name der aktuell aufgerufenen Funktion.
- `__CLASS__`: Der Name der aktuellen Klasse (siehe Vorlesung 4).
- `__TRAIT__`: Der Name des aktuellen Traits (siehe Vorlesung 4).
- `__METHOD__`: Der Name der aktuellen Klassenmethode (siehe Vorlesung 4).
- `__PROPERTY__`: Der Name einer Eigenschaft in einem Property-Hook (siehe Vorlesung 4).
- `__NAMESPACE__`: Der Name des aktuellen Namespace (siehe Vorlesung 4).
- `ClassName::class`: Der voll qualifizierte Name einer Klasse (siehe Vorlesung 4).

Core-Konstanten

Diese Konstanten werden vom PHP-Core (also von der Engine) definiert. Sie enthalten Informationen über Versionen, verwendete Betriebssysteme, maximale Größe eines Integers am aktuellen System etc. Im Folgenden werden einige aufgelistet, eine komplette Liste findet sich unter <https://www.php.net/manual/de/reserved.constants.php>.

- `PHP_VERSION`: Die aktuelle PHP-Version.
- `PHP_OS`: Das Betriebssystem, auf dem PHP gerade läuft.
- `PHP_INT_MAX`: Der größtmögliche Integerwert im System.
- `PHP_INT_SIZE`: Die Bytegröße eines Integers im System.
- ...

Standard-Konstanten

Standard-Konstanten sind eine Reihe weiterer vordefinierter Konstanten, die von den mit PHP mitgelieferten Erweiterungen stammen. Zu ihnen zählen etwa mathematische Konstanten wie Pi oder die Eulersche Zahl. Aufgelistet sind sie auf der Seite der jeweiligen Extension⁵.

- `M_PI`: Die Zahl Pi (π).
- `M_E`: Die Eulersche Zahl (e).
- ...

3.3 Arrays

Arrays sind Kollektionen von verschiedenen Werten, die geordnet abgebildet werden. Diese Abbildung erfolgt über Schlüssel/Wert Paare. Der Schlüssel kann entweder ein Integer – in diesem Fall spricht man von einem indizierten Array – oder ein String sein.

⁵Eine Liste aller Erweiterungen findet sich unter <https://www.php.net/manual/de/extensions.membership.php#extensions.membership.core>.

Arrays mit Strings als Schlüssel werden assoziative Arrays genannt. Die Werte können in beiden Fällen beliebig sein.

Arrays sind auf vielfältige Arten und Weisen verwendbar und daher sehr gebräuchlich. Es gibt viele PHP-Funktionen, die mit Arrays arbeiten (z. B. sortieren), aber auch Funktionen, die Arrays als Parameter akzeptieren.

3.3.1 Erzeugen von Arrays

Arrays werden mit der Kurzsyntax `[item1, item2, ...]` erzeugt. Alternativ ist auch das Sprachkonstrukt `array()` möglich, es gilt jedoch mittlerweile als veraltet. Einige Beispiele dazu:

```
1 $a1 = []; // Leeres Array
2 $a2 = ["a", "b", "c"]; // Indiziertes Array
3 $a3 = [3, "Hallo", 5.1]; // Gemische Datentypen als Werte
```

Werden Arrays auf diese Art und Weise angelegt, entstehen automatisch *indizierte* Arrays mit (Integer) Schlüsseln von 0 bis $n - 1$. Sollen diese explizit vergeben werden, kann zur Syntax für assoziative Arrays gegriffen werden. Dabei wird zwischen Schlüssel und Wert ein Pfeil (`=>`) gesetzt. Die so definierten Schlüssel können auch Lücken aufweisen.

```
1 $a4 = [0 => "a", 1 => "b", 2 => "c"]; // Explizit zugewiesene Schlüssel
2 $a5 = [0 => "a", 3 => "b", 4 => "c"]; // Zuweisung mit Lücken
```

Auf diese Art und Weise werden auch *assoziative* Arrays mit Strings als Schlüssel erzeugt:

```
1 $a6 = ["Vorname" => "John", "Nachname" => "Doe", "Alter" => "34"]; // Assoz. Array
2 $a7 = ["farbe" => "rot", "form" => "rund", 3]; // Gemischtes Array
```

Wie in der zweiten Zeile ersichtlich, kann auch ein gemischtes Array erzeugt werden. Der Zugriff auf die Zahl 3 erfolgt hierbei über den Index 0.

Arrays werden aufgrund der Lesbarkeit auch gerne in einer mehzeiligen Schreibweise deklariert. In diesem Fall steht die öffnende Klammer in der Zeile mit dem `=`-Zeichen und die schließende Klammer steht in der nächsten Zeile nach dem letzten Wert. Jede Zeile schließt mit einem Komma, auch die letzte (Trailing Comma):

```
1 $a8 = [
2     "multi",
3     "line",
4     "declaration",
5 ];
```

3.3.2 Zugriff auf Array-Elemente

Um auf einzelne Elemente eines Arrays zuzugreifen, wird der Schlüssel in eckigen Klammern angegeben. Dieser kann ein Integer bei indizierten Arrays oder ein String bei assoziativen Arrays sein. Der erste Indexwert ist immer 0.

```
1 $val1 = $a2[1]; // b
2 $val2 = $a4[0]; // a
3 $val3 = $a6["Vorname"]; // John
4 $val4 = $a7["farbe"]; // rot
5 $val5 = $a7[0]; // 3
```

Vorsicht: Würde mit `$a6[Vorname]` (also ohne Anführungszeichen) zugegriffen, so würde der Zugriff über den Wert der Konstante `Vorname` erfolgen, sofern eine solche definiert wurde.

3.3.3 Erweitern von Arrays

Arrays können in PHP jederzeit erweitert werden. Um einem indizierten Array einen neuen Wert anzuhängen, werden nur die Array-Klammern, aber keine explizite Position angegeben:

```
1 $arr1 = [7, 4, 3]; // Stellen 0, 1, 2 sind belegt
2 $arr1[] = 14; // Landet an Stelle 3
```

Es kann jedoch auch explizit ein Schlüssel angegeben werden, an dem der neue Wert eingefügt werden soll:

```
1 $arr1 = [7, 4, 3]; // Stellen 0, 1, 2 sind belegt
2 $arr1[7] = 14; // Landet an Stelle 7
3 $arr1["test"] = 5; // Landet an Stelle "test"
```

Eine Ausgabe mit `var_dump($arr1);` zeigt dies:

```
/var/www/html/wet2vo-examples/vo03/arrays/arrays.php:25:
array (size=5)
  0 => int 7
  1 => int 4
  2 => int 3
  7 => int 14
  'test' => int 5
```

3.3.4 Arrays durchlaufen

Arrays können ganz normal mit `while`- oder `for`-Schleifen durchlaufen werden. Einfacher ist jedoch die `foreach`-Schleife. Sie durchläuft immer das gesamte Array und gibt in jedem Durchlauf den aktuellen Wert in einer Variable zurück. Die prinzipielle Syntax sieht wie folgt aus:

```
1 foreach (Array as Wert) {
2     Anweisung;
3 }
```

Der erste Parameter ist also das Array, das durchlaufen werden soll. Der zweite Parameter erhält bei jedem Durchlauf den aktuellen Wert. Sein Name kann beliebig gewählt werden. Hier ein konkretes Beispiel:

```
1 $array1 = [3, 14, 4, 9];
2 foreach($array1 as $value) {
3     echo $value; // Ausgabe: 31449
4 }
```

Liegt ein assoziatives Array vor, existiert eine zweite Syntax, die es ermöglicht, auch den Schlüssel in einer Variable auszugeben:

```
1 foreach (Array as Schlüssel => Wert) {
2     Anweisung;
3 }
```

Der erste Parameter ist wiederum das zu durchlaufende Array, dann folgen zwei Variablen für den Schlüssel und den dazugehörigen Wert, deren Namen wieder frei gewählt werden können. In jedem Durchlauf sind darin die aktuellen Werte enthalten und können verarbeitet werden. Ein Beispiel:

```
1 $array2 = ["eins" => 1, "zwei" => 2, "drei" => 3];
2 foreach($array2 as $key => $value) {
3     echo "$key: $value"; // Ausgabe: eins: 1zwei: 2drei: 3
4 }
```

3.4 Funktionen der PHP-Bibliothek

Im Folgenden sollen, wie bereits in Vorlesung 2 für Strings geschehen, einige Funktionen aus der *PHP-Bibliothek* aufgezeigt werden, die für das Arbeiten mit Variablen und Arrays interessant und praktisch sind.

3.4.1 Funktionen für Variablen

Für Variablen gibt es einige praktische Funktionen, um deren Vorhandensein oder Typ zu überprüfen.

Auf Vorhandensein einer Variable prüfen

Die Funktion `isset($var [, $var2, ...])`⁶ überprüft, ob eine oder mehrere Variablen vorhanden sind und gibt `true` oder `false` zurück. Die Funktion akzeptiert beliebig viele Parameter. Sind mehrere angegeben, gibt sie nur dann `true` zurück, wenn alle angegebenen Variablen auch vorhanden sind.

```
1 $a = "hallo";
2 $b = "welt";
3 isset($a); // true
4 isset($a, $b); // true
5 isset($c); // false
6 isset($a, $c); // false
```

Seit PHP 7 kann statt einer Überprüfung mittels `isset` vor der Verwendung einer Variable auch der Null Coalescing Operator (siehe Abschnitt 2.3.6) verwendet werden.

Variablen löschen

Die Funktion `unset($var [, $var2, ...])`⁷ löscht eine (oder mehrere) Variablen. Die Variable existiert danach nicht mehr, eine Überprüfung mit `isset()` ergibt also `false`.

```
1 $a = "hallo";
2 if (isset($a)) { ... } // true
3 unset($a);
4 if (isset($a)) { ... } // false
```

⁶<https://www.php.net/manual/de/function.isset.php>

⁷<https://www.php.net/manual/de/function.unset.php>

Typ-Prüfungen für Variablen

Den Typ einer Variable zu kennen, kann oft wichtig sein. PHP bietet dafür verschiedene Funktionen an. Sie alle geben `true` oder `false` zurück:

- `is_array($var)`:⁸ Überprüft, ob eine Variable ein Array ist.
- `is_bool($var)`:⁹ Überprüft, ob es sich um eine boolesche Variable handelt.
- `is_float($var)`:¹⁰ Überprüft, ob es sich um eine Gleitkommazahl handelt.
- `is_int($var)`:¹¹ Überprüft, ob es sich um eine Ganzzahl handelt.
- `is_null($var)`:¹² Überprüft, ob eine Variable `null` enthält.
- `is_numeric($var)`:¹³ Überprüft, ob eine Variable eine Zahl oder ein numerischer String (ein String, der nur aus Zahlen besteht) ist.
- `is_string($var)`:¹⁴ Überprüft, ob es sich um einen String handelt.

Dazu einige Beispiele:

```
1 $a = "hallo";
2 $b = 34;
3 $c = "1234";
4 is_array($a); // false
5 is_bool($a); // false
6 is_float($b); // false
7 is_int($b); // true
8 is_null($c); // false
9 is_numeric($c); // true
10 is_string($c); // true
```

Ausgabe von Variablen-Inhalten

Inhalte von einfachen Variablen können mit `echo` oder `print()` einfach im Browser ausgegeben werden. Gerade aber bei Arrays oder später Objekten ist dies nicht mehr so einfach möglich. PHP bietet dazu spezielle Funktionen, die dies erleichtern.

Mittels `print_r($var [, $return = false])`¹⁵ wird der Inhalt einer Variable in menschenlesbarer Form ausgegeben. Vor allem im Zusammenhang mit Arrays ist dies sehr nützlich.

```
1 $var = ["test", "wert", "eintrag"];
2 print_r($var);
```

Dies erzeugt folgende Ausgabe:

```
Array ( [0] => test [1] => wert [2] => eintrag )
```

Soll der Rückgabewert nicht im Browser ausgegeben, sondern weiterverarbeitet werden, kann der optionale zweite Parameter auf `false` gesetzt werden.

⁸<https://www.php.net/manual/de/function.is-array.php>

⁹<https://www.php.net/manual/de/function.is-bool.php>

¹⁰<https://www.php.net/manual/de/function.is-float.php>

¹¹<https://www.php.net/manual/de/function.is-int.php>

¹²<https://www.php.net/manual/de/function.is-null.php>

¹³<https://www.php.net/manual/de/function.is-numeric.php>

¹⁴<https://www.php.net/manual/de/function.is-string.php>

¹⁵<https://www.php.net/manual/de/function.print-r.php>

Die Funktion `var_dump($var [, $var2, ...])`¹⁶ erzeugt eine detailliertere Ausgabe wie `print_r` und gibt auch den Typ des jeweiligen Eintrags mit an.

```
1 $var = ["test", "wert", "eintrag"];
2 var_dump($var);
```

Dies erzeugt folgende Ausgabe:

```
/var/www/html/wet2vo-examples/vo03/php-funktionen/variablen-funktionen.php:9:
array (size=3)
  0 => string 'test' (length=4)
  1 => string 'wert' (length=4)
  2 => string 'eintrag' (length=7)
```

3.4.2 Funktionen für Arrays

Für Arrays sind einerseits Funktionen zum Ermitteln der Größe oder zum Auffinden von Werten interessant, andererseits sind Funktionen im Zusammenhang mit Strings wichtig.

Größe eines Arrays

Die Funktion `count($array [, $mode])`¹⁷ (bzw. ihr Alias `sizeof()`) geben die Größe eines Arrays – also die Anzahl der darin enthaltenen Elemente – zurück. Wird der zweite, optionale Parameter `$mode` mit dem Wert `COUNT_RECURSIVE` angegeben, so zählt die Funktion rekursiv. Dies ist dann nützlich, wenn ein Array wieder andere Arrays enthält und die einzelnen Elemente aller Arrays summiert werden sollen.

```
1 $arr1 = [2, "Hallo", 4.3];
2 $arr2 = ["Test", 7, [3, 5, "Hi"]];
3 echo count($arr1); // 3
4 echo count($arr2); // 3
5 echo count($arr2, COUNT_RECURSIVE); // 6
```

Werte prüfen und zurückgeben

Die Funktion `in_array($needle, $haystack)`¹⁸ überprüft, ob sich ein gesuchter Wert (`$needle`) in einem Array (`$haystack`) befindet. Diese Funktion erspart ein manuelles Vergleichen in einer Schleife.

```
1 $arr3 = [1, 3, 5, 7];
2 in_array(3, $arr3); // true (1)
```

Mit `array_first($array)`¹⁹ und `array_last($array)`²⁰ gibt es seit PHP 8.5 die Möglichkeit, den ersten und letzten Wert im Array abzufragen.

```
1 echo array_first($arr3); // 1
2 echo array_last($arr3); // 7
```

¹⁶<https://www.php.net/manual/de/function.var-dump.php>

¹⁷<https://www.php.net/manual/de/function.count.php>

¹⁸<https://www.php.net/manual/de/function.in-array.php>

¹⁹<https://www.php.net/manual/en/function.array-first.php>

²⁰<https://www.php.net/manual/en/function.array-last.php>

Schlüssel prüfen und zurückgeben

Ebenso lässt sich abprüfen, ob ein Schlüsselwert existiert. Dazu dient die Funktion `array_key_exists($key, $array)`²¹. Ist der Schlüssel vorhanden, wird `true` zurückgegeben, ansonsten `false`.

```
1 $arr4 = ["name" => "John Doe", "alter" => 34, "geschlecht" => "männlich"];
2 echo array_key_exists("alter", $arr4); // true (1)
```

Mit `array_key_first($array)`²² und `array_key_last($array)`²³ können der erste und letzte Schlüssel im Array abgefragt werden.

```
1 echo array_key_first($arr4); // name
2 echo array_key_last($arr4); // geschlecht
```

Einen String in ein Array aufteilen

Mit der Funktion `explode($delimiter, $string)`²⁴ kann ein String zerteilt und als Array zurückgegeben werden. Der Teiler `$delimiter` ist eine beliebige Zeichenkette. Sie dient nur zur Trennung und scheint bei den Einzelteilen nicht mehr auf. Im folgenden Beispiel wird ein String mit einem Leerzeichen aufgeteilt.

```
1 $text = "Hello little world!";
2 $pieces = explode(" ", $text);
3 print_r($pieces); // Array ( [0] => Hello [1] => little [2] => world! )
```

Ein Array zu einem String zusammenfügen

Die Umkehrfunktion zu `explode()` stellt `implode($glue, $pieces)`²⁵ dar. Unter Angabe eines Verbindungselements `$glue` werden hier die einzelnen Einträge eines Arrays zu einem String zusammengefügt. Im folgenden Beispiel werden die Einträge eines Arrays in einen kommaseparierten String eingefügt.

```
1 $parts = ["name", "alter", "geschlecht"];
2 $string = implode(",", $parts);
3 echo $string; // name,alter,geschlecht
```

Einen String per Regex aufteilen

Anstatt wie bei `explode()` eine einfache Zeichenkette anzugeben, kann ein String auch anhand einer Regular Expression aufgeteilt werden. Dies ist mit Hilfe der Funktion `preg_split($pattern, $string)`²⁶ möglich. Details zu Regular Expressions folgen in Abschnitt 3.6. Das folgende Beispiel zerteilt ein Datum an typischen Trennzeichen wie Schrägstrich (/), Punkt (.) oder Bindestrich (-). Durch die Regular Expression kann angegeben werden, dass die Aufteilung erfolgt, egal welches der drei Zeichen im Datum vorkommt.

²¹ <https://www.php.net/manual/de/function.array-key-exists.php>

²² <https://www.php.net/manual/de/function.array-key-first.php>

²³ <https://www.php.net/manual/de/function.array-key-last.php>

²⁴ <https://www.php.net/manual/de/function.explode.php>

²⁵ <https://www.php.net/manual/de/function.implode.php>

²⁶ <https://www.php.net/manual/de/function.preg-split.php>


```
1 $date = "13/03/2025";  
2 $values = preg_split("#[/.-]#", $date);  
3 print_r($values); // Array ( [0] => 13 [1] => 03 [2] => 2025 )
```

Arrays mit Callbacks durchsuchen

Mit PHP 8.4 sind weitere, mächtige Array-Funktionen hinzugekommen. Diese erlauben mit einer Callback-Funktion die Suchkriterien zu definieren:

- `array_find($array, $callback)`²⁷: Ein Element finden, das den Kriterien des Callbacks entspricht.
- `array_find_key($array, $callback)`²⁸: Einen Schlüssel finden, der den Kriterien des Callbacks entspricht.
- `array_any($array, $callback)`²⁹: Überprüfen, ob mindestens ein Element im Array den Kriterien des Callbacks entspricht.
- `array_all($array, $callback)`³⁰: Überprüfen, ob alle Elemente im Array den Kriterien des Callbacks entsprechen.

3.5 Formulardaten verarbeiten

Das Arbeiten mit *Formularen* gehört zu den wichtigsten Dingen im PHP-Workflow. Es stellt die einfachste Möglichkeit dar, Daten von Benutzer*innen abzufragen. Im Folgenden wird der serverseitige Ablauf besprochen, wie auf die eingegebenen Daten zugegriffen werden kann.

3.5.1 Vordefinierte Variablen

Für den Zugriff auf Formulardaten stellt PHP drei sogenannte *superglobale Arrays* zur Verfügung. Diese enthalten alle Daten aus einem abgeschickten Formular. Je nachdem, auf welche Art und Weise das Formular übertragen wurde (GET oder POST), liegen diese Werte in unterschiedlichen Arrays.

Übertragung mittels GET

GET ist eine HTTP-Übertragungsmethode (siehe Abschnitt 1.10). Um diese Methode zu verwenden, muss das Attribut `method` im Formularfeld den Wert `get` aufweisen:

```
1 <form method="get" action="get.php">  
2   <label for="infolabel">Info:</label>  
3   <input type="text" name="info" id="infolabel">  
4   <button type="submit">Abschicken</button>  
5 </form>
```

²⁷ <https://www.php.net/manual/de/function.array-find.php>

²⁸ <https://www.php.net/manual/de/function.array-find-key.php>

²⁹ <https://www.php.net/manual/de/function.array-any.php>

³⁰ <https://www.php.net/manual/de/function.array-all.php>

Wird dieses Formular abgeschickt, wird ein Request auf `get.php` erstellt und der Browser erhält vom Webserver diese als Response zurückgeliefert. Die Werte des Eingabefelds kommen mit einem Fragezeichen angehängt in `key=value`-Form beim Request mit. So könnte der URL des obigen Formulars etwa dann `http://localhost:8080/wet2vo-examples/vo03/formulardaten/get.php?info=Test` lauten, falls das Wort „Test“ im Formularfeld eingegeben wurde. Relevant für den Schlüssel ist der Name des Formularfelds, der durch das `name`-Attribut (nicht `id`!) gesetzt wird. Weitere Schlüssel-Wert-Paare werden mit dem kaufmännischen Und (`&`) angehängt.

Um nun mit PHP auf diesen Wert aus dem Request zugreifen zu können, stellt PHP das superglobale Array `$_GET` zur Verfügung. Die Arraystellen erhalten ihren Namen von den Formularfeldern, demnach kann über `$_GET["info"]` der Wert aus dem Formularfeld ausgelesen werden.

GET-Parameter können auch einfach „so“ an eine URL angehängt werden und scheinen ebenfalls im Array `$_GET` auf.

Übertragung mittels POST

POST ist eine weitere, in HTTP vorgesehene Übertragungsart. Die Daten werden hier nicht im URL sondern gesondert übertragen, weshalb auch größere Mengen möglich sind. Darüber hinaus bleibt der URL unverändert und die übertragenen Daten werden nicht sofort preisgegeben. Ein POST-Formular muss beim Attribut `method` den Wert `post` aufweisen:

```
1 <form method="post" action="post.php">
2   <label for="info">Info:</label>
3   <input type="text" name="info" id="info">
4   <button type="submit">Abschicken</button>
5 </form>
```

Wird das Formular hier abgeschickt, lautet der URL z.B. `http://localhost:8080/wet2vo-examples/vo03/formulardaten/post.php` – es sind keine Parameter sichtbar. Für den Zugriff in PHP existiert das superglobale Array `$_POST`. Die jeweilige Arraystelle wird wiederum über den Namen des Formularfelds bestimmt, in diesem Fall ist der Zugriff also über `$_POST["info"]` möglich.

Übertragungsunabhängiges Array

Das superglobale Array `$_REQUEST` enthält sowohl die Daten von `$_POST` als auch von `$_GET` und kann somit universell eingesetzt werden. Generell sollte jedoch explizit `$_POST` oder `$_GET` eingesetzt werden, um Fehler und Sicherheitslücken zu vermeiden. Würde etwa im Code mit einer Schleife über `$_REQUEST` iteriert und die Werte ausgegeben, könnte einfach durch das Anhängen an die URL-Zeile ein weiterer GET-Parameter „eingeschmuggelt“ werden, der dann auch angezeigt wird, obwohl alle Formulare auf der Seite nur mit POST arbeiten.

3.5.2 Übergabe von Werten

Die Übergabe von Werten von Formularen nach PHP wurde bereits soeben angesprochen. Je nach Übertragungsart kann ein superglobales Array ausgelesen werden. An der

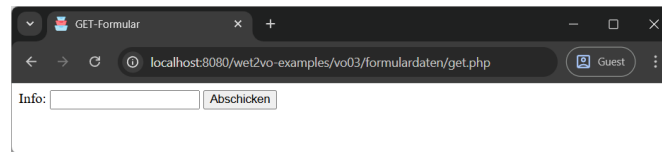


Abbildung 3.2: Die Seite zeigt nur das Formularfeld an. Da noch kein Inhalt eingegeben wurde, gibt der PHP-Teil nichts aus.

Stelle des Formularfeldnamens finden sich dann die Daten. Anbei zwei kleine Beispiele mit den beiden Übertragungsarten GET und POST.

```
1 <!DOCTYPE html>
2 <html lang="de">
3   <head>
4     <title>GET-Formular</title>
5     <meta charset="utf-8">
6   </head>
7   <body>
8     <form method="get" action="get.php">
9       <label for="infolabel">Info:</label>
10      <input type="text" name="info" id="infolabel">
11      <button type="submit">Abschicken</button>
12    </form>
13
14    <?php
15      if (isset($_GET["info"])) {
16        echo "<p>{$_GET["info"]}</p>";
17      }
18    ?>
19  </body>
20 </html>
```

Dieses Beispiel zeigt die Datei `get.php`. Sie enthält ein Formular mit einem Eingabefeld. Darunter ist ein kleiner PHP-Block, der überprüft, ob `$_GET["info"]` einen Eintrag enthält. Falls dies so ist, wird dieser ausgegeben. Zu Beginn wird nur das Formularfeld angezeigt, denn im superglobalen Array befinden sich noch keine Einträge (siehe Abbildung 3.2).

Danach wird der String „Hi“ eingetippt und das Formular abgeschickt. In der URL-Zeile wird der Parameter angehängt und `get.php` wird erneut geladen da das `action`-Attribut wieder auf die gleiche Seite verweist. Es hätte auch eine zweite, eigene PHP-Seite aufgerufen werden können.

Dieses Mal liefert `isset($_GET["info"])` den Wert `true` zurück und somit wird dieser Wert als Absatz ausgegeben. Abbildung 3.3 stellt dies dar.

Derselbe Ablauf lässt sich auch mit POST durchführen. Der Code dafür:

```
1 <!DOCTYPE html>
2 <html lang="de">
3   <head>
4     <title>POST-Formular</title>
5     <meta charset="utf-8">
6   </head>
7   <body>
8     <form method="post" action="post.php">
```

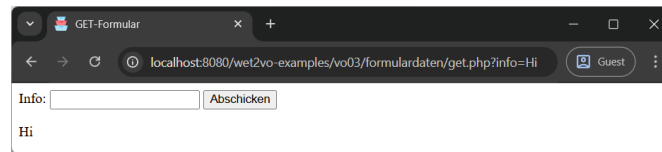


Abbildung 3.3: Nachdem das Formular abgeschickt wurde, wird der eingegebene Wert angezeigt.

```

 9      <label for="info" >Info:</label>
10      <input type="text" name="info" id="info">
11      <button type="submit">Abschicken</button>
12  </form>
13
14  <?php
15      if (isset($_POST["info"])) {
16          echo "<p>{$_POST["info"]}</p>";
17      }
18      ?>
19  </body>
20 </html>

```

Geändert haben sich die Übertragungsart im Formular und der Zugriff über das superglobale Array `$_POST` (und die Datei heißt nun `post.php`). Die Ausgabe im Browser ist ebenfalls gleich, nur die angehängten Parameter in der URL-Zeile fehlen.

Kommen im Formularfeld mehrere Eingabefelder vor, funktioniert die Werteübergabe dafür analog. Wichtig ist, dass alle einzigartigen Namen im `name`-Attribut aufweisen, um sie unterscheiden zu können.

Anmerkung: Die ungeprüfte Übernahme und Weiterverarbeitung von User*innen-Input kann schwere Sicherheitslücken öffnen. Vorlesung 7 geht genauer auf diese Thematik ein.

3.5.3 Übergabe von Mehrfachauswahl

Im Falle von *Mehrfachauswahl* bei Checkboxes oder Multi-Select-Feldern muss eine leicht geänderte Strategie bei der Wertübergabe angewandt werden. In diesem Fall müssen alle Elemente denselben Namen aufweisen. Zusätzlich muss dieser mit abschließenden Array-Klammern (`[]`) versehen sein. Dies macht eine serverseitige Verarbeitung als Array möglich.

Das folgende Beispiel zeigt eine Mehrfachauswahl mit Checkboxes:

```

1 <!DOCTYPE html>
2 <html lang="de">
3   <head>
4     <title>Mehrfach-Auswahl</title>
5     <meta charset="utf-8">
6   </head>
7   <body>
8     <form method="post" action="checkboxes.php">
9       <p>Meine Hobbys:</p>
10      <label for="musik">Musik:</label>
11      <input type="checkbox" name="hobbies[]" value="musik" id="musik">

```

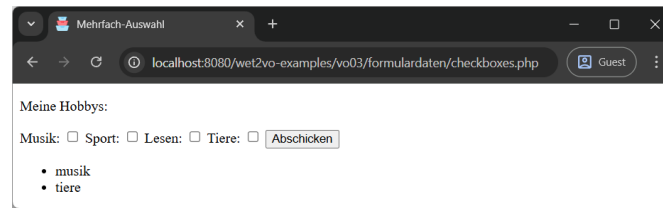


Abbildung 3.4: Die Checkboxes 1 (Musik) und 4 (Tiere) wurden ausgewählt. Die ausgegebenen Werte sind klein geschrieben, da sie im `value`-Attribut der jeweiligen Checkbox so angegeben sind.

```

12     <label for="sports">Sport:</label>
13     <input type="checkbox" name="hobbies[]" value="sport" id="sports">
14     <label for="reading">Lesen:</label>
15     <input type="checkbox" name="hobbies[]" value="lesen" id="reading">
16     <label for="animals">Tiere:</label>
17     <input type="checkbox" name="hobbies[]" value="tiere" id="animals">
18     <button type="submit">Abschicken</button>
19 </form>
20
21 <?php
22 if (isset($_POST["hobbies"])) {
23     echo "<ul>";
24     foreach ($_POST["hobbies"] as $hobby) {
25         echo "<li>$hobby</li>";
26     }
27     echo "</ul>";
28 }
29 ?>
30 </body>
31 </html>

```

Die Checkboxes haben alle den Namen `hobbies[]`, somit existiert in PHP ein Eintrag im superglobalen Array `$_POST["hobbies"]`, der ebenfalls wieder ein Array ist. Mit einer `foreach`-Schleife kann dann ganz einfach über das Array iteriert und jeder einzelne Eintrag in Form einer unsortierten Liste ausgegeben werden. Die Werte im Array werden von den Werten im Attribut `value` bei den Checkboxes bestimmt. Werden also Checkbox Nummer 1 (Musik mit dem Wert `musik`) und 4 (Tiere mit dem Wert `tiere`) ausgewählt, sieht die Ausgabe im Browser wie in Abbildung 3.4 aus.

3.6 Regular Expressions

Einige der Stringfunktionen in Abschnitt 2.3.9 von Vorlesung 2 boten eine simple Methode zu überprüfen, ob eine Zeichenkette in einer anderen vorkommt, oder nicht. Oft jedoch ist nicht bekannt, wie diese Zeichenkette genau aussieht. Es soll etwa nur überprüft werden, ob am Ende eines Strings zwei Zahlen stehen, es ist aber nicht klar, welche das sind, da sie von Eingaben der Benutzer*innen abhängen. In diesem Fall ist eine Überprüfung auf eine konkrete Zeichenkette nicht mehr möglich. Vielmehr muss auf ein Muster hin überprüft werden – in diesem Fall „zwei Zahlen am Ende des Strings“. Überprüfungen mit derlei Mustern werden *Regular Expressions* (kurz Regexp) oder zu

deutsch reguläre Ausdrücke genannt.

3.6.1 Regex-Funktionen

Reguläre Ausdrücke sind nicht allein auf PHP beschränkt. Es gibt sie in fast jeder Programmier- oder Skriptsprache. Noch dazu kommt, dass es verschiedene Syntaxen gibt, die verwendet werden können. In PHP herrscht die sogenannte *Perl-Syntax* vor.

Die erste Übereinstimmung finden

Um zu überprüfen, ob ein bestimmtes Muster in einem Text (d. h. einer Zeichenkette) vorkommt, wird die Funktion `preg_match($pattern, $subject [, &$matches])`³¹ benötigt. Hierbei ist `$pattern` das Muster, nach dem gesucht werden soll (siehe dazu Abschnitt 3.6.2), `$subject` ist die Zeichenkette, die durchsucht werden soll. Optional kann mit `$matches` noch ein dritter Parameter angegeben werden, der von der Funktion mit Teil-Suchergebnissen befüllt wird. Mehr dazu in Abschnitt 3.6.2 auf Seite 3-28. Die Funktion gibt 1 zurück, wenn eine Übereinstimmung gefunden wurde oder 0 falls nicht.

Alle Übereinstimmungen finden

Die Funktion `preg_match()` stoppt nach der ersten gefundenen Übereinstimmung. Sollen alle Vorkommen eines Musters erkannt werden, muss stattdessen auf die Funktion `preg_match_all($pattern, $subject [, &$matches, $flags])`³² zurückgegriffen werden. Die einzelnen Vorkommen können in `$matches` abgelegt werden (siehe Seite 3-28), `$flags` bestimmt dabei die Reihenfolge. Die Funktion gibt die Anzahl der Vorkommen als Zahl zurück.

Übereinstimmungen ersetzen

Soll nicht nur gesucht werden, ob ein Muster in einer Zeichenkette erkannt wird, sondern bei einer Übereinstimmung der gefundenen Text durch einen anderen ersetzen werden, kommt `preg_replace($pattern, $replacement, $subject)`³³ zum Einsatz. Die Parameter sind gleich, hinzu kommt lediglich `$replacement`, das Muster welches als Ersatz für die gefundene Zeichenkette dient.

Mit `preg_replace_callback($pattern, $callback, $subject)`³⁴ und dem ähnlichen `preg_replace_callback_array($patterns_callbacks, $subject)`³⁵ gibt es zwei Funktionen, die statt eines konkreten Ersetzungsmusters eine eigene Callback-Funktion aufrufen. Erste ruft dabei für ein Muster genau eine Funktion auf, letztere existiert seit PHP 7 und erlaubt es, ein Array mit Mustern und dazugehörigen Callbacks zu hinterlegen.

Beispiele mit Regex-Funktionen folgen im nächsten Abschnitt.

³¹ <https://www.php.net/manual/de/function.preg-match.php>

³² <https://www.php.net/manual/de/function.preg-match-all.php>

³³ <https://www.php.net/manual/de/function.preg-replace.php>

³⁴ <https://www.php.net/manual/de/function.preg-replace-callback.php>

³⁵ <https://www.php.net/manual/de/function.preg-replace-callback-array.php>

3.6.2 Muster definieren

Um ein zu suchendes *Muster* zu definieren, gibt es eine spezielle Syntax. Ein Muster ist immer eine Zeichenkette, steht also unter Anführungszeichen. Zudem verlangt die Perl-Syntax, dass am Anfang und Ende des Musters ein identisches Begrenzungszeichen steht. Dieses Zeichen kann frei gewählt werden, sollte aber im Muster nicht vorkommen. In der Regel werden zwei Schrägstriche (/) gewählt. Wird der Schrägstrich als Teil des Musters benötigt, kann z. B. auch die Raute (#) oder ein ähnliches Zeichen eingesetzt werden. Nur der Backslash ist nicht möglich. Er wird zum Escapen von reservierten Zeichen benötigt.

Eine Regular Expression sieht also prinzipiell wie folgt aus (statt den drei Punkten stehen dann die jeweiligen Muster-Zeichen, die im folgenden erklärt werden):

```
1 if (preg_match("/.../", $string)) { ... }
```

Einfache Zeichen

Soll mit einer Regular Expression nach einfachen Zeichen gesucht werden, genügt es, diese einfach im Muster zwischen den Begrenzungszeichen anzugeben:

```
1 "/a/"
2 "/test/"
```

Das erste Muster sucht, ob der Buchstabe „a“ vorkommt, das zweite sucht nach dem Muster „test“. Dies lässt sich jedoch mit String-Funktionen genauso effizient erledigen. Dennoch ein Beispiel:

```
1 preg_match("/a/", "Hallo Welt!"); // 1 (true)
```

Eine Sonderstellung nimmt der Punkt (.) ein. Er ist eine sogenannte Wildcard, steht also für jedes beliebige Zeichen. Ein Beispiel dazu:

```
1 "/.aus/"
```

Dieses Muster passt auf eine Reihe von Zeichenketten: „Maus“, „Haus“, „raus“ etc.

Soll explizit nach einem Punkt-Zeichen gesucht werden (z. B. um zu überprüfen, ob ein Datum im europäischen Format TT.MM.YYYY geschrieben ist), muss der Punkt mit einem Backslash escaped werden.

```
1 "/\.\aus/"
```

Dieses Muster überprüft explizit auf die Zeichenkette „.aus“.

Zeichenklassen

Zeichenklassen lassen im Gegensatz zu den eben gezeigten Beispielen bereits Alternativen zu. Es wird nicht nur nach „a“ sondern z. B. nach „entweder a oder b oder c“ gesucht. Eine solche Klasse wird in eckigen Klammern angegeben und kann frei definiert werden. Der Bindestrich (-) wird verwendet, um Bereiche anzugeben. Einige Beispiele:

```
1 "[abc]/"
2 "[a-z]/"
3 "[0-5]/"
4 "[a-zA-Z0-9_]/"
```

Der erste Ausdruck passt auf alle Zeichenketten, die entweder a, b, c oder das Leerzeichen enthalten. Der zweite sucht nach allen Kleinbuchstaben von a bis z, der dritte sucht Zahlen von 0 bis 5, der vierte sucht alle Klein- und Großbuchstaben, alle Zahlen und den Unterstrich.

```
1 preg_match("/[abc ]/", "Hallo Welt!"); // 1 (true)
```

Zusätzlich gibt es noch eine Reihe an benannten Zeichenklassen, die das Definieren von Bereichen oft erübrigen. Diese Klassennamen werden zwischen zwei eckige Klammern und Doppelpunkte gesetzt, z. B. `[:digit:]`.

Folgende solcher Klassen sind definiert:

- `[:alnum:]`: Alle Zahlen und Buchstaben.
- `[:alpha:]`: Alle Buchstaben.
- `[:ascii:]`: Alle 128 ASCII Zeichen.
- `[:blank:]`: Leerzeichen oder Tabulator.
- `[:cntrl:]`: Diverse Steuerzeichen.
- `[:digit:]`: Dezimalzahlen.
- `[:graph:]`: Alle druckbaren Zeichen, ohne Leerzeichen.
- `[:lower:]`: Alle Kleinbuchstaben.
- `[:print:]`: Alle druckbaren Zeichen, mit Leerzeichen.
- `[:punct:]`: Alle druckbaren Zeichen, ohne Buchstaben und Zahlen.
- `[:space:]`: Leerzeichen.
- `[:upper:]`: Alle Großbuchstaben.
- `[:word:]`: Buchstaben, Ziffern und der Unterstrich.
- `[:xdigit:]`: Hexadezimale Zahlen.

Allerdings können diese benannten Zeichenklassen nur innerhalb einer Zeichenklasse verwendet werden, weshalb nochmals zwei eckige Klammern benötigt werden:

```
1 preg_match("/[[:alnum:]]/", "Hallo 2026!"); // 1 (true)
```

Sollen in Zeichenklassen bestimmte Mengen an Zeichen explizit ausgeschlossen werden, kann dies mit dem Zirkumflex-Zeichen (^) geschehen. Mit dem folgenden Muster kann nach Zeichenketten gesucht werden, die keine Selbstlaute enthalten:

```
1 "/[^aeiou]/"
```

Im Beispiel wird ein Muster gesucht, das mit einem großen „H“ beginnt, dann darf *kein* Vokal folgen, danach kommen „llo“. Dies kommt so nicht vor, also schlägt der Vergleich fehl. Stünde statt „Hallo“ das Wort „Hyllo“ oder „Hbllo“ dort, gäbe es einen Treffer.

```
1 preg_match("/H[^aeiou]llo/", "Hallo Welt!"); // 0 (false)
```

Zeichen am Anfang und Ende eines Strings

Die Spezialzeichen Zirkumflex (^) und Dollar (\$) haben eine spezielle Bedeutung. Steht ein Zirkumflex am Beginn eines Muster-Ausdrucks, so bedeutet dies, dass eine übereinstimmende Zeichenkette mit diesem Muster beginnen muss. Umgekehrt bedeutet ein

Dollarzeichen am Ende, dass der String mit diesem Muster enden muss. Hier einige Beispiele:

```
1 "/^Die/"
2 "/^[0-9]/"
3 "/er$/"
4 "/^ABC$/"
```

Der erste Ausdruck findet Strings die mit „Die“ beginnen, der zweite findet Strings, die mit einer Zahl beginnen, der dritte Ausdruck findet Strings, die mit „er“ enden, der vierte findet Strings, die mit „ABC“ beginnen und enden, d. h. genau aus ABC (und keinen weiteren Zeichen) bestehen.

Wichtig ist, `"/^[0-9]/"` und `"/[^0-9]/"` nicht zu verwechseln. Das erste Muster findet Strings, die mit Zahlen von 0–9 beginnen, das zweite findet Strings, die keine Zahlen von 0–9 enthalten. Das Zirkumflex-Zeichen hat in diesem Fall eine doppelte Bedeutung, je nachdem, ob es außerhalb oder innerhalb der eckigen Klammern steht.

Angabe der Häufigkeit

Oft ist es nötig, bei einem Teil einer Regular Expression angeben zu können, wie häufig dieser Teil auftreten kann. Dafür stehen zunächst drei Zeichen zur Verfügung:

- `?`: Der Ausdruck kann 0 oder 1 Mal auftreten.
- `*`: Der Ausdruck kann 0 bis n Mal auftreten.
- `+`: Der Ausdruck kann 1 bis n Mal auftreten.

Darüber hinaus kann in geschwungenen Klammern auch eine explizite Häufigkeit angegeben werden:

- `{m}`: Der Ausdruck muss genau m Mal auftreten.
- `{m,}`: Der Ausdruck muss mindestens m Mal auftreten.
- `{,n}`: Der Ausdruck darf höchstens n Mal auftreten.
- `{m,n}`: Der Ausdruck darf zwischen m und n Mal auftreten.

Die folgenden Regular Expressions zeigen einige Beispiele:

```
1 preg_match("/ca+t/", "caaaaaat"); // 1 (true)
2 preg_match("/ca+t/", "ct"); // 0 (false)
3 preg_match("/ca?t/", "caaaaaat"); // 0 (false)
4 preg_match("/ca*t/", "ct"); // 1 (true)
```

Der Modifikator gilt hier immer nur für das Zeichen, hinter dem er steht. In diesen Beispielen also für das „a“. Wird mit Zeichenklassen (also Bereichen von Zeichen) gearbeitet, gilt er natürlich für den gesamten Bereich.

```
1 "/[0-9]{3}-[0-9]{3}-[0-9]{4}/"
```

Dies wäre etwa eine Regular Expression, um US-Telefonnummern im Format xxx-xxx-xxxx zu erkennen. Es werden 3 beliebige Zahlen gefolgt von einem Bindestrich, gefolgt von drei Zahlen, einem Bindestrich und schließlich 4 Zahlen erkannt. Die Nummer 123-456-7890 wäre etwas korrekt, 12-3456-7890 hingegen nicht.

Gruppierung/Subpatterns

Mit Hilfe von Klammerung können Einzelzeichen auch gruppiert werden. Somit kann ein Modifikator auch auf mehrere Zeichen angewandt werden:

```
1 "/a(bc)*/"
```

In diesem Fall wird eine Zeichenkette gesucht, die ein „a“ enthält, und optional die zwei Zeichen „bc“. Das eigentliche Kriterium ist hier nur „a“, denn ob „bc“ vorkommt oder nicht, ist in diesem Fall ja egal. Darum ist auch ein String wie „ab“ korrekt („a“ kommt vor, „ab“ nicht).

```
1 preg_match("/a(bc)*/", "aaaaa"); // 1 (true)
2 preg_match("/a(bc)*/", "abc"); // 1 (true)
3 preg_match("/a(bc)*/", "bc"); // 0 (false)
4 preg_match("/a(bc)*/", "ab"); // 1 (true)
```

Ein weiterer Nebeneffekt der Klammerung ist, dass geklammerte Teile von der Funktion `preg_match()` in ein Array zerlegt werden. Dieses wird als dritter Parameter angegeben und dann automatisch befüllt. Das Array an der Stelle 0 enthält den gesamten Ausdruck, der gefunden wurde. Die Stelle 1 enthält den Inhalt der ersten Klammer, usw. Hier ein Beispiel:

```
1 if (preg_match("/([0-9]{1,2})\.([0-9]{1,2})\.([0-9]{4})/", "23.04.2026", $matches)){
2     echo $matches[0]; // 23.04.2026
3     echo $matches[1]; // 23
4     echo $matches[2]; // 04
5     echo $matches[3]; // 2026
6 }
```

Diese Regular Expression sucht europäische Datumsformate. Zunächst stehen 1–2 Ziffern, dann ein Punkt, dann wieder 1–2 Ziffern, ein Punkt und schließlich 4 Ziffern. Durch die Angabe des Parameters `$matches` (kann beliebig heißen), wird ein Array erzeugt. Die Regex wurde geklammert (jeweils um die Ziffern), somit wird für jeden Klammerninhalt ein eigener Eintrag im Array erzeugt und die einzelnen Teile sind leicht zugreifbar.

Alternativen

Mit der sogenannten Pipe (`|`) lässt sich eine Alternative erzeugen. D. h. entweder der eine oder andere angegebene Teil des Musters muss vorkommen. Einige Beispiele:

```
1 preg_match("/cat|dog/", "The cat bit me"); // 1 (true)
2 preg_match("/cat|dog/", "The dog bit me"); // 1 (true)
3 preg_match("/cat|dog/", "The rabbit bit me"); // 0 (false)
```

Vorsicht bei Klammerung und Spezialzeichen wie `(^)` und `($)`:

```
1 "/^cat|dog$/" // Muss mit cat beginnen oder dog enden
2 "/^(cat|dog)$/" // Muss entweder mit cat oder dog beginnen und enden
```

Escape-Sequenzen

Der Backslash dient nicht nur als reines Escape-Zeichen (um z. B. mittels `\.` tatsächlich den Punkt zu suchen), sondern ist auch die Grundlage für einige sogenannte *Escape-Sequenzen*. Dies sind entweder nicht druckbare Zeichen, generische Zeichen-Typen oder

Tabelle 3.1: Die wichtigsten Regular Expression-Zeichen im Überblick.

Zeichen	Bedeutung
<code>string</code>	Sucht genau das Wort „string“
<code>.</code>	Ein beliebiges Zeichen
<code>^</code> , <code>\$</code>	Anfang und Ende des Strings
<code>()</code> , <code> </code>	Gruppierung, Alternative
<code>*</code> , <code>+</code> , <code>?</code>	Wiederholungen ($0-n$, $1-n$, $0-1$)
<code>{m}</code> , <code>{m,n}</code>	Explizite Wiederholungen
<code>[]</code> , <code>[^]</code>	Zeichenklasse und ausgeschlossene Zeichenklasse
<code>\</code>	Zum Escapen von Zeichen wie z. B. <code>.</code>

auch sogenannte Bedingungen, die an einem Teil des Strings auftreten können (z. B. ein Wort ist zu Ende). Die folgende Liste stellt einige wichtige Escape-Sequenzen dar, weitere finden sich unter <https://www.php.net/manual/de/regexp.reference.escape.php>.

- `\n`: Ein Zeilenumbruch (nicht druckbares Zeichen).
- `\t`: Tabulator (nicht druckbares Zeichen).
- `\d`: Jede beliebige Dezimalzahl (Zeichen-Typ).
- `\D`: Jedes Zeichen, das keine Dezimalzahl ist (Zeichen-Typ).
- `\s`: Jede Art von Whitespace-Zeichen (Zeichen-Typ).
- `\S`: Jedes Zeichen, das kein Whitespace ist (Zeichen-Typ).
- `\b`: Eine Wortgrenze (Bedingung).
- `\B`: Keine Wortgrenze (Bedingung).

3.6.3 Muster-Übersicht

Zur besseren Übersicht stellt Tabelle 3.1 nochmals alle wichtigen Zeichen im Zusammenhang mit Regular Expressions dar.

3.7 Weiterführende Literatur

Wie auch bereits im ersten Teil empfiehlt sich [41] als das primäre Nachschlagewerk auf Englisch. Funktionen, Arrays und Konstanten sowie die Verarbeitung von Formularfeldern sind hier sehr gut beschrieben. Wichtig ist, die aktuelle Auflage zu verwenden, da die Vorgänger die mittlerweile nicht mehr gebräuchliche POSIX Regular Expression-Syntax beinhalten.

PHP-Grundlagen werden ebenfalls von [44] gut erläutert, leider wurde auf Regular Expressions in diesem Buch vollkommen verzichtet. [49] hingegen widmet den regulären Ausdrücken ein ganzes Kapitel und bietet natürlich auch detaillierte Erklärungen zu den Grundlagen.

Wer sich intensiv den regulären Ausdrücken widmen möchte und ein umfangreiches Nachschlagewerk sucht, dem sei mit [10] *das* Werk zu dem Thema ans Herz gelegt. Für schnelles Nachschlagen und praktische Beispiele eignen sich passend dazu [12, 39].

Wer Lösungen zu fortgeschritteneren Problemen im Zusammenhang mit Arrays, Funktionen, Variablen oder auch Formularen sucht, der ist mit [27] gut bedient.

Vorlesung 4

Objektorientiertes PHP

4.1 Modularisierung

Modularisierung ist ein bewährtes Konzept in der Softwareentwicklung. Teile des Programms, die wiederverwendet werden können oder sollen, werden in eigene Teile ausgelagert. Dies kann eine Funktion sein, die ein kleines Stück Code enthält und mehrfach aufgerufen wird, eine ganze Gruppe von Funktionen und/oder Variablen oder auch ganze Klassen (siehe Abschnitt 4.2). Diese lassen sich in PHP in eigene Dateien auslagern und bequem in verschiedenen anderen Dateien wieder inkludieren. Dies verbessert nicht nur die Struktur, sondern erhöht auch die Übersicht, da sich z. B. globale Konfigurationsvariablen schneller finden lassen.

4.1.1 Dateien inkludieren

PHP bietet vier Sprachkonstrukte an, um den Inhalt von externen Dateien in einer anderen Datei zu inkludieren:

- `include`,
- `require`,
- `include_once` und
- `require_once`.

Die Inhalte der eingebundenen Dateien werden wie PHP-Dateien angesehen, d. h. PHP-Code muss (wie in anderen PHP-Dateien auch) in den PHP-Klammern (`<?php ?>`) stehen. Sind in den inkludierten Dateien Variablen oder Funktionen vorhanden, so ist deren Gültigkeitsbereich derselbe, als ob sie direkt an der Datei an dieser Stelle stünden. Variablen, die vor einer inkludierten Datei existierten, sind in dieser dann auch sichtbar.

Für *inkludierte Dateien*, die nur prozeduralen Code (Funktionen, Variablen, Konstanten) enthalten, kommt oft eine Namenskonvention mit der Dateiendung `.inc.php` (z. B. `functions.inc.php`) zur Anwendung. Dateien, die eine Klasse enthalten, werden in der Regel mit dem Namen der Klasse in PascalCase und der regulären PHP-Endung benannt (z. B. `MyClass.php`). Es kann aber jede Datei mit jeder beliebigen Endung eingebunden werden.

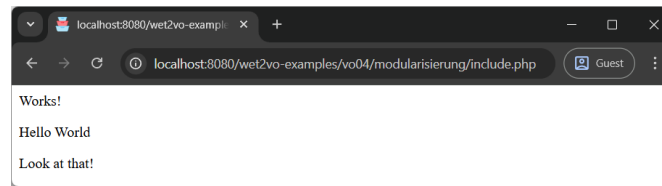


Abbildung 4.1: Obwohl `include.php` aufgerufen wurde, werden die Inhalte der anderen beiden Dateien angezeigt.

include

Mit dem Sprachkonstrukt `include "filename"`¹ wird der Inhalt einer externen Datei an der Stelle des Aufrufs inkludiert, d. h. eingefügt. Als Argument kann entweder ein relativer Pfad (z. B. `include "functions.inc.php"`) oder ebenso ein absoluter (z. B. `include "/var/www/html/functions.inc.php"`) angegeben werden. Es empfiehlt sich jedoch immer, den relativen Pfad zu verwenden. Dieser bezieht sich immer auf die aktuelle Datei. Wird dort nichts gefunden, werden die Verzeichnisse, die bei der Direktive `include_path` in der Datei `php.ini` angegeben sind, durchsucht. Wird dort auch nichts gefunden, wird eine Warnung ausgegeben, das aktuelle PHP-Skript wird jedoch weiter abgearbeitet.

Ein Beispiel. Gegeben sei die Include-Datei `vars.inc.php` mit folgendem Inhalt:

```
1 <?php
2 $test = "<p>Works!</p>";
3 const MESSAGE = "<p>Hello World</p>";
```

Diese soll in der Datei `include.php` eingebunden und ihr Inhalt verwendet werden. Dies ist ganz einfach möglich:

```
1 <?php
2 include "vars.inc.php";
3 echo $test;
4 echo MESSAGE;
```

Der Inhalt der Variablen und Konstanten wird genauso übernommen, als ob sie direkt in der Datei stünden.

Steht eine `return`-Anweisung am Ende einer inkludierten Datei, kann dieser Wert auch direkt weiterverwendet werden. Gegeben sei der Inhalt der Datei `return.inc.php`:

```
1 <?php
2 return "<p>Look at that!</p>";
```

Der dort zurück gegebene Wert kann wie folgt z. B. in `include.php` ausgegeben werden:

```
1 echo include "return.inc.php"; // <p>Look at that!</p>
```

Abbildung 4.1 zeigt den Aufruf von `include.php` im Browser.

¹<https://www.php.net/manual/de/function.include.php>

require

`require`² funktioniert gleich wie `include`. Wird jedoch die einzubindende Datei nicht gefunden, resultiert `require` in einem Fehler (anstatt einer Warnung) und das Skript wird abgebrochen.

include_once

Das Sprachkonstrukt `include_once`³ funktioniert auch wie `include`, stellt jedoch sicher, dass einzubindende Files nur ein Mal inkludiert werden. Dies ist dann sinnvoll, wenn mehrere Dateien ein und dieselbe Include-Datei einbinden und ein mehrfaches Inkludieren problematisch sein könnte.

require_once()

`require_once`⁴ ist wiederum das Pendant zu `include_once`. Der Unterschied ist hier derselbe wie zwischen `include` und `require`: Wird die einzubindende Datei nicht gefunden, bricht `require_once` mit einem Fehler ab.

4.2 Objektorientierung in PHP

Seit Version 5.0 unterstützt PHP ein ausgedehntes Objektmodell. Objektorientierung selbst war bereits in PHP 4 möglich, jedoch noch sehr eingeschränkt. Das Objektmodell ist dabei sehr stark an das von Java angelehnt.

4.2.1 Allgemeines und Terminologie

Objektorientierung ist ein fundamentales Prinzip in der Programmierung. Es impliziert eine Verbindung zwischen Daten und dem Code, der diese Daten verarbeitet. In einer rein funktionalen Sprache (wie PHP in dieser Vorlesung bis jetzt verwendet wurde) existieren die Daten lediglich in Datenstrukturen (z.B. Arrays). Dazu gibt es Funktionen, die mit diesen Daten arbeiten. In einer objektorientierten Sprache existieren Objekte. Diese speichern einerseits die Daten und beinhalten gleichzeitig Methoden, die diese Daten verarbeiten können. Dieses Konzept ermöglicht ein besseres Softwaredesign, leichtere Wartbarkeit und eine bessere Wiederverwendbarkeit des Codes. Zunächst sollen überblicksartig die wichtigsten Begriffe kurz erläutert werden:

Klasse

Als *Klasse* werden im objektorientierten Modell die Baupläne für Objekte bezeichnet. Eine Klasse definiert, welche Daten gespeichert werden (*Eigenschaften*) und welche Operationen auf diese Daten durchgeführt werden können (*Methoden*).

²<https://www.php.net/manual/de/function.require.php>

³<https://www.php.net/manual/de/function.include-once.php>

⁴<https://www.php.net/manual/de/function.require-once.php>

Objekt

Ein *Objekt* ist eine konkrete Instanz einer Klasse. Von einer Klasse kann es beliebig viele Objekte geben. Während die Klasse vorgibt, welche Daten gespeichert werden können, enthält jedes Objekt sein eigenes Set an solchen Daten.

Die Daten selbst werden *Eigenschaften* (engl. „properties“) oder auch Member-, Instanz- oder Exemplarvariablen genannt.

Die Funktionen, die den Zugriff auf die Daten des Objekts ermöglichen, werden im objektorientierten Kontext *Methoden* genannt.

Interface

Als *Interface* wird ein Set an vorgegebenen Methoden und Eigenschaften genannt, an das sich eine Klasse halten kann. Dies erleichtert die Kommunikation zwischen einzelnen Klassen, da eine zweite Klasse sich darauf verlassen kann, dass bestimmte Methoden vorhanden sind, wenn ein Interface implementiert ist.

Vererbung und abstrakte Klasse

Das Konzept der *Vererbung* ermöglicht es, dass Klassen von anderen abgeleitet werden. Sie übernehmen dabei die Eigenschaften und Methoden der sogenannten Eltern- oder Super-Klasse und können neue hinzu definieren. Soll verhindert werden, dass die Eltern-Klasse nicht instanziiert wird, kann diese als *abstrakte Klasse* definiert werden. Die abgeleitete Klasse bleibt davon unbetroffen.

Trait

Als *Trait* wird eine wiederverwendbare Sammlung an Eigenschaften und Methoden bezeichnet, die jedoch keine vollständige Klassen ist. Traits werden in mehreren unterschiedlichen Klassen eingebunden, wenn diese dieselben Funktionalitäten benötigen, eine Vererbung aber aufgrund einer zu großen Verschiedenheit in der Natur der Klassen nicht sinnvoll ist.

Enums

Enums, im Deutschen auch Aufzählungen genannt, sind spezielle Arten von Objekten. Mit ihnen wird ein eigener Datentyp spezifiziert, der nur eine bestimmte Menge an Werten annehmen kann.

4.2.2 Objekte erzeugen

PHP-Objekte werden – wie in anderen Sprachen auch üblich – mit dem Schlüsselwort `new` erzeugt. Die Syntax lautet:

```
1 $object = new Class();
```

Die Klammern sind dabei optional, sodass auch `$object = new Class;` geschrieben werden könnte.

Manche Klassen erlauben es, beim Anlegen eines Objekts Argumente mitzugeben. Diese werden in Klammern angegeben:


```
1 $object = new Class($arg1, $arg2);
```

Das folgende Beispiel legt zwei Objekte der Klasse `Rectangle` an. Einmal ohne Parameter, einmal mit fünf Parametern:

```
1 $rect1 = new Rectangle();  
2 $rect2 = new Rectangle(45, 60, 110, 120, $green);
```

4.2.3 Zugriff auf Objekteigenschaften und -methoden

Sobald ein Objekt angelegt wurde, kann auf seine Eigenschaften und Methoden zugegriffen werden. Für den Zugriff wird der Objekt-Operator `->` verwendet:

```
1 $object->property;  
2 $object->method();
```

Beim Zugriff ist darauf zu achten, dass das `$`-Zeichen nur vor die Objektvariable gesetzt wird und nicht mehr vor die Member-Variable. Ebenfalls zu beachten ist der Zugriffsmodifikator. Nur auf Eigenschaften und Methoden, die `public` sind, kann auf diese Art zugegriffen werden (siehe dazu auch Abschnitt 4.2.4).

Im folgenden Beispiel werden die Eigenschaften eines der zuvor angelegten Rechtecke geändert und eine Methode aufgerufen:

```
1 $rect1->x1 = 5;  
2 $rect1->y1 = 5;  
3 $rect1->x2 = 10;  
4 $rect1->y2 = 10;  
5 $rect1->color = $red;  
6 $rect1->draw();
```

4.2.4 Klassen deklarieren

Klassen stellen die Baupläne von Objekten dar. Sie definieren, welche Daten gespeichert werden und welche Methoden vorhanden sind. Um eine Klasse zu definieren, wird das Schlüsselwort `class` benötigt. Danach folgt der Name der Klasse. Bei diesem spielt Groß- und Kleinschreibung keine Rolle, üblicherweise werden Klassen jedoch im *PascalCase* (auch *StudlyCaps* genannt), d. h. mit großem Anfangsbuchstaben und Großbuchstaben für weitere Wörter, verfasst.

Eine Klasse wird üblicherweise in einer eigenen Datei gespeichert (die auch den Namen der Klasse trägt), es ist aber auch genauso möglich, mehrere Klassen in einer Datei zu definieren.

Nach dem Namen der Klasse folgen geschwungene Klammern, innerhalb derer alle Eigenschaften und Methoden angegeben werden. Die Syntax dazu sieht wie folgt aus:

```
1 class ClassName  
2 {  
3     private $property;  
4  
5     public function method()  
6     {  
7         // ...  
8     }  
9 }
```

Eigenschaften

Eigenschaften oder Exemplarvariablen einer Klasse geben an, welche Daten später von einem Objekt gespeichert werden können. Sie sind optional, d. h. eine Klasse muss nicht zwingend Eigenschaften aufweisen, in der Regel sind aber solche vorhanden. Eigenschaften werden meist am Beginn einer Klasse deklariert und in *camelCase* geschrieben.

Zugriffsmodifikatoren: PHP ermöglicht es, analog zu Java und anderen Sprachen, die Sichtbarkeit von deklarierten Eigenschaften anzugeben. Dafür stehen drei (bekannte) Schlüsselwörter zur Verfügung:

- **public:** Die Eigenschaft kann von überall her (außerhalb des Objekts, von anderen Klassen etc.) verwendet werden.
- **protected:** Auf die Eigenschaft kann nur von Objekten der eigenen Klasse und ggf. davon abgeleiteten Klassen verwendet werden.
- **private:** Die Eigenschaft kann nur von Methoden der eigenen Klasse verändert werden.

Die Standardsichtbarkeit ist **public**. Generell sollte der Zugriffsschutz immer sehr streng gewählt werden, um das Prinzip der Datenkapselung aufrecht zu erhalten. Der Zugriff kann dann geordnet über Methoden oder Property Hooks (s.u.) erfolgen.

Das folgende Beispiel deklariert die Eigenschaften der Klasse **Rectangle**. Dies sind zwei (x/y) -Koordinatenpaare für die Punkte links oben und rechts unten und eine Farbe. Um den Zugriff wie in Abschnitt 4.2.3 zu gewährleisten, müssen in diesem Fall alle Eigenschaften als **public** deklariert werden:

```
1 class Rectangle
2 {
3     public $x1;
4     public $y1;
5     public $x2;
6     public $y2;
7     public $color;
8
9     // ...
10 }
```

Typed Properties: Wie auch bei Funktionen können Typdeklarationen bei Eigenschaften einer Klasse verwendet werden. Diese werden zwischen dem Zugriffsmodifikator und dem Namen der Eigenschaft angegeben. Um für die Klasse **Rectangle** nur Ganzzahlen als Koordinaten sowie einen String für die Farbe zuzulassen, sieht dies wie folgt aus:

```
1 class Rectangle
2 {
3     public int $x1;
4     public int $y1;
5     public int $x2;
6     public int $y2;
7     public string $color;
8
9     // ...
10 }
```

Wie bei Funktionen (Methoden) und Rückgabewerten erzeugt die Typisierung hier einen Fehler, wenn andere Datentypen als die angegebenen zugewiesen werden.

Asymmetrische Sichtbarkeit: Seit PHP 8.4 ist es möglich, die Zugriffsmodifikatoren asymmetrisch zu gestalten. Dies ist vor allem dann sinnvoll, wenn der setzende/schreibende Kontext strenger als der lesende sein soll. So könnte etwa für die Eigenschaft `$color` festgelegt werden, dass diese zwar grundsätzlich `public` ist, der schreibende Zugriff aber `private` sein soll. Dies bedeutet, dass die Eigenschaft zwar öffentlich gelesen, aber nur intern in der Klasse gesetzt/geschrieben werden kann. Dazu wird eine zweite Sichtbarkeit mit dem Modifikator `set` für schreibend (oder auch `get`) für lesend hinzugefügt. In der Klasse `Rectangle` sieht dies dann so aus.

```
1 class Rectangle
2 {
3     public int $x1;
4     public int $y1;
5     public int $x2;
6     public int $y2;
7     public private(set) string $color;
8
9     // ...
10 }
```

Die `set`-Sichtbarkeit muss dabei gleich oder restriktiver als die Standardsichtbarkeit (das erste Schlüsselwort) sein, außerdem funktioniert asymmetrische Sichtbarkeit nur mit typed Properties. Ist die Standardsichtbarkeit `public`, so kann diese auch weggelassen werden. In diesem Fall wäre also `private(set) string $color` ausreichend.

Methoden

Eine *Methode* ist eine Funktion innerhalb einer Klasse. Sie wird genau gleich mit dem Schlüsselwort `function` definiert, jedoch wird in der Regel auch ein Zugriffsmodifikator (`public`, `protected` oder `private`) vorgestellt, um bestimmen zu können, wann die Methoden aufgerufen werden darf.

Zugriff auf Eigenschaften und Methoden: Um auf die Eigenschaften und Methoden des aktuellen Objekts zugreifen zu können, muss das Schlüsselwort `$this` benutzt werden. Es referenziert das aktuelle Objekt. Um also auf die Eigenschaft `$x1` eines `Rectangle` Objekts in einer Methode des Objekts zugreifen zu können, muss dieser Zugriff immer über `$this->x1` erfolgen. Würde nur `$x1` verwendet, so wird eine lokale Variable angelegt bzw. verwendet werden.

Im folgenden Beispiel wird eine Methode `getX1()` implementiert, die den x -Wert des linken oberen Punktes im Rechteck zurückgibt. Die Methode wird als `public` deklariert, um den Zugriff von außerhalb zu ermöglichen. Weiters wird eine Funktion `move(int $dx, int $dy)` angelegt, die alle Koordinaten um die Werte `$dx` und `$dy` verschiebt. Diese beiden Werte sind die (als `int` typisierten) Parameter der Methode. Schließlich soll es noch eine Funktion `draw()` geben, die das Rechteck zeichnet. Ihr Inhalt würde Funktionen einer Grafikbibliothek aufrufen, um das Rechteck darstellen zu können.

```
1 class Rectangle
2 {
3     public int $x1;
4     public int $y1;
5     public int $x2;
6     public int $y2;
7     public private(set) string $color;
8
9     public function move(int $dx, int $dy): void
10    {
11        $this->x1 += $dx;
12        $this->y1 += $dy;
13        $this->x2 += $dx;
14        $this->y2 += $dy;
15    }
16
17    public function getX1(): int
18    {
19        return $this->x1;
20    }
21
22    public function draw(): void
23    {
24        // Zeichnen des Rechtecks
25    }
26 }
```

Property Hooks: In PHP 8.4 wurden so genannte Property Hooks eingeführt, welche Getter- und Setter-Methoden (wie `getX1()`) überflüssig machen. Dabei wird die Definition der Eigenschaft nicht mit einem Strichpunkt sondern mit geschwungenen Klammern (`{ }`) beendet. In den Klammern finden sich die Schlüsselwörter `get` oder `set` die wiederum in geschwungenen Klammern den Getter- bzw. Setter-Code beinhalten. Für die Eigenschaft `$x1` sieht dies in der Klasse wie folgt aus:

```
1 class Rectangle
2 {
3     public int $x1 {
4         get {
5             return $this->x1;
6         }
7     }
8
9     // ...
10 }
```

Der Zugriff erfolgt dann direkt über die Eigenschaft: `$rect1->x1` anstatt über `$rect1->getX1()`. Hierbei verbindet sich die Einfachheit des Zugriffs auf eine **public** Eigenschaft mit der Sicherheit einer Getter-Methode, denn letztere stellt sicher, dass ggf. noch Code ausgeführt wird, bevor der Wert zurückgegeben wird, was mit einem direkten Zugriff nicht möglich ist. Geschieht jedoch keine weitere Logik, als den Wert der Eigenschaft zurückzugeben, ist **public**-Sichtbarkeit völlig ausreichend.

Konstruktor

Beim Erzeugen eines Objekts mit `new` wird dessen *Konstruktor* aufgerufen. Mit diesem können alle wichtigen Initialisierungen vorgenommen werden. In PHP ist der Konstruktor jedoch etwas anders definiert, als es etwa in Java der Fall ist. Es handelt sich um eine sogenannte *magische Methode* (engl. „magic method“). Derlei Methoden beginnen immer mit zwei Unterstrichen (`__`), darum sollten keine eigenen Methodennamen benützt werden, die so beginnen, da sie sonst von PHP als magisch angesehen werden.

Die magische Methode für den Konstruktor lautet `__construct()`. Die Anzahl der Parameter kann dabei beliebig gewählt werden.

Im Konstruktor der Klasse `Rectangle` werden alle vier Koordinaten und die Farbe mit Standardwerten initialisiert, Typdeklarationen sichern die Initialisierung zusätzlich ab:

```
1 class Rectangle
2 {
3     public int $x1 {
4         get {
5             return $this->x1;
6         }
7     }
8     public int $y1;
9     public int $x2;
10    public int $y2;
11    public private(set) string $color;
12
13    public function __construct(
14        int $x1 = 0,
15        int $y1 = 0,
16        int $x2 = 0,
17        int $y2 = 0,
18        string $color = "white"
19    ) {
20        $this->x1 = $x1;
21        $this->y1 = $y1;
22        $this->x2 = $x2;
23        $this->y2 = $y2;
24        $this->color = $color;
25    }
26
27    // ...
28 }
```

PHP erlaubt es nicht, Methoden zu überladen, d. h. zwei Methoden mit demselben Namen aber unterschiedlichen Parametern anzugeben. Diesem Umstand kann wie im Beispiel gezeigt entgegnet werden. Durch die Verwendung von Default-Parametern ist es möglich, den Konstruktor mit einer Anzahl zwischen null und allen fünf Parametern aufzurufen und dabei immer noch eine korrekte Initialisierung zu gewährleisten. In Abschnitt 4.2.2 wurde genau dies demonstriert.

Constructor Property Promotion: Beachtet man den Konstruktor der Klasse `Rectangle` genau, so fällt auf, dass die Initialisierung von Eigenschaften meist derselbe Prozess ist: Eigenschaft deklarieren, Parameter deklarieren, das Argument der Eigenschaft zuweisen.

Dabei wird jede Eigenschaft (etwa `$x1`) viermal angegeben.

Mit PHP 8 wurde versucht, diesen Umstand zu lösen. Mit Hilfe der Constructor Property Promotion wird die Initialisierung drastisch verkürzt. Dazu wird einfach der Zugriffsmodifikator der Eigenschaft (hier überall `public`) vor den jeweiligen Parameter im Konstruktor gestellt. Der Inhalt des Konstruktors sowie die Deklaration der Eigenschaften kann entfallen. Etwaige Property Hooks wie bei `$x1` werden direkt in den Parametern inkludiert. Der Konstruktor der Klasse `Rectangle` sieht dadurch wie folgt aus:

```
1 class Rectangle
2 {
3     public function __construct(
4         public int $x1 = 0 {
5             get {
6                 return $this->x1;
7             }
8         },
9         public int $y1 = 0,
10        public int $x2 = 0,
11        public int $y2 = 0,
12        public string $color = "white"
13    ) {}
14
15    // ...
16 }
```

Destruktor

Seit PHP5 gibt es auch *Destruktoren* in PHP. Diese erfüllen den gegenteiligen Zweck eines Konstruktors, werden also bei der Zerstörung eines Objekts aufgerufen. Dies geschieht, wenn die letzte Referenz auf ein Objekt entfernt wird (z. B. dadurch, dass die einzige Variable, die auf das Objekt zeigt, auf `null` gesetzt wird) oder ganz einfach das Ende des Skripts erreicht wird. PHP räumt alle seine Ressourcen automatisch auf, somit ist das Einsatzgebiet von Destruktoren sehr eingeschränkt. Sie können jedoch z. B. zum Schließen von Datenbankverbindungen oder einfach zum Debugging verwendet werden (um loggen zu können, wann ein Objekt zerstört wurde).

Ein Destruktor ist ebenfalls eine magische Methode und wird mit dem Namen `__destruct()` deklariert. Im Destruktor der Klasse `Rectangle` wird eine simple Meldung ausgegeben, wenn das Objekt zerstört wird.

```
1 class Rectangle
2 {
3     // ...
4
5     public function __destruct()
6     {
7         echo "Destructor called!";
8     }
9
10    // ...
11 }
```

Statische Elemente

Zusätzlich zu normalen Eigenschaften eines Objekts können auch *statische Elemente* definiert werden. Während Eigenschaften (Member-Variablen) für jedes Objekt einzeln existieren und für jede Instanz einen eigenständigen Wert haben, existiert eine statische Variable genau ein Mal pro *Klasse*. Ihr Wert ist also über alle Objekte der Klasse hinweg gleich.

Statische Variablen werden mit Hilfe des Schlüsselworts **static** angelegt. Auch für sie gelten dieselben Zugriffsmodifikatoren wie für Eigenschaften oder Methoden. Der Zugriff auf eine statische Variable, erfolgt jedoch immer über die Angabe der Klasse, *nicht* über das Objekt. Handelt es sich um eine statische Variable, die als **public** deklariert wurde, kann der Zugriff von außerhalb über die folgende Syntax erfolgen:

```
1 ClassName::$staticVar;
```

Soll innerhalb eines Objekts auf die statische Variable zugegriffen werden, so geschieht dies ebenfalls über eine derartige Syntax. Nur kann der Name der aktuellen Klasse durch das Schlüsselwort **self** ersetzt werden (da im Objekt ja der Name der Klasse bekannt ist). Die zwei Doppelpunkte werden Gültigkeitsbereichsoperator (engl. „scope resolution operator“), oder offiziell Paamayim Nekudotayim (aus dem Hebräischen für „zweimal Doppelpunkt“) genannt.

Im **Rectangle**-Beispiel wird eine statische Variable **\$version** angelegt, welche die Versionsnummer der Klasse enthält. Diese Nummer wird einmal außerhalb des Objekts und einmal über die Methode **getVersion()** innerhalb angefragt und nach außen zurück gegeben. In der letzten Zeile wird versucht, auf die statische Variable wie auf eine normale Eigenschaft zuzugreifen. Dies führt zu einer Notice über den falschen Zugriff auf ein statische Property und zu einem Warning, dass die Eigenschaft, auf die Zugriffen wird, nicht existiert.

```
1 class Rectangle
2 {
3     // ...
4     public static string $version = "1.0.0";
5
6     // ...
7
8     public function getVersion(): string
9     {
10         return self::$version;
11     }
12
13     // ...
14 }
15
16 $rect1 = new Rectangle();
17 echo Rectangle::$version; // 1.0.0
18 echo $rect1->getVersion(); // 1.0.0
19 echo $rect1->version; // NOTICE & WARNING!
```

Statische Variablen können auf dieselbe Art und Weise verändert werden. Über **Rectangle::\$version = "2.0.0"**; kann der Wert jederzeit neu gesetzt werden.

Ebenso können Methoden statisch deklariert werden. Dies bewirkt ebenfalls, dass sie zur Klasse und nicht zum Objekt gehören und somit ohne konkrete Instanz aufge-

rufen werden. Im Unterschied zu statischen Eigenschaften kann jedoch eine statische Methode auch über eine Instanz aufgerufen werden, wenngleich dies nicht sinnvoll ist und vermieden werden sollte.

Das folgende Beispiel definiert eine statische Methode, die die statische Variable `$version` mit `echo` ausgibt. Der Aufruf wird zunächst statisch, d. h. über den Klassennamen und dann am Objekt durchgeführt.

```
1 class Rectangle
2 {
3     // ...
4     public static string $version = "1.0.0";
5
6     // ...
7
8     public static function printVersion(): void
9     {
10         echo self::$version;
11     }
12
13     // ...
14 }
15
16 $rect1 = new Rectangle();
17 Rectangle::printVersion(); // 1.0.0
18 $rect1->printVersion(); // Funktioniert, aber nicht nötig/empfehlenswert
```

Konstanten

Genauso wie bei globalen Konstanten können *Klassen-Konstanten* mit dem Schlüsselwort `const` angelegt werden, seit PHP 8.3 ist auch eine Typdeklaration möglich. Ihr Wert kann nicht mehr geändert werden, sobald eine Konstante einmal angelegt wurde. Wie auch bei globalen Konstanten hat es sich eingebürgert, Großschreibung zu verwenden.

Konstanten können seit PHP 7.1 mit einem vorangestellten Zugriffsmodifikator wie `public`, `protected` oder `private` verwendet werden. In früheren Versionen bzw. ohne Modifikator sind sie automatisch `public`.

Für die Klasse `Rectangle` wird eine Konstante `TYPE` eingeführt, die den Typ des geometrischen Objekts („Rectangle“) zurückgibt. Der Zugriff erfolgt wiederum von außerhalb, über eine Methode und schließlich über das Objekt. Zugriff Nummer drei schlägt erneut fehl, da auch Konstanten in der Klasse und nicht beim Objekt gespeichert sind.

```
1 class Rectangle
2 {
3     // ...
4
5     public const string TYPE = "Rectangle";
6
7     // ...
8
9     public function getType(): string
10    {
11        return self::TYPE;
12    }
```



```
13
14 // ...
15 }
16
17 $rect1 = new Rectangle();
18 echo Rectangle::TYPE; // Rectangle
19 echo $rect1->getType(); // Rectangle
20 echo $rect1->TYPE; // FEHLER!
```

Schreibgeschützte Eigenschaften

Mit PHP 8.1 wurden *schreibgeschützte Eigenschaften* (engl. „readonly properties“) eingeführt. Dabei handelt es sich um Eigenschaften der Klasse, denen nur einmal ein Wert zugewiesen werden kann. Dies mag auf den ersten Blick wie eine Konstante wirken – für diese gilt prinzipiell dasselbe – Konstanten sind jedoch Klassenkonstanten, d. h. sie existieren bei der Klasse und nicht beim Objekt. Eine Konstante gilt also für alle Objekte einer Klasse und hat dort denselben Wert. Nur lesbare Eigenschaften existieren im Kontext des Objekts und können für jedes Objekt einen unterschiedlichen Wert haben.

Um eine nur lesende Eigenschaft zu definieren, wird das Schlüsselwort **readonly** zwischen Zugriffsmodifikator und Typhinweis gestellt. Letzteres ist zwingend nötig, denn nur schreibgeschützte Eigenschaften funktionieren nicht ohne die Angabe des Typs.

Im folgenden Beispiel wird eine nur lesbare Eigenschaft `$id` eingeführt. Ihr wird im Konstruktor ein Argument zugewiesen. Es ist keine Default-Wert angegeben, da dies nicht zielführend ist (man könnte ihn nicht mehr ändern) und es kommt wieder Constructor Property Promotion zum Einsatz, um den Zuweisungsprozess im Konstruktor zu verkürzen. Im Anschluss wird ein Objekt mit der verpflichtenden ID angelegt, für alle anderen Parameter werden die Standardwerte angenommen. Beim Versuch, die ID durch eine Zuweisung nochmals zu ändern, wird ein Fehler angezeigt.

```
1 class Rectangle
2 {
3     public function __construct(
4         public readonly int $id,
5         public int $x1 = 0,
6         public int $y1 = 0,
7         public int $x2 = 0,
8         public int $y2 = 0,
9         public string $color = "white"
10    ) {
11    }
12
13    // ...
14 }
15
16 $rect1 = new Rectangle(1);
17 $rect1->id = 3; // FEHLER!
```

Schreibgeschützte Klassen: In PHP 8.2. wurden zudem *readonly Klassen* eingeführt. Hätte eine Klasse nur Eigenschaften mit **readonly** Eigenschaften, so kann nun einfach die gesamte Klasse derart markiert werden. Das Schlüsselwort entfällt dann bei den einzelnen Eigenschaften.

Aus folgendem Beispiel zu einer hypothetischen `Post`-Klasse (etwa für einen Blog), bei dem die Daten über ein Posting einmal festgelegt und dann nicht mehr veränderbar sind...

```
1 class Post
2 {
3     public function __construct(
4         public readonly string $title,
5         public readonly string $author,
6         public readonly string $content
7     ) {
8     }
9 }
```

wird dann dieses:

```
1 readonly class Post
2 {
3     public function __construct(
4         public string $title,
5         public string $author,
6         public string $content
7     ) {
8     }
9 }
```

4.2.5 Interfaces

Ein *Interface* (zu deutsch „Schnittstelle“) stellt eine Art Vertrag da, den eine Klasse eingeht. Im Interface selbst sind Methodenprototypen und ggf. Property Hooks und Konstanten angegeben, die eine Klasse konkret implementieren muss, wenn sie dieses Interface einsetzt. Dies ist wertvoll, da andere Klassen sich darauf verlassen können, dass bestimmte Methoden vorhanden sind, wenn ein gewisses Interface implementiert wird. Ob eine Klasse ein Interface implementiert, lässt sich wie bei Vererbung (siehe Abschnitt 4.2.6) durch den `instanceof` Operator prüfen.

Interfaces werden ähnlich wie Klassen definiert. Als Schlüsselwort dient hierbei jedoch `interface`. Zudem werden Methoden in Interfaces nur angeführt und mit einem Strichpunkt abgeschlossen. Die konkrete Implementierung in geschweiften Klammern entfällt. Dies muss dann in der Klasse geschehen, die das Interface implementiert.

Um bei einer Klasse anzudeuten, dass sie ein Interface implementiert (also den Vorgaben daraus folgt), wird neben den Klassennamen das Schlüsselwort `implements`, gefolgt vom Namen des Interface, gestellt.

Umgelegt auf das `Rectangle`-Beispiel, wird ein neues Interface mit dem Namen `GeometricComponentInterface` erstellt, das allgemein jegliche geometrische Komponente repräsentieren soll. Das Interface gibt die Methoden `move(int $dx, int $dy)` (zum verschieben der Komponente) und `draw()` (zum Zeichnen der Komponente) vor.

```
1 interface GeometricComponentInterface
2 {
3     public function move(int $dx, int $dy): void;
4     public function draw(): void;
5 }
```

Die Klasse `Rectangle` implementiert nun dieses Interface mit seinen zwei vorgeschriebenen Methoden.

```
1 require "GeometricComponentInterface.php";
2
3 class Rectangle implements GeometricComponentInterface
4 {
5     // ...
6
7     public function move(int $dx, int $dy): void
8     {
9         $this->x1 += $dx;
10        $this->y1 += $dy;
11        $this->x2 += $dx;
12        $this->y2 += $dy;
13    }
14
15    // ...
16
17    public function draw(): void
18    {
19        // ...
20    }
21 }
```

Wichtig ist hierbei, dass die Methoden genau dieselben Parameter aufweisen. Voraussetzung ist, dass die Datei mit dem definierten Interface zunächst eingebunden wird, damit die Informationen darüber vorhanden sind.

Gibt es neben der Klasse `Rectangle` noch andere, die dieses Interface implementieren (z. B. `Circle` oder `Triangle`), so kann bei all diesen Klassen davon ausgegangen werden, dass sie `draw()` und `move(int $dx, int $dy)` implementieren. Es kann also etwa mit `instanceof` überprüft werden, ob ein Objekt dieses Interface implementiert und dann gefahrlos `draw()` aufgerufen werden. Gäbe es das Interface nicht, müsste überprüft werden, ob das Objekt entweder vom Typ `Rectangle` oder vom Typ `Circle` oder vom Typ `Triangle` ist, bevor die Methode zum Zeichnen aufgerufen wird.

Attribute

Attribute (englisch „Attributes“, seit PHP 8.0 verfügbar) sind Metadaten, die an Klassen, Methoden, Properties oder Parameter geheftet werden können. Sie verändern den Code nicht direkt, sondern liefern Zusatzinformationen, die entweder von der PHP-Engine selbst, von Frameworks oder von Werkzeugen wie statischen Analysetools und IDEs ausgewertet werden. Syntaktisch werden Attribute in eckigen Klammern mit vorangestelltem Hash-Zeichen notiert und der zu annotierenden Code-Stelle vorangestellt.

Ein besonders nützliches Attribut ist `#[Override]` (verfügbar seit PHP 8.3). Es wird einer Methode vorangestellt und teilt der PHP-Engine mit, dass diese Methode eine Methode einem Interface, einer Elternklasse (siehe Abschnitt 4.2.6), oder einem Trait (siehe Abschnitt 4.2.8) überschreibt bzw. implementiert. Existiert keine entsprechende Methode im Original, wirft PHP einen Fehler. Dies schützt insbesondere vor Tippfehlern im Methodennamen sowie vor Fehlern, die entstehen, wenn sich die ursprüngliche Methode nachträglich ändert (z. B. durch Umbenennung im Interface).

Im Beispiel der Klasse `Rectangle` aus dem vorigen Abschnitt lässt sich das Attribut auf die beiden vom Interface vorgegebenen Methoden anwenden:

```

1 class Rectangle implements GeometricComponentInterface
2 {
3     // ...
4
5     #[Override]
6     public function move(int $dx, int $dy): void
7     {
8         $this->x1 += $dx;
9         $this->y1 += $dy;
10        $this->x2 += $dx;
11        $this->y2 += $dy;
12    }
13
14    #[Override]
15    public function draw(): void
16    {
17        // ...
18    }
19 }
```

Würde beim Methodennamen ein Tippfehler (etwa `drwa()` statt `draw()`) passieren, so meldet PHP direkt einen Fehler, da keine entsprechende Methode im Interface existiert. Ohne das Attribut wäre der Fehler nur indirekt erkennbar, da PHP lediglich melden würde, dass `Rectangle` das Interface `GeometricComponentInterface` nicht vollständig implementiert. `#[Override]` kann also immer gesetzt werden, wenn eine Methode eine Vorgabe aus einem Interface, einer Elternklasse oder einem Trait erfüllt.

4.2.6 Vererbung

PHP kennt das Konzept der *Vererbung*. Eine Klasse kann von einer anderen erben, d. h. abgeleitet werden und übernimmt damit alle Eigenschaften und Methoden, die entweder als `public` oder `protected` deklariert sind. Um eine Klasse von einer anderen abzuleiten, wird nach dem Klassennamen das Schlüsselwort `extends` angestellt. Danach folgt der Name der Basisklasse, von der abgeleitet wird. Abgeleitete Klassen können neue Variablen und Methoden hinzufügen oder auch bestehende Methoden mit einer neuen Implementierung überschreiben.

Beim Zugriff wird bei einer überschriebenen Methode immer die Version aus der Kindklasse (d. h. der abgeleiteten Klasse) bevorzugt. Soll explizit bestimmt werden, dass die Methode der Elternklasse aufgerufen werden soll, so kann das über das Schlüsselwort `parent` und dem Gültigkeitsbereichsoperator (`::`) erfolgen. Um explizit die abgeleitete Klasse anzusprechen, kommt wieder das Schlüsselwort `self` zum Einsatz.

Im folgenden Beispiel soll eine Klasse `Box` angelegt werden, die eine dreidimensionale Repräsentation des normalen Rechtecks darstellt. Da eine Box aus sechs Rechtecken besteht, wird eine neue Variable für die Tiefe der Box eingeführt und dann die Methode `draw()` mehrmals aufgerufen. Beim ersten Mal wird die Methode der Elternklasse aufgerufen, da dies die Vorderseite, also ein normales Rechteck zeichnet. Auch hier wird das Attribut `#[Override]` angegeben, um zu kennzeichnen, dass hier eine Methode (in diesem Fall aus der Basisklasse `Rectangle`) überschrieben wird.

Damit der abgeleiteten Klasse jedoch die Informationen über die Elternklasse vorliegen, muss diese mittels `include` oder `require` eingebunden werden, falls sich die Klassendefinition nicht in derselben Datei befindet.

```
1 require "Rectangle.php";
2
3 class Box extends Rectangle
4 {
5     public int $depth;
6
7     #[Override]
8     public function draw(): void
9     {
10         // Draw the front (Rectangle)
11         parent::draw();
12         // Draw the other sides
13         // ...
14     }
15 }
```

Wichtig: Im Konstruktor einer abgeleiteten Klasse wird nicht automatisch der Konstruktor der Basisklasse aufgerufen. Ist dies gewünscht – und das ist in der Regel der Fall – so muss es explizit mit `parent::__construct()`; geschehen.

Typüberprüfungen durchführen

Soll überprüft werden, von welchem Typ ein bestimmtes Objekt ist, d. h. welche Klasse ihm zugrunde liegt, oder ob ein gewisses Interface implementiert ist, so kann der `instanceof`-Operator verwendet werden. Dabei ist zu beachten, dass ein Objekt einer abgeleiteten Klasse auch immer als ein Objekt der Basisklasse angesehen wird. Folgende Beispiele illustrieren dies:

```
1 class Human
2 {
3     // ...
4 }
5 class Employee extends Human
6 {
7     // ...
8 }
9 class Animal
10 {
11     // ...
12 }
13
14 $obj = new Employee();
15
16 if ($obj instanceof Employee) { // true
17     echo "Object is Employee";
18 }
19 if ($obj instanceof Human) { // true
20     echo "Object is Human";
21 }
22 if ($obj instanceof Animal) { // false
23     echo "Object is Animal";
24 }
```

Finale Klassen, Methoden und Konstanten

Mit dem Schlüsselwort **final** können ganze Klassen, einzelne Methoden oder seit PHP 8.1 auch Konstanten gekennzeichnet werden, um Vererbung zu verbieten bzw. einzuschränken. Werden eine Methode oder eine Konstante in der Elternklasse mit **final** gekennzeichnet, können sie in einer Kindklasse nicht überschrieben werden. Der Versuch führt zu einem Fehler. Wird die gesamte Klasse mit **final** markiert, so kann von ihr überhaupt nicht mehr abgeleitet werden.

Die Verwendung von **final** kann durchaus freizügig erfolgen. Wenn eine Vererbung nicht vorgesehen oder erwünscht ist, können dadurch ungewollte Probleme, die durch nachträgliche Codeänderungen in einer Klasse auftreten, verhindert werden.

```
1 final class NoInheritance
2 {
3     public function foo(): void
4     {
5         // ...
6     }
7 }
8
9 class CantOverrideEverything
10 {
11     public const int CAN_OVERRIDE = 1;
12     final public const int CANNOT_OVERRIDE = 2;
13
14     public function canOverride(): void
15     {
16         // ...
17     }
18
19     final public function cantOverride(): void
20     {
21         // ...
22     }
23 }
```

4.2.7 Abstrakte Klassen

Abstrakte Klassen ermöglichen es, eine Art unvollständiges Grundgerüst einer Klasse zu erstellen, das selbst nicht instanziiert werden kann. In diesem Grundgerüst sind sowohl die Klasse selbst als auch einzelne Methoden (mindestens eine) mit dem Schlüsselwort **abstract** gekennzeichnet. Dies verhindert einerseits die Instanziierung und zwingt andererseits eine Klasse, die von dieser abstrakten Klasse erbt, dass diese Methoden implementiert werden müssen. Diese Vorgehensweise ist vor allem dann sinnvoll, wenn es für eine Methode einer Elternklasse keine sinnvolle Standardimplementierung gibt, da diese sehr stark von dem jeweiligen Anwendungsfall abhängt. In diesem Fall übernehmen konkrete Kindklassen dann die jeweilige Implementierung, teilen sich aber durch die gemeinsame Elternklasse einen großen Teil des Codes, sodass dieser nicht mehrmals implementiert werden muss.

Somit unterscheiden sich abstrakte Klassen von Interfaces. Denn während auch abstrakte Klassen Methoden vorgeben, die implementiert werden müssen, können sie auch fertigen Code enthalten, der einfach vererbt wird.

Im folgenden Beispiel wird eine abstrakte Klasse erstellt, die eine abstrakte Methode und eine vollständig implementierte enthält. Die abstrakte Methode ist wie bei einem Interface mit einem Semikolon terminiert und enthält keinen konkreten Code. Durch das Schlüsselwort **abstract** bei der Methode wird angedeutet, dass diese in einer abgeleiteten Klasse implementiert werden muss. Weiters muss das Schlüsselwort **abstract** dann auch vor die Klassendefinition gesetzt werden.

```
1 abstract class AbstractClass
2 {
3     abstract protected function getValue(): string;
4
5     public function printOut(): void
6     {
7         echo $this->getValue();
8     }
9 }
```

Zu dieser Klasse gibt es eine konkrete Implementierung. Diese muss zunächst die Datei der abstrakten Klasse mit **require** einbinden und leitet danach von ihr ab. Die konkrete Klasse überschreibt nur die abstrakte Methode **getValue()**. **printOut()** wird nicht überschrieben und wird daher von der Basisklasse verwendet. Auch hier kann mit **#[Override]** gekennzeichnet werden, dass eine Methode der Basisklasse überschrieben (bzw. erst implementiert) wird.

```
1 require "AbstractClass.php";
2
3 class ConcreteClass extends AbstractClass
4 {
5     #[Override]
6     protected function getValue(): string
7     {
8         return "ConcreteClass";
9     }
10 }
```

4.2.8 Traits

PHP verwendet ein klassisches Vererbungsmodell, auch Einfachvererbung genannt. Somit kann jede Klasse nur von einer einzigen Klasse erben (d. h. abgeleitet werden). Andere Sprachen (wie etwa C++) erlauben auch Mehrfachvererbung, d. h. eine Klasse kann die Funktionalitäten von mehreren Elternklassen übernehmen.

Einfachvererbung funktioniert für den Großteil der Fälle, es gibt jedoch auch Situationen, wo sie an ihre Grenzen stößt. Etwa wenn zwei inhaltlich völlig unterschiedliche Klassen eine identische Funktionalität benötigen. Angenommen, es existieren die Klassen **Person** und **Car**, die inhaltlich sehr unterschiedlich sind. Beide jedoch sollen die Möglichkeit haben, ihre aktuelle Position (z. B. in GPS-Koordinaten) zu speichern, aktualisieren und auszugeben, also lokalisierbar zu sein (um etwa auf einer Karte darstellbar zu sein). Dafür gibt es nun mehrere Herangehensweisen:

1. Der Positionscode wird in beiden Klassen identisch implementiert.
2. Es wird ein Interface – z.B. **LocatableInterface** erstellt, das die Methoden zur Positionsbestimmung definiert.

3. Es wird eine abstrakte Elternklasse – z. B. **AbstractLocatable** – erstellt, die den Code beinhaltet und von der beide erben.
4. Es wird ein Trait – z. B. **LocatableTrait** – erstellt, der die entsprechenden Methoden definiert und implementiert und in beiden Klassen zur Anwendung kommt.

Ansatz 1 ist nicht empfehlenswert, da er Codeverdoppelung bedeutet und das DRY (Don't Repeat Yourself)-Prinzip verletzt. Die Wartbarkeit leidet ebenso, da bei einer Änderung immer beide Implementierungen geändert werden müssen.

Herangehensweise 2 ist minimal besser, da zumindest formal festgelegt wird, dass beide Klassen bestimmte Methoden definieren, mit denen das Positionsverhalten manipuliert werden kann. Allerdings muss die Implementierung immer noch in beiden Klassen erfolgen, was ebenfalls das DRY-Prinzip verletzt und schwer wartbar ist.

Ansatz 3 ist ebenfalls nicht zu empfehlen, da er eine Vererbungshierarchie zwischen zwei Klassen erzeugt, die inhaltlich nichts gemeinsam haben. Dies sollte bei Vererbung immer vermieden werden.

Die vierte Lösung, nämlich die Erstellung eines Traits, ist die beste. Ein Trait definiert sowohl die notwendigen Methoden und stellt auch gleich die Implementierung zur Verfügung. Dabei handelt es sich nicht um eine komplette Klasse, sondern um ein begrenztes Set an Funktionalität, das auch in mehrere Klassen integriert werden kann.

Traits definieren

Traits werden ähnlich wie Klassen definiert, es kommt lediglich das Schlüsselwort **trait** zum Einsatz. Ansonsten werden Eigenschaften und Methoden (inklusive Zugriffsmodifikatoren) genauso angegeben wie bei Klassen.

```
1 trait TraitName
2 {
3     private $property;
4
5     public function method()
6     {
7         // ...
8     }
9 }
```

Ein Trait zum Verwalten einer GPS-Position könnte wie folgt aussehen. Die beiden Eigenschaften **\$latitude** und **\$longitude** sind **public**, da beim Lesen und Schreiben keinerlei zusätzliche Logik erfolgt und werden mit **null** initialisiert.

```
1 trait LocatableTrait
2 {
3     public ?float $latitude = null;
4     public ?float $longitude = null;
5
6     public function getPosition(): array
7     {
8         return [$this->latitude, $this->longitude];
9     }
10
11     public function setPosition(array $pos): void
12     {
13         $this->latitude = $pos[0];
```



```
14     $this->longitude = $pos[1];
15 }
16 }
```

Traits verwenden

Um einen Trait in einer (oder mehreren Klassen) zu verwenden, wird das Schlüsselwort **use** mit dem Trait-Namen verwendet. Dadurch sind alle Methoden und Eigenschaften des Traits in der Klasse verwendbar. Zuvor muss die Datei des Traits mit **include** oder **require** eingebunden werden, um verfügbar zu sein.

Die Klasse **Person** würde den Trait wie folgt verwenden:

```
1 <?php
2
3 require "LocatableTrait.php";
4
5 class Person
6 {
7     use LocatableTrait;
8
9     // ...
10 }
11
12 $pers = new Person();
13 $pers->setPosition([48.368201, 14.514065]);
14 echo "<p>Latitude: {$pers->latitude}</p>";
15 echo "<p>Longitude: {$pers->longitude}</p>";
```

Analog funktioniert die Anwendung in weiteren Klassen wie etwa **Car**.

4.2.9 Enums (Aufzählungen)

Enums, auch Aufzählungen genannt, sind ein spezieller Typ von Klasse. Sie werden nicht dazu verwendet, einen komplexen Bauplan für ein Objekt zu definieren, sondern sind etwas reduzierter. Sie werden eingesetzt, um einen neuen Datentyp zu spezifizieren, der eine bestimmte Menge an konkreten Werten aufweist. Sie tragen zur Typsicherheit, Fehlerreduktion und Lesbarkeit bei, wann immer etwa nur bestimmte Zustände zugelassen werden sollen. Das bekannteste Beispiel für einen Enum ist der Datentyp **bool**, der die zwei Werte **true** und **false** haben kann.

Enums definieren

Ein Enum wird ähnlich wie eine Klasse, jedoch mit dem Schlüsselwort **enum** angelegt. Innerhalb der geschwungenen Klammern folgen mit **case** definierte Werten. Um etwa einen Datentyp **ExportFormat** anzulegen, der vorgibt, in welche Formate Daten exportiert werden können, könnte ein Enum wie folgt definiert werden:

```
1 enum ExportFormat
2 {
3     case XML;
4     case JSON;
5     case PDF;
6 }
```

In diesem Fall gäbe es drei mögliche Formate: XML, JSON und PDF. Etwas anderes wird nicht zugelassen. `ExportFormat` ist dabei ein eigener Datentyp, genauso wie es Klassen sind.

Enums verwenden

Dieser Enum kann nun verwendet werden, wenn eine Entscheidung zwischen diesen drei Exportformaten benötigt wird. Angenommen, es existiert die Klasse `ProductExporter`, welche eine Methode `export()` besitzt. Diese soll anhand eines Parameters entscheiden, in welches Format exportiert werden soll. Mit Hilfe des `ExportFormat`-Enums sieht dies wie folgt aus:

```
1 class ProductExporter
2 {
3     public function export(ExportFormat $format): void
4     {
5         match ($format) {
6             ExportFormat::XML => $this->exportToXML(),
7             ExportFormat::JSON => $this->exportToJSON(),
8             ExportFormat::PDF => $this->exportToPDF(),
9         };
10    }
11 }
```

`ExportFormat` ist der Datentyp des Parameters. Mit Hilfe einer `match`-Anweisung wird unterschieden, welchen Wert der Enum hat. Die Syntax entspricht hier der von Konstanten, es wird also der Name (`ExportFormat`), gefolgt vom Scope Resolution Operator (`::`) und einem der drei Werte des Enums (`XML`, `JSON`, `PDF`) angegeben. Andere Werte sind nicht möglich und führen zu einem Fehler. Danach wird etwa eine private Methode aufgerufen, die den tatsächlichen Export in das jeweilige Format durchführt.

Somit bieten Enums hier starke Typsicherheit. Würde in der Methode alternativ die Auswahl mit einem String als Parameter zugelassen (`public function export(string $format): void`), so müsste innerhalb des `match`-Ausdrucks zwangsläufig ein zusätzlicher Zweig für ungültige Eingaben überlegt werden — etwa für den Fall, dass weder `"XML"`, `"JSON"` noch `"PDF"` übergeben wird. Damit muss auch ein Fehlerfall behandelt werden: Wird der Export gar nicht durchgeführt? Soll eine Exception geworfen werden? Wie wird die Fehlermeldung angezeigt? Bei der Verwendung eines Enums als Parametertyp entfallen diese Entscheidungen, weil die fix definierten Werte sicherstellen, dass nur gültige Formate übergeben werden können – die Typsicherheit verhindert ungültige Eingaben bereits zur Übersetzungszeit.

Enum-Werte ausgeben und Backed Enums

Die Werte eines Enums können auch als String ausgegeben werden. Jeder Wert eines Enums besitzt die `name`-Eigenschaft, die den Namen als Zeichenkette ausgibt.

```
1 echo ExportFormat::XML->name; // XML
2 echo ExportFormat::JSON->name; // JSON
3 echo ExportFormat::PDF->name; // PDF
```

Im obigen Coden werden die Strings `„XML“`, `„JSON“` und `„PDF“` ausgegeben.

Sollen die einzelnen Fälle eines Enums mit Werten hinterlegt werden, kann ein so genannter *Backed Enum* verwendet werden. Möglich sind `string`- und `int`-Werte, der Datentyp wird nach dem Namen des Enums angegeben. Im folgenden wird ein Enum erzeugt, dessen drei Fälle mit Integer-Werten hinterlegt sind:

```
1 enum BackedExportFormat: int
2 {
3     case XML = 1;
4     case JSON = 2;
5     case PDF = 3;
6 }
```

Die Eigenschaft `name` ergibt nach wie vor den Bezeichner (z.B. „XML“), über die Eigenschaft `value` kann nun der Wert (z.B. „1“ für „XML“) abgefragt werden.

```
1 echo BackedExportFormat::XML->name; // XML
2 echo BackedExportFormat::XML->value; // 1
```

4.2.10 Exceptions

Zusammen mit dem objektorientierten Programmiermodell wurde in PHP 5 ein Modell zur *Ausnahmebehandlung* (engl. „exceptions“) eingeführt. Dieses Modell gleicht sehr stark dem von Java, ist jedoch bei weitem nicht so integral implementiert wie dort. So geben die meisten Funktionen aus PHP noch normale Fehlermeldungen aus und lediglich moderne Erweiterungen nützen das Exception-Modell.

Um es zu verwenden, steht die Klasse `Exception` zur Verfügung. Von ihr erzeugte Objekte repräsentieren allgemeine Fehler, die „geworfen“ und mittels eines speziellen Mechanismus behandelt werden können.

Um eine Exception zu werfen, wird das Schlüsselwort `throw` verwendet. Geworfen wird ein neues `Exception`-Objekt, dem als Parameter eine konkrete Fehlermeldung mitgegeben werden kann.

```
1 throw new Exception("General error!");
```

Jede auf diese Art ausgelöste Exception muss abgefangen und behandelt werden. Dafür steht der sogenannte `try/catch`-Block zur Verfügung. Die Exception wird im `try`-Teil ausgelöst, der `catch`-Teil spezifiziert, welcher Typ von Exception (im allgemeinsten Fall eine Ausnahme vom Typ `Exception`) abgefangen und behandelt wird. Nach der Behandlung läuft der Code normal weiter. Folgendes Beispiel zeigt einen kompletten Ablauf mit Auslösung und Behandlung des Fehlers.

```
1 try {
2     throw new Exception("General error!");
3     echo "This output isn't shown anymore.";
4 } catch (Exception $e) {
5     echo "Exception: " . $e->getMessage();
6 }
7 echo "Continue after exception handling.";
```

Sobald der Fehler mit `throw` ausgelöst wurde, stoppt die Programmausführung an dieser Stelle. Die Ausgabe mit `echo` in der Zeile darunter wird nie ausgeführt. Anstatt dessen wird zum `catch`-Block gewechselt und die Fehlerbehandlung (in diesem Fall die reine Ausgabe der Meldung) durchgeführt. Danach läuft der Code jedoch wieder weiter sodass das finale `echo` wieder angezeigt wird.

Auf einen `try`-Block können auch mehrere `catch`-Blöcke folgen, wenn verschiedene Exceptions abgefangen werden sollen. Dabei ist zu beachten, dass die spezifischeren jedoch zuerst und die allgemeinsten zuletzt kommen sollten, damit nicht die allgemeinste Exception alle spezifischeren abfängt:

```
1 try {  
2     // ...  
3 } catch (OutOfBoundsException $e) { // spezifischer  
4     echo "Exception: " . $e->getMessage();  
5 } catch (Exception $e) { // allgemeiner  
6     echo "Exception: " . $e->getMessage();  
7 }
```

Die Variable im Catch-Block (hier `$e`) kann auch weggelassen werden, wenn sie nicht verwendet wird: `catch (Exception)`.

Eigene Exceptions

Wie auch in Java können eigene Exceptions definiert werden. Dazu muss einfach von der Klasse `Exception` abgeleitet werden.

```
1 class MyException extends Exception  
2 {  
3     // ...  
4 }
```

Es empfiehlt sich jedoch, auf die vordefinierten Exceptions der Standard-PHP-Library (SPL)⁵ zurückzugreifen, in der bereits die wichtigsten Ausnahmen vordefiniert sind.

4.2.11 Namespaces

Die Beispiele von Klassen, Interfaces oder auch Traits haben demonstriert, wie sich mit Hilfe von Objektorientierung Funktionalitäten kapseln und somit auch wiederverwenden lassen. Ein Problem besteht jedoch nach wie vor: Namensgleichheit. In den Beispielen wurden Klassen wie `Rectangle`, `Person` oder `Car` verwendet. Sollen diese anderen Entwickler*innen zur Verfügung gestellt werden, besteht durchaus die Möglichkeit, dass sich in deren Code ebenfalls Klassen befinden, die diese Namen tragen und es kommt zu einem Konflikt.

Um derlei Konflikte zu vermeiden, existieren *Namespaces* (deutsch „Namensräume“). Diese gliedern PHP-Code in virtuelle Hierarchien, vergleichbar mit Verzeichnissen in einem Dateisystem. Namespaces können dabei mehrere Ebenen haben, bei der obersten spricht man vom sogenannten Vendor-Namespace. Dies sollte ein Bezeichner sein, der global einzigartig ist und somit den eigenen Code unverwechselbar zuordnet. Darunter gibt es Subnamespaces, die den Code noch weiter gliedern.

Namespaces definieren

Eine Namespace-Deklaration steht am Beginn einer PHP-Datei und wird mit dem Schlüsselwort `namespace` eingeleitet. Danach folgt der Name des Vendor-Namespace. Weitere Subnamespaces werden mit einem Backslash (`\`) getrennt angegeben. Namespaces werden in der Regel in Pascal Case angegeben.

⁵<https://www.php.net/manual/de/spl.exceptions.php>

```
1 namespace VendorNamespace\SubNamespace\AnotherSubNamespace;
```

Angenommen, es sollen in PHP Klassen erstellt werden, die Web Technologies repräsentieren. Dazu könnte es die Klassen **Lecture** und **Exercise** geben. Diese gehören inhaltlich eindeutig zusammen, somit macht es Sinn, sie unter einem gemeinsamen Namespace zu führen. Als Vendor-Namespace könnte hier z. B. **Fhooe** verwendet werden, um zu kennzeichnen, dass es sich um eine Lehrveranstaltung an der FH Oberösterreich handelt. Als Subnamespaces bieten sich dann **Mtd** und weiters **WebTechnologies** an. Somit wird die Hierarchie klar und deutlich repräsentiert.

Die Grundgerüste der beiden Klassen, zunächst die Klasse **Lecture** in **Lecture.php**:

```
1 <?php
2
3 namespace Fhooe\Mtd\WebTechnologies;
4
5 class Lecture
6 {
7     // ...
8 }
```

Danach die Klasse **Exercise** in **Exercise.php**:

```
1 <?php
2
3 namespace Fhooe\Mtd\WebTechnologies;
4
5 class Exercise
6 {
7     // ...
8 }
```

Namespaces beim Instanzieren angeben

Sollen nun in einer Datei **webtechnologies.php** Objekte von den Klassen **Lecture** und **Exercise** angelegt (d. h. instanziiert) werden, muss der Namespace mit angegeben werden. Es wird zunächst ein Objekt der Klasse **Lecture** unter Angabe des vollen Namespaces angelegt (die Klasse muss zuvor mit **include** oder **require** eingebunden werden):

```
1 $lecture = new \Fhooe\Mtd\WebTechnologies\Lecture();
```

Die Angabe des Namespaces beginnt bereits mit einem Backslash, der sozusagen den globalen Namespace darstellt, von dem ausgegangen wird. Danach folgen Vendor-Namespace, die beiden Subnamespaces und schließlich wie gewohnt der Name der Klasse. Dies wird auch als der voll qualifizierte Name einer Klasse (engl. „full qualified class name“) bezeichnet.

Da diese Angabe vor allem wenn sie öfters benötigt wird, viel Schreibarbeit bedeutet und unübersichtlich ist, können Namespaces auch für die Datei importiert werden. Dies geschieht mit dem **use** Schlüsselwort. Danach kann die Angabe des voll qualifizierten Namespaces entfallen – der Klassenname reicht. Dies soll am Beispiel eines **Exercise**-Objekts demonstriert werden:

```
1 use Fhooe\Mtd\WebTechnologies\Exercise;
2
```

```
3 // ...  
4  
5 $exercise = new Exercise();
```

Zunächst wird am Beginn der Datei der Namespace importiert, danach reicht für das Anlegen des Objekts der Klassenname.

Wird innerhalb einer Klasse, die in einem Namespace liegt, ein Objekt einer anderen Klasse instanziiert, so muss nicht der gesamte Namespace angegeben werden. PHP nimmt immer zunächst an, dass sie die Klasse im selben Namespace befindet. Würde also innerhalb der **Lecture**-Klasse ein **Exercise**-Objekt angelegt, so reicht die folgende Angabe, es muss nicht der Namespace mittels **use** importiert werden:

```
1 $exercise = new Exercise();
```

Wird ein Objekt einer Klasse angelegt, die in einem anderen Namespace liegt, muss dieses entweder als voll qualifizierter Name angegeben oder mit **use** importiert werden.

Code, der in keinem Namespace liegt, gehört zum sogenannten *globalen Namespace*. Ein gutes Beispiel hierfür ist die Klasse **Exception** (siehe Abschnitt 4.2.10). Um ein **Exception**-Objekt innerhalb einer Klasse mit Namespace anzulegen, muss der Backslash vorangestellt werden, um Explizit den globalen Namespace anzugeben. Wird darauf verzichtet oder vergessen, sucht der PHP-Interpreter im aktuellen Namespace nach einer Klasse **Exception**, wird (vermutlich) nicht fündig und erzeugt einen Fehler. Die korrekte Angabe lautet also wie folgt:

```
1 $exception = new \Exception();
```

Namespaces und Verzeichnisstruktur

PHP-Klassen, die einem Namespace angehören, können in einem beliebigen Verzeichnis liegen, da es sich bei Namespaces prinzipiell um eine virtuelle Hierarchie handelt. D. h., es müssen sich etwa die beiden Klassen **Lecture** und **Exercise** nicht zwingend in einem Verzeichnis **Fhooe>Mtd>WebTechnologies** befinden. Betrachtet man allerdings moderne PHP-Projekte, so zeigt sich schnell, dass die Hierarchie der Namespaces in der Regel auch in der Verzeichnisstruktur abgebildet wird. Der Grund hierfür ist die Autoloading-Funktionalität, die sehr stark auf diesem Prinzip basiert. Abschnitt 4.3 geht initial darauf ein, Vorlesung 5 wird die zugrundeliegenden Standards im Detail behandeln.

4.3 Autoloading

Bei der Arbeit mit Klassen und Objekten wurde ersichtlich, dass diese zunächst bekannt sein müssen, bevor sie initialisiert werden können. Sind die Klassen in eigenen Dateien definiert (was wie bereits erwähnt so üblich ist), so müssen diese Dateien zunächst mit **include** oder **require** wie in Abschnitt 4.1 beschrieben eingebunden werden. Dies funktioniert problemlos für kleine PHP-Projekte, wird jedoch umständlich, wenn nicht gar zum Problem, wenn Projekte aus mehr als nur ein paar PHP-Dateien (sprich hunderten von Dateien) bestehen.

Eine elegante und zeitgemäße Lösung stellt *Autoloading* dar. Diese Strategie bedient sich der PHP-Funktion `spl_autoload_register($autoloader)`⁶ die es erlaubt, dem

⁶<https://www.php.net/manual/de/function.spl-autoload-register.php>

PHP-Interpreter Funktionen (die sogenannten Autoloader) bekannt zu geben, die sich um das Laden von Klassen kümmern, wenn ein Objekt derselben erzeugt wird. Die Autoloader Funktion hat die Aufgabe, den voll qualifizierten Namen der Klasse, also Namespace und Klassenname, zu verwenden, um im Dateisystem die entsprechende Datei zu finden und mittels `require` einzubinden.

Um diesen Prozess einfacher zu gestalten, empfiehlt es sich, wie bereits in Abschnitt 4.2.11 erwähnt, Namespaces als Verzeichnisse abzubilden, somit muss die Autoloader-Funktion den Backslash, der die einzelnen Ebenen des Namespaces trennt, nur noch in den betriebssystemabhängigen Verzeichnisseparator umwandeln (Backslash unter Windows, Slash unter macOS bzw. Linux) und die Endung `.php` anfügen und kann dann die Datei einbinden.

4.3.1 Autoloading an einem Beispiel erklärt

Gegeben seien wiederum die Klassen `Lecture` und `Exercise`, die beide im Namespace `Fhooe\Mtd\WebTechnologies` definiert sind. Beide Dateien liegen – ausgehend vom Webverzeichnis (also dem Ordner, in dem die im Webserver aufzurufenden Dateien liegen) – unter `Fhooe\Mtd\WebTechnologies`. Im Webverzeichnis selbst liegt die Datei `autoload.php`. Sie soll von jeder der beiden Klassen ein Objekt anlegen und wird auch den Code für den Autoloader enthalten. Dieser wird wie folgt definiert:

```
1 <?php
2 function autoload(string $fullyQualifiedClassName): void
3 {
4     $fullyQualifiedClassName = ltrim($fullyQualifiedClassName, "\\");
5     $filePath = "";
6
7     if ($lastBackslashPosition = strrpos($fullyQualifiedClassName, "\\")) {
8         $namespace = substr($fullyQualifiedClassName, 0, $lastBackslashPosition);
9         $shortClassName = substr($fullyQualifiedClassName, $lastBackslashPosition + 1);
10        $filePath = str_replace("\\", DIRECTORY_SEPARATOR, $namespace) .
            DIRECTORY_SEPARATOR;
11    } else {
12        $shortClassName = $fullyQualifiedClassName;
13    }
14    $filePath .= "$shortClassName.php";
15
16    if (file_exists($filePath)) {
17        require $filePath;
18    }
19 }
20 spl_autoload_register(autoload(...));
```

In Zeile 20 wird `spl_autoload_register()` aufgerufen. Der Parameter ist eine Referenz auf eine Callback-Funktion in der First-Class Callable Syntax, in diesem Fall `autoload()`. Diese Referenz wird dem PHP-Interpreter übergeben. Somit weiß dieser, dass er die Funktion aufrufen kann bzw. muss, wenn Objekte instanziiert werden sollen, deren Klassendefinitionen nicht bekannt sind.

`autoload()` selbst ist ab Zeile 2 definiert und bekommt als Argument automatisch den voll qualifizierten Klassennamen übergeben (egal, ob dieser beim `new`-Aufruf komplett angegeben oder über `use` importiert wird). Im Falle von `Lecture` nimmt `$fullyQualifiedClassName` also den Wert `Fhooe\Mtd\WebTechnologies\Lecture` an.

Der Backslash am Anfang wird nicht mit übergeben, würde aber zur Vorsicht in Zeile 4 entfernt werden.

Als nächstes wird dieser String in den Pfad zur Klassendatei (`$filePath`) und den reinen Klassennamen ohne Namespace (`$shortClassName`) aufgeteilt. Dies geschieht ab Zeile 7, wo zunächst das letzte Vorkommen des Backslashes gesucht und dann der String von Beginn bis zu diesem Zeichen in `$namespace` und der Rest in `$shortClassName` gespeichert wird. Danach werden im String des Namespaces die Backslash-Zeichen durch den im aktuellen Betriebssystem vorherrschenden Verzeichnistrenner (festgelegt in `DIRECTORY_SEPARATOR`) ersetzt. Unter Windows bleibt der Backslash, unter macOS oder Linux wird ein Slash eingesetzt. Schließlich wird der Klassenname um die Endung `.php` ergänzt und an den bisherigen Pfad angehängt. Hat die Klasse keinen Namespace, so wird im `else`-Zweig der Klassenname unverändert übernommen und `$filePath` bleibt leer, sodass die Datei direkt im Webverzeichnis erwartet wird. Bevor der so entstandene Pfad mit `require` eingebunden wird, prüft Zeile 16 mit `file_exists()`, ob die Datei tatsächlich existiert. Andernfalls würde `require` einen Fatal Error werfen und die Ausführung sofort abbrechen – mit der Existenzprüfung kann der Autoloader bei unbekannten Klassen hingegen sauber aufgeben, sodass etwa weitere registrierte Autoloader die Möglichkeit bekommen, die Klasse zu finden.

Um Autoloading zu verwenden, muss diese Datei einmal im Projekt eingebunden sein. Sie kümmert sich fortan darum, unbekannte Klassendefinitionen zu laden. Um Objekte von `Lecture` und `Exercise`, wie in Abschnitt 4.2.11 gezeigt, zu erzeugen, muss nur der Autoloader, nicht die spezifischen Klassendefinitionen eingebunden werden:

```
1 require "autoload.php";
2
3 use Foooe\Mtd\WebTechnologies\Exercise;
4 use Foooe\Mtd\WebTechnologies\Lecture;
5
6 $lecture = new Lecture();
7 $exercise = new Exercise();
```

Diese Art des Autoloadings folgt dem Grundprinzip, dass die Verzeichnisstruktur die Namespace-Hierarchie eins zu eins abbildet. Historisch wurde dieses Vorgehen im *PSR-0 Standard*⁷ festgeschrieben, der zusätzlich auch Underscores in Klassennamen als Verzeichnistrenner interpretierte – ein Erbe aus der Zeit vor der Einführung von Namespaces in PHP 5.3. PSR-0 ist mittlerweile veraltet und durch den flexibleren *PSR-4 Standard* abgelöst, der unter anderem erlaubt, einzelne Namespace-Präfixe auf konfigurierbare Basisverzeichnisse abzubilden. Der hier gezeigte Autoloader entspricht in seiner Funktionsweise einer vereinfachten Variante von PSR-4. Letzterer wird, zusammen mit weiteren PSR-Standards, in Vorlesung 5 besprochen.

4.4 Weiterführende Literatur

Das Kapitel der Objektorientierung wird in [41] gut behandelt, einen guten Überblick bieten auch wieder [44, 49]. Hier sind auch neuere Konzepte durchaus abgedeckt, wenngleich auch nicht allzu umfangreich.

⁷<https://www.php-fig.org/psr/psr-0/>

Einen Fokus auf die Objektorientierung der Sprache legt [51], darüber hinaus werden bewährte Entwicklungsmuster (Patterns) erläutert.

Vorlesung 5

Komponenten und Standards

5.1 Allgemeines zu Komponenten

Die Art und Weise, PHP-Applikationen zu entwickeln, hat sich in den letzten Jahren stark verändert.

5.1.1 Von DIY zu Frameworks

In den Anfängen der Sprache war die Wiederverwendbarkeit von Code nur ein untergeordnetes Ziel. PHP wurde meist als Mittel zum Zweck eingesetzt, um dynamische Inhalte zu erzeugen. Lösungen für bestimmte Problemstellungen mussten selbst umgesetzt werden.

Etwa ab Mitte der 2000er Jahre entstanden die ersten PHP-Frameworks wie CakePHP und Symfony im Jahr 2005 oder CodeIgniter und das Zend Framework im Jahr 2006 [21]. Diese Frameworks boten erstmals ein großes Set an oft benötigten Funktionalitäten an, wie etwa Datenbankzugriffe, integrierte Template-Systeme, Session-Verwaltung u. v. m. Mit ihnen konnten nun Webprojekte zum ersten Mal in geordneteren Bahnen abgewickelt werden. Hinter den Frameworks versammelte sich schnell eine Community, die nicht nur für die Weiterentwicklung sondern auch am Support für Anwender*innen eine maßgebliche Rolle spielte. PHP-Entwickler*innen waren nun mit ihren Problemen nicht mehr alleine bzw. mussten diese selbst lösen, sondern konnten sich Unterstützung bei anderen Entwickler*innen holen, die ebenfalls Erfahrungen mit dem jeweiligen Framework hatten.

Frameworks bedeuten jedoch auch immer eine Art von geschlossenem Ökosystem, in das zum Erlernen viel Zeit investiert werden muss. Wird das Framework dann beherrscht, so erfordern andere Projekte möglicherweise andere Frameworks, was einen erneuten (wenn wohl auch verkürzten) Lernprozess zur Folge hat. Wird ein Framework für ein Webprojekt ausgewählt, so kommt dies auch einer Investition in dessen Zukunft gleich. Es gilt abzuschätzen, ob das Framework auch weiterentwickelt wird, um nicht Gefahr zu laufen, dass in naher Zukunft keine Updates mehr zur Verfügung gestellt werden.

Schließlich stellt sich auch immer die Frage, ob ein Framework überhaupt benötigt wird, da es, wie bereits erwähnt, eine Vielzahl von Funktionalitäten enthält, die oft nur bei großen Projekten Sinn machen. Beispiele hierfür können große öffentliche

Seiten oder Plattformen sein. Für kleinere Projekte, wie etwa private Anwendungen oder Webprojekte mit sehr spezifischen Anwendungsgebieten stellt ein PHP-Framework möglicherweise zu viel unnötigen Overhead dar.

Diesem Umstand kann seit einigen Jahren mit einem neuen Konzept begegnet werden: Komponenten.

5.1.2 PHP-Komponenten

Eine *Komponente* ist eine Sammlung von PHP-Code, d. h. zusammengehörige Klassen, Interfaces und Traits – die eine spezifische Aufgabe erfüllt bzw. ein Problem löst. Im Unterschied zu Frameworks ist dies genau *eine* Aufgabe. So existieren etwa Komponenten, um E-Mails zu versenden, HTTP-Requests zu tätigen oder Logging für Webapplikationen zur Verfügung zu stellen. Gute PHP-Komponenten sollten dabei die folgenden Kriterien erfüllen:

- **Fokussiert auf ein Problem:** Eine gute Komponente existiert, um genau ein Problem zu lösen bzw. eine Aufgabe zu erfüllen. Dies ermöglicht es, Overhead und unnötigen Code in der eigenen Applikation zu vermeiden, da durch die Verwendung einer Komponente nicht unnötige Funktionalitäten „eingekauft“ werden.
- **Klein:** PHP-Komponenten sind nicht größer, als sie sein müssen. Sie enthalten nur so viel Code wie notwendig, um ihre Aufgabe zu erfüllen. Diese Größe kann natürlich variieren. So gibt es Komponenten, die nur aus einer Klasse bestehen, während andere eine Sammlung verschiedener Klassen darstellen.
- **Kooperativ:** PHP-Komponenten arbeiten gut mit anderen Komponenten zusammen bzw. behindern diese nicht. So sollten Komponenten ihren eigenen Namespace verwenden, um Namenskollisionen zu vermeiden. Weiters können Komponenten auf anderen aufbauen, wenn Funktionalitäten benötigt werden, die schon existieren.
- **Gut getestet:** Komponenten sollten gut getestet sein, um potentielle Probleme zu minimieren. Durch ihren Fokus und ihre Kleinheit sind Tests in der Regel nicht umfangreich. Gute Komponenten inkludieren diese Tests, sodass sie von anderen Entwickler*innen ausgeführt und nachvollzogen werden können. Mehr zu Tests in Vorlesung 12.
- **Gut dokumentiert:** Um die Verwendung zu erleichtern, sollten Komponenten über eine gute Dokumentation verfügen. Eine README-Datei erklärt, was die Komponente tut, wie sie installiert werden kann und wie sie eingesetzt wird. Kommentare im Code (so genannte Docblocks) erläutern Klassen und Methoden näher, daraus kann in wenigen Schritten eine webbasierte API-Dokumentation generiert werden.

Eine moderne PHP-Webapplikation muss also nicht zwingend auf ein großes Framework setzen (wenngleich moderne Frameworks wie Symfony¹, Laravel² oder Yii³ wie erwähnt für größere Projekte absolut empfehlenswert sind). Vielmehr sollte zunächst genau überlegt und analysiert werden, welche Funktionalitäten benötigt werden. Da-

¹<https://symfony.com/>

²<https://laravel.com/>

³<https://www.yiiframework.com/>

nach kann recherchiert werden, ob es für diese Aufgaben bereits Komponenten gibt und falls ja, werden diese in das eigene Projekt integriert und verwendet. Somit muss das Rad an mehreren Stellen nicht neu erfunden werden (d. h. Probleme gelöst werden, für die es bereits etablierte Lösungen gibt) und der Overhead und die Einlernphase für ein großes Framework entfallen ebenfalls.

5.2 Komponenten verwalten: Composer & Packagist

Für die Installation und Verwaltung von Komponenten hat sich in PHP eine Gesamtlösung etabliert: Das Tool *Composer* in Kombination mit dem Komponentenverzeichnis *Packagist*.

5.2.1 Composer

*Composer*⁴ ist ein sogenannter Dependency-Manager, manchmal auch Paketmanager genannt. Seine Aufgabe besteht darin, Komponenten für ein Projekt zu installieren und deren Abhängigkeiten (andere Komponenten, die benötigt werden) aufzulösen und ebenfalls herunterzuladen (und ggf. wiederum deren Abhängigkeiten usw.). Die Komponenten – auch Packages genannt – werden dabei auf Projekt-Basis installiert (nicht systemweit). Somit ist Composer nicht mit klassischen Paketmanagern wie Apt unter Linux vergleichbar, die Pakete für das gesamte System installieren. Composer hat jedoch eine Option, Pakete global zu installieren, etwa wenn es sich um Tools (Binaries) handelt, die in mehreren Projekten verwendet werden können. Die aus dem ersten Semester schon bekannte Entsprechung in JavaScript ist *npm*⁵.

Darüber hinaus kümmert sich Composer auch um das Autoloading der installierten Komponenten, d. h. es wird eine zentrale Autoloading-Datei erstellt (*autoload.php*), die in der Hauptdatei des eigenen Projekts eingebunden wird und sich um die Einbindung aller Komponenten selbst kümmert.

Composer ist eine Commandline-Applikation, die auf dem Computer, auf dem der Webserver läuft, d. h. in der Regel ein entfernter Server, genauso aber ein Docker-Container oder der eigene Computer mit einer lokalen PHP-Installation, installiert wird. Die aktuelle Version ist 2.9.7.

Composer installieren

Composer kann lokal im Projekt oder global im System installiert werden. Die ersten vier Schritte sind dabei identisch. Unter Linux, Unix bzw. macOS lauten sie:

```
1 php -r "copy('https://getcomposer.org/installer', 'composer-setup.php');"
2 php -r "if (hash_file('sha384', 'composer-setup.php') === '
    c8b085408188070d5f52bcfe4ecfbee5f727afa458b2573b8eaaf77b3419b0bf2768dc67c86944d
    a1544f06fa544fd47') { echo 'Installer verified'.PHP_EOL; } else { echo '
    Installer corrupt'.PHP_EOL; unlink('composer-setup.php'); exit(1); }"
3 php composer-setup.php
4 php -r "unlink('composer-setup.php');"
```

⁴<https://getcomposer.org/>

⁵<https://www.npmjs.com/>

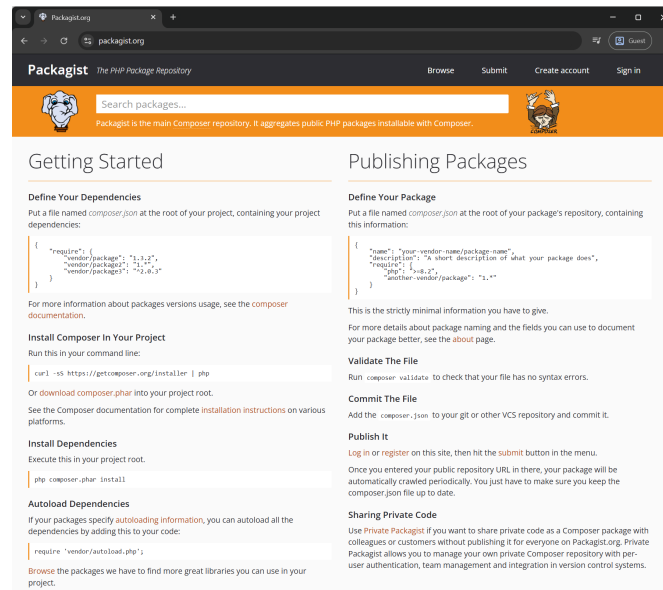


Abbildung 5.1: Die Startseite von Packagist. In der Suchzeile kann nach Paketen gesucht werden, darunter werden Informationen zur Verwendung mit Composer angezeigt.

Dies lädt zunächst den aktuellen Composer-Installer herunter und benennt ihn in `composer-setup.php` um. Danach wird die SHA384-Checksumme überprüft, um sicherzustellen, dass der Installer korrekt heruntergeladen wurde. Falls dies der Fall ist, wird `composer-setup.php` ausgeführt und anschließend gelöscht. Das Ergebnis ist die Datei `composer.phar` (PHAR steht für PHP Archive), die mit `php composer.phar` ausgeführt werden kann.

In der Regel wird Composer jedoch systemweit installiert. Dies wird dadurch erreicht, dass die PHAR-Datei nach `usr/local/bin/composer` verschoben wird. Dadurch ist sie im gesamten System mit dem Befehl `composer` aufrufbar (so ist dies etwa im *fhooe-web-dock* Docker-Container der Fall):

```
1 mv composer.phar /usr/local/bin/composer
```

Um Composer für Windows zu installieren, wird der Installer `Composer-Setup.exe` von <https://getcomposer.org/download/> heruntergeladen und ausgeführt. Dies installiert alles notwendige und ändert die Pfadvariablen, sodass der Befehl `composer` von überall her ausgeführt werden kann.

5.2.2 Packagist Repository

Das zentrale Verzeichnis für PHP-Komponenten ist *Packagist*⁶ (siehe Abbildung 5.1). In ihm werden öffentlich verfügbare Komponenten aggregiert und über Schlüsselworte und Packagenamen durchsuchbar gemacht. Packagist listet dabei verfügbare Komponenten auf und gibt dabei auch aus, wie viele Downloads die Komponente aufweist bzw. wie oft sie von Nutzer*innen mit Sternen versehen wurde (um sie positiv hervorzuheben).

⁶<https://packagist.org/>

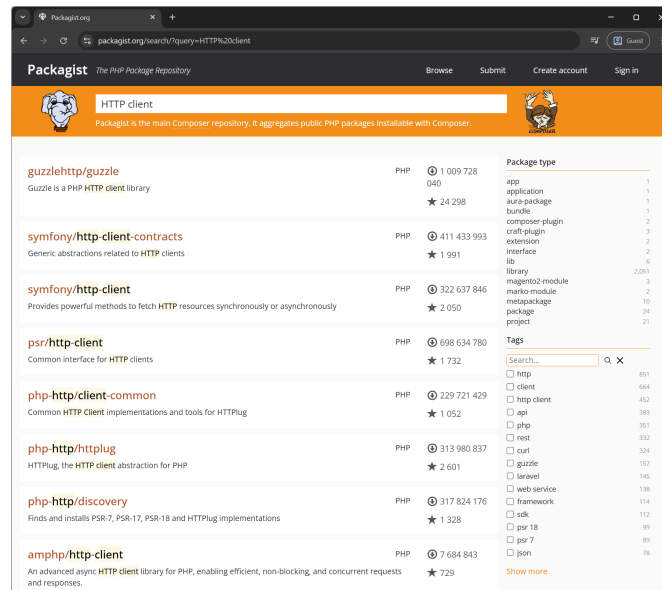


Abbildung 5.2: Auf Packagist wird eine PHP-Komponente für HTTP-Requests gesucht. Es werden mehrere Ergebnisse angezeigt.

Die Anzahl der auf Packagist geführten PHP-Komponenten ist sehr hoch. Mit Stand Mai 2026 sind es 451.631, die in insgesamt 5.532.688 einzelnen Versionen vorliegen [34]. Um einen Überblick über die populärsten Komponenten zu bekommen, kann auf die Downloadzahlen zurückgegriffen werden. Diese sind jedoch nicht immer aussagekräftig, darum empfiehlt sich die Auflistung *Awesome PHP* [50], die eine kuratierte Liste an PHP-Komponenten darstellt.

Packages finden

Um nun eine Komponente zu installieren, wird diese zunächst über Packagist gesucht und dann mittels Composer im eigenen Projekt installiert.

Als Beispiel soll eine kleine Anwendung geschrieben werden, die Anfragen an die öffentliche Chuck Norris Jokes API⁷ sendet, um einen Fakt (Witz) über Chuck Norris abzufragen. Eine API ist eine Schnittstelle, die auf GET- oder POST-Anfragen hört und daraufhin Daten in einem maschinenlesbaren Format – meist JSON – zurückliefert. Mehr zu APIs folgt in Vorlesung 11.

Als erstes wird eine Komponente gesucht, die HTTP-Requests verarbeiten kann, da die Chuck Norris Jokes API den Fakt (d. h. die Daten) als Response auf einen GET-Request liefert. Dazu wird auf Packagist „HTTP client“ als Suchbegriff verwendet (siehe dazu Abbildung 5.2).

Eines der ersten Resultate ist **guzzlehttp/guzzle**: „Guzzle is a PHP HTTP client library“. Dies klingt vielversprechend und die hohe Anzahl der Downloads lässt ebenfalls darauf schließen, dass es sich um eine solide Komponente handelt.

⁷<https://api.chucknorris.io/>

Komponentennamen

Am Beispiel der Komponente `guzzlehttp/guzzle` zeigt sich das Namensschema der Komponenten auf Packagist. Dieses lautet immer `vendor/package`. Jede volle Packagebezeichnung besteht aus einem Vornamen – hier `guzzlehttp` – und einem Packagenamen – hier `guzzle`. Der Vorname ist global einzigartig und ist entweder der Name eines Projekts oder einer Firma bzw. Organisation, die diese Komponente (und auch andere) entwickelt. Unter dem Vornamen `guzzlehttp` finden sich etwa 18 verschiedene Packages. Der Packagename selbst bezeichnet die Komponente, die installiert werden soll. Für die Installation des Packages sind beide Teile notwendig.

Komponenten installieren

Mit dem Wissen, dass die benötigte HTTP-Client-Komponente unter dem vollen Namen `guzzlehttp/guzzle` zu finden ist, kann nun Composer verwendet werden, um sie zu installieren.

Die Syntax für eine Installation mit Composer in der Kommandozeile lautet wie folgt:

```
1 $ composer require vendor/package
```

`composer` ruft die Composer-Executable auf, mit dem Schlüsselwort `require` wird eine benötigte Komponente angefordert. Es folgt der volle Name aus Vendor- und Packagenamen.

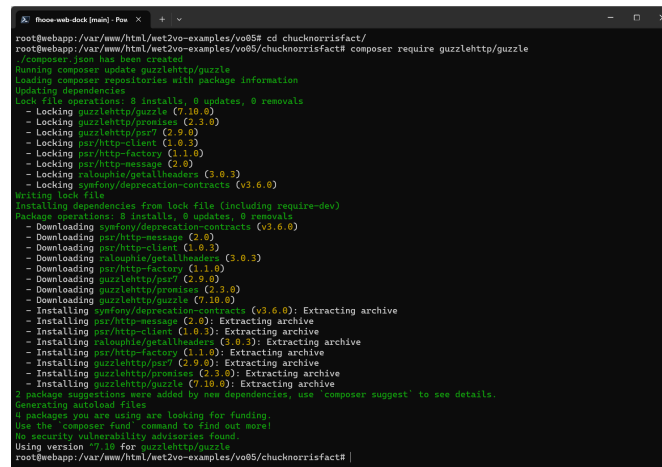
Für das konkrete Beispiel wird in der Kommandozeile (im Docker Container mittels `docker exec -it webapp /bin/bash`) zunächst ein Projektordner `chucknorrisfact` angelegt, dann in diesen gewechselt und schließlich der Composer Executable mitgeteilt, dass das Package `guzzlehttp/guzzle` benötigt wird:

```
1 $ mkdir chucknorrisfact
2 $ cd chucknorrisfact
3 $ composer require guzzlehttp/guzzle
```

Dies startet den Installationsvorgang (siehe Abbildung 5.3) und erzeugt zunächst zwei Dateien im Ordner `chucknorrisfact`: `composer.json` und `composer.lock`. In die erstgenannte Datei werden die benötigten Komponenten im JSON-Format für die aktuelle Anwendung notiert. Die Datei kann entweder von Composer selbst (über den eben erfolgten `composer require`-Aufruf) modifiziert oder auch händisch bearbeitet werden. Der Inhalt nach dem initialen Aufruf sieht wie folgt aus:

```
1 {
2   "require": {
3     "guzzlehttp/guzzle": "^7.10"
4   }
5 }
```

Die Datei `composer.lock` ist eine Ergänzung zu `composer.json` und speichert die genauen Versionen, die installiert wurden. Dies hilft dabei, dass mehrere Personen, die an dem Projekt arbeiten, dieselben Versionen der Komponenten installieren, auch wenn es inzwischen neuere gibt. Beide Dateien sollten bei einer Arbeit mit Versionskontrolle (z. B. Git) unbedingt mitgespeichert werden, da sie zu einer späteren Installation der benötigten Komponenten unerlässlich sind.



```

root@webapp:/var/www/html/net2vo-examples/vo65# cd chucknorrisfact# composer require guzzlehttp/guzzle
./composer.json has been created
Running composer update guzzlehttp/guzzle
Loading composer repositories with package information
Updating dependencies
Lock file operations: 8 installs, 0 updates, 0 removals
- Locking guzzlehttp/guzzle (7.10.0)
- Locking guzzlehttp/promises (2.3.0)
- Locking guzzlehttp/psr7 (2.9.0)
- Locking psr/http-client (1.0.3)
- Locking psr/http-factory (1.1.0)
- Locking psr/http-message (2.0)
- Locking ralouphie/getallheaders (3.0.3)
- Locking symfony/deprecation-contracts (v3.6.0)
Writing lock file
Installing dependencies from lock file (including require-dev)
Package operations: 8 installs, 0 updates, 0 removals
- Downloading symfony/deprecation-contracts (v3.6.0)
- Downloading psr/http-message (2.0)
- Downloading psr/http-client (1.0.3)
- Downloading ralouphie/getallheaders (3.0.3)
- Downloading psr/http-factory (1.1.0)
- Downloading guzzlehttp/psr7 (2.9.0)
- Downloading guzzlehttp/promises (2.3.0)
- Downloading guzzlehttp/guzzle (7.10.0)
- Installing symfony/deprecation-contracts (v3.6.0): Extracting archive
- Installing psr/http-message (2.0): Extracting archive
- Installing psr/http-client (1.0.3): Extracting archive
- Installing ralouphie/getallheaders (3.0.3): Extracting archive
- Installing psr/http-factory (1.1.0): Extracting archive
- Installing guzzlehttp/psr7 (2.9.0): Extracting archive
- Installing guzzlehttp/promises (2.3.0): Extracting archive
- Installing guzzlehttp/guzzle (7.10.0): Extracting archive
3 package suggestions were added by new dependencies, use 'composer suggest' to see details.
Generating autoload files
4 packages you are using are looking for funding.
Use the 'composer fund' command to find out more!
No security vulnerability advisories found.
Using version ^7.10 for guzzlehttp/guzzle
root@webapp:/var/www/html/net2vo-examples/vo65/chucknorrisfact#

```

Abbildung 5.3: Composer installiert das Paket `guzzlehttp/guzzle` inklusive seiner Abhängigkeiten.

Neben den beiden Dateien wurde auch ein Ordner `vendor` angelegt, in den Composer die Dateien der Komponente, weitere Komponenten auf denen die eingebundene aufbaut und eigene Autoloading-Dateien abgelegt hat. Dieser Ordner kann sehr groß werden, es sollte daher auch nicht unter Versionskontrolle gestellt oder weitergegeben werden. Seine Inhalte können von Composer mit einem einfachen Aufruf sofort wieder generiert werden.

Existieren `composer.json` und `composer.lock` für ein Projekt nämlich bereits, so sind die benötigten Komponenten bereits bekannt und es reicht der folgende Aufruf, um diese zu installieren:

```
1 $ composer install
```

Versionsangaben

Wie ersichtlich ist, notiert Composer neben dem Packagenamen auch eine Versionsnummer, in diesem Fall `~7.10`. Dies gibt an, dass minimal Version 7.10 installiert werden, zudem werden auch kompatible, größere Updates installiert. Diese Syntax verlässt sich auf das sogenannte *Semantic Versioning*⁸, das Versionsnummern dreistellig im Format `MAJOR.MINOR.PATCH`, also z. B. 1.2.4 oder 3.0.8 angibt. Die Änderung der Versionsnummern folgt dabei genauen Regeln:

1. MAJOR ändert sich nur, wenn inkompatible, d. h. sehr fundamentale Änderungen an den Methoden (der API) durchgeführt werden.
2. MINOR ändert sich, wenn neue Funktionalitäten hinzugefügt werden, die jedoch rückwärtskompatibel sind.
3. PATCH ändert sich, wenn Fehler behoben werden, die auch rückwärtskompatibel sind.

⁸<https://semver.org/>

Da sich alle Komponenten, die auf Packagist veröffentlicht wurden, an Semantic Versioning halten (sollten), kann mit einer Versionsangabe genau gesteuert werden, welche Versionen Composer installieren darf. Die Versionsangabe folgt dem Packagenamen mit einem Doppelpunkt:

```
1 $ composer require vendor/package:^1.0
```

Die folgenden Notationen sind dabei möglich:

- Genaue Versionsangabe, z. B. `1.2.3`: Installiert nur genau diese Version.
- Bereichsangaben, z. B. `>=1.0` oder `>= 1.0 <2.0`: Mit Hilfe der Vergleichsoperatoren `>`, `>=`, `<`, `<=` und `!=` können Reichweiten angegeben werden. Ein Leerzeichen dient als logische UND-Verknüpfung, mit Hilfe des `||`-Zeichens kann ein logisches ODER angegeben werden.
- Bereichsangaben mit Bindestrich, z. B. `1.0 - 2.0`: Entspricht dabei einer Angabe von `>=1.0.0 <=2.0.0` schließt also die beiden Angaben mit ein.
- Wildcard-Angabe, z. B. `1.0.*`: Ermöglicht jede beliebige Versionsnummer an der PATCH-Stelle, entspricht also `>=1.0.0 <1.1`.
- Nächstmögliches Release, z. B. `~1.2`: Installiert immer die neueste Version auf der aktuell angegebenen Semantic Versioning Ebene. Das angegebene Beispiel entspricht also `>=1.2.0 <2.0.0`, während `~1.2.3 >=1.2.3 <1.3.0` entspricht.
- Nächstmögliches Release (großzügiger), z. B. `^1.2.3`: Verhält sich ähnlich wie der Tilde-Operator, ist jedoch großzügiger, sodass alle rückwärtskompatiblen Updates installiert werden. Das angegebene Beispiel entspricht in diesem Fall `>=1.2.3 <2.0.0`, da neue MINOR-Versionen keine Kompatibilitätsprobleme erzeugen (sollten). Dieser Operator sollte wann immer möglich zum Einsatz kommen.

5.2.3 Packages updaten

Um nach einiger Zeit die eingebundenen Komponenten auf den neuesten Stand zu bringen, kann folgender Befehl (im Projektverzeichnis, also z. B. in `chucknorrisfact`) ausgeführt werden:

```
1 $ composer update
```

Dabei werden die neuesten Versionen der eingebundenen Komponenten – sofern vorhanden – heruntergeladen und im Projekt eingebunden, `composer.lock` wird aktualisiert. Dabei werden jedoch immer die Versionsauflagen aus der `composer.json`-Datei beachtet. Gibt es neue Versionen einer Komponente – etwa Version 8.0.0 von `guzzlehttp/guzzle` – so muss zunächst die Versionsnummer in `composer.json` entsprechend angepasst werden, damit ein Update möglich ist.

5.2.4 Autoloading

Nachdem Komponenten mittels Composer installiert wurde, bleibt die Frage, wie diese eingebunden werden. Während Composer die notwendigen Komponenten herunterlädt, wird auch ein Autoloader generiert, der direkt im `vendor` Verzeichnis abgelegt wird. Somit muss nur diese Datei eingebunden werden, somit sind alle Komponenten verfügbar.

```
1 require "vendor/autoload.php";
```

5.2.5 Beispielprojekt: Custom Chuck Norris Fact

Mit der heruntergeladenen HTTP-Komponente und dem Wissen über das einfach Einbinden des Autoloaders kann nun das Chuck Norris-Projekt umgesetzt werden.

Auf der Webseite der API⁹ steht ihre Verwendung. Es können GET-Requests an mehrere URLs geschickt werden, um verschiedene Ergebnisse zu erhalten:

- GET <https://api.chucknorris.io/jokes/random>: Liefert einen zufälligen Fakt.
- GET <https://api.chucknorris.io/jokes/random?category={category}>: Liefert einen zufälligen Fakt aus einer übergebenen Kategorie.
- GET <https://api.chucknorris.io/jokes/categories>: Liefert alle verfügbaren Kategorien zurück.

Das Ergebnis ist immer eine JSON-Datenstruktur. Da es sich um GET-Requests handelt, können die URL-Aufrufe direkt in die Browserzeile kopiert werden und das Ergebnis ist sichtbar.

Für das Projekt wird nun eine Datei `index.php` angelegt, die – der Einfachheit halber – in ihrem Ablauf prozedural gehalten wird.

Am Beginn der Datei steht ein PHP-Block, der zunächst die verfügbaren Kategorien abfragen muss, damit diese dann in einem HTML-Formular angezeigt werden können.

Zunächst wird der Autoloader aus dem `vendor`-Verzeichnis eingebunden, damit die mit Composer installierte Komponente direkt verwendet werden kann. Dann wird der Guzzle HTTP-Client angelegt, indem ein Objekt der Klasse `GuzzleHttp\Client` erstellt wird. Das entstehende Objekt, gespeichert in der Variable `$client` hat eine Methode `request($method, $uri, $options)`, die einen Request absenden kann. Der erste Parameter bestimmt die Art (hier GET), der zweite den URL, der aufgerufen werden soll. Im dritten Parameter können Optionen als Array angegeben werden. Wird ein Eintrag `query` angegeben, so können GET-Parameter mitübergeben werden. Das Ergebnis wird in einem Response-Objekt (hier `$categoriesResponse`) gespeichert.

```
1 require "vendor/autoload.php";
2
3 $client = new GuzzleHttp\Client();
4
5 $categoriesResponse = $client->request(
6     "GET",
7     "https://api.chucknorris.io/jokes/categories",
8 );
```

Die API liefert ihre Daten im JSON Format. Um diese Antwort abzufragen, wird mit der Methode `getBody()` der Inhalt der Response entnommen und mit Hilfe von `json_decode()` in eine PHP-Datenstruktur umgewandelt. In diesem Fall handelt es sich um ein Array, da auch in JSON ein Array enthalten ist. `$categories` ist also ein PHP-Array in dem alle Kategorien enthalten sind.

```
1 $categoriesBody = $categoriesResponse->getBody();
2 $categories = json_decode($categoriesBody);
```

Nun folgt in der Datei ein HTML-Formular, das mit dem Kategorien-Array eine Dropdown-Liste (`<select>`) erstellt und die Kategorien darin anzeigt. Die Dropdown-Liste hat den Namen `category`, die einzelnen `<option>`-Elemente erhalten den Namen

⁹<https://api.chucknorris.io/>

der Kategorie als `value`-Attribut – dieser Wert wird beim Abschieken übertragen und steht, da das Formular mit POST abgeschickt wird, somit in `$_POST["category"]`. Das Formular ruft dabei dieselbe Seite erneut auf, denn in `$_SERVER["SCRIPT_NAME"]` steht immer der Pfad zur aktuellen Datei.

```

1 <!DOCTYPE html>
2 <html lang="en">
3   <head>
4     <title>Random Chuck Norris Fact</title>
5     <meta charset="UTF-8">
6   </head>
7   <body>
8     <h1>Get a Random Chuck Norris Fact</h1>
9     <form action="<?= $_SERVER["SCRIPT_NAME"] ?>" method="post">
10      <label for="categorySelector">Select a category for a fact:</label>
11      <select name="category" id="categorySelector">
12        <?php
13          foreach ($categories as $category) {
14            echo "<option value=\"\$category\">\"$category\"</option>";
15          }
16        ?>
17      </select>
18      <button type="submit">Roundhouse!</button>
19    </form>

```

Bevor `<body>` und `<html>` wieder geschlossen werden, wird nun die mögliche Ausgabe des zufälligen Chuck Norris Fakts vorgenommen.

Diese soll nur erfolgen, wenn in `$_POST["category"]` ein Wert enthalten ist, was nur der Fall ist, wenn das Formular auch abgeschickt wurde (nicht beim initialen Aufruf). Dann wird erneut das Guzzle-Client-Objekt `$client` verwendet, um einen GET-Request abzuschicken. Dieses Mal wird ein zufälliger Fakt abgefragt und als Query-Parameter wird die ausgewählte Kategorie mitgegeben. War die Kategorie etwa „animal“, entspricht dies folgendem Aufruf: <https://api.chucknorris.io/jokes/random?category=animal>.

Dann wird wieder der Body der Response ausgelesen und mit `json_decode()` in eine PHP-Datenstruktur umgewandelt. Dieses Mal entsteht ein Objekt in dessen Eigenschaft `value` der Fakt enthalten ist. Auf diesen wird bei der Ausgabe mit `echo` zugegriffen.

```

1 if (isset($_POST["category"])) {
2   echo "<h2>Random fact from category {$_POST["category"]}</h2>";
3
4   $factResponse = $client->request(
5     "GET",
6     "https://api.chucknorris.io/jokes/random",
7     [
8       "query" => [
9         "category" => $_POST["category"],
10      ],
11    ],
12  );
13
14  $factBody = $factResponse->getBody();
15  $fact = json_decode($factBody);
16  echo "<p>$fact->value</p>";
17 }

```

Abbildung 5.4 zeigt einen zufälligen Chuck Norris Fakt aus der Kategorie „science“.

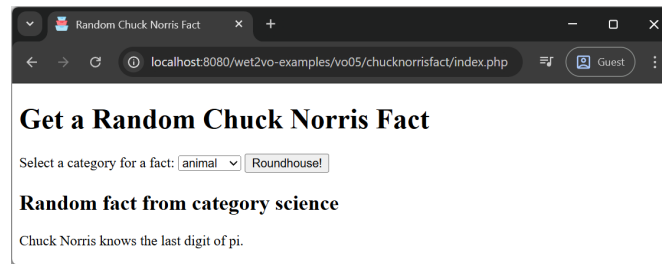


Abbildung 5.4: Ein Chuck Norris Fakt aus der Kategorie „science“. Die Dropdown-Liste zeigt nach dem Absenden des Formulars immer den ersten Eintrag in der Kategorienliste „animal“.

5.2.6 Eigene Komponenten definieren

Für das gezeigte Beispielprojekt wurde für die Umsetzung auf bestehende Komponenten zurückgegriffen. Es ist jedoch natürlich genauso möglich, eine eigene Komponente zu erstellen und diese über Packagist anderen zur Verfügung zu stellen. Dazu muss `composer.json` mehr Informationen enthalten als die bloße Angabe der verwendeten Dependencies. So muss etwa ein gültiger Packagename im Format `vendor/package` angegeben werden, sinnvoll sind auch eine Beschreibung, der Typ (z. B. `library` im Falle einer Bibliothek oder `project` im Falle eines gesamten Projekts), die Angabe einer Lizenz und der Autor*innen. Wenn Dependencies nötig sind, werden diese in gewohnter Form eingebunden.

Um eine initiale Version der `composer.json` zu erstellen, kann der folgende Befehl ausgeführt werden.

```
1 $ composer init
```

Dies startet einen kleinen interaktiven Assistenten, der die obigen Informationen abfragt und daraus die Datei generiert. Weitere Einträge können gemäß der Composer Dokumentation für das `composer.json`-Format [3] vorgenommen werden.

Der folgende Code zeigt `composer.json` für das `fhoee/router` Paket in Version 3.0.0, welches in dieser Lehrveranstaltung noch öfters zum Einsatz kommen wird:

```
1 {
2   "name": "fhoee/router",
3   "description": "A simple object-oriented router for educational purposes.",
4   "license": "MIT",
5   "type": "library",
6   "keywords": [
7     "routing",
8     "php",
9     "education"
10  ],
11  "authors": [
12    {
13      "name": "Wolfgang Hochleitner",
14      "email": "wolfgang.hochleitner@fh-hagenberg.at",
15      "role": "Developer"
16    }
17  ],
```

```
18  "require": {
19      "php": "^8.5",
20      "ext-mbstring": "*",
21      "psr/log": "^3.0"
22  },
23  "require-dev": {
24      "ergebnis/composer-normalize": "^2.50",
25      "friendsofphp/php-cs-fixer": "^3.94",
26      "mockery/mockery": "^1.6",
27      "pestphp/pest": "^4.4",
28      "phpstan/phpstan": "^2.1"
29  },
30  "minimum-stability": "stable",
31  "autoload": {
32      "psr-4": {
33          "Fhooe\\Router\\": "src/"
34      }
35  },
36  "config": {
37      "allow-plugins": {
38          "ergebnis/composer-normalize": true,
39          "pestphp/pest-plugin": true
40      }
41  },
42  "scripts": {
43      "cs-check": "php-cs-fixer fix --dry-run --diff",
44      "cs-fix": "php-cs-fixer fix",
45      "pest": "pest",
46      "phpstan": "phpstan analyse src --memory-limit=-1 --level 9 || true",
47      "test": [
48          "@cs-check",
49          "@phpstan",
50          "@pest"
51      ]
52  }
53 }
```

5.3 Standards in PHP: PHP-FIG

Neben der Arbeit mit Komponenten gehört auch das Einhalten von Standards mittlerweile zum modernen Arbeitsstil in PHP.

5.3.1 Allgemeines zu Standards

Lange Zeit gab es für die Arbeit mit PHP nicht wirklich verbindliche Standards. Jedes Framework, jede*r Entwickler*in verfasste PHP-Code nach bestem Wissen und Gewissen bzw. nach eigenen Vorstellungen und Richtlinien. Dies führte zu einem Kommunikationsproblem. Code aus einem Framework konnte kaum in anderen Frameworks verwendet werden, da im Bezug auf Coding Standards Uneinigkeit herrschte.

Dieses Problem wurde von Entwickler*innen erkannt und ab 2009 mit Gründung der PHP Framework Interopability Group, kurz PHP-FIG¹⁰ aktiv bekämpft. Diese Or-

¹⁰<https://www.php-fig.org/>

ganisation hat es sich zum Ziel gesetzt, De-facto Standards zu entwickeln, die Zusammenarbeit zwischen verschiedenen Projekten erleichtern bzw. überhaupt ermöglichen.

Diese De-facto Standards werden *PSR* (PHP Standard Recommendation) und *PER* (PHP Evolving Recommendation) genannt. Sie sind in keinsten Weise verbindlich, haben sich aber in kürzester Zeit innerhalb der PHP-Community etabliert und erfreuen sich daher weiter Verbreitung. Im Gegensatz zu anderen Sprachen (etwa JavaScript) sind diese Standards auch die einzig akzeptierten. PSRs sind nummeriert¹¹, zur Zeit gibt es 14 akzeptierte, vier im Entwurfsstatus, drei eingestellte und zwei überholte.

PSRs lassen sich grob in drei Kategorien einteilen:

- **Styles:** Code-Richtlinien, die für die einheitliche Formatierung von PHP-Code in den verschiedensten Projekten sorgen sollen.
- **Autoloading:** Definierte Autoloading-Strategien, die es ermöglichen, mehrere Komponenten mit einem einzigen Autoloader zu laden.
- **Interfaces:** Standardisierte Schnittstellen, für immer wiederkehrende Aufgaben (wie z. B. Logging).

Neben den vielen PSRs existiert seit Juni 2022 auch der erste *PER*¹² (kurz für PHP Evolving Recommendation). Diese wurden eingeführt, um auf die kontinuierliche Weiterentwicklung der Sprache PHP besser reagieren zu können und betrifft vor allem Coding-Richtlinien (Styles). Denn ein PSR durchläuft einen langwierigen Prozess, bis er verabschiedet wird und stellt den Status zu genau einem Zeitpunkt da. Eine Coding-Guideline muss sich aber mit der Sprache weiterentwickeln, weshalb diese nun als PER abgebildet werden. Dies entspricht dem System der Living Standards in HTML und JavaScript.

In den folgenden Abschnitten sollen einige der wichtigsten PSRs und PERs überblicksartig besprochen werden. Weitere Details können auf der PHP-FIG Webseite entnommen werden.

5.3.2 PSR-1

PSR-1 ist der *Basic Coding Standard*. Er enthält einfache Richtlinien, wie PHP-Code formatiert werden soll um in Komponenten und Frameworks einheitlich zu erscheinen. Folgende Richtlinien zählen dazu:

- **PHP Tags:** PHP-Code darf nur in den langen `<?php ?>` Tags oder den kurzen Echo-Tags `<?= ?>` stehen. Andere Tags sind nicht zulässig.
- **Zeichenkodierung:** PHP-Dateien müssen UTF-8 kodiert sein. Die Dateien dürfen keine Byte Order Mark (BOM) enthalten.
- **Nebeneffekte:** Eine PHP-Datei soll *entweder* Dinge definieren (wie Klassen, Funktionen, Konstanten etc.) *oder* Logik mit Nebeneffekten ausführen, jedoch niemals beides. Als Nebeneffekte werden etwa das Einbinden von externen Dateien mittels `require`, das Erzeugen von Output (mittels `echo`) und dergleichen bezeichnet.
- **Autoloading:** Alles Namespaces und Klassen müssen einer Autoloading PSR (PSR-4 oder auch PSR-0, siehe dazu Abschnitt 5.3.4) folgen, d. h. jede Klasse

¹¹<https://www.php-fig.org/psr/>

¹²<https://www.php-fig.org/per/>

muss sich in einer eigenen Datei befinden und einen Namespace mit mindestens einer Ebene (einen Vendor-Namespace) aufweisen.

- **Klassennamen:** Klassennamen werden im StudlyCaps (auch Upper Camel Case, Studly Case oder Pascal Case) Format geschrieben, z. B. `MyClassName`.
- **Konstantennamen:** Konstanten werden komplett in Großbuchstaben geschrieben, Underscores werden zur Worttrennung verwendet, z. B. `SUM` oder mit Underscores `NR_OF_ELEMENTS`.
- **Methodennamen:** Methoden werden im Camel Case notiert, bei dem der erste Buchstabe klein geschrieben wird, z. B. `calculateTotalSum()`.
- **Variablennamen:** PSR-1 überlässt die Formatierung von Variablennamen explizit den Autor*innen. Es können dabei `$StudlyCaps`, `$camelCase` oder auch `$under_score` zum Einsatz kommen, jedoch immer konsistent.

5.3.3 PER Coding Style

Aufbauend auf PSR-1 setzt die *PER Coding Style* auf striktere Regeln und deren Einhaltung. Dieser Coding-Stil erweitert und ersetzt seit Juli 2022 PSR-12 (welche wiederum den Vorläufer PSR-2 ersetzte) und wurde wie dieser aus vielen bekannten Frameworks übernommen bzw. erstellt und stellt somit einen verbreiteten Konsens dar. Im Gegensatz zu den statischen PSRs handelt es sich hierbei um einen *Living Standard*. Aktuell ist Version 3.0.0 vom 14. Juli 2025. Folgende Regeln gehören dazu:

- **PSR-1 implementieren:** Der Code folgt den Regeln aus PSR-1.
- **Dateien und Zeilenumbrüche:** Alle Dateien enthalten nur Linux-Zeilenumbrüche (LF), die letzte Zeile darf nicht leer sein. Das schließende PHP-Tag `?>` muss in Dateien, die nur PHP-Code beinhalten, entfallen. Es gibt keine harte Obergrenze für die Zeilenlänge; die weiche Grenze beträgt 120 Zeichen, empfohlen werden maximal 80 Zeichen pro Zeile. Überflüssige Whitespace-Zeichen müssen entfernt werden. Leerzeilen für bessere Lesbarkeit sind erlaubt, jede Zeile darf nur eine PHP-Anweisung beinhalten.
- **Einrückungen:** Code darf ausschließlich mit 4 Leerzeichen (nicht mit Tabulatoren) eingerückt werden.
- **Abschließendes Komma (Trailing Commas):** Erstreckt sich eine Liste über mehrere Zeilen, muss nach dem letzten Element ein abschließendes Komma gesetzt werden. Dies gilt zwingend für Arrays, Parameterlisten in Funktionsdefinitionen, Argumentlisten bei Funktionsaufrufen, `use`-Listen in Closures und Zweige in `match`-Ausdrücken. In einzeiligen Listen darf kein abschließendes Komma stehen.
- **Schlüsselwörter und Typen:** PHP-Schlüsselwörter und Typen (`int`, `string`, `true`, `null` etc.) werden ausschließlich klein geschrieben. Es ist die Kurzform der Typbezeichner zu verwenden (`bool` statt `boolean`, `int` statt `integer`). Bei Union Types (`int|string`) und Intersection Types (`A&B`) dürfen keine Leerzeichen um die Trenner gesetzt werden; ist `null` Bestandteil eines Union Types, so steht es an letzter Stelle bzw. wird bei einem einzelnen Typ als `?T` abgekürzt.
- **Reihenfolge von Deklarationen:** Die Reihenfolge am Dateianfang ist strikt

vorgegeben: `<?php`, Datei-Docblock, `declare`-Statements, `namespace`, `use`-Statements (für Klassen, Funktionen, Konstanten in separaten Blöcken).

- **Klassen, Traits und Interfaces:** Die öffnende Klammer steht in einer eigenen Zeile. Sichtbarkeitsmodifikatoren sind für alle Eigenschaften, Konstanten und Methoden zwingend. Bei der *Constructor Property Promotion* werden Sichtbarkeit und Typ direkt im Konstruktor angegeben. Bei der Instanziierung müssen die runden Klammern auch ohne Argumente gesetzt werden (`new Foo()`).
- **Reihenfolge der Modifikatoren:** Wenn mehrere Modifikatoren auftreten, ist deren Reihenfolge fest vorgeschrieben: `abstract/final`, Sichtbarkeit (`public`, `protected`, `private`), Set-Sichtbarkeit (z. B. `private(set)`), `static`, `readonly`, Typdeklaration, Name.
- **Methoden und Funktionsaufrufe:** Die öffnende Klammer folgt dem Namen ohne Leerzeichen. Erstrecken sich die Argumente eines Aufrufs über mehrere Zeilen, müssen sie jeweils um eine Ebene eingerückt werden; die schließende Klammer muss dann in einer eigenen Zeile (zusammen mit der schließenden Klammer der Anweisung, falls vorhanden) stehen. Bei benannten Argumenten steht kein Leerzeichen vor dem Doppelpunkt, jedoch eines danach.
- **Kontrollstrukturen:** Ein Leerzeichen nach dem Schlüsselwort (`if`, `switch` etc.) ist zwingend. Die öffnende Klammer steht in derselben Zeile. `else` und `elseif` stehen in derselben Zeile wie die schließende Klammer des vorangegangenen Blocks. Der Körper jeder Struktur muss in geschweiften Klammern eingeschlossen sein.
- **Operatoren:** Binäre Operatoren (Arithmetik, Vergleich, Zuweisung, logisch etc.) sowie der ternäre Operator müssen von je einem Leerzeichen umgeben sein. Inkrement-/Dekrement-Operatoren stehen ohne Whitespace direkt am Operanden, Type Casts ohne Leerzeichen innerhalb der Klammern (`(int) $x`). Wird ein Ausdruck umgebrochen, muss der Operator am Beginn der neuen Zeile stehen.
- **Arrays:** Arrays müssen mit der kurzen Syntax (`[]`) deklariert werden. Bei mehrzeiligen Arrays steht die öffnende Klammer in derselben Zeile wie das Gleichheitszeichen, jeder Wert auf einer eigenen Zeile, mit abschließendem Komma nach dem letzten Element.
- **Closures und Arrow Functions:** Closures werden mit einem Leerzeichen nach `function` deklariert. Bei Arrow Functions darf hingegen kein Leerzeichen zwischen `fn` und der öffnenden Klammer stehen: `fn(int $x) => $x * 2;`.
- **Enumerations:** Die Fälle (`case`) werden in PascalCase geschrieben. Enums können `public` Konstanten enthalten, unterstützen aber keine Vererbung; daher müssen nicht-öffentliche Methoden `private` (statt `protected`) sein.
- **Property Hooks:** Mit PHP 8.4 eingeführte `get`- und `set`-Hooks folgen einer eigenen Formatierungsregel: Mehrzeilige Hook-Bodys werden wie Methodenkörper eingerückt; besteht der Hook aus einem einzigen Ausdruck, kann er mit `=>` verkürzt werden (z. B. `get => $this->value;`).
- **Heredoc und Nowdoc:** Der schließende Identifikator darf eingerückt werden, muss jedoch auf derselben Ebene wie der restliche Code innerhalb des Blocks stehen.
- **Attribute:** Attribute (z. B. `#[Override]`) müssen in einer eigenen Zeile direkt vor

der Struktur (Klasse, Methode, Eigenschaft) stehen. Werden sie bei Parametern genutzt, stehen sie direkt vor dem jeweiligen Parameter in derselben oder einer eigenen Zeile.

5.3.4 PSR-4 und PSR-0

PSR-4 und PSR-0 sind sogenannte Autoloading-Strategien und beschreiben, wie Klassen und Namespaces mit der Platzierung im Dateisystem korrelieren.

Eine erste, lange gültige Fassung war PSR-0, die besagte, dass jeder Namespace als eigenes Verzeichnis zu existieren hatte. An den Klassennamen wurde schließlich noch `.php` gehängt und so konnten sehr einfach Klassen gefunden und geladen werden.

Die Klasse `Acme\Logging\AdvancedLogging` würde im Verzeichnis `lib\vendor` des Projekts vom Autoloader unter `lib\vendor\Acme\Logging\AdvancedLogging.php` gesucht. Die Klasse `Fhooe\MTD\WebTechnologies` entsprechend unter `lib\vendor\Fhooe\MTD\WebTechnologies.php`.

Diese Strategie ist jedoch unflexibel, da sie – je nach Verschachtelungstiefe der Namespaces – viele Verzeichnisse erzeugt. Außerdem muss immer vom selben Grundverzeichnis – in diesem Fall `lib\vendor` – ausgegangen werden. Aus diesem Grund wurde mit PSR-4 eine flexiblere Autoloading-Strategie entworfen, die nun statt PSR-0 eingesetzt wird.

Im Unterschied zu PSR-0 kann bei PSR-4 ein Basisverzeichnis definiert werden, was die Verzeichnisstruktur weniger komplex gestaltet. So kann festgelegt werden, dass Klassen des `Acme` Namespaces im Verzeichnis `src` liegen. Somit liegt dann die Klasse `Acme\Logging\AdvancedLogging` dann in `src\Logging\AdvancedLogging.php`. Alternativ kann auch `Acme\Logging` auf `src` gemapped werden, dann ergibt sich folgende Verzeichnisstruktur: `src\AdvancedLogging.php`.

In der in Abschnitt 5.2.6 gezeigten Datei `composer.json` ist etwa ersichtlich, dass Klassen mit dem Namespace `Fhooe\Router` direkt im Verzeichnis `src` liegen. D. h., es gibt keine Unterordner-Struktur `Fhooe\Router`, da dies nur unnötige Komplexität mit sich bringen würde.

PSR-4 gibt auch genau vor, wie der Autoloader selbst implementiert werden muss. Da jedoch Composer einen PSR-4 kompatiblen Autoloader generiert (siehe Abschnitt 5.2.4), gibt es eigentlich keinen Grund, dies selbst zu tun.

5.3.5 Weitere PSRs

Neben den erwähnten PSRs gib es noch eine Vielzahl weiterer, von denen hier nur sechs oberflächlich erwähnt werden sollen. Genaue Details und weitere PSRs finden sich auf der PHP-FIG-Webseite.

- **PSR-3 Logger Interface:** Definiert ein standardisiertes Interface für Klassen, die Ereignisse loggen. Dies macht es möglich, dass Logger-Komponenten einfach ausgetauscht werden können.
- **PSR-5 PHPDoc Standard:** Beschreibt wie die Dokumentation von PHP-Code aussehen soll (derzeit noch ein Entwurf).
- **PSR-6 Caching Interface:** Beschreibt ein Interface für Komponenten, die Caching unterstützen.

- **PSR-7 HTTP Message Interface:** Beschreibt Interfaces für HTTP-Messages. Die Komponente Guzzlehttp etwa implementiert dieses Interface für Requests und Responses.
- **PSR-18 HTTP Client:** Ein gemeinsames Interface zum Versenden und Empfangen von HTTP Requests und Responses. Komponenten wie Guzzlehttp können so vereinheitlichte Interfaces nützen und sind austauschbar.
- **PSR-20 Clock:** Beschreibt ein Interface, mit dem die Systemzeit ausgelesen werden kann.

5.3.6 Standards einhalten

Nachdem nun über die Wichtigkeit von Standards geschrieben wurde, bleibt noch die Frage, wie diese einfach eingehalten werden können. Entwickler*innen können unmöglich immer alle Details im Kopf behalten, weshalb es der Unterstützung von Werkzeugen bedarf, um den Standards zu folgen. Dies betrifft in erster Linie PSR-1 und PER Coding Style, da sich diese mit der Formatierung von Code auseinandersetzen.

IDEs konfigurieren

Der einfachste Weg, bereits beim Schreiben von Code den PSR/PER-Richtlinien zu folgen ist, die verwendete IDE (sofern möglich) bereits auf PSR-1 und PER-CS zu konfigurieren.

PhpStorm etwa bietet unter `Settings > Editor > Code Style > PHP` die Möglichkeit, den Code-Stil nach dem PER Coding Style 2.0.0 zu setzen. Dazu wird unter `Set from...` die Option `PER-CS 2.0.0` ausgewählt.

Diese Option setzt die Platzierung von Klammern, Leerzeichen und einigen anderen Dingen auf die von PER-CS 2.0.0 vorgegebenen Regeln. Abbildung 5.5 zeigt einen Screenshot aus der Konfiguration von PhpStorm. PER-CS 3.0.0 ist zur Zeit noch nicht verfügbar, jedoch bereits als fehlend gemeldet¹³.

Überprüfung mittels Tools: PHP_CodeSniffer

Während die korrekte Konfiguration einer IDE hilft, grundsätzliche Probleme beim Editieren zu vermeiden, so können sich dennoch Fehler einschleichen, die subtiler sind, oder auch durch die automatischen Editierhilfen nicht immer abgefangen werden. Typische Beispiele sind etwa der korrekte Zeilenumbruch, der unter Windows nicht LF, sondern CRLF lautet, eine fehlende Leerzeile am Ende oder auch Zeilenlängen, die 120 Zeichen übersteigen.

Solche Fehler können im Nachhinein jedoch leicht mit einem Tool überprüft und auch korrigiert werden, dem *PHP_CodeSniffer*¹⁴ – kurz **phpcs**. Es kann als PHAR-Datei oder global über Composer installiert werden und wird über die Kommandozeile aufgerufen. Im **fhooe-web-dock** Docker Container ist **phpcs** bereits installiert und kann ohne Installation verwendet werden.

Die Syntax ist denkbar einfach:

¹³<https://youtrack.jetbrains.com/issue/WI-81988/Implement-PER-3-codestyle-in-PHP-formatters>

¹⁴https://github.com/PHP-CSStandards/PHP_CodeSniffer

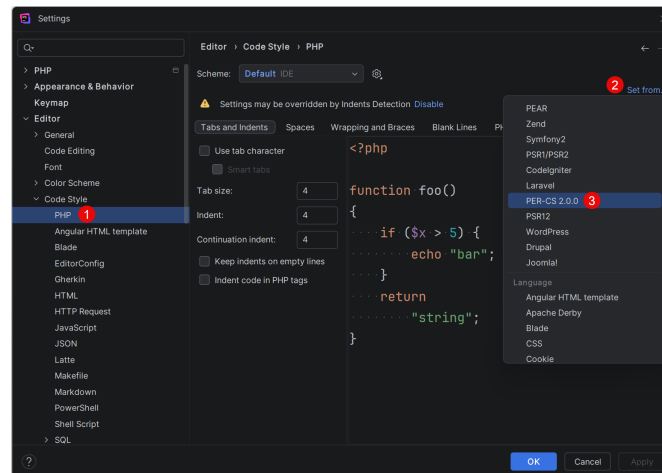


Abbildung 5.5: Der Code-Stil von PhpStorm kann nach den Regeln von PER 2.0.0 konfiguriert werden.

```
1 $ phpcs /path/to/code/myfile.php
```

Alternativ kann auch nur ein Verzeichnis angegeben werden, was alle Dateien innerhalb überprüft. Standardmäßig verwendet `phpcs` jedoch den PEAR Coding-Standard. Um PSR-12 (PER Coding Style ist noch nicht verfügbar) zu verwenden, kann ein optionaler Parameter angegeben werden:

```
1 $ phpcs --standard=PSR12 /path/to/code/myfile.php
```

Nach der Ausführung generiert `phpcs` einen Fehlerreport.

Im Falle des Chuck Norris-Beispiels werden zur Simulation von Problemen in einer neuen Datei Einrückungen verschoben, ein Trailing Comma entfernt und die öffnende geschwungene Klammer der if-Bedingung in die nächste Zeile verschoben. Danach wird das Tool wie folgt aufgerufen und generiert diesen Output:

```
1 $ phpcs --standard=PSR12 index_phpcerrors.php
```

```
FILE: /var/www/html/wet2vo-examples/vo05/chucknorrisfact/index_phpcerrors.php
```

```
-----
FOUND 2 ERRORS AFFECTING 2 LINES
```

```
-----
37 | ERROR | [x] Expected 1 space after closing parenthesis; found newline
53 | ERROR | [x] Line indented incorrectly; expected at least 4 spaces, found 2
```

```
-----
PHPCBF CAN FIX THE 2 MARKED SNIFF VIOLATIONS AUTOMATICALLY
-----
```

```
Time: 75ms; Memory: 8MB
```

Gefunden wurden in diesem Fall zwei Fehler:

- In Zeile 37 sollte die geschwungene Klammer nach einem Leerzeichen zur vorigen Klammer stehen, gefunden wurde jedoch ein Zeilenumbruch.
- In Zeile 53 ist der Code um zwei Leerzeichen zu wenig eingerückt.

Das Trailing Comma wird nicht gefunden, da dies in PSR-12 noch nicht enthalten war. Die restlichen Fehler können nun händisch korrigiert werden oder mit Hilfe eines zweiten Tools, dem *PHP Code Beautifier and Fixer* – kurz **phpcbf** – automatisch korrigiert werden. In der Regel wird **phpcs** im Aufruf einfach durch **phpcbf** ersetzt:

```
1 phpcbf --standard=PSR12 index_phpcserrors.php
```

Die führt zur Korrektur der Fehler mit folgendem Output:

PHPCBF RESULT SUMMARY

FILE	FIXED	REMAINING
/.../wet2vo-examples/vo05/chucknorrisfact/index_phpcserrors.php	3	0

A TOTAL OF 2 ERRORS WERE FIXED IN 1 FILE

Time: 103ms; Memory: 8MB

Ein erneutes Aufrufen von **phpcs** zeigt, dass keine Fehler mehr vorhanden sind.

PHP_CodeSniffer kann auch in viele IDEs, wie z. B. PhpStorm automatisch integriert werden. Unter **Settings** » **Editor** » **Inspections** » **PHP** » **Quality tools** kann ein Haken bei der Einstellung **PHP_CodeSniffer validation** gesetzt werden. Voraussetzung ist, dass dieser zuvor korrekt unter **Settings** » **PHP** » **Quality Tools** » **PHP_CodeSniffer** konfiguriert wurde. Die automatische Inspektion führt **phpcs** nach jedem Speichervorgang aus und zeigt Fehler bereits in PhpStorm an.

Alternative: PHP-CS-Fixer

Eine weit verbreitete Alternative zu PHP_CodeSniffer ist der *PHP CS Fixer*¹⁵ – kurz **php-cs-fixer**. Im Gegensatz zum Gespann **phpcs**/**phpcbf** vereint er die Analyse und die automatische Korrektur in einem einzigen Werkzeug. Vor allem aber unterstützt er den aktuellen *PER Coding Style 3.0* bereits direkt als eingebautes Regelset und ist damit – anders als **phpcs** – auf dem aktuellen Stand der PHP-FIG-Spezifikation.

Wie **phpcs** kann auch **php-cs-fixer** als PHAR-Datei oder über Composer (Paket **friendsofphp/php-cs-fixer**) installiert werden und ist im **fhooe-web-dock** Docker Container bereits vorhanden. Der zentrale Befehl ist **fix**, dem als Argument eine Datei oder ein Verzeichnis übergeben wird. Standardmäßig verwendet **php-cs-fixer** jedoch das **@PSR12**-Regelset; um stattdessen den *PER Coding Style 3.0* anzuwenden, wird das Regelset **@PER-CS3x0** über die Option **--rules** angegeben:

```
1 php-cs-fixer fix --rules=@PER-CS3x0 /path/to/code/myfile.php
```

Ein wesentlicher Unterschied zu **phpcs** besteht darin, dass **fix** die Dateien unmittelbar verändert – das Tool ist also primär als Fixer und nicht als Linter konzipiert. Soll **php-cs-fixer** – analog zu **phpcs** – lediglich auflisten, welche Verstöße er finden würde, ohne die Dateien zu modifizieren, kann der Befehl **check** verwendet werden, eine Kurzform für **fix --dry-run**. Mit der zusätzlichen Option **--diff** werden die konkreten Änderungen im **udiff**-Format ausgegeben:

¹⁵ <https://github.com/PHP-CS-Fixer/PHP-CS-Fixer>

```
1 php-cs-fixer check --rules=@PER-CS3x0 --diff index_phpcserrors.php
```

Auf das gleiche manipulierte Chuck Norris-Beispiel angewendet, generiert das Tool folgenden Output:

```
1) index_phpcserrors.php
----- begin diff -----
--- /var/www/html/wet2vo-examples/vo05/chucknorrisfact/index_phpcserrors.php
+++ /var/www/html/wet2vo-examples/vo05/chucknorrisfact/index_phpcserrors.php
@@ -28,14 +28,13 @@
foreach ($categories as $category) {
echo "<option value=\"\$category\">{$category}</option>";
}
-      ?>
+?>
</select>
<button type="submit">Roundhouse!</button>
</form>

<?php
-if (isset($_POST["category"]))
-{
+if (isset($_POST["category"])) {
echo "<h2>Random fact from category {$_POST["category"]}</h2>";

$factResponse = $client->request(
@@ -43,7 +42,7 @@
"https://api.chucknorris.io/jokes/random",
[
"query" => [
-      "category" => $_POST["category"]
+      "category" => $_POST["category"],
],
],
);

----- end diff -----
```

```
Found 1 of 1 files that can be fixed in 0.018 seconds, 19.41 MB memory used
```

Die Verstöße entsprechen inhaltlich jenen, die zuvor auch **phpcs** gemeldet hat, allerdings wird auch das trailing Comma gefunden und das schließende PHP-Tag wird an den Anfang der Zeile verschoben.

Wo bei **phpcs** jedoch im Anschluss **phpcbf** aufgerufen werden müsste, genügt bei **php-cs-fixer** der Wechsel des Befehls von **check** auf **fix**, um die Korrektur tatsächlich durchzuführen:

```
1 php-cs-fixer fix --rules=@PER-CS3x0 index_phpcserrors.php
```

```
1) index_phpcserrors.php
```

```
Fixed 1 of 1 files in 0.021 seconds, 19.41 MB memory used
```

Ein erneuter **check**-Lauf bestätigt, dass keine Verstöße mehr vorhanden sind.

Während sich `phpcs` sehr gut für eine schnelle Überprüfung über die Kommandozeile eignet, ist `php-cs-fixer` stärker auf die dauerhafte Verwendung im selben Projekt ausgerichtet. In diesem Fall wird statt der Option `--rules` eine Konfigurationsdatei `.php-cs-fixer.dist.php` im Projekt-Wurzelverzeichnis abgelegt, in der Regelset und einzubeziehende Pfade festgelegt werden. Liegt eine solche Datei vor, genügt der Aufruf von `php-cs-fixer fix` ohne weitere Argumente. Eine minimale Konfiguration für PER 3.0 sieht etwa so aus:

```
1 <?php
2 return (new PhpCsFixer\Config())
3     ->setRules(['@PER-CS3x0' => true])
4     ->setFinder((new PhpCsFixer\Finder())->in(__DIR__));
```

Auch `php-cs-fixer` kann in PhpStorm integriert werden. Unter **Settings** **Editor** **Inspections** **PHP** **Quality tools** kann ein Haken bei der Einstellung **PHP CS Fixer validation** gesetzt werden, sofern das Tool zuvor unter **Settings** **PHP** **Quality Tools** **PHP CS Fixer** korrekt konfiguriert wurde. Verstöße werden dann auch nach jedem Speichervorgang direkt im Editor angezeigt. Über **Code** **Reformat Code with PHP CS Fixer** lassen sie sich unmittelbar beheben.

5.4 Weiterführende Literatur

Die beste Referenz zum Thema Komponenten und Standards bietet mit Sicherheit [52]. Es ist die modernste Referenz zu diesem Thema. Eine gute Übersicht online bietet ebenfalls [25]. Eine etwas kompaktere Einführung zum Thema Package Management bietet [49].

Vorlesung 6

Templates und Routing

6.1 Template-Systeme

Template-Systeme dienen zur Entkopplung von HTML- und PHP-Code. PHP bietet in seiner Basisinstallation kein System an, kann jedoch streng gesehen selbst als Template-System gesehen werden, da statischem HTML-Markup mit PHP-Code dynamische Inhalte hinzugefügt werden können. Da diese Vermischung jedoch meist unübersichtlich ist und auch durchaus noch mehr Funktionen haben könnte bzw. sollte, haben sich diverse Libraries am Markt etabliert.

6.1.1 Allgemeines

Die Trennung einzelner Teile einer Webseite kann zu einer besseren Übersicht und leichter Wartbarkeit beitragen. Durch die Trennung von HTML und CSS in separate Dateien muss der Markup-Code nicht verändert werden, wenn irgendwo eine Farbe ausgetauscht werden soll. Durch die Trennung von HTML und JavaScript kann die lokale Programmlogik ebenfalls in einer eigenen Datei implementiert werden, ohne dazu jedes Mal das HTML-Gerüst editieren zu müssen.

Auch für PHP gilt diese Anforderung, wenngleich sie bis jetzt nicht umgesetzt wurde, denn alle Beispiele bisher haben Logik und Ausgabe (also PHP- und HTML-Code) miteinander vermischt. Die Befehle `echo` und `print()` im PHP-Code sind dafür hauptverantwortlich, erzeugen sie doch HTML-Code mitten in PHP-Skripten. Sollen einzelne HTML-Teile wiederverwendet werden, müssen diese in eine Funktion oder Methode ausgelagert werden. Techniken wie Advanced Escaping, bei denen zwischen logischen PHP-Anweisungen (z. B. `for`-Schleifen) und erzeugter HTML-Ausgabe hin und her gewechselt werden, oder die Heredoc-Syntax komplettieren die Vermischung von Code und Struktur, die aber eigentlich nicht gewünscht ist.

Template-Systeme schaffen hierbei Abhilfe. Konstante HTML-Teile werden in eigene Dateien ausgelagert und somit vom PHP-Code entfernt. Dynamische Teile, die aufgrund der Programmlogik unterschiedlich sein können, werden durch Platzhalter aus PHP entsprechend befüllt und ergeben somit wieder eine dynamische Seite. Dies erlaubt zum einen ein konsistentes Seitendesign, auf der anderen Seite eine einfachere Wartbarkeit und größere Übersicht.

6.1.2 Bekannte Template-Engines

Es gibt viele unterschiedliche Template-Systeme für PHP. Viele davon sind als eigenständige Bibliotheken verfügbar:

- Latte¹,
- Twig²,
- Smarty³,
- mustache⁴,
- Plates⁵.

Andere Template-Systeme sind Teil von PHP-Frameworks:

- Blade (Teil des Laravel Frameworks)⁶,
- Volt (Teil des phalcon Frameworks)⁷.

Die Auswahl des Template-Systems ist vielfach persönliche Präferenz oder hängt vom genauen Einsatzgebiet ab. Das Erlernen eines Systems ist jedoch in der Regel ein guter Grundstein, um auch einen schnellen Einstieg in andere Template-Engines zu finden. Im Folgenden wird Latte vorgestellt, ein intuitives, sicheres und sehr modernes Template-System.

6.1.3 Latte Template Engine

In den letzten Jahren hat sich *Latte* als extrem sichere und performante Template-Engine einen Namen gemacht. Latte zeichnet sich besonders durch sein „Context-Aware Escaping“ aus. Das bedeutet, die Engine erkennt automatisch, ob eine Variable in einem HTML-Text, einem HTML-Attribut oder gar in einem JavaScript-Block ausgegeben wird und wendet selbstständig die korrekten Sicherheitsmechanismen an, um Cross-Site Scripting (XSS, siehe Vorlesung 7) zu verhindern.

Die Syntax von Latte orientiert sich stark an nativem PHP, was den Einstieg deutlich erleichtert. Die aktuelle Version ist 3.1.4 vom 23. April 2026.

Installation

Latte ist eine moderne und modular aufgebaute PHP-Anwendung und besteht aus über 170 PHP-Dateien (Klassen, Interfaces etc.). Damit dieses System funktioniert, spielt Autoloading eine zentrale Rolle, damit die Klassen untereinander gefunden werden. Die bevorzugte Installationsweise⁸ für Latte ist daher mittels Composer, da dieser das Autoloading für Komponenten bereits beinhaltet.

```
1 $ composer require latte/latte
```

¹<https://latte.nette.org/>

²<https://twig.symfony.com/>

³<https://www.smarty.net/>

⁴<https://mustache.github.io/>

⁵<https://platesphp.com/>

⁶<https://laravel.com/docs/master/blade>

⁷<https://docs.phalcon.io/latest/volt/>

⁸<https://latte.nette.org/en/develop#toc-installation>

Funktionsweise und Konfiguration

HTML-Inhalte werden bei Template-Systemen in Dateien ausgelagert. Bei Latte haben diese üblicherweise die Endung `.latte`. Um diese in PhpStorm sinnvoll verwenden zu können, empfiehlt sich das Plugin *Latte Support*⁹.

Ein Template ist zunächst wie eine statische HTML-Datei. Es gibt jedoch Platzhalter für Variablen, die von PHP aus beim Rendern des Templates befüllt werden. Somit bleibt die dynamische Natur, die auch bei der nativen Ausgabe mit PHP vorherrscht, erhalten.

Um Templates mit Latte anzuzeigen, muss ein zentrales Engine-Objekt angelegt werden. Es ist empfehlenswert, ein Verzeichnis für den Cache anzugeben (hier `cache`, da Latte die Templates für maximale Performance im Hintergrund in reinen PHP-Code kompiliert. Während der Entwicklung erkennt Latte Änderungen an den `.latte`-Dateien automatisch und kompiliert diese bei Bedarf neu. Weiters kann noch explizit ein `FileLoader` mit dem Pfad der Template-Dateien angegeben werden. Dies schränkt einerseits das Laden von Template-Dateien auf ein Verzeichnis ein (hier `templates`) und erspart die Angabe des gesamten Verzeichnispfades.

Der initiale Konfigurationsaufwand sieht wie folgt aus:

```
1 use Latte\Engine;
2 use Latte\Loaders\FileLoader;
3
4 $latte = new Engine();
5 $latte->setLoader(new FileLoader(__DIR__ . "/templates"));
6 $latte->setCacheDirectory(__DIR__ . "/cache");
```

Ausgabe, Variablen und Zuweisung

Latte verfügt über die Methode `render($name, $params)`¹⁰, um das Template anzuzeigen. Der erste Parameter ist der Pfad zum anzuzeigenden Template, der zweite ein Array oder Objekt mit den zu übergebenden Parametern. Soll das Template nur gerendert, d. h. dessen HTML-Code erzeugt und als String zurückgegeben werden, kann `renderToString($name, $params)` verwendet werden. Der folgende Aufruf zeigt das Template `message.latte` an und weist die Parameter `name` und `message` im Template zu:

```
1 $latte->render("message.latte", [
2     "name" => "John Doe",
3     "message" => "I'm back baby!",
4 ]);
```

Damit die angegebenen Parameter im Template auch angezeigt werden, müssen Platzhalter gesetzt werden. Latte nutzt dafür einfache geschwungene Klammern mit dem Variablennamen (inklusive dem Dollar-Zeichen, genau wie die Curly Syntax in PHP):

```
1 <p>Name: {$name}</p>
```

⁹<https://plugins.jetbrains.com/plugin/24218-latte-support>

¹⁰<https://latte.nette.org/en/develop#toc-how-to-render-a-template>

Beispiel

Dies alles wird in einem Beispiel demonstriert. Die Datei `message.latte` enthält das HTML-Template, die Datei `latte.php` nimmt die Konfiguration vor und zeigt das Template an.

Das Template enthält die beiden Platzhalter `{ $name }` und `{ $message }`:

```
1 <!DOCTYPE html>
2 <html lang="en">
3   <head>
4     <title>Message</title>
5     <meta charset="utf-8">
6   </head>
7   <body>
8     <p>Name: { $name }</p>
9     <p>Message: { $message }</p>
10  </body>
11 </html>
```

Die PHP-Datei initialisiert die Engine und zeigt das Template an:

```
1 use Latte\Engine;
2 use Latte\Loaders\FileLoader;
3
4 $latte = new Engine();
5 $latte->setLoader(new FileLoader(__DIR__ . "/templates"));
6 $latte->setCacheDirectory(__DIR__ . "/cache");
7
8 $latte->render("message.latte", [
9   "name" => "John Doe",
10  "message" => "I'm back baby!",
11 ]);
```

Die Ausgabe ist das folgende HTML-Markup, das Ergebnis ist in Abbildung 6.1 ersichtlich.

```
1 <!DOCTYPE html>
2 <html lang="en">
3   <head>
4     <title>Message</title>
5     <meta charset="utf-8">
6   </head>
7   <body>
8     <p>Name: John Doe</p>
9     <p>Message: I'm back baby!</p>
10  </body>
11 </html>
```

Fortgeschrittene Variablenübergabe

In Latte können zusammengesetzte Datentypen exakt wie in nativem PHP abgefragt werden.

Arrays und Objekte: Der Zugriff auf Arrays im Template ist analog zu PHP möglich:

```
1 $array = [
2   "John Doe",
```

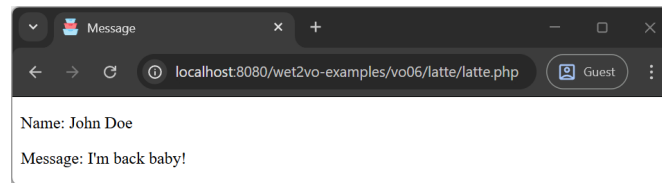


Abbildung 6.1: Bei der Ausgabe der Seite ist nicht ersichtlich, dass Latte im Hintergrund arbeitet. Aufgerufen wird ja nach wie vor die PHP-Seite (hier `latte.php`).

```
3  "male",
4  25,
5 ];
6 $latte->render("array-object.latte", [
7  "data1" => $array,
8 ]);
```

In der Template-Datei kann dann über die gewohnte Syntax auf die einzelnen Stellen zugegriffen werden:

```
1 <p>Name: {{$data1[0]}}</p>
2 <p>Gender: {{$data1[1]}}</p>
3 <p>Age: {{$data1[2]}}</p>
```

Auch bei assoziativen Arrays und Objekten bleibt die Syntax konsistent. Folgendes Array soll ans Template übergeben werden:

```
1 $assocArray = [
2  "name" => "Jane Doe",
3  "details" => [
4    "gender" => "female",
5    "age" => 23,
6  ],
7 ];
8 $latte->render("array-object.latte", [
9  "data2" => $assocArray,
10 ]);
```

Der Zugriff im Template:

```
1 <p>Name: {{$data2["name"]}}</p>
2 <p>Gender: {{$data2["details"]["gender2"]}}</p>
3 <p>Age: {{$data2["details"]["age"]}}</p>
```

Auch Objekte können an Latte-Templates übergeben werden. Auf die Eigenschaften eines Objekts wird über den `->`-Operator zugegriffen. Zunächst wird ein Objekt angelegt und übergeben:

```
1 $object = new Person(
2  "Jim Doe",
3  "male",
4  3,
5 );
6 $latte->render("array-object.latte", [
7  "data3" => $object,
8 ]);
```

Im Template wird auf die einzelnen (public)-Eigenschaften wie folgt zugegriffen:

```

1 <p>Name: {$data3->name}</p>
2 <p>Gender: {$data3->gender}</p>
3 <p>Age: {$data3->age}</p>

```

Filter: Um zugewiesene Variablen im Template noch weiter zu modifizieren, existieren *Filter*. Diese werden mit dem |-Zeichen (Pipe) an die Variable gehängt. Dabei können auch mehrere Filter angewandt und Parameter an den Filter übergeben werden.

Latte bietet eine Vielzahl eingebauter Filter¹¹. Einige davon sind:

- Groß-/Kleinschreibung: `capitalize`, `upper`, `lower`, `firstUpper`, `firstLower`,
- Transformation: `stripHtml`, `breakLines`, `trim`, `sort`, `date`,
- Escaping: `noescape`, `escapeUrl`, `query`,
- ...

Das folgende Beispiel formatiert einige Werte:

```

1 <p>Name: {$name|lower}</p>
2 <p>Message: {$message|stripHtml|trim}</p>
3 <p>Timestamp: {=now|date:'d.m.Y H:M'}</p>

```

Hinweis: Latte erlaubt das direkte Aufrufen von PHP-Funktionen (wie `date()`) innerhalb der Tags `{}`: `{date("d.m.Y, H:i:s")}`

Tags und Kontrollstrukturen

Mit Latte lässt sich Logik im Template hervorragend abbilden. Auch hier orientieren sich die Tags¹² an PHP.

Schleifen: Mit `{foreach}`¹³ lassen sich Schleifen über komplette Arrays realisieren.

```

1 <ul>
2   {foreach $colors as $color}
3     <li>{$color}</li>
4   {/foreach}
5 </ul>

```

Anstatt Tags zu verwenden, kann auch die *n:attributes*-Syntax verwendet werden. Dabei wird die Logik als HTML-Attribut verpackt. Latte erkennt dies beim Kompilieren und wandelt es in korrektes HTML um. Eine `foreach`-Schleife sieht damit wie folgt aus:

```

1 <ul>
2   <li n:foreach="$colors as $color">{$color}</li>
3 </ul>

```

Auch eine klassische `for`-Schleife mit einer Zählervariable kann in Latte verwendet werden:

```

1 <ul>
2   {for $i = 1; $i <= 3; $i++}
3     <li>{$i}</li>
4   {/for}
5 </ul>

```

¹¹<https://latte.nette.org/en/filters>

¹²<https://latte.nette.org/de/tags>

¹³<https://latte.nette.org/en/tags#toc-loops>

Auch diese Schleife kann als *n:attribute* formuliert werden:

```
1 <ul>
2   <li n:for="$i = 1; $i <= 3; $i++">{$i}</li>
3 </ul>
```

Innerhalb einer `foreach`-Schleife stellt Latte die implizite Variable `$iterator` zur Verfügung, mit der sich etwa abfragen lässt, ob es der erste (`$iterator->first`) oder letzte (`$iterator->last`) Durchlauf ist.

Bedingungen: Bedingungen werden mit `if`, `elseif` und `else` abgebildet.

```
1 {if $gender === "male"}
2   <p>Welcome Sir!</p>
3 {elseif $gender === "female"}
4   <p>Welcome Madam!</p>
5 {else}
6   <p>Welcome non-binary Person!</p>
7 {/if}
```

Mit `n:if` und `n:else` existieren für einfache Bedingungen auch *n:attributes*. Ein `n:elseif` gibt es jedoch nicht, weshalb das obige Beispiel nur mit Tags abgebildet werden kann.

Soll überprüft werden, ob eine Variable existiert (d. h. ins Template übergeben wurde), bietet Latte den speziellen `ifset`-Tag:

```
1 {ifset $name}
2   <p>Welcome {$name}!</p>
3 {/ifset}
```

Diese kann wiederum kürzer formuliert werden:

```
1 <p n:ifset="$name">Welcome {$name}!</p>
```

Weitere Templates inkludieren: Um Templates modular aufzubauen (z. B. Header und Footer auszulagern), nutzt Latte die `include`-Anweisung¹⁴.

```
1 {include "filename.latte"}
```

Zusätzlich können beliebige Variablen direkt bei der Einbindung übergeben werden. Parameter werden durch Kommas getrennt und mit dem `=>` Operator zugewiesen:

```
1 // main.latte
2 <!DOCTYPE html>
3 <html lang="en">
4   <head>
5     <title>Hello World</title>
6   </head>
7   <body>
8     {include "body.latte", heading => "Welcome to the site!"}
9   </body>
10 </html>
```

```
1 // body.latte
2 <h1>{$heading}</h1>
3 <p>Hello World!</p>
```

¹⁴<https://latte.nette.org/en/tags#toc-include>

Latte bietet noch weitere Funktionalitäten und lässt sich auch exzellent erweitern. All diese Dinge finden sich in der offiziellen Latte Dokumentation [13].

6.2 Routing

Routing im Kontext der Webentwicklung ist ein Mechanismus, bei dem HTTP-Requests, d. h. der Aufruf von bestimmten URLs, an den jeweils definierten Code, der sich um deren Abarbeitung kümmert, weitergeleitet werden. Einfach ausgedrückt bedeutet dies, dass Routing bestimmt, was passiert, wenn Nutzer*innen bestimmte URLs aufrufen.

6.2.1 Allgemeines

In seiner grundsätzlichen Konfiguration verlässt sich PHP beim Routing auf den Webserver. Dabei wird der Pfad des Requests, also der Teil nach der Domain oder IP-Adresse, direkt auf ein Verzeichnis am Webserver und die darin enthaltenen Dateien abgebildet.

Beim URL `https://www.example.com/mywebapp/index.php` ergeht der Request an den Server, der unter `www.example.com` verfügbar ist. Der Webserver sucht danach im Ordner `mywebapp` eine Datei `index.php`, übergibt diese an die PHP-Engine und nach Ausführung des Inhalts wird die HTML-Antwort (Response) zurückgegeben. Genau so läuft diese auf einem Entwicklungsserver ab: `http://localhost:8080/wet2vo-examples/vo06/routing/index.php` sendet den Request an den Server unter `localhost` am Port 8080. Dort muss – ausgehend vom Web-Root, das üblicherweise `var/www/html` lautet) – die Verzeichnishierarchie `wet2vo-examples/vo06/routing` existieren und darin die Datei `index.php` liegen, damit die Datei ausgeführt und eine Antwort zurückgegeben werden kann. Existiert unter dem angegebenen Pfad keine Datei, so liefert der Webserver die Antwort `404: Not found`, da die Ressource nicht vorhanden war.

Dieser Mechanismus ist einfach und trägt nicht zuletzt auch zum Erfolg von PHP bei. Dieses Prinzip ist schnell erlernt und verstanden, der Webserver muss dabei nicht aufwändig konfiguriert werden. Wurde eine PHP-Datei erstellt, kann sie am Webserver in den gewünschten Ordner gelegt werden und ist darunter sofort verfügbar.

Diese Einfachheit hat jedoch auch den Nachteil, dass sich der URL und die Verzeichnisstruktur immer decken müssen. Zudem hört jeder URL zwangsläufig mit einer PHP-Datei auf, denn diese enthält ja den Code, der aufgerufen werden muss. Eine Webseite, die also aus Hauptseite, Registrierung, Login und Logout besteht, benötigt die folgenden PHP-Dateien, die etwa unter diesen URLs aufgerufen werden können:

- `https://www.example.com/mywebapp/index.php`: Hauptseite
- `https://www.example.com/mywebapp/register.php`: Registrierung
- `https://www.example.com/mywebapp/login.php`: Login
- `https://www.example.com/mywebapp/logout.php`: Logout

Nehmen wir jedoch an, es besteht nun die Anforderung, folgende URLs (genau in dieser Schreibweise) zu erreichen:

- `https://www.example.com/mywebapp/`: Hauptseite
- `https://www.example.com/mywebapp/account/register`: Registrierung
- `https://www.example.com/mywebapp/account/login`: Login
- `https://www.example.com/mywebapp/account/logout`: Logout

Der vielleicht zunächst in den Sinn kommende Ansatz wäre, im Ordner `mywebapp` den Unterordner `account` anzulegen und darin wiederum `register`, `login` und `logout`. Die PHP-Dateien im Root-Ordner und den Unterordnern müssten dann jeweils `index.php` heißen, somit müssten sie nicht angegeben werden, denn ein Webserver sucht bei einer reinen Verzeichnisangabe immer nach Dateien mit dem Namen `index` und verschiedenen Endungen und ruft diese automatisch auf, wenn er sie findet.

Dieses Prinzip funktioniert hier zwar, wird jedoch schnell umständlich. Denn für jede Pfadebene müsste tatsächlich am Webserver ein Verzeichnis angelegt werden, darüber hinaus funktioniert ein URL wie `https://www.example.com/mywebapp/index.php` dann auch und das ist eigentlich nicht gewünscht. Und dann gibt es womöglich auch noch URLs, deren genauer Inhalt gar nicht bekannt ist. Soll etwa ein Produkt mit einer bestimmten ID abgefragt werden, so könnte die URL `https://www.example.com/mywebapp/product/4` lauten, um das Produkt mit der ID 4 anzuzeigen. Auch hier bräuchte es für jede ID ein Verzeichnis – wenn Produkte dynamisch aus einer Datenbank kommen, ein nicht mehr wartbares Unterfangen.

Es benötigt also Routing, welches unabhängig von einer physischen Verzeichnisstruktur am Server funktioniert. Dieses wird im folgenden Abschnitt beschrieben.

6.2.2 Funktionsweise

Um beliebige URLs in der eigenen Webapplikation zu ermöglichen, diese also von der Verzeichnisstruktur unabhängig zu gestalten, müssen alle Requests an einer Stelle zusammenlaufen, um sie dort zentral zu behandeln und zu entscheiden, ob es für diesen Pfad etwas anzuzeigen gibt, oder nicht. Dies ist ein bekanntes Prinzip in der Webapplikationsentwicklung, ein sogenanntes Entwurfsmuster (engl. „Design Pattern“) namens *Front Controller* [9, S. 344]. Dabei übernimmt ein Objekt – in PHP kann dies auch eine Datei sein, wenn Objektorientierung nicht zum Einsatz kommt – die Abarbeitung aller eingehenden Requests und entscheidet, welche Aktionen aufgerufen werden.

Ein Router in einer Webapplikation repräsentiert eben diesen Front Controller. Sein Code wird in der Regel in `index.php` im Root-Verzeichnis der Webapplikation platziert. Damit jedoch alle Requests – egal mit welcher Pfadangabe – dorthin umgeleitet werden, bedarf es zunächst einer Einstellung am Webserver.

Requests auf `index.php` umschreiben

Die hierfür benötigte Technologie nennt sich URL-Rewriting. Dabei werden mit Hilfe eines Webserver-Moduls die eingehenden Requests verändert, d. h. umgeschrieben.

Für den Apache Webserver – dieser kommt in den Docker-Containern zum Einsatz – muss dazu das Modul `mod_rewrite` aktiviert sein und für das Web-Verzeichnis die Direktive `AllowOverride All` gesetzt sein. Dies erlaubt, dass eine spezielle Datei namens `.htaccess` erstellt werden kann, die entsprechende Rewrite-Regeln enthält. Diese Datei wird dann ins Root-Verzeichnis der Webapplikation gelegt und sieht wie folgt aus:

```
1 <IfModule mod_rewrite.c>
2 RewriteEngine On
3
4 RewriteCond %{REQUEST_URI}::$1 ^(/.+)/(.*):~2$
5 RewriteRule ^(.*) - [E=BASE:%1]
```

```
6
7 RewriteCond %{REQUEST_FILENAME} !-f
8 RewriteRule ^ index.php [QSA,L]
9 </IfModule>
```

Zunächst wird überprüft, ob `mod_rewrite` aktiv ist und falls ja, die Rewrite-Engine aktiviert. In den Zeilen 4 und 5 wird dann das Basisverzeichnis ermittelt. Liegt das Projekt nämlich in einem Unterverzeichnis, muss dieses vor die anschließende Regel gesetzt werden. In Zeile 7 wird schließlich als Bedingung abgefragt, dass die Datei nicht existiert, in diesem Fall wird dann `index.php` in Zeile 8 aufgerufen. Das Argument `QSA` (Query String Append) gibt an, dass GET-Parameters angehängt werden, `L` (Last) stoppt die Verarbeitung von Regeln nach der aktuellen.

Die aktuelle Route ermitteln

Nachdem nun alle Requests auf `index.php` umgeschrieben wurden, kann die Verarbeitung in der PHP-Datei beginnen. Dazu werden nun zunächst die verwendete HTTP-Methode und der Pfad des Requests abgefragt. Beide Dinge sind im superglobalen Array `$_SERVER` vorhanden und können dort ausgelesen werden:

```
1 $method = $_SERVER["REQUEST_METHOD"];
2 $path = $_SERVER["REQUEST_URI"];
```

Die Methode ist ein String und in den einfachen Fällen „GET“ oder „POST“. Jeder einfache Request ist dabei ein GET-Request, POST kommt etwa beim Versenden von Formularen zum Einsatz. Es existieren auch noch weitere HTTP-Methoden, diese sind jedoch für diese Lehrveranstaltung noch nicht relevant und können etwa unter [28] nachgelesen werden.

Der Pfad ist ebenfalls ein String und enthält den gesamten URI ohne Domain- oder IP-Angabe. Bei `http://localhost:8080` ist dies einfach nur `/`, da es sich um das Root-Verzeichnis am Server (die oberste Ebene) handelt. Bei `http://localhost:8080/wet2vo-examples/vo06/routing/` ist dies `/wet2vo-examples/vo06/routing/`.

Hier zeigt sich nun schon das erste Problem: liegt die Applikation in einem oder mehreren Unterordnern, so gibt PHP diese natürlich mit aus. Die Routen sollen aber relativ zum Ordner des Projekts – in diesem Fall `routing` – sein und nicht zum Root-Verzeichnis. Daher ist die Einführung eines Basispfades (engl. „base path“) sinnvoll, der die Verzeichnisstruktur vom Root-Ordner bis zum Projektordner enthält. Mit ihm können später diese Teile des Pfades leicht entfernt werden.

```
1 $basePath = "/wet2vo-examples/vo06/routing";
```

Mit diesen Bestandteilen kann nun die Route gebildet werden. Sie besteht also aus Protokoll und Pfad. Der Basispfad wird dabei vorher mittels `substr()` entfernt, sodass nur noch der Relevante Teil übrig bleibt.

```
1 if (!empty($basePath)) {
2     $path = substr($path, strlen($basePath));
3 }
4
5 $route = "$method $path";
```


`$route` enthält nun also einen String mit Protokoll und Pfad. Für einen GET-Aufruf von `http://localhost:8080/wet2vo-examples/vo06/routing/` ist dies `GET /`, für einen GET-Aufruf von `http://localhost:8080/wet2vo-examples/vo06/routing/other` ist es `GET /other`.

Mit diesen Routen lässt sich nun das Verhalten des Front-Controllers steuern.

Aktionen für Routen ausführen

Im einfachsten Fall entscheidet nun ein `switch`-Statement, welcher Code ausgeführt wird. Verschiedene Fälle (`case`) enthalten Routen und den Code, der ausgeführt oder angezeigt werden soll. Im einfachsten Fall ist dies eine Ausgabe mit `echo`, es können aber genauso PHP-Dateien eingebunden (und somit ausgeführt) werden oder Latte-Templates angezeigt werden.

Trifft keine der Abfragen im `switch`-Statement zu, wird der `default`-Abschnitt ausgelöst. In ihm wird der HTTP-Statuscode 404 gesetzt und ausgegeben, dass keine Seite (oder Route) gefunden wurde. Damit ist die Abarbeitung beendet.

```
1 switch ($route) {  
2     case "GET /":  
3         echo "Here's the root page";  
4         break;  
5     case "GET /other":  
6         echo "Here's the other page.";  
7         break;  
8     default:  
9         http_response_code(404);  
10        echo "Page not found!";  
11 }
```

Auf dieselbe Art und Weise können beliebig viele Routen definiert werden, auch mit anderen Methoden wie etwa POST. Dieser gesamte Prozess ist in Abbildung 6.2 dargestellt.

Diese Art des Routings ist zwar simpel, jedoch nicht sehr flexibel, denn die Logik des Routings wird mit der eigenen Applikation vermischt. Aus diesem Grund empfiehlt es sich, bestehende Routing-Packages zu verwenden. Abschnitt 6.2.3 stellt das Paket `fhoee/router` vor, welches die gerade gezeigte Funktionalität objektorientiert umsetzt und für die Verwendung in dieser Lehrveranstaltung eine gute Basis darstellt. Für aufwändigere Ansprüche im Produktivbetrieb sind dann jedoch Router mit mehr Funktionalität gefordert; diese werden in Abschnitt 6.2.4 vorgestellt.

6.2.3 Einfaches Routing mit `fhoee/router`

`fhoee/router` ist eine PHP-Komponente, die das soeben beschriebene Prinzip des Routings objektorientiert umsetzt. Der Router wurde dabei bewusst für das Erlernen des Routings simpel gehalten und verfügt dabei nur über die grundsätzlichen Funktionen. Das Projekt ist open-source auf GitHub¹⁵ und über Packagist¹⁶ verfügbar.

¹⁵<https://github.com/Digital-Media/fhoee-router>

¹⁶<https://packagist.org/packages/fhoee/router>

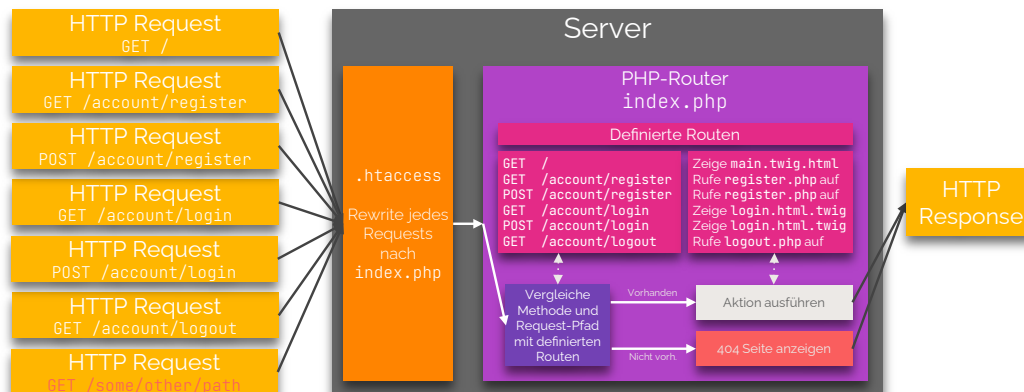


Abbildung 6.2: Das Prinzip des Routings. Alle Requests werden über die `.htaccess`-Datei auf `index.php` umgeleitet. Dort werden Methode und Pfad mit den im Router hinterlegten Routen verglichen. Gibt es eine Übereinstimmung, wird die bei der Route hinterlegte Aktion ausgeführt, die eine Response erzeugt. Gibt es keine Übereinstimmung, wird eine 404-Seite als Response zurückgegeben.

Verwendung und Ablauf von `fhooe/router`

Um `fhooe/router` im eigenen Projekt einzusetzen, muss die Komponente zunächst mittels Composer installiert werden.

```
1 $ composer require fhooe/router
```

Dies erzeugt die bereits in Vorlesung 5 vorgestellten Dateien `composer.json` und `composer.lock` sowie den Ordner `vendor`, in dem sich im Unterordner `fhooe/router/src` die Datei `Router.php` befindet. Sie enthält den gesamten Code des Routers.

Um den Router einzusetzen, wird nun im Projektordner eine Datei `index.php` angelegt und ebenfalls eine Datei `.htaccess` mit dem in Abschnitt 6.2.2 beschriebenen Inhalt. Letztere schreibt wiederum alle Requests auf `index.php` um. Ohne sie kann der gesamte Prozess nicht funktionieren.

`fhooe/router` bildet in der Klasse den in Abschnitt 6.2.2 beschriebenen Ablauf ab. Die Klasse kümmert sich also um das Erfragen der Methode und des Request-Pfades. Jedoch wird die Abfrage, ob dies einer definierten Route entspricht, nicht mehr wie gezeigt in einem `switch`-Statement abgehandelt. Vielmehr ermöglicht es die Klasse `Router`, Routen zu hinterlegen, die dann im Objekt gespeichert, bei der Ausführung verglichen und abgearbeitet werden. Die Arbeitsschritte sind also wie folgt:

1. Lege ein Router-Objekt an. Zuvor `autoload.php` aus dem `vendor`-Verzeichnis einbinden, damit die Klasse gefunden wird.
2. Optional: Setze einen Basispfad, falls `index.php` nicht auf der untersten (Root) Ebene des Webservers liegt, sondern in einem Unterverzeichnis.
3. Definiere alle gewünschten Routen und die dazugehörigen Aktionen, die eine Response erzeugen.
4. Definiere eine 404-Aktion, falls ein Pfad aufgerufen wird, für den es keine Route gibt.

5. Starte den Routing-Mechanismus.

Diese Schritte werden nun in den folgenden Abschnitten im Detail erläutert.

Ein Router-Objekt anlegen: Um ein Objekt von `fhoee/router` anzulegen, wird zunächst `autoload.php` aus dem `vendor`-Verzeichnis eingebunden. Somit ist das Autoloading von Composer aktiviert und alle benötigten Klassen werden automatisch gefunden. Danach wird mit `new` ein Objekt erzeugt. Der Konstruktor hat keine Parameter.

```
1 require __DIR__ . "/vendor/autoload.php";  
2  
3 $router = new Fhoee\Router\Router();
```

Um die Angabe des Namespaces bei der Objekt-Initialisierung zu vermeiden, kann alternativ ein `use`-Statement am Beginn der Datei eingesetzt werden. Die Initialisierung sieht dann wie folgt aus:

```
1 use Fhoee\Router\Router;  
2  
3 require __DIR__ . "/vendor/autoload.php";  
4  
5 $router = new Router();
```

Basispfad setzen: Befindet sich die Applikation in einem Unterverzeichnis des Webserver, so muss der Router diesen Pfad kennen. Im folgenden Szenario liegen die Dateien (also `index.php` und `.htaccess`) im Docker-Container unter `wet2vo-examples ▶ vo06 ▶ fhoee-router`. Dieser Teil des Verzeichnisses wird zum Basispfad. In Version 3.0 von `fhoee/router` wird dieser direkt über die öffentliche Eigenschaft `$basePath` gesetzt. Der Pfad muss dabei mit einem Schrägstrich (Slash) beginnen und darf am Ende keinen aufweisen:

```
1 $router->basePath = "/wet2vo-examples/vo06/fhoee-router";
```

Routen definieren: Der wohl wichtigste Schritt ist das Festlegen von Routen. Eine Route besteht im Falle der `fhoee/router`-Komponente aus drei Dingen:

1. Der HTTP-Methode (GET oder POST).
2. Dem Pfad, auch Muster (engl. „pattern“) genannt.
3. Einer Aktion, welche die Response dieser Route, d. h. Ausgabe, erzeugt. Diese Aktion wird in Form einer Callback-Funktion hinterlegt, die vom Router aufgerufen wird, wenn die Route zutrifft.

Durch `addRoute(HttpMethod $method, string $pattern, Closure $callback)` wird eine Route hinzugefügt. Die Methode enthält die drei genannten Teile als Parameter. `$method` ist vom Typ `HttpMethod`, ein Enum mit den Werten `HttpMethod::GET` und `HttpMethod::POST`. Der Typ `Closure`¹⁷ für den Callback repräsentiert dabei eine anonyme Funktion und bietet dabei etwas mehr Kontrolle als der ebenfalls verfügbare Typ `callable`, der auch normale Funktionen zulässt.

¹⁷<https://www.php.net/manual/de/class.closure.php>

Für die beiden in dieser Lehrveranstaltung relevanten HTTP-Methoden GET und POST existieren jedoch zwei Shorthand-Methoden, die die Angabe der Methode vereinfachen: `get(string $pattern, Closure $callback)` und zusätzlich der Shorthand `post(string $pattern, Closure $callback)`.

Mit ihnen können nun Routen definiert werden:

```
1 $router->get("/", function () {
2     require "views/main.php";
3 });
4
5 $router->get("/form", function () {
6     require "views/form.php";
7 });
8
9 $router->post("/form", function () {
10    require "views/form.php";
11 });
```

Diese Aufrufe definieren drei Routen:

- Eine GET-Route, die auf beim Aufruf des Root-Verzeichnisses des Projekts ausgelöst wird. Sie wird durch `/` repräsentiert und ergibt mit dem Basispfad dann den URL `http://localhost:8080/wet2vo-examples/vo06/fhooe-router/`. Wird dieser URL aufgerufen, wird der Inhalt der Datei `views▶main.php` ausgegeben, die durch einen `require`-Aufruf eingebunden wird.
- Eine GET-Route, die beim Aufruf des Pfades `/form` ausgelöst wird. Sie wird durch den URL `http://localhost:8080/wet2vo-examples/vo06/fhooe-router/form` aufgerufen und zeigt ein Formularfeld (aus `views▶form.php`) an, welches eine Nachricht mit POST abschickt.
- Eine POST-Route, die beim Aufruf des Pfades `/form` ausgelöst wird. Sie wird ebenfalls durch `http://localhost:8080/wet2vo-examples/vo06/fhooe-router/form` aufgerufen, jedoch nur, wenn der Request mit POST, d. h. durch Absenden des Formulars, ausgelöst wurde.

Die Hauptdatei `views▶main.php` sieht wie folgt aus:

```
1 <?php
2
3 global $router;
4 ?>
5
6 <!DOCTYPE html>
7 <html lang="en">
8     <head>
9         <meta charset="UTF-8">
10        <title>Main Page</title>
11    </head>
12    <body>
13        <h1>Main</h1>
14        <p>This is the main page!</p>
15
16        <p>Why not also try the <a href="<?=$router->urlFor("/form") ?>">form</a>?</p>
17    </body>
18 </html>
```

Zu Beginn wird das Router-Objekt global eingebunden. Da die Datei `main.php` mit `require` in `index.php` eingebunden wird, kann das dort angelegte Objekt mittels `global`-Statement ganz einfach verfügbar gemacht werden.

Anschließend erfolgt die Ausgabe einer fast vollständig statischen HTML-Seite. Lediglich beim Link zum Formular findet sich PHP-Code: `$router->urlFor("/form")`. Die Methode `urlFor()` erhält als Argument eine Route – in diesem Fall `/form` – und löst diese in einen vollständigen URL auf, indem ein eventuell gesetzter Basispfad wieder davor gesetzt wird. Es entsteht also `http://localhost:8080//wet2vo-examples/vo06/fhooe-router/form`. Dies ist notwendig, denn würde nur `/form` als Wert des `href`-Attributs gesetzt, würde der Schrägstrich den Link ausgehend vom Root des Webserver setzen, wodurch `http://localhost:8080/form` entstünde, was nicht korrekt und gewünscht ist. Befindet sich die Applikation tatsächlich im Root des Webserver und nicht in einem Unterverzeichnis, so ist dies natürlich möglich.

Die Formulardatei `views/form.php` sieht ähnlich aus. Auch hier wird zunächst das Router-Objekt global eingebunden, um es bei URLs verwenden zu können.

```

1 <?php
2
3 global $router;
4 ?>
5
6 <!DOCTYPE html>
7 <html lang="en">
8   <head>
9     <meta charset="UTF-8">
10    <title>Form Page</title>
11  </head>
12  <body>
13    <h1>Form</h1>
14    <p>This is the form page!</p>
15
16    <form action="<?= $router->urlFor("/form") ?>" method="post">
17      <label for="message">Message:</label>
18      <input type="text" name="msg" id="message">
19      <button type="submit">Send Message</button>
20    </form>
21
22 <?php
23 if (isset($_POST["msg"])) {
24   echo "<p>Message: {$_POST["msg"]}</p>";
25 }
26 ?>
27
28   <p>Why not also try the <a href="<?= $router->urlFor("/") ?>">main page</a>?</p>
29 </body>
30 </html>

```

Dann wird ein Formularfeld eingebunden. Im `action`-Attribut kommt wiederum die Methode `urlFor()` zum Einsatz, da auch hier der korrekte URL aufgelöst werden muss. Die Ausgabe des Formularinhalts erfolgt nur dann, wenn sich auch Daten im superglobalen Array `$_POST["msg"]` befinden.

404 Callback setzen: Neben den Routen darf auch der Fehlerfall nicht außer Acht gelassen werden. Ruft der*die User*in eine Route auf, die nicht im Router hinterlegt wurde, muss eine Fehlerseite angezeigt werden. Dies geschieht wie folgt:

```
1 $router->set404Callback(function () {
2     require "views/404.html";
3 });
```

Die Methode `set404Callback(Closure $callback)` erwartet wiederum eine anonyme Funktion, die eine 404-Antwort zurückgibt. In diesem Fall wird eine statische HTML-Seite, nämlich `views/404.html` eingebunden und somit zurückgegeben:

```
1 <!DOCTYPE html>
2 <html lang="en">
3   <head>
4     <meta charset="UTF-8">
5     <title>404</title>
6   </head>
7   <body>
8     <h1>404</h1>
9     <p>File not found.</p>
10  </body>
11 </html>
```

Die Klasse `Router` setzt dabei bereits selbst den HTTP Statuscode 404, dies muss also nicht extra geschehen.

Den Routing-Mechanismus starten: Nachdem nun im `Router`-Objekt alle notwendigen Informationen hinterlegt wurden, wird abschließend der Routing-Mechanismus gestartet:

```
1 $router->run();
```

Der Router beginnt nun den in Abbildung 6.2 dargestellten Ablauf. Er liest Methode und Pfad aus und vergleicht sie mit den zuvor hinterlegten Routen. Gibt es eine Übereinstimmung, so wird die hinterlegte Aktion aufgerufen und eine Response erzeugt. Wurden alle hinterlegten Routen überprüft und keine Übereinstimmung gefunden, wird der 404 Callback ausgelöst und diese Response generiert.

Weitere Funktionalitäten von `fhoee/router`

Im Folgenden werden noch weitere Methoden und Eigenschaften der `fhoee/router`-Klasse aufgelistet, die nützlich sein können:

public string \$basePath: Über diese Eigenschaft kann der Basispfad nicht nur gesetzt, sondern auch ausgelesen werden.

```
1 $basePath = $router->basePath;
```

public function redirect(string \$url, ?array \$par = null): void: Wird für eine Weiterleitung auf einen kompletten URL verwendet. Dadurch kann etwa auf eine externe Seite weitergeleitet werden. Mit dem optionalen Parameter `$par` können eventuelle GET-Parameter an den URL gehängt werden. Dabei wird automatisch der

Statuscode 302 Found gesetzt, der einen temporären Redirect anzeigt. Suchmaschinen können erkennen, dass es sich hier nicht um eine dauerhafte Weiterleitung handelt, die sie sich merken müssen.

```
1 $router->redirect("https://www.fh-ooe.at/");
```

`public function redirectTo(string $pattern): void:` Führt eine Weiterleitung auf einen Routingpfad durch. Dies ist sinnvoll für Redirects, die innerhalb der eigenen Seite geschehen. Der Pfad wird dabei zuvor in eine vollen URL aufgelöst und dann weitergeleitet.

```
1 $router->redirectTo("/form");
```

Basisprojekt fhooe/router-skeleton

Der soeben gezeigte Ablauf, mit dem die Komponente **fhooe/router** in ein eigenes Projekt integriert wird, ist universell einsetzbar. Die dabei notwendigen fünf Schritte sind jedoch immer dieselben. Aus diesem Grund, gibt es zum **fhooe/router** Paket ergänzend das **fhooe/router-skeleton**-Projekt. Es ist ebenfalls auf GitHub¹⁸ open-source verfügbar und über Packagist¹⁹ installierbar.

Da es sich nicht um eine Bibliothek (Library) handelt, sondern um ein Starter-Projekt, lautet der Composer-Aufruf hier anders:

```
1 $ composer create-project fhooe/router-skeleton path/to/install
```

`path/to/install` ist hierbei eine Pfadangabe, die noch nicht existieren darf. Der Pfad wird neu angelegt und das Starterprojekt wird dorthin installiert. Dies ist eigentlich nur ein verstecktes Git-Clone, weshalb auch der Pfad noch nicht existieren darf (denn Git klonet keine Repositories in bestehende Verzeichnisse).

Ausgehend vom Verzeichnis `wet2vo-examples▶vo06` wird somit das Unterverzeichnis **fhooe-router-skeleton** erzeugt. In ihm befinden sich alle zum Projekt gehörigen Dateien.

```
1 $ composer create-project fhooe/router-skeleton fhooe-router-skeleton
```

Nach dem Aufruf existieren in diesem Ordner die folgenden Unterverzeichnisse:

- **logs:** Verzeichnis für Applikations-Logdateien.
- **public:** Das Verzeichnis mit `index.php` und `.htaccess`. In einer Webserverkonfiguration für ein Produktivprojekt sollte **public** später das einzig öffentlich zugreifbare Verzeichnis sein.
- **views:** Verzeichnis mit den Views, also den Dateien, die bei den verschiedenen Routen angezeigt werden sollen. Dies können HTML-, PHP- oder auch Latte-Dateien sein.
- **vendor:** Verzeichnis mit allen benötigten Dateien wie dem **fhooe/router** oder dem **latte/latte** Paket.

¹⁸<https://github.com/Digital-Media/fhooe-router-skeleton>

¹⁹<https://packagist.org/packages/fhooe/router-skeleton>

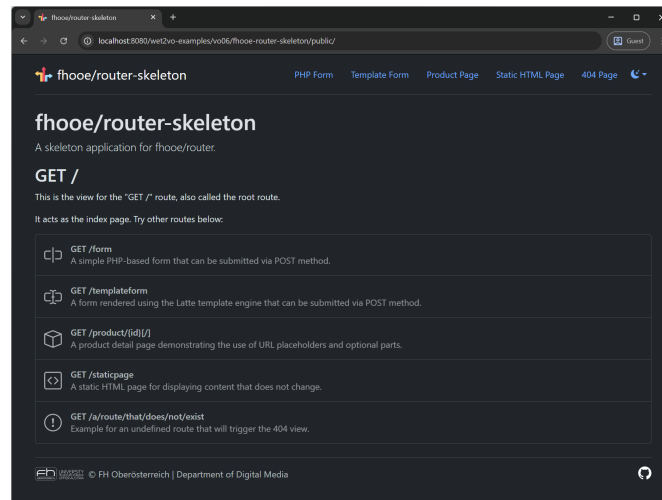


Abbildung 6.3: Die Startseite – auch als Root-Route bezeichnet – des `fhoove/router-skeleton` im dunklen Modus.

Damit das Projekt lauffähig ist, sollte zunächst in der Datei `public/index.php` der Basispfad in Zeile 49 gesetzt, d. h. angepasst werden. Das `public`-Verzeichnis muss hier mitbedacht werden.

```
1 $router->basePath = "/wet2vo-examples/vo06/fhoove-router-skeleton/public";
```

Danach kann dieser URL – hier `http://localhost:8080/wet2vo-examples/vo06/fhoove-router-skeleton/public/` im Browser geöffnet werden und zeigt die Startseite, wie in Abbildung 6.3 ersichtlich.

Sollte eine Fehlermeldung angezeigt werden, die etwas wie „Failed to open stream: Permission denied“ enthält, so darf Monolog aufgrund fehlender Berechtigungen nicht ins Verzeichnis `logs` schreiben. Dies geschieht meist auf Windows-Rechnern, bei denen Docker die Verzeichnisberechtigungen nicht korrekt mappen kann. In diesem Fall löst ein `chmod -R 777 fhoove-router-skeleton` das Problem.

Von hier aus können verschiedene bereits hinterlegte Routen ausprobiert und in `public/index.php` nachvollzogen werden:

- `GET /`: Zeige das Latte-Template `views/index.latte`.
- `GET /form`: Zeige ein Formular in der PHP-Datei `views/form.php`.
- `POST /form`: Zeige das Ergebnis des Formulars in der PHP-Datei `views/form.php`.
- `GET /templateform`: Zeige `views/templateform.latte`: ein Formular als Latte-Template.
- `POST /templateformresult`: Zeige das Ergebnis des Formulars im Latte-Template `views/templateformresult.latte`.
- `GET /product/{id}/[ ]`: Zeige die Seite `views/product.latte` für ein variables Produkt. Dabei ist `{id}` ein Platzhalter, der mit einem beliebigen Wert ersetzt und im Callback abrufbar ist. Der abschließende Schrägstrich ist durch das Setzen in eckige Klammern optional. Die Route funktioniert also mit und ohne.
- `GET /staticpage`: Zeige `views/staticpage.html`: eine statische HTML-Seite.

Latte-Templates verwenden: Der `fhoee/router-skeleton` definiert bereits alles Notwendige in `public/index.php`, um die Latte Template-Engine nahtlos mit dem Router zu verbinden. Die Latte-Engine wird instanziiert und das Verzeichnis für gecachte Templates gesetzt, die Template-Dateien selbst werden im Verzeichnis `views` erwartet. Weiters wird die Latte-Erweiterung `RouterExtension` hinzugefügt, welche die beiden Funktionen `url_for()` und `get_base_path()` definiert, damit diese auch in Template-Dateien verwendet werden können. Auch das superglobale Array `$_SESSION` (siehe Vorlesung 8) wird in den Templates durch die `SessionExtension` verfügbar gemacht.

```
1 $latte = new Engine();
2 $latte->setLoader(new FileLoader(__DIR__ . '/../views'));
3 $latte->setCacheDirectory(__DIR__ . '/../cache');
4 $latte->addExtension(new RouterExtension($router));
5 $latte->addExtension(new SessionExtension());
```

In einem Latte-Template kann danach beispielsweise der komplette URL zur Route `/form` ganz einfach über den Aufruf `{url_for("/form")}` dynamisch generiert werden.

Logging verwenden: Ebenfalls ist im Skeleton das Paket `monolog/monolog` eingebunden, welches einfach Logging-Aufruf in eine Datei absetzt. Das Paket wird ebenfalls in `public/index.php` initialisiert:

```
1 $logger = new Logger("skeleton-logger");
2 $logger->pushProcessor(new PsrLogMessageProcessor());
3 $formatter = new LineFormatter(
4     "[%datetime%] %channel%.%level_name%: %message%\n",
5     "d.m.Y H:i:s T",
6     true,
7     true,
8 );
9 $handler = new StreamHandler(__DIR__ . '/../logs/router.log");
10 $handler->setFormatter($formatter);
11 $logger->pushHandler($handler);
12
13 $router = new Router($logger);
```

Aufrufe des Loggers werden bereits in der Klasse `Router` vorgenommen und im Skeleton lediglich in die Datei weitergeleitet. Natürlich kann das `Logger`-Objekt auch jederzeit für eigene Logging-Aufruf verwendet werden:

```
1 $logger->info("Das ist eine Information.");
2 $logger->debug("Das ist eine Debug-Meldung.");
```

6.2.4 Komplexere Router

Wie bereits erwähnt, eignet sich `fhoee/router` für einfache Routing-Operationen und für das Erlernen des Prinzips, da er klein und übersichtlich gehalten wurde. Dies bedeutet aber selbstverständlich, dass der Funktionsumfang dementsprechend beschränkt ist. Darum sollte der Router auch nicht im Produktivbetrieb eingesetzt werden. Hierfür gibt es andere Komponenten.

bramus/router

Die Komponente **bramus/router** wurde von Bramus van Damme, einem Webentwickler aus Belgien entworfen. Das Projekt ist open-source auf GitHub²⁰ und auf Packagist²¹ und wird mit Composer installiert:

```
1 $ composer require bramus/router
```

Der Router ist ebenfalls einfach gehalten – er besteht nur aus einer Klasse – weist jedoch einen größeren Funktionsumfang auf. Folgende Dinge gehören dazu:

- Volle Unterstützung aller HTTP-Methoden wie GET, POST, PUT, DELETE, OPTIONS, PATCH und HEAD.
- Dynamische Routing-Muster (Pfade) über Regular Expressions.
- Untermuster für Routen, bei denen Teile optional gemacht werden können.
- Middleware Support (Code, der vor oder nach dem Routing ausgeführt werden kann).
- u. v. m.

Dieser Router eignet sich gut für kleine Projekte, wenn das eigene Wissen über Routing noch nicht allzu groß ist, denn viele der Dinge lassen sich gut im nur knapp über 500 Zeilen langen Kern nachvollziehen.

nikic/fast-route

Die Komponente **nikic/fast-route** wurde von Nikita Popov, einem Langzeit-Mitentwickler des PHP-Kerns, entworfen und stellt einen schnellen, auf regulären Ausdrücken basierenden Router dar. Das Projekt ist open-source auf GitHub²² und auf Packagist²³ verfügbar und wird über Composer installiert:

```
1 $ composer require nikic/fast-route
```

Der Fokus dieses Routers liegt auf Geschwindigkeit. Diese ist vor allem in umfangreichen Webapplikationen wichtig, wenn die Routing-Komponente möglicherweise hunderte verschiedene Routing-Muster zu verwalten hat. Wird nun der Routing-Mechanismus ausgelöst, so ist das Durchsuchen dieser vielen hinterlegten Routen möglicherweise ein langwieriger Prozess. **nikic/fast-route** löst dieses Problem mit Hilfe kombinierter Regular Expressions. Die Routen-Pfade werden als Regular-Expression hinterlegt, doch anstatt alle hinterlegten Muster einzeln zu durchlaufen, werden sie in ein großes Muster zusammengefügt und geklammert. Danach wird das `$matches`-Array durchlaufen: der erste Eintrag, der einen Wert enthält, ist die gesuchte Route. Nikita Popov beschreibt diesen Prozess ausführlich in einem Blog-Posting²⁴.

Die **nikic/fast-route** Komponente ist extrem umfangreich und zugleich performant, weshalb sie auch in vielen anderen Frameworks, wie etwa dem Slim Framework (siehe Vorlesung 12) eingesetzt wird. Sie eignet sich gut für große Projekte, das Verständnis von Routing sollte dementsprechend gut ausgeprägt sein.

²⁰<https://github.com/bramus/router>

²¹<https://packagist.org/packages/bramus/router>

²²<https://github.com/nikic/fastroute>

²³<https://packagist.org/packages/nikic/fast-route>

²⁴<https://www.npopov.com/2014/02/18/Fast-request-routing-using-regular-expressions.html>

league/route

Die Komponente `league/route` ist ein Router, der auf `nikic/fast-route` aufbaut und vollständige Unterstützung bzw. Implementierung vieler PSRs achtet. Dazu zählen PSR-4 (Autoloading), PSR-7 (HTTP Messages), PSR-11 (Container) und PSR-15 (HTTP Request Handler), PSR-16 (Caching) und PSR-17 (HTTP Factories). Sie ist open-source auf GitHub²⁵ und Packagist²⁶ verfügbar und wird mit Composer installiert:

```
1 $ composer require league/route
```

Die Ziele des Projekts bestehen darin, die performante `nikic/fast-route`-Komponenten benutzer*innenfreundlicher zu gestalten und durch strikte Einhaltung verschiedener PSRs bessere Kompatibilität und Austauschbarkeit in Projekten zu garantieren. Sie ist sicherlich derzeit die modernste PHP-Routing-Komponente, die sich für eine Vielzahl von Projekten eignet und mit dem entsprechenden Routing-Wissen eingesetzt werden kann.

6.3 Weiterführende Literatur

Zur Template-Engine Latte gibt es so gut wie keine brauchbare Literatur. Hier sei auf die offizielle Dokumentation [13] verwiesen, auch finden sich im Web zahlreiche Tutorials, welche die Abläufe erklären.

Auch zum Thema Routing ist die Literatur spärlich. In [37] wird in Kapitel 3 ein Router entworfen und dabei die Prinzipien erklärt. [51] greift das Prinzip des Front Controllers auf und demonstriert die Umsetzung anhand von PHP 8 Code. Ansonsten sei auf die diversen Ressourcen der jeweiligen Komponenten verwiesen, welche oft Einblicke in die Funktionsweise und das Design geben.

²⁵ <https://github.com/theleague/route>

²⁶ <https://packagist.org/packages/league/route>

Vorlesung 7

Formularverarbeitung

7.1 Wiederholung: Formulardatenverarbeitung

Die *Verarbeitung von Formulardaten* wurde bereits in Abschnitt 3.5 grundsätzlich besprochen. Hier wird nochmals eine kurze Wiederholung gegeben, bevor noch einige weitere Konzepte besprochen werden, die im Kapitel 3 noch ausgelassen wurden, jedoch für eine sichere und Verarbeitung von Daten unbedingt erforderlich sind.

7.1.1 Das HTML-Formular

Zur Formularübertragung wird ein HTML-Formular, bestehend aus dem `<form>`-Element, mindestens einem Formularelement (z. B. `<input>`) und meist einem Button zum Absenden (z. B. `<button>`) benötigt. Ein einfaches Formular sieht etwa wie folgt aus:

```
1 <form method="post" action="<?= $_SERVER["SCRIPT_NAME"] ?>">
2   <label for="datenlabel">Info:</label>
3   <input type="text" name="daten" id="datenlabel">
4   <button type="submit">Abschicken</button>
5 </form>
```

Dieses Formular verwendet die POST-Übertragungsmethode (siehe dazu Abschnitt 7.1.2) im `action`-Attribut und ruft sich selbst wieder auf. Dies wird durch das Einsetzen von `$_SERVER["SCRIPT_NAME"]` erreicht. Dieser Eintrag im superglobalen Array `$_SERVER` enthält den Pfad zum aktuellen Skript. Somit kann dieses frei umbenannt werden, ohne dass der Inhalt darauf angepasst werden muss.

7.1.2 Übertragungsmethode und Zugriff

Formulare können in PHP mit den HTTP Methoden GET oder POST übertragen werden. Angegeben wird dies im `action`-Attribut des Formulars mit den Werten `get` oder `post`. Der Unterschied liegt in der Übertragung der Inhalte.

Übertragung mittels GET

GET-Parameter werden in einem sogenannten Query-String an den URL gehängt. Dies geschieht im Format `?key1=value1&key2=value2`.

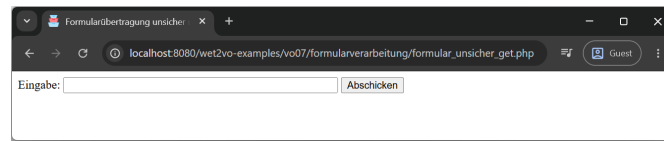


Abbildung 7.1: Das Formular zeigt ein einfaches Formularfeld zum Eingeben eines Wertes.

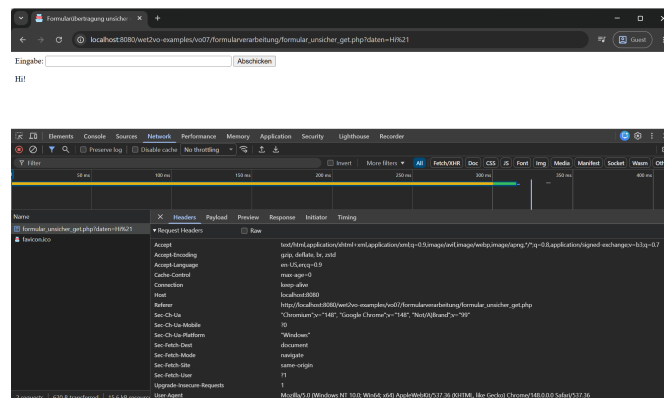


Abbildung 7.2: Die Request-Header für eine Formularübertragung mit GET. Es ist ersichtlich, dass keine Daten übertragen werden, diese hängen ja am URL.

Wird das obige Formular in der Datei `formular_unsicher_get.php` gespeichert, sein `action`-Attribut auf `get` geändert, und über `http://localhost:8080/wet2vo-examples/vo07/formularverarbeitung/formular_unsicher_get.php` aufgerufen, so wird ein Formular wie in Abbildung 7.1 gezeigt.

Wird in das Feld mit dem Namen `daten` (definiert über das `name`-Attribut im Formular) „Hi!“ eingegeben, erfolgt ein GET-Request an den Server, der wie folgt aussieht: `http://localhost:8080/wet2vo-examples/vo07/formularverarbeitung/formular_unsicher_get.php?daten=Hi%21`. Es werden also der Schlüssel „daten“ und der Wert „Hi!“ im Format `?daten=Hi%21` an den ursprünglichen URL gehängt. Dieser ist derselbe, weil im `action`-Attribut angegeben wurde, dass sich die Datei selbst aufrufen soll. `%21` ist hier das Ausrufezeichen, welches URL-kodiert wurde, da in URLs nur eine bestimmte Menge von erlaubten Zeichen vorkommen dürfen.

Der Request kann in den DevTools des Browsers im Tab `Network > Headers` betrachtet werden. Er ist in Abbildung 7.2 für den vorigen Request dargestellt.

Betrachtet man die Payload, also die Daten, die mitübertragen werden – verfügbar in den DevTools über `Network > Payload`, so ist in Abbildung 7.3 ersichtlich, dass hier von „Query String Parameters“ gesprochen wird. Bei einem genaueren Blick auf die Request Header ist auch ersichtlich, dass nirgends eine Größenangabe in Bytes mitgeschickt wird, da der Request eben keinerlei Daten enthält. Bei einer POST-Übertragung ist dies anders (siehe nächster Abschnitt).

Der Zugriff erfolgt über das superglobale Array `$_GET` an der Stelle des Formularfeldnamens, also in diesem Fall über `$_GET["daten"]`. Um den eingegebenen Wert

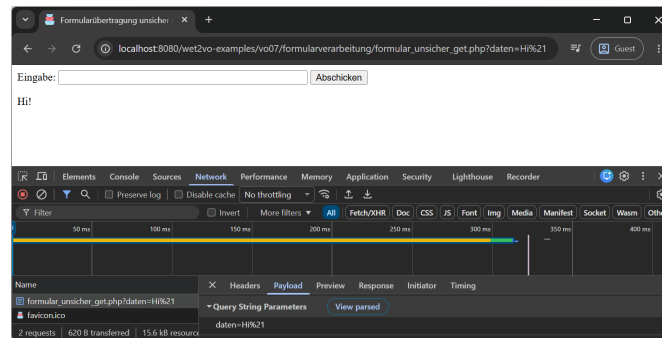


Abbildung 7.3: Die Payload, also die mitgeschickten Daten bei einer GET-Übertragung, ist nur als Query String Parameter ausgewiesen.

unterhalb des Formularfeldes anzuzeigen (wie in den Abbildungen 7.2 und 7.3 ersichtlich), ist etwa folgender Code nötig:

```
1 if (isset($_GET["daten"])) {
2     echo "<p>".$_GET["daten"]."</p>";
3 }
```

Die Abfrage mit `isset()` stellt sicher, dass nur dann eine Ausgabe erfolgt, wenn `$_GET["daten"]` auch vorhanden ist. Dies ist nur nach dem Absenden des Formulars der Fall. Diese Art der Ausgabe ist jedoch unsicher. Dies wird in Abschnitt 7.2.1 erläutert. In Abschnitt 7.2.5 werden Gegenstrategien vorgestellt.

Übertragung mittels POST

POST-Parameter werden gesondert im Request-Body als Schlüssel/Wert-Paare übertragen und tauchen im URL nicht auf.

Wird der Code für das Formular in eine Datei `formular_unsicher_post.php` gespeichert, und über `http://localhost:8080/wet2vo-examples/vo07/formularverarbeitung/formular_unsicher_post.php` aufgerufen, so sieht das Ergebnis zunächst identisch zu dem in Abbildung 7.1 aus.

Wird in das Feld mit dem Namen `daten` wieder „Hi!“ eingegeben, erfolgt nun ein *POST*-Request an den Server, der wie folgt aussieht: `http://localhost:8080/wet2vo-examples/vo07/formularverarbeitung/formular_unsicher_post.php`. Es handelt sich also wieder um denselben URL wie zuvor, Parameter sind keine sichtbar. Diese werden im Request-Body gesondert übertragen. Dies kann wieder in den DevTools über **Network** » **Headers** betrachtet werden. Der Request ist in Abbildung 7.4 für den vorigen Request dargestellt.

Beim Blick auf die Payload des Requests (**Network** » **Payload**) zeigt sich in Abbildung 7.5, dass hier wirklich Daten verschickt werden (die Daten werden nun als „Form Data“ bezeichnet). Diese sind in der Form `key=value` hinterlegt und lauten in diesem Fall `daten=Hi%21`. Zählt man die Zeichen, die jeweils 1 Byte benötigen, so kommt man auf die in der Header-Angabe ausgewiesenen 11 Bytes. Diese Form der Übertragung ist im Content-Type `application/x-www-form-urlencoded` festgelegt und stellt die Standardübertragung für Formulare mit reinem Textinhalt dar. Der Content-Type ist dabei auch in den Request-Headern (Abbildung 7.4) ersichtlich. Er muss nicht speziell

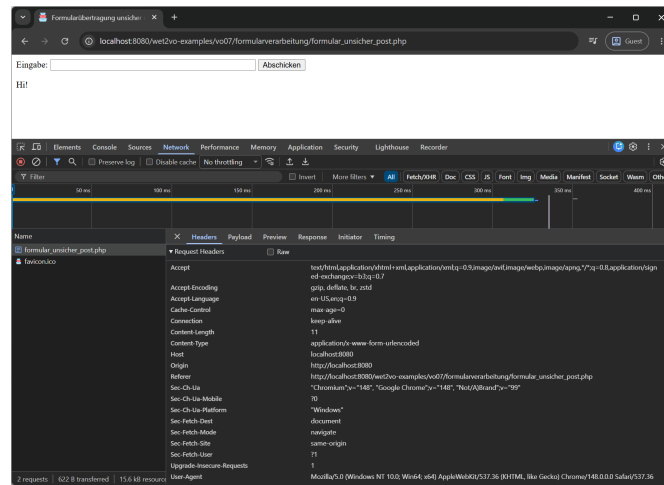


Abbildung 7.4: Die Request-Header für eine Formularübertragung mit POST. Der Eintrag **Content-Length** zeigt an, dass hier tatsächlich Inhalt im Request mitgesendet wird. In diesem Fall 11 Bytes. Der **Content-Type** der Übertragung lautet **application/x-www-form-urlencoded**.

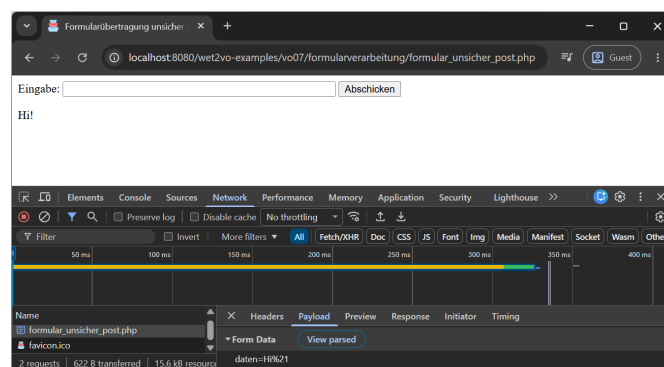


Abbildung 7.5: Die Payload für eine POST Übertragung ist ein URL-kodierter String aller Parameter. Der Browser deutet dies über „Form Data“ an.

angegeben werden.

Der Zugriff erfolgt über das superglobale Array `$_POST` an der Stelle des Formularfeldnamens, also in diesem Fall über `$_POST["daten"]`. Um den eingegebenen Wert unterhalb des Formularfeldes anzuzeigen (wie in den Abbildungen 7.4 und 7.5 ersichtlich), ist etwa folgender Code nötig:

```
1 if (isset($_POST["daten"])) {
2     echo "<p>{"$_POST["daten"]}</p>";
3 }
```

Auch dieser Zugriff weist ein schwerwiegendes Sicherheitsproblem auf, welches in Abschnitt 7.2.1 besprochen wird.

Vergleich GET und POST

GET- und POST-Übertragungen bieten beide Vor- und Nachteile, die im Folgenden aufgelistet werden. Der Vorteil einer Methode ist meist im Umkehrschluss der Nachteil der anderen.

Vorteile von GET-Übertragungen:

- Der Request kann mit allen Parametern als Lesezeichen (Bookmark) gespeichert oder geteilt werden.
- Leichteres Debuggen, da die übermittelten Eingaben im URL sichtbar und auch veränderbar sind.
- Die Zielseiten (URLs mit Parametern) können von Suchmaschinen indiziert werden.

Vorteile von POST-Übertragungen:

- Die übermittelten Formulardaten sind nicht im Verlauf (History) des Browsers enthalten. Dies bedeutet mehr Sicherheit.
- Da die Parameter nicht im URL übertragen werden, gibt es keine Längen- bzw. Mengenbeschränkung der Daten (bei GET durch die URL-Maximallänge von 2048 Zeichen der Fall).
- Mit POST können Dateien versandt werden (siehe Abschnitt 7.3).
- POST-Requests werden vom Browser nicht zwischengespeichert (cached).

Generell ist meist POST zu bevorzugen, wenn ein Formular mit Daten aktiv verschickt wird. GET macht dann Sinn, wenn die Parameter Dinge enthalten, die ein bestimmtes Verhalten nachstellen sollen, etwa die Suche eines bestimmten Produkts in einem Webshop: <https://somewebshop.site/search?query=someproduct>.

7.2 Gefahren für Webanwendungen

Web-Anwendungen, die mit Eingaben von Benutzer*innen arbeiten, sind bei der Weiterverarbeitung solcher Eingaben prinzipiell einigen Gefahren ausgesetzt. Diese müssen erkannt und entsprechend neutralisiert werden. Im Folgenden wird ein kurzer Überblick über mögliche Probleme und Gefahren und deren Lösungen gegeben.

7.2.1 Cross-Site-Scripting

Cross-Site-Scripting, kurz XSS, bezeichnet eine Sicherheitslücke (bzw. deren Ausnutzung), die durch das ungeprüfte Weiterverwenden von Benutzer*inneneingaben entsteht. Ziel des Cross-Site-Scriptings kann im schlimmsten Fall das Ausspionieren von sensiblen Daten der User*innen sein.

Bei XSS handelt es sich um eine Art der so genannten *HTML-Injection*, also das ungewollte Einfügen von HTML-Code zwischen zwei Aufrufen einer Seite. Dieser HTML-Code ist meist schadhaftes JavaScript, daher auch der Ausdruck Cross-Site-Scripting.

XSS kann immer dann auftreten, wenn Eingaben von Benutzer*innen ungeprüft wieder ausgegeben werden. Angenommen, es gibt, wie in Abschnitt 7.1 gezeigt, ein Eingabefeld mit dem Namen **daten** und dessen Inhalt wird beim Absenden so ausgegeben:

```
1 echo "<p>{$_POST["daten"]}</p>";
```

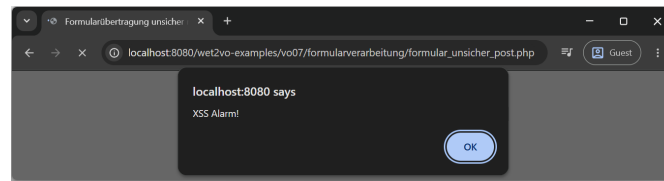



Abbildung 7.6: Die ungeprüfte Eingabe eines HTML-Fragments mit JavaScript lässt die Alert-Box im Browser erscheinen.

Dies reicht aus, um schadhafte Skripte ausführen zu können. Die folgende Eingabe im Formularfeld erzeugt bereits den ungewollten Aufruf eines Skripts (siehe Abbildung 7.6):

```
1 <script>alert("XSS Alarm!")</script>
```

Da das PHP-Skript einfach den Inhalt des Formularfeldes ausgibt, wird auch die Eingabe in Form eines HTML-`<script>`-Elements genauso bei der Ausgabe erzeugt. In diesem Fall wird nur ein Popup geöffnet, es könnte jedoch weit mehr mittels JavaScript aufgerufen werden, etwa

- das unendliche Neuladen der Seite mittels `location.reload()`,
- die Umleitung von Benutzer*innen über den Aufruf von JavaScript-Code wie `location.href = "https://malicious.site"`,
- das Auslesen von Cookies der aktuellen Seite über `document.cookie` (sofern diese nicht durch das `HttpOnly`-Flag geschützt sind),
- das Auslesen von `localStorage` und `sessionStorage`, wo moderne Webapplikationen häufig sensible Zugriffs-Tokens (wie JWTs) ablegen,
- das Auslesen aller gedrückten Tasten über Key-Events (Keylogging) und das Versenden derselben an eine Zieladresse,
- das Verändern der angezeigten Webseite (Defacement) oder das dynamische Einblenden von gefälschten Formularfeldern (Phishing) im visuellen Kontext der eigentlich vertrauenswürdigen Domain,
- das unbemerkte Ausführen von Aktionen im Namen des Opfers (etwa das Ändern von Passwörtern, Hinzufügen von Admin-Accounts oder Tätigen von Käufen) via `fetch()`-Requests, da das Skript mit der aktiven Sitzung der eingeloggten Person läuft,
- und viele weitere Dinge, die meist noch viel unerfreulicher sind (wie etwa das Ausführen von Krypto-Minern im Hintergrund).

7.2.2 SQL-Injection

Die *SQL-Injection* basiert auf einem ähnlichen Konzept. Speichert eine Seite Eingaben seiner Benutzer*innen in einer SQL-Datenbank und der Input besteht aus SQL-Anweisungen, so können diese ungefiltert in die Datenbank gelangen und dort auf diese wirken. So können etwa Datensätze verändert oder ausgespäht werden.

Ein Beispiel wäre eine Suche, z. B. in einem Webshop. Angenommen, es gibt eine Seite `find.php`, die einen GET-Parameter `id` verwendet, um ein Produkt mit einer

bestimmten ID-Nummer zu suchen. Diese könnte wie folgt aufgerufen werden: <https://localhost/mysite/find.php?id=42>

Das PHP-Skript verwendet dann den Wert des GET-Parameters (hier 42) ungefiltert in einer SQL-Abfrage weiter:

```
1 SELECT author, subject, text FROM artikel WHERE ID=42
```

Angreifer*innen können sich dies zunutze machen, indem der Wert des GET-Parameters um ein Update-Statement erweitert wird: [https://localhost/mysite/find.php?id=42; UPDATE+USER+SET+TYPE="admin"+WHERE+ID=23](https://localhost/mysite/find.php?id=42; UPDATE+USER+SET+TYPE='admin'+WHERE+ID=23)

Somit wird an die ursprünglichen SQL-Abfrage noch eine zweite angehängt, in diesem Fall wird der*die User*in mit der ID 23 zum*zur Administrator*in gemacht. Wenn dies der*die User*in der angreifenden Person ist, besteht damit umfangreicher Zugriff auf das System:

```
1 SELECT author, subject, text FROM artikel WHERE ID=42; UPDATE USER SET TYPE="admin"
WHERE ID=23
```

Das Thema SQL-Injection wird detaillierter in der Lehrveranstaltung Relational Databases (RDB2VO) behandelt.

7.2.3 Cross-Site Request Forgery (CSRF)

Bei der *Cross-Site Request Forgery*, kurz CSRF, führen fremde Benutzer*innen auf einer Webseite einen Request mit den Rechten von eingeloggten User*innen ohne deren Wissen aus. Wenn diese Opfer über administrativen Zugang verfügen, kann der Schaden das gesamte System betreffen.

Dazu wird meistens ein Link präpariert, der vom System als gültige Anweisung interpretiert wird. Weiß eine angreifende Person etwa, dass über den Link <https://localhost/mysite/user.php?action=new&user=me&pass=secret> ein neuer Account „me“ angelegt werden kann, muss sie es nur noch schaffen, diesen Link einer administrativen Kraft der Seite unterzububeln (z. B. versteckt in einer E-Mail oder auf einer präparierten Webseite). Ist diese Person in ihrem Browser noch mit erweiterten Rechten angemeldet und klickt auf den Link, hängt der Browser das gültige Session-Cookie automatisch an den Request an – der neue Account wird angelegt. Der*die Angreifer*in kann das System anschließend übernehmen.

Wichtig: Das obige Beispiel verwendet einen GET-Request, die bloße Verwendung von POST-Requests für Zustandsänderungen schützt jedoch nicht vor CSRF! Angreifende können auf ihrer präparierten Webseite versteckte HTML-Formulare platzieren, die beim Laden der Seite via JavaScript automatisch und unbemerkt per POST an die Zielseite abgesendet werden. Auch HTTPS bietet hier *keinen* Schutz, da die Authentifizierung (das Cookie) vom Browser bei jedem Request automatisch mitgesendet wird, unabhängig davon, wer den Request initiiert hat.

Als wirksame Gegenmaßnahmen sind in der modernen Webentwicklung folgende Konzepte essenziell:

- **SameSite-Cookies:** Dies ist die wichtigste moderne Verteidigungslinie. Wird beim Setzen des Session-Cookies entweder das Attribut **SameSite=Lax** oder auch **SameSite=Strict** verwendet, verbietet der Browser das Mitsenden des Cookies bei Anfragen, die von einer fremden Domain (Cross-Site) stammen.

- **Anti-CSRF-Tokens:** Hierbei handelt es sich um das *Synchronizer Token Pattern*. Bei jedem Formularaufruf generiert der Server eine lange, kryptografisch sichere Zufallszeichenkette (Token) und speichert diese in der Session. Dasselbe Token wird als verstecktes Feld in das HTML-Formular eingefügt. Beim Absenden prüft der Server, ob das mitgesendete Token mit dem in der Session übereinstimmt. Angreifer*innen können dieses Token nicht erraten.
- **Re-Authentifizierung:** Bei besonders kritischen Aktionen (Passwortänderung, Account-Löschung, Überweisungen) sollte immer das aktuelle Passwort erneut abgefragt werden.
- **XSS-Prävention:** Eine Cross-Site-Scripting-Schwachstelle (siehe Abschnitt 7.2.1) hebt jeden CSRF-Schutz aus. Werden Skripte im Kontext der Zielseite ausgeführt, können diese Tokens auslesen und in beliebigen Requests mitsenden. Ein stringenter XSS-Schutz ist daher zwingend erforderlich.

Das Grundproblem von CSRF ist untrennbar mit dem Status des Logins verbunden, weshalb Sessions in Vorlesung 8 im Detail thematisiert werden.

7.2.4 OWASP Top Ten

OWASP, kurz für *Open Web Application Security Project*, ist eine Non-Profit-Organisation, die sich das Ziel gesetzt hat, die Sicherheit von Webapplikationen zu verbessern. Unter <https://owasp.org/www-project-top-ten/> gibt die OWASP in mehrjährigen Abständen die Top 10 der kritischsten Sicherheitsprobleme heraus. Dort werden die Probleme im Detail erläutert und Gegenstrategien vorgeschlagen.

Die aktuell gültige Liste stammt aus dem Jahr 2025 und umfasst die folgenden Top 10:

- A01: Broken Access Control,
- A02: Security Misconfiguration,
- A03: Software Supply Chain Failures,
- A04: Cryptographic Failures,
- A05: Injection,
- A06: Insecure Design,
- A07: Authentication Failures,
- A08: Software or Data Integrity Failures,
- A09: Security Logging and Alerting Failures,
- A10: Mishandling of Exceptional Conditions.

Die Punkte Cross-Site-Scripting und SQL-Injection sind dabei weiterhin unter „Injection“ (A05) zusammengefasst und gehören zu den kritischsten Problemen von Webapplikationen. Cross-Site Request Forgery (CSRF) ist kein eigenständiger Punkt, sondern ist in „Broken Access Control“ (A01) integriert, da es im Kern eine missbräuchliche Rechteauserweiterung darstellt.

Konzepte zur Vermeidung von Problemen bei „Broken Access Control“ (A01) und „Authentication Failures“ (A07) werden in Vorlesung 8 detaillierter behandelt. Der Punkt „Software Supply Chain Failures“ (A03) betrifft in PHP-Projekten vor allem über Composer installierte Fremd-Komponenten – eine kompromittierte Abhängigkeit

(Dependency) kann das gesamte Projekt gefährden. Eine regelmäßige Überprüfung und Aktualisierung ist hier zwingend erforderlich.

7.2.5 PHP Filter-Funktionen

Wie in Abschnitt 7.2.1 bereits erwähnt wurde, ist die ungeprüfte Ausgabe von Benutzer*inneneingaben die Hauptursache für Cross-Site-Scripting. Als Devise muss daher gelten:

Keine Benutzer*inneneingaben dürfen in PHP ungefiltert (d. h. ohne entsprechende Überprüfung) wieder ausgegeben werden!

Für eine derartige Überprüfung, auch *Filterung* genannt, stellt PHP einige Funktionen zur Verfügung:

htmlspecialchars

`htmlspecialchars($string [, $flags [, $encoding]])`¹ wandelt fünf der wichtigsten, in HTML reservierten, Zeichen in ihre Entitäten um:

- `&` – wird zu `&`;
- `<` – wird zu `<`;
- `>` – wird zu `>`;
- `"` – wird zu `"`; (je nach gesetzten Flags),
- `'` – wird zu `'` oder `'`; (je nach gesetzten Flags).

Der Parameter `$flags` erlaubt mehrere Optionen, die mit dem bitweisen ODER (`|`) verknüpft werden können. Die wichtigsten sind:

- `ENT_COMPAT`: Konvertiert nur doppelte Anführungszeichen zu `"`; und lässt einfache Anführungszeichen unverändert.
- `ENT_QUOTES`: Konvertiert sowohl doppelte (`"`;) als auch einfache Anführungszeichen (`'`; bei HTML5, sonst `'`).
- `ENT_NOQUOTES`: Lässt doppelte und einfache Anführungszeichen unverändert.
- `ENT_SUBSTITUTE`: Ersetzt ungültige Code-Unit-Sequenzen mit dem Unicode-Ersatzzeichen U+FFFD (eine Raute mit einem Fragezeichen).
- `ENT_HTML401`: Handle Code als HTML 4.01.
- `ENT_HTML5`: Handle Code als HTML5.

`ENT_QUOTES | ENT_SUBSTITUTE | ENT_HTML401` ist der Standardwert für `$flags` seit PHP 8.1. Wird PHP Version 5.4 oder höher verwendet, sollte der letzte Teil auf HTML5 angepasst werden, vorausgesetzt HTML5 kommt im Dokument zum Einsatz.

`$encoding` erlaubt die Angabe des verwendeten Zeichensatzes. Der Standardwert ist seit PHP 5.6 der aktuell konfigurierte Standardzeichensatz (in der Regel UTF-8) und muss nicht mehr angegeben werden. Bei älteren PHP-Versionen ist dies noch nötig.

¹<https://www.php.net/manual/de/function htmlspecialchars.php>

htmlspecialchars

`htmlspecialchars()`² hat dieselben Parameter wie `htmlspecialchars()`. Die Funktion ist auch ansonsten komplett identisch, wandelt jedoch *alle* Zeichen, für die es HTML-Entitäten gibt, in solche um. Also auch etwa „ä“ zu `ä`, „ö“ zu `ö` usw.

filter_var und filter_input

Universeller sind die beiden Filterfunktionen `filter_var($variable [, $filter])`³ und `filter_input($type, $variable_name [, $filter])`⁴. Erstere filtert den Inhalt einer Variable, durch Angabe des Filters `FILTER_SANITIZE_FULL_SPECIAL_CHARS` wird das Verhalten der Funktion `htmlspecialchars()` erreicht. Dieser Filter kann auch bei `filter_input()` verwendet werden. Bei dieser Funktion wird nicht der Inhalt einer Variable, sondern der Inhalt eines `$_GET`- oder `$_POST`-Eintrages gefiltert. Die folgenden zwei Zeilen machen prinzipiell dasselbe (lediglich bei den Rückgabewerten gibt es kleine Unterschiede):

```
1 filter_input(INPUT_POST, "daten", FILTER_SANITIZE_FULL_SPECIAL_CHARS);
2 htmlspecialchars($_POST["daten"], ENT_QUOTES | ENT_HTML401);
```

strip_tags

Die Funktion `strip_tags($string [, $allowable_tags])`⁵ entfernt alle HTML- und PHP-Tags aus einem String. Mit dem optionalen Parameter `$allowable_tags` können alle Tags angegeben werden, die nicht entfernt werden sollen.

Diese Funktion eignet sich aufgrund einiger Umgehungsmöglichkeiten kaum für einen effektiven XSS-Schutz und sollte nicht eingesetzt werden. So wird beispielsweise der Tag `<a href="javascript:alert(1)">` von `strip_tags` nicht bereinigt, wenn `<a>` erlaubt ist.

Beispiel

Im folgenden Beispiel wird der Text `<h1>"Häullo" 'World'</h1>` in ein Formularfeld eingegeben. Die Ausgabe erfolgt in der ersten Zeile ungefiltert (also potentiell gefährlich), dann mittels `htmlspecialchars()`, `htmlspecialchars()`, `filter_var()`, `filter_input()` und schließlich `strip_tags()`. Das Ergebnis ist in Abbildung 7.7 zu sehen, der generierte Quellcode (zwischen den `<div>`-Elementen) sieht wie folgt aus:

```
1 unfiltered:      <h1>"Häullo" 'World'</h1>
2 htmlspecialchars: &lt;h1&gt;&quot;Häullo&quot; &apos;World&apos;&lt;/h1&gt;
3 htmlspecialchars: &lt;h1&gt;&quot;H&auml;llo&quot; &apos;World&apos;&lt;/h1&gt;
4 filter_var:      &lt;h1&gt;&quot;H&auml;llo&quot; &#039;World&#039;&lt;/h1&gt;
5 filter_input:    &lt;h1&gt;&quot;H&auml;llo&quot; &#039;World&#039;&lt;/h1&gt;
6 strip_tags:      "Häullo" 'World'
```

²<https://www.php.net/manual/de/function.htmlspecialchars.php>

³<https://www.php.net/manual/de/function.filter-var.php>

⁴<https://www.php.net/manual/de/function.filter-input.php>

⁵<https://www.php.net/manual/de/function.strip-tags.php>



Abbildung 7.7: Ungefiltert wird die Eingabe `<h1>"Hällo" 'World'</h1>` tatsächlich als Überschrift erster Ordnung angezeigt. Mit den Filter-Funktionen werden die HTML-Spezialzeichen in Entitäten umgewandelt, sodass sie nur mehr als Zeichen erscheinen, oder ganz entfernt.

7.2.6 Symfony HTML Sanitizer: Komponente zur Filterung von Inhalten

Zur Verarbeitung von User*innendaten aus Formularen ist `htmlspecialchars()` eine verlässliche und meist absolut ausreichende Methode. Es gibt jedoch auch Fälle, in denen formatierte HTML-Eingaben erlaubt und sicher gefiltert werden müssen. Ein typisches Beispiel sind Eingaben aus visuellen Editoren (WYSIWYG) in Content-Management-Systemen. Hier soll Textformatierung (z. B. `<b>`, `<i>`) erhalten bleiben, während Cross-Site-Scripting-Vektoren (z. B. `<script>`-Tags oder `onload`-Attribute) zuverlässig entfernt werden müssen.

Die moderne Standardkomponente hierfür ist `symfony/html-sanitizer` aus dem Symfony-Framework. Er wird über Composer installiert:

```
1 $ composer require symfony/html-sanitizer
```

Die Nutzung basiert auf einem strikten „Allowlisting“-Ansatz. Es wird zunächst eine Konfiguration erstellt, die definiert, was erlaubt ist. Die Methode `allowSafeElements()` ist dabei ein guter Startpunkt, da sie Standard-Tags für Textauszeichnungen zulässt, aber jegliche ausführende Logik blockiert. Anschließend wird das eigentliche Sanitizer-Objekt erzeugt:

```
1 use Symfony\Component\HtmlSanitizer\HtmlSanitizerConfig;
2 use Symfony\Component\HtmlSanitizer\HtmlSanitizer;
3
4 $config = new HtmlSanitizerConfig()->allowSafeElements();
5 $sanitizer = new HtmlSanitizer($config);
```

Nun kann das Formularfeld (in diesem Fall `daten`) über die Methode `sanitize()` bereinigt werden, bevor es ausgegeben oder in einer Datenbank gespeichert wird:

```
1 if (isset($_POST["daten"])) {
2     $datenBereinigt = $sanitizer->sanitize($_POST["daten"]);
3     echo "<div>$datenBereinigt</div>";
4 }
```

Die Ausgabe gestaltet sich prinzipiell wie folgt:

- Sicheres HTML (wie `<p>`, `<strong>`) bleibt erhalten.
- Schadhaftes HTML (z. B. `<script>`, `<iframe>` oder `javascript:-`Links) wird restlos entfernt.

- Fehlerhaft verschachteltes HTML wird vom Sanitizer automatisch erkannt und strukturell korrigiert, um eine valide Ausgabe (W3C-Standard) zu gewährleisten.

Aufgrund der enormen Komplexität von HTML und den sich ständig wandelnden Angriffsvektoren sollte für das Filtern von HTML *immer* eine spezialisierte, gut gewartete Komponente wie der Symfony HTML Sanitizer herangezogen und niemals versucht werden, dies mit regulären Ausdrücken selbst zu erstellen.

7.3 Dateiuploads

Das *Hochladen von Dateien* gehört – wie auch das Erfassen von textuellen Eingaben der Benutzer*innen – zum Themengebiet der Formularverarbeitung. Während sich jedoch textuelle Daten rein clientseitig erfassen und dann z. B. per E-Mail versenden lassen, können hochgeladene Dateien nur serverseitig weiterverarbeitet werden. Dafür sind zunächst einige Dinge notwendig.

7.3.1 Voraussetzungen und Ablauf

Für das Erfassen von Dateien sind einige Voraussetzungen zu beachten. Zunächst muss ein entsprechendes Formular vorhanden sein, aber auch die maximal erlaubte Dateigröße spielt eine Rolle.

Formular

Beim Formular für einen Dateiupload spielen zwei Attribute eine besondere Rolle. Zunächst muss beim `<form>`-Element selbst das Attribut `enctype` mit dem speziellen Wert `multipart/form-data` angegeben sein, um die Übertragung größerer Datenmengen zu ermöglichen. Des Weiteren muss es ein `<input>`-Element geben, dessen `type`-Attribut den Wert `file` aufweist. Dies erzeugt ein Eingabefeld mit dem typischen „Browse“-Button, der es ermöglicht, eine Datei im Dateisystem für den Upload auszuwählen.

Anmerkung: Das Aussehen des Dateiupload-Eingabefeldes wird sowohl vom verwendeten Betriebssystem als auch vom jeweiligen Browser bestimmt. Sein Aussehen variiert daher stark. Auch eine Formatierung mittels CSS ist nur mit einigen Tricks möglich.

Weiters ist bei Dateiuploads die Übertragungsmethode POST vorausgesetzt.

Wie bereits eingehend erwähnt, ist es für derartige Formulare üblich, eine Größenbeschränkung für Dateien anzugeben. Dies kann auf zwei Arten geschehen:

- mittels verstecktem Formularfeld `MAX_FILE_SIZE` (Soft-Limit),
- durch Einstellungen in der Konfigurationsdatei `php.ini` (Hard-Limit).

Die Angabe über ein verstecktes Formularfeld (`type="hidden"`) muss *vor* dem Eingabefeld für Dateien erfolgen. Das Feld muss (durch das Attribut `name`) den Namen `MAX_FILE_SIZE` haben, das Attribut `value` enthält die Obergrenze in Bytes. Die Angabe einer Obergrenze auf diese Art und Weise ist keineswegs sicher und kann clientseitig leicht ausgehebelt werden (sie steht ja im Quelltext). Trotzdem kann es sinnvoll sein, sie anzugeben, da der Browser *sofort* weiß, wie groß die Datei maximal sein darf, noch bevor er mit dem Upload begonnen hat. Es verhindert also, dass Benutzer*innen auf den Upload einer großen Datei warten, nur um dann die Fehlermeldung zu erhalten,

dass die Datei zu groß war. Derzeit implementiert jedoch kein moderner Browser eine derartige Überprüfung, weshalb sie auch durchaus ignoriert werden kann.

Ein fertiges HTML-Formular inklusive maximaler Uploadgröße sieht wie folgt aus:

```
1 <form method="post" enctype="multipart/form-data"
   action="<?= $_SERVER["SCRIPT_NAME"] ?>">
2   <input type="hidden" name="MAX_FILE_SIZE" value="2097152">
3   <input type="file" name="userfile">
4   <button type="submit">Hochladen</button>
5 </form>
```

Die angegebene Maximalgröße von 2.097.152 Bytes entspricht genau 2 Megabytes ($1024 \cdot 1024 \cdot 2$ Bytes).

php.ini Einstellungen

Wichtiger als die Angabe eines Soft-Limits über das Formular, ist das so genannte Hard-Limit in der Konfigurationsdatei `php.ini`. Dieses kann von den Nutzer*innen (im Gegensatz zur Angabe im Formular) nicht umgangen werden. Zusätzlich können hier weitere Dinge rund um Dateiuploads gesteuert werden.

Tabelle 7.1 enthält eine Übersicht über alle Konfigurationsvariablen in der `php.ini`, die einen Einfluss auf das Hochladen von Dateien haben.

Je nach Webserver-Konfiguration können andere Werte für die einzelnen Variablen voreingestellt sein. Die Werte können jedoch einfach geändert werden. Dabei ist zu beachten, dass nach einer Änderung ein Neustart des Apache Servers nötig ist, um die Konfigurationsdatei neu einzulesen.

7.3.2 Verarbeitung im superglobalen Array

Nachdem das Formularfeld sowie der Webserver auf Dateiuploads vorbereitet wurden, kann mit der Verarbeitung begonnen werden. Nachdem das Formular abgeschickt wurde, wird die Datei in ein temporäres Verzeichnis auf den Webserver hochgeladen (dieses kann in `php.ini` im Eintrag `upload_tmp_dir` festgelegt werden). Um das Überschreiben von eventuell gleichlautenden, bereits hochgeladenen Dateien zu vermeiden, bekommt die Datei zunächst einen zufällig gewählten Namen.

Der Zugriff auf die hochgeladene Datei erfolgt in weiterer Folge über ein weiteres superglobales Array: `$_FILES`.

Das Array ist zweidimensional und hat sechs Eigenschaften, die Auskunft über die hochgeladene Datei geben. Im Folgenden wird angenommen, dass der Name des Upload-Feldes `userfile` lautet:

- `$_FILES["userfile"]["name"]`: Der ursprüngliche Name der Datei, wie er auf dem Computer des*der Benutzer*in lautete.
- `$_FILES["userfile"]["full_path"]`: Der vollständige Pfad, wie er vom Browser übermittelt wurde (seit PHP 8.1).
- `$_FILES["userfile"]["type"]`: Der MIME-Type der hochgeladenen Datei. Bei einem GIF-Bild etwa `image/gif`, Bei einer Textdatei lautet der Wert `text/plain`.
- `$_FILES["userfile"]["size"]`: Die Größe der hochgeladenen Datei in Bytes.
- `$_FILES["userfile"]["tmp_name"]`: Der temporäre Dateiname, unter dem die hochgeladene Datei auf dem Server gespeichert wurde.

Tabelle 7.1: Die Konfigurationsvariablen aus `php.ini`, die das Hochladen von Dateien beeinflussen. Die Werte in der zweiten Spalte sind Beispiele – andere bzw. ähnliche Werte sind möglich.

Variable	Wert	Bedeutung
<code>upload_max_filesize</code>	2M	Maximale Größe, die eine hochgeladene Datei haben darf.
<code>file_uploads</code>	On	Legt fest, ob generell Dateiuploads per HTTP erlaubt sind.
<code>post_max_size</code>	8M	Setzt die maximal erlaubte Größe von POST-Daten.
<code>max_file_uploads</code>	20	Die maximale Anzahl an Dateien, die in einem Request hochgeladen werden können.
<code>memory_limit</code>	128M	Setzt den Maximalwert des Speichers, den ein Skript verbrauchen darf. –1 legt fest, dass es kein Limit gibt.
<code>max_execution_time</code>	30	Legt die maximale Zeit in Sekunden fest, die ein Skript laufen darf, bevor der Parser die Ausführung stoppt.
<code>max_input_time</code>	60	Legt die maximale Zeit in Sekunden fest, die ein Skript verbrauchen darf, um Eingabedaten (wie POST, GET und Dateiuploads) zu verarbeiten.

- `$_FILES["userfile"]["error"]`: Der Fehlercode, der im Zusammenhang mit dem Hochladen der Datei generiert wurde.

7.3.3 Fehlercodes

Nachdem die Datei hochgeladen wurde, wird die Eigenschaft `error` des superglobalen Arrays `$_FILES` immer mit einem Wert befüllt. Dieser Wert ist ein Integer, für den es jedoch sprechendere Konstanten gibt:

- `UPLOAD_ERR_OK` (0): Kein Fehler, die Datei wurde erfolgreich hochgeladen.
- `UPLOAD_ERR_INI_SIZE` (1): Die Größe der hochgeladenen Datei überschritt den in der `php.ini` unter `upload_max_filesize` eingestellten Wert.
- `UPLOAD_ERR_FORM_SIZE` (2): Die Größe der Datei überschritt den im Formular unter `MAX_FILE_SIZE` angegebenen Wert.
- `UPLOAD_ERR_PARTIAL` (3): Die Datei wurde nur zum Teil hochgeladen.
- `UPLOAD_ERR_NO_FILE` (4): Es wurde keine Datei hochgeladen.
- `UPLOAD_ERR_NO_TMP_DIR` (6): Es existiert kein temporäres Verzeichnis am Webserver, in das die Datei hochgeladen werden kann.

- `UPLOAD_ERR_CANT_WRITE` (7): Der Webserver kann die Datei nicht ins Dateisystem schreiben (z. B. wegen fehlender Schreibrechte).
- `UPLOAD_ERR_EXTENSION` (8): Eine PHP-Erweiterung hat einen Fehler verursacht.

Es existiert kein Fehlercode mit dem Wert 5 mehr. Dieser wurde ursprünglich erzeugt, wenn eine leere Datei hochgeladen wurde, dieses Verhalten wurde aber später nicht mehr als Fehler angesehen und der Code daher entfernt.

Bevor mit der weiteren Verarbeitung der hochgeladenen Datei fortgefahren wird, sollte der Fehlercode immer abgefragt werden. Lautet dieser 0, so war alles in Ordnung, ist ein anderer Wert im Array enthalten, so sollte der Fehlercode interpretiert werden. Die Fehler 1–4 können von den Nutzer*innen behoben werden (etwa durch erneutes Hochladen oder Hochladen einer kleineren Datei). Die Fehler 6–8 müssen von dem*der Betreiber*in des Webserver korrigiert werden. In diesem Fall bringt eine genaue Fehlermeldung an die Benutzer*innen nur bedingt etwas.

7.3.4 Verschieben der Datei und Sicherheitsaspekte

Nachdem sichergestellt wurde, dass der Upload funktioniert hat, muss die Datei aus dem temporären Verzeichnis an ihren Bestimmungsort verschoben werden. Das geschieht mit der Funktion `move_uploaded_file($filename, $destination)`⁶. Diese prüft zunächst, ob die angegebene Datei tatsächlich durch einen gültigen HTTP-POST-Upload entstanden ist, was das Einschleusen lokaler Systemdateien durch Angreifer verhindert. Danach verschiebt sie die Datei an das angegebene Ziel.

Sicherheitswarnung: Die Verwendung von `basename()` für den Dateinamen (wie im folgenden Beispiel) verhindert zwar sogenannte *Directory Traversal*-Angriffe (z. B. das Einschleusen von Pfaden wie `../../../../../etc/passwd`), löst aber nicht alle Sicherheitsprobleme. Wenn der Originalname beibehalten wird, können bestehende Dateien überschrieben werden oder ausführbare Skripte (z. B. `shell.php`) auf den Server gelangen. Für Produktionssysteme sollten Dateinamen serverseitig immer neu generiert (z. B. mit `uniqid()`) und die Dateiendung anhand des vom Server geprüften MIME-Typs vergeben werden.

Der folgende Code zeigt die Verarbeitung der hochgeladenen Datei, die Auswertung spezifischer Fehlercodes über ein Array und das Verschieben unter Verwendung von `basename()`, um Pfadmanipulationen durch den Browser zu unterbinden. Die Fehlermeldungen sind in einer eigenen Datei definiert und werden im Beispiel eingebunden:

```

1 $uploadErrors = [
2     UPLOAD_ERR_INI_SIZE    => "Die Datei überschreitet die in php.ini konfigurierte
        maximale Größe.",
3     UPLOAD_ERR_FORM_SIZE   => "Die Datei überschreitet die im Formular angegebene
        maximale Größe.",
4     UPLOAD_ERR_PARTIAL     => "Die Datei wurde nur teilweise hochgeladen.",
5     UPLOAD_ERR_NO_FILE     => "Es wurde keine Datei ausgewählt.",
6     UPLOAD_ERR_NO_TMP_DIR  => "Temporäres Verzeichnis fehlt.",
7     UPLOAD_ERR_CANT_WRITE  => "Datei konnte nicht auf dem Server gespeichert werden.",
8     UPLOAD_ERR_EXTENSION  => "Upload wurde durch eine PHP-Erweiterung abgebrochen.",
9 ];

```

⁶<https://www.php.net/manual/de/function.move-uploaded-file.php>

Dann erfolgt das eigentliche Upload-Handling:

```

1 require "upload_errors.php";
2
3 const UPLOAD_DIR = "uploads/";
4
5 if ($_SERVER["REQUEST_METHOD"] === "POST") {
6     $error = $_FILES["userfile"]["error"];
7
8     if ($error !== UPLOAD_ERR_OK) {
9         $message = $uploadErrors[$error] ?? "Unbekannter Fehler (Code $error).";
10        echo "<p>Fehler: $message</p>";
11    } else {
12        $originalName = basename($_FILES["userfile"]["name"]);
13
14        if (move_uploaded_file($_FILES["userfile"]["tmp_name"], UPLOAD_DIR .
            $originalName)) {
15            echo "<p>Upload von " . htmlspecialchars($originalName, ENT_QUOTES | ENT_HTML5
                ) . " erfolgreich!</p>";
16        } else {
17            echo "<p>Fehler: Datei konnte nicht gespeichert werden.</p>";
18        }
19    }
20 }

```

7.3.5 Hochladen mehrerer Dateien

Das Hochladen mehrerer Dateien gestaltet sich analog zu dem bisher gezeigten. Beim Formular sind zwei kleine Änderungen notwendig:

```

1 <form method="post" enctype="multipart/form-data"
    action="<?= $_SERVER["SCRIPT_NAME"] ?>">
2 <input type="hidden" name="MAX_FILE_SIZE" value="2097152">
3 <input type="file" name="userfiles[]" multiple>
4 <button type="submit">Hochladen</button>
5 </form>

```

Das Formularfeld trägt nun den Namen `userfiles[]`, repräsentiert also ein Array. Zudem wird das Attribut `multiple` hinzugefügt, um die Auswahl mehrerer Dateien gleichzeitig zuzulassen. Da es auf diese Weise im Browser nicht limitiert werden kann, wie viele Dateien maximal ausgewählt werden dürfen, können alternativ auch mehrere einzelne Eingabefelder mit demselben `name`-Attribut (jeweils mit eckigen Klammern) erstellt werden.

Die Daten landen wieder im `$_FILES`-Array. Dieses ist bei multiplen Uploads jedoch um eine Ebene tiefer verschachtelt. Jeder der regulären Einträge (wie `name`, `tmp_name` oder `error`) ist nun selbst ein indiziertes Array mit so vielen Einträgen, wie Dateien hochgeladen wurden.

Jede Datei wird einzeln verarbeitet. Dazu wird zunächst mittels `count()` bei einem der Einträge ermittelt, wie viele Dateien übermittelt wurden. Anschließend wird in einer Schleife über jeden Index iteriert. Dabei kommt dieselbe robuste Logik wie beim Einzel-Upload zum Einsatz: Der Originalname wird mittels `basename()` entschärft, die Ausgabe in HTML mit `htmlspecialchars()` maskiert und anhand des Fehlercodes entschieden, ob die Datei verschoben wird.

```

1 $nrOfFiles = count($_FILES["userfiles"]["name"]);
2 for ($i = 0; $i < $nrOfFiles; $i++) {
3     $error = $_FILES["userfiles"]["error"][$i];
4     $originalName = basename($_FILES["userfiles"]["name"][$i]);
5     $safeName = htmlspecialchars($originalName, ENT_QUOTES | ENT_HTML5);
6
7     if ($error !== UPLOAD_ERR_OK) {
8         $message = $uploadErrors[$error] ?? "Unbekannter Fehler (Code $error).";
9         echo "<p>Fehler bei $safeName: $message</p>";
10    } elseif (move_uploaded_file($_FILES["userfiles"]["tmp_name"][$i], UPLOAD_DIR .
        $originalName)) {
11        echo "<p>Upload von $safeName erfolgreich!</p>";
12    } else {
13        echo "<p>Fehler: $safeName konnte nicht gespeichert werden.</p>";
14    }
15 }

```

7.3.6 Hochladen ganzer Verzeichnisse

Seit PHP 8.1 gibt es die Möglichkeit, ein Verzeichnis auszuwählen und dessen gesamte Inhalte (inklusive aller Unterverzeichnisse) hochzuladen. Das Formular ähnelt dem für den Upload mehrerer Dateien, wird jedoch um das Attribut `webkitdirectory` ergänzt. Dieses Attribut ist formal nicht standardisiert, wird aber von allen modernen Browsern problemlos unterstützt.

```

1 <form method="post" enctype="multipart/form-data"
    action="<?= $_SERVER["SCRIPT_NAME"] ?>">
2 <input type="hidden" name="MAX_FILE_SIZE" value="2097152">
3 <input type="file" name="userfiles[]" webkitdirectory multiple>
4 <button type="submit">Hochladen</button>
5 </form>

```

Die Verarbeitung im `$_FILES`-Array verläuft ähnlich wie im vorigen Abschnitt. Um jedoch die Verzeichnisstruktur serverseitig identisch nachzubilden, muss anstelle des Schlüssels `name` der Schlüssel `full_path` ausgelesen werden. Dieser enthält neben dem Dateinamen auch den vom Browser übermittelten relativen Pfad.

Sicherheitswarnung: Wie der Dateiname wird auch der Wert `full_path` direkt vom Client übermittelt. Ein Angreifer kann diesen manipulieren und *Directory Traversal*-Vektoren wie `../` einschleusen. In einem Produktivsystem muss der Pfad vor der Weiterverarbeitung strikt validiert und bereinigt werden, um ein Ausbrechen aus dem Upload-Verzeichnis zu verhindern.

Bevor die Datei mit `move_uploaded_file()` an ihren Bestimmungsort verschoben wird, muss sichergestellt sein, dass das entsprechende Zielverzeichnis existiert. Der folgende Code extrahiert dazu mit `dirname()`⁷ den Ordnerpfad aus der Zielvariable und prüft mittels `is_dir()`⁸, ob dieser existiert. Ist das nicht der Fall, wird er mit `mkdir()`⁹ erstellt. Der dritte Parameter (`true`) sorgt dafür, dass das Anlegen rekursiv passiert,

⁷<https://www.php.net/manual/de/function.dirname.php>

⁸<https://www.php.net/manual/de/function.is-dir.php>

⁹<https://www.php.net/manual/de/function.mkdir.php>

also die gesamte Ordnerhierarchie aufgebaut wird. Die Dateiberechtigungen sind hier beispielhaft auf 0777 gesetzt, was ausschließlich in lokalen Entwicklungsumgebungen verwendet werden darf.

```

1 $nrOfFiles = count($_FILES["userfiles"]["name"]);
2 for ($i = 0; $i < $nrOfFiles; $i++) {
3     $error = $_FILES["userfiles"]["error"][$i];
4     $relativePath = $_FILES["userfiles"]["full_path"][$i];
5     $destination = UPLOAD_DIR . $relativePath;
6     $safePath = htmlspecialchars($relativePath, ENT_QUOTES | ENT_HTML5);
7
8     if ($error !== UPLOAD_ERR_OK) {
9         $message = $uploadErrors[$error] ?? "Unbekannter Fehler (Code $error).";
10        echo "<p>Fehler bei $safePath: $message</p>";
11    } elseif (!is_dir(dirname($destination)) && !mkdir(dirname($destination), 0777,
12        true)) {
13        echo "<p>Fehler: Verzeichnis für $safePath konnte nicht erstellt werden.</p>";
14    } elseif (move_uploaded_file($_FILES["userfiles"]["tmp_name"][$i], $destination))
15    {
16        echo "<p>Upload von $safePath erfolgreich!</p>";
17    } else {
18        echo "<p>Fehler: $safePath konnte nicht gespeichert werden.</p>";
19    }
20 }

```

7.3.7 bulletproof: Dateiuploads mit einer fertigen Komponente

Auch für Dateiuploads stehen auf Packagist fertige Komponenten zur Verfügung, die den Boilerplate-Code und die vielen manuellen Sicherheitschecks verringern. Für das sichere Hochladen von *Bildern* ist die leichtgewichtige Komponente **samayo/bulletproof** eine gute Wahl. Da sich Bulletproof ausschließlich auf Bildformate spezialisiert hat, sollten für den Upload beliebiger anderer Dateitypen (z.B. PDFs, Office-Dokumente oder ZIP-Archive) generische Bibliotheken wie **siriusphp/upload** oder die robuste **UploadedFile**-Klasse der **symfony/http-foundation** Komponente herangezogen werden.

Bulletproof wird regulär über Composer installiert:

```
1 $ composer require samayo/bulletproof
```

Die Bibliothek abstrahiert das mühsame Arbeiten mit dem `$_FILES`-Array. Im folgenden kompletten Beispiel wird ein HTML-Formular bereitgestellt. Sobald dieses per POST abgesendet wird, übernimmt Bulletproof die Validierung.

Zunächst wird ein neues **Image**-Objekt mit dem superglobalen Array instanziiert. Anschließend werden die Sicherheitsregeln als Kettenaufrufe (Method Chaining) konfiguriert. In diesem Beispiel wird der Upload auf gängige Bildformate bis maximal 2 Megabyte ($2 * 1024 * 1024$ Bytes) beschränkt und das Zielverzeichnis mit **setStorage()** festgelegt.

```

1 $image = new Bulletproof\Image($_FILES);
2 $image
3     ->setStorage("uploads/")
4     ->setMime(["jpeg", "png", "gif"])
5     ->setSize(1, 2 * 1024 * 1024);
6

```

```
7 // Prüfen, ob eine Datei im Upload-Feld übergeben wurde
8 if ($image["userfile"]) {
9     $upload = $image->upload();
10
11     if ($upload) {
12         // getName() liefert den von Bulletproof neu generierten, sicheren Namen
13         echo "<p>Upload von {$upload->getName()} erfolgreich!</p>";
14     } else {
15         echo "<p>Fehler: {$image->getError()}</p>";
16     }
17 } else {
18     echo "<p>Fehler: {$image->getError()}</p>";
19 }
```

Der finale Upload-Prozess wird durch den Aufruf von `upload()` angestoßen. Bulletproof kümmert sich dabei automatisch um die Vergabe eines sicheren, zufällig generierten Dateinamens. Zudem prüft die Bibliothek die echte Datei-Signatur (MIME-Type) des Inhalts, um sicherzustellen, dass sich nicht etwa ein ausführbares Skript hinter einer gefälschten `.jpg`-Endung versteckt. Liefert die Methode `true` (bzw. das generierte Image-Objekt) zurück, war der Upload erfolgreich; andernfalls kann die exakte Fehlerursache über `getError()` ausgelesen werden.

7.4 Weiterführende Literatur

Als primäre Referenz für Formularverarbeitung, Filter-Funktionen und Dateiuploads sollte in diesem Fall [42] eingesetzt werden. Sie ist naturgemäß am aktuellsten Stand, während die meisten Bücher zu dem Thema oft veraltete Informationen beinhalten. Etwa wenn es um diverse Parameter von Funktionen geht.

Dateiuploads werden ausführlich in [41, 44, 49] behandelt, jedoch decken die Bücher dabei nur unwesentlich mehr Details ab, als in dieser Vorlesung vorgestellt wurden.

Vorlesung 8

Sessions und Authentifizierung

Bevor die Hauptthemen *Sessions* und *Authentifizierung* erläutert werden, ist es notwendig, zunächst über *HTTP-Response Header* und *Cookies* zu sprechen, da diese Dinge dort eine wesentliche Rolle spielen.

8.1 HTTP-Response Header

In Vorlesung 7 wurde ein Blick auf die Header geworfen, die bei der Anfrage (Request) an einen Webserver bei der GET- und POST-Methode übertragen werden. Die Antwort (Response), die der Webserver auf eine Anfrage eines Clients (Request) sendet, enthält ebenfalls solche *Header*. Wie auch in HTML enthalten Header verschiedene Metadaten, die dann vom Client ausgewertet werden können. In den Headern sind etwa ein Statuscode (200 – OK oder 404 – Not found), die Länge der übertragenen Daten, der Content-Type (z. B. `text/html`) oder auch Cookies (siehe Abschnitt 8.2) enthalten.

Diese Header-Daten werden in der Regel automatisch vom Webserver oder auch von PHP generiert und es gibt nur selten Grund zum Eingreifen. In manchen Fällen sollen aber explizit Anweisungen hinzugefügt werden. Dies kann über die Funktion `header($string)`¹ geschehen.

Die wichtigste Grundregel hierbei lautet: Header müssen *vor* jeglichem Output gesendet werden, d. h. vor eventuellem HTML-Markup oder auch `echo`-Aufrufen.

Folgende wichtige Dinge lassen sich mit der `header()`-Funktion erreichen:

8.1.1 Unterschiedliche Content-Types

Der normale Content-Type für PHP-generierte Ausgabe lautet `text/html`. Soll nun aber der Browser dazu bewegt werden, dass er die Ausgabe als reine Textdatei oder etwa auch als XML-Datei behandelt, so kann jeweils ein anderer Content-Type im Header mitgeschickt werden. Die entsprechenden Angaben lauten wie folgt:

```
1 header("Content-type: text/plain"); // Text
2 header("Content-type: text/xml"); // XML
```

¹<https://www.php.net/manual/de/function.header.php>

8.1.2 Weiterleitungen

Ein weiterer häufiger Einsatzgrund der Funktion `header()` ist das Weiterleiten auf einen andere URL. In diesem Fall müssen die Angabe `Location` sowie der neue URL im Header-String enthalten sein:

```
1 header("Location: https://www.fh-ooe.at/");  
2 exit();
```

Nach einer Weiterleitung sollte `exit()`;² aufgerufen werden. Dies terminiert das aktuelle PHP-Skript sofort. Da eine Weiterleitung auf einer andere Seite initiiert wurde, sollte auch die Verarbeitung der aktuellen am Server sofort gestoppt werden, um nicht unnötig Ressourcen zu verwenden. `exit()`; tut genau dies. Seit PHP 8.4 ist `exit()` eine echte Funktion (vorher war es ein Sprachkonstrukt). Sie sollte daher mit Klammern (also `exit()`) aufgerufen werden. Da die Funktion aber vom Parser nach wie vor speziell behandelt wird, ist es auch nach wie vor ohne Klammern (also `exit;`) möglich.

8.2 Cookies

Dass es sich bei HTTP um ein zustandsloses Protokoll handelt, wurde bereits im ersten Semester erläutert. Dies macht es besonders schwer, einen zusammenhängenden Zustand abzubilden. Denn um diesen zu erhalten, müssen Daten konsistent weitergegeben werden können.

Eine Möglichkeit, dies zu tun, sind versteckte Formularfelder. Alle wichtigen Daten werden von einem Seitenaufruf zum nächsten über solche versteckten Formularfelder weitergegeben. Da dieses Verfahren jedoch eine Sequenz von Formularen voraussetzt, ist es nicht sehr universell einzusetzen.

Eine zweite Möglichkeit ist das sogenannte URL-Rewriting (siehe auch Abschnitt 8.3), bei dem alle wichtigen Informationen als GET-Parameter von Seite zu Seite weitergegeben werden. Während diese Technik zwar überall funktioniert, kann die Weitergabe von vielen Parametern durchaus mühsam werden.

8.2.1 Allgemeines

Eine dritte Möglichkeit sind die sogenannten *Cookies*. Bei ihnen handelt es sich um eine sehr kleine, textbasierte Dateneinheit, die im Browser gespeichert wird. Sie werden mit einem Webserver assoziiert und auch bei jedem Request-Response-Zyklus mit versandt. Aus diesem Grund sind Cookies auch sehr klein und können nur beschränkte Datenmengen aufnehmen. Ein Cookie ist auf eine Größe von 4 Kilobyte beschränkt, weiters dürfen maximal 20 Cookies pro Server existieren.

Cookies dienen also quasi als Gedächtnisvariable, die einen Zustand zwischen einzelnen HTTP-Aufrufen speichern können. Sie dienen also primär der serverseitigen Verarbeitung und Informationsübertragung. Sollen nur clientseitige Informationen gespeichert werden, so empfiehlt sich der im ersten Semester besprochene Local Storage. Nichtsdestotrotz können Cookies aber auch clientseitig mittels JavaScript angesprochen werden.

²<https://www.php.net/manual/de/function.exit>

8.2.2 Cookie-Eigenschaften

Ein Cookie hat verschiedene Parameter:

- **name:** Der Name eines Cookies. Er kann beliebig gewählt werden und dient primär zur Identifikation.
- **value:** Der Wert eines Cookies, also die eigentlichen Daten, die lokal zwischengespeichert werden sollen.
- **expires:** Die Lebensdauer eines Cookies. Wird keine angegeben, so wird das Cookie beim Beenden des Browsers gelöscht. Ansonsten existiert es bis zum angegebenen Zeitstempel. Dieser wird in Sekunden seit dem 1.1.1970 (GMT) angegeben.
- **path:** Der Pfad am Webserver, für den das Cookie gelten soll. Standardmäßig ist dies der aktuelle Pfad inklusive aller Unterverzeichnisse.
- **domain:** Legt fest, für welche Domain das aktuelle Cookie vom Browser zurückgegeben werden soll. Standardmäßig wird immer der aktuelle Server angenommen.
- **secure:** Erlaubt es einzustellen, dass ein Cookie nur bei einer verschlüsselten HTTPS-Verbindung übertragen wird. Der Standardwert ist jedoch **false**, sodass Cookies auch bei unsicheren Verbindungen übertragen werden.
- **httponly:** Ist der Parameter auf **true** gestellt, ist das Cookie nur mehr über HTTP zugreifbar. Lokale Skriptsprachen wie JavaScript können nicht mehr darauf zugreifen. Diese Einstellung kann einen effektiven Schutz gegen XSS bieten.
- **samesite:** Legt fest, ob das Cookie bei Anfragen von anderen Webseiten (Cross-Site-Requests) mitgesendet wird.

8.2.3 Cookies setzen

Um ein Cookie mit den restlichen HTTP-Headern an den Browser zu senden, wird `setcookie($name [, $value, $expires, $path, $domain, $secure, $http])`³ verwendet. Die Parameter entsprechen den in Abschnitt 8.2.2 angeführten. Alle Parameter, außer der Name, sind optional. Sollen einzelne davon umgangen werden, kann einfach ein leerer String ("") dafür angegeben werden. Ausnahme ist `$expires`, das einen Integer verlangt, hier muss 0 gesetzt werden.

Seit PHP 7.3 gibt es eine alternative – und zu bevorzugende, weil flexiblere – Syntax zum Setzen von Cookies: `setcookie($name [, $value, [$options]])`. Der Parameter `$options` ist ein assoziatives Array mit möglichen Schlüsseln wie `expires`, `path`, `domain`, `secure`, `httponly` und `samesite`. Letzterer kann einen der folgenden Werte haben:

- **Strict:** Das Cookie wird *nur* bei sogenannten *Same-Site-Requests* mitgesendet, also wenn der Request direkt von derselben Domain stammt (z. B. bei Navigation oder Formularen innerhalb der eigenen Seite). Dies ist die restriktivste Einstellung.
- **Lax:** Wie **Strict**, aber Cookies werden auch bei *einfachen Cross-Site-GET-Requests* (z. B. durch einen Link von einer fremden Seite) mitgesendet. POST-Requests und andere komplexe Requests bleiben ausgeschlossen.

³<https://www.php.net/manual/de/function.setcookie>

- **None:** Das Cookie wird *auch bei Cross-Site-Requests* (z.B. über eingebettete Inhalte wie iFrames oder externe Skripte) mitgesendet. Voraussetzung ist, dass zusätzlich der **secure**-Parameter auf **true** gesetzt ist (nur über HTTPS).

Wird etwa ein YouTube-Video per **iframe** auf der eigenen Seite eingebunden, versucht YouTube ggf. ein Authentifizierungs- oder Personalisierungs-Cookie zu setzen. Damit dieses im Kontext einer Drittanbieter-Anfrage funktioniert, muss es mit den Attributen **samesite=None** und **secure=true** gesetzt sein. Viele Browser (z.B. Google Chrome ab Version 80) setzen **samesite=Lax** als Standardwert, wenn kein expliziter **samesite**-Wert angegeben ist.

Wichtig: Cookies werden an den Browser im Response-Header gesendet. Daher gilt für **setcookie()** dasselbe wie für **header()**: Der Aufruf muss *vor* jeglichem Output geschehen.

Session-Cookies

Sogenannte Session-Cookies sind der einfachste Fall eines Cookies. Sie haben kein Ablaufdatum und werden daher mit Schließen des Browsers wieder gelöscht. Ein Session-Cookie kann wie folgt erzeugt werden:

```
1 setcookie("user", "John Doe");
```

In diesem Fall wird unter dem Namen **user** der Wert **John Doe** abgespeichert.

Cookies mit Ablaufdatum

Um Cookies längerfristig zu speichern, muss ein Ablaufdatum gesetzt werden. Dieses muss in Sekunden seit dem 1.1.1970 vorliegen. Um einen sogenannten Unix-Zeitstempel zu erzeugen, kann die Funktion **time()**⁴ verwendet werden. Diese gibt den Zeitstempel für den aktuellen Augenblick zurück. Soll das Cookie von diesem Moment an genau eine Woche gültig sein, werden die entsprechenden Sekunden ($60 \cdot 60 \cdot 24 \cdot 7 = 604.800$) addiert (der Name des Cookies wird geändert, um sich vom Session-Cookie zu unterscheiden):

```
1 setcookie("username", "Jane Doe", ["expires" => time() + 604800]);
```

8.2.4 Cookies auslesen

Ruft der Browser eine Seite auf, für die ein gültiges Cookie in seinem lokalen Speicher existiert, so schickt er dieses automatisch an den Server. Dort kann es in PHP über ein weiteres superglobales Array, nämlich **\$_COOKIE**, ganz einfach ausgelesen werden. Der Schlüssel für den Zugriff ist der Name des Cookies. Um den Benutzer*innennamen aus dem zweiten der oben gesetzten Cookies auszulesen, ist folgender Code nötig:

```
1 echo "<p>Hello {$_COOKIE['username']}!</p>";
```

8.2.5 Cookies löschen

Um ein Cookie zu löschen, wird ein neues Cookie mit demselben Namen, aber leermem Wert erzeugt, dessen Ablaufdatum in der Vergangenheit liegt. Dafür wird von der

⁴<https://www.php.net/manual/de/function.time.php>

Funktion `time()` einfach ein Wert abgezogen (z. B. 43.200 Sekunden, was 12 Stunden entspricht). Dies veranlasst den Browser dazu, das Cookie lokal zu entfernen. Wichtig ist dabei, dass der vierte Parameter `$path` gleich wie beim Setzen des Cookies sein muss, da es sonst nicht funktioniert. Ebenso kann der Wert im `$_COOKIE` Superglobal mit `unset()` entfernt werden, um den bereits empfangenen Wert des Cookies für PHP in der aktuellen Datei nicht mehr verfügbar zu machen. Denn für den aktuellen Request kam das Cookie vom Browser noch mit und wird erst mit der Response der PHP-Datei gelöscht. Der nächste Request enthält es dann nicht mehr, weshalb der Eintrag in `$_COOKIE` erst dann leer wäre.

```
1 if (isset($_COOKIE["username"])) {  
2     setcookie("username", "", ["expires" => time() - 43200]);  
3     unset($_COOKIE["username"]);  
4 }
```

8.3 Sessions

Wie bereits am Beginn von Abschnitt 8.2 erwähnt, stellt das zustandslose HTTP Webentwickler*innen vor Herausforderungen, wenn es um die Aufrechterhaltung von Zuständen geht. Auch, wenn es, wie soeben gezeigt, Mechanismen wie Cookies gibt, um Werte zwischen zwei Seitenaufrufen zu speichern, so stellt das Speichern und Auslesen vieler Werte doch einen ständigen (mitunter lästigen) Mehraufwand dar.

8.3.1 Allgemeines

Die Lösung für dieses Problem ist das Session-Management. In PHP wird dazu eine eindeutige ID – die sogenannte *Session-ID* – vergeben. Diese muss bei jedem Request mitgeschickt werden, wodurch wiederum zusammengehörige Requests identifiziert werden können. Informationen (in Form von Variablen) sind über diese zusammengehörigen Requests hinweg persistent, bleiben also erhalten. Ähnlich wie Variablen innerhalb einer PHP-Datei verfügbar sind, sind diese Variablen dann auf allen Seiten, die durch die Session-ID als zusammengehörig identifiziert wurden, verfügbar. Derartige zusammengehörige Seiten werden im Allgemeinen als *Session* bezeichnet.

Die Session-ID

Um nun die Session-ID zwischen verschiedenen Seiten mitführen zu können, existieren drei verschiedene Mechanismen:

- Cookies: Die Session-ID wird in einem Cookie beim Browser gespeichert.
- URL-Codierung (URL-Rewriting): Die Session-ID wird in Form eines GET-Parameters an den URL gehängt.
- Versteckte Formularfelder: Die Session-ID wird in versteckten Formularfeldern mitgeschickt (selten bis gar nicht verwendet).

Cookies sind die Standardvariante, mit der die Session-ID übertragen wird. Das Cookie erhält dabei einen Namen – meist `PHPSESSID` – der Wert ist die generierte ID. Dies ist die einfachste Variante. Sie ist transparent für Benutzer*innen und einfach für Entwickler*innen, hat aber natürlich den Nachteil, dass Sessions bei ausgeschalteten

Cookies nicht verfügbar sind. Dies ist der Grund, warum viele Webseiten explizit darauf hinweisen, dass Cookies für einen reibungslosen Betrieb aktiviert sein müssen.

Damit der Webserver Cookies zur Übertragung der Session verwendet, muss die Einstellung `session.use_cookies = 1` in der Datei `php.ini` gesetzt sein. Dies ist jedoch in der Regel der Fall.

URL-Rewriting allgemein

URL-Rewriting ist eine Alternative zur Übertragung der Session-ID. Um es zu implementieren, muss jeder Link auf eine zur Session gehörende Folgeseite dynamisch um die Session-ID erweitert werden. Diese wird wie ein GET-Parameter mitgeführt: `http://localhost/sessions/index.php?PHPSESSID=104f342ad28f82f`

Diese Variante hat jedoch einige Nachteile. Wird die Seite als Bookmark gespeichert, wird die Session-ID mitgespeichert. Diese gilt jedoch zu einem späteren Zeitpunkt vielleicht nicht mehr. Ebenso muss bei einer Rückkehr auf die Seite (solange der Browser geöffnet bleibt) die Session-ID wieder an die URL gehängt werden, sonst geht die Session verloren. Des Weiteren stellt das Anhängen der Session-ID im Klartext ein Sicherheitsrisiko dar und begünstigt Angriffe wie etwa Session-Hijacking.

8.3.2 PHP-Sessions

PHP besitzt ein sehr klares und einfaches Session-Konzept. Eine Session wird am Beginn einer Seite gestartet oder fortgeführt. Danach können Variablen in der Session über ein superglobales Array gespeichert und ausgelesen werden. Die Übergabe der Session-ID geschieht standardmäßig über Cookies, ist aber auch per URL-Rewriting möglich. Am Server werden die Session-Daten unter der Session-ID gespeichert. Das Verzeichnis dazu wird in der `php.ini` durch die Variable `session.save_path` festgelegt.

Starten einer Session

Um eine Session zu starten wird die Funktion `session_start()`⁵ verwendet. Dieser Aufruf sollte ganz zu Beginn der Seite stehen, jedenfalls aber bevor Output erzeugt wurde.

Ist noch keine Session vorhanden, wird eine neue Session-ID generiert und gespeichert. Ist bereits eine Session-ID (in einem Cookie) vorhanden, wird diese Session fortgeführt.

Durch den Aufruf der Funktion werden gespeicherte Sessionvariablen verfügbar. Diese werden im superglobalen Array `$_SESSION` abgelegt. Dieses Array dient also als Sessionspeicher und kann aktiv beschrieben werden. Soll eine Variable in der Session gespeichert werden, muss nur ein neuer Eintrag im Array angelegt werden:

```
1 $_SESSION["user"] = "John Doe";
```

Beispiel: Reload-Counter

Im folgenden Beispiel wird ein simpler Reload-Counter implementiert. Er zählt, wie oft eine Seite neu geladen wurde. Dafür wird zunächst eine Session gestartet, dann wird die

⁵<https://www.php.net/manual/de/function.session-start.php>



Abbildung 8.1: Die typischen Schritte eines Kommunikationsablaufs mit Sessions: vom initialen Request über die Erstellung, Speicherung und Weitergabe der Session bzw. ihrer ID und der Wiederherstellung der damit assoziierten Daten.

Anzahl der Reloads als Variable in der Session (genauer gesagt im Array `$_SESSION`) gespeichert. Ist kein Wert vorhanden, wird er mit 0 neu initialisiert, sonst einfach um 1 erhöht.

```

1 <?php
2 session_start();
3
4 if (!isset($_SESSION["counter"])) {
5     $_SESSION["counter"] = 0;
6 } else {
7     $_SESSION["counter"]++;
8 }
9
10 echo "Anzahl Reloads: {$_SESSION["counter"]}";
11 ?>

```

Ablauf einer Session

Am Beispiel des Reload-Counters lässt sich der Ablauf einer Session analysieren. Abbildung 8.1 stellt ihn dar.

Die folgenden 7 Schritte sind involviert.

1. Der initiale Request vom Browser wird an den Server gestellt. Die Session existiert noch nicht, darum ist auch kein Cookie involviert.
2. Am Server wird bei der aufgerufenen PHP-Seite (in der PHP-Engine) eine Session-ID erzeugt.
3. Der Server schreibt die Daten der Session in ein Session-File mit dieser ID. Die Session ist somit logisch am Server angelegt.
4. Die Response wird mit einem Cookie, das die Session-ID enthält, zurückgeschickt.
5. Das Cookie wird im Browser gespeichert.
6. Der nächste Request des Browsers schickt das Cookie mit der Session ID wieder zurück zum Server.
7. Dort wird die Session-ID mit der gespeicherten verglichen und die Daten können aus dem Session-File wiederhergestellt werden.

URL-Rewriting als Fallback, wenn Cookies nicht verfügbar sind

URL-Rewriting wurde früher als Fallback verwendet, wenn Cookies deaktiviert waren. Dabei wird die Session-ID über einen GET-Parameter im URL weitergegeben. Dies funktioniert über die Konstante `SID`. Sie enthält das komplette Schlüssel-Wert-Paar mit den Session-ID-Informationen. Aus Sicherheitsgründen (z. B. Gefahr von Session Hijacking durch mitgesendete URLs) wird dies heute nicht mehr empfohlen.

Moderne PHP-Versionen setzen in `php.ini` standardmäßig `session.use_only_cookies = 1`, wodurch URL-Rewriting gar nicht mehr automatisch funktioniert (`SID` enthält dann keinen Wert). Dieses Verhalten dient der Sicherheit und ist seit PHP 5.3 (2009) Standard.

Beenden einer Session

Um eine Session zu beenden, sind mehrere Schritte notwendig. Zwar löscht die Funktion `session_destroy()`⁶ alle, mit einer Session in Verbindung stehende Daten am Server, das globale Array `$_SESSION` sowie das Cookie zur Übertragung der Session-ID sind jedoch nicht betroffen. Diese müssen daher extra gelöscht werden.

Der komplette Vorgang zum Löschen einer Session sieht wie folgt aus. Dieser Code findet sich oft in einer eigenen Datei (z. B. `logout.php`).

```
1 session_start();
2
3 $_SESSION = [];
4
5 if (isset($_COOKIE[session_name()])) {
6     $params = session_get_cookie_params();
7     setcookie(
8         session_name(),
9         "",
10        [
11            "expires" => time() - 42000,
12            "path" => $params["path"],
13            "domain" => $params["domain"],
14            "secure" => $params["secure"],
15            "httponly" => $params["httponly"],
16            "samesite" => $params["samesite"],
17        ],
18    );
19 }
20
21 session_destroy();
```

Zunächst wird die Session mittels `session_start()` fortgeführt. Im darauffolgenden Schritt wird das superglobale Array `$_SESSION` gelöscht, indem es mit einem leeren Array überschrieben wird. Der dritte Schritt ist das Löschen des Session-Cookies. Dazu wird zunächst überprüft, ob es gesetzt ist. Wenn dies der Fall ist, werden mittels `session_get_cookie_params()`⁷ die einzelnen Informationen des Session Cookies (Ablaufdatum, Pfad, Domain, HTTP only und SameSite) abgefragt. Danach wird das Session Cookie mit einem ungültigen Cookie überschrieben. Dieses erhält den Namen des Session Cookies (abgefragt durch `session_name()`⁸), hat jedoch keinen Inhalt. Un-

⁶<https://www.php.net/manual/de/function.session-destroy.php>

⁷<https://www.php.net/manual/de/function.session-get-cookie-params.php>

⁸<https://www.php.net/manual/de/function.session-name.php>

gültig wird es dadurch, dass sein Ablaufdatum in der Vergangenheit liegt. Dazu wird vom aktuellen Zeitstempel einfach ein frei gewählter Sekundenbetrag subtrahiert. Der Browser erkennt dadurch, dass das Cookie abgelaufen ist und löscht es. Schließlich wird durch den Aufruf von `session_destroy()` die Session auch logisch am Server beendet.

8.3.3 Klassen und Sessions

Bei den Abschnitten über Vererbung und Interfaces in Vorlesung 4 wurde bereits gezeigt, dass PHP zwingend den Inhalt einer Basisklasse oder eines Interface kennen muss, bevor damit gearbeitet werden kann. Da Basisklasse und abgeleitete Klasse bzw. Interface meist in verschiedenen Dateien liegen, müssen diese mittels `include` oder `require` (ggf. mit Hilfe eines Autoloaders) zunächst eingebunden werden.

Auch bei der Arbeit mit Sessions kommt ein ähnliches Thema zum Tragen. Beim Session-Management werden alle Session-Variablen zur Übergabe zwischen den einzelnen Seiten in einer temporären Datei am Webserver abgespeichert. Dieser Vorgang wird Serialisierung genannt. Werden die Variablen auf der Folgeseite wiederhergestellt, wird von Deserialisierung gesprochen.

Nicht nur einfache Datentypen können serialisiert und deserialisiert werden. Auch Objekte können auf diese Art in der Session gespeichert werden. Die Wiederherstellung von Objekten ist jedoch komplexer als die eines einfachen Datentyps, da ein Objekt meist aus einer Reihe von verschiedenen Eigenschaften besteht. Nichtsdestotrotz kann PHP diesen Prozess vollständig automatisieren, sodass für Entwickler*innen kein Mehraufwand entsteht. Die einzige Voraussetzung ist, dass die Klassendefinition zum Zeitpunkt der Deserialisierung bekannt ist. Daher sollte diese am besten in eine eigene Datei ausgelagert werden und *vor* dem Aufruf von `session_start()` eingebunden werden.

Soll etwa ein Objekt der Klasse `Rectangle` in der Session gespeichert werden, so sollte die Klassendefinition in einer eigenen PHP-Datei namens `Rectangle.php` oder auch `Rectangle.inc.php` gespeichert sein:

```
1 // Rectangle.inc.php
2 class Rectangle implements GeometricComponentInterface
3 {
4     // ...
5 }
```

Erstreckt sich die Session etwa über zwei Dateien, so muss in beiden diese Datei vor dem Beginn/Weiterführen der Session eingebunden werden:

```
1 // SessionPage1.php
2 require("Rectangle.inc.php");
3 session_start();
4 // ...
```

```
1 // SessionPage2.php
2 require("Rectangle.inc.php");
3 session_start();
4 // ...
```

8.3.4 Session-Sicherheit und weitere Funktionen

Sessions stehen sehr oft in Zusammenhang mit geschützten oder sensiblen Daten, etwa im Falle eines Login-Mechanismus (siehe Abschnitt 8.4) oder auch bei Dingen wie Onlineeinkäufen. Daher sollten im Umgang mit Sessions einige Dinge beachtet werden. Darüber hinaus gibt es weitere Funktionen im Zusammenhang mit Sessions, die für die Arbeit hilfreich sein können.

Session-ID geheim halten

Die Session-ID sollte nach Möglichkeit immer geheim gehalten werden, da sie den Schlüssel zum Übernehmen einer Session („Session Hijacking“) darstellt. Darum sollte auf Dinge wie URL-Rewriting aus Sicherheitsgründen verzichtet werden. Auch das Verwenden einer sicheren Verbindung (über SSL) trägt zur sicheren Übertragung der Session-ID bei.

Session-ID wechseln

Die Session-ID sollte im Verlauf einer Applikation an wichtigen Stellen gewechselt, d. h. neu generiert werden. Dies verhindert Session Hijacking in Kombination mit der sogenannten „Session Fixation“, wo ein*e Angreifer*in eine Session-ID anfordert und diese einem Opfer mit einem Link etwa zum Login unterjubelt. Meldet sich der*die User*in an, so gilt die Session-ID als eingeloggt und der*die Angreifer*in hat Zugriff zum System. Wird die Session-ID jedoch nach einem Login automatisch neu generiert, bietet sich dieses Angriffsszenario nicht. PHP bietet mit `session_regenerate_id()`⁹ eine einfache Möglichkeit, die Session-ID zu wechseln.

Session benennen

Die Funktion `session_name()`¹⁰ erlaubt es, Sessions zu benennen bzw. den gesetzten Namen abzufragen. Der Standardwert lautet `PHPSESSID`, existieren jedoch auf einem Server zwei Webapplikationen (etwa ein Blog und ein Wiki), so kann es vorkommen, dass beide diesen Standardwert verwenden und Sessions daher (unfreiwillig) geteilt bzw. wiederverwendet werden. Um dies zu trennen, können beide Applikationen einen eindeutigen Sessionnamen wählen. Der Aufruf muss dabei *vor* `session_start()` erfolgen.

Status einer Session abfragen

Um zu erfragen, ob eine Session existiert oder die aktuellen Einstellungen überhaupt Sessions zulassen, kann `session_status()`¹¹ zum Einsatz kommen. Die Funktion liefert eine Ganzzahl zurück, die sich gegen die folgenden drei Konstanten vergleichen lässt:

1. `PHP_SESSION_DISABLED`: Sessions sind in der Datei `php.ini` deaktiviert.
2. `PHP_SESSION_NONE`: Sessions sind aktiviert, es existiert jedoch keine.
3. `PHP_SESSION_ACTIVE`: Sessions aktiviert und es gibt derzeit eine aktive.

⁹<https://www.php.net/manual/en/function.session-regenerate-id.php>

¹⁰<https://www.php.net/manual/en/function.session-name.php>

¹¹<https://www.php.net/manual/en/function.session-status.php>

Session-ID setzen oder abfragen

Mittels `session_id()`¹² kann die aktuelle Session-ID abgefragt oder auch selbst generiert werden. Für letzteres gibt es jedoch Auflagen hinsichtlich verwendeter Zeichen und Länge. Wird die Session-ID selbst generiert, muss dies vor dem Aufruf von `session_start()` geschehen.

Session-Einstellungen in `php.ini` überschreiben

Seit PHP 7.0 können beim Starten einer Session Session-Konfigurationswerte aus der Datei `php.ini` überschrieben werden. Dazu wird `session_start()` mit einem assoziativen Array aufgerufen, das die zu überschreibenden Parameter neu setzt. Im folgenden Beispiel wird eine Session gestartet in der die Lebensspanne eines Session-Cookies (bis zum Schließen des Browsers) statt des Standardwerts von 0 eine Stunde (3.600 Sekunden) beträgt.

```
1 session_start(["cookie_lifetime" => 3600]);
```

Die folgenden Werte können angepasst werden (jeweils ohne den Prefix `session.`): <https://www.php.net/manual/de/session.configuration.php>.

8.4 Authentifizierung mit PHP

Unter *Authentifizierung* (auch Authentifikation) im Web wird ganz allgemein der Nachweis einer behaupteten Eigenschaft einer Partei verstanden. Im Falle von Computersystemen wird dabei in der Regel die Identität einer Person oder eines Programms nachgewiesen. Den positiven Abschluss einer Authentifizierung nennt man auch Autorisierung und dieser gestattet wiederum im Fall von Computersystemen den Zugriff auf bestimmte Bereiche oder Ressourcen, die zuvor nicht erreichbar waren.

Ein gängiger Mechanismus zur Authentifizierung ist die Abfrage eines Benutzer*innennamens in Kombination mit einem Passwort. Andere Möglichkeiten sind die modernen Passkeys, Login-PINs, die Verwendung von Hardware-Token oder auch biometrische Merkmale wie Fingerabdrücke.

Im Folgenden wird der Schutz einer Seite mittels Authentifizierung durch eine typische Login-Seite mit Benutzer*innenname (in Form einer E-Mail-Adresse) und Passwort basierend auf Sessions und Redirects erläutert.

8.4.1 Funktionsweise und Ablauf

Eine Voraussetzung für die Implementierung mittels PHP ist der Browser-Redirect über HTTP-Header. Wie bereits in Abschnitt 8.1.2 besprochen, bewirkt dieser, dass der Browser vom PHP-Skript angewiesen werden kann, eine bestimmte Seite zu laden. Die Syntax zur Wiederholung:

```
1 header("Location: newSite.php");
```

Nach diesem Aufruf wird die Seite `newSite.php` angezeigt, als Statuscode wird *302 Redirect* angegeben. Vor dem Aufruf von `header()` darf keinerlei Ausgabe erfolgen. In

¹²<https://www.php.net/manual/en/function.session-id.php>

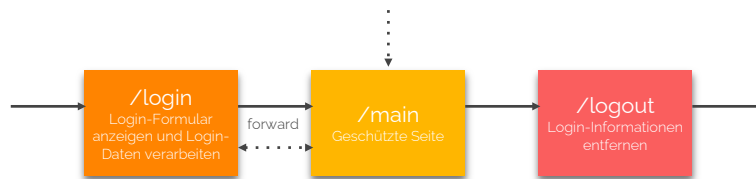


Abbildung 8.2: Die logische Abfolge der Routenaufrufe ist mit den durchgehenden Pfeilen dargestellt. Mögliche Abweichungen mit den gestrichelten. Wird `GET /main` direkt ohne Login aufgerufen, wird sofort auf `GET /login` weitergeleitet. Wird der Login dort korrekt durchgeführt, geschieht die Weiterleitung auf `GET /main` und die Seite darf betreten werden.

einem allgemeineren Fall kann auch ein vollständiger URL angegeben werden. In diesem Beispiel wird auf eine Seite am gleichen Webserver weitergeleitet, die auch im selben Verzeichnis liegt, wie die Datei, von der aus der Redirect erfolgt.

Die vorgestellte *PHP-Authentifizierung* ist eine mit `fhooe/router` realisierte Applikation. Die folgenden vier Routen sind involviert:

- `GET /login` & `POST /login`: Zeigt ein Formular mit den Feldern für Benutzer*innenname in Form einer E-Mail-Adresse und einem Passwort für einen Login an. Beim Abschicken wird in `POST /login` überprüft, ob die Angaben korrekt sind und ggf. auf die Route der Hauptseite `GET /main` weitergeleitet. Bei falschen Angaben wird auf der Seite verweilt.
- `GET /main`: Die Hauptseite mit dem eigentlichen Inhalt. Sie darf nur betreten werden, wenn ein korrekter Login durchgeführt wurde. Daher wird zunächst immer überprüft, ob dieser stattgefunden hat. Wenn nicht, wird sofort auf `GET /login` umgeleitet.
- `GET /logout`: Führt einen vollständigen Logout durch und entfernt alle Informationen über einen korrekten Login.

Der Ablauf zwischen diesen drei Routen ist in Abbildung 8.2 dargestellt.

8.4.2 Erzeugen der Login-Daten

Bevor mit dem Login begonnen werden kann, müssen überhaupt *Login-Daten* vorhanden sein, auf die später überprüft werden kann. In der Regel können sich Benutzer*innen auf einer Registrierungsseite einen Benutzer*innennamen (in Form einer E-Mail-Adresse) mit dazugehörigem Passwort angeben. Beide Werte müssen nun im System gespeichert werden, damit später beim Login überprüft werden kann, ob die korrekte Kombination eingegeben wurde. Der Benutzer*innennamen kann dabei im Klartext gespeichert werden, das Passwort sollte jedoch *niemals* in lesbarer Form gespeichert werden. Egal ob die Speicherung dabei in einer Datenbank oder einer Textdatei erfolgt. Klartextpasswörter stellen ein Sicherheitsrisiko dar, falls sich Angreifer*innen unerlaubten Zugriff auf den Speicherort verschaffen. Daher ist eine Verschlüsselung unumgänglich.

Passwörter verschlüsseln

Die derzeit sicherste Methode zur *Verschlüsselung von Passwörtern* ist die Verwendung der Funktion `password_hash($password, $algo, $options)`¹³. Der erste Parameter ist das zu hashende Passwort, der zweite spezifiziert den zu verwendenden Algorithmus. Dabei empfiehlt es sich, entweder `PASSWORD_DEFAULT` anzugeben (was den Standard-Algorithmus der aktuellen PHP-Version, meist `bcrypt`, verwendet) oder direkt auf moderne Standards wie `PASSWORD_ARGON2ID` zu setzen. Das Ergebnis ist ein Hashwert, der den Salt und die Informationen über den verwendeten Algorithmus bereits enthält, sodass die Gegenprüfung ganz ohne zusätzliche Datenbankfelder erfolgen kann.

Um einen mit dieser Funktion generierten Passwort-Hash zu überprüfen, existiert die Funktion `password_verify($password, $hash)`.¹⁴ Der erste Parameter ist dabei das zu überprüfende Passwort im Klartext, das zweite Argument ist der gehashte Wert aus der Datenbank. Die Funktion führt den Hash-Vorgang erneut durch; wenn das Ergebnis übereinstimmt, gibt sie `true`, ansonsten `false` zurück.

Um zu überprüfen, ob der im gespeicherten Hash verwendete Algorithmus oder dessen Parameter noch aktuell sind, kann `password_needs_rehash($hash, $algo)`¹⁵ nach dem erfolgreichen Login verwendet werden. Falls sich die Anforderungen geändert haben (z. B. Server-Upgrade oder höhere Sicherheitsrichtlinien), gibt die Funktion `true` zurück. Das Passwort kann dann aus dem Klartext der aktuellen Login-Anfrage direkt neu verschlüsselt und in der Datenbank aktualisiert werden. Somit ist stets die höchste Sicherheit gewährleistet.

Sollen Informationen über einen mit `password_hash()` erzeugten Hash abgefragt werden, kann `password_get_info($hash)`¹⁶ zum Einsatz kommen. Es gibt ein assoziatives Array mit den Schlüsseln `algo`, `algoName` und `options` zurück, welches die verwendeten Speicher- und Zeitkosten des Hash-Vorgangs offenlegt.

Diese nativen PHP-Passwort-Funktionen sollten immer jeglicher selbstgeschneider-ten Lösung vorgezogen werden, um sicheres Passwortmanagement zu gewährleisten.

8.4.3 Der Login-Vorgang

Beim *Login-Vorgang* wird ein Benutzer*innenname in Form einer E-Mail-Adresse mit Passwort abgefragt. Stimmen beide mit dem gespeicherten Datensatz überein, darf die Hauptseite betreten werden. Wird die Hauptseite direkt betreten, ohne vorher einen Login durchgeführt zu haben, wird auf die Login-Seite verwiesen. In den folgenden Abschnitten werden die einzelnen Codestücke beschrieben, die zum Ablauf notwendig sind.

Definitionen der Routen

Zunächst werden in der Hauptdatei `index.php` (im Verzeichnis `public`) die Routen angelegt. Neben der Hauptroute `GET /`, welche der Startpunkt des Ablaufs ist, werden vier weitere Routen angelegt:

¹³ <http://php.net/manual/de/function.password-hash.php>

¹⁴ <https://www.php.net/manual/de/function.password-verify.php>

¹⁵ <https://www.php.net/manual/de/function.password-needs-rehash.php>

¹⁶ <https://www.php.net/manual/de/function.password-get-info.php>

- GET /login: Zeigt das Login-Formular an.
- POST /login: Verarbeitet das Login-Formular und überprüft, ob korrekte Login-Daten vorhanden sind.
- GET /main: Zeigt die Hauptseite an.
- GET /logout: Entfernt alle gültigen Login-Informationen.

Die Definitionen hierfür lauten wie folgt:

```

1 $router->get("/login", function () use ($latte, $router) {
2     // Login-Formular anzeigen
3 });
4
5 $router->post("/login", function () use ($latteig, $router) {
6     // Login-Formular verarbeiten
7 });
8
9 $router->get("/main", function () use ($latte, $router) {
10    // Geschützte Hauptseite anzeigen
11 });
12
13 $router->get("/logout", function () use ($router) {
14    // Ausloggen
15 });

```

Die ersten drei Routen sollen eine Response (HTML-Output) anzeigen und bekommen daher das Latte-Objekt übergeben, damit sie die entsprechenden Templates ausgeben können. Alle Routen bekommen das Router-Objekt mitgegeben, damit sie Weiterleitungen auf andere Routen durchführen können.

Überprüfung der Login-Daten

Die Route GET /login startet den eigentlichen Login-Vorgang. Sie muss zunächst lediglich ein Latte-Template mit dem Login-Formular ausgeben:

```

1 $router->get("/login", function () use ($latte, $router) {
2     $latte->render("login.latte");
3 });

```

Das Formular spezifiziert in seinem action-Attribut, dass die Route POST /login aufgerufen wird. Somit findet die gesamte Verarbeitung in der Handler-Funktion dieser Route statt:

```

1 $router->post("/login", function () use ($latte, $router) {
2     $login = new Login($latte, $router);
3     $login->isValid();
4     $login->displayOutput();
5 });

```

Zunächst wird ein Objekt der Klasse Login angelegt. Diese Klasse kapselt alle Funktionalitäten, um die Eingaben aus dem Formularfeld zu validieren, ggf. Fehlermeldungen anzuzeigen und bei Erfolg auf die geschützte Seite weiterzuleiten.

Die Klasse hat eine Eigenschaft `$pdo` für die PDO-Datenbankverbindung, ein Array `$messages`, das mögliche Fehlermeldungen speichert, ein Latte-Objekt `$latte`, welches die Templatefunktionalitäten beinhaltet und ein Router-Objekt `$router` für Weiterleitungen.

Im Konstruktor der Klasse werden die übergebenen Latte- und Router-Objekte gespeichert und eine Datenbankverbindung aufgebaut.

```
1 final class Login
2 {
3     private PDO $pdo;
4     private array $messages = [];
5     private Engine $latte;
6     private Router $router;
7
8     public function __construct(Engine $latte, Router $router)
9     {
10         $this->latte = $latte;
11         $this->router = $router;
12         $this->initDB();
13     }
14
15     // ...
16 }
```

In der Methode `isValid()` muss nun überprüft werden, ob die eingegebenen Daten korrekt sind:

```
1 public function isValid(): void
2 {
3     if (Utilities::isEmptyString($_POST["email"])) {
4         $this->messages["email"] = "Please enter your email address.";
5     } elseif (!Utilities::isEmail($_POST["email"])) {
6         $this->messages["email"] = "Please enter a valid email address";
7     }
8     if (Utilities::isEmptyString($_POST["password"])) {
9         $this->messages["password"] = "Please enter your password.";
10    }
11
12    if (count($this->messages) === 0) {
13        if (!$this->authenticateUser()) {
14            $this->messages["login"] = "Invalid account email address or password.";
15        } else {
16            $this->business();
17        }
18    }
19 }
```

Beide Felder – die E-Mail-Adresse und das Passwort – dürfen zunächst nicht leer sein. Bei der E-Mail-Adresse wird weiters überprüft, ob es sich auch um eine korrekte Format handelt. Dies wird mit einer statischen Hilfsfunktion aus der Klasse `Utilities` erledigt.

Sind diese Punkte sichergestellt, wird die Methode `authenticateUser()` aufgerufen, die überprüft, ob die angegebene E-Mail-Passwort-Kombination in den gespeicherten Login-Daten enthalten ist. Gibt die Methode `true` zurück, sind beide Angaben korrekt, bei `false` sind die Daten falsch. Aus Sicherheitsgründen wird nicht angegeben, ob E-Mail-Adresse oder Passwort oder beides falsch bzw. nicht vorhanden ist, sondern nur eine generelle Fehlermeldung.

Die Methode `authenticateUser()` fragt nun aus der Tabelle `user` den Datensatz ab, dessen E-Mail-Adresse mit der eingegebenen übereinstimmt. Erhält das Ergebnis einen

Datensatz, so wird das Passwort dieses Datensatzes mit dem eingegebenen verglichen. Dabei kommt, wie in Abschnitt 8.4.2 beschrieben, `password_verify()` zum Einsatz. Gibt die Funktion `true` zurück, handelt es sich um das korrekte Passwort und der*die Benutzer*in gilt als korrekt authentifiziert.

Mit einer zusätzlichen Abfrage wird noch überprüft, ob das Passwort neu verschlüsselt werden muss. Mit Hilfe von `password_needs_rehash()` wird getestet, ob der gespeicherte Hash noch den Anforderungen an den aktuellen gewünschten Algorithmus (hier `PASSWORD_DEFAULT`) erfüllt. Ist dies der Fall, passiert nichts, ansonsten wird das Passwort mit neu gehashed und der Eintrag in der Datenbank mittels `updateUser()` aktualisiert.

Ein*e User*in gilt dann *nicht* als korrekt authentifiziert, wenn die E-Mail-Adresse nicht in den gespeicherten Datensätzen vorhanden war oder das eingegebene Passwort nicht mit dem gespeicherten übereinstimmt.

```
1 private function authenticateUser(): bool
2 {
3     $query = "SELECT user_id, email, password FROM user WHERE email = :email";
4     $params = [":email" => $_POST["email"]];
5
6     $statement = $this->pdo->prepare($query);
7     $statement->execute($params);
8     $row = $statement->fetch();
9
10    if ($row && password_verify($_POST["password"], $row["password"])) {
11        if (password_needs_rehash($row["password"], PASSWORD_DEFAULT)) {
12            $password = password_hash($_POST["password"], PASSWORD_DEFAULT);
13            $this->updateUser($row["user_id"], $password);
14        }
15        return true;
16    }
17    return false;
18 }
```

Weiterleitung bei erfolgreichem Login

Wurden E-Mail-Adresse und Passwort als gültig anerkannt, erfolgt in `business()` die Weiterleitung auf den geschützten Bereich, z. B. Route `GET /main`. Dafür wird auf die Methode `redirectTo()` der Klasse `Router` zurückgegriffen. Diese verwendet die PHP-Funktion `header()` gefolgt von einem `exit()`-Aufruf und erlaubt als Parameter eine einfache Routen-Angabe. Diese wird in einen vollen URL aufgelöst und die Weiterleitung erfolgt. Als Wert wird hier `/main` übergeben.

Vor der eigentlichen Weiterleitung werden noch wichtige Werte in die Session geschrieben. Zunächst die E-Mail-Adresse des*der angemeldeten Benutzers*Benutzerin. Somit ist diese auf den Folgeseiten verfügbar. Weiters wird durch Aufruf der statischen Methode `generateLoginToken()` aus der `Utilities`-Klasse noch ein kryptografisch sicheres Token generiert. Die Methode verwendet die Funktion `random_bytes(32)`, um 32 zufällige, kryptografisch sichere, Bytes zu erzeugen. Die erzeugten Bytes werden anschließend mit der Funktion `bin2hex()` in eine hexadezimale Darstellung umgewandelt, um sicherzustellen, dass das Token nur aus lesbaren Zeichen besteht. Später wird auf das Vorhandensein dieses Tokens in der Session überprüft, um als eingeloggt zu gelten.

Um die Sicherheit noch weiter zu erhöhen könnte das Token zusätzlich noch clientseitig (etwa in einem versteckten Formularfeld oder in einem Header) übertragen werden. Solche *Client-synchronisierten* Token müssen dann auch validiert (verglichen) werden, bieten aber erweiterten Schutz gegen Session-Fixation, Session-Hijacking oder CSRF. Solange die Session sicher ist, ist dieser erweiterte Mechanismus nicht zwingend nötig.

Die zuvor erwähnte Methode `business()` in der Klasse `Login` sieht wie folgt aus:

```
1 protected function business(): void
2 {
3     $_SESSION["email"] = $_POST["email"];
4
5     $_SESSION["loginToken"] = Utilities::generateLoginToken();
6
7     $this->router->redirectTo("/main");
8 }
```

Die verwendete Hilfsmethode ist in `Utilities.php` wie folgt definiert:

```
1 public static function generateLoginToken(): string
2 {
3     return bin2hex(random_bytes(32));
4 }
```

Da ein Login von einem einzigen Wert in der Session abhängt, ist es wichtig, diese gut zu sichern. So sollte das Session-Cookie nur über HTTPS übertragen und zugreifbar sein und die Same-Site-Policy auf Strict gesetzt sein. Dies kann beim `session_start()`-Aufruf als Optionen in einem Array spezifiziert werden:

```
1 session_start([
2     "cookie_secure" => true,
3     "cookie_httponly" => true,
4     "cookie_samesite" => "Strict",
5 ]);
```

Browser sehen `localhost` als sichere Quelle an, weshalb dies auch über `http://localhost:8080` funktioniert. Auf nicht-lokalen Webservern muss HTTPS verwendet werden, sonst wird das Cookie nicht mit dem Request an den Server zurückgeschickt und der gesamte Ablauf schlägt fehl.

Zusätzliche Sicherheit bietet das Beschränken des Cookies auf Domain und Pfad, das (Um-)Benennen des Session-Cookies (um zu verhindern in mehreren Applikationen am Server eingeloggt zu sein, wenn der Login-Mechanismus derselbe ist) und das Erneuern der Session-ID nach erfolgreichem Login mittels `session_regenerate_id()`.

Anzeige von Fehlermeldungen bei erfolglosem Login

War der Login erfolgreich, erfolgte aufgrund der Weiterleitung auf die geschützte Seite keinerlei Ausgabe. Gibt es jedoch Probleme, weil die Benutzer*innendaten nicht mit den gespeicherten übereinstimmen, so muss wieder das Login-Formular angezeigt werden. Dies geschieht in der Methode `displayOutput()`, die nach `isValid()` aufgerufen wird und eben nur dann auch tatsächlich ausgeführt wird, wenn die Überprüfung nicht erfolgreich war.

Die Methode zeigt das Latte-Template des Login-Formulars an, übergibt aber zusätzlich die bereits eingegebene E-Mail-Adresse, welche in `value`-Attribut des Eingabefeldes eingefügt wird. Somit ist die Adresse bereits vorausgefüllt. Weiters wird das

Array mit den Fehlermeldungen übergeben, welches diese im Template anzeigt, wenn `messages` übergeben wurde.

```
1 public function displayOutput(): void
2 {
3     $this->latte->render("login.latte", [
4         "email"     => $_POST["email"],
5         "messages" => $this->messages,
6     ]);
7 }
```

Absichern logingeschützter Seiten

Der Login-Mechanismus ist soweit komplett, jedoch könnte immer noch die Route der zu schützenden Seite (`GET /main`) ohne Login erreicht werden, da dort noch nicht überprüft wird, ob ein Login stattgefunden hat. Dies wird dadurch erreicht, dass vor dem Anzeigen des Templates der Hauptseite eine Abfrage auf das in der Session gespeicherten Login-Token getätigt wird. Ist dieses nicht vorhanden, wird zurück auf die Login-Seite geleitet, noch bevor die Hauptseite angezeigt wird. Diese Funktionalität ist als statische Methode `requireLoggedIn()` in einer Klasse `RouteGuard` angelegt.

Der Aufruf sieht wie folgt aus:

```
1 $router->get("/main", function () use ($latte, $router) {
2     RouteGuard::requireLoggedIn("loginToken", $router, "/login");
3     $latte->render("main.latte");
4 });
```

Die Methode ist folgendermaßen implementiert:

```
1 public static function requireLoggedIn(string $tokenName, Router $router, string
    $exitRoute): void
2 {
3     if (!isset($_SESSION[$tokenName])) {
4         $router->redirectTo($exitRoute);
5     }
6 }
```

Eine ähnliche Absicherung kann bei Bedarf auch für die Route `GET /login` verwendet werden. Sind User*innen eingeloggt, macht der Aufruf der Login-Seite wenig Sinn. Daher kann mit der umgekehrten Bedingung bei einem erfolgreichen Login direkt wieder auf die Hauptseite weitergeleitet werden. Dies ist in der Methode `requireNotLoggedIn()` in der Klasse `RouteGuard` implementiert:

```
1 public static function requireNotLoggedIn(string $tokenName, Router $router, string
    $exitRoute): void
2 {
3     if (isset($_SESSION[$tokenName])) {
4         $router->redirectTo($exitRoute);
5     }
6 }
```

Damit ändert sich die Routendefinition für `GET /login` wie folgt:

```
1 $router->get("/login", function () use ($latte, $router) {
2     RouteGuard::requireNotLoggedIn("loginToken", $router, "/main");
3     $latte->render("login.latte");
4 });
```


Logout

Benutzer*innen gelten so lange als eingeloggt, so lange die Session bzw. der Hashwert darin existiert. Während dieser Zeit können sie sich nicht erneut anmelden, da ja sofort auf die Hauptseite weitergeleitet würde, wenn ein korrekter Login existiert. Um den*die Benutzer*in abzumelden, muss die Session gelöscht werden, dies bewirkt automatisch einen *Logout*.

Die Route GET `/logout` enthält den notwendigen Code. Sie muss keinerlei HTML-Output erzeugen, sondern nur die Session zerstören und danach auf eine andere Route weiterleiten. Das Löschen der Session geschieht wie in Abschnitt 8.3.2 beschrieben. Zunächst wird das superglobale Session-Array gelöscht, dann das Session-Cookie überschrieben und schließlich die Session logisch entfernt.

```
1 $router->get("/logout", function () use ($router) {  
2     $_SESSION = [];  
3     if (isset($_COOKIE[session_name()])) {  
4         setcookie(session_name(), "", [  
5             "expires" => time() - 86400,  
6             "path"     => "/",  
7         ]);  
8     }  
9     session_destroy();  
10    $router->redirectTo("/");  
11 });
```

8.5 Weiterführende Literatur

Die Themen HTTP-Request-Header, Cookies und Sessions werden sehr ausführlich in [41] behandelt. Jedoch deckt das Buch dabei nur unwesentlich mehr Details ab, als in dieser Vorlesung vorgestellt wurden.

[44, 49] widmen den Themen Sessions und Cookies jeweils ganze Kapitel, die sowohl eine Einführung bieten als auch konkrete Beispiele besprechen.

Literatur zum Thema Authentifizierung im Web ist sehr dünn gestreut.

[44, 49] diskutieren beide jeweils ein PHP-Login-System, welche mit Datenbanken arbeiten. Sie funktionieren auf einer ähnlichen Basis, ist jedoch etwas rudimentärer implementiert.

Vorlesung 9

Datum, Zeit und Internationalisierung

9.1 Datum und Zeit

Datum und Uhrzeiten sind wichtige Dinge in fast allen PHP-Applikationen. Sei es, um den Zeitpunkt eines Forenpostings zu kennzeichnen, die genaue Zeit zu der ein*e User*in das letzte Mal angemeldet war, die Berechnung von Zeiträumen, um eine Online-Reservierung vorzunehmen oder einfach nur, um die aktuelle Uhrzeit anzuzeigen. Die Arbeit mit Datum und Uhrzeit ist jedoch eine komplexere Thematik als einfach nur die Ausgabe von Zahlen und Zeichenketten. Verschiedene Zeitzonen, Zeit- und Datumsformate, Schaltjahre, Sommer- und Winterzeit, die Addition von Stunden zu einer Uhrzeit, die einen Tageswechsel zur Folge haben – kurzum, viele Dinge machen eine händische Berechnung und Verwaltung von Datum und Uhrzeit zur Herausforderung.

Daher sollte unbedingt auf die von PHP bereitgestellten Klassen `DateTime` (zur Verwaltung von Datum und Uhrzeit), `DateInterval` (zur Verwaltung von Zeiträumen) und `DateTimeZone` (zur Arbeit mit Zeitzonen) zurückgegriffen werden.

9.1.1 Datum und Zeit ausgeben

Um ein Datum und/oder die Zeit auszugeben, wird also immer ein Objekt der Klasse `DateTime` benötigt. Wird dieses ohne jegliche Parameter angelegt, so enthält es das aktuelle Datum und die Uhrzeit:

```
1 $d = new DateTime();
```

Um diese auszugeben, enthält das Objekt die Methode `format($format)`¹, die unter Angabe eines bestimmten Formats diese ausgibt. Dabei können die Angaben der Funktion `date()`² verwendet werden, Tabelle 9.1 gibt einen Auszug über einige Formatparameter.

Das aktuelle Datum und die Uhrzeit werden zunächst mit der Option `r` ausgegeben, dann mit der Option `d.m.Y, H:i:s e`. Das Ergebnis ist in Abbildung 9.1 zu sehen.

```
1 echo "<p>It's now {$d->format('r')}</p>";  
2 echo "<p>It's now {$d->format('d.m.Y, H:i:s e')}</p>";
```

¹<https://www.php.net/manual/de/datetime.format.php>

²<https://www.php.net/manual/de/function.date.php>

Tabelle 9.1: Einige Formatzeichen zur Datums- und Uhrzeitformatierung. Die volle Auflistung findet sich unter <https://www.php.net/manual/de/datetime.format.php>.

Zeichen	Beschreibung	Beispielwerte
d	Tag numerisch, mit führender Null	01–31
j	Tag numerisch, ohne führende Null	1–31
l	Wochentag textuell, ausgeschrieben	Monday–Sunday
N	Wochentag numerisch	1 (Montag)–7 (Sonntag)
m	Monat numerisch mit führender Null	01–12
F	Monat textuell, ausgeschrieben	January–December
Y	Jahr, vierstellig	2026, 1975
L	Schaltjahr oder nicht	1 (ja), 0 (nein)
G	Stunde, 24 h, ohne führende Null	0–23
g	Stunde, 12 h, ohne führende Null	1–12
H	Stunde, 24 h, mit führender Null	00–23
h	Stunde, 12 h, mit führender Null	01–12
i	Minute, mit führender Null	00–59
s	Sekunde, mit führender Null	00–59
e	Zeitzonebezeichner	UTC, GMT, Europe/Vienna
c	Vollständiges Datum/Uhrzeit, ISO 8601	2026-06-02T10:30:00+02:00
r	Vollständiges Datum/Uhrzeit, RFC 2822	Tue, 02 Jun 2022 10:30:00+0200

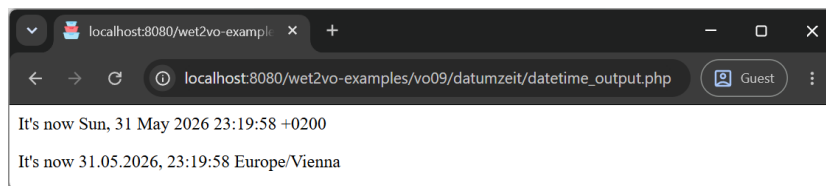


Abbildung 9.1: Die aktuelle Zeit wird einmal mit der Option **r**, dann mittels **d.m.Y, H:i:s e** ausgegeben.

9.1.2 Zeitzonen setzen

Unter Umständen wird im vorigen Beispiel nicht wie in Abbildung 9.1 „Europe/Vienna“, sondern „UTC“ (bzw. „+0000“ statt „+0200“), „Europe/Helsinki“ oder auch „America/Los_Angeles“ angezeigt. PHP übernimmt standardmäßig die am Webserver eingestellte Zeitzone, sofern diese nicht in der Datei `php.ini` explizit definiert ist. Steht dieser Webserver nun nicht in der lokalen Zeitzone oder seine Zeitzone ist einfach falsch konfiguriert, kommt es zur Ausgabe einer anderen bzw. unerwarteten Zeitzone.

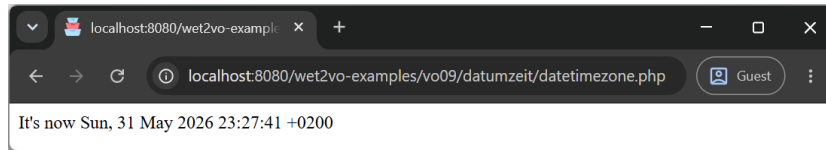


Abbildung 9.2: Der aktuelle Zeitstempel wird Dank `DateTimeZone`-Objekt in der richtigen Zeitzone dargestellt.

Standard-Zeitzone setzen

Dies kann auf zwei Arten gelöst werden. Zunächst kann die PHP-Konfiguration des aktuellen Webserver in der Datei `php.ini` mit folgender Konfiguration auf die für Österreich geltende Zeitzone gesetzt werden:

```
date.timezone = "Europe/Vienna"
```

Besteht keine Möglichkeit, dies bereits dort zu ändern, kann es auch zur Laufzeit im PHP-Code mit der Funktion `date_default_timezone_set()`³ gesetzt werden:

```
1 date_default_timezone_set("Europe/Vienna");
```

Eine Auflistung aller gültigen Zeitzone-Bezeichner findet sich unter <https://www.php.net/manual/de/timezones.php>.

Zeitzone verwalten

Oftmals ist es jedoch sinnvoll, UTC als Standard-Zeitzone anzunehmen bzw. zu belassen und alle Datums- und Zeitinformationen damit abzuspeichern (in Dateien oder einer Datenbank). UTC beachtet keine Sommer- und Winterzeit und kann als kleinster gemeinsamer Nenner oder Ausgangspunkt gesehen werden, wenn eine Applikation mit Daten aus verschiedenen Zeitzone arbeiten muss. In diesem Fall ist es sinnvoller, die Zeitzone erst bei der Anzeige von Daten der User*innen zu setzen.

Dafür existiert die Klasse `DateTimeZone`, die Zeitzone im Zusammenhang mit der `DateTime`-Klasse repräsentiert. Objekte dieser Klasse werden wiederum mit den gängigen Zeitzone-Bezeichnern als Argument erzeugt.

Sie können dann etwa als zweites Argument beim Konstruktor eines `DateTime`-Objekts angegeben werden. So wird die Zeitzone nur für dieses eine Objekt und nicht global im PHP-System gesetzt. Im folgenden Beispiel wird genau dies gemacht, Abbildung 9.2 zeigt das Ergebnis.

```
1 $timezone = new DateTimeZone("Europe/Vienna");
2 $d1 = new DateTime("now", $timezone);
3 echo "<p>It's now {"$d1->format("r")}</p>";
```

Alternativ kann die Zeitzone auch mit der Methode `setTimezone($timezone)`⁴ nachträglich gesetzt werden. Eine zuvor festgelegte Zeit wird dabei entsprechend angepasst. Im folgenden Beispiel wird ein `DateTime`-Objekt mit einer bestimmten Datums-

³<https://www.php.net/manual/de/function.date-default-timezone-set.php>

⁴<https://www.php.net/manual/de/datetime.settimezone.php>

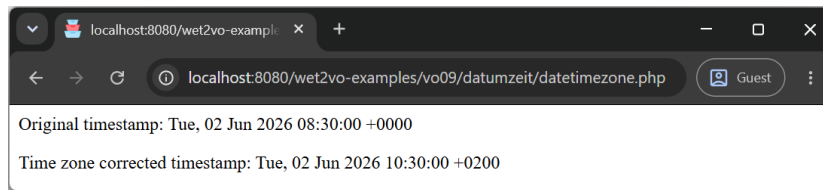


Abbildung 9.3: Ein Zeitstempel wird mit UTC angelegt und ausgegeben. Danach wird eine neue Zeitzone gesetzt und entsprechend dargestellt.

und Uhrzeitkombination in UTC angelegt. Dies könnte so etwa nach dem Einlesen eines (in einer Datei oder Datenbank) gespeicherten Eintrags der Fall sein. Vor der Ausgabe mit `format()` wird eine Zeitzone gesetzt. Das Ergebnis ist in Abbildung 9.3 ersichtlich.

```
1 $timezone = new DateTimeZone("Europe/Vienna");
2 $d2 = new DateTime("2026-06-02T08:30:00+00:00");
3 echo "<p>Original timestamp: {$d2->format("r")}</p>";
4 $d2->setTimezone($timezone);
5 echo "<p>Time zone corrected timestamp: {$d2->format("r")}</p>";
```

9.1.3 Mit Datum und Zeit rechnen

Ein `DateTime`-Objekt repräsentiert einen bestimmten Zeitpunkt. Oft ist es jedoch nötig, ausgehend von diesem einen neuen Zeitstempel zu berechnen. Dazu müssen bestimmte Zeitintervalle addiert oder subtrahiert werden, etwa „jetzt in 15 Stunden“ oder „heute vor zwei Wochen“. Diese Intervalle werden mit Hilfe der `DateInterval`-Klasse definiert.

Intervalle definieren

Um einen Zeitintervall festzulegen, wird ein Objekt der Klasse instanziiert. Als Argument wird ein String übergeben, der das Intervall festlegt. Er beginnt immer mit dem Buchstaben P für „Period“, gefolgt von einem Integerwert, der eine Dauer angibt. Abschließend wird ein Bezeichner gesetzt, der angibt, ob es sich um Tage, Wochen, Monate oder Jahre handelt. Werden Zeiten angegeben, werden diese durch den Buchstaben T getrennt spezifiziert. Tabelle 9.2 gibt die möglichen Bezeichner wieder.

Im folgenden Code werden drei Intervalle erstellt:

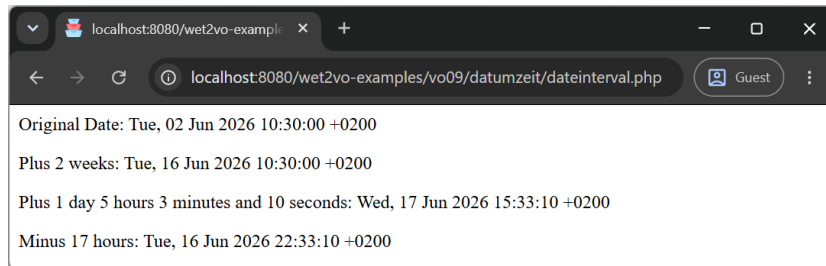
- zwei Wochen,
- ein Tag, fünf Stunden, drei Minuten und zehn Sekunden und
- 17 Stunden.

```
1 $twoWeeks = new DateInterval("P2W");
2 $oneDay5Hours3Mins10Secs = new DateInterval("P1DT5H3M10S");
3 $seventeenHours = new DateInterval("PT17H");
```

Zu beachten ist, dass bei der Instanziierung von `DateInterval`-Objekten eine Exception geworfen wird, falls der Intervallbezeichner nicht korrekt interpretiert werden kann. Es empfiehlt sich hier einen `try/catch`-Block zu verwenden, um dieser Fehlermöglichkeit zu begegnen.

Tabelle 9.2: Gültige Periodenbezeichner für `DateInterval`-Objekte.

Periodenbezeichner	Beschreibung
Y	Jahre
M	Monate
D	Tage
W	Wochen
H	Stunden
M	Minuten
S	Sekunden
F	Mikrosekunden

**Abbildung 9.4:** Zu einem Datum werden mehrere Intervalle addiert bzw. subtrahiert.

Intervalle addieren und subtrahieren

Mit Hilfe der Methoden `add($interval)`⁵ und `sub($interval)`⁶ können nun diese Intervalle zu einem `DateTime`-Objekt addiert bzw. davon subtrahiert werden. Das folgende Beispiel addiert zu einem Datum `$d` zuerst die ersten beiden Intervalle und subtrahiert dann den dritten. Abbildung 9.4 zeigt das Ergebnis.

```
1 echo "<p>Original Date: {$d->format("r")}</p>";
2 echo "<p>Plus 2 weeks: {$d->add($twoWeeks)->format("r")}</p>";
3 echo "<p>Plus 1 day 5 hours 3 minutes and 10 seconds:
    {$d->add($oneDay5Hours3Mins10Secs)->format("r")}</p>";
4 echo "<p>Minus 17 hours: {$d->sub($seventeenHours)->format("r")}</p>";
```

Zeitdifferenzen berechnen

Um die Differenz zwischen zwei `DateTime`-Objekten zu berechnen, steht die Methode `diff($datetime2)`⁷ zur Verfügung. Das Ergebnis ist ein `DateInterval`-Objekt.

⁵<https://www.php.net/manual/de/datetime.add.php>

⁶<https://www.php.net/manual/de/datetime.sub.php>

⁷<https://www.php.net/manual/de/datetime.diff.php>

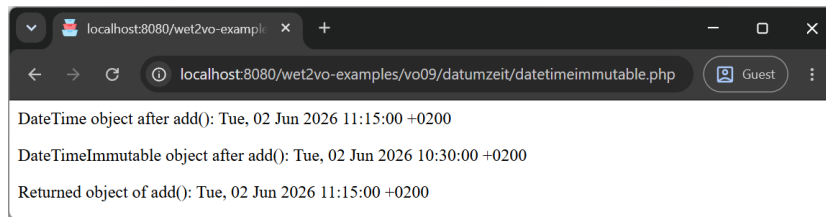


Abbildung 9.5: Ein `DateTimeImmutable`-Objekt wird nicht durch Intervall-Operationen verändert.

9.1.4 Unveränderbare Zeitstempel

Objekte der Klasse `DateTime` werden, wie in Abschnitt 9.1.3 demonstriert, verändert, wenn Zeitintervalle hinzugefügt werden. Dabei geht der ursprüngliche Zeitstempel verloren. Dies kann nicht erwünscht, oder sogar problematisch sein. Aus diesem Grund gibt es alternativ dazu die Klasse `DateTimeImmutable`. Sie bietet dieselben Methoden wie `DateTime`, jedoch sind Objekte der Klasse „immutable“, also unveränderbar. Wird eine Methode wie `add()` aufgerufen, wird also nicht das bestehende Objekt verändert, sondern ein neues Objekt erzeugt und zurückgegeben.

```
1 $d1 = new DateTime("2026-06-02T10:30:00+02:00");
2 $d2 = new DateTimeImmutable("2026-06-02T10:30:00+02:00");
3 $fortyFiveMinutes = new DateInterval("PT45M");
4
5 $d1->add($fortyFiveMinutes);
6 $d3 = $d2->add($fortyFiveMinutes);
7
8 echo "<p>DateTime object after add(): {$d1->format("r")}</p>";
9 echo "<p>DateTimeImmutable object after add(): {$d2->format("r")}</p>";
10 echo "<p>Returned object of add(): {$d3->format("r")}</p>";
```

In diesem Beispiel werden ein `DateTime`- und ein `DateTimeImmutable`-Objekt angelegt. Zu beiden werden 45 Minuten hinzugezählt und das Resultat wird ausgegeben. Wie in Abbildung 9.5 ersichtlich, ändert sich das `DateTime`-Objekt, das `DateTimeImmutable`-Objekt bleibt jedoch gleich. Um hier den neu entstandenen Zeitstempel (+45 Minuten) zu erhalten, muss der Rückgabewert von `add()` verwendet werden.

9.1.5 Erweiterte Funktionalitäten mit Carbon

Während die mit PHP mitgelieferten Klassen gute Basisfunktionalität für die Arbeit mit Datum und Zeit bieten, gibt es Fälle, in denen noch zusätzliche Funktionalität benötigt wird. Beispiele dafür könnten das Erstellen von Zeitstempeln aus verschiedensten Formaten, das einfachere Berechnen von Differenzen oder eine Lokalisierung von Monatsnamen und Wochentagen sein.

All diese Dinge und noch viel mehr bietet die PHP-Komponente `nesbot/carbon`⁸. Sie erweitert die `DateTime`-Klasse und bietet daher all ihre Funktionalitäten und vieles mehr. Carbon kann einfach über Composer installiert werden:

```
1 composer require nesbot/carbon
```

⁸<https://carbon.nesbot.com/>

Eine ausführliche Dokumentation aller Funktionen bietet die Carbon-Webseite.

9.2 Internationalisierung

Genauso wie beinahe jede PHP-Applikation Datum und Uhrzeit benötigt und dabei auf die unterschiedlichen Gegebenheiten wie Formate und Zeitzonen Rücksicht nehmen muss, so muss diese Anpassung an lokale Gegebenheiten meist noch eine Stufe höher gedacht werden. PHP-Applikationen laufen nicht nur in unterschiedlichen Zeitzonen, sondern werden von Benutzer*innen in unterschiedlichen Regionen, mit unterschiedlichen Sprachen und Schriften oder Kulturen verwendet. Dies muss bei der Erstellung der Applikation berücksichtigt werden, sollte diese global verwendet werden. Dieser Prozess wird *Internationalisierung* genannt.

9.2.1 Allgemeines zu Internationalisierung

Das W3C definiert *Internationalisierung* wie folgt [19]:

Internationalisierung entspricht dem Design und der Entwicklung eines Produkts, einer Anwendung oder eines Dokuments, das leichte Lokalisierung für Zielgruppen, die in Kultur, Region oder Sprache variieren, ermöglicht.

Internationalisierung, (engl. „internationalization“, auch mit „i18n“ – der Anzahl der Buchstaben zwischen „i“ und „n“ – abgekürzt) ermöglicht es also, in eine Applikation auf die Bedürfnisse von Nutzer*innen mit unterschiedlichen, so genannten *Locales* Rücksicht zu nehmen.

Locales

Eine Locale fasst die wichtigsten Internationalisierungs-Parameter für eine Region zusammen. Dazu gehören u. a.:

- die Sprache und Schrift (inklusive Leserichtung),
- das Land,
- das Datums- und Zeitformat,
- das Währungsformat.

Eine Locale wird in PHP (und anderen Sprachen) meist als IETF BCP 47 (RFC 5646) [35] Sprachtags angegeben. Diese setzen sich aus einer Reihe von Subtags zusammen, die durch einen Bindestrich getrennt sind. Begonnen wird mit dem verpflichtenden Subtag für die Primärsprache, danach können verschiedene Zusatzinformationen optional angegeben werden. Dazu gehören etwa Subtag für die Region oder ein Subtag für das verwendete Schriftsystem. Je genauer die Locale durch die Subtags beschrieben wird, desto spezifischer ist sie auf regionale Gegebenheiten angepasst. **de** bedeutet etwa Deutsch und umfasst alle deutschsprachigen Länder, während **de-AT** spezifisch österreichisches Deutsch kennzeichnet. Tabelle 9.3 gibt einen Überblick über einige solche Sprachtags.

Tabelle 9.3: Einige Locales in Form von IETF BCP 47 Sprachtags.

Sprachtag	Bedeutung
de	Deutsch
de-AT	Österreichisches Deutsch
de-DE	Standard-Deutsch (Deutschland)
en-US	US-Englisch
fr-CA	Kanadisches Französisch
fr-FR	Standard-Französisch (Frankreich)
zh-Hant-TW	Chinesisch, Han traditional, für Taiwan
sr-Cyrl	Serbisch, kyrillisch

Zum Arbeiten mit Locales stellt PHP die Klasse `Locale`⁹ zur Verfügung. Sie ermöglicht es, Sprachtags zu validieren und korrekt zurückzugeben.

Locales bilden also die Eigenschaften ab, die für die Internationalisierung einer Applikation wichtig sind. Der Prozess der Anpassung der Applikation an diese Eigenschaften wird nun Lokalisierung genannt.

Lokalisierung

Wie in der Definition am Beginn des Abschnitts angegeben, ermöglicht Internationalisierung die Lokalisierung eines Produktes oder eine Applikation. *Lokalisierung* (engl. „localization“ oder kurz „l10n“) ist also der tatsächliche Prozess der Angleichung einer Applikation an lokale Gegebenheiten. Also die Übersetzung in die Landessprache und das dabei verwendete Schriftsystem, die Angleichung an die verwendeten Datums-, Zeit- und Währungsformate oder auch der sorgfältige sprachliche Umgang mit kulturellen Eigenschaften. Lokalisierung kann also als die konkrete Anwendung von Internationalisierung gesehen werden.

Die folgenden Abschnitte zeigen Herangehensweisen in PHP, die dabei helfen sollen, die grundsätzlichen Abläufe von Lokalisierungen zu verstehen und sie einsetzen zu können. Dazu sollten aber die Herausforderungen, denen man als Entwickler*in dabei begegnet, bekannt sein.

Herausforderungen bei Internationalisierung und Lokalisierung

Bei der Internationalisierung und Lokalisierung von Anwendungen sollten sich Entwickler*innen der Tragweite dieses Prozesses bewusst sein. Es gibt weltweit um die 7.000 Sprachen in mehreren hundert Sprachfamilien. Allein der Prozess des Schreibens wurde auf der Welt vier Mal unabhängig voneinander auf verschiedene Arten und Weisen erfunden: von den Sumerern, den Ägyptern, den Chinesen und den Mayas. Dabei entstanden verschiedene Schriftsysteme. Ausgehend von den ägyptischen Hieroglyphen entwickelte

⁹<https://www.php.net/manual/de/class.locale.php>

sich etwa das lateinische Alphabet, kyrillisch, arabisch, oder die runische Schrift. Im asiatischen Raum erfolgte eine völlig andere Entwicklung, hier haben Schriftzeichen Bedeutungen. Diese (sehr grobe) Zusammenfassung alleine macht bereits deutlich, dass nicht einfach nur von „Sprache und Schrift“ gesprochen werden kann, sondern dass diese Dinge für Personen in unterschiedlichen Regionen der Erde auch eine völlig unterschiedliche Bedeutung und Herangehensweisen haben. Internationalisierung ist also nicht nur eine technische Herausforderung – es ist vor allem wichtig, sich vor Augen zu führen, dass es diese Unterschiede gibt und sie bei der Entwicklung von Applikationen zu bedenken.

Berücksichtigt man diese Vielfalt in Sprache und Schrift und versucht sie auf den Kontext der Internationalisierung von Softwareapplikationen herunterzubrechen, ergeben sich eine Reihe von Herausforderungen:

- *Geschlechtsspezifische Konjugationen* bei Verben und Adjektiven. Bei Sprachen wie Spanisch oder Italienisch werden etwa Adjektive an das Geschlecht des Hauptworts angepasst.
- *Pluralbildungen*. In deutsch und anderen romanischen Sprachen repräsentiert ein Gegenstand die Einzahl, während kein Gegenstand oder mehrere Gegenstände als Pluralform wiedergegeben werden (Ich habe eine Tomate. Ich habe keine Tomaten. Ich habe fünf Tomaten.). Andere Sprachen (z. B. slawische Sprachen) haben jedoch noch weitere Formen der Pluralbildung (z. B. unter 5 vs. über 5 etc.).
- *Datums-, Zeit- und Währungsformatierung*. Tag, Monat und Jahr werden etwa in unterschiedlicher Reihenfolge angegeben und haben in den jeweiligen Sprachen andere Namen. Darüber hinaus gibt es Länder, die eigene Kalendersysteme haben (z. B. der Iran). Auch Währungen werden in Ländern unterschiedlich dargestellt.
- *Lese- und Schreibrichtung*. Viele europäische Sprachen werden von links nach rechts und von oben nach unten (LTR, TTB – left to right, top to bottom) gelesen. Sprachen wie Hebräisch oder Arabisch hingegen werden von rechts nach links und von oben nach unten (RTL, TTB) gelesen. Im Chinesischen und Japanischen gibt es Texte, die zuerst von oben nach unten und dann von rechts nach links (TTB, RTL) gelesen werden.

Diese und noch viele Herausforderungen müssen mitbedacht werden. Die in den folgenden Abschnitten vorgestellten Lösungen werden einige dieser Themen dabei aufgreifen.

9.2.2 Internationalisierung und Lokalisierung mit PHP

Um nun Applikationen mit PHP zu internationalisieren, gibt es eine Reihe von technischen Möglichkeiten. Diese richten sich oft nach der Größe der Applikation und wie diese skalieren soll. Vorgestellt werden drei Ansätze:

- ein einfacher, arraybasierter Ansatz,
- die Bibliothek Gettext und
- die Symfony Translation Komponente.

Jeder dieser Ansätze hat gewisse Vor- und Nachteile, die in den Abschnitten erläutert werden. Es ist wichtig zu verstehen, dass es nicht *den* Ansatz gibt, der immer die beste Wahl ist. Vielmehr muss das Ergebnis den Erwartungen an die Applikation

genügen. Werden etwa nur wenige Zeichenketten übersetzt, ist die Auswahl eines komplexeren Systems wahrscheinlich nicht notwendig. Soll die Applikation aber dynamisch erweiterbar sein oder sollen etwa externe Übersetzer*innen eingebunden werden, muss auf Systeme zurückgegriffen werden, die einen strukturierten und aufteilbaren Workflow unterstützen.

9.2.3 Übersetzung mit PHP-Arrays

Die einfachste Möglichkeit, Applikationen zu lokalisieren, sind PHP-Arrays. Dabei ist zu beachten, dass diese vor allem für die Übersetzung von Zeichenketten (Strings) funktionieren. Pluralbildung kann gut abgebildet werden, Dinge wie Datumsformate müssen selbständig verwaltet werden. Die komplette Verwaltung der Locales sowie die Auswahl der richtigen Strings muss selbst erfolgen. Daher eignet sich dieses System nur für sehr kleine Applikationen, die nicht erweitert werden, sprich skalieren müssen.

Aufbau der Applikation

Bei der Arbeit mit PHP-Arrays als Datenquellen für Übersetzungen empfiehlt es sich, einen eigenen Unterordner (z.B. `locale`) für die Übersetzungen zu erstellen. Darin befinden sich PHP-Dateien, die nach Sprachtags benannt sind, z.B. `locale › en.php` oder `locale › de.php`. Auch Dateien wie `locale › de-AT.php` sind möglich, wenn eine genauere Unterscheidung notwendig ist.

Jede dieser Dateien wird ein assoziatives PHP-Array enthalten, das identisch aufgebaut ist und unter bestimmten Schlüsseln die anzuzeigenden Zeichenketten in der jeweiligen Sprache enthält. Per `include` oder `require` wird die Datei für die gewünschte Locale eingebunden und somit die jeweilige Sprache ausgegeben.

Zur Demonstration wird die Datei `index.php` angelegt, die ein Formular mit einem `<select>`-Element zur Sprachauswahl enthält. Darüber hinaus noch ein Eingabefeld zur Erfassung einer Zahl, die später zur Demonstration der Pluralbildung benötigt wird. Bei den `<label>`-Elementen ist ersichtlich, dass nicht ein statischer Text, sondern der Wert eines PHP-Arrays `$messages` ausgegeben wird. Dies betrifft bereits die Übersetzung, da ja auch die Beschriftungen in der richtigen Sprache angezeigt werden sollen.

```

1 <form method="get" action="<?=$_SERVER["SCRIPT_NAME"] ?>">
2   <label for="language"><?=$messages["selectLabel"] ?></label><br>
3   <select name="locale" id="language">
4     <option value="en"><?=$messages["optionLabel"] ?></option>
5     <option value="en">English</option>
6     <option value="de">Deutsch</option>
7   </select><br>
8   <label for="messages"><?=$messages["messageLabel"] ?></label><br>
9   <input type="number" id="messages" name="nrOfMessages"><br>
10  <button type="submit"><?=$messages["sendButton"] ?></button>
11 </form>

```

Das `<select>`-Element setzt also nun unter dem Wert `locale` die Sprache. In diesem Fall entweder `en` oder `de`. Wird das Formular nun abgeschickt, wird dieser Wert mit übergeben und kann beim Anzeigen der Seite für die Lokalisierung verwendet werden. Dazu befindet sich am Anfang der PHP-Datei die Auswahl der Sprache in diesem Code-Fragment:

```

1 <?php
2
3 $locale = "en";
4 if (isset($_GET["locale"])) {
5     $locale = $_GET["locale"];
6 }
7
8 require __DIR__ . "/locale/$locale.php";

```

Zunächst wird als Standard-Locale `en` festgelegt. Diese Einstellung bestimmt auch den initialen Aufruf. Wurde aber über das Formular im `<select>`-Element ausgewählt, so wird diese Locale verwendet. Im Anschluss wird aus dem Unterverzeichnis `locale` jene PHP-Datei eingebunden, deren Name mit der Locale übereinstimmt. In der Datei ist nun das Array `$messages` definiert.

Zunächst in `locale/en.php`:

```

1 $messages = [
2     "selectLabel" => "Please select the language.",
3     "optionLabel" => "Select language",
4     "messageLabel" => "Please select the number of messages",
5     "sendButton" => "Send",
6     "welcome" => "Welcome to the site!",
7     "messages" => [
8         "zero" => "You have no messages.",
9         "one" => "You have one message.",
10        "other" => "You have %d messages.",
11    ],
12 ];

```

Dann in `locale/de.php`:

```

1 $messages = [
2     "selectLabel" => "Bitte die Sprache wählen.",
3     "optionLabel" => "Sprache wählen",
4     "messageLabel" => "Bitte die Anzahl der Nachrichten wählen.",
5     "sendButton" => "Absenden",
6     "welcome" => "Willkommen auf der Seite!",
7     "messages" => [
8         "zero" => "Du hast keine Nachrichten.",
9         "one" => "Du hast eine Nachricht.",
10        "other" => "Du hast %d Nachrichten.",
11    ],
12 ];

```

Übersetzen von Zeichenketten

Die hier hinterlegten Strings können nun auf der Seite einfach ausgegeben werden. In `$messages["selectLabel"]` befindet sich etwa der Text für die Beschriftung des `<select>`-Elements, `$messages["welcome"]` zeigt eine Begrüßung an, die unter dem Formular ausgegeben wird.

```

1 echo "<p>{$messages["welcome"]}</p>";

```

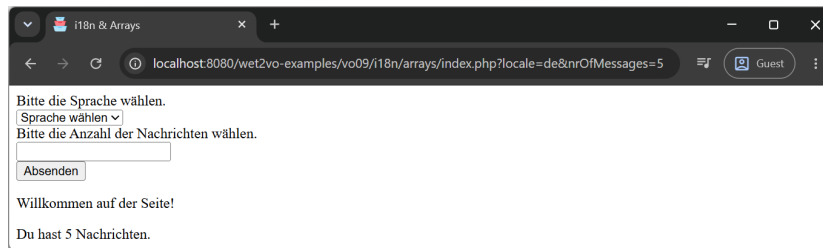


Abbildung 9.6: Im Dropdown-Menü wurde „Deutsch“ ausgewählt und die Zahl „5“ eingegeben. Das Ergebnis ist eine deutsche Seite mit einer korrekten Pluralbildung.

Pluralbildung

Das Formularfeld mit der ID `messages` erlaubt es, eine Zahl einzugeben. Basierend auf dieser soll angezeigt werden, wie viele Nachrichten ein*e Nutzer*in erhalten hat. Wird 0 eingegeben, soll (auf Deutsch) „Du hast keine Nachrichten.“ angezeigt werden, bei 1 erscheint „Du hast eine Nachricht.“ und bei jeder anderen Zahl soll „Du hast %d Nachrichten.“ ausgegeben werden, wobei %d hier für die eingegebene Zahl steht. Es benötigt also drei unterschiedliche Zeichenketten, die unter dem Schlüsselwort `messages` abgelegt sind. Unter `zero` liegt der Text für 0 Nachrichten, unter `one` der Text für eine Nachricht und unter `other` der Text für alle anderen Fälle.

Bei der Ausgabe wird nun mit einem `match`-Statement (moderne Alternative zu `switch`) basierend auf der eingegebenen Zahl entschieden, auf welchen String im Array zugegriffen wird:

```
1 if (isset($_GET["nrOfMessages"])) {
2     $nrOfMessages = $_GET["nrOfMessages"];
3
4     echo match ($nrOfMessages) {
5         "0" => $messages["messages"]["zero"],
6         "1" => $messages["messages"]["one"],
7         default => sprintf($messages["messages"]["other"], $nrOfMessages),
8     };
9 }
```

Wurde eine Zahl eingegeben wird anhand dieser entschieden. Bei der Zahl 0 wird `$messages["messages"]["zero"]` mit `echo` ausgegeben, bei der Eingabe der Zahl 1 `$messages["messages"]["one"]`. Bei jedem anderen Wert kommt der `default`-Zweig zum Zug. Hier kann nun nicht einfach mit `echo` der String ausgegeben werden, denn es muss ja die konkrete Zahl noch eingesetzt werden. Der gewählte Platzhalter `%d` kommt dabei von der PHP-Funktion `sprintf($format, $values)`¹⁰, die einen formatierten String zurückgibt. In das Muster, das in `$format` angegeben wird, können Platzhalter eingefügt werden. `%d` steht etwas für eine Ganzzahl, `%s` für einen String oder `%f` für eine Gleitkommazahl. `$values` sind die einzufügenden Werte die der Reihe nach in die Platzhalter eingefügt werden. Das Ergebnis ist der „befüllte“ String.

Das Ergebnis für die Applikation mit der Sprache Deutsch und 5 Nachrichten wird in Abbildung 9.6 dargestellt.

¹⁰<https://www.php.net/manual/de/function.sprintf.php>

Resümee und weitere Funktionalitäten

Sollen korrekte Datums-, Zeit oder Währungsformate ausgegeben werden, so muss selbst anhand von Logik entschieden werden, wie diese zu handhaben sind. Dazu müssen die jeweiligen Gegebenheiten einer Locale bekannt sein und dementsprechend umgesetzt werden. Auch gilt es zu beachten, dass die Daten in PHP-Datenstrukturen erfasst werden. Sollen externe Übersetzer*innen eingesetzt werden, so müssen diese ihre Übersetzungen in PHP-Dateien ablegen.

Somit eignet sich diese Vorgehensweise hauptsächlich dann, wenn wenige Zeichenketten übersetzt werden müssen, ansonsten aber keine weiteren Anforderungen an die Internationalisierung gestellt werden.

9.2.4 Übersetzung mit Gettext

GNU gettext – in PHP als *Gettext* verfügbar – ist eine Programmbibliothek zur Internationalisierung von Software. Sie ist ursprünglich in C verfasst und wurde in zahlreiche andere Programmiersprachen portiert, darunter auch PHP.

Grundsätzliche Funktionsweise

Der Kern der Bibliothek ist die Funktion `gettext($string)`¹¹, die als Parameter eine Zeichenkette erhält, die in der Originalsprache der Applikation angezeigt wird. Dies ist in der Regel Englisch bzw. sollte eine Sprache sein, die alle Übersetzer*innen beherrschen. Jeder String, der übersetzt werden soll, muss in dieser Funktion „verpackt“ werden. Um die Lesbarkeit zu erhöhen, steht auch der Alias `_( $string)` zur Verfügung.

Anstatt also in einer Applikation

```
1 echo "Hello World!";
```

auszugeben, wird

```
1 echo _("Hello World!");
```

verwendet.

```
1 echo gettext("Hello World!");
```

ist gleichbedeutend, aber wesentlich länger.

Vorbereitung der Umgebung

Da Gettext eine tief im Betriebssystem verankerte Bibliothek ist, arbeitet sie mit den dort installierten Locales. Um zu überprüfen, welche am Webserver installiert sind, kann `locale -a` in der Shell eingegeben werden. Dies ergibt z. B. die folgenden Locales, im `fhooe-web-dock` Docker Container sind jedoch wesentlich mehr Locales über das Debian Paket `locales-all` installiert:

`C.utf8`

`de_AT.utf8`

`de_DE.utf8`

¹¹<https://www.php.net/manual/de/function.gettext.php>

```
en_GB.utf8
en_US.utf8
POSIX
```

Um zu bestimmen, welche Locale der systemweite Standard ist, wird `locale` (ohne die Option `-a`) eingegeben. Im Docker Container ist `en_US.utf8` überall als Standard hinterlegt.

```
LANG=en_US.utf8
LANGUAGE=en_US.utf8
LC_CTYPE="en_US.utf8"
LC_NUMERIC="en_US.utf8"
LC_TIME="en_US.utf8"
LC_COLLATE="en_US.utf8"
LC_MONETARY="en_US.utf8"
LC_MESSAGES="en_US.utf8"
LC_PAPER="en_US.utf8"
LC_NAME="en_US.utf8"
LC_ADDRESS="en_US.utf8"
LC_TELEPHONE="en_US.utf8"
LC_MEASUREMENT="en_US.utf8"
LC_IDENTIFICATION="en_US.utf8"
LC_ALL=en_US.utf8
```

Vorbereitung der Applikation

Wie bereits in Abschnitt 9.2.3 gezeigt, soll eine kleine Applikation geschrieben werden, die mit einem `<select>`-Element die Sprache wechseln und einen Singular- und Plural-String anzeigen kann.

Die Datei `index.php` enthält also wieder ein Formular. Dieses Mal sind alle zu übersetzenden Zeichenketten mit der Gettext-Syntax gekennzeichnet. Die Werte der `<option>`-Elemente sind nun Locales, wie sie im Linux-Betriebssystem soeben installiert bzw. vorgefunden wurden.

```
1 <form method="get" action="<?=$_SERVER["SCRIPT_NAME"] ?>">
2   <label for="language"><?=$__("Please select the language.") ?></label><br>
3   <select name="locale" id="language">
4     <option value="en_US.utf8"><?=$__("Select language") ?></option>
5     <option value="en_US.utf8">English</option>
6     <option value="de_AT.utf8">Deutsch</option>
7   </select><br>
8   <label for="messages"><?=$__("Please select the number of messages.") ?></label><br>
9   <input type="number" id="messages" name="nrOfMessages"><br>
10  <button type="submit"><?=$__("Send") ?></button>
11 </form>
```

Nach dem Formular wird ein String zur Begrüßung ausgegeben:

```
1 echo "<p>" . _("Welcome to the site!") . "</p>";
```

Am Beginn der PHP-Datei findet nun wieder die Auswahl der Sprache statt. Zusätzlich werden die Einstellungen für Gettext vorgenommen:

```

1 $locale = "en_US.utf8";
2 if (isset($_GET["locale"])) {
3     $locale = $_GET["locale"];
4 }
5
6 putenv("LANG=$locale");
7 setlocale(LC_ALL, $locale);
8 $domain = "messages";
9 bindtextdomain($domain, __DIR__ . "/locale");
10 textdomain($domain);
11 bind_textdomain_codeset($domain, "UTF-8");

```

Als Locale wird `en_US.utf8` angenommen. Dies ist die im System verwendete Locale. Wiederum kann über das Formular etwas anderes gesetzt werden.

Danach finden einige Einstellungen statt. Zunächst wird in Zeile 6 eine Umgebungsvariable für die Sprache gesetzt, in Zeile 7 folgt dann die Locale für die eingebauten PHP Internationalisierungsfunktionen. `LC_ALL` bedeutet, dass eine Locale für alle Features (von Stringvergleichen über Währungsformatierung bis hin zu Zahlen- und Datumsformatierung) verwendet wird. In den Zeilen 8 und 9 wird die Domäne angegeben. Sie bestimmt später den Namen der Dateien, welche die übersetzten Strings beinhalten werden. Diese Domain wird dann einem Verzeichnis, in diesem Fall dem Unterordner `locale` zugewiesen. Dort müssen diese Dateien dann abgespeichert werden. Schließlich wird die Domäne in Zeile 10 aktiviert und schließlich in Zeile 11 die Zeichenkodierung für die Domäne definiert. Nun können die Dateien mit den Übersetzungen erstellt werden.

Erstellung der Übersetzungen

Gettext verwendet zur Übersetzung sogenannte *Portable Objects* bzw. PO-Dateien. Damit diese korrekt gefunden werden, muss eine korrekte Dateistruktur angelegt werden. Der oberste Ordner `locale` wurde in PHP bereits spezifiziert. Darunter muss für jede Sprache aber eine festgelegte Struktur erzeugt werden. Die Übersetzungsdateien für `de_AT` müssen im Verzeichnis `locale▶de_AT▶LC_MESSAGES` liegen. Der Dateiname muss mit der angegebenen Domain übereinstimmen, also `messages.po` lauten.

Nun können mit den Commandline-Tools `xgettext` und `msginit` zunächst ein PO-Template (`.pot`-Datei) und daraus eine PO-Datei erstellt werden. In diese PO-Datei werden dann die übersetzten Strings eingetragen. Schließlich wird die PO-Datei mittels `msgfmt` in ein Machine Object (MO-Datei) umgewandelt, die die übersetzten Texte in einem Binärformat enthält und die zur Laufzeit verwendet werden.

Um diesen Prozess zu Vereinfachen kann auch ein grafisches Werkzeug verwendet werden. Das bekanntest ist Poedit¹². Mit ihm wird zunächst eine leere POT-Datei angelegt, dann wird Poedit mitgeteilt, wo sich die Quelldateien befinden. Aus diesen liest das Werkzeug die zu übersetzenden Strings aus und erzeugt die PO-Datei. In der grafischen Oberfläche werden die Übersetzungen eingetragen und gespeichert. Dies erzeugt im Hintergrund die passende MO-Datei `messages.mo`. Abbildung 9.7 stellt die Oberfläche von Poedit dar.

¹²<https://poedit.net/>

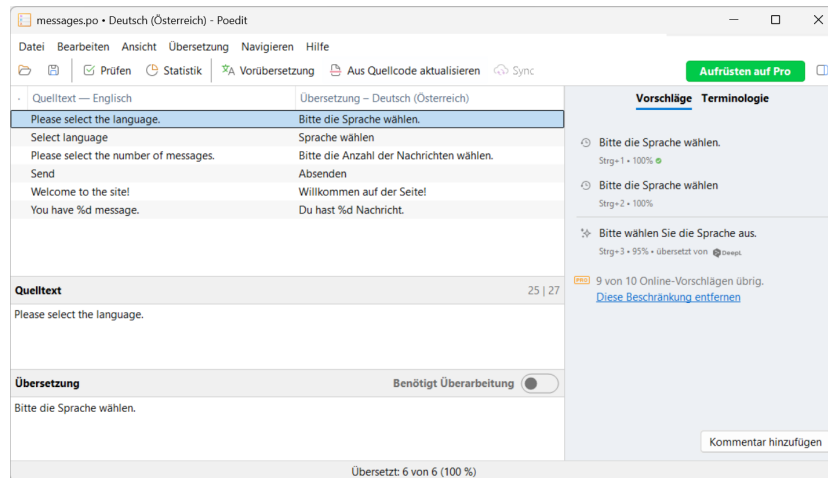


Abbildung 9.7: Poedit zeigt alle zu übersetzenden Strings zusammen mit der Übersetzung an und generiert alle notwendigen Dateien.

Die entstandene PO-Datei enthält den Originalstring zusammen mit seiner Übersetzung und einigen Metadaten.

```

1 #: index.php:24
2 msgid "Please select the language."
3 msgstr "Bitte die Sprache wählen."
4
5 #: index.php:26
6 msgid "Select language"
7 msgstr "Sprache wählen"
8
9 #: index.php:30
10 msgid "Please select the number of messages."
11 msgstr "Bitte die Anzahl der Nachrichten wählen."
12
13 #: index.php:32
14 msgid "Send"
15 msgstr "Absenden"
16
17 #: index.php:36
18 msgid "Welcome to the site!"
19 msgstr "Willkommen auf der Seite!"

```

Nachdem die MO-Dateien generiert wurden, kann die Applikation ausgeführt werden. Die Strings werden korrekt übersetzt.

MO-Dateien werden vom Betriebssystem gecacht, um maximale Performance zu bieten. Werden die PO-Dateien editiert und neue PO-Dateien generiert, muss der Apache Webserver neu gestartet werden (unter Debian im Docker Container etwa mittels `service apache2 restart`), damit die Änderungen aus dem Dateisystem übernommen werden. Ein Aufruf von `clearstatcache()`; am Ende des PHP-Skripts, kann während der Entwicklungsphase diesen Cache löschen und einen Restart vermeiden. Dies funktioniert jedoch nicht immer.

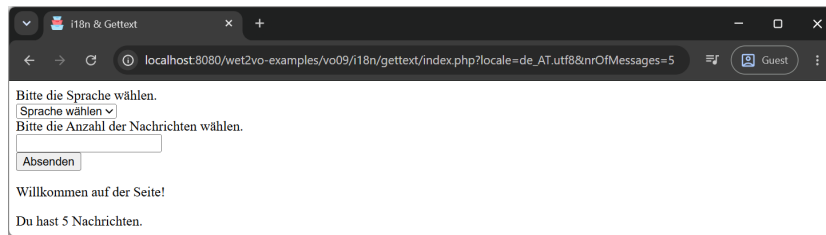


Abbildung 9.8: Im Dropdown-Menü wurde „Deutsch“ ausgewählt und die Zahl „5“ eingegeben. Das Ergebnis ist eine deutsche Seite mit einer korrekten Pluralbildung.

Pluralbildung

Für die Pluralbildung ist die Funktion `ngettext($singular, $plural, $count)`¹³ verfügbar. Das erste Argument ist die Einzahl, das zweite die Mehrzahl. Das dritte Argument dient Gettext festzustellen, welche Form gewählt werden muss.

Im Beispiel wird dies genutzt, um eine einfache Form der Benachrichtigung über neue Nachrichten zu erstellen.

```
1 if (isset($_GET["nrOfMessages"])) {
2     $nrOfMessages = intval($_GET["nrOfMessages"]);
3
4     echo sprintf(ngettext("You have %d message.", "You have %d messages.",
5         $nrOfMessages), $nrOfMessages);
6 }
```

Die Einzahl „You have %d message.“ mit der Mehrzahl „You have %d messages.“ zusammen mit der Anzahl der Nachrichten aus dem Formular an `ngettext()` übergeben. In der PO-Datei wird mit Poedit eine Übersetzung angelegt. Diese erzeugt folgenden Eintrag:

```
1 #: index.php:41
2 #, php-format
3 msgid "You have %d message."
4 msgid_plural "You have %d messages."
5 msgstr[0] "Du hast %d Nachricht."
6 msgstr[1] "Du hast %d Nachrichten."
```

Je nachdem, welche Anzahl in `$nrOfMessages` enthalten wird, wählt Gettext die richtige Form aus und gibt sie als String zurück. Ist deutsch aktiviert, etwa wie folgt:

- 0: Du hast %d Nachrichten.
- 1: Du hast %d Nachricht.
- 2: Du hast %d Nachrichten.

Über `sprintf()` wird dann %d durch die eigentliche Zahl ersetzt, sodass eine passende Nachricht ausgegeben wird. Für 5 Nachrichten sieht dies wie in Abbildung 9.8 dargestellt aus.

¹³<https://www.php.net/manual/de/function.ngettext.php>

Resümee und weitere Funktionalitäten

Gettext ist ein langjähriger Standard über Übersetzungen in Programmen. Das System der PO-Dateien ist weit verbreitet, es gibt viele Werkzeuge, auch auf kommerzieller Ebene, die damit arbeiten. Nachdem die Applikation mit den Gettext-Funktionen erstellt und die PO-Dateien generiert wurden, können diese Dateien an externe Übersetzer*innen weitergegeben werden, diese können völlig unabhängig von der Applikation die Zeichenketten übersetzen und die Dateien retournieren. Wissen über PHP oder den Aufbau der Applikation sind dabei nicht notwendig.

Gettext hat jedoch keinerlei Werkzeuge oder Funktionalitäten, um etwa mit Datum, Zeit oder Währungen zu arbeiten. Diese Dinge müssen selbst verwaltet werden. Darüber hinaus ist die Installation nicht immer einfach und überall möglich. Auch bedeutet die enge Integration ins Betriebssystem, dass etwas an Flexibilität verloren geht, vor allem, wenn die Applikation auf anderen Betriebssystemen laufen soll bzw. Betriebssysteme nicht für den Einsatz mit Gettext korrekt konfiguriert sind. Fehlende Locales müssen etwa nachinstalliert werden. Fehlen hierfür die Rechte, kann die Applikation mit diesen Sprachen nicht funktionieren.

Gettext hat seine Stärken jedoch vor allem bei Dingen wieder Pluralbildung, wo die Bibliothek selbst die Auswahl der korrekten Formen übernimmt.

Somit eignet sich Gettext immer dann, wenn die Übersetzung im Vordergrund steht. Durch die gute Toolunterstützung kann die Übersetzung dabei schnell und effizient erfolgen.

9.2.5 Übersetzung mit symfony/translation

Die dritte Möglichkeit zur Internationalisierung und Lokalisierung von Applikationen bieten Komponenten. Viele der großen Frameworks beinhalten Internationalisierungskomponenten, die auch einzeln verwendbar sind. Eine solche ist `symfony/translation` aus dem Symfony Framework.

Installation und Einbindung

Die Komponente wird zunächst über Composer installiert:

```
1 $ composer require symfony/translation
```

Bei der Einbindung wird, wie bereits von anderen Komponenten bekannt, der Composer-Autoloader am Beginn der Datei eingebunden und zunächst ein `Translator`-Objekt angelegt. Diesem wird die Locale übergeben, in die es übersetzen soll. In dem Beispiel soll die Locale wieder über ein `<select>`-Menü gesetzt werden können.

```
1 use Symfony\Component\Translation\Loader\JsonFileLoader;
2 use Symfony\Component\Translation\Translator;
3
4 require "vendor/autoload.php";
5
6 $locale = "en-US";
7 if (isset($_GET["locale"])) {
8     $locale = $_GET["locale"];
9 }
10
11 $translator = new Translator($locale);
```

Nachdem das `Translator`-Objekt erzeugt wurde, müssen die Ressourcen mit den übersetzten Zeichenketten hinzugefügt werden. `symfony/translation` unterstützt dabei eine Reihe von Loader-Klassen, mit denen diese Daten dem Translator übergeben werden können. Die Klasse kann JSON-, YAML-, XML-, PO-, MO- oder einfache PHP-Dateien laden. Zusätzlich gibt es einen Array-Loader und auch aus Datenbanken können Übersetzungen geladen werden. Für das folgende Beispiel kommt ein JSON-Loader zum Einsatz. Die Übersetzungen sind dabei in einem JSON-Objekt hinterlegt, das die Strings eindeutigen Schlüsseln zuordnet.

Zunächst wird ein Objekt der Klasse `JsonFileLoader` angelegt und dem Translator unter dem Namen `json` hinzugefügt. Danach werden zwei Ressourcen – die JSON-Dateien mit den Übersetzungen – angegeben. Beim Hinzufügen mit `addResource()` wird der Name des Loaders (`json`), der Pfad zur Datei (`translations/messages.de.json`), sowie die Locale der Datei (d.h. die Sprache der enthaltenen Strings) angegeben.

Wichtig: `symfony/translation` nimmt an, dass die Datei den Namen `messages`, gefolgt von einem Sprachtag und der Dateiendung trägt. `messages` ist hierbei die sogenannte Domäne, unter der die Übersetzungen verfügbar sind (ähnlich wie bei Gettext). Soll ein anderer Name verwendet werden, so muss die geänderte Domäne als vierter Parameter bei `addResource()` angegeben werden (`message` wird als Standard angenommen und muss nicht extra angeführt werden).

```
1 $translator->addLoader("json", new JsonFileLoader());
2 $translator->addResource("json", "translations/messages.de.json", "de-AT");
3 $translator->addResource("json", "translations/messages.en.json", "en-US");
```

Vorbereitung der Applikation und Übersetzungen

Die Applikation besteht wieder aus einem Formular, in dem in einem `<select>`-Element die Sprache und in einem Eingabefeld die Anzahl von Nachrichten eingegeben werden können. Darunter wird eine Willkommensnachricht ausgegeben.

```
1 <form method="get" action="<?=$_SERVER["SCRIPT_NAME"] ?>">
2   <label for="language"><?=$translator->trans("selectLabel") ?></label><br>
3   <select name="locale" id="language">
4     <option value="en-US"><?=$translator->trans("optionLabel") ?></option>
5     <option value="en-US">English</option>
6     <option value="de-AT">Deutsch</option>
7   </select><br>
8   <label for="messages"><?=$translator->trans("messageLabel") ?></label><br>
9   <input type="number" id="messages" name="nrOfMessages"><br>
10  <button type="submit"><?=$translator->trans("sendButton") ?></button>
11 </form>
12 <?php
13
14 echo "<p>{<?=$translator->trans("welcome")}</p>";
```

Jede Zeichenkette, die übersetzt werden soll, wird in einen `$translator->trans()` Aufruf verpackt. Übergeben wird ein Schlüsselwort, unter dem die Übersetzungen in den verschiedenen JSON-Dateien hinterlegt sind. `selectLabel` beinhaltet den Label-Text für das `<select>`-Element, unter `welcome` ist der Begrüßungstext hinterlegt. Die JSON-Dateien dazu sehen wie folgt aus:

`translations/messages.en.json`:

```

1 {
2   "selectLabel": "Please select the language.",
3   "optionLabel": "Select language",
4   "messageLabel": "Please select the number of messages",
5   "sendButton": "Send",
6   "welcome": "Welcome to the site!",
7   "messages": {
8     "zero": "You have no messages.",
9     "one": "You have one message.",
10    "other": "You have %d messages."
11  }
12 }

```

translations/messages.de.json:

```

1 {
2   "selectLabel": "Bitte die Sprache wählen.",
3   "optionLabel": "Sprache wählen",
4   "messageLabel": "Bitte die Anzahl der Nachrichten wählen.",
5   "sendButton": "Absenden",
6   "welcome": "Willkommen auf der Seite!",
7   "messages": {
8     "zero": "Du hast keine Nachrichten.",
9     "one": "Du hast eine Nachricht.",
10    "other": "Du hast %d Nachrichten."
11  }
12 }

```

Bei der Ausführung wählt das **Translator**-Objekt nun die Strings aus der jeweiligen Locale und zeigt diese an.

Pluralbildung

Für die Demonstration der Pluralbildung wurde der Eintrag **messages** in den JSON-Dateien angelegt. Er enthält ein Unterobjekt mit den drei Schlüsseln **zero**, **one** und **other** in denen die entsprechenden Formulierungen für keine Nachrichten, eine Nachricht und mehr als eine Nachricht hinterlegt sind.

Die Logik dazu wird in der PHP-Datei anhand des Wertes **nrOfMessages** aus dem Formular gesteuert. Je nach Wert wird einer der drei Werte in einem **match**-Statement ausgegeben. Bei der Position **other** wird zusätzlich der Parameter **%d** mit dem Wert aus **\$nrOfMessages** ersetzt. Dies ist über die Angabe eines zweiten Parameters bei **trans()** möglich.

```

1 if (isset($_GET["nrOfMessages"])) {
2   $nrOfMessages = $_GET["nrOfMessages"];
3
4   echo match ($nrOfMessages) {
5     "0" => $translator->trans("messages.zero"),
6     "1" => $translator->trans("messages.one"),
7     default => $translator->trans("messages.other", ["%d" => $nrOfMessages])
8   };
9 }

```

Abbildung 9.9 zeigt die Anwendung, nachdem Deutsch ausgegeben und die Zahl 5 ausgewählt wurde.

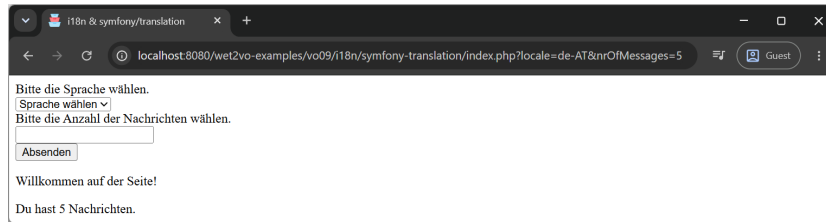


Abbildung 9.9: Im Dropdown-Menü wurde „Deutsch“ ausgewählt und die Zahl „5“ eingegeben. Das Ergebnis ist eine deutsche Seite mit einer korrekten Pluralbildung.

Übersetzung in Latte Templates

Die `symfony/translation`-Komponente lässt sich nahtlos in die Latte Template-Engine integrieren. Latte bietet hierfür native Übersetzungs-Tags in der Form `{_ 'key' }`. Um diese Funktionalität zu aktivieren, muss die `TranslatorExtension` von Latte registriert und mit dem Symfony-Translator verknüpft werden.

Zunächst wird in `index.php` das `Translator`-Objekt mit einem Loader und den Übersetzungs-Ressourcen wie zuvor gezeigt angelegt. Danach wird die Latte-Engine wie etwa aus Vorlesung 6 bekannt, erzeugt. Schließlich wird mittels `addExtension()` eine Instanz der `TranslatorExtension` hinzugefügt. Sie erwartet als Parameter ein `Callable`, also etwas, das sie aufrufen kann. Hier könnte etwa eine anonyme Funktion (eine sogenannte Closure) hinterlegt werden – die bereits bekannte First-Class-Callable-Syntax ist hier jedoch wesentlich eleganter und kürzer. Sie reicht die Parameter aus dem Template direkt an die `symfony/translation`-Komponente weiter.

```

1 $locale = "en-US";
2 if (isset($_GET["locale"])) {
3     $locale = $_GET["locale"];
4 }
5
6 $translator = new Translator($locale);
7 $translator->addLoader("json", new JsonFileLoader());
8 $translator->addResource("json", "translations/messages.de.json", "de-AT");
9 $translator->addResource("json", "translations/messages.en.json", "en-US");
10
11 $latte = new Engine();
12 $latte->setLoader(new FileLoader(__DIR__ . "/templates"));
13 $latte->setCacheDirectory(__DIR__ . "/cache");
14
15 $latte->addExtension(new TranslatorExtension($translator->trans(...)));

```

Nun kann eine Template-Datei `index.latte` angelegt werden. Diese enthält wieder ein Formular zur Sprachauswahl und zur Eingabe einer Zahl, gibt eine Willkommensnachricht und die Anzahl der Nachrichten (zur Demonstration der Pluralbildung) aus:

```

1 <!DOCTYPE html>
2 <html lang="{ $locale }">
3   <head>
4     <meta charset="UTF-8">
5     <title>i18n & symfony/translation + Latte Templates</title>
6   </head>
7   <body>

```

```

8     <form method="get" action="{$_scriptName}">
9         <label for="language">{$_selectLabel}</label><br>
10        <select name="locale" id="language">
11            <option value="en-US">{$_optionLabel}</option>
12            <option value="en-US">English</option>
13            <option value="de-AT">Deutsch</option>
14        </select><br>
15        <label for="messages">{$_messageLabel}</label><br>
16        <input type="number" id="messages" name="nrOfMessages"><br>
17        <button type="submit">{$_sendButton}</button>
18    </form>
19
20    <p>{$_welcome}</p>
21
22    {if $nrOfMessages != null}
23        <p n:if="$nrOfMessages == 0">{$_messages.zero}</p>
24        <p n:elseif="$nrOfMessages == 1">{$_messages.one}</p>
25        <p n:else>{$_messages.other, ['%d' => $nrOfMessages]}</p>
26    {/if}
27 </body>
28 </html>

```

Alle Zeichenketten, die übersetzt werden sollen, werden mit dem Schlüssel unter dem sie in den Übersetzungs-Ressourcen zu finden sind, innerhalb des `{_}`-Tags angegeben. Dies weist die `Translator`-Klasse an, den jeweiligen String in der aktiven Locale an dieser Stelle einzusetzen. Um bei der Pluralbildung Parameter mitzugeben, wird dem Tag einfach ein assoziatives Array übergeben (hier `['%d' => $nrOfMessages]`).

In der Hauptdatei muss nun nur noch das Template gerendert werden:

```

1 $latte->render("index.latte", [
2     "locale" => $locale,
3     "scriptName" => $_SERVER["SCRIPT_NAME"],
4     "nrOfMessages" => isset($_GET["nrOfMessages"]) ? intval($_GET["nrOfMessages"]) :
        null,
5 ]);

```

Die einzigen Platzhalter, die übergeben werden müssen sind die, die nicht übersetzt werden. Dies sind der Wert des `action`-Attributs des Formulars sowie der eingegebene Wert aus dem Formularfeld. Ist dieser nicht vorhanden (beim ersten Aufruf) wird `null` übergeben. Zudem wird der Locale-String übergeben, um das `lang`-Attribut beim `<html>`-Element korrekt setzen zu können.

Übersetzung mit dem ICU MessageFormat

Alle bisher gezeigten Übersetzungen arbeiteten mit weitgehend statischen Zeichenketten. Maximal wurde an einer Stelle (den Nachrichten) ein Wert eingesetzt. Oft jedoch sind Strings komplexer und ergeben sich aus mehreren Bedingungen, die verschiedene Werte wie Namen, Zahlen oder ein Datum enthalten. Da die Wortfolge hier in unterschiedlichen Sprachen meist völlig unterschiedlich ist, bleibt meist nur die Aufteilung in mehrere Substrings, die einzeln übersetzt und wieder zusammengefügt werden müssen. Dies verkompliziert jedoch Übersetzungen, da die Zusammenhänge oft nicht mehr gegeben sind. Daher muss versucht werden, einen gesamten String zu erzeugen, der je nach Sprache anders zusammengesetzt ist und diese variablen Werte enthält.

Tabelle 9.4: Beispiele für ICU MessageFormat Muster. Für die Beispielausgabe wird die Locale `de-AT` angenommen.

Muster	Werte	Resultat
Hallo {name}!	<i>name</i> : Wolfgang	Hallo Wolfgang!
Das kostet {value,number,currency}.	<i>value</i> : 3	Das kostet € 3,00.
Heute ist der {datum,date,short}	<i>datum</i> : <code>new DateTime()</code>	Heute ist der 02.06.26
Es ist jetzt {uhrzeit,time}	<i>uhrzeit</i> : <code>new DateTime()</code>	Es ist jetzt 21:02:57

Dies ist mit Hilfe des ICU MessageFormats Möglich. ICU steht für „International Components for Unicode“ und stellt ein Set an C/C++ und Java-Bibliotheken dar, die bei der Internationalisierung von Applikationen verwendet werden können. In PHP steht MessageFormat über die Klasse `MessageFormatter`¹⁴ zur Verfügung und kann verwendet werden, um sprachneutrale Strings zu erzeugen.

Ein solcher String besteht aus einem Muster, ähnlich wie bei PDO Prepared Statements. Variable Inhalte wie Zahlen, Daten, wechselnde adjektive oder Anreden werden über Platzhalter (Argumente) repräsentiert und in geschwungen Klammern angegeben. Steht nur der Platzhalter, wird dieser unformatiert ausgegeben. Zusätzlich können mit Beistrich getrennt noch Formatierungsangaben gemacht werden. Zur Verfügung stehen `number`, `date` und `time` und Zusatzargumente zur detaillierten Formatierung¹⁵. Bei der Ausgabe werden die Platzhalter dann befüllt und der fertige String wird ausgegeben. Tabelle 9.4 zeigt einige Beispiele.

Neben diesen simplen Argumenten gibt es auch komplexe. Erwähnenswert sind `plural` und `select`.

Plural ermöglicht es, Strings basierend auf einem numerischen Wert zusammen mit Pluralregeln für eine Sprache zu selektieren. Über die Kategorien `zero`, `one`, `two`, `few`, `many` und `other` können – je nach Sprache – die Pluralregeln abgebildet werden. Für Deutsch werden in der Regel nur `zero`, `one` und `other` benötigt:

```
1 {nr,plural,
2   zero {Niemand kam.}
3   one {Jemand kam.}
4   other {Viele kamen.}
5 }
```

Wird nun für `nr` etwa 5 übergeben, wird „Viele kamen.“ ausgewählt. Bei 1 „Jemand kam“.

Ähnlich verhält sich `select`. Hier erfolgt die Auswahl aufgrund von selbst festlegbaren Schlüsselwörtern – lediglich die Fallback-Option `other` muss existieren. `select` ist für die Auswahl des Geschlechts hilfreich:

```
1 {gender,select,
2   male {Er}
3   female {Sie}
4   other {They}
```

¹⁴<https://www.php.net/manual/de/class.messageformatter.php>

¹⁵Siehe dazu <https://unicode-org.github.io/icu-docs/apidoc/released/icu4j/com/ibm/icu/text/MessageFormat.html> für eine vollständige Auflistung der Parameter.


```
5 }
```

Wird *male* übergeben, so wird „Er“ ausgegeben, bei *female* „Sie“ und bei jedem anderen Wert, z. B. *divers* oder *other*, wird „They“ ausgewählt.

Der Online ICU Message Editor¹⁶ erlaubt ein einfaches Experimentieren mit Mustern und den zu befüllenden Werten.

Die `symfony/translate`-Komponente unterstützt den Umgang mit dem ICU MessageFormat. Um dem `Translator`-Objekt mitzuteilen, dass die enthalten Zeichenketten in diesem Format vorliegen, müssen Dateinamen und Domäne mit dem Suffix `+intl-icu` ergänzt werden:

```
1 $translator = new Translator("de-AT");
2 $translator->addLoader("yaml", new YamlFileLoader());
3 $translator->addResource("yaml", "translations/messages+intl-icu.en.yaml", "en-US",
   "messages+intl-icu");
4 $translator->addResource("yaml", "translations/messages+intl-icu.de.yaml", "de-AT",
   "messages+intl-icu");
```

Da ICU MessageFormat Strings besser lesbar sind, wenn sie mit Zeilenumbrüchen formatiert werden, bietet sich hier YAML als Format für die Übersetzungs-Ressourcen an, denn JSON erlaubt in den Zeichenketten keine Zeilenumbrüche (bzw. nur durch Angabe von `\n`).

Um YAML verwenden zu können, muss die Komponente `symfony/yaml` mit Composer installiert werden:

```
1 $ composer require symfony/yaml
```

Zum Laden der Ressourcen kommt nun die Klasse `YamlFileLoader` zum Einsatz. Die Dateien enthalten Strings zu einer Party-Einladung, basierend auf der einladenden Person, dem Gast, der Anzahl der Gäste und dem Datum und sehen wie folgt aus:

`translations>messages+intl-icu.en.yaml`

```
1 ---
2 party: |-
3 {gender_of_host, select,
4   female {
5     {num_guests, plural, offset:1
6       =0 {{host}} does not give a party on {party_date, date}.}
7       =1 {{host}} invites {guest} to her party on {party_date, date}.}
8       =2 {{host}} invites {guest} and one other person to her party on {party_date,
9         date}.}
9     other {{host}} invites {guest} and # other people to her party on {party_date,
10      date}.}
10   }
11 }
12 male {
13   {num_guests, plural, offset:1
14     =0 {{host}} does not give a party on {party_date, date}.}
15     =1 {{host}} invites {guest} to his party on {party_date, date}.}
16     =2 {{host}} invites {guest} and one other person to his party on {party_date,
17       date}.}
17   other {{host}} invites {guest} and # other people to his party on {party_date,
18     date}.}
18 }
```

¹⁶<https://format-message.github.io/icu-message-format-for-translators/editor.html>

```

19   }
20   other {
21     {num_guests, plural, offset:1
22       =0 {{host}} does not give a party on {party_date, date}.}
23       =1 {{host}} invites {guest} to their party on {party_date, date}.}
24       =2 {{host}} invites {guest} and one other person to their party on {party_date,
25         date}.}
26     other {{host}} invites {guest} and # other people to their party on {party_date
27       , date}.}
28   }

```

translations>messages+intl-icu.de.yaml

```

1 ---
2 party: |-
3 {gender_of_host, select,
4   female {
5     {num_guests, plural, offset:1
6       =0 {{host}} veranstaltet keine Party am {party_date, date}.}
7       =1 {{host}} lädt {guest} zur ihrer Party am {party_date, date} ein.}
8       =2 {{host}} lädt {guest} und eine weitere Person zu ihrer Party am {party_date,
9         date} ein.}
9     other {{host}} lädt {guest} und # andere Personen zu ihrer Party am {party_date
10       , date} ein.}
10   }
11 }
12 male {
13   {num_guests, plural, offset:1
14     =0 {{host}} veranstaltet keine Party am {party_date, date}.}
15     =1 {{host}} lädt {guest} zur seiner Party am {party_date, date} ein.}
16     =2 {{host}} lädt {guest} und eine weitere Person zu seiner Party am {party_date
17       , date} ein.}
17   other {{host}} lädt {guest} und # andere Personen zu seiner Party am {
18     party_date, date} ein.}
18 }
19 }
20 other {
21   {num_guests, plural, offset:1
22     =0 {{host}} veranstaltet keine Party am {party_date, date}.}
23     =1 {{host}} lädt {guest} zur their Party am {party_date, date} ein.}
24     =2 {{host}} lädt {guest} und eine weitere Person zu their Party am {party_date,
25       date} ein.}
25   other {{host}} lädt {guest} und # andere Personen zu their Party am {party_date
26     , date} ein.}
26 }
27 }
28 }

```

Zunächst wird auf der obersten Ebene mit einem `select`-Argument je nach Geschlecht des*der Gastgebers*Gastgeberin (`host`) entschieden. Danach folgt die jeweilige Pluralbildung je nach Anzahl der Gäste. Bei 0 wird keine Party veranstaltet, bei 1 eine Party mit der in `guest` genannten Person und bei 2 und `other` eine Party mit der genannten Person und weiteren. `party_date` gibt das Datum der Party im jeweils lokalisierten Format aus.

Nun wird der Translator aufgerufen und die Argumente werden mit übergeben:



Abbildung 9.10: Durch die übergebenen Parameter entsteht ein einzelner String, der grammatikalisch korrekt und nach den Regeln der Locale (hier **de-AT**) formatiert ist.

```
1 $translator->trans("party", [  
2     "host" => "Wolfgang",  
3     "gender_of_host" => "male",  
4     "guest" => "John Doe",  
5     "num_guests" => 3,  
6     "party_date" => new DateTime(),  
7 ])
```

Dieser Aufruf erzeugt die Ausgabe in Abbildung 9.10.

Resümee und weitere Funktionalitäten

Die `symfony/translation`-Komponente ist sicherlich die kompletteste Lösung zur Internationalisierung. Sie ermöglicht einfaches Übersetzen von Zeichenketten genauso wie die modulare Verwendung des ICU MessageFormats. Sie deckt damit auch Datum, Uhrzeit, Währung und viele andere Dinge ab. Auch das Zusammenspiel mit Twig Templates ist ein Argument für die Verwendung.

Die Komponente kann noch einige Dinge mehr, die unter <https://symfony.com/doc/current/translation.html> nachgelesen werden können.

9.3 Weiterführende Literatur

Die Arbeit mit Datum und Zeit gehört zum Standardrepertoire von PHP und ist daher in den meisten modernen Büchern gut abgedeckt. Erwähnt seien dabei wie immer [44, 49].

Literatur zur Internationalisierung und Lokalisierung ist jedoch kaum vorhanden. Alle großen und bekannten PHP-Bücher sparen dieses Thema gekonnt aus. Somit kann nur auf [42] verwiesen werden. Die Dokumentation enthält ein eigenes Kapitel zu allen Internationalisierungsfunktionen und zeigt Beispiele, wie sie eingesetzt werden können. Tutorials und in den Weiten des WWW können dies noch ergänzen.

Vorlesung 10

Medienverarbeitung

Medienverarbeitung bezeichnet den Prozess des Erstellens und Einlesens von Medienformaten abseits von HTML. Im Konkreten werden hier mehrere Dinge angesprochen: das Lesen und Schreiben von XML und JSON, die Erstellung von PDF und das Laden und Konvertieren von Grafiken.

10.1 XML-Verarbeitung

XML steht für *Extensible Markup Language*, eine vom W3C standardisierte Markup-Sprache. Eine detaillierte Ausführung zu den Grundlagen von XML findet sich in den Unterlagen zu Vorlesung 7 aus dem Wintersemester.

Um XML maschinell zu verarbeiten, gibt es eine Reihe von Werkzeugen, die eingesetzt werden können: Parser, Transformatoren und Renderer. Für diese Lehrveranstaltung relevant sind die XML-Parser.

10.1.1 Der Parser

Die Aufgabe eines *XML-Parsers* ist es, ein XML-Dokument einzulesen, und die darin enthaltenen Informationen programmiertechnisch aufzubereiten. Der Parser muss dafür die Struktur des Dokuments, das er abarbeiten soll, genau kennen. D. h., es gibt nicht *den* XML-Parser, sondern es gibt einen spezifischen Parser für jede XML-Anwendung. Wird eine eigene XML-Anwendung definiert, so muss meist auch ein eigener Parser dazu geschrieben werden, um Daten auslesen zu können. Dieser Parser weiß somit genau, welche Elemente und Attribute im XML-Dokument zu erwarten sind. Die Unbekannten sind allerdings die Daten dahinter, also die Werte der einzelnen Elemente und Attribute. Diese werden mit Hilfe des Parsers ausgelesen.

Die Arbeit mit einem Parser lässt sich in drei große Schritte unterteilen:

1. **Initialisieren des Parsers:** Hierbei wird der Parser im Programm angelegt und alle notwendigen Einstellungen werden vorgenommen.
2. **Aufruf des Parsers:** Dem Parser wird das XML-Dokument, das ausgelesen werden soll, übergeben und der Parser beginnt, es abzuarbeiten.
3. **Verarbeiten der Ergebnisse:** Die vom Parser gelieferten Daten werden verarbeitet.

Ein XML-Parser kann bei der Abarbeitung des Dokuments validieren, d. h. die XML-Grammatik überprüfen und ggf. auch korrigieren (implementierungsabhängig). Tut er dies, so ist er in der Lage, auftretende Fehler zu benennen und anzuzeigen.

Arten von XML-Parsern

Um ein XML-Dokument zu verarbeiten, gibt es in sehr vielen Programmiersprachen (z. B. C++, Java, PHP etc.) XML-Bibliotheken, die XML-Parser beinhalten. Diese lassen sich in drei große Kategorien einteilen:

Eventbasierte Parser: Eventbasierte Parser, auch Push- oder SAX-Parser (Simple API for XML) genannt, repräsentieren das XML-Dokument als sequentiellen Datenstrom. Das XML-Dokument wird Schritt für Schritt durchlaufen und an wichtigen Stellen (z. B. Beginn eines Tags, Beginn von Inhalt, Ende eines Tags etc.) wird automatisch eine von dem*der Entwickler*in gewählte Funktion (genannt Callback-Funktion) aufgerufen, die die Verarbeitung übernimmt. Das Antreffen einer solchen wichtigen Position im Dokument wird als Event bezeichnet, daher auch der Name. Eventbasierte Parser sind sehr speichereffizient, sind jedoch aufwändiger für Entwickler*innen, da diese nur die Informationen bekommen, wo der Parser gerade arbeitet (z. B. am Beginn eines Elements). Entwickler*innen müssen sich selber im Code darum kümmern, alle weiteren Informationen (wie Name des Tags etc.) zu erhalten und sich ebenfalls merken, welche Teile vom Dokument bereits verarbeitet wurden.

Eventbasierte Parser sind gut geeignet für einfache Operationen, wenn etwa nur bestimmte Elemente eines Dokuments ausgelesen werden sollen. Da Schritt für Schritt gelesen wird, sind sie sehr gut für große XML-Dokumente zu gebrauchen. Die Struktur eines XML-Dokuments kann damit jedoch nicht verändert und neue Dokumente können nicht erzeugt werden, da der Parser nur zum Lesen geeignet ist. Weiters können Inhalte nicht validiert werden.

PHP bietet mit dem Modul *XML Parser*¹ einen eventbasierten (SAX) Parser an. Dieser ist der am längsten verfügbare, er existiert bereits seit PHP 4 und ist von der API am umständlichsten zu verwenden. Er wird daher nicht näher erwähnt.

Pull-Parser: Ein Pull-Parser behandelt ein XML-Dokument als eine Serie von aufeinanderfolgenden Elementen und verwendet einen so genannten Iterator, um diese der Reihe nach abzuarbeiten. Dabei wird das Durchlaufen der verschiedenen Elemente von dem*der Entwickler*in bestimmt. Während bei SAX-Parsing die Informationen vom Parser permanent beim Auftreten eines Events geschickt werden (ob benötigt oder nicht), entscheiden Entwickler*innen beim Pull-Parsing, ob sie an einer bestimmten Stelle im Dokument Daten verarbeiten möchten. D. h. diese werden nur angefordert, wenn es nötig ist, wovon sich auch der Name Pull-Parser ableitet.

In PHP steht mit dem Modul *XMLReader*² ein voll implementierter Pull-Parser zur Verfügung.

¹<https://www.php.net/manual/de/book.xml.php>

²<https://www.php.net/manual/de/book.xmlreader.php>

Objektorientierte Parser: Ein objektorientierter Parser stellt das XML-Dokument als Objektstruktur dar und bietet wahlfreien Zugriff auf die einzelnen Elemente – in diesem Zusammenhang meist Knoten genannt. Ein Knoten ist ein Element mit Inhalt und ggf. Attributen. Objektorientierte Parser sind sehr praktisch, weil auf Elemente sehr einfach zugegriffen werden kann, bei großen Dateien benötigen sie jedoch viel Speicher, da sie ja die komplette Struktur auf einmal laden müssen.

In PHP stehen standardmäßig zwei Parser zur Verfügung, die nach diesem Modell arbeiten: *DOM*³ und *SimpleXML*⁴.

10.1.2 XML einlesen in PHP

Im Folgenden werden zwei der drei in PHP verfügbaren Parser-Arten – der Pull-Parser und ein objektorientierter Ansatz – anhand eines Beispiels besprochen. Als Dokument dient das Rezept für Tirolerknödel, welches bereits im Wintersemester vorgestellt wurde. Dieses Rezept soll geparsed und sein Inhalt lesbar ausgegeben werden.

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <rezept quelle="https://www.ichkoche.at/der-klassische-tiroler-knoedel-rezept-4386">
3   <gericht>Der klassische Tiroler Knödel</gericht>
4   <zutaten>
5     <zutat>
6       <ingredienz>Semmelwürfel</ingredienz>
7       <menge>350</menge>
8       <einheit>g</einheit>
9     </zutat>
10    <zutat>
11      <ingredienz>Milch</ingredienz>
12      <menge>200</menge>
13      <einheit>ml</einheit>
14    </zutat>
15    <zutat>
16      <ingredienz>Eier</ingredienz>
17      <menge>4</menge>
18      <einheit>Stück</einheit>
19    </zutat>
20    <zutat>
21      <ingredienz>Speck</ingredienz>
22      <menge>100</menge>
23      <einheit>g</einheit>
24    </zutat>
25    <zutat>
26      <ingredienz>Bergsteigerwurst</ingredienz>
27      <menge>100</menge>
28      <einheit>g</einheit>
29    </zutat>
30    <zutat>
31      <ingredienz>Butter</ingredienz>
32      <menge>20</menge>
33      <einheit>g</einheit>
34    </zutat>
35    <zutat>
```

³<https://www.php.net/manual/de/book.dom.php>

⁴<https://www.php.net/manual/de/book.simplexml.php>

```

36     <ingredienz>Zwiebel</ingredienz>
37     <menge>1</menge>
38     <einheit>Stück</einheit>
39 </zutat>
40 <zutat>
41     <ingredienz>Petersilie</ingredienz>
42     <menge>10</menge>
43     <einheit>g</einheit>
44 </zutat>
45 <zutat>
46     <ingredienz>Salz</ingredienz>
47     <menge>1</menge>
48     <einheit>Prise</einheit>
49 </zutat>
50 <zutat>
51     <ingredienz>Muskatnuss</ingredienz>
52     <menge>1</menge>
53     <einheit>Prise</einheit>
54 </zutat>
55 <zutat>
56     <ingredienz>Mehl</ingredienz>
57     <menge>80</menge>
58     <einheit>g</einheit>
59 </zutat>
60 <zutat>
61     <ingredienz>Schnittlauch</ingredienz>
62     <menge>10</menge>
63     <einheit>g</einheit>
64 </zutat>
65 </zutaten>
66 <zubereitung>
67     <schritt>Semmelwürfel mit Milch und Eiern vermengen und 20 bis 30 Minuten ziehen
        lassen.</schritt>
68     <schritt>Kleinwürfelig geschnittenen Speck und fein gewiegte Bergsteigerwurst in
        zerlassener Butter gemeinsam mit Zwiebeln, Petersilie und Schnittlauch anbraten
        und mit Salz und Muskatnuss abschmecken. Mehl darüber streuen und aus allen
69     <schritt>Mit befeuchteten Händen daraus Knödel formen.</schritt>
70     <schritt>Einen davon als Probeknödel in Salzwasser knappe 15 Minuten kochen. Hat
        der Probeknödel eine zu weiche Konsistenz, muss man noch etwas Mehl unter die
        Knödelmasse kneten.</schritt>
71     <schritt>Erst wenn die Masse kompakt genug ist, kocht man die restlichen Knödel.
        </schritt>
72     <schritt>Tiroler Knödel mit Schnittlauch bestreut servieren.</schritt>
73 </zubereitung>
74 </rezept>

```

Pull-Parser: XMLReader

Pull-Parser arbeiten das Dokument sequentiell ein. Man spricht hier auch von cursorbasiertem Einlesen, da quasi ein Cursor durch das Dokument „geschoben“ wird). Wann immer der Parser auf einen der unterschiedlichen Teile des Dokuments (Tags, Inhalt, Attribute etc.) stößt, gibt es die Möglichkeit, diese zu verarbeiten. Dabei wird das Knoten-Konzept von den objektorientierten DOM-Parsern übernommen: jeder Eintrag im XML-Baum wird als Knoten angesehen.

In einer `while`-Schleife läuft der Parser über das Dokument, dabei wird der Cursor von Knoten zu Knoten weiterbewegt. Nun muss festgestellt werden, um welche Art von Knoten (Element, Text, schließendes Element) es sich handelt. Danach können die Eigenschaften des Knoten abgefragt und der Inhalt verarbeitet werden.

In PHP wird ein Pull-Parser durch die Erweiterung *XMLReader* zur Verfügung gestellt. Sie ist seit PHP 5.1 Teil der Standarddistribution und arbeitet vollkommen objektorientiert.

Im Folgenden wird das Knödelrezept mit dem Pull-Parser verarbeitet und ausgegeben. Es wird dabei auf die einzelnen Teile des Parse-Vorgangs genauer eingegangen. Der ganze Code wird in der Klasse `MyPullParser` verpackt.

Parser initialisieren: Zum Speichern der auszulesenden Werte werden zunächst Eigenschaften in `MyPullParser` mit *Asymmetric Visibility* (`private(set)`) angelegt. Dies erlaubt es, Eigenschaften von außen lesend, aber nur von innen schreibend zuzugreifen (Getter-Methoden entfallen). Für die Initialisierung des Parsers muss nur ein Objekt der Klasse `XMLReader` angelegt werden. Die Klasse `MyPullParser` sieht im Grundgerüst wie folgt aus:

```
1 namespace XMLExample;
2
3 use XMLReader;
4
5 class MyPullParser
6 {
7     private XMLReader $parser;
8
9     private(set) string $source = "";
10    private(set) string $dish = "";
11    private(set) array $ingredients = [];
12    private(set) array $steps = [];
13
14    public function __construct()
15    {
16        $this->parser = new XMLReader();
17    }
18
19    // ...
20 }
```

Parsen starten: Das Starten des Parse-Vorgangs beginnt mit dem Öffnen der einzulesenden Datei mit Hilfe der Methode `open($file)`⁵. Danach wird das Einlesen durch Aufruf der Methode `read()`⁶ am Parser-Objekt begonnen. Diese Methode muss in einer `while`-Schleife aufgerufen werden, denn sie liefert solange `true`, solange Inhalt zum Einlesen vorhanden ist, und bricht die Schleife dann am Ende des XML-Dokuments durch das Zurückgeben von `false` ab.

```
1 public function parse(string $file): void
2 {
3     $this->parser->open($file);
```

⁵<https://www.php.net/manual/de/xmlreader.open.php>

⁶<https://www.php.net/manual/de/xmlreader.read.php>


```
4
5 while ($this->parser->read()) {
6     // Verarbeitung...
7 }
8 }
```

Inhalte verarbeiten: Die Methode `read()` läuft nun sequentiell durch das XML-Dokument und stoppt bei jedem Knoten. Als Knoten wird – wie auch schon vom DOM bekannt – ein jeder Bestandteil des Dokuments angesehen. Knoten haben wiederum verschiedene Typen und werden durch Konstanten in der `XMLReader`-Klasse bezeichnet. Eine Auflistung findet sich im PHP Handbuch⁷, die drei wichtigsten lauten:

- `XMLReader::ELEMENT`: Öffnendes Element.
- `XMLReader::END_ELEMENT`: Schließendes Element.
- `XMLReader::TEXT`: Text zwischen Tags.

Um nun festzustellen, bei welcher Art von Knoten der Cursor stehen geblieben ist, kann die Eigenschaft `nodeType` (z. B. mittels einer `switch`-Anweisung) gegen diese Konstanten überprüft werden. Anschließend können im jeweiligen `case`-Teil die Operationen für öffnende Elemente, schließende Elemente und Text durchgeführt werden. Im aktuellen Fall genügt es jedoch, mittels `if`-Bedingung auf ein öffnendes Element abzufragen, da alle Operationen dort erledigt werden können.

Handelt es sich nun um ein öffnendes Element, erfolgt das Auslesen der Inhalte. Dazu wird in einer `switch`-Anweisung der Name des aktuellen Element-Knotens als Entscheidungskriterium herangezogen. Handelt es sich um das `<rezept>`-Element, wird das Attribut `quelle` abgefragt und in der Eigenschaft `$source` gespeichert. Beim Element `<gericht>` wird die Methode `readString()`, die den Textinhalt des Knotens zurückgibt, aufgerufen und das Ergebnis in die Eigenschaft `$dish` gespeichert. Die Elemente `<ingredienz>`, `<menge>`, `<einheit>` und `<schritt>` werden auf dieselbe Art und Weise in die Arrays `$ingredients` bzw. `$steps` ausgelesen. Somit ist die Verarbeitung abgeschlossen.

Anmerkung: Anstatt mit `readString()` beim öffnenden Element den Inhalt auszulesen, hätte auch auf einen `TEXT`-Knoten überprüft und dessen Wert zurückgegeben werden können. Hierbei muss aber wiederum sichergestellt werden, zu welchem Element dieser Text gehört (z. B. durch eine Variable `$currentElement`, die sich das öffnende Element merkt).

```
1 while ($this->parser->read()) {
2     if ($this->parser->nodeType === XMLReader::ELEMENT) {
3         switch ($this->parser->name) {
4             case "rezept":
5                 $this->source = $this->parser->getAttribute("quelle") ? "";
6                 break;
7             case "gericht":
8                 $this->dish = trim($this->parser->readString());
9                 break;
10            case "zutat":
11                $this->ingredients[] = [];
12                break;
```

⁷<https://www.php.net/manual/de/class.xmlreader.php>

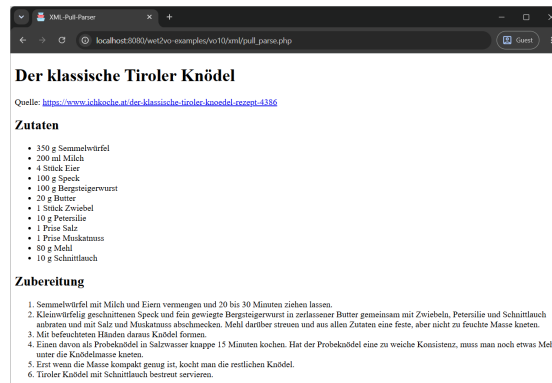


Abbildung 10.1: Im Browser wird der Inhalt des XML-Dokuments strukturiert angezeigt.

```

13     case "ingredienz":
14     case "menge":
15     case "einheit":
16         $lastIndex = array_key_last($this->ingredients);
17         $this->ingredients[$lastIndex][$this->parser->name] = trim($this->parser->
    readString());
18         break;
19     case "schritt":
20         $this->steps[] = trim($this->parser->readString());
21         break;
22     }
23 }
24 }
```

Ablauf starten: Um den Ablauf zu starten, muss ein Objekt der Klasse `MyPullParser` angelegt und darauf die Methode `parse()` mit der zu parsenden XML-Datei aufgerufen werden. Danach kann über die Eigenschaften, die öffentlich lesbar sind, das Ergebnis abgefragt und entsprechend ausgegeben werden. Das Ergebnis im Browser ist in Abbildung 10.1 ersichtlich.

```

1 $xmlParser = new MyPullParser();
2 $xmlParser->parse("rezept.xml");
```

Parser zerstören: Um den Parser zu zerstören, bietet sich der Destruktor an. Dazu nötig ist lediglich der Aufruf der Methode `close()`⁸, welche sich auch selbst darum kümmert, den Parser zu stoppen, sollte dieser gerade mit dem Verarbeiten eines Dokuments beschäftigt sein.

```

1 public function __destruct()
2 {
3     $this->parser->close();
4 }
```

⁸<https://www.php.net/manual/de/xmlreader.close.php>

Objektorientierter Parser: DOM

Neben dem soeben vorgestellten Pull-Parser (und dem hier nicht erwähnten eventbasierten Parser) existiert das dritte Modell der *objektorientierten Parser*. Diese Parser arbeiten mit dem Document Object Model (DOM) und repräsentieren die XML-Struktur als DOM-Baum. Da der Parser immer die gesamte Struktur des Dokuments kennt (da er den Baum vollständig aufbauen muss), ist es einfach, bestimmte Teile des Dokuments selektiv auszulesen. Dies bedeutet jedoch, dass das gesamte Dokument im Speicher gehalten werden muss, was bei großen XML-Dokumenten zum Problem werden kann.

PHP bietet seit Version 5 zwei verschiedene objektorientierte Parser an. Zunächst *SimpleXML*⁹, eine einfache Erweiterung, die XML-Objekte als eine Ansammlung von verschachtelten PHP-Objekten repräsentiert. Weiters auch *DOM*, das den W3C DOM Living Standard implementiert und somit einen Zugriff auf das Dokument mit den bekannten DOM-Methoden ermöglicht. Letzteres wird hier vorgestellt.

Der *DOM-Parser* wurde mit PHP 8.4 grundlegend überarbeitet und entspricht nun vollständig dem DOM Living Standard. Er bildet das gesamte XML-Dokument in einer Baumstruktur ab. Über die aus der JavaScript DOM API bekannten Methoden lassen sich leicht bestimmte Knoten (Elemente) finden.

Dies sei am Beispiel des Knödelrezepts demonstriert. In der Klasse *MyDOMParser* werden die folgenden DOM-Parser Funktionen abgebildet.

Parser initialisieren und Parsen starten: Beim DOM-Parser ist der Initialisierungsaufwand gering. Soll ein XML-Dokument eingelesen werden, wird ein Objekt der Klasse *Dom\XMLDocument*¹⁰ mit Hilfe des statischen Konstruktors *createFromFile()* angelegt. Darum ist es auch flexibler, dies gleich in der Methode *parse()* und nicht im Konstruktor zu erledigen. Dies startet den Parser, dieser läuft über das gesamte Dokument und bildet dessen Struktur als Baum im Objekt *\$dom* ab. Danach ist das DOM-Objekt befüllt und kann für Abfragen verwendet werden.

```

1 class MyDOMParser
2 {
3     private(set) string $source = "";
4     private(set) string $dish = "";
5     private(set) array $ingredients = [];
6     private(set) array $steps = [];
7
8     public function parse($file)
9     {
10         $this->source = "";
11         $this->dish = "";
12         $this->ingredients = [];
13         $this->steps = [];
14
15         $dom = XMLDocument::createFromFile($file);
16
17         // Verarbeitung
18     }
19 }
```

⁹<https://www.php.net/manual/de/book.simplexml.php>

¹⁰<https://www.php.net/manual/de/class.dom-xml-document.php>

Inhalte verarbeiten: Die Stärke des DOM-Ansatzes liegt darin, direkte Abfragen machen zu können. Dabei muss die Struktur des Dokuments bekannt sein. Analog zu JavaScript stehen die üblichen Funktionen und Eigenschaften zur Verfügung:

- `documentElement`¹¹: Enthält den Wurzelknoten.
- `querySelector($selectors)`¹²: Gibt das erste Element zurück, das dem übergebenen CSS-Selektor entspricht.
- `querySelectorAll($selectors)`¹³: Gibt eine `NodeList` mit allen Elementen zurück, die dem übergebenen CSS-Selektor entsprechen.
- `getElementById($id)`¹⁴: Gibt das Element mit der gegebenen ID zurück.
- ...

Wurden die gewünschten Elemente gefunden, gibt es wiederum bekannte Eigenschaften und Methoden, um Informationen daraus zu erhalten:

- `nodeName`: Der Name eines Knotens.
- `nodeType`: Der Typ eines Knotens (Element, Text etc.).
- `textContent`: Der Textinhalt eines Knotens.
- `firstChild`: Der erste Kindknoten.
- `hasChildNodes()`: Hat der Knoten weitere Kindknoten?
- ...

Diese Methoden und Eigenschaften können nun verwendet werden, um entsprechende Informationen aus dem Dokument abzufragen. Das Wurzelelement `<rezept>` wird über die Eigenschaft `documentElement` abgefragt. `getAttribute($name)`¹⁵ wird verwendet, um das dazugehörige `quelle`-Attribut abzufragen.

Um das `<gericht>`-Element zu erhalten, wird mit `querySelector("gericht")` das erste `<gericht>`-Element abgefragt und dessen Wert mit `textContent` gespeichert.

Ähnlich werden die einzelnen Zutaten verarbeitet. Zunächst werden alle `<zutat>`-Elemente mit `querySelectorAll()` abgefragt und dann mit einer `foreach`-Schleife durchlaufen. Danach werden für jedes Element dessen Kind-Knoten abgefragt. Durch die Typprüfung `$child instanceof Element` wird sichergestellt, dass es sich um einen echten Element-Knoten und nicht um unnötige Text-Knoten (wie Zeilenumbrüche) handelt. Ist dies der Fall, wird der Wert in `$ingredients` gespeichert.

Auch beim Auslesen der Zubereitungsschritte läuft es ähnlich: Zunächst werden alle `<schritt>`-Elemente abgefragt, danach werden in einer Schleife die Inhalte in `$steps` gespeichert.

```
1 $this->source = $dom->documentElement->getAttribute("quelle") ?? "";
2
3 $this->dish = trim($dom->querySelector("gericht")->textContent);
4
5 foreach ($dom->querySelectorAll("zutat") as $ingredient) {
6     $this->ingredients[] = [];
7     foreach ($ingredient->childNodes as $child) {
```

¹¹<https://www.php.net/manual/de/class.domdocument.php#domdocument.props.documentelement>

¹²<https://www.php.net/manual/de/dom-parentnode.querySelector.php>

¹³<https://www.php.net/manual/de/dom-parentnode.querySelectorAll.php>

¹⁴<https://www.php.net/manual/de/domdocument.getelementbyid.php>

¹⁵<https://www.php.net/manual/de/domelement.getattribute.php>

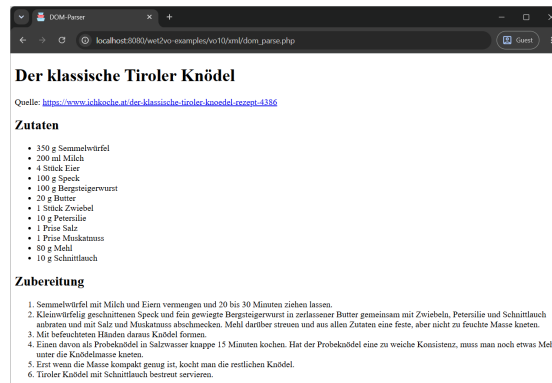


Abbildung 10.2: Im Browser wird das XML-Dokument strukturiert ausgegeben. Das Ergebnis identisch zum vorhergehenden Ansatz.

```

8     if ($child instanceof Element) {
9         $lastIndex = array_key_last($this->ingredients);
10        $this->ingredients[$lastIndex][$child->tagName] = trim($child->textContent);
11    }
12 }
13 }
14
15 foreach ($dom->querySelectorAll("schritt") as $step) {
16     $this->steps[] = trim($step->textContent);
17 }

```

Ablauf starten und Ausgabe: Um den Ablauf zu starten, wird ein Objekt der Klasse `MyDOMParser` erzeugt und darauf die `parse()`-Methode aufgerufen:

```

1 $xmlParser = new MyDOMParser();
2 $xmlParser->parse("rezept.xml");

```

Die Ausgabe des DOM-Parse-Vorgangs ist in Abbildung 10.2 ersichtlich.

10.1.3 XML erzeugen in PHP

Das *Erzeugen von XML* ist der umgekehrte Prozess zum Parsen. Aus einem Set von Daten wird eine strukturierte XML-Datei erstellt. Dabei kann die XML-Datei entweder im Browser ausgegeben oder in einer Datei gespeichert werden. Als Beispiele sollen TV-Shows in XML-Form abgespeichert werden.

Die Sendungen seien im folgenden Array gegeben:

```

1 $this->shows = [
2     [
3         "name" => "The Mandalorian",
4         "service" => "Disney+",
5         "resolution" => "2160p",
6         "duration" => "40",
7     ],
8     [
9         "name" => "Rick and Morty",

```

```

10     "service" => "Netflix",
11     "resolution" => "1080p",
12     "duration" => "20",
13 ],
14 ];

```

Da XML ein Textformat ist, kann es auch völlig *ohne weitere Tools* erzeugt werden. Für simple Strukturen ist dies oft auch völlig ausreichend, wenngleich es nicht die eleganteste Art und Weise ist. In diesem Abschnitt werden jedoch die Erweiterungen *XMLWriter* und *DOM* vorgestellt, die diese Aufgabe effizienter erledigen.

XMLWriter

Eine komfortable Methode des XML-Schreibens ist die Verwendung der Erweiterung *XMLWriter*. Sie ist das Gegenstück zum *XMLReader* und auch seit PHP 5.1 standardmäßig inkludiert. XMLWriter arbeitet ebenfalls sequentiell, d. h. die einzelnen Teile der XML-Datei müssen in der richtigen Reihenfolge definiert werden, man bezeichnet sie daher auch als Stream-Writer. Die Stärke dieser Erweiterung liegt vor allem im Neuerstellen von XML-Inhalten, weniger in der Veränderung von bestehenden Dateien. Dies ist durch die sequentielle Abarbeitung bedingt.

XMLWriter kann sowohl funktional als auch objektorientiert verwendet werden. Die objektorientierte Variante ist jedoch wesentlich eleganter und wird daher in weiterer Folge verwendet. Die folgenden Programmteile gehören zur Klasse *MyStreamWriter*.

Writer initialisieren: Zunächst muss ein neues *XMLWriter*-Objekt erzeugt werden. Diesem Objekt wird dann mitgeteilt, ob die XML-Datei ausgegeben, oder in eine Datei geschrieben werden soll. Für die Ausgabe am Bildschirm wird die XML-Datei in den Speicher geschrieben, daher wird die Methode `openMemory()`¹⁶ benötigt. Für die Ausgabe in eine Datei kommt `openUri($uri)`¹⁷ zum Einsatz. Damit die Elemente korrekt und gut leserlich eingerückt sind, kann über die Funktion `setIndent($indent)`¹⁸ diese Einrückung aktiviert werden.

Im Beispiel wird lediglich das Objekt im Konstruktor angelegt. Das Öffnen der Datei sowie das Setzen der Einrückung findet bereits in der Methode zum Schreiben statt.

```

1 class MyStreamWriter
2 {
3     private XMLWriter $writer;
4     private array $shows;
5
6     public function __construct(array $shows)
7     {
8         $this->writer = new XMLWriter();
9         $this->shows = $shows;
10    }
11
12    public function generateXML(string $file): void
13    {
14        $this->writer->openUri($file);

```

¹⁶<https://www.php.net/manual/de/xmlwriter.openmemory.php>

¹⁷<https://www.php.net/manual/de/xmlwriter.openuri.php>

¹⁸<https://www.php.net/manual/de/xmlwriter.setindent.php>

```

15     $this->writer->setIndent(true);
16
17     // Verarbeitung
18 }
19 }

```

XML erzeugen: Nachdem der Writer konfiguriert wurde, werden der Reihe nach die Bestandteile der XML-Datei erzeugt. Begonnen wird mit der XML-Deklaration. Diese wird durch den Aufruf von `startDocument($version, $encoding)`¹⁹ erzeugt. Gleichzeitig wird damit auch das Dokument begonnen. Als Version sollte 1.0, als Encoding UTF-8 angegeben werden.

In der Regel folgt im nächsten Schritt eine DTD, die unter Zuhilfenahme der Methode `writeDtd($name [, $publicId, $systemId [, $content]])`²⁰ geschrieben werden kann. Im aktuellen Beispiel wird darauf verzichtet.

Es soll nun das Wurzelement `<shows>` erzeugt werden. Da es zunächst nur geöffnet und erst ganz am Schluss wieder geschlossen wird, kommt `startElement($name)`²¹ zum Einsatz. Die Methode erzeugt ein öffnendes Element mit einem gegebenen Namen. Hier wäre der Zeitpunkt, ein XML-Schema anzugeben. Dazu würden die entsprechenden Attribute wie `xmlns:xsi` mit der Methode `writeAttribute($name, $value)`²² hinzugefügt.

Danach sollen innerhalb eines `<show>`-Elements die Details der einzelnen Serien erzeugt werden. Es wird wiederum eine `foreach`-Schleife verwendet, um über das Array mit den Shows zu iterieren. Mittels `startElement()` wird das `<show>`-Element geöffnet. Dann kommt die innere `foreach`-Schleife, die über Name, Service, Auflösung und Dauer iteriert. Zu jedem dieser Werte wird nun ein Element angelegt. Da hier ein ganzes Element, also öffnender Tag, Textinhalt und schließender Tag auf einmal erzeugt werden, kann die Methode `writeElement($tag, $content)`²³ verwendet werden. Sie erzeugt ein komplettes Element mit Inhalt. Nach dieser Schleife wird `endElement()`²⁴ aufgerufen. Diese Methode schließt das zuletzt geöffnete Element, in diesem Fall `<show>` mit seinem Gegenstück `</show>`.

Nun ist noch das Element `<shows>` offen, welches durch einen erneuten Aufruf von `endElement()` geschlossen wird. Danach folgt der Aufruf von `endDocument()`²⁵, welches das Dokument als geschlossen markiert. Ein Aufruf von `flush()`²⁶ leert den Speicher und schreibt das Dokument in die Datei.

```

1 $this->writer->startDocument("1.0", "UTF-8");
2 $this->writer->startElement("shows");
3
4 foreach($this->shows as $show) {
5     $this->writer->startElement("show");
6     foreach ($show as $tag => $data) {

```

¹⁹<https://www.php.net/manual/de/xmlwriter.startdocument.php>

²⁰<https://www.php.net/manual/de/xmlwriter.writedtd.php>

²¹<https://www.php.net/manual/de/xmlwriter.startelement.php>

²²<https://www.php.net/manual/de/xmlwriter.writeattribute.php>

²³<https://www.php.net/manual/de/xmlwriter.writeelement.php>

²⁴<https://www.php.net/manual/de/xmlwriter.endelement.php>

²⁵<https://www.php.net/manual/de/xmlwriter.enddocument.php>

²⁶<https://www.php.net/manual/de/xmlwriter.flush.php>

```
7     $this->writer->writeElement($tag, $data);
8   }
9   $this->writer->endElement();
10 }
11 $this->writer->endElement();
12
13 $this->writer->endDocument();
14
15 $this->writer->flush();
```

Ausgabe: Der soeben gezeigte Code erzeugt eine XML-Datei mit folgendem Inhalt:

```
<?xml version="1.0" encoding="UTF-8"?>
<shows>
  <show>
    <name>The Mandalorian</name>
    <service>Disney+</service>
    <resolution>2160p</resolution>
    <duration>40</duration>
  </show>
  <show>
    <name>Rick and Morty</name>
    <service>Netflix</service>
    <resolution>1080p</resolution>
    <duration>20</duration>
  </show>
</shows>
```

DOM

Ähnlich wie schon in JavaScript, als das DOM verwendet wurde, um neue HTML-Inhalte zu generieren, kann die *DOM XML-Erweiterung* auch verwendet werden, um neue XML-Dateien zu erstellen oder bestehende zu verändern. Vor allem für Zweites ist DOM unerlässlich, da eine XML-Struktur geladen, an einer Stelle verändert und wieder gespeichert werden kann, ohne die gesamte Struktur neu generieren zu müssen.

DOM ist rein objektorientiert aufgebaut. Das Schreiben verläuft sehr ähnlich zum Parsen (siehe Abschnitt 10.1.2). Der folgende Code stammt aus einer Klasse namens `MyDOMWriter`.

Writer initialisieren und XML erzeugen: Wie auch beim Parsen muss ein Objekt vom Typ `Dom\XMLDocument` angelegt werden, dieses mal jedoch mit dem statischen Konstruktor `createEmpty()`. Es empfiehlt sich, die Versionsnummer (1.0) und die Zeichenkodierung (UTF-8) als Argumente anzugeben. Weiters ist es von Vorteil, die Eigenschaft `formatOutput` auf `true` zu setzen, um Zeilenumbrüche und Einrückungen zu erhalten. Beides wird nicht im Konstruktor, sondern direkt in `generateXML()` erledigt. Da ein DOM-Dokument zunächst im Speicher angelegt und dann befüllt und gespeichert wird, sollte dies zusammen geschehen. Ansonsten würde bei einem mehrmaligen Aufruf von `generateXML()` immer wieder die XML-Struktur in dasselbe DOM-Dokument geschrieben.

Nachdem ein neues (leeres) `XMLDocument`-Objekt angelegt wurde, kann nun begonnen werden, die Baumstruktur darin zu generieren. Zunächst wird das Wurzelement `<shows>` erzeugt. Hierfür kommt `createElement($name)` (analog zu JavaScript) zum Einsatz. Sie erzeugt ein leeres Element. Damit dieses aber auch seinen Weg ins XML-Dokument findet, muss es in der DOM-Struktur platziert werden. Hierfür wird `appendChild($node)` verwendet. Die Methode hängt einen Knoten an einen anderen und gibt den neu angehängten wieder zurück. Das Wurzelement wird also erzeugt und an das neu erzeugte `XMLDocument`-Objekt gehängt. Der Rückgabewert wird in `$shows` gespeichert. An dieses Element werden dann die weiteren angehängt.

Um die einzelnen Shows zu erzeugen, kommt wiederum eine `foreach`-Schleife zum Einsatz. Sie läuft über alle Shows und erzeugt zunächst ein `<show>`-Element und hängt es an das bestehende `<shows>`-Wurzelement. Danach wird über die einzelnen Eigenschaften der Show iteriert und jeweils eigene Elemente mit `createElement()` erzeugt. Als Name dient der Array-Schlüssel, als Wert wird der Wert im Array übergeben und mittels `textContent`-Eigenschaften ins leere Element gesetzt. Danach wird das Element in das `<show>`-Element gehängt.

Schließlich wird die XML-Datei mit der Methode `saveXmlFile($file)` in die angegebene Datei geschrieben.

```

1 use Dom\XMLDocument;
2
3 class MyDOMWriter
4 {
5     private array $shows;
6
7     public function __construct(array $shows)
8     {
9         $this->shows = $shows;
10    }
11
12    public function generateXML(string $file): void
13    {
14        $dom = XMLDocument::createEmpty("1.0", "UTF-8");
15        $dom->formatOutput = true;
16
17        $shows = $dom->appendChild($dom->createElement("shows"));
18
19        foreach ($this->shows as $show) {
20            $showElem = $shows->appendChild($dom->createElement("show"));
21            foreach ($show as $tag => $data) {
22                $showData = $dom->createElement($tag);
23                $showData->textContent = $data;
24                $showElem->appendChild($showData);
25            }
26        }
27
28        $dom->saveXmlFile($file);
29    }
30 }
```

Ausgabe: Die erzeugte Ausgabe gleicht der des XMLWriters:

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<shows>
  <show>
    <name>The Mandalorian</name>
    <service>Disney+</service>
    <resolution>2160p</resolution>
    <duration>40</duration>
  </show>
  <show>
    <name>Rick and Morty</name>
    <service>Netflix</service>
    <resolution>1080p</resolution>
    <duration>20</duration>
  </show>
</shows>
```

10.1.4 Diskussion der einzelnen Erweiterungen

Jede der demonstrierten Erweiterungen bietet ihre Vor- und Nachteile. Diese sollen in aller Kürze nochmals zusammen gefasst werden.

Parser

Beim Parsen stehen die Modell, Pull- und objektorientiertes Parsing zur Verfügung. Während sich das erste Modell gut für große Dateien eignen, ist der objektorientierte Vertreter vor allem für den wahlfreien Zugriff zu gebrauchen.

Pull-Parsing stellt einen Mittelweg zwischen veraltetem (und daher ausgelassenem) eventbasiertem und objektorientiertem Parsing dar und kombiniert die Vorteile aus beidem. Einerseits geringer Speicherverbrauch, andererseits Anlehnung an das DOM, was die Verarbeitung der Elemente betrifft.

DOM-Parsing ist vor allem dann interessant, wenn Zugriff auf spezifische Elemente gebraucht wird. Soll ohnehin das gesamte Dokument abgearbeitet werden, stehen der hohe Speicherverbrauch und das Fehlen eines praktischen Iterators dem entgegen.

Writer

Beim Schreiben von XML besteht die Auswahl zwischen dem sequentiellen XMLWriter und einem DOM-basierten Ansatz.

Die XMLWriter-Erweiterung ist modern und objektorientiert und verfolgt einen natürlichen Ansatz. Das Dokument wird von vorne nach hinten aufgebaut, dabei muss die gewollte Struktur jedoch stets im Kopf behalten werden. Ein Stolperstein kann das Vergessen von schließenden Elementen sein, was die Struktur durcheinander bringt.

Die DOM-Erweiterung ist die vom Umfang her mächtigste, aber auch komplizierteste. Sie ist sowohl gut zum Erzeugen von neuem Markup als auch zum Verändern von bestehendem geeignet. Zur Verwendung ist eine gute Vorstellung des DOM-Baums unerlässlich. Da aber Elemente erzeugt und dann hierarchisch aneinander gehängt werden, gibt es weniger Fallstricke im Bezug auf vergessene Elemente und dergleichen.

10.1.5 XML-Komponenten

Auch für die Arbeit mit XML gibt es von der PHP-Community entwickelte Komponenten, die mit Composer einfach installiert werden können.

sabre/xml

Die Bibliothek *sabre/xml*²⁷ folgt so genannten Entwurfsmustern (engl. „design patterns“) und bietet einen strukturierten, objektorientierten Zugriff auf XML – sowohl lesend als auch schreibend. Dabei werden im Hintergrund XMLReader und XMLWriter verwendet, ohne aber deren Methoden verwenden zu müssen. *sabre/xml* ist dabei sehr effizient im Umgang mit XML-Namespaces und versucht generell die Anzahl der Codezeilen, die zu schreiben sind, zu minimieren (vor allem im Vergleich etwa mit DOM).

FluidXML

*FluidXML*²⁸ (installierbar als *servo/fluidxml*) ist eine an das DOM angelehnte XML-Bibliothek. Sie verhält sich nach dem Prinzip des Fluent Programming, das eine natürliche Aneinanderreihung von Methoden vorsieht – quasi als ob Sätze in natürlicher Sprache gesprochen würden. FluidXML greift dabei das Knotenprinzip des DOM auf und ist auch mit ihm vollständig kompatibel, versucht jedoch die manchmal etwas umständlich anmutenden Herangehensweisen des DOM zu vereinfachen.

10.2 JSON lesen und schreiben

Die JavaScript Object Notation – kurz JSON – hat sich neben Formaten wie XML oder YAML schon lange als Datenaustauschformat etabliert. In der Vorlesung Web Fundamentals wurden der Aufbau und die Funktionsweise des Formats erläutert. Auch PHP bietet Funktionen, die in JSON notierte Daten in eine PHP-Datenstruktur umwandeln können bzw. auch den umgekehrten Prozess beherrschen. Diese Funktionen sind streng genommen keine Dateifunktionen, da die JSON-Daten als String eingelesen bzw. generiert werden. Mit Hilfe von `file_get_contents()`²⁹ und `file_put_contents()`³⁰ ist das Lesen und Schreiben von bzw. in Dateien mit einem Zusatzschritt erledigt. Somit bietet sich JSON auch für PHP-Applikationen als Format an, um damit Daten in Dateien zu persistieren.

10.2.1 Lesen von JSON-Dateien

JSON-Daten werden mit der Funktion `json_decode($json [, $assoc])`³¹ eingelesen. Sie verarbeitet den String `$json` und liefert eine PHP-Datenstruktur – im Regelfall ein Objekt, das die JSON-Eigenschaften abbildet. Wird der optionale zweite Parameter `$assoc` auf `true` gesetzt, wird ein assoziatives Array anstelle eines Objekts erzeugt.

²⁷<https://sabre.io/xml/>

²⁸<https://packagist.org/packages/servo/fluidxml>

²⁹<https://www.php.net/manual/de/function.file-get-contents.php>

³⁰<https://www.php.net/manual/de/function.file-put-contents.php>

³¹<https://www.php.net/manual/de/function.json-decode.php>

Das folgende Beispiel verwendet einen Adressbucheintrag in JSON, der zu Demonstrationszwecken etwas mehr Verschachtelung bietet, als das Knödelrezept. Der Eintrag sei in einer Datei `addressbook.json` gespeichert.

```
1 {
2   "firstName": "John",
3   "lastName": "Doe",
4   "age": 25,
5   "address": {
6     "street": "21 Doe St.",
7     "city": "Doeville",
8     "zip": "12345",
9     "state": "NY",
10    "country": "USA"
11  },
12  "phoneNumbers": [
13    {
14      "type": "home",
15      "number": "800 123 456-789"
16    },
17    {
18      "type": "work",
19      "number": "800 999 666-333"
20    }
21  ]
22 }
```

Ausgabe als Objekt

Diese Datei wird nun mit `file_get_contents()` in eine Zeichenkette gelesen. Diese wird der Funktion `json_decode()` übergeben. Das resultierende Objekt wird mittels `var_dump()` ausgegeben.

```
1 $jsonString = file_get_contents("addressbook.json");
2 $jsonData = json_decode($jsonString);
3 var_dump($jsonData);
```

Das Ergebnis ist folgendes Objekt:

```
/var/www/html/wet2vo-examples/vo10/json/jsondecode.php:11:
object(stdClass)[1]
  public 'firstName' => string 'John' (length=4)
  public 'lastName' => string 'Doe' (length=3)
  public 'age' => int 25
  public 'address' =>
    object(stdClass)[2]
      public 'street' => string '21 Doe St.' (length=10)
      public 'city' => string 'Doeville' (length=8)
      public 'zip' => string '12345' (length=5)
      public 'state' => string 'NY' (length=2)
      public 'country' => string 'USA' (length=3)
  public 'phoneNumbers' =>
    array (size=2)
      0 =>
        object(stdClass)[3]
          public 'type' => string 'home' (length=4)
```

```

        public 'number' => string '800 123 456-789' (length=15)
1 =>
    object(stdClass)[4]
        public 'type' => string 'work' (length=4)
        public 'number' => string '800 999 666-333' (length=15)

```

Der Zugriff auf die einzelnen Eigenschaften erfolgt nun mittels der gewohnten PHP-Objekt- bzw. Arraysyntax. Im Folgenden werden Nachname, Straße und die zweite Telefonnummer ausgegeben.

```

1 echo $jsonData->lastName; // Doe
2 echo $jsonData->address->street; // 21 Doe St.
3 echo $jsonData->phoneNumbers[1]->number; // 800 999 666-333

```

Ausgabe als Array

Wird die Funktion mit einem zweiten Parameter, der den Wert `true` hat, aufgerufen, erzeugt `json_decode()` ein assoziatives Array anstelle eines Objekts.

```

1 $jsonString = file_get_contents("addressbook.json");
2 $jsonData = json_decode($jsonString, true);
3 var_dump($jsonData);

```

Dieses sieht wie folgt aus:

```

/var/www/html/wet2vo-examples/vo10/json/jsondecode.php:11:
array (size=5)
    'firstName' => string 'John' (length=4)
    'lastName' => string 'Doe' (length=3)
    'age' => int 25
    'address' =>
        array (size=5)
            'street' => string '21 Doe St.' (length=10)
            'city' => string 'Doeville' (length=8)
            'zip' => string '12345' (length=5)
            'state' => string 'NY' (length=2)
            'country' => string 'USA' (length=3)
    'phoneNumbers' =>
        array (size=2)
            0 =>
                array (size=2)
                    'type' => string 'home' (length=4)
                    'number' => string '800 123 456-789' (length=15)
            1 =>
                array (size=2)
                    'type' => string 'work' (length=4)
                    'number' => string '800 999 666-333' (length=15)

```

Der Zugriff auf die einzelnen Eigenschaften geschieht ausschließlich über die Array-syntax. Es werden wieder Nachname, Straße und die zweite Telefonnummer ausgegeben.

```

1 echo $jsonData["lastName"]; // Doe
2 echo $jsonData["address"]["street"]; // 21 Doe St.
3 echo $jsonData["phoneNumbers"][1]["number"]; // 800 999 666-333/

```

10.2.2 Schreiben von JSON-Dateien

Das Schreiben bzw. Erstellen eines JSON-Datensatzes geschieht mit Hilfe der Funktion `json_encode($value [, $options])`³². Sie akzeptiert mit `$value` jede beliebige PHP-Datenstruktur (ausgenommen Ressourcen). Diese Datenstruktur wird dann in eine valide JSON-Zeichenkette umgewandelt. Mit `$options` können zusätzliche Optionen spezifiziert werden. Hier empfiehlt sich eine Kombination aus `JSON_UNESCAPED_SLASHES` und `JSON_PRETTY_PRINT` um Schrägstriche nicht zu escapen und das Ergebnis schön formatiert aufzubereiten. Letzteres macht beim Schreiben in Dateien umso mehr Sinn, wenn diese von Menschen gelesen werden sollen.

Im folgenden Beispiel werden vier unterschiedliche Datentypen angelegt: ein Objekt, ein indiziertes Array, ein assoziatives Array und eine einfache Ganzzahl. Die Klassendefinition verwendet dabei die in Abschnitt 4.2.4 gezeigte Constructor Property Promotion, die die Definition von Eigenschaften und die Zuweisung von Parametern an selbe deutlich verkürzt.

```
1 class Person
2 {
3     public function __construct(public string $name, public int $age, public string
4         $country) {}
5 }
6 $array = [
7     "Good",
8     "Bad",
9     "Ugly",
10 ];
11
12 $assocArray = [
13     "key" => "value",
14     "otherkey" => "othervalue",
15 ];
16
17 $simpleNumber = 42;
```

Diese werden jeweils mit `json_encode()` in eine JSON-Datenstruktur umgewandelt und danach mit `file_put_contents()` in eine Datei geschrieben.

```
1 $jsonObject = json_encode(
2     new Person("John Doe", 34, "Doeland"),
3     JSON_UNESCAPED_SLASHES | JSON_PRETTY_PRINT
4 );
5 $jsonArray = json_encode(
6     $array,
7     JSON_UNESCAPED_SLASHES | JSON_PRETTY_PRINT
8 );
9 $jsonAssocArray = json_encode(
10    $assocArray,
11    JSON_UNESCAPED_SLASHES | JSON_PRETTY_PRINT
12 );
13 $jsonInteger = json_encode(
14    $simpleNumber,
15    JSON_UNESCAPED_SLASHES | JSON_PRETTY_PRINT
```

³²<https://www.php.net/manual/de/function.json-encode.php>

```
16 );  
17  
18 file_put_contents("object.json", $jsonObject);  
19 file_put_contents("array.json", $jsonArray);  
20 file_put_contents("assocarray.json", $jsonAssocArray);  
21 file_put_contents("integer.json", $jsonInteger);
```

Das Ergebnis sieht wie folgt aus. PHP-Objekte und assoziative Arrays erzeugen JSON-Objekte. Ein indiziertes Array erzeugt ein JSON-Array. Eine einfache Integerzahl wird auch als solche in JSON dargestellt. Um also die gewünschte JSON-Struktur zu erhalten (in der Regel ein Objekt), muss auch die entsprechende PHP-Datenstruktur gewählt werden.

```
** object.json **  
{  
  "name": "John Doe",  
  "age": 34,  
  "country": "Doeland"  
}  
  
** array.json **  
[  
  "Good",  
  "Bad",  
  "Ugly"  
]  
  
** assocarray.json **  
{  
  "key": "value",  
  "otherkey": "othervalue"  
}  
  
** integer.json **  
42
```

10.2.3 JSON Fehlerbehandlung

Sind beim Kodieren oder Dekodieren von JSON-Daten Fehler aufgetreten, so können diese mit Hilfe der Funktionen `json_last_error()`³³ oder `json_last_error_msg()`³⁴ identifiziert werden. Erstere Funktion gibt dabei den Fehlercode als Integerzahl zurück, die mit Hilfe von Konstanten interpretiert werden muss. Einfacher ist daher die Verwendung der zweiten Funktion, die diese Interpretation bereits selbst vornimmt und eine Meldung als String zurück gibt. Ist kein Fehler passiert, wird „No error“ ausgegeben.

10.2.4 JSON validieren

Seit PHP 8.3 steht mit `json_validate()`³⁵ eine Funktion zur Verfügung, die eine JSON-Datenstruktur auf Korrektheit überprüft. Sie kann eingesetzt werden, wenn es nicht

³³ <https://www.php.net/manual/de/function.json-last-error.php>

³⁴ <https://www.php.net/manual/de/function.json-last-error-msg.php>

³⁵ <https://www.php.net/manual/de/function.json-validate.php>

um den Inhalt geht, sondern nur darum, ob die JSON-Syntax eingehalten wurde. Da auch `json_decode()` eine Validierung durchführt, sollten die beiden Funktionen nicht hintereinander verwendet werden, weil dies ein (unnötiges) doppeltes Verarbeiten der JSON-Daten zur Folge hat.

```
1 $jsonString = file_get_contents("addressbook.json");  
2 $jsonIsValid = json_validate($jsonString);  
3  
4 echo $jsonIsValid ? "JSON structure is valid." : "JSON structure is not valid.";
```

10.3 Grafiken mit der GD Graphics Library

Webseiten bestehen in der Regel nicht nur aus Text, meist sind auch *Bilder* in irgendeiner Form involviert. Logos, Buttons, Werbebanner oder Icons sind nur einige Beispiele dafür. Viele, wenn nicht der Großteil dieser Bilder sind rein statisch und wurden mit Werkzeugen wie Photoshop erstellt. Andere jedoch müssen dynamisch generiert werden, da ihr Inhalt von bestimmten Faktoren abhängt. Beispiele hierfür sind Grafiken zum aktuellen Aktienkurs oder personalisierte Werbebanner.

PHP ermöglicht die dynamische Erzeugung von Grafiken durch diverse *Grafikbibliotheken*. Dies sind ganz allgemein Softwarepakete, die es ermöglichen, Grafikausgabe am Bildschirm darzustellen oder in einer Datei zu speichern, d. h. Grafiken zu erstellen.

Bekannte Vertreter sind *ImageMagick*³⁶, *GraphicsMagick*³⁷ oder auch die GD Graphics Library, welche hier für einige wichtige Funktionalitäten vorgestellt wird.

10.3.1 Allgemeines zur GDlib

Die *GD Graphics Library*³⁸, auch *GD Library*, *GDlib*, *LibGD* oder kurz *GD*, ist eine Grafikbibliothek geschrieben in C, entwickelt von Thomas Boutell und mittlerweile fortgeführt von Pierre Joye. Der Name „GD“ stand ursprünglich für „GIF Draw“, nachdem aber aufgrund patentrechtlicher Umstände die GIF-Unterstützung kurzzeitig entfernt werden musste, wurde der Name auf „Graphics Draw“ geändert.

Die Bibliothek kann mit den bekannten Formaten JPEG, GIF, PNG, WebP, AVIF, TGA und BMP umgehen (teilweise müssen dazu aber die entsprechenden Bibliotheken im Betriebssystem installiert sein). Daneben werden noch WBMP, XBM und XPM sowie die eigenen Formate GD und GD2 unterstützt. Dies ist im Vergleich zu ImageMagick eine eher bescheidene Auswahl an verarbeitbaren Formaten, der große Vorteil von GD liegt jedoch in der Tatsache, dass die Bibliothek seit PHP 4.3.0 ein Standardbestandteil der Distribution ist und somit auf allen aktuellen PHP-Systemen verfügbar ist.

Zusätzlich zur guten Verfügbarkeit zeichnet sich die GD Graphics Library durch ihre einfache Verwendung aus. Einen Überblick über alle GD-Funktionalitäten bietet das PHP-Handbuch³⁹.

³⁶<https://imagemagick.org/>

³⁷<http://www.graphicsmagick.org/>

³⁸<https://libgd.github.io/>

³⁹<https://www.php.net/manual/de/book.image.php>

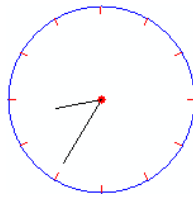


Abbildung 10.3: Die Grafik der Uhr wird mittels GD Library erzeugt und zeigt somit immer die richtige Uhrzeit an. Quelle: [22].

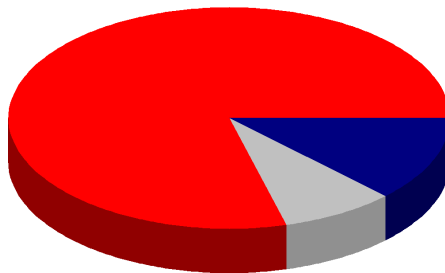


Abbildung 10.4: Das dynamische Kuchendiagramm ist ideal, um Daten anzuzeigen, die sich verändern können. Quelle: [17].

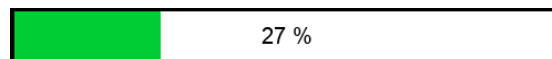


Abbildung 10.5: Die Fortschrittsanzeige wird dynamisch durch GD berechnet und erzeugt.

Einsatzgebiete

Die Einsatzgebiete für dynamische Grafiken im Web sind vielfältig. Zunächst ist die *Erzeugung* von Grafiken ein wichtiger Punkt. Wann immer nicht bekannt ist, wie eine Grafik zu einem gegebenen Zeitpunkt aussehen wird oder wenn die Grafik auf sich verändernden Daten beruht, muss die Grafik dynamisch erzeugt werden. Ein Beispiel ist etwa eine Uhr, wie in Abbildung 10.3 ersichtlich. Soll eine analoge Uhr, die immer die richtige Uhrzeit anzeigt, grafisch dargestellt werden, würden 720 (mit Sekunden sogar 43.200) verschiedene Bilder benötigt. Mit einer dynamisch erzeugten Grafik wird die Position der einzelnen Zeiger einfach anhand der aktuellen Zeit gesetzt.

Ein weiteres Beispiel ist die Erzeugung von Diagrammen, etwa Balken- oder Kuchendiagrammen. Die Verteilung des Diagramms kann hier etwa auf einem sich verändernden Datensatz beruhen. Dies macht es schwer möglich, das Diagramm im Vorhinein zu erstellen. Abbildung 10.4 zeigt ein 3D-Kuchendiagramm, das mit GD erzeugt wurde.

Ein weiterer Anwendungsfall sind Fortschrittsanzeigen. Wann immer es nötig ist, einen Fortschritt oder Verlauf anzuzeigen, können mit GD grafische Fortschrittsanzeigen erzeugt werden. Abbildung 10.5 stellt ein Beispiel dar.

Schließlich sind Grafikbibliotheken wie GD auch ideal, um bestehende Bilder zu bearbeiten. Häufige Szenarien sind hierbei das Verkleinern von Bildern, das Entnehmen

eines Ausschnitts oder auch das Konvertieren von einem Grafikformat in ein anderes. Dies wird auch der hier demonstrierte Einsatzzweck sein.

Die folgenden Abschnitte erläutern nun, wie mittels der GD Graphics Library Bilder erstellt und verarbeitet werden.

10.3.2 Neue Bilder erstellen

Jedem neuen Bild liegt eine *Zeichenfläche* (engl. „canvas“) zugrunde. Diese ist pixelbasiert und hat eine feste Größe. Die Zeichenfläche selbst ist noch nicht sichtbar und enthält noch nichts (genauer gesagt ist sie komplett schwarz). Sie ist vielmehr eine einfache Repräsentation des Bildes, auf der dann gearbeitet wird. Die Zeichenfläche wird als Objekt vom Typ `GdImage` erstellt. Damit das Bild dann wirklich angezeigt wird, muss eine eigene Funktion aufgerufen werden (siehe Abschnitt 10.3.4).

Um nun eine neue Zeichenfläche zu erzeugen, bietet GD zwei Funktionen:

- `imagecreate($width, $height)` und
- `imagecreatetruecolor($width, $height)`.

`imagecreate`

Die Funktion `imagecreate($width, $height)`⁴⁰ erzeugt eine neue Zeichenfläche mit 256 Farben. Die Funktion gibt das `GdImage`-Objekt zurück, das in weiterer Folge für alle Operationen benötigt wird.

Der folgende Code erzeugt eine Zeichenfläche mit 200×200 Pixeln und 256 möglichen Farben. Das Objekt ist dann über `$image` verfügbar.

```
1 $image = imagecreate(200, 200);
```

`imagecreatetruecolor`

`imagecreatetruecolor($width, $height)`⁴¹ erzeugt ebenso eine Zeichenfläche, bietet jedoch ein Truecolor-Bild mit 16,7 Millionen Farben. Die Syntax verlangt ebenfalls die Angabe einer Breite und Höhe, zurückgegeben wird wieder ein `GdImage`-Objekt.

Im Folgenden wird eine Zeichenfläche mit 16,7 Millionen Farben und ebenfalls 200×200 Pixeln erzeugt:

```
1 $image = imagecreatetruecolor(200, 200);
```

Generell sollte immer `imagecreatetruecolor()` verwendet werden, um die komplette Farbpalette zur Verfügung zu haben. Eine Ausnahme ist die Erzeugung von GIF-Bildern. Diese unterstützen ohnehin nur 256 Farben, somit ist auch das Anlegen eines Truecolor-Canvas nicht zielführend.

10.3.3 Bestehende Bilder öffnen

Soll eine bestehende Bilddatei *geöffnet* werden, kann das Anlegen einer leeren Zeichenfläche entfallen. Vielmehr gibt es eigene Funktionen, die ein `GdImage`-Objekt mit den

⁴⁰<https://www.php.net/manual/de/function.imagecreate.php>

⁴¹<https://www.php.net/manual/de/function.imagecreatetruecolor.php>

Daten einer bestehenden Bilddatei zurückgegeben können. Je nach Dateityp kommt eine andere Funktion zum Einsatz:

- `imagecreatefrompng($filename)`⁴²,
- `imagecreatefromjpeg($filename)`⁴³,
- `imagecreatefromgif($filename)`⁴⁴,
- `imagecreatefromwebp($filename)`⁴⁵,
- `imagecreatefromavif($filename)`⁴⁶,
- `imagecreatefromtga($filename)`⁴⁷,
- `imagecreatefrombmp($filename)`⁴⁸,
- `imagecreatefromwbmp($filename)`⁴⁹,
- `imagecreatefromxbm($filename)`⁵⁰,
- `imagecreatefromxpm($filename)`⁵¹,
- `imagecreatefromgd($filename)`⁵²,
- `imagecreatefromgd2($filename)`⁵³.

Im folgenden Code wird die bestehende Datei `bild1.png` geöffnet. Die Daten (d. h. die Zeichenfläche) stehen dann wieder als Objekt in der Variable `$image` zur Verfügung. Der Öffnungsprozess ist derselbe für alle anderen Formate. Die Formate GD und GD2 sind eigene GD Library Formate, die verwendet werden können, wenn eine große Menge an Bildern zwischengespeichert und wieder geöffnet werden muss, da sie native Formate der Bibliothek sind.

```
1 $image = imagecreatefrompng("bild1.png");
```

Eine weitere Möglichkeit, verschiedenste bestehende Bilder zu öffnen, stellt die Funktion `imagecreatefromstring($image)`⁵⁴ dar. Als Parameter erwartet sie einen so genannten Datenstrom mit Bildinhalten, also einen String, der die Dateiinhalte repräsentiert. Ein solcher kann, wie bereits bei JSON in Abschnitt 10.2 bereits erwähnt, mit der Funktion `file_get_contents($filename)` erhalten werden. Werden beide Funktionen kombiniert, kann diese zum Öffnen verschiedenster Dateitypen (sofern unterstützt) verwendet werden:

```
1 $image = imagecreatefromstring(file_get_contents("bild1.jpg"));
```

⁴²<https://www.php.net/manual/de/function.imagecreatefrompng.php>

⁴³<https://www.php.net/manual/de/function.imagecreatefromjpeg.php>

⁴⁴<https://www.php.net/manual/de/function.imagecreatefromgif.php>

⁴⁵<https://www.php.net/manual/de/function.imagecreatefromwebp.php>

⁴⁶<https://www.php.net/manual/de/function.imagecreatefromavif.php>

⁴⁷<https://www.php.net/manual/de/function.imagecreatefromtga.php>

⁴⁸<https://www.php.net/manual/de/function.imagecreatefrombmp.php>

⁴⁹<https://www.php.net/manual/de/function.imagecreatefromwbmp.php>

⁵⁰<https://www.php.net/manual/de/function.imagecreatefromxbm.php>

⁵¹<https://www.php.net/manual/de/function.imagecreatefromxpm.php>

⁵²<https://www.php.net/manual/de/function.imagecreatefromgd.php>

⁵³<https://www.php.net/manual/de/function.imagecreatefromgd2.php>

⁵⁴<https://www.php.net/manual/de/function.imagecreatefromstring.php>

10.3.4 Bilder ausgeben

Um Bilder *auszugeben*, gibt es zwei Möglichkeiten. Entweder wird das Bild dem Browser direkt zur Anzeige übergeben oder in einer Datei gespeichert.

Gleichgültig, auf welche Art die Ausgabe dann schlussendlich erfolgt, muss zuvor die Entscheidung getroffen werden, in welchem Grafikformat die Ausgabe erfolgen soll. Ähnlich wie beim Einlesen von bestehenden Bildern gibt es dann auch für die Ausgabe eine Funktion pro Grafikformat (TGA- und XPM-Ausgabe werden jedoch nicht unterstützt):

- `imagepng($image [, $filename])`⁵⁵,
- `imagejpeg($image [, $filename])`⁵⁶,
- `imagegif($image [, $filename])`⁵⁷,
- `imagewebp($image [, $filename])`⁵⁸,
- `imageavif($image [, $filename])`⁵⁹,
- `imagebmp($image [, $filename])`⁶⁰,
- `imagewbmp($image [, $filename])`⁶¹,
- `imagexbm($image [, $filename])`⁶²,
- `imagegd($image [, $filename])`⁶³,
- `imagegd2($image [, $filename])`⁶⁴.

Im Browser

Wurde ein Ausgabeformat gewählt, in dem das Bild im Browser angezeigt werden soll, so muss die jeweilige Funktion mit einem Parameter – nämlich dem `GdImage`-Objekt – aufgerufen werden. *Davor* jedoch muss dem Browser mitgeteilt werden, dass es sich bei der angezeigten Datei um eine Bilddatei im jeweils gewählten Format handelt.

Wird das Bild direkt im Browser ausgegeben, d. h. wird die PHP-Datei aufgerufen, deren alleiniger Zweck es ist, ein Bild zu erzeugen und darzustellen, so muss mit Hilfe der Funktion `header()` der **Content-Type-Header** auf diesen Bildtyp gesetzt werden. Somit weiß der Browser, wenn er die Response vom Server erhält, dass ein Bild und keine HTML-Seite retour kommt.

Der folgende Code wird in der Datei `emptyimage.php` gespeichert. Es wird ein neues Bild angelegt und danach sofort ausgegeben. Da keinerlei Zeichenoperationen geschehen, wird lediglich ein schwarzes Rechteck angezeigt, da ein neues Bild nur schwarz enthält. Das Bild wird als PNG ausgegeben (durch die Verwendung von `imagepng()`), zuvor

⁵⁵<https://www.php.net/manual/de/function.imagepng.php>

⁵⁶<https://www.php.net/manual/de/function.imagejpeg.php>

⁵⁷<https://www.php.net/manual/de/function.imagegif.php>

⁵⁸<https://www.php.net/manual/de/function.imagewebp.php>

⁵⁹<https://www.php.net/manual/de/function.imageavif.php>

⁶⁰<https://www.php.net/manual/de/function.imagebmp.php>

⁶¹<https://www.php.net/manual/de/function.imagewbmp.php>

⁶²<https://www.php.net/manual/de/function.imagexbm.php>

⁶³<https://www.php.net/manual/de/function.imagegd.php>

⁶⁴<https://www.php.net/manual/de/function.imagegd2.php>

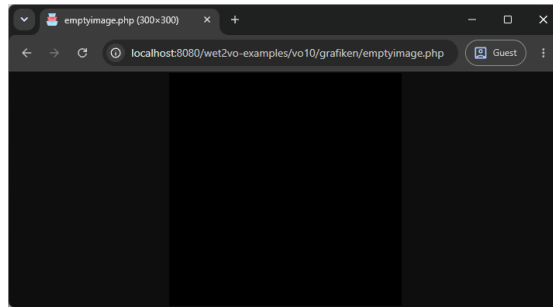


Abbildung 10.6: Ein leeres Bild wird ausgegeben. Die Zeichenfläche ist standardmäßig schwarz. Die Ausgabe erfolgt als 300×300 Pixel PNG-Bild. Interessant ist die URL-Zeile des Browsers: Hier wird die PHP-Datei (und keine Grafikdatei) aufgerufen.

wird mit der Funktion `header()` der **Content-Type** auf `image/png` gesetzt. Abbildung 10.6 zeigt die Ausgabe.

```
1 $image = imagecreatetruecolor(300, 300);
2
3 header("Content-type: image/png");
4 imagepng($image);
```

Als Datei

Das Schreiben einer Bilddatei gestaltet sich ganz ähnlich. Hier kommt jedoch bei den Ausgabemethoden ein zweiter Parameter hinzu, der den zu verwendenden Dateinamen angibt. Das Setzen des **Content-Type** Headers kann entfallen, da die PHP-Datei in der Response ja kein Bild zurück liefert, sondern dieses direkt auf die Festplatte schreibt.

Die Datei `writeimage.php` erzeugt ebenfalls ein 300×300 Pixel großes Bild, ohne (d.h. nur mit schwarzem) Inhalt. Die Ausgabe wird jedoch direkt in die Datei `emptyimage.png` geschrieben. Wird also `writeimage.php` im Browser aufgerufen, so wird nichts angezeigt, stattdessen wird aber eine PNG-Datei im aktuellen Verzeichnis erstellt.

```
1 $image = imagecreatetruecolor(300, 300);
2
3 imagepng($image, "emptyimage.png");
```

10.3.5 Zeichenfunktionen

Um mit Hilfe der GD-Library dynamisch Inhalte zu erstellen, können die diversen Zeichenfunktionen verwendet werden. Diese seien an dieser Stelle jedoch nur kurz aufgelistet:

- Farben definieren: `imagecolorallocate($image, $red, $green, $blue)`⁶⁵,
- Linienstärke: `imagesetthickness($image, $thickness)`⁶⁶,

⁶⁵ <https://www.php.net/manual/de/function.imagecolorallocate.php>

⁶⁶ <https://www.php.net/manual/de/function.imagesetthickness.php>

- Linienstil: `imagesetstyle($image, $style)`⁶⁷,
- Einzelne Pixel: `imagesetpixel($image, $x, $y, $color)`⁶⁸,
- Linien: `imageline($image, $x1, $y1, $x2, $y2, $color)`⁶⁹,
- Rechtecke: `imagerectangle($image, $x1, $y1, $x2, $y2)`⁷⁰,
- Gefüllte Rechtecke: `imagefilledrectangle($image, $x1, $y1, $x2, $y2)`⁷¹,
- Polygone: `imagepolygon($image, $points, $color)`⁷²,
- Gefüllte Polygone: `imagefilledpolygon($image, $points, $color)`⁷³,
- Kreisbögen: `imagearc($image, $cx, $cy, $w, $h, $s, $e, $c)`⁷⁴,
- Gefüllte Kreisbögen: `imagefilledarc($image, $cx, $cy, $w, $h, $s, $e, $c, $st)`⁷⁵,
- Ellipsen: `imageellipse($image, $cx, $cy, $w, $h, $c)`⁷⁶,
- Gefüllte Ellipsen: `imagefilledellipse($image, $cx, $cy, $w, $h, $c)`⁷⁷,
- Text mit TrueType-Schriften: `imagefttext($image, $size, $angle, $x, $y, $color, $font, $text)`⁷⁸

10.3.6 Bilder kopieren

Die GD Graphics Library ermöglicht es, Inhalte eines Bildes in ein zweites zu *kopieren*. Es kann das gesamte Bild oder nur ein Ausschnitt kopiert werden. Die Funktion `imagecopy($dstIm, $srcIm, $dstX, $dstY, $srcX, $srcY, $srcW, $srcH)`⁷⁹ hat die folgenden Parameter:

- `$dstIm`: Das Zielbild, in das kopiert werden soll.
- `$srcIm`: Das Quellbild, aus dem die Inhalte entnommen werden sollen.
- `$dstX`: Die *x*-Koordinate im Zielbild.
- `$dstY`: Die *y*-Koordinate im Zielbild.
- `$srcX`: Die *x*-Koordinate im Quellbild.
- `$srcY`: Die *y*-Koordinate im Quellbild.
- `$srcW`: Die Breite des Ausschnitts im Quellbild.
- `$srcH`: Die Höhe des Ausschnitts im Quellbild.

Abbildung 10.7 illustriert, wie ein Ausschnitt aus dem Quellbild entnommen und im Zielbild eingefügt wird.

⁶⁷<https://www.php.net/manual/de/function.imagesetstyle.php>

⁶⁸<https://www.php.net/manual/de/function.imagesetpixel.php>

⁶⁹<https://www.php.net/manual/de/function.imageline.php>

⁷⁰<https://www.php.net/manual/de/function.imagerectangle.php>

⁷¹<https://www.php.net/manual/de/function.imagefilledrectangle.php>

⁷²<https://www.php.net/manual/de/function.imagepolygon.php>

⁷³<https://www.php.net/manual/de/function.imagefilledpolygon.php>

⁷⁴<https://www.php.net/manual/de/function.imagearc.php>

⁷⁵<https://www.php.net/manual/de/function.imagefilledarc.php>

⁷⁶<https://www.php.net/manual/de/function.imageellipse.php>

⁷⁷<https://www.php.net/manual/de/function.imagefilledellipse.php>

⁷⁸<https://www.php.net/manual/de/function.imagefttext.php>

⁷⁹<https://www.php.net/manual/de/function.imagecopy.php>

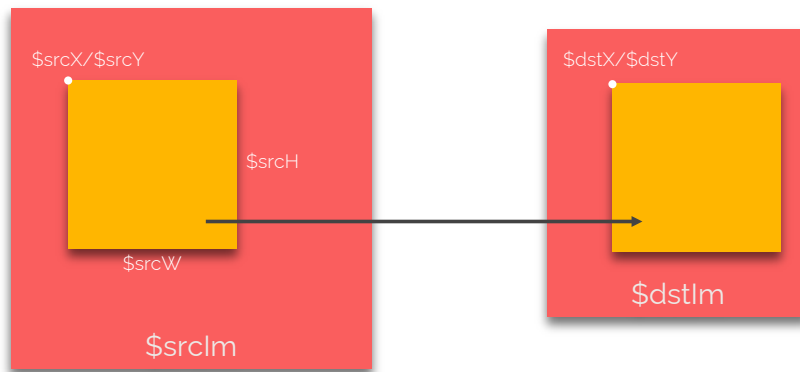


Abbildung 10.7: Ein Ausschnitt aus dem Quellbild wird ins Zielbild kopiert.

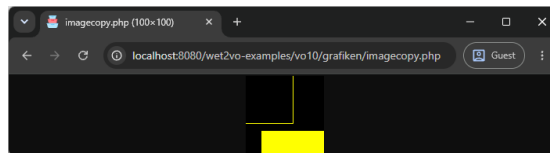


Abbildung 10.8: Ein Ausschnitt aus dem 300×300 Pixel großen Bild wird in ein neues, 100×100 Pixel großes Bild kopiert.

Der folgende Code erstellt zunächst mit zwei der Zeichenfunktionen zwei Rechtecke. Es wird dann, beginnend bei den Koordinaten (50, 50) ein 100×100 Pixel großer Ausschnitt entnommen und in ein neues, ebenfalls 100×100 großes Bild kopiert. Abbildung 10.8 zeigt das Ergebnis.

```
1 $image1 = imagecreatetruecolor(300, 300);
2 $image2 = imagecreatetruecolor(100, 100);
3
4 $yellow = imagecolorallocate($image1, 255, 255, 0);
5
6 imagerectangle($image1, 10, 10, 110, 110, $yellow);
7 imagefilledrectangle($image1, 70, 120, 280, 250, $yellow);
8 imagecopy($image2, $image1, 0, 0, 50, 50, 100, 100);
9
10 header("Content-type: image/png");
11 imagepng($image2);
```

10.3.7 Bildgröße ändern

Analog zum Kopieren funktioniert auch die Änderung der *Bildgröße*. Eigentlich handelt es sich auch hierbei um einen Kopiervorgang, jedoch ist beim Einfügen eine Größenänderung des Zielbereichs möglich, sodass der Ausschnitt entweder vergrößert oder verkleinert dargestellt werden kann.

Zwei Funktionen stehen für diesen Vorgang zur Verfügung:

- `imagecopyresampled($dstIm, $srcIm, $dstX, $dstY, $srcX, $srcY,`

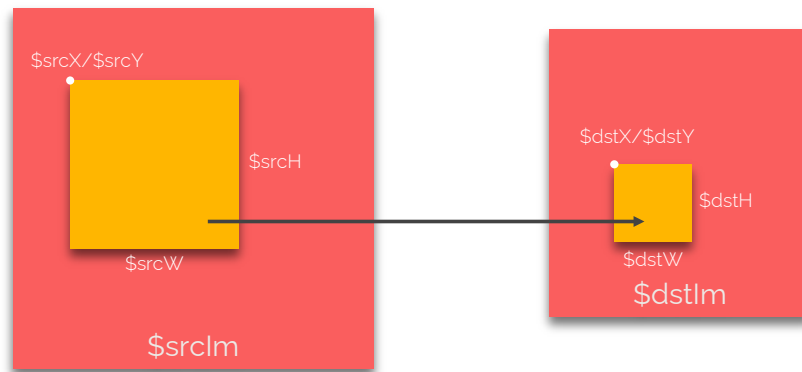


Abbildung 10.9: Ein Ausschnitt aus dem Quellbild wird ins Zielbild kopiert und dabei skaliert.

`$dstW, $dstH, $srcW, $srcH`)⁸⁰ und

- `imagecopyresized($dstIm, $srcIm, $dstX, $dstY, $srcX, $srcY, $dstW, $dstH, $srcW, $srcH)`.⁸¹

Beide weisen eine ähnliche Funktionsweise auf, `imagecopyresampled()` ist jedoch vorzuziehen, da beim Skalieren die einzelnen Farbwerte interpoliert werden, was zu einem visuell besseren Ergebnis führt. Die Parameter beider Funktionen lauten wie folgt:

- `$dstIm`: Das Zielbild, in das kopiert werden soll.
- `$srcIm`: Das Quellbild, aus dem die Inhalte entnommen werden sollen.
- `$dstX`: Die x -Koordinate im Zielbild.
- `$dstY`: Die y -Koordinate im Zielbild.
- `$srcX`: Die x -Koordinate im Quellbild.
- `$srcY`: Die y -Koordinate im Quellbild.
- `$dstW`: Die Breite des Ausschnitts im Zielbild.
- `$dstH`: Die Höhe des Ausschnitts im Zielbild.
- `$srcW`: Die Breite des Ausschnitts im Quellbild.
- `$srcH`: Die Höhe des Ausschnitts im Quellbild.

Abbildung 10.9 zeigt, wie ein Ausschnitt aus dem Quellbild entnommen und skaliert im Zielbild eingefügt wird.

Der folgende Code erstellt wiederum zwei Rechtecke. Es wird nun aber das gesamte, 300×300 Pixel große Bild entnommen und in ein neues, 100×100 großes Bild skaliert eingefügt. Abbildung 10.10 zeigt das Ergebnis.

```
1 $image1 = imagecreatetruecolor(300, 300);
2 $image2 = imagecreatetruecolor(100, 100);
3
4 $yellow = imagecolorallocate($image1, 255, 255, 0);
```

⁸⁰<https://www.php.net/manual/de/function.imagecopyresampled.php>

⁸¹<https://www.php.net/manual/de/function.imagecopyresized.php>



Abbildung 10.10: Das gesamte, 300×300 Pixel großes Quellbild wird in das 100×100 Pixel große Zielbild kopiert und dabei skaliert.

```

5
6 imagerectangle($image1, 10, 10, 110, 110, $yellow);
7 imagefilledrectangle($image1, 70, 120, 280, 250, $yellow);
8
9 imagecopyresampled($image2, $image1, 0, 0, 0, 0, 100, 100, 300, 300);
10
11 header("Content-type: image/png");
12 imagepng($image2);

```

10.3.8 Bilder rotieren

Um Bilder zu *rotieren*, steht `imagerotate($image, $angle, $bgcolor)`⁸² zur Verfügung. Dabei wird das Bild um seinen Mittelpunkt um den in `$angle` spezifizierten Winkel rotiert. Mit `$bgcolor` wird die Farbe der Region bestimmt, die nach der Rotation sichtbar wird.

Die Funktion gibt eine neue `GdImage`-Instanz zurück. Die Größe des neuen Bildes kann sich verändern, denn das Bild wird notfalls erweitert und nicht abgeschnitten.

Der folgende Code rotiert zwei erstellte Rechtecke um 45 Grad. Das resultierende Bild ist nicht mehr 300×300 Pixel sondern 426×426 Pixel groß und in Abschnitt 10.11 ersichtlich.

```

1 $width = $height = 300;
2 $image = imagecreatetruecolor($width, $height);
3
4 $yellow = imagecolorallocate($image, 255, 255, 0);
5 $black = imagecolorallocate($image, 0, 0, 0);
6
7 imagerectangle($image, 10, 10, 110, 110, $yellow);
8 imagefilledrectangle($image, 70, 120, 280, 250, $yellow);
9
10 $image = imagerotate($image, 45, $black);
11
12 header("Content-type: image/png");
13 imagepng($image);

```

10.4 PDFs erzeugen

PDF, das *Portable Document Format* von Adobe, ist der globale Standard für plattformunabhängige Dokumente. Es wurde im Jahr 1992 erstmals veröffentlicht und wird bis

⁸²<https://www.php.net/manual/de/function.imagerotate.php>

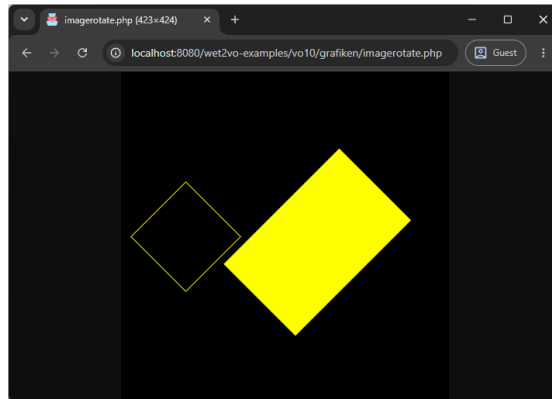


Abbildung 10.11: Das ursprüngliche Bild wurde um 45° rotiert und dadurch von 300×300 Pixel auf 423×424 Pixel vergrößert.

heute aktiv weiterentwickelt. Die aktuelle Version ist 2.0, welche 2017 zusammen mit der Software Adobe Acrobat DC veröffentlicht wurde.

PDF-Dokumente können Text, Grafiken (Vektor- und Rastergrafiken) oder auch andere, multimediale Inhalte wie Videos enthalten. Das PDF-Format bedient sich dabei einer Untermenge der vektorbasierten Seitenbeschreibungssprache PostScript, mit der die Ausgabe von Inhalten layoutgetreu möglich ist. Das Einbetten dieser Inhalte, sowie auch damit verbundener Dinge wie Schriften in eine einzige Datei und die Unabhängigkeit von Ausgabegeräten wie Druckern hat die enorme Verbreitung von PDF zunächst überhaupt ermöglicht und dann in weiterer Folge stark gefördert.

In der modernen Webentwicklung werden PDFs (wie Rechnungen, Tickets oder Reports) kaum noch mühsam über das Platzieren von Elementen an x/y -Koordinaten erstellt. Stattdessen hat sich ein effizienterer Workflow etabliert: Das Dokument wird als reguläres HTML/CSS-Template entworfen und anschließend serverseitig in ein PDF konvertiert.

Der PHP-Core bietet hierfür keine Funktionalitäten. Auch offizielle Erweiterungen existieren nicht, weshalb auf Komponenten zurückgegriffen werden muss. Die derzeit populärste und am aktivsten gepflegte PHP-Bibliothek für diesen Zweck ist **Dompdf**⁸³. Dompdf ist ein (größtenteils) CSS 2.1-konformer HTML-Renderer in PHP, der HTML-Strings einliest und daraus strukturierte PDF-Dateien generiert.

10.4.1 Dompdf initialisieren und Grundlagen

Die Bibliothek wird zunächst wie gewohnt über Composer installiert:

```
1 composer require dompdf/dompdf
```

Die grundlegende Verwendung von Dompdf ist in drei logische Schritte unterteilt: Laden des HTML-Codes, Konfiguration des Seitenformats und Rendern/Ausgeben des PDFs.

Nach dem Einbinden des Autoloaders wird ein Objekt der Klasse Dompdf erzeugt.

⁸³<https://github.com/dompdf/dompdf>

Die Methode `loadHtml()`⁸⁴ nimmt den zu rendernden HTML-String entgegen. Mit `setPaper()`⁸⁵ wird das Format (meist „A4“) und die Ausrichtung („portrait“ für Hochformat oder „landscape“ für Querformat) festgelegt. Der Aufruf von `render()`⁸⁶ startet schließlich den Konvertierungsprozess.

```
1 use Dompdf\Dompdf;
2
3 require __DIR__ . "/vendor/autoload.php";
4
5 $dompdf = new Dompdf();
6
7 $dompdf->loadHtml("<h1>Hello PDF World!</h1><p>My first PDF with Dompdf!</p>");
8
9 $dompdf->setPaper("A4", "portrait");
10
11 $dompdf->render();
```

Ausgabe des PDFs

Nachdem das PDF gerendert wurde, muss es ausgeliefert werden. Dompdf bietet hierfür die Methode `stream($filename, $options)`⁸⁷. Der Parameter `$options` ist ein Array, in dem das Verhalten des Browsers gesteuert werden kann:

- `["Attachment" => false]`: Das PDF wird direkt im Browser (im integrierten PDF-Viewer) *angezeigt* (Inline-Anzeige).
- `["Attachment" => true]`: Der Browser erzwingt einen *Download* der Datei.

Soll das PDF nicht an den Browser gesendet, sondern direkt als Datei auf dem Webserver gespeichert werden, kann der erzeugte Binärcode des PDFs mit der Methode `output()` ausgelesen und über die native PHP-Funktion `file_put_contents()` auf die Festplatte geschrieben werden:

```
1 // Direkt im Browser anzeigen
2 $dompdf->stream("hallo-welt.pdf", ["Attachment" => false]);
3
4 // Am Server speichern (überschreibt bestehende Dateien)
5 file_put_contents(__DIR__ . "/hallo-welt.pdf", $dompdf->output());
```

Anpassen der Optionen

Dompdf erlaubt es, gewisse Standardeinstellungen über ein eigenes Konfigurationsobjekt vom Typ `Dompdf\Options` zu definieren und anzuwenden. Dieses Objekt wird zuerst angelegt und dann bei der Erstellung des `Dompdf`-Objekts als Parameter mitgegeben. Folgende Optionen können u. a. als Methoden gesetzt werden:

- `setDefaultFont()`: Erlaubt die Angabe einer Standardschriftart. Dompdf kennt die Optionen „serif“, „sans-serif“ oder „monospace“. Weitere Schriftarten müssen als TTF-Dateien im Projekt inkludiert und über `setFontDir()` verfügbar gemacht werden.

⁸⁴<https://github.com/dompdf/dompdf/wiki/Usage#loadhtml>

⁸⁵<https://github.com/dompdf/dompdf/wiki/Usage#setpaper>

⁸⁶<https://github.com/dompdf/dompdf/wiki/Usage#render>

⁸⁷<https://github.com/dompdf/dompdf/wiki/Usage#stream>

- `setIsPdfAEnabled()`: Erlaubt die Erzeugung von PDF/A-Dokumenten (schreibgeschützte Archivdokumente).
- `setPdfBackend()`: Ermöglicht die Angabe des Backends, das die Erzeugung des PDFs übernimmt. Der Standard ist „Cpdf“.

Um im Hello World-Beispiel die Standardschriftfamilie auf „Helvetica“ zu setzen, ist folgender Code nötig:

```
1 use Dompdf\Dompdf;
2 use Dompdf\Options;
3
4 $options = new Options();
5 $options->setDefaultFont("Helvetica");
6
7 $dompdf = new Dompdf($options);
8
9 // PDF-Erstellung...
```

Metadaten setzen und Zugriff auf den Canvas

PDF-Dokumente enthalten Metadaten. Dies sind zum einen der Titel, zum anderen aber auch Dinge wie der*die Autor*in, das Thema des PDFs, Schlüsselwörter oder auch die Software, welche das PDF erzeugt hat. Diese Dinge können im `Dompdf`-Objekt über die Methode `addInfo()` gesetzt werden. Im Beispiel sind dies Titel, Autor*in, Betreff, Schlüsselwörter und die erstellende Applikation:

```
1 // PDF rendern
2 $dompdf->render();
3
4 // Metadaten setzen
5 $dompdf->addInfo("Title", "Dompdf Example");
6 $dompdf->addInfo("Author", "Wolfgang Hochleitner");
7 $dompdf->addInfo("Subject", "Dompdf Demo");
8 $dompdf->addInfo("Keywords", "PHP, PDF, Dompdf, Web Technologies");
9 $dompdf->addInfo("Creator", "Dompdf v3.1.5");
```

Diese Aufruf sollten nach `render()` erfolgen, denn sie greifen intern auf den Canvas zu. Dieser ist die Zeichenfläche auf dem mit Low-Level-Zeichenmethoden (ähnlich wie bei der GD Library) gezeichnet und geschrieben werden kann, falls das HTML-Rendering nicht ausreicht. Der Zugriff erfolgt über die Methode `getCanvas()`⁸⁸. Der Canvas ist auch die Schnittstelle zur darunterliegenden PDF-Bibliothek. Die Standardbibliothek „Cpdf“ kann dort über `get_cpdf()` abgefragt werden.

Fertiges Hallo Welt PDF

Das fertige Hallo Welt PDF mit serifenloser Standardschriftart (und Metadaten) ist in Abbildung 10.12 ersichtlich.

10.4.2 PDFs mit einer Template-Engine erstellen

Der Workflow, HTML-Strings direkt im PHP-Code zusammenzubauen, wird bei komplexen Dokumenten (wie Rechnungen mit Tabellen) schnell unübersichtlich. Eine mo-

⁸⁸ <https://github.com/dompdf/dompdf/wiki/Usage#canvas>

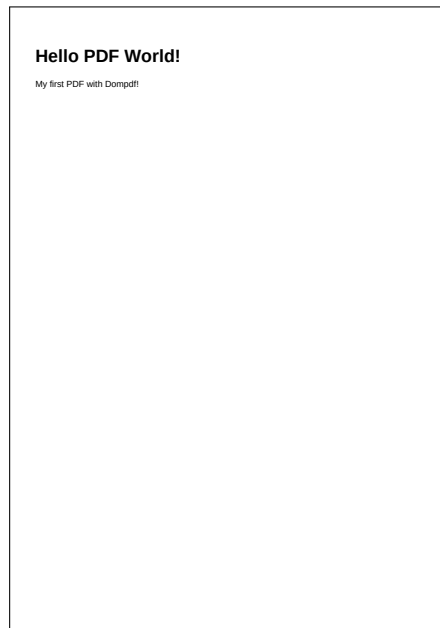


Abbildung 10.12: Das fertige Hallo Welt-PDF.

derne und professionelle Architektur trennt – wie schon in Vorlesung 6 erwähnt – aber ohnehin zwischen Logik und Präsentation. Dies trifft auch auf die PDF-Erstellung zu. Die Template-Engine *Latte* kann einfach mit Dompdf kombiniert werden.

1. In **PHP** werden die Daten zur Verfügung gestellt (als Variablen oder z. B. aus einer Datenbank).
2. **Latte** fügt die Daten in ein HTML-Template ein und generiert das fertige HTML als String.
3. **Dompdf** wandelt diesen String in ein PDF um.

Das Latte-Template

Im folgenden wird eine Übersicht über die Web Technologies-Vorlesung als Tabelle gegeben. Neben einer Überschrift wird auch noch ein Bild eingebunden. Bei diesem ist zu beachten, dass der Pfad relativ zur PHP-Datei gesetzt wird, welche die PDF-Erzeugung aufruft.

Zunächst wird eine Datei `webtechnologies.latte` im Template-Verzeichnis angelegt. Sie enthält die CSS-Formatierung im `<style>`-Tag, bindet ein lokales Bild (`images/wet2icon.png`) ein und iteriert über ein Array, um eine Tabelle aufzubauen. Dompdf unterstützt die meisten CSS 2.1-Eigenschaften zur korrekten Darstellung.

```
1 <!DOCTYPE html>
2 <html lang="de">
3   <head>
4     <meta charset="UTF-8">
5     <title>{$title}</title>
6   <style>
7     body {
```

```

 8     font-family: Helvetica, Arial, sans-serif;
 9     }
10     table {
11         border-collapse: collapse;
12         width: 100%;
13         margin-top: 20px;
14     }
15     th, td {
16         border: 1px solid #333;
17         padding: 8px;
18         text-align: left;
19     }
20     th {
21         background-color: #f2f2f2;
22     }
23 </style>
24 </head>
25 <body>
26     <h1>{$title}</h1>
27
28     <div class="image">
29         
30     </div>
31
32     <p>Übersicht der Vorlesungsinhalte:</p>
33
34     <table>
35         <tr>
36             <th>Einheit</th>
37             <th>Thema</th>
38         </tr>
39         <tr n:foreach="$lectures as $unit => $topic">
40             <td>{$unit}</td>
41             <td>{$topic}</td>
42         </tr>
43     </table>
44 </body>
45 </html>

```

PHP-Logik und Konvertierung

In der PHP-Datei wird nun Latte initialisiert und die Datenstruktur aufgebaut. Anstatt das Template mit `render()` direkt anzuzeigen, wird die Ausgabe über die Methode `renderToString()` in einen String geschrieben. Dieser wird dann der Methode `loadHtml()` von `Dompdf` übergeben.

Eine Besonderheit beim Verwenden von Bildern oder externen Stylesheets in `Dompdf` ist der Dateisystemzugriff. Standardmäßig blockiert `Dompdf` das Laden lokaler Dateien, um *Directory Traversal*-Angriffe zu verhindern. Um das Bild im Template korrekt rendern zu können, muss dem `Dompdf`-Objekt über die Klasse `Options` explizit ein Arbeitsverzeichnis zugewiesen werden. Der Aufruf `$options->setChroot([__DIR__]);` berechtigt den Renderer, auf das aktuelle Verzeichnis und dessen Unterverzeichnisse (wie `images`) zuzugreifen.

Nach dem Konvertieren des Strings in das PDF-Format werden über `addInfo()`

noch Metadaten gesetzt. Abschließend wird das Dokument sowohl an den Browser zur Inline-Anzeige gesendet (`stream()`) als auch dauerhaft auf dem Server gespeichert.

```

1 // Latte initialisieren
2 $latte = new Engine();
3 $latte->setLoader(new FileLoader(__DIR__ . "/templates"));
4 $latte->setCacheDirectory(__DIR__ . "/cache");
5
6 // Daten fürs Template
7 $data = [
8     "title" => "Willkommen zu Web Technologies",
9     "lectures" => [
10         "Vorlesung 1" => "Grundlagen der Kommunikation im Web",
11         "Vorlesung 2" => "PHP-Grundlagen Teil 1",
12         "Vorlesung 3" => "PHP-Grundlagen Teil 2 und Regular Expressions",
13         "Vorlesung 4" => "Objektorientiertes PHP",
14         "Vorlesung 5" => "Komponenten und Standards",
15         "Vorlesung 6" => "Templates und Routing",
16         "Vorlesung 7" => "Formularverarbeitung",
17         "Vorlesung 8" => "Sessions und Authentifizierung",
18         "Vorlesung 9" => "Datum, Zeit und Internationalisierung",
19         "Vorlesung 10" => "Medienverarbeitung",
20         "Vorlesung 11" => "REST APIs & AJAX",
21         "Vorlesung 12" => "Microframeworks und Testing",
22     ],
23 ];
24
25 // Template in einen String rendern
26 $html = $latte->renderToString("webtechnologies.latte", $data);
27
28 // Dompdf mit Optionen erzeugen
29 $options = new Options();
30 $options->setChroot(__DIR__);
31
32 $dompdf = new Dompdf($options);
33 $dompdf->loadHtml($html);
34 $dompdf->setPaper("A4", "portrait");
35 $dompdf->render();
36
37 // Metadaten hinzufügen
38 $dompdf->addInfo("Title", "Dompdf & Latte Example");
39 $dompdf->addInfo("Author", "Wolfgang Hochleitner");
40 $dompdf->addInfo("Subject", "Dompdf & Latte Demo");
41 $dompdf->addInfo("Keywords", "PHP, PDF, Dompdf, Latte, Web Technologies");
42 $dompdf->addInfo("Creator", "Dompdf v3.1.5");
43
44 // PDF anzeigen und speichern
45 $dompdf->stream("pdf-latte-webtechnologies.pdf", ["Attachment" => false]);
46 file_put_contents(__DIR__ . "/pdf-latte-webtechnologies.pdf", $dompdf->output());

```

Fertiges PDF mit Vorlesungsinhalten

Das PDF mit den Inhalten der zwölf Vorlesungen und einer Grafik, welches unter Zuhilfenahme von Latte erstellt wurde, ist in Abbildung 10.13 ersichtlich.

Willkommen zu Web Technologies



Übersicht der Vorlesungsinhalte:

Einheit	Thema
Vorlesung 1	Grundlagen der Kommunikation im Web
Vorlesung 2	PHP-Grundlagen Teil 1
Vorlesung 3	PHP-Grundlagen Teil 2 und Regular Expressions
Vorlesung 4	Objektorientiertes PHP
Vorlesung 5	Komponenten und Standards
Vorlesung 6	Templates und Routing
Vorlesung 7	Formularverarbeitung
Vorlesung 8	Sessions und Authentifizierung
Vorlesung 9	Datum, Zeit und Internationalisierung
Vorlesung 10	Medienverarbeitung
Vorlesung 11	REST APIs & AJAX
Vorlesung 12	Microframeworks und Testing

Abbildung 10.13: Das fertige PDF Dokument. Die Tabelle wurde in Latte erzeugt und gerendert, die Grafik ist aus dem Dateisystem eingebunden.

10.5 Weiterführende Literatur

Bei den in PHP verfügbaren XML-Werkzeugen hat sich in den letzten Jahren viel getan. Leider existiert nur wenig brauchbare Literatur zu dem Thema. [41] behandelt lediglich die Erweiterungen XML Parser sowie DOM und SimpleXML. XMLReader fehlt und auch auf das Thema XML schreiben wird nicht eingegangen.

Im deutschsprachigen Bereich behandelt [49] das Thema XML mit allen Erweiterungen sehr ausführlich, [44] gibt einen nicht ganz so detaillierten aber dennoch ausreichenden Überblick. JSON wird in der Literatur nur am Rande angesprochen. [44, 49] zeigen die Funktionen auf wenigen Seiten.

Da die GD Graphics Library zur PHP-Standarddistribution gehört, ist das Thema in der Literatur entsprechend gut abgedeckt. [41] widmet sich ausführlich dem Thema und wartet mit vielen Beispielen und Erklärungen auf, die auf einem aktuellen Stand sind. Auch die beiden deutschsprachigen Werke [44, 49] warten mit guten Kapiteln zum Thema GDlib auf.

Die Erzeugung von PDFs über den Workflow der HTML-zu-PDF-Konvertierung wird am besten über die Online-Dokumentation von Dompdf⁸⁹ studiert, da Printliteratur bei der Vorstellung externer Bibliotheken naturgemäß oft hinterherhinkt. [41, 49] zeigen PDF-Generierung zwar ausführlich, fokussieren sich aber auf Bibliotheken wie TCPDF, die mittlerweile als veraltet anzusehen sind.

Schließlich sei noch auf [42] verwiesen. Die offizielle Dokumentation enthält eigene Kapitel zu den nativen Themen XML, JSON und GD, in denen alle Funktionen mit Beispielen illustriert sind.

⁸⁹<https://github.com/dompdf/dompdf/wiki>

Vorlesung 11

REST APIs & AJAX

11.1 REST APIs

In modernen Webapplikationen finden sich fast überall *Webservices*. Diese sind Schnittstellen, mit denen eine Maschine-zu-Maschine-Kommunikation stattfinden kann. An einem Ende – in der Regel auf einem Webserver – werden strukturiert Daten zur Verfügung gestellt, die dann von einer anderen Webapplikation (am selben oder auch auf einem anderen Server) konsumiert, d. h. weiterverarbeitet werden.

Konkret sichtbar wird dies, wenn eine Webapplikation eine strikte Trennung zwischen Frontend und Backend vornimmt, indem etwa unterschiedliche Technologien und Frameworks zum Einsatz kommen. So kümmert sich ein in PHP geschriebenes Backend etwa um die Verbindung zur Datenbank, der Speicherung, Abfrage und strukturierten Aufbereitung von Daten. Diese werden dann über einen Webservice, sprich eine definierte Schnittstelle zur Verfügung gestellt. Das Frontend – z. B. in Form eines JavaScript Frontend Frameworks wie Vue.js, Angular oder React – verbindet sich zu dieser Schnittstelle, und verwendet diese Daten, um damit das User-Interface der Webseite zu befüllen und die Informationen ansprechend und nutzer*innenfreundlich aufzubereiten.

Der Webservice arbeitet dabei plattformunabhängig, denn er gibt nur Daten in einem maschinenlesbaren Format wie JSON, XML oder auch einfachem Text aus. Diese Daten können von beliebigen anderen Applikationen konsumiert werden.

Bekannte Webservice-Technologien, -Protokolle bzw. auch -Architekturen sind XML-RPC¹, JSON-RPC², SOAP³ oder REST. Um letzteres dreht sich dieser Abschnitt.

11.1.1 Allgemeines zu REST

REST, kurz für *Representational State Transfer*, ist ein Paradigma oder auch Softwarearchitekturstil zur Erstellung von Webservices. Vergleicht man die Herangehensweise mit den zuvor aufgezählten Technologien, so fällt REST durch seine Einfachheit und das Verwenden bestehender Technologien auf. Während Technologien wie XML-RPC oder JSON-RPC es erlauben, Methoden einer über einen URI verfügbaren Webapplikation über das Netzwerk aufzurufen (RPC bedeutet „Remote Procedure Call“) und dadurch

¹<https://xmlrpc.com/>

²<https://www.jsonrpc.org/>

³<http://www.w3.org/TR/SOAP>

Daten zu erhalten geht REST hier einen anderen Weg: Es werden einfache HTTP- oder HTTPS-Requests mit den Methoden GET, POST, PUT, DELETE und HEAD an einen eindeutigen URI abgesetzt. Diese Aufrufe werden am Server verarbeitet und liefern die gewünschten Daten. Wie bei HTTP üblich, ist die Kommunikation damit beendet. Die Anfrage und die damit erhaltene Antwort muss so vollständig sein, dass sie für den*die Empfänger*in verständlich ist. Dies ist in den sechs REST-Prinzipien auch definiert.

Sechs Prinzipien einer REST-Architektur

REST ist keine standardisierte Technologie, sondern ein Architekturstil. Um zu definieren, wann ein Webservice, in weiterer Folge auch Schnittstelle oder API (Application Programming Interface) genannt, als REST API bezeichnet werden kann, wurden sechs Prinzipien festgelegt.

Client-Server: REST-Architekturen folgen dem Client-Server-Prinzip, das eine Trennung von Aufgaben vorsieht. Wie bereits in der Einleitung erwähnt, ist der Server für die Bereitstellung eines Dienstes und die Aufbereitung der Daten verantwortlich (z. B. Backend), welche von einem Client konsumiert und verarbeitet werden (z. B. Frontend).

Zustandslosigkeit: Da REST HTTP für die Anfragen und Antworten verwendet, zählt es auch zu den zustandslosen Protokollen. Daher muss die Antwort alle Informationen enthalten, um die Anfrage zu beantworten, da keinerlei Zustand (z. B. in Form von Sessions) gespeichert wird. Somit ist jede Anfrage an eine REST API unabhängig von vorangehenden oder nachfolgenden Anfragen.

Caching: Antworten eines REST-Aufrufs sollen zwischengespeichert werden können, sollen dies jedoch über die Header der Antwort kundtun.

Einheitliche Schnittstelle: Bei der Gestaltung der REST API gelten vier zentrale Anforderungen an die Schnittstelle:

- **Adressierbarkeit von Ressourcen:** Jede über die API zur Verfügung gestellte Ressource muss eindeutig identifizierbar sein. Dies geschieht über einen eindeutigen URI.
- **Repräsentationen zur Veränderung von Ressourcen:** Eine REST-API kann je nach Anforderung des Clients die Daten, die unter einer Ressource ausgeliefert werden, in verschiedenen Formaten (z. B. in JSON oder XML) zurückliefern.
- **Selbstbeschreibende Nachrichten:** Jede REST-Nachricht (die Antwort einer REST API) soll selbstbeschreibend sein, sodass klar ist, wie sie verarbeitet werden kann. Die nötigen Informationen dafür liefern die Metadaten der Antwort selbst: Der **Content-Type-Header** gibt das Inhaltsformat an (z. B. `application/json`), der Statuscode das Ergebnis der Anfrage (z. B. 200 oder 404). Der Client benötigt also keine zusätzlichen, externen Informationen, um eine Antwort korrekt interpretieren zu können.
- **Hypermedia as the Engine of Application State (HATEOAS):** Der REST-Client muss eine REST-Schnittstelle nur anhand der URLs, die vom Server zur

Verfügung gestellt werden, navigieren können. So kann in einer Antwort wiederum ein Link auf die nächste aufzurufende Ressource enthalten sein.

Mehrschichtiges System: Für den Client ist es nicht ersichtlich, ob er direkt zum Endserver verbunden ist, oder ob noch Systeme zwischengeschaltet sind. Die können etwa ein Load Balancer oder ein Proxy sein. Dies garantiert, dass das REST-System skalierbar bleibt.

Code on Demand (optional): Optional können REST-Schnittstellen auch ausführbaren Code wie z. B. JavaScript-Skripte mit übertragen, wenn dies notwendig ist.

11.1.2 Aufbau einer REST API

Eine REST-Schnittstelle ist also wie soeben definiert eine konkrete URI, die unter den verschiedenen HTTP-Methoden [7, Abschnitt 9] – allen voran GET, POST, PUT, DELETE und HEAD – Daten zur Verfügung stellt. Dabei haben die Methoden eine entsprechende semantische Bedeutung, die bei der Umsetzung beachtet werden sollte:

- **GET** ohne Angabe einer ID: Liefert eine Liste von Ressourcen (z. B. alle User*innen: GET `https://api.foo.com/users`).
- **GET** mit Angabe einer ID: Liefert eine spezifische Ressource mit einer ID (z. B. den*die User*in mit der bestimmten ID: GET `https://api.foo.com/users/3`).
- **POST**: Legt eine neue Ressource an (z. B. eine*n neue*n User*in: POST `https://api.foo.com/users`).
- **PUT** mit Angabe einer ID: Aktualisiert eine bestehende Ressource mit einer ID (z. B. aktualisiert den*die User*in mit dieser ID: PUT `https://api.foo.com/users/3`).
- **DELETE** mit Angabe einer ID: Löscht die Ressource mit einer ID (z. B. den*die User*in mit der ID: DELETE `https://api.foo.com/users/3`).
- **HEAD** mit oder ohne ID: Liefert Header-Informationen über eine Liste von Ressourcen oder eine einzige Ressource (z. B. die Größe der abzufragenden User*innenliste: HEAD `https://api.foo.com/users`).

Darüber hinaus werden auch die Methoden **OPTIONS** (Überprüfung, welche Methoden bei einer Ressource akzeptiert werden), **TRACE** und **CONNECT** unterstützt. Diese werden hier jedoch nicht weiter besprochen. Generell beziehen sich die in den weiteren Abschnitten angeführten Beispiele primär auf die Methoden **GET** und **POST**.

Datenformat bekanntgeben

Client und Server müssen einander mitteilen, welche Formate sie verarbeiten können (auf der Seite des Clients, denn dieser erhält die Antwort) und welche Formate tatsächlich geliefert werden (auf der Seite des Servers, denn dieser generiert die Antwort).

Dies geschieht für den Client bei der Anfrage über den HTTP-Header **Accept**, mit dem er mitteilt, dass er etwa Antworten in Plain-Text und JSON verarbeiten kann: **Accept: text/plain,application/json**.

Der Server schickt in seiner Antwort den Header **Content-Type** mit, in dem er den tatsächlich ausgelieferten Typ bekanntgibt: **Content-Type: application/json**.

Statuscodes einer REST-Antwort

In den Headerdaten der Antwort auf eine REST-Anfrage wird wie bei HTTP ein Statuscode mitgeschickt. Anhand dieses Codes wird dem Client mitgeteilt, ob die Anfrage erfolgreich war oder nicht. Hier können prinzipiell alle verfügbaren HTTP-Statuscodes [7, Abschnitt 15] verwendet werden, die gebräuchlichsten sind aber die folgenden:

- **200 OK:** Die Anfrage wurde erfolgreich ausgeführt. Ein Ergebnis wird mitgeliefert. Wird bei GET-Anfragen zurückgegeben.
- **201 CREATED:** Die Anfrage wurde erfolgreich ausgeführt und eine Ressource angelegt. Wird bei POST-Anfragen zurückgegeben.
- **204 NO CONTENT:** Die Anfrage wurde ausgeführt, es gibt jedoch keinen Inhalt, der zurückgegeben werden kann. Wird meist bei DELETE-Operationen verwendet, nachdem eine Ressource gelöscht wurde.
- **400 BAD REQUEST:** Genereller Fehlercode, wenn in der Anfrage ein Syntaxfehler enthalten ist und der Server diese nicht versteht (z. B. ID des*der User*in ist keine Zahl).
- **401 UNAUTHORIZED:** Authentifizierungsfehler. Die Ressource kann nur mit entsprechenden Credentials (d. h. einer Anmeldung) abgefragt werden.
- **403 FORBIDDEN:** Autorisierungsfehler. Der Client ist zwar angemeldet (im Gegensatz zur 401), besitzt aber nicht die Rechte, um diese Ressource abzufragen. Etwa wenn Admin-Rechte benötigt werden, man aber nur als User*in angemeldet ist.
- **404 NOT FOUND:** Die Ressource kann nicht gefunden werden. Entweder der URI existiert nicht oder die Ressource an sich (z. B. ein*e entsprechende*r User*in) ist nicht vorhanden.
- **406 NOT ACCEPTABLE:** Der Server kann keine Antwort erzeugen, die dem vom Client angeforderten Format entspricht.
- **500 INTERNAL SERVER ERROR:** Ein generischer, serverseitiger Fehler. Die Anfrage konnte nicht ausgeführt werden.
- **501 NOT IMPLEMENTED:** Die HTTP Methode für diesen URI ist nicht implementiert. Wenn etwa eine POST-Anfrage an eine URI geschickt wird, die aber nur GET implementiert.

Datenformate einer REST-Antwort

Die Antwort einer erfolgreichen Anfrage an eine REST API kann in verschiedenen *Datenformaten* erfolgen. Als Beispiel dient die Anfrage an die fiktive API `GET https://api.foo.com/users`, die alle User in einem System zurückgibt. Dies sind in diesem Beispiel jeweils drei Einträge.

XML: *XML* ist eines der am längsten verwendeten Markup-Formate, die zum REST-Datenaustausch zum Einsatz kommen. Es ist außerdem das namensgebende Format in *AJAX* (siehe 11.2) und ermöglicht es, komplexe Strukturen zurückzugeben und diese auch menschenlesbar aufzubereiten. Um mit PHP eine XML-Antwort zu erzeugen, können die in Vorlesung 10 demonstrierten Bibliotheken *XMLWriter* und *DOM* zum Einsatz kommen. Die Antwort kann wie folgt aussehen:

```
1 <users size="3">
2   <link rel="self" href="/users" />
3   <user id="1">
4     <username>jdoe</username>
5     <realname>John Doe</realname>
6     <link rel="self" href="/users/1" />
7   </user>
8   <user id="2">
9     <username>jane789</username>
10    <realname>Jane Doe</realname>
11    <link rel="self" href="/users/2" />
12  </user>
13  <user id="3">
14    <username>jimmy</username>
15    <realname>Jim Doe</realname>
16    <link rel="self" href="/users/3" />
17  </user>
18 </users>
```

Der Tag `<users>` leitet die Antwort ein und teilt in einem Attribut die Größe mit. Danach folgen `<user>`-Elemente, das Attribut `id` teilt die ID mit. Innerhalb des Elements werden die relevanten Informationen zurückgegeben. Die `<link>`-Elemente erfüllen das HATEOAS-Prinzip, damit aus der Antwort der URI für eine Detail-Abfrage gewonnen werden kann.

JSON: *JSON* ist ein einfaches, menschenlesbares Datenaustauschformat. Es wurde in der Vorlesung zu Web Fundamentals bereits im Detail besprochen. Der Vorteil von JSON gegenüber XML ist, dass JSON-Dokumente in der Regel kürzer und präziser sind, durch die Einteilung in Objekte und Arrays lassen sich Daten, die strukturiert vorliegen, sehr gut damit abbilden. JSON hat sich bei vielen APIs als Standard etabliert.

Eine Antwort in JSON kann wie folgt aussehen:

```
1 {
2   "size": 3,
3   "link": "/users",
4   "users": [
5     {
6       "id": 1,
7       "username": "jdoe",
8       "realname": "John Doe",
9       "link": "/users/1"
10    },
11    {
12      "id": 2,
13      "username": "jane789",
14      "realname": "Jane Doe",
15      "link": "/users/2"
16    },
17    {
18      "id": 3,
19      "username": "jimmy",
20      "realname": "Jim Doe",
21      "link": "/users/3"
22    }
23  ]
}
```

24 }

HTML: Die große Stärke von REST ist die Unabhängigkeit zwischen Ausgabeformat und darunterliegenden Daten. So kann auch HTML zurückgegeben werden, wenn dies für eine Applikation Sinn macht und der Client dies mit seinem **Accept**-Header auch als mögliches Format indiziert. Vor allem in Kombination mit AJAX (Abschnitt 11.2) ist dies nicht ungewöhnlich. Der Vorteil dabei ist, dass das HTML-Fragment schnell und ohne weitere Konvertierung etwa im Frontend eingebaut werden kann. Der Nachteil dabei ist, dass die verwendeten HTML-Elemente (und eventuell Klassen, IDs etc.) nicht für jedes Frontend geeignet sind. Dadurch leidet die Austauschbarkeit.

Eine HTML-Antwort kann etwa so aussehen. Da hier die Ausgabe direkt verarbeitet wird, wird auf die Links auf die jeweiligen einzelnen Userdaten (im Sinne von HATEOAS) verzichtet.

```

1 <div>
2   <div class="user" id="user1">
3     <p class="username">jdoe</p>
4     <p class="realname">John Doe</p>
5   </div>
6   <div class="user" id="user2">
7     <p class="username">jane789</p>
8     <p class="realname">Jane Doe</p>
9   </div>
10  <div class="user" id="user3">
11    <p class="username">jimmy</p>
12    <p class="realname">Jim Doe</p>
13  </div>
14 </div>

```

Text: Eine weitere Möglichkeit, Daten vom Server zu erhalten, ist einfacher *Text* ohne jegliches Markup. In diesem Fall wird der Text oft zeilenweise zurückgegeben, um ihn dann am Zeilenumbruch wieder trennen zu können. Ebenfalls möglich sind CSV-Formate, die nicht `\n` oder `\r\n`, sondern ein anderes Zeichen zum Trennen einzelner Daten verwenden.

Die User*innenliste kann – hier als einfaches CSV-Format – wie folgt aussehen:

```

1 1;jdoe;John Doe;/users/1
2 2;jane789;Jane Doe;/users/2
3 3;jimmy;Jim Doe;/users/3

```

11.1.3 Eine REST API in PHP implementieren

Das Grundprinzip einer REST API besteht also darin, Ressourcen unter einem eindeutigen URI in Kombination mit einer HTTP-Methode (GET, POST etc.) verfügbar zu machen und diese in einem bestimmten Format (oder mehreren) zurückzuliefern. Dies soll nun mit dem User*innenbeispiel, das bereits in Abschnitt 11.1.2 thematisiert wurde, praktisch umgesetzt werden. Um HTTP-Methode und URI einfach abfragen zu können, bietet sich ein Router an. In diesem Fall kommt **fhooe/router** unter Zuhilfenahme des **fhooe/router-skeleton** zum Einsatz. Da dieser Router sehr einfach gehalten ist,

werden jedoch nur die Methoden GET und POST demonstriert. Vollständige Router, wie in Abschnitt 6.2.4 gezeigt, unterstützen auch weitere Methoden. Dies wird auch in Vorlesung 12 anhand des Slim Frameworks demonstriert.

Als Ausgabeformat wird JSON (MIME-Type `application/json`) verwendet.

Definitionen der Routen

Zunächst werden in der Hauptdatei `index.php` (im Verzeichnis `public`) die Routen angelegt. Neben der Hauptroute GET `/`, welche der Startpunkt des Ablaufs ist, werden vier weitere Routen angelegt:

- GET `/users`: Listet alle User*innen im System auf.
- GET `/users/{id}`: Wird ein GET-Parameter `id` nach `/users` angegeben, wird nur der*die User*in mit dieser ID zurückgegeben.
- POST `/users`: Fügt eine*n neue*n User*in hinzu.
- GET `/addUser`: Zeigt ein Formular zum Hinzufügen eines*einer User*in an und ruft dann POST `/users` auf, um die eingegeben Daten an die API zum Eintragen zu übergeben.

Die Definitionen hierfür lauten wie folgt:

```
1 $router->get("/users", function () {
2     $userManager = new UserManager();
3     $userManager->listUsers();
4 });
5
6 $router->get("/users/{id}", function ($id) {
7     $userManager = new UserManager();
8     $userManager->listUser($id);
9 });
10
11 $router->post("/users", function () {
12     $userManager = new UserManager();
13     $userManager->addUser();
14 });
15
16 $router->get("/adduser", function () use ($latte) {
17     $latte->render("adduser.latte");
18 });
```

GET `/users`: Die Verarbeitung und Generierung der Ausgabe erfolgt in einem Objekt der Klasse `UserManager`. Durch Aufruf von `listUsers()` werden alle gespeicherten User*innen ausgegeben.

GET `/users/{id}`: Durch Angabe einer ID nach `/users` wird beim `UserManager`-Objekt `listUser($id)` aufgerufen, um den*die User*in mit dieser ID abzufragen und auszugeben. Das Error-Handling (z. B. bei nicht existenter oder nicht-numerischer ID) übernimmt die Klasse selbst.

POST `/users`: Auch hier wird ein Objekt der Klasse `UserManager` angelegt. In der Methode `addUser()` wird ein*e neue*r User*in hinzugefügt. Die Daten kommen dabei

aus der Payload des POST-Aufrufs und müssen unter dem Schlüssel `username` (in PHP als `$_POST["username"]` verfügbar) und `realname` (in PHP als `$_POST["realname"]` verfügbar) mitgeschickt werden. Dies geschieht hier über ein Formular unter der Route `GET /adduser`.

`GET /adduser`: Um POST-Daten erzeugen zu können, wird ein Formular angezeigt, welches User*innenname und echten Namen erfasst und danach die Router `POST /users` aufruft. Die Formularfelder haben die `name`-Attribute `username` und `realname`, wodurch in der POST-Payload die korrekten Einträge entstehen.

Alle User*innen abfragen: `listUsers()`

Die Methode `listUsers()` der Klasse `UserManager` gibt alle User*innen in einer JSON-Struktur zurück.

```

1 public function listUsers(): void
2 {
3     $query = "SELECT * FROM rest_api_example.user";
4
5     $statement = $this->pdo->query($query);
6     $users = $statement->fetchAll();
7
8     $jsonResponse = [
9         "size" => count($users),
10        "link" => $this->link
11    ];
12
13    foreach ($users as $user) {
14        $newUser = $user;
15        $newUser["link"] = "$this->link/{$user["id"]}";
16        $jsonResponse["users"][] = $newUser;
17    }
18
19    $this->showResponse(200, $jsonResponse);
20 }

```

Dazu werden zunächst mit einer Datenbankabfrage alle Zeilen und Spalten der Tabelle `user` abgefragt. Die Tabelle hat die Spalten `id` (der Primärschlüssel), `username` und `realname`. Das Ergebnis wird als assoziatives Array abgefragt (Option beim PDO-Objekt `PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC`).

Dann wird die JSON-Antwort erzeugt. Dazu werden in die Variable `$jsonResponse` die Einträge `size` und `link` hinzugefügt. Ersterer zählt die Anzahl der abgefragten Datensätze und setzt sie als Wert. Der Link (hier `/users`) wird in der Eigenschaft `$this->link` gespeichert und könnte auch direkt vom Router abgefragt werden (wenn dieser es unterstützt).

Schließlich wird über das Ergebnisarray der Datenbankabfrage iteriert. Jeder Eintrag mit den Positionen `id`, `username` und `realname` wird zunächst in der Variable `$newUser` gespeichert. Dann wird noch eine Position `link` für diesen einen User hinzugefügt. Ihr Wert besteht aus der Link-URL und der jeweiligen ID.

Ist das Array fertig konstruiert, wird `showResponse()` aufgerufen. Die Methode bekommt den Statuscode 200 für eine erfolgreiche Abfrage und das soeben erstellte Array `$jsonResponse` übergeben.

Ausgabe erzeugen: `showResponse()`

Diese Methode kommt immer zum Einsatz, wenn die REST API eine Antwort zurückgibt. Sie setzt den gewünschten HTTP- bzw. REST-Statuscode, den `Content-Type`-Header und gibt eine JSON-Struktur zurück, wenn es gewünscht ist.

```

1 private function showResponse(int $statusCode, ?array $content = null): void
2 {
3     $acceptHeader = $_SERVER["HTTP_ACCEPT"] ?? "";
4
5     if (str_contains($acceptHeader, $this->contentType) || str_contains($acceptHeader,
6         "*/")) {
7         http_response_code($statusCode);
8         header("Content-Type: " . $this->contentType);
9         if (isset($content)) {
10             echo json_encode($content, JSON_PRETTY_PRINT | JSON_UNESCAPED_SLASHES |
11                 JSON_UNESCAPED_UNICODE);
12         }
13     } else {
14         http_response_code(406);
15     }
16 }

```

Zunächst wird der HTTP-Header **Accept** abgefragt – abgesichert über den Null-Coalescing-Operator `??`, falls dieser Header gar nicht gesendet wird – und überprüft, ob der hier zurückgegebene Typ `application/json` (gesetzt im Konstruktor der Klasse in die Eigenschaft `$this->contentType`) darin enthalten ist. Ist dies nicht der Fall, weil der Client etwa gezielt XML über **Accept: application/xml** anfordert, dann wird der Statuscode 406 zurückgegeben. Dieser teilt mit, dass das gewünschte Format nicht erzeugt werden kann. Diese Prüfung ist allerdings sehr streng, weshalb zusätzlich geprüft wird, ob der **Accept**-Header den Wildcard-Wert `*/` enthält. Dieser bedeutet, dass der Client jedes beliebige Format akzeptiert – in diesem Fall wird die Antwort ebenfalls erzeugt. Das deckt gängige Clients wie `curl` oder `fetch()`-Aufrufe ohne explizit gesetzten **Accept**-Header ab (die `*/` senden) sowie Webbrowser, deren **Accept**-Header `*/` am Ende ihrer Präferenzliste enthält. Sind also JSON oder die Wildcard in der Header-Information enthalten, so wird die Ausgabe erzeugt.

Zunächst wird der übergebene Statuscode gesetzt, dann der **Content-Type**-Header auf `application/json`, denn dies ist der Antwort-Typ. Wurde nun ein Array im Parameter `$content` übergeben, wird dieses mit der Funktion `json_encode()` in eine JSON-Datenstruktur umgewandelt und mit `echo` ausgegeben.

Im Falle von `GET /users` entsteht dies bei drei hinterlegten User*innen. Dies deckt sich mit dem Beispiel aus Abschnitt 11.1.2.

```

1 {
2     "size": 3,
3     "link": "/users",
4     "users": [
5         {
6             "id": 1,
7             "username": "jdoe",
8             "realname": "John Doe",
9             "link": "/users/1"
10        },
11    ]
12 }

```

```

12     "id": 2,
13     "username": "jane789",
14     "realname": "Jane Doe",
15     "link": "/users/2"
16 },
17 {
18     "id": 3,
19     "username": "jimmy",
20     "realname": "Jim Doe",
21     "link": "/users/3"
22 }
23 ]
24 }

```

Anmerkung: Den `Accept`-Header mit `str_contains()` auf Inhalte zu vergleichen, ist eine schnelle und oft zu einfache Lösung. Mit `willdurand/negotiation`⁴ existiert etwa eine Komponente, die eine vollständige und professionelle Lösung implementiert.

Eine*n User*in abfragen: `listUser()`

Die Methode `listUser($id)` der Klasse `UserManager` gibt eine*n User*in mit gegebener ID in einer JSON-Struktur zurück.

```

1 public function listUser(int|string $id): void
2 {
3     if (is_string($id) && !is_numeric($id)) {
4         $this->showResponse(400, ["error" => "The given ID has to be a number"]);
5         return;
6     }
7
8     $query = "SELECT * FROM rest_api_example.user WHERE id = :id";
9
10    $statement = $this->pdo->prepare($query);
11    $params = [":id" => $id];
12    $statement->execute($params);
13    $users = $statement->fetchAll();
14
15    if (count($users) === 1) {
16        $jsonResponse = $users[0];
17        $jsonResponse["link"] = "$this->link/$id";
18        $this->showResponse(200, $jsonResponse);
19    } else {
20        $this->showResponse(404, ["error" => "No entry with ID $id found"]);
21    }
22 }

```

Zunächst wird überprüft, ob der Parameter – der eine Ganzzahl oder ein String (so kommt der Parameter aus der URL vom Router) sein kann – eine Zahl ist. Handelt es sich etwa um Buchstaben, wird eine Fehlermeldung mit dem Fehlercode 400 (Bad Request) und einer Fehlermeldung zurückgegeben. Handelt es sich um eine Ganzzahl oder eine Zahl im String wird fortgefahren.

Das `SELECT`-Statement ist ähnlich zu dem in `listUsers()`, nur wird auf die übergebene ID eingeschränkt. Das Ergebnis sollte daher immer ein*e User*in sein. Alternativ keine*r, wenn die ID nicht existiert.

⁴<https://packagist.org/packages/willdurand/negotiation>

Nach der Abfrage wird das Abfrageergebnis direkt in die Variable `$jsonResponse` gespeichert. Damit existieren die Positionen `id`, `username` und `realname`. Es wird noch `link` hinzugefügt und danach `showResponse()` aufgerufen.

Dies generiert diese Struktur für den User mit der ID 1:

```
1 {  
2   "id": 1,  
3   "username": "jdoe",  
4   "realname": "John Doe",  
5   "link": "/users/1"  
6 }
```

Ist die ID in der Datenbank nicht vorhanden, besteht das Abfrageergebnis nicht aus genau einem Eintrag (sondern aus null). In diesem Fall wird der Statuscode 404 zurückgegeben, weil die angeforderte Ressource nicht existiert.

Eine*n User*in hinzufügen: `addUser()`

Die Methode `addUser()` der Klasse `UserManager` erstellt eine*n User*in in der Datenbank.

```
1 public function addUser(): void  
2 {  
3   $query = "INSERT into rest_api_example.user SET username = :username, realname = :  
4     realname";  
5  
6   $statement = $this->pdo->prepare($query);  
7   $params = [":username" => $_POST["username"], ":realname" => $_POST["realname"]];  
8   $success = $statement->execute($params);  
9  
10  if ($success) {  
11    $this->showResponse(201, [  
12      "message" => "User successfully created",  
13      "link" => "$this->link/{$this->pdo->lastInsertId()}",  
14    ]);  
15  } else {  
16    $this->showResponse(500, ["error" => "An error occurred while creating the user"]);  
17  }
```

Zunächst wird ein Prepared Statement mit Username und Realname vorbereitet, mit den Parametern aus `$_POST` befüllt und ausgeführt. Das Ergebnis (erfolgreich oder nicht) wird in der Variable `$success` gespeichert. War das Einfügen erfolgreich, wird der Statuscode 201 für „Created“ zurückgegeben. Ist beim Einfügen in die Datenbank etwas schief gegangen, wird 500 ausgegeben. Dies weist auf einen internen Serverfehler hin.

Beim erfolgreichen Erstellen wird nur einfacher JSON-Output generiert. Wenn es gewünscht ist, kann hier etwa auch der soeben erstellte Datensatz wieder zurückgegeben werden.

11.1.4 Weitere Funktionalitäten von REST APIs

Die hier gezeigten Funktionalitäten zeigen nur einige wenige Dinge, die mit REST APIs erreicht werden können. Zentral sind etwa Authentifikation und Autorisierung: diese stellen sicher, dass bestimmte Endpunkte der API nur mit gültigen Credentials erreicht werden können und bestimmte Abfragen nur Ergebnisse liefern, wenn diese Credentials für die jeweilige Abfrage berechtigt sind.

Weiters sind selbstverständlich noch andere HTTP-Methoden ein wichtiger Bestandteil. Dies und mehr wird umfassend in Web Architectures (WAR3VO) besprochen und angewandt.

11.2 AJAX

In Abschnitt 11.1 wurde gezeigt, wie auf der Serverseite Daten strukturiert mit einfachen HTTP(S)-Aufrufen zur Verfügung gestellt werden können. Auf der Clientseite kommt dabei oft AJAX zum Einsatz, wenn es darum geht, Daten von einem Server abzufragen und im Frontend einer Webseite ohne Neuladen der Seite einzubauen.

11.2.1 Allgemein

AJAX steht für *Asynchronous JavaScript And XML* und beschreibt das Konzept des dynamischen Änderns von Webseiten mit Hilfe einer asynchronen Datenübertragung.

Synchronität vs. Asynchronität

Synchronität im technischen Sinn bedeutet, dass alle Abläufe in einem Softwareprogramm oder einer Webseite zeitlich gestaffelt ablaufen. Lädt die Webseite neue Daten, so wird gewartet, bis diese Daten empfangen wurden und erst dann wird der Rest der Seite geladen. Die Seite (oder auch das Programm) blockiert also solange alle weiteren Vorgänge, bis der aktuelle Vorgang abgeschlossen ist.

Von *Asynchronität* wird hingegen gesprochen, wenn solche Vorgänge quasi parallel ablaufen können. Die Webseite beginnt mit dem Laden von Daten und fährt danach gleich mit dem Anzeigen von anderen Dingen fort, die nicht auf diese Daten angewiesen sind. Ist das Laden der Daten beendet (der Zeitpunkt steht hier aber nicht fest), wird das System benachrichtigt und kann mit der Verarbeitung der Daten beginnen.

Asynchronität trägt in der Regel entscheidend dazu bei, dass (Web-) Applikationen reaktionsfreudig (engl. „responsive“) bleiben.

Das Prinzip von AJAX

AJAX ist keine eigene oder neue Technologie sondern die Verbindung bekannter Web-Technologien wie etwa JavaScript oder XML. Mit Hilfe dieser Technologien wird der klassische Ablauf einer Webanwendung verändert und dynamischer gestaltet. Die zentrale Veränderung ist dabei, dass der Seiten-Reload entfällt.

Klassischer Ablauf einer Webanwendung: Beim klassischen Schema fordert der Client mittels eines Requests am Server eine Seite an. Der Server sucht diese Seite in einem

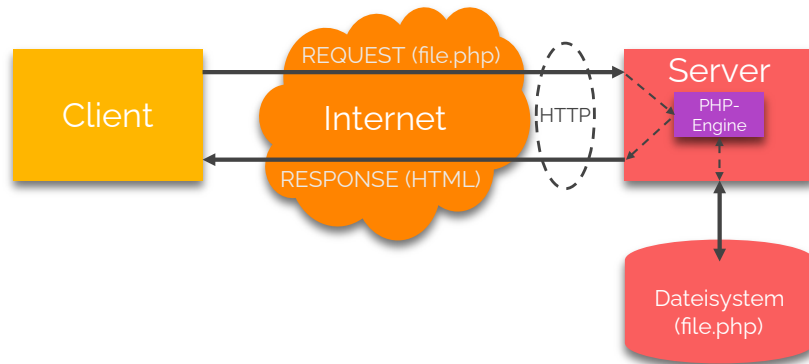


Abbildung 11.1: Bei einer klassischen Webanwendung (z. B. wie bisher rein in PHP) fordert der Browser eine Seite an, der Webserver verarbeitet diese und sendet den HTML-Code zurück.

Dateisystem, holt ggf. Informationen aus einer Datenquelle (z. B. Datenbank) und sendet den fertigen HTML-Code (ggf. mit Bildern, CSS etc.) an den Client zurück. Dies erzeugt den typischen Reload, da eine neue Seite angefordert wird. Abbildung 11.1 stellt dies schematisch dar.

Ablauf einer Webanwendung mit AJAX: Kommt nun AJAX zum Einsatz, so wird zwischen Client und Server noch eine Zwischenschicht eingeführt, die die Kommunikation mit dem Server übernimmt. Der Client schickt nun nicht mehr direkt einen HTTP-Request, sondern tätigt einen JavaScript-Aufruf. Dieser wird von der AJAX-Schicht verarbeitet, indem ein Request an den Server abgeschickt und die notwendigen Daten angefordert werden (z. B. wird eine einfache PHP-Datei oder eine REST API aufgerufen, welche die Daten abfragt und aufbereitet zurücksendet). Diese kommen vom Server nun nicht als zwingend als HTML retour, sondern oft auch in einem Datenformat wie XML (namensgebend), JSON, HTML oder als reiner Text, wie bereits in Abschnitt 11.1.2 gezeigt.

Die AJAX-Zwischenschicht nimmt diese Daten entgegen und kann nun die neuen Inhalte mittels JavaScript in die bestehende Seite integrieren. Auf diese Art und Weise wird das Neuladen der Seite vermieden. Zudem können auch nur einzelne Teile einer Seite ausgetauscht werden. Abbildung 11.2 veranschaulicht den AJAX-Ablauf.

Datenformate

Zur Übertragung der Daten vom Server zurück zur AJAX-Schicht kommen die Datenformate zum Einsatz, die bereits in Abschnitt 11.1.2 besprochen wurden:

- **XML:** Wenn die über AJAX am Server aufgerufene Ressource (z. B. eine REST API) XML zurück liefert, so muss natürlich zuvor ein XML-Format definiert worden sein. Einerseits muss die Datei am Server in diesem Format generiert werden, andererseits muss die AJAX-Schicht das Format kennen und parsen können, um die zurückgelieferten Daten auch verwenden zu können. Das Parsen der XML-Daten geschieht mit DOM-Methoden in JavaScript (`querySelector()`),

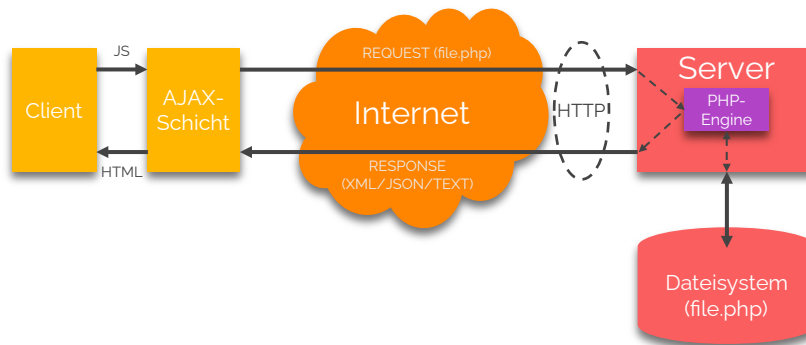


Abbildung 11.2: Bei einer AJAX-basierten Webanwendung kommuniziert der Browser über JavaScript mit einer Zwischenschicht. Diese setzt die eigentliche Anfrage zum Server ab (z. B. eine PHP-Datei oder REST API) und erhält Daten in XML, JSON oder einem anderen Format zurück. Diese Daten werden von der Zwischenschicht an den Browser retour gegeben bzw. eingebaut.

`getElementById()` etc.), denn diese funktionieren genauso mit XML-Dokumenten (wie auch schon in PHP in Vorlesung 10 demonstriert).

- **JSON:** Der Vorteil von JSON gegenüber XML ist, dass JSON-Dokumente in der Regel kürzer und präziser sind, durch die Einteilung in Objekte und Arrays lassen sich Daten, die strukturiert vorliegen, sehr gut damit abbilden. Ebenso ist JSON ein Subset von JavaScript, weshalb JSON-Objekte direkt als JavaScript-Objekte verstanden werden können, ohne zuvor geparsed werden zu müssen.
- **HTML:** Liefert der Server direkt *HTML-Code* zurück, so wird auch oft von *AHAH* (kurz für „Asynchronous HTML and HTTP“) gesprochen. Hierbei werden nie komplette HTML-Seiten zurückgeliefert sondern nur kleine Fragmente (z. B. `<article>`-Element oder ein Absatz). Diese werden dann mittels JavaScript (von der AJAX-Schicht) an der entsprechenden Stelle in der Webseite eingebaut. Diese Fragmente müssen am Server nicht zwingend dynamisch generiert werden (z. B. mittels PHP). Es kann auch einfach eine HTML-Datei angefordert werden, die genau dieses Fragment enthält.
- **Text:** Einfacher Text eignet sich dann sehr gut, wenn nur wenig Daten (etwa nur einzelne, kurze Strings) vom Server zurückgeschickt werden. Dadurch kann der Overhead des Parsens von Datenformaten entfallen.

11.2.2 Die Fetch API

Für die Kommunikation mit einem Server steht in modernem JavaScript die *Fetch API*⁵ zur Verfügung. Sie ist der native, in allen aktuellen Browsern verfügbare Standard, um asynchrone HTTP(S)-Requests abzusetzen und löst das ältere `XMLHttpRequest`-Objekt ab (siehe Abschnitt 11.2.6).

Das grundlegende Muster eines Fetch-Aufrufs besteht aus drei Bausteinen, die zunächst wie ein Rezept verwendet werden können. Das technische Fundament dahin-

⁵https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

ter – das Konzept der *Promises* – wird in der Lehrveranstaltung Web Architectures (WAR3VO) im dritten Semester ausführlich behandelt. Für die folgenden Beispiele genügt das Muster:

- **async**: Eine Funktion, die auf Daten aus dem Netzwerk wartet, wird mit dem Schlüsselwort **async** markiert.
- **await**: Ein Aufruf, dessen Ergebnis abgewartet werden muss, wird mit **await** versehen. Das Script „pausiert“ an dieser Stelle, bis das Ergebnis vorliegt, und liefert es dann direkt zurück. Dadurch liest sich der Code von oben nach unten, fast wie synchroner Code.
- **try/catch**: Tritt während des Requests ein Fehler auf, wird er im **catch**-Block abgefangen.

Ein vollständiger Fetch-Aufruf sieht damit grundsätzlich so aus:

```
1 async function loadData() {
2   try {
3     const response = await fetch("daten.php");
4
5     // Antwort enthält HTTP Status zwischen 200 und 299
6     if (response.ok) {
7       const data = await response.json();
8       // data weiterverarbeiten ...
9     } else {
10      // HTTP-Fehler (z. B. 404, 500) behandeln
11    }
12  } catch (error) {
13    // Netzwerkfehler behandeln (Server nicht erreichbar)
14  }
15 }
```

An diesem Muster fallen zwei Dinge auf, die für den korrekten Einsatz von Fetch zentral sind und im Folgenden kurz erläutert werden.

Zweimal await: `fetch()` wartet zunächst nur darauf, dass die *Antwort* des Servers eintrifft, also Statuszeile und Header. Der eigentliche *Inhalt* (der Body) wird erst durch einen zweiten Aufruf ausgelesen: `response.json()` für das Verarbeiten von JSON-Daten oder `response.text()` für reinen Text. Da auch das Auslesen des Bodys – gerade bei großen Datenmengen – Zeit in Anspruch nehmen kann, ist auch hier ein **await** notwendig. Man wartet also zuerst auf die Antwort, dann auf deren Inhalt.

HTTP-Fehler landen nicht im catch: Eine wichtige Besonderheit von Fetch: Ein HTTP-Fehlercode wie 404 Not Found oder 500 Internal Server Error gilt für `fetch()` *nicht* als Fehler. Die Anfrage war ja erfolgreich – der Server hat geantwortet, nur eben mit einem Fehlercode. Der **catch**-Block wird daher nur bei echten *Netzwerkfehlern* ausgelöst, etwa wenn der Server gar nicht erreichbar ist.

Um auf HTTP-Statuscodes zu reagieren, muss daher die Eigenschaft `response.ok` (**true** bei Statuscodes von 200–299) oder direkt `response.status` ausgewertet werden. Dies passt unmittelbar zur Trennung der Fehlerebenen in REST APIs aus Abschnitt 11.1.2: Der **catch**-Block deckt die *Transport*-Ebene ab (kam überhaupt eine Antwort

an?), die Prüfung von `response.ok` die *Applikations*-Ebene (war die Antwort inhaltlich erfolgreich?).

11.2.3 Beispiel: Textdaten verarbeiten

In diesem und in den Abschnitten 11.2.4 und 11.2.5 folgen nun drei Beispiele, welche die Verwendung von AJAX mit Hilfe der *Fetch API* illustrieren. Im ersten wird zunächst die grundsätzliche Verwendung mit einfachen Textdaten vom Server gezeigt. Neuer Inhalt wird auf einer HTML-Seite asynchron von einem Webserver angefordert und das Ergebnis eingebaut. Konkret befindet sich auf einer Seite eine Auswahlliste mit den neun Bundesländern Österreichs. Je nach Auswahl wird die entsprechende Landeshauptstadt auf der Seite angezeigt. Die Zuordnung von Bundesland zu Hauptstadt passiert dabei in einer PHP-Datei am Server, welche quasi als Datenquelle (wie eine REST API) dient. Diese erhält einen Parameter und liefert die entsprechende Stadt zurück, die dann angezeigt wird – alles ohne Neuladen der Seite.

Im Folgenden werden die einzelnen Bestandteile des Beispiels erläutert.

Die HTML-Seite

Ausgangspunkt ist eine einfache HTML-Datei. Diese enthält neben etwas Text ein Auswahlfeld (`<select>`) mit zehn Einträgen. Neben dem Standardwert „Please select“ sind die neun österreichischen Bundesländer erfasst. Die Liste hat die ID `state`, um sie später einfacher ansprechen zu können.

Darüber hinaus gibt es ein `<h2>`-Element mit der ID `capital`, in dem zunächst der Platzhaltertext „Select a state to have it replaced with its capital“ enthalten ist. Dieser Text wird durch die jeweilige Landeshauptstadt ersetzt werden.

Im `<head>` des Dokuments ist die Datei `ajax.js` mit `defer` eingebunden. In ihr findet der gesamte AJAX-Aufruf statt.

```
1 <!DOCTYPE html>
2 <html lang="en">
3   <head>
4     <title>AJAX using Fetch and Text Data</title>
5     <meta charset="utf-8">
6     <script src="ajax.js" defer></script>
7   </head>
8   <body>
9     <h1>Show the Capital</h1>
10
11     <p>The following example retrieves the capital of the selected state
12       asynchronously and displays it in the heading.</p>
13
14     <h2 id="capital">Select a state to have it replaced with its capital</h2>
15
16     <form action="#">
17       <label for="state">Please select a state:</label>
18       <select id="state">
19         <option>Please select</option>
20         <option>Burgenland</option>
21         <option>Carinthia</option>
22         <option>Lower Austria</option>
```



```
23     <option>Styria</option>
24     <option>Tyrol</option>
25     <option>Upper Austria</option>
26     <option>Vorarlberg</option>
27     <option>Vienna</option>
28   </select>
29 </form>
30 </body>
31 </html>
```

Die JavaScript-Datei

Der Aufruf mit Fetch gestaltet sich einfach. Es genügt, die Auswahlliste abzufragen und ihr einen Event-Handler für den `change`-Event zuzuweisen. Jedes Mal, wenn ein neues Bundesland gewählt wird, ruft dieser die Funktion `sendRequest()` auf.

```
1 const select = document.getElementById("state");
2 select.addEventListener("change", sendRequest);
```

In der Funktion `sendRequest()` wird nun der Fetch-Aufruf an `states.php` abgesetzt. Da die Funktion auf Netzwerkdaten wartet, wird sie mit `async` markiert. Wie bei allen Funktionen, die von einem Event-Handler aufgerufen werden, wird das `event`-Objekt automatisch mitgegeben. Dieses besitzt das Unterobjekt `target`. Dies ist das Element, bei dem der Event ausgelöst wurde. Dieses Objekt besitzt wiederum die Eigenschaft `selectedIndex`, welche die Index-Position der Auswahl enthält. 0 ist dabei das erste, 1 das zweite Element, usw. Somit lässt sich der ausgewählte Index im Drop-down über `event.target.selectedIndex` ermitteln und als GET-Parameter `index` an den Aufruf von `states.php` angehängt.

Mit `await fetch(...)` wird der Request abgesetzt und auf die Antwort gewartet. Als Parameter wird ein Konfigurations-Objekt mit dem Schlüssel `headers` mitgeschickt. Dies informiert den Server, dass der Client mit Textdaten umgehen kann. Anschließend wird über `response.ok` geprüft, ob die Anfrage erfolgreich war. Falls ja, wird mit `await response.text()` der Antworttext ausgelesen und in das Element mit der ID `capital` gesetzt. Andernfalls wird eine Fehlermeldung angezeigt.

```
1 async function sendRequest(event) {
2   const index = event.target.selectedIndex;
3   const capital = document.getElementById("capital");
4
5   try {
6     const response = await fetch("states.php?index=" + index, {
7       headers: {"Accept": "text/plain"}
8     });
9
10    if (response.ok) {
11      capital.textContent = await response.text();
12    } else {
13      capital.textContent = "A problem occurred with requesting the selected capital.";
14    }
15  } catch (error) {
16    capital.textContent = "The server could not be reached.";
17  }
18 }
```

Im Vergleich zu einer Umsetzung mit `XMLHttpRequest` (siehe Abschnitt 11.2.6) entfällt bei Fetch das manuelle Vorbereiten, Konfigurieren und Abschicken des Requests über mehrere Methoden. Der gesamte Ablauf liest sich linear von oben nach unten.

Die PHP-Datei

Da AJAX ja verwendet wird, um vom Webserver dynamisch Daten im Hintergrund nachzuladen, muss es auch eine Instanz geben, die diese Daten entsprechend aufbereitet. Dies ist in vielen Fällen eine REST API, wie in Abschnitt 11.1 gezeigt, welche die benötigten Daten aus den Ressourcen (Datenbank, Dateien etc.) abfragt und entsprechend aufbereitet retour gibt. In diesem Fall ist es jedoch eine einfache, einzelne Datei `states.php`, mit einem Array als Datenquelle, das alle Landeshauptstädte sowie einen Standardeintrag zu Beginn enthält. Die Positionen decken sich hierbei mit denen aus der Auswahlliste.

Die PHP-Datei wird über die Fetch API aufgerufen und bekommt den GET-Parameter `index` mit übergeben. Zunächst wird mit `isset()` geprüft, ob dieser Parameter überhaupt vorhanden ist. Fehlt er, wird sofort Statuscode 400 (Bad Request) zurückgegeben, da ein wichtiger Parameter fehlt. Andernfalls wird mittels `array_key_exists()` geprüft, ob der übergebene Index im Array existiert, also zwischen 0 und 9 liegt. Wenn ja, wird Statuscode 200 (OK) gesetzt und die zugehörige Landeshauptstadt zurückgegeben. Existiert der Index nicht (etwa bei 10), wird Statuscode 404 zurückgegeben, da es unter diesem Index keinen Datensatz gibt.

Zuvor wird noch der Header `Content-Type` auf `text/plain` gestellt. Somit handelt es sich hierbei um reinen Text als Datenaustauschformat. Die PHP-Datei könnte diesen Wert auch in XML oder JSON verpackt retour geben. Da es sich aber nur um einen einzelnen Wert handelt, wäre der Overhead größer als der Nutzen daraus.

```
1 <?php
2
3 $capitals = [
4     "Please select a state",
5     "Eisenstadt",
6     "Klagenfurt",
7     "Sankt Pölten",
8     "Salzburg",
9     "Graz",
10    "Innsbruck",
11    "Linz",
12    "Bregenz",
13    "Vienna",
14 ];
15
16 header("Content-Type: text/plain");
17
18 if (!isset($_GET["index"])) {
19     http_response_code(400);
20 } elseif (array_key_exists($_GET["index"], $capitals)) {
21     http_response_code(200);
22     echo $capitals[$_GET["index"]];
23 } else {
24     http_response_code(404);
25 }
```

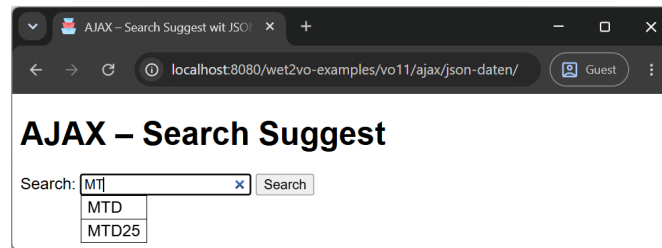


Abbildung 11.3: Nachdem die Buchstaben „MT“ eingetippt wurden, schlägt das System alle Wörter vor, die genau diese beiden Buchstaben an einer beliebigen Position enthalten.

11.2.4 Beispiel: JSON-Daten verarbeiten

Bei dem im Abschnitt 11.2.3 gezeigten Beispiel lieferte der AJAX-Aufruf ein Wort als reinen Text zurück. Im Folgenden soll nun ein Beispiel demonstriert werden, in dem die Rückgabewerte im JSON-Format vorliegen, wie dies bereits im Abschnitt 11.1 demonstriert wurde. Das Beispiel ist ein so genannter *Search Suggest*, d. h. bei der Eingabe von Buchstaben in ein Suchfeld werden alle Begriffe aus einem Datenbestand zurückgeliefert, in denen die eingegebenen Buchstaben vorkommen.

Prinzip

Ein Eingabefeld einer HTML-Datei wird als Suchfeld deklariert. Bei einem klassischen Ablauf wird beim Absenden des Formulars eine Suche in PHP durchgeführt und nach dem Reload der Seite werden die Suchergebnisse angezeigt.

Mit Hilfe von AJAX ist es nun möglich, bereits während der Eingabe des Suchbegriffs den Anwender*innen mögliche gültige Ergebnisse vorzuschlagen. Diese wissen dadurch bereits während der Eingabe, welche Begriffe ein Ergebnis liefern werden. Alternativ können genauso häufig verwendete Suchbegriffe vorgeschlagen werden (wie z. B. bei Google).

AJAX im Ablauf: Jedes Mal, nachdem ein Buchstabe getippt wurde, muss mittels AJAX nachgesehen werden, ob der bisher eingegebene Begriff (Needle) in einer Menge von definierten Wörtern (Haystack) gefunden wird. Wenn ja, wird der Begriff in einer Liste vorgeschlagen (siehe Abbildung 11.3).

Dieser gesamte Ablauf passiert im Hintergrund ohne Reload, also asynchron.

Die HTML-Datei

Die Datei `index.html` enthält das Formular. Das Eingabefeld erhält in der JavaScript-Datei `ajaxsearch.js` einen Event-Handler für den `keyup`-Event, der die zentrale Funktion des Search Suggests aufruft.

Das `<div>`-Element mit der ID `suggestions` ist zu Beginn leer und wird im Verlauf mit den möglichen Suchergebnissen befüllt.

```
1 <!DOCTYPE html>
2 <html lang="en">
3   <head>
```

```
4 <title>AJAX &ndash; Search Suggest wit JSON Data</title>
5 <meta charset="utf-8">
6 <link rel="stylesheet" href="styles.css">
7 <script src="ajaxsearch.js" defer></script>
8 </head>
9 <body>
10 <h1>AJAX &ndash; Search Suggest</h1>
11
12 <form id="form" method="get" action="output.php" autocomplete="off">
13   <div>
14     <label for="search">Search:</label>
15     <input type="search" id="search" name="search">
16     <button>Search</button>
17   </div>
18
19   <div id="suggestions"></div>
20 </form>
21 </body>
22 </html>
```

Die JavaScript-Datei

Auch hier wird zunächst das Suchfeld abgefragt und mit einem Event-Handler für den **keyup**-Event versehen. Jedes Mal, wenn eine Taste losgelassen wird, ruft dieser die Funktion `sendRequest()` auf.

```
1 const search = document.getElementById("search");
2 search.addEventListener("keyup", sendRequest);
```

In `sendRequest()` wird der aktuelle Suchbegriff über `encodeURIComponent()` für die Verwendung in einer URL aufbereitet (dies stellt sicher, dass für URLs problematische Zeichen durch ihre Escape-Sequenzen ersetzt werden) und als GET-Parameter **search** an `search.php` angehängt. Über das Konfigurationsobjekt wird angegeben, dass als Datentyp JSON erwartet wird. Da die Antwort hier (wie erwartet) im JSON-Format vorliegt, wird sie mit `await response.json()` direkt in ein JavaScript-Objekt umgewandelt. Das Ergebnis wird an `handleResponse()` übergeben.

```
1 async function sendRequest() {
2   const str = encodeURIComponent(search.value);
3
4   try {
5     const response = await fetch("search.php?search=" + str, {
6       headers: {"Accept": "application/json"}
7     });
8
9     if (response.ok) {
10      const data = await response.json();
11      handleResponse(data);
12    }
13  } catch (error) {
14    console.error("Request failed:", error);
15  }
16 }
```

Die Funktion `handleResponse()` erhält nun das fertig aufbereitete JSON-Objekt als Parameter. Falls der Eintrag `count` (die Anzahl der zurückgegebenen Einträge) größer als 0 ist, wird für jeden Eintrag ein `<div>` mit Event-Handlern für `mouseover`-, `mouseout`- und `click`-Events sowie einer CSS-Klasse erzeugt. Diese neu erzeugten Elemente werden dann in das bestehende `<div>`-Element mit der ID `suggestions` eingefügt.

```
1 function handleResponse(data) {
2   const suggestDiv = document.getElementById("suggestions");
3   suggestDiv.innerHTML = "";
4
5   if (data.count > 0) {
6     for (let i = 0; i < data.count; i++) {
7       const entry = document.createElement("div");
8       entry.addEventListener("mouseover", function () {
9         this.classList.add("suggestLinkOver");
10      });
11      entry.addEventListener("mouseout", function () {
12        this.classList.remove("suggestLinkOver");
13      });
14      entry.addEventListener("click", function () {
15        search.value = this.textContent;
16        suggestDiv.innerHTML = "";
17      });
18      entry.classList.add("suggestLink");
19      entry.textContent = data.words[i];
20      suggestDiv.appendChild(entry);
21    }
22  }
23 }
```

In den anonymen Funktionen der `mouseover`- und `mouseout`-Handler wird eine CSS-Klasse gewechselt, sobald sich die Maus über ein `<div>`-Element bzw. wieder hinaus bewegt. In der Funktion des `click`-Handlers wird der Wert des angeklickten Elements schließlich in das Eingabefeld gesetzt und das `<div>`-Element mit allen Vorschlägen wieder geleert.

Die PHP-Datei

In der PHP-Datei `search.php` wird zunächst eine Datenquelle (hier eine PHP-Klasse, die ein Array verwaltet) eingebunden. Von dieser Datenquelle wird nun ein Objekt erzeugt und darauf die Methode `search()` aufgerufen. Sie bekommt als Argument den als GET-Parameter übermittelten Sucheintrag. Ist dieser GET-Parameter nicht vorhanden, wird sofort der Statuscode 400 für „Bad Request“ zurückgegeben. Im Erfolgsfall ist es der normale Statuscode 200. Zuvor wird auch der `Content-Type`-Header auf `application/json` gesetzt.

```
1 $data = new SearchData();
2
3 header("Content-Type: application/json");
4 if (isset($_GET["search"])) {
5   http_response_code(200);
6   echo $data->search($_GET["search"]);
7 }
```

```

8 else {
9     http_response_code(400);
10 }

```

Die Methode `search()` läuft nun in einer Schleife durch das Array an hinterlegten Wörtern und überprüft mittels `mb_stripos()`, ob der Suchbegriff im Wort enthalten ist, wenn der Suchbegriff kein leerer String ist. Falls ja, wird es in das Array `$jsonResponse["words"]` geschrieben. Dieses assoziative Array wird später zur Eigenschaft `words` im JSON-Objekt. Nachdem alle Wörter durchlaufen wurden, wird das Array sortiert und mittels `count()` seine Länge ermittelt. Diese wird anschließend in `$jsonResponse["count"]` eingetragen. `$jsonResponse` wird dann mit Hilfe von `json_encode()` in ein JSON-Objekt verwandelt und mit `echo` zurückgegeben.

```

1 public function search(string $searchString): string
2 {
3     $jsonResponse["words"] = [];
4
5     if ($searchString !== "") {
6         foreach ($this->titles as $entry) {
7             if (mb_stripos($entry, $searchString, 0, "UTF-8") !== false) {
8                 $jsonResponse["words"][] = $entry;
9             }
10        }
11        sort($jsonResponse["words"]);
12    }
13
14    $jsonResponse["count"] = count($jsonResponse["words"]);
15
16    return json_encode($jsonResponse);
17 }

```

Als Datenbestand dient das folgende Array:

```

1 $this->titles = [
2     "Die FH",
3     "WET2V0",
4     "FH",
5     "Hagenberg",
6     "Datenbanken",
7     "Medientechnik und -design",
8     "MTD25",
9     "MTD",
10    "PHP",
11    "JavaScript",
12    "Österreich",
13    "Bla",
14    "Gut/Böse",
15 ];

```

Wird die Suche mit dem Begriff „MT“ ausgeführt, so wird folgender JSON-Output von `search.php` erzeugt:

```
{"words": ["MTD", "MTD25"], "count": 2}
```

11.2.5 Beispiel: Formulardaten mit POST übertragen

Beide gezeigten Beispiele wurden mit der Methode GET übertragen. Gerade aber bei Formulardaten wird in der Regel die Methode POST benötigt, vor allem wenn nicht nur Textdaten, sondern etwa auch Dateien mit übertragen werden sollen. POST-Daten werden im Body des Requests mit übertragen und müssen daher über die Option `body` im Konfigurationsobjekt des `fetch()`-Aufrufs angegeben werden. Handelt es sich nur um Textdaten aus Eingabefeldern, kann als Wert grundsätzlich eine Zeichenkette in der folgenden Form übergeben werden:

```
1 fetch("details.php", {  
2   method: "POST",  
3   headers: { "Content-Type": "application/x-www-form-urlencoded" },  
4   body: "foo=bar&test=something",  
5 });
```

`foo` und `test` wären in diesem Fall die Namen zweier Formularfelder und `bar` und `something` deren Werte. Damit die Daten serverseitig im `$_POST`-Array ankommen, muss zusätzlich der `Content-Type`-Header auf `application/x-www-form-urlencoded` gesetzt werden – jenes Format, das auch ein HTML-Formular beim Absenden erzeugt.

Dieser händische Weg ist allerdings aus mehreren Gründen mühsam: Die Werte müssten zuvor aus den einzelnen Formularfeldern ausgelesen und zu dieser Zeichenkette zusammengesetzt werden, was bei vielen Feldern schnell unübersichtlich wird. Zudem müsste jeder Wert einzeln über `encodeURIComponent()` kodiert werden, da Sonderzeichen wie `&` oder `=` die Zeichenkette sonst zerstören würden. Und schließlich funktioniert dieser Ansatz ohnehin nur für reine Textfelder, nicht für Dateien.

Aus diesem Grund gibt es zur Aufbereitung von Formularinhalten für den Versand das `FormData`-Objekt⁶. Es nimmt einem das manuelle Zusammensetzen, das Kodieren und das Setzen des `Content-Type`-Headers vollständig ab und kann darüber hinaus auch Dateien übertragen.

POST-Requests mit Fetch und FormData

Zur Aufbereitung von Formularinhalten für den Versand dient das `FormData`-Objekt⁷. Sollen alle Felder eines Formulars berücksichtigt werden, wird das `<form>`-Element abgefragt und im Konstruktor übergeben. Wichtig ist, dass nur Formularfelder berücksichtigt werden, die ein `name`-Attribut aufweisen.

```
1 const form = document.querySelector("form");  
2 const formData = new FormData(form);
```

Bei Fetch wird das `FormData`-Objekt einfach als `body` im Optionen-Objekt des Requests übergeben. Die Methode wird über die Option `method` auf `"POST"` gesetzt. Anders als beim manuellen Versand einer Zeichenkette muss der `Content-Type`-Header hier *nicht* gesetzt werden: Übergibt man ein `FormData`-Objekt, erkennt der Browser dies und setzt den passenden `Content-Type` (`multipart/form-data`) automatisch – inklusive der technisch notwendigen `boundary`-Information, die die einzelnen Formularfelder im Body voneinander trennt. Dieser Header darf bei `FormData` sogar *nicht* manuell gesetzt werden, da die `boundary` nur dem Browser bekannt ist.

⁶<https://developer.mozilla.org/en-US/docs/Web/API/FormData>

⁷<https://developer.mozilla.org/en-US/docs/Web/API/FormData>

```
1 const response = await fetch("somefile.php", {  
2   method: "POST",  
3   body: formData,  
4 });
```

Auf der Serverseite stehen die Daten nun unter den entsprechenden Namen zur Verfügung: Textdaten in `$_POST` und Dateien in `$_FILES`.

Alternativ kann ein `FormData`-Objekt auch „manuell“ befüllt werden, falls nicht alle Inhalte des Formulars im AJAX-Request mitgeschickt werden sollen. In diesem Fall wird das Objekt ohne Argument angelegt, Einträge werden mittels `append()` hinzugefügt. Im folgenden Beispiel werden zwei Einträge angelegt:

```
1 const formData = new FormData();  
2 formData.append("foo", "bar");  
3 formData.append("test", "something");
```

Auch dieses `FormData`-Objekt wird dann der Methode `fetch()` übergeben.

Dateiuploads über FormData: Ein besonderer Vorteil des `FormData`-Objekts zeigt sich beim Hochladen von Dateien. Enthält das Formular ein Eingabefeld vom Typ `file` (siehe Abschnitt 7.3), so wird die ausgewählte Datei beim Erzeugen des `FormData`-Objekts automatisch mit übernommen und beim Fetch-Versand mitgeschickt. Serverseitig steht sie dann – wie beim klassischen Upload aus Abschnitt 7.3 – im gewohnten superglobalen Array `$_FILES` zur Verfügung. Die gesamte serverseitige Verarbeitung (Fehlercode-Prüfung, `move_uploaded_file()` usw.) bleibt also identisch. Der einzige Unterschied ist, dass der Upload nun asynchron und ohne Seiten-Reload erfolgt.

Ein wichtiger Kontrast zum klassischen Upload betrifft das `<form>`-Element selbst: Beim herkömmlichen Upload ist das Attribut `enctype="multipart/form-data"` zwingend erforderlich (siehe Abschnitt 7.3). Beim Versand über Fetch und `FormData` ist dies *nicht* nötig – der Browser setzt den passenden `Content-Type`-Header inklusive der nötigen `boundary`-Information beim Absenden automatisch und korrekt. Aus demselben Grund darf dieser Header bei `FormData` auch nicht manuell gesetzt werden.

Abschließend soll das Wissen der POST-Übertragung mit Hilfe eines `FormData`-Objekts verwendet werden, um ein einfaches Formular mit AJAX abzusenden und die Ergebnisse ohne Neuladen der Seite anzuzeigen.

Die HTML-Datei

Die HTML-Datei enthält hauptsächlich das `<form>`-Element, das zwei Formularfelder – einmal für Text und einmal für ein Passwort – enthält. Darüber hinaus existiert eine leere, ungeordnete Liste, in der die Ergebnisse eingebettet werden.

```
1 <!DOCTYPE html>  
2 <html lang="en">  
3   <head>  
4     <meta charset="UTF-8">  
5     <title>AJAX POST Request with FormData</title>  
6     <script src="postform.js" defer></script>  
7   </head>  
8   <body>  
9     <h1>AJAX POST Request with FormData</h1>  
10    <form action="#" method="post" id="registerForm">
```



```
11     <label for="user">Name:</label>
12     <input type="text" id="user" name="user">
13     <label for="pass">Password:</label>
14     <input type="password" id="pass" name="pass">
15     <button type="submit">Register</button>
16   </form>
17   <ul id="loginDetails"></ul>
18 </body>
19 </html>
```

Die JavaScript-Datei

In `postform.js` wird die gesamte Logik realisiert. Sie ist ähnlich aufgebaut wie in den vorangegangenen Beispielen. Das Formularfeld wird abgefragt und ein Event-Handler für den `submit`-Event angehängt. Tritt der Event auf, wird `sendRequest()` aufgerufen.

```
1 document.getElementById("registerForm").addEventListener("submit", sendRequest);
```

Die Funktion `sendRequest()` verhindert zunächst mit `event.preventDefault()` das klassische Absenden des Formulars (und somit einen Seiten-Reload). Danach wird aus dem Formular ein `FormData`-Objekt erzeugt und per `Fetch` an `details.php` geschickt. Bei erfolgreicher Antwort wird der zeilenweise zurückgegebene Text ausgelesen und verarbeitet. Beim Absenden wurde bereits mit dem `Accept`-Header kommuniziert, dass Text verarbeitet werden kann.

```
1 async function sendRequest(event) {
2   event.preventDefault();
3
4   const formData = new FormData(event.target);
5   const list = document.getElementById("loginDetails");
6   list.innerHTML = "";
7
8   try {
9     const response = await fetch("details.php", {
10       method: "POST",
11       headers: {"Accept": "text/plain"},
12       body: formData,
13     });
14
15     if (response.ok) {
16       const text = await response.text();
17       const entries = text.split("\n");
18
19       for (let i = 0; i < entries.length; i++) {
20         const item = document.createElement("li");
21         item.textContent = entries[i];
22         list.appendChild(item);
23       }
24     } else {
25       const item = document.createElement("li");
26       item.textContent = "A problem occurred with the request!";
27       list.appendChild(item);
28     }
29   } catch (error) {
30     const item = document.createElement("li");
```

```
31     item.textContent = "The server could not be reached.";
32     list.appendChild(item);
33 }
34 }
```

Zunächst wird die Liste hier zu Beginn jedes Aufrufs mit `list.innerHTML = ""` geleert. Damit wird verhindert, dass sich die Ergebnisse bei mehrmaligem Absenden immer wieder unterhalb anhäufen. Die PHP-Datei liefert kein JSON-Format, sondern einfach zeilenweise eine Zeichenkette im Format „Schlüssel: Wert“ für jeden Eintrag zurück. Um diese Daten zu verarbeiten, wird die in `text` gespeicherte Antwort an den Zeilenumbrüchen (`\n`) mit Hilfe von `split()` in ein Array aufgeteilt. Dieses wird durchlaufen und für jeden Eintrag ein `<li>`-Element erzeugt, welches in die ungeordnete Liste gehängt wird. Im Fehlerfall wird eine Fehlermeldung entsprechend ausgegeben.

Die PHP-Datei

Die PHP-Datei muss das soeben angesprochene Format erzeugen. Zunächst wird überprüft, ob in `_POST` Einträge vorhanden sind. Wenn nein, wird der Statuscode 400 zurückgegeben. Wenn ja, wird mit einer `foreach`-Schleife über alle Einträge in `$_POST` iteriert. Für jeden Eintrag stehen dabei der Schlüssel (`$key`) und der Wert (`$value`) direkt zur Verfügung, woraus jeweils ein String im Format `"key: $value"` gebildet und in das Array `$lines` geschrieben wird. Anschließend wird dieses Array mittels `implode()` in einen String umgewandelt, als Trennzeichen dient `PHP_EOL`, der Zeilenumbruch. Die Ausgabe wird mit `echo` und dem Statuscode 200 zurückgegeben. Die Antwort ist durch den `Content-Type-Header` als reine Textantwort gekennzeichnet.

```
1 header("Content-Type: text/plain");
2
3 if (!empty($_POST)) {
4     $lines = [];
5     foreach ($_POST as $key => $value) {
6         $lines[] = "$key: $value";
7     }
8     $output = implode(PHP_EOL, $lines);
9
10    http_response_code(200);
11    echo $output;
12 } else {
13    http_response_code(400);
14 }
```

Ausgabe und Anmerkungen

Bei Eingabe von „johndoe“ als Name und „topsecret“ als Passwort, werden diese Daten asynchron an die PHP-Datei geschickt. Diese liest diese aus dem `$_POST`-Array aus und gibt sie Zeilenweise zurück. Diese Rückgabe wird ohne Neuladen in die ungeordnete Liste eingefügt, sodass ein Ergebnis wie in Abbildung 11.4 entsteht.

Dieses Beispiel enthält keinerlei Validierung. Diese müsste in der PHP-Datei stattfinden, entsprechende Fehlermeldungen müssten mit der Response zurückgegeben und in JavaScript verarbeitet, d. h. angezeigt werden.

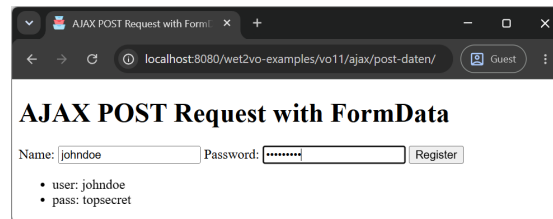


Abbildung 11.4: Nachdem Benutzer*innenname und Passwort eingegeben und abgeschickt wurden, werden diese, ohne die Seite neu zu laden, unterhalb in der Liste angezeigt.

11.2.6 Historischer Kontext: XMLHttpRequest

Bevor die Fetch API zum Standard wurde, war das *XMLHttpRequest*-Objekt (kurz *XHR*) über viele Jahre der einzige Weg, um in JavaScript asynchrone Requests abzusetzen. Es wurde ursprünglich von Microsoft entwickelt und später von den anderen Browserherstellern übernommen. Auf XHR geht auch der Name AJAX zurück: *Asynchronous JavaScript And XML* – auch wenn heute statt XML meist JSON übertragen wird.

Das Objekt ist nach wie vor in allen Browsern verfügbar und begegnet einem in älterem Code und Tutorials. Für neue Projekte wird es jedoch nicht mehr empfohlen, die Fetch API ist der moderne Ersatz.⁸ Zum Verständnis älterer Codebestände wird die grundsätzliche Arbeitsweise hier kurz am Bundesländer-Beispiel aus Abschnitt 11.2.3 gezeigt.

XHR ist *event-basiert*: Man legt ein Anfrage-Objekt an, bereitet den Request mit `open(method, url)` vor, registriert über `addEventListener()` einen Handler für den `load`-Event (der ausgelöst wird, sobald die Antwort vorliegt) und schickt den Request mit `send()` ab. Da die Antwort jederzeit eintreffen kann, wird die Verarbeitung in den Event-Handler ausgelagert – im Unterschied zum linearen Ablauf bei `async/await`.

```

1 const request = new XMLHttpRequest();
2
3 function sendRequest(event) {
4   request.open("GET", "states.php?index=" + event.target.selectedIndex);
5   request.addEventListener("load", handleResponse);
6   request.send();
7 }
8
9 function handleResponse() {
10  if (request.status === 200) {
11    document.getElementById("capital").textContent = request.responseText;
12  } else {
13    document.getElementById("capital").textContent = "A problem occurred.";
14  }
15 }
16
17 const select = document.getElementById("state");
18 select.addEventListener("change", sendRequest);

```

Im Vergleich zum Fetch-Beispiel aus Abschnitt 11.2.2 fällt auf, dass der Ablauf hier

⁸<https://developer.mozilla.org/docs/Glossary/XMLHttpRequest>

über zwei getrennte Funktionen verteilt ist (Absenden und Verarbeiten) und der Status manuell über `request.status` geprüft wird. Genau diese Verteilung über Event-Handler und das manuelle State-Tracking sind die Gründe, warum die modernere, sequenziell lesbare Fetch API XHR abgelöst hat.

Die WHATWG pflegt jedoch im Rahmen des *XMLHttpRequest* Living-Standards [48] weiterhin die Dokumentation und Funktionsweise des `XMLHttpRequest`-Objekts.

11.2.7 Ausblick: Promises

Das in diesem Kapitel verwendete `async/await`-Muster ist bewusst als Rezept eingeführt worden. Tatsächlich handelt es sich dabei um eine besonders gut lesbare Schreibweise für den Umgang mit *Promises*.

Ein *Promise* repräsentiert die finale Beendigung einer asynchronen Operation. Je nachdem, ob diese erfolgreich war oder fehlgeschlagen ist, kann ein entsprechender Callback aufgerufen werden. Historisch wurde dies über die Methoden `then()` (Erfolgsfall) und `catch()` (Fehlerfall) ausgedrückt:

```
1 fetch(url)
2   .then(response => response.json())
3   .then(data => {
4     // data verarbeiten
5   })
6   .catch(error => {
7     // Fehler verarbeiten
8   });
```

Dieser Code tut exakt dasselbe wie die `async/await`-Variante aus Abschnitt 11.2.2 – `await` ist lediglich eine neuere, sequenziell lesbare Schreibweise (eingeführt mit ES2017) für denselben Promise-Mechanismus. Beide Varianten liegen also *derselben* Fetch API zugrunde; sie unterscheiden sich nur in der Syntax. Das vollständige Konzept der Promises – Verkettung, Zustände und Methoden wie `Promise.all()` – wird in der Lehrveranstaltung Web Architectures (WAR3VO) im dritten Semester behandelt.

11.3 Serverseitige Konsumierung von REST APIs

Bisher wurde in diesem Kapitel gezeigt, wie REST APIs *bereitgestellt* werden und wie ein Browser als Client diese clientseitig über AJAX abfragt. In der Praxis müssen Webapplikationen jedoch häufig selbst auf externe Datenquellen zugreifen. Der eigene Webserver wird dabei zum Client, der Anfragen an eine fremde REST API stellt, die erhaltenen Daten (meist JSON) verarbeitet und in den eigenen Ablauf integriert.

Die zentrale Komponente für diesen Ablauf ist eine HTTP-Client-Bibliothek. Die PHP Framework Interoperability Group (PHP-FIG) definiert hierfür den Standard *PSR-18: HTTP Client* [36], der ein gemeinsames Interface für solche Clients festlegt. Die bekannteste Implementierung ist `guzzlehttp/guzzle`, die bereits in Vorlesung 5 für das Chuck Norris-Beispiel vorgestellt wurde.

Guzzle wird zunächst im Projekt mit Composer installiert:

```
1 composer require guzzlehttp/guzzle
```

11.3.1 Das Client-Objekt erstellen

Im ersten Schritt wird ein `Client`-Objekt angelegt, nachdem der Composer-Autoloader eingebunden wurde. Werden mehrere Requests an unterschiedliche Pfade derselben Domain gerichtet, kann im Konstruktor ein Basispfad (`base_uri`) gesetzt werden, sodass die einzelnen Requests nur noch den relativen Pfad angeben müssen.

```
1 use GuzzleHttp\Client;
2
3 require "vendor/autoload.php";
4
5 $client = new Client([
6     "base_uri" => "https://httpbin.org",
7 ]);
```

Das folgende Beispiel verwendet diesen Basispfad, um Requests abzusetzen. *httpbin.org*⁹ ist ein HTTP Request- und Response-Service, der eine REST API für Testzwecke bereitstellt und die gesendeten Daten zur Kontrolle wieder zurückgibt.

11.3.2 Requests senden

Requests werden über Shorthand-Methoden wie `get()` oder `post()` abgesetzt. Das zweite Argument ist ein assoziatives Array mit Optionen, etwa `query` für GET-Parameter oder `json` für einen JSON-Request-Body. Eine vollständige Auflistung bietet die Guzzle-Dokumentation¹⁰.

Im Folgenden werden ein GET- und ein POST-Request an den jeweiligen URL bei *httpbin.org* gesandt. Die Ergebnisse werden in zwei Response-Objekten gespeichert.

```
1 $getResponse = $client->get("/get", [
2     "query" => ["search" => "Hagenberg"],
3 ]);
4
5 $postResponse = $client->post("/post", [
6     "json" => ["username" => "jdoe", "realname" => "John Doe"],
7 ]);
```

11.3.3 Responses verarbeiten

Das Ergebnis eines Guzzle-Requests ist ein PSR-7 Response-Objekt. Über definierte Methoden können Statuscode, Header und Body ausgelesen werden:

- `getStatusCode()`: Liefert den Statuscode der Response.
- `getReasonPhrase()`: Liefert den textuellen Status.
- `getHeader($name)`: Gibt den Wert eines Headers zurück.
- `getBody()`: Liefert den Inhalt (Body) der Response.

Im folgenden Beispiel werden die Antworten des GET- und POST-Requests ausgewertet und ausgegeben. *httpbin.org* schickt als Antwort eine JSON-Datenstruktur mit den übermittelten Daten aus dem Request. Das Ergebnis ist in Abbildung 11.5 zu sehen.

⁹<https://httpbin.org/>

¹⁰<https://docs.guzzlephp.org/en/stable/request-options.html>

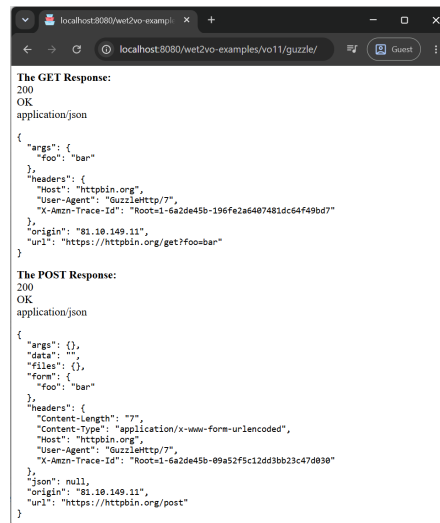


Abbildung 11.5: Die Antworten der Guzzle-Requests werden mit `echo` als Heredoc ausgegeben.

```

1 echo <<<GETRESPONSE
2 <p><strong>The GET Response:</strong><br>
3 {$getResponse->getStatusCode()}<br>
4 {$getResponse->getReasonPhrase()}<br>
5 {$getResponse->getHeader("Content-Type")[0]}</p>
6 <pre>{$getResponse->getBody()->getContents()}</pre>
7 GETRESPONSE;
8
9 echo <<<POSTRESPONSE
10 <p><strong>The POST Response:</strong><br>
11 {$postResponse->getStatusCode()}<br>
12 {$postResponse->getReasonPhrase()}<br>
13 {$postResponse->getHeader("Content-Type")[0]}</p>
14 <pre>{$postResponse->getBody()->getContents()}</pre>
15 POSTRESPONSE;

```

11.3.4 Fehlerbehandlung

Liefert eine Abfrage einen 400er-Fehlercode (Client Error, z. B. 404, weil die Ressource nicht existiert), so gibt Guzzle kein Response-Objekt zurück, sondern wirft eine `ClientException`. 500er-Fehlercodes (Server Error) lösen eine `ServerException` aus; die allgemeinere, beide Fälle abdeckende Exception ist `GuzzleException`.

Diese Exceptions werden mit einem `try/catch`-Block behandelt. Aus dem Exception-Objekt kann mit `getResponse()` die ursprüngliche Antwort des Servers abgefragt werden, um etwaige von der API mitgelieferte Fehlermeldungen auszuwerten:

```

1 try {
2     $getResponse = $client->get("/get", [
3         "query" => ["foo" => "bar"],
4     ]);
5 } catch (ClientException $e) {

```

```
6  $getResponse = $e->getResponse();
7  }
8
9  try {
10     $postResponse = $client->post("/post", [
11         "form_params" => ["foo" => "bar"],
12     ]);
13 } catch (ClientException $e) {
14     $postResponse = $e->getResponse();
15 }
```

Auf diese Weise existiert in jedem Fall ein verwertbares Response-Objekt, mit dem die Applikation weiterarbeiten kann.

500er-Fehlercodes werden mit einer `ServerException` bedacht, die allgemeinere (und beide Fälle abdeckende) Exception ist eine `GuzzleException`.

11.4 Weiterführende Literatur

Das Thema REST APIs wird in der PHP-Literatur etwas spärlich behandelt. [49] widmet ihm nur zwei der über 1.000 Seiten. Auch in [41] wird das Thema REST nur aus Client-Seite betrachtet.

Mit [24] existiert ein aktuelles Buch, das sich prinzipiell mit dem Design von APIs (abseits von PHP) beschäftigt.

Eines der ausführlichsten Werke zum Thema AJAX bietet sicherlich [16], wenngleich es das Buch sich in seinen über 900 Seiten etwas verliert und viele Dinge abdeckt, für Einsteiger*innen eher nebensächlich sind. Es ist außerdem schon etwas in die Jahre gekommen und arbeitet noch mit `XmlHttpRequest` und nicht der `Fetch API`.

Ein praxisbezogenes, in Deutsch gehaltenes Buch ist [20]. Hier wird das Thema AJAX zusammen mit einigen umgebenden Themen wie Sicherheit oder auch Usability ergänzt.

Auch in den großen JavaScript-Kompendien wie etwa [2, 43] finden sich Abschnitte zum Thema AJAX, die durchaus zur Einführung geeignet und viel aktueller als alle Spezialwerke sind.

Vorlesung 12

Microframeworks und Testing

12.1 Einführung in Microframeworks

Wie bereits in Kapitel 5 erläutert wurde, hat sich die Art und Weise, mit PHP-Code zu arbeiten, in den letzten Jahren stark verändert. Statt Do It Yourself (DIY) Lösungen existieren große Frameworks, die einen großen Funktionsumfang mit sich bringen, oder auch Komponenten, die genau eine Aufgabe haben und für sehr spezifische Anwendungsfälle eingesetzt werden können. Dazwischen existiert jedoch noch eine weitere Ebene, die bis jetzt noch nicht erwähnt wurde: Microframeworks.

12.1.1 Definition

Ein *Microframework* ist ein minimalistisches Webframework, das im Vergleich zu den sogenannten Full-Stack-Frameworks (wie etwa Symfony, Laravel etc.) einen wesentlich geringeren Funktionsumfang bietet. Dieser wird jedoch in vielen Fällen gar nicht benötigt, etwa wenn nur ein kleines Projekt oder eine REST-API (im Sinne eines Web-Services) umgesetzt werden sollen.

Sogenannte Full-Stack Frameworks, im Englischen auch oft „Kitchen Sink Solution“ genannt (vom Ausspruch „everything but the kitchen sink“, also etwas, das so ziemlich alles umfasst) bieten in ihrem Umfang Lösungen für die verschiedensten Bereiche einer Webseite an. Dazu gehören u. a. Routing (erläutert in Kapitel 6), Logging (das Mitspeichern von Log-Informationen, ebenfalls in Kapitel 6 erwähnt), Datenbank-Management, Komponenten zur Authentifizierung (Login), Abstraktion des Dateisystems und noch viele mehr. Viele dieser Komponenten werden möglicherweise für ein Projekt gar nicht benötigt, das Full-Stack Framework sieht sich jedoch als Werkzeugkasten, der die Dinge einfach anbietet, auch wenn Sie nicht benötigt werden. Abbildung 12.1 stellt die Struktur eines solchen Frameworks schematisch dar.

Ein Microframework im Gegenzug enthält nur die notwendigsten Komponenten. Dies ist in der Regel das Routing, alle anderen Dinge sind nicht enthalten. Sie können jedoch als Dependencies (d. h. als Komponenten) ganz einfach nachinstalliert werden und werden dabei vom Microframework in einem eigenen Container verwaltet. Zusätzlich unterstützen viele Microframeworks noch sogenannte Middleware. Dies sind ebenfalls Komponenten, deren Aufgaben jedoch explizit darin bestehen, die Request- und Response-Objekte (die Anfrage und Ergebnis beinhalten) zu verändern. Abbildung 12.2

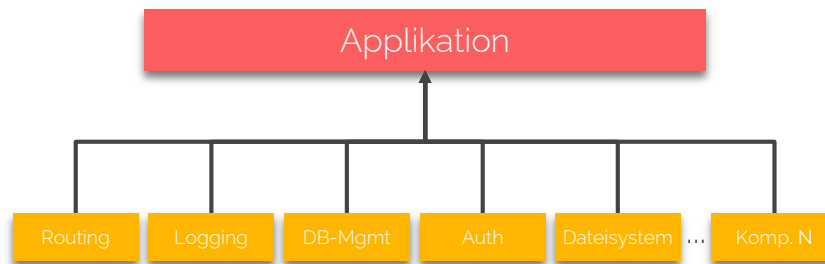


Abbildung 12.1: Ein Full-Stack Framework beinhaltet eine Vielzahl von verschiedenen Komponenten. Abbildung basierend auf [11].

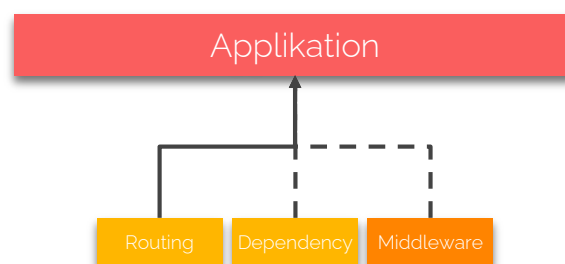


Abbildung 12.2: Das Microframework enthält im Kern nur das Routing. Andere Dependencies oder Middleware sind optional. Abbildung basierend auf [11].

stellt die grundsätzliche Architektur eines Microframeworks dar.

Anders ausgedrückt: Ein Microframework enthält von sich aus nur das Nötigste, um eine Webapplikation zu erstellen: das Empfangen eines HTTP-Requests, die Weiterleitung (Routing) dieses Requests an eine Callback-Methode (die dann die Response erstellt) und Rückgabe der HTTP-Response.

12.1.2 Vertreter

Es gibt eine Reihe von Microframeworks in PHP, diese erfüllen alle denselben Zweck, nämlich eine einfache Routing-Basis für eine Web-Applikation zu sein, ihr Funktionsumfang unterscheidet sich aber mitunter leicht. Im Folgenden werden einige Vertreter kurz beschrieben.

Slim Framework

Das *Slim Framework* wird in dieser Vorlesung im Detail in Abschnitt 12.2 behandelt.

Fat-Free Framework

Das *Fat-Free Framework*¹ (kurz F3) existiert seit 2009 und ist somit eines der ältesten Microframeworks. Es enthält Routing, Caching und Unterstützung für mehrsprachige

¹<https://fatfreeframework.com/>

Applikationen. Weitere Funktionalitäten können über eigene Plug-ins nachgerüstet werden, dazu zählen eine Template-Engine, ein Unit-Testing Toolkit oder auch Datenbank-Anbindungen. Der Quellcode des Frameworks hat nur 90 kB, was F3 zu einem der kleinsten Frameworks macht.

Mezzio

*Mezzio*² ist ein Microframework, das auf den Komponenten des Laminas (vormals Zend) Frameworks aufbaut und sich vor allem als Alternative für Entwickler*innen sieht, die mit diesem Full-Stack Framework bereits Erfahrungen haben. Umgekehrt lassen sich auch Anwendungen, die mit Mezzio erstellt wurden, leicht auf Laminas upgraden, falls dies erforderlich wird. Mezzio beinhaltet neben dem Routing Komponenten für Authentifikation und Autorisierung sowie Sessions.

Flight

*Flight*³ ist ein weiteres Microframework, das auf eine kompakte Codebasis setzt. Es besteht im Kern aus einem Router, besitzt jedoch auch Möglichkeiten zum Rendern des Outputs mittels PHP. Ebenfalls enthalten ist eine Möglichkeit des Cachings und sogenannte Filter, die vor und nach Funktionen aufgerufen werden können und Parameter und Output verändern.

12.2 Slim Framework

Das *Slim Framework*⁴ ist ein PHP Microframework zum effizienten Erstellen von Webapplikationen und APIs (Schnittstellen). Es besteht im Kern aus folgenden Funktionalitäten bzw. unterstützt die folgenden Dinge:

- **HTTP-Routing:** Der Router von Slim erlaubt es, Callback-Methoden zu spezifischen HTTP-Methoden (GET, POST etc.) und entsprechenden URL-Mustern zu erstellen, die aufgerufen werden, wenn der entsprechende Request die Webapplikation erreicht. Der Router unterstützt dabei fortgeschrittene Funktionalitäten wie Parameter oder Pattern-Matching.
- **Middleware:** Middleware-Komponenten können verwendet werden, um Request- und Response-Objekte zu verändern, z. B. um Autorisierung zu überprüfen, bevor ein Request beantwortet wird.
- **PSR-7 Unterstützung:** PSR-7⁵ ist ein Standard der PHP-FIG und definiert, wie Request- und Response-Objekte (HTTP-Messages) funktionieren sollen, um dies zu vereinheitlichen. Slim unterstützt dabei alle PSR-7-kompatiblen HTTP-Message-Implementierungen.
- **Dependency Injection:** Werden weitere Dependencies, d. h. Komponenten benötigt, können diese in einen Container geladen und damit verwaltet werden. Sie

²<https://docs.mezzio.dev/>

³<https://docs.flightphp.com/>

⁴<https://www.slimframework.com/>

⁵<https://www.php-fig.org/psr/psr-7/>

werden erst geladen, wenn sie benötigt werden. Jede Implementierung, die dem PSR-11-Standard für Container⁶ folgt, wird dabei unterstützt.

Die aktuelle Version ist 4.15.2 vom 22. Mai 2026.

12.2.1 Technische Voraussetzungen

Slim Framework benötigt wie alle PHP-Anwendungen einen Webserver, z. B. Apache oder nginx. Da die Routing-Funktionalität jedoch darauf baut, dass URLs aufgerufen werden, die nicht existieren (d. h. die auf kein existierendes Verzeichnis am Server zeigen), ist es nötig, dass der Webserver sogenanntes URL-Rewriting unterstützt. Dabei werden URL-Angaben, die am Webserver nicht existieren, über ein Set an Regeln zurück an die Hauptdatei (z. B. `index.php`) geleitet. Diese Datei kümmert sich dann darum, die angegebene Route zu verarbeiten und etwas Sinnvolles (oder auch eine Fehlerseite) anzuzeigen. Dies ist das bereits in Kapitel 6 angesprochene *Front-Controller-Pattern*.

Wie beim `fhoee/router` muss für den Apache Webserver das Modul `mod_rewrite` aktiviert sein und für das Web-Verzeichnis die Direktive `AllowOverride All` gesetzt sein. Dies erlaubt, dass die Datei `.htaccess` erstellt werden kann, die die entsprechenden Rewrite-Regeln enthält.

Die `.htaccess`-Datei für Slim gleicht der des `fhoee/router`:

```
1 <IfModule mod_rewrite.c>
2   RewriteEngine On
3
4   RewriteCond %{REQUEST_URI}::$1 ^(/.+)/(.*):\:2$
5   RewriteRule ^(.*) - [E=BASE:%1]
6
7   RewriteCond %{REQUEST_FILENAME} !-f
8   RewriteRule ^ index.php [QSA,L]
9 </IfModule>
```

Zunächst wird überprüft, ob `mod_rewrite` aktiv ist und falls ja, die Rewrite-Engine aktiviert. In den Zeilen 4 und 5 wird dann das Basisverzeichnis ermittelt, falls das Projekt nicht im Root des Webserver liegt. In Zeile 7 wird schließlich als Bedingung abgefragt, dass die Datei nicht existiert und in diesem Fall dann `index.php` in Zeile 8 aufgerufen. Das Argument `QSA` gibt an, dass GET-Parameters angehängt werden, `L` stoppt die Verarbeitung von Regeln nach der aktuellen.

Die Konfiguration der Rewrite-Rule für nginx kann der Slim-Dokumentation⁷ entnommen werden. Ebenso findet sich dort die Möglichkeit, mit dem eingebauten PHP-Webserver zu testen.

12.2.2 Hello World-Beispiel

Um eine Applikation mit Slim zu erstellen, bietet sich der Slim-Skeleton⁸ an, der – ähnlich wie der `fhoee/router-skeleton` – eine simple Ausgangsbasis darstellt. Für dieses Hello World-Beispiel wird jedoch komplett von vorne begonnen.

⁶<https://www.php-fig.org/psr/psr-11/>

⁷<https://www.slimframework.com/docs/v4/start/web-servers.html>

⁸<https://github.com/slimphp/Slim-Skeleton>

Verzeichnisstruktur

Zunächst wird ein Verzeichnis für das gesamte Projekt – in diesem Fall **helloworld** angelegt. Anschließend noch ein Unterverzeichnis **public**, in dem sich die aufzurufende(n) Datei(en), nämlich **index.php** und auch **.htaccess** befinden.

Einbinden von Slim

Danach wird im Verzeichnis **helloworld** Slim mittels Composer installiert:

```
1 $ composer require slim/slim
```

Dies erzeugt **composer.json**, **composer.lock** und das **vendor**-Verzeichnis mit den von Slim benötigten Dependencies. Es empfiehlt sich, stets die aktuelle Version des Frameworks verwenden (aktuell 4.15.2), da diese laufend Fehler- und Sicherheitskorrekturen erhält.

Eine PSR-7 Implementierung einbinden

PSR-7 ist der Standard, der HTTP-Messages (Requests und Responses) als Interfaces definiert. Slim baut auf diesem Standard auf und benötigt daher eine konkrete Implementierung dieser Interfaces. Verwendet werden kann dabei eine Reihe von Komponenten, einige davon sind in der Slim-Dokumentation⁹ aufgelistet. Slim bietet selbst eine Implementierung an, nämlich **slim/psr7**, die mit Composer im nächsten Schritt eingebunden wird:

```
1 $ composer require slim/psr7
```

Die Applikation erzeugen

In **public** wird zunächst eine **.htaccess**-Datei mit dem Inhalt aus Abschnitt 12.2.1 erzeugt.

Anschließend wird die Hauptdatei **index.php** im selben Verzeichnis angelegt und mit folgendem Inhalt befüllt:

```
1 use Slim\Factory\AppFactory;
2 use Psr\Http\Message\ResponseInterface as Response;
3 use Psr\Http\Message\ServerRequestInterface as Request;
4
5 require __DIR__ . '/../vendor/autoload.php';
6
7 $app = AppFactory::create();
8
9 $app->setBasePath("/wet2vo-examples/vo12/slim-framework/helloworld/public");
10 $app->addErrorMiddleware(true, false, false);
11
12 $app->get("/", function (Request $request, Response $response, array $args) {
13     $response->getBody()->write("Hello world!");
14     return $response;
15 });
16
17 $app->run();
```

⁹<https://www.slimframework.com/docs/v4/start/installation.html>

Importe: Um den Code lesbarer zu machen, werden zu Beginn die benötigten Namespaces mittels `use`-Direktive angegeben. Für Request und Response werden dabei die PSR-7-Interfaces (`ServerRequestInterface` bzw. `ResponseInterface`) importiert, so dass nicht gegen eine konkrete Implementierung, sondern gegen das Interface programmiert wird (lose Kopplung).

Mit dem Schlüsselwort `as` wird dabei ein *Alias* für den Namen des Interfaces vergeben: `use Psr\Http\Message\ResponseInterface as Response` macht den vollqualifizierten Namen unter der Kurzbezeichnung `Response` verfügbar. Ein Alias ist rein lokal und ändert nichts an der Klasse selbst – er dient nur dazu, lange Namen abzukürzen oder Namenskonflikte zwischen gleichnamigen Klassen aus unterschiedlichen Namespaces aufzulösen. In den folgenden Beispielen kann dadurch die kürzere Schreibweise `Request` bzw. `Response` in den Methoden-Signaturen verwendet werden. Im Anschluss folgt die Einbindung des Composer-Autoloaders.

App-Objekt erzeugen: Der Einstiegspunkt in eine Slim-Applikation ist ein Objekt der Klasse `\Slim\App`. Die Applikation verwaltet primär die Routen, d.h. die möglichen URL-Pfade, unter denen Inhalte zurückgeliefert werden und beinhaltet den Container, der Dependencies und auch Einstellungen verwaltet. Ein App-Objekt wird jedoch nicht direkt mit `new` angelegt, sondern über eine sogenannte Factory (Deutsch auch Fabrik-methode genannt) erzeugt.

Base-Path angeben: Soll eine Slim-Applikation in einem Unterverzeichnis des Document-Roots laufen, muss dieses – wie auch schon vom `fhooe/router` bekannt – als Base-Path angegeben werden. Angenommen, das Document-Root ist (wie unter vielen Linux-Distributionen üblich) unter `var/www/html` zu finden und die Datei `index.php` soll im Unterordner `wet2vo-examples/vo12/slim-framework/helloworld/public` liegen, so wird dies wie folgt angegeben:

```
1 $app->setBasePath("/wet2vo-examples/vo12/slim-framework/helloworld/public");
```

Fehlerbehandlung aktivieren: Damit aussagekräftige Fehlermeldungen ausgegeben werden, wird die Error-Middleware registriert. In Slim 4 ist diese – anders als in früheren Versionen – nicht mehr standardmäßig aktiv; ohne sie führt eine geworfene Exception oder ein nicht gefundener Pfad zu einer (unschönen) PHP-Fehlermeldung.

```
1 $app->addErrorMiddleware(true, false, false);
```

Der erste Parameter aktiviert die Anzeige der Fehlerdetails, die beiden weiteren steuern das Logging der Fehler bzw. der Fehlerdetails. Die Anzeige der Details (`true`) ist ausschließlich für die Entwicklung vorgesehen; in Produktion ist dieser Wert auf `false` zu setzen, um keine internen Informationen (Pfade, Stacktraces) preiszugeben. Eine ausführlichere Betrachtung der Middleware folgt in Abschnitt 12.2.4.

Route definieren: Um etwas anzuzeigen, wird mindestens eine Route benötigt. In Slim stehen in der `\Slim\App`-Instanz dazu Methoden für jede HTTP-Methode zur Verfügung. Die wichtigste ist `get()`, die es ermöglicht, auf einen GET-Request unter einem bestimmten URL zu reagieren. In diesem Fall wird mit `/` spezifiziert, dass direkt das Verzeichnis, in dem die Datei `index.php` liegt, gemeint sein soll. Der zweite Parameter ist

die Callback-Methode, die aufgerufen wird, wenn ein GET-Request mit dem passenden URL eintrifft. Diese Methode enthält immer drei Parameter:

- **\$request**: Das Request-Objekt, welches alle Daten des aktuellen Requests enthält.
- **\$response**: Das Response-Objekt, welches an den Client zurückgegeben werden soll. Es kann (und soll) hier verändert werden.
- **\$args**: Ein Array mit benannten Platzhaltern der Route. Siehe dazu Abschnitt 12.2.5.

Um die „Hello world!“ Ausgabe zu erzeugen, wird das Response-Objekt benötigt. Über `getBody()` wird auf den Body, d. h. den Inhalt dieses Objekts zugegriffen und mittels `write()` der Text „Hello world!“ geschrieben. Anschließend wird das Response-Objekt zurückgegeben.

Applikation ausführen: Um die Applikation zu starten, muss schließlich die Methode `run()` des App-Objekts aufgerufen werden. Sie setzt das Routing (und andere Funktionalitäten) in Gang.

12.2.3 Dependencies

Um nun mit Slim praktisch arbeiten zu können, sollen zwei Dependencies (zusätzliche Softwarekomponenten) hinzugefügt werden. Zunächst sollen die Logging-Komponente *Monolog* verwendet werden, um einfach Debug-Ausgaben in eine Log-Datei (unabhängig von der des Webservers) schreiben zu können, weiters sollen *Latte*-Templates zum Einsatz kommen, um komplexere HTML-Ausgaben als im Hello-World-Beispiel machen zu können.

Dazu wird eine zweite Slim-Applikation namens `slimexample` angelegt. Zunächst wird der gleichnamige Ordner erstellt und Slim sowie die PSR-7-Komponente wie beim Hello World-Beispiel mit Composer installiert:

```
1 $ composer require slim/slim slim/psr7
```

In der Datei `public › index.php` wird nun – anders als beim Hello-World-Beispiel – zunächst ein Dependency-Container erstellt und konfiguriert, bevor das App-Objekt erzeugt wird. Der Grund dafür wird im nächsten Abschnitt deutlich: Der Container muss der `AppFactory` bekannt sein, *bevor* diese die Applikation erzeugt.

Dependency Injection Container

Um Dependencies einzubinden, benötigt Slim einen sogenannten Dependency Container. Einzubindende Komponenten werden zunächst in diesem Container registriert und konfiguriert. Wenn sie dann im Code benötigt werden, werden sie vom Container erzeugt und bereitgestellt.

Slim stellt selbst keinen Container bereit, kann aber mit jeder Komponente, die den PSR-11 Standard für Dependency Injection Container implementiert, arbeiten. Eine bekannte Dependency-Container-Komponente, die dieses Kriterium erfüllt, ist PHP-DI¹⁰. Diese wird zunächst mit Composer eingebunden:

```
1 $ composer require php-di/php-di
```

¹⁰<https://php-di.org/>

Container erzeugen: Um einen Dependency-Container zu erzeugen, existiert die Klasse `\DI\ContainerBuilder`. Sie erlaubt es, den Container zunächst zu konfigurieren und ihn dann mit einer Factory-Methode zu erzeugen. Anschließend wird der so erstellte Container der `AppFactory` mittels `setContainer()` übergeben. Dies muss vor dem Erzeugen des App-Objekts geschehen:

```
1 $containerBuilder = new ContainerBuilder();
2 // Definitionen werden hier hinzugefügt (siehe unten)
3 $container = $containerBuilder->build();
4 AppFactory::setContainer($container);
5
6 $app = AppFactory::create();
7 $app->setBasePath("/wet2vo-examples/vo12/slim-framework/slimexample/public");
```

Einstellungen hinterlegen: In diesen Container können nun verschiedene Dinge hinterlegt werden. Im einfachsten Fall sind dies simple Werte in Form eines assoziativen Arrays. Dies wird gerne verwendet, um Einstellungen der Applikation oder auch einzelner Dependencies zentral zu hinterlegen.

Dazu werden beim `ContainerBuilder`-Objekt sogenannte Definitionen (engl. „definitions“) in Form eines assoziativen Arrays hinzugefügt. Diese Definitionen erzeugen einen Eintrag im Container-Objekt unter einem frei wählbaren Namen. Dies kann ein beliebiger String sein, für die Einstellungen lautet der Eintrag dabei oft `settings`. Der Wert ist in der Regel ein assoziatives Array mit den verschiedenen Einstellungen.

```
1 $containerBuilder->addDefinitions(
2     [
3         "settings" => [
4             "settingName" => "settingValue",
5         ],
6     ],
7 );
```

Im Folgenden werden für die Applikation drei Einstellungen hinterlegt, die später für die Fehlerbehandlung in Abschnitt 12.2.4 herangezogen werden. Diese sind die drei Werte der Error-Middleware, die auch schon beim Hello World-Beispiel verwendet wurden: Ob Fehlermeldungen im Browser ausgegeben werden, ob Fehler (in eine Datei) geloggt werden sollen und ob Details ins Log geschrieben werden sollen. Alle drei Einstellungen werden zunächst als aktiv gesetzt.

```
1 $containerBuilder->addDefinitions(
2     [
3         "settings" => [
4             "displayErrorDetails" => true,
5             "logErrors" => true,
6             "logErrorDetails" => true,
7         ],
8     ],
9 );
```

Wichtig ist dabei, dass alle Einstellungen Teil *einer* einzigen `settings`-Definition sind. Die folgenden Ausschnitte zeigen jeweils nur die hinzukommenden Einträge innerhalb desselben Arrays. Ein erneuter Aufruf von `addDefinitions()` mit dem Schlüssel `settings` würde die zuvor gesetzten Werte überschreiben, da PHP-DI gleichnamige Definitionen nicht zusammenführt, sondern ersetzt.

Dependencies registrieren: Die Hauptaufgabe des Containers ist es jedoch, Dependencies als Service zu registrieren. Auch hier wird eine Definition hinzugefügt, als Name wird jedoch in der Regel der Klassenname der Dependency verwendet, da so automatisch erkannt werden kann, wenn eine Instanz benötigt wird. Unter diesem Namen (den man einfach durch das Anhängen der Klassenkonstante `::class` an den Klassennamen als String erhält) wird nun eine Funktion (genannt Closure) hinterlegt. In dieser wird die Dependency angelegt und zurückgegeben. Der Container ruft diese Funktion zur Erstellung der Dependency dann auf, wenn sie später das erste Mal benötigt wird.

Um auf den Container zugreifen zu können, um etwa Einstellungen zu entnehmen, wird dieser als Parameter mitgegeben. Über die Methode `get()` können die Werte unter dem hinterlegten Schlüssel entnommen werden.

```
1 $containerBuilder->addDefinitions(
2     [
3         MyService::class => function (ContainerInterface $container) {
4             $settings = $container->get("settings");
5             return new MyService($settings);
6         },
7     ],
8 );
```

Logger-Dependency: Monolog

Während der Entwicklung einer Applikation aber auch im Betrieb ist gutes Logging ein praktisches Tool, um Informationen über den Verlauf der Applikation zu erhalten. Nicht immer kann ein Debugger eingesetzt werden bzw. ist praktisch. *Monolog* ist das bekannteste Logging-Framework. Es basiert auf dem PSR-3 Logging Standard und lässt sich einfach über Composer installieren:

```
1 $ composer require monolog/monolog
```

Grundeinstellungen für den Logger (Name, Pfad der Log-Datei und Minimum-Log-Level) werden im `settings`-Eintrag des Containers unter dem Schlüssel `logger` hinterlegt:

```
1 $containerBuilder->addDefinitions(
2     [
3         "settings" => [
4             // Einstellungen zum Errorhandling
5             "logger" => [
6                 "name" => "slimexample-logger",
7                 "path" => "../logs/app.log",
8                 "level" => Level::Debug,
9             ],
10        ],
11    ],
12 );
```

Nun wird, wie soeben beschrieben, eine Definition mit dem Namen des LoggerInterfaces im Container angelegt. Hier wird bewusst nicht der Klassenname von Monolog sondern das generische PSR-3 Interface verwendet, um den Logger austauschbar zu machen. Im Code muss später nur angegeben werden, dass eine Instanz des LoggerInterfaces angelegt werden soll. Welche konkrete Implementierung dahinter liegt, spielt

keine Rolle. Dies ist wiederum lose Kopplung (engl. „loose coupling“), was ein erstrebenswertes Ziel in der Softwareentwicklung ist.

```

1 $containerBuilder->addDefinitions(
2   [
3     // Einstellungen wie zuvor beschrieben
4     LoggerInterface::class => function (ContainerInterface $container) {
5       $settings = $container->get("settings");
6       $loggerSettings = $settings["logger"];
7
8       $logger = new Logger($loggerSettings["name"]);
9
10      $fileHandler = new StreamHandler($loggerSettings["path"],
11      $loggerSettings["level"]);
12      $logger->pushHandler($fileHandler);
13
14      return $logger;
15    },
16  );

```

Zunächst werden aus dem Container die Einstellungen für den Logger abgefragt. Der Container wird dabei in der Closure als Parameter `$container` mitgegeben. Dann wird eine neue `\Monolog\Logger` Instanz angelegt – der Name wird aus den Einstellungen übernommen.

Um den Zielort bzw. die Art der Ausgabe zu definieren, kommt ein Objekt der Klasse `\Monolog\Handler\StreamHandler` zum Einsatz. Dies ermöglicht das Loggen in eine Datei (Dateien liegen sogenannten Streams – Datenströme – zugrunde). Als Argumente werden der Pfad und das Log-Level für jeden Eintrag mitgegeben. Im Anschluss wird dieser Handler der Logger-Instanz zugewiesen und das fertige Logger-Objekt zurückgegeben.

Sobald diese Zuweisung aktiv ist, kann über den Eintrag im Container mittels `get()` auf die Logging-Instanz zugegriffen werden. Dabei wird diese erst angelegt, wenn sie das erste Mal benötigt wird und belegt somit nicht unnötig Ressourcen.

```

1 $logger = $container->get(LoggerInterface::class);
2 $logger->info("Logger dependency created.");

```

Dies erzeugt (falls noch nicht vorhanden) eine Datei `logs › app.log` und erstellt folgenden Eintrag:

```
[2026-06-23T06:50:00+00:00] slimexample-logger.INFO: Logger dependency created. [] []
```

Derartige Log-Aufrufe können an allen wichtigen Stellen eingebaut werden und somit bei Problemen schnell Aufschluss über die Ursache geben. Monolog bietet noch eine Vielzahl an weiteren Funktionalitäten. Diese können aus der Dokumentation¹¹ entnommen werden.

In Abschnitt 12.2.4 wird dieser Logger ebenfalls eingesetzt, um Fehlermeldungen in die Logdatei zu schreiben.

¹¹<https://github.com/Seldaek/monolog/blob/main/doc/01-usage.md>

Renderer: Latte

Wird das Slim Framework nicht für eine REST-API die nur Daten retour liefert, sondern zum Erstellen einer kompletten Webapplikation verwendet, ist die Verwendung eines Renderers, d. h. eines Template-Systems, unumgänglich. Dieser Renderer ermöglicht die Ausgabe komplexer HTML-Dokumente, die allerdings nicht direkt ausgegeben, sondern wieder in ein Response-Objekt verpackt werden müssen.

Slim selbst bietet mit `slim/php-view` eine einfache Komponente zur Ausgabe mit reinem PHP an: HTML-Seiten werden mit PHP-Echo-Tags erstellt, und Slim übergibt diesen Seiten die Daten, die dann ausgegeben werden. Für komplexere Anwendungen empfiehlt sich jedoch eine vollwertige Template-Engine. In dieser Vorlesung kommt wie immer *Latte*¹² zum Einsatz. Die Integration in Slim folgt der offiziellen Anleitung von Daniel Opitz¹³.

Zunächst wird Latte mittels Composer installiert:

```
1 $ composer require latte/latte
```

Dann werden im Container für das Template-System entsprechende Einstellungen hinterlegt:

```
1 $containerBuilder->addDefinitions(
2     [
3         "settings" => [
4             // Einstellungen für Errorhandling und Logger
5             "template" => [
6                 "path" => "../templates",
7                 "cache" => "../cache",
8                 "auto_refresh" => true,
9             ],
10    ],
11 ],
12 );
```

Unter dem Schlüssel `template` werden der Pfad zu den Templates, das Cache-Verzeichnis für die kompilierten Templates und der Status des Auto-Refresh angegeben.

Damit aus den Templates heraus URLs anhand von Routennamen erzeugt werden können (wie weiter unten bei der Formularverarbeitung gezeigt), benötigt die Latte-Engine den *RouteParser* von Slim. Dieser übersetzt einen Routennamen in den zugehörigen Pfad und stellt dabei automatisch den Base-Path der Applikation voran. Der *RouteParser* stammt aus dem App-Objekt und wird – nach dessen Erstellung – im Container hinterlegt, sodass die Engine-Definition später darauf zugreifen kann:

```
1 $container->set("routeParser", $app->getRouteCollector()->getRouteParser());
```

Im Anschluss wird die Latte-Engine im Container als Service registriert. In der Definition wird zusätzlich über `addFunction()` eine eigene Latte-Funktion `url_for` hinterlegt, die einen Routennamen entgegennimmt und daraus den vollständigen Pfad (inklusive Base-Path) erzeugt:

```
1 $containerBuilder->addDefinitions(
2     [
3         // Einstellungen und Logger-Dependency
```

¹²<https://latte.nette.org/>

¹³<https://latte.nette.org/en/cookbook/slim-framework>

```

4     Engine::class => function (ContainerInterface $container) {
5         $settings = $container->get("settings");
6         $templateSettings = $settings["template"];
7
8         $latte = new Engine();
9         $latte->setLoader(new FileLoader($templateSettings["path"]));
10        $latte->setCacheDirectory($templateSettings["cache"]);
11        $latte->setAutoRefresh($templateSettings["auto_refresh"]);
12
13        $routeParser = $container->get("routeParser");
14        $latte->addFunction("url_for",
15            fn(string $name, array $data = [], array $query = []): string
16                => $routeParser->urlFor($name, $data, $query),
17        );
18
19        return $latte;
20    },
21    ],
22 );

```

Aus dem Container werden zunächst die Einstellungen abgerufen. Mit ihnen wird eine `\Latte\Engine`-Instanz konfiguriert: Der `FileLoader` legt das Template-Verzeichnis fest, `setCacheDirectory()` das Verzeichnis für die kompilierten (und damit schnellen) Templates und `setAutoRefresh()` steuert, ob Latte den Cache bei Änderungen automatisch neu erzeugt – im Produktivbetrieb wird dies aus Performance-Gründen deaktiviert.

Abschließend wird über `addFunction()` die Funktion `url_for` in der Engine registriert. Sie steht damit in allen Templates (wie beim *foo/router-skeleton*) zur Verfügung und übersetzt einen Routennamen mithilfe des `RouteParsers` in den zugehörigen Pfad. Auf diese Weise bleibt die URL-Erzeugung an einer einzigen Stelle gebündelt und muss nicht in jedem Routing-Callback wiederholt werden. Da die Engine-Definition eine Closure ist, die erst beim ersten Rendern ausgewertet wird, sind die zuvor mit `$container->set()` hinterlegten Werte zu diesem Zeitpunkt bereits vorhanden.

Die Engine-Instanz allein genügt jedoch noch nicht, um direkt ein PSR-7-Response-Objekt zu erzeugen. Gemäß der offiziellen Anleitung wird daher eine schlanke Hilfsklasse `TemplateRenderer` erstellt, die das gerenderte Template in das Response-Objekt schreibt. Sie wird in `src▶Renderer▶TemplateRenderer.php` abgelegt:

```

1  <?php
2
3  namespace App\Renderer;
4
5  use Latte\Engine;
6  use Psr\Http\Message\ResponseInterface;
7
8  final class TemplateRenderer
9  {
10     public function __construct(
11         private Engine $engine,
12     ) {}
13
14     public function template(ResponseInterface $response, string $template, array
15         $data = []): ResponseInterface {
16         $output = $this->engine->renderToString($template, $data);

```

```
16     $response->getBody()->write($output);
17
18     return $response;
19 }
20 }
```

Die Klasse erhält im Konstruktor die `Engine`-Instanz. Da diese im Container registriert ist, erzeugt PHP-DI die `TemplateRendererer`-Instanz bei Bedarf automatisch (Autowiring) – eine eigene Definition oder eine zusätzliche Middleware ist nicht nötig. Damit die Klasse über Composer gefunden wird, ist das `src`-Verzeichnis per PSR-4 in der `composer.json` einzutragen:

```
1 "autoload": {
2   "psr-4": {
3     "App\\": "src/"
4   }
5 }
```

Nach dem Eintragen wird der Autoloader mit `composer dump-autoload` aktualisiert.

Um ein Template zu rendern, wird die `TemplateRendererer`-Instanz aus dem Container angefordert und ihre Methode `template()` aufgerufen. Diese rendert das Latte-Template und schreibt das Ergebnis in das Response-Objekt:

```
1 return $this->get(TemplateRendererer::class)->template($response, "template.latte",
    ["param1" => "value1"]);
```

Innerhalb eines Routing-Callbacks verweist `$this` dabei auf den Container. Die Anwendung dieses Aufrufs wird in Abschnitt 12.2.5 im Rahmen verschiedener Routen demonstriert. Latte escaped übergebene Werte standardmäßig kontextsensitiv, was einen wirksamen Schutz vor Cross-Site Scripting (XSS) darstellt.

Die Beispiele setzen voraus, dass am Beginn von `index.php` die jeweiligen Namespaces `\Latte\Engine`, `\Latte\Loaders\FileLoader`, `\Slim\Routing\RouteContext` und `\App\Renderer\TemplateRendererer` per `use` importiert wurden.

12.2.4 Middleware

Slim unterstützt sogenannte Middleware. Dies ist Code, der sowohl vor als auch nach dem Start der Slim-App ausgeführt wird und dabei die Request- und Response-Objekte auslesen oder verändern kann. Somit könnte z. B. ein automatischer Schutz vor Cross-Site Request Forgery (CSRF) bei jedem Request implementiert oder jeder Request automatisch authentifiziert werden. Auch Caching ist möglich.

Middleware in Slim funktioniert dabei nach dem Zwiebel-Prinzip. Hinzugefügte Middleware wird dabei wie Ringe um die eigentliche Applikationslogik gelegt. Beim Request werden zuerst die Middlewareschichten durchlaufen, dann wird die Applikationslogik zur Generierung der Response ausgeführt und danach wird die Middleware wieder in umgekehrter Reihenfolge ausgeführt. Abbildung 12.3 stellt dies dar.

Technisch basiert Middleware in Slim auf dem Standard PSR-15¹⁴. Eine Middleware kann entweder als Closure umgesetzt werden oder als Klasse, die das Interface

¹⁴<https://www.php-fig.org/psr/psr-15/>

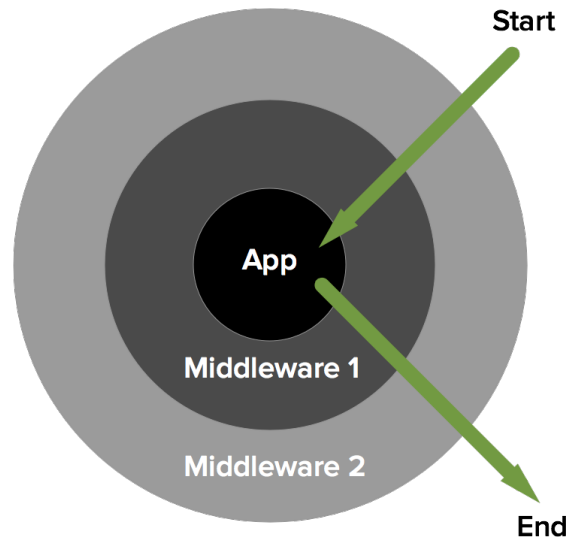


Abbildung 12.3: Middleware wird vor der Applikationslogik von außen nach innen (LIFO – last in, first out) und nach der Applikationslogik erneut von innen nach außen ausgeführt. Quelle: [26].

`Psr\Http\Server\MiddlewareInterface` implementiert. Dieses verlangt eine Methode `process()`, die das Request-Objekt sowie einen Request-Handler erhält. Innerhalb dieser Methode kann der Request verändert, mit `$handler->handle($request)` an den nächsten Handler weitergereicht und die zurückkommende Response anschließend nachbearbeitet werden.

Eine Middleware, die Caching übernimmt, analysiert den ankommenden Request. Ist eine gecachte Antwort vorhanden, wird diese sofort zurückgegeben und die Applikation wird nicht mehr betreten. Gab es keine gecachte Version, wird die Response in der Applikation erstellt und danach speichert die Middleware die Response nach dem Verlassen der App im Cache, sodass sie beim nächsten Aufruf zur Verfügung steht.

Ein weiteres Einsatzgebiet ist Error-Handling: um PHP Exceptions abzufangen, kann eine mit Slim mitgelieferte Error-Handling-Middleware registriert werden.

Middleware wird im Gegensatz zu Dependencies zum fertigen App-Objekt hinzugefügt. Für das Error-Handling geschieht dies mittels `addErrorMiddleware()`. Diese Methode hat drei verpflichtende und einen optionalen booleschen Parameter:

- `$displayErrorDetails`: Zeigt Fehlermeldungen an. `true` während der Entwicklung, `false` im Produktivbetrieb.
- `$logErrors`: Aktiviert das Logging von Fehlern.
- `$logErrorDetails`: Zeigt Details im Fehlerlog an.
- `$logger`: Optional kann im vierten Parameter eine Instanz eines PSR-3 kompatiblen Loggers angegeben werden.

Die Werte der ersten drei Einstellungen wurden bereits im Container unter `settings` (jeweils mit dem Wert `true`) hinterlegt, können also von dort abgefragt werden. Als Logger wird das Monolog-Objekt verwendet, das ebenfalls schon über den Container angefordert und erzeugt wurde.

```
1 $settings = $container->get("settings");
2 $app->addErrorMiddleware(
3     $settings["displayErrorDetails"],
4     $settings["logErrors"],
5     $settings["logErrorDetails"],
6     $logger,
7 );
8 $logger->info("Error middleware added.");
```

Tritt nun ein Fehler auf, so wird dieser am Bildschirm angezeigt und ebenfalls mit Details in die Logdatei geschrieben.

Weitere Einstellungen für die Error-Middleware sind in der Slim-Dokumentation¹⁵ einsehbar. Darüber hinaus gibt es noch weitere Middleware, die mit Slim mitgeliefert wird, wie das Routing, Output-Buffering oder Bestimmung der Content-Length.

12.2.5 Routing

Das Herzstück jeder Slim-Applikation sind ihre Routen. Diese bestimmen, welche Pfade in der Applikation aufgerufen werden können und entsprechende Ergebnisse liefern. Eine Route besteht aus einem Protokoll (GET, POST, PUT, DELETE, OPTIONS, PATCH), einem URI (d. h. URL) und einer Callback-Methode, die aufgerufen wird, wenn ein Request auf diese Route eingeht. Die HTTP-Methode wird über eine von sechs Methoden des App-Objekts ausgewählt, URI und Callback sind deren Parameter. Für einen GET-Request sieht dies für das Hello-World-Beispiel wie folgt aus:

```
1 $app->get("/", function (Request $request, Response $response, array $args) {
2     $response->getBody()->write("Hello World!");
3     return $response;
4 });
```

Die Methode heißt `get()` und reagiert damit nur auf eingehende GET-Requests. / legt fest, dass das Root-Verzeichnis, in dem sich die aufrufende Datei befindet, gemeint ist. Angenommen, der Code liegt im Docker-Container in der Datei `index.php` im Ordner `var/www/html/wet2vo-examples/vo12/slim-framework/slimexample/public`, dann reagiert diese Route auf `http://localhost:8080/wet2vo-examples/vo12/slim-framework/slimexample/public/`. Voraussetzung ist ein korrekt gesetzter Base-Path.

Der Callback hat immer drei Argumente: das Request-Objekt, aus dem alle Details des Requests ausgelesen werden können, das Response-Objekt, in das die Response geschrieben werden muss und benannte Argumente, falls das Latte-Template dieser Route Platzhalter enthält. Die Request- und Response Objekte sollten, wie schon gezeigt, nicht die konkreten Implementierungen (etwa von `\Slim\Psr7`) verwenden, sondern nur auf das jeweilige PSR-Interface (`Psr\Http\Message\ResponseInterface` bzw. `Psr\Http\Message\ServerRequestInterface`) verwenden. Somit ist es jederzeit möglich, die PSR-7 Implementierung zu tauschen (lose Kopplung).

Im Callback selbst kann mittels `getBody()` der Inhalt der Response abgefragt und mittels `write()` beschrieben werden. Auch mittels `echo` ausgegebene Inhalte werden standardmäßig an die Response gehängt. Dann wird das Response-Objekt zurückgegeben und von Slim angezeigt.

¹⁵<https://www.slimframework.com/docs/v4/middleware/error-handling.html>

Durch das Hinzufügen der Slim Routing Middleware kann beim Routing Einfluss auf die Reihenfolge der Middleware genommen werden. Da sie für einige andere Middlewares verwendet wird, empfiehlt sich das Hinzufügen:

```
1 $app->addRoutingMiddleware();
```

Routing für eine Formularverarbeitung

Nachfolgend soll ein simples Routing für Formularverarbeitung erstellt werden. Dazu wird eine GET-Route benötigt, die das Formular anzeigt und eine POST-Route, die das Ergebnis des mit POST versendeten Formulars darstellt. Dargestellt wird alles mittels Latte-Templates, die wie in Abschnitt 12.2.3 gezeigt initialisiert wurden.

Zunächst wird eine GET-Route auf das Root-Verzeichnis definiert. Wird sie aufgerufen, soll das Formular – gespeichert in `templates/form.latte` – angezeigt werden. Auf den Container kann im Routing-Callback direkt über `$this->get()` zugegriffen werden, somit kann der `TemplateRenderer` angefordert und das Template gerendert werden. Es werden dabei keine Daten übergeben: Das Formularfeld bleibt leer, weil das Template über `{parameters}` einen Default für den Platzhalter deklariert (siehe unten), und den Pfad für das `action`-Attribut erzeugt das Template selbst über die Funktion `url_for`.

```
1 $app->get("/", function (Request $request, Response $response, array $args) {
2     return $this->get(TemplateRenderer::class)->template($response, "form.latte");
3 });
```

Das Template sieht wie folgt aus, die Ausgabe im Browser ist in Abbildung 12.4 ersichtlich:

```
1 {parameters string $placeholder = ''}
2 <!DOCTYPE html>
3 <html lang="en">
4     <head>
5         <meta charset="UTF-8">
6         <title>Tell Me Your Name</title>
7     </head>
8     <body>
9         <h1>Tell Me Your Name</h1>
10        <form action="{url_for('result')}" method="post">
11            <label for="name">Name:</label>
12            <input type="text" id="name" name="name" value="{ $placeholder }">
13            <button type="submit">Send</button>
14        </form>
15    </body>
16 </html>
```

Hinweis zum Template: Der `parameters`-Tag am Beginn deklariert die vom Template erwartete Eingabevariable `$placeholder` samt Typ und Default-Wert. Dadurch muss die GET-Route auf / keinen Platzhalter übergeben – fehlt er, greift der Default (leerer String). Ohne diese Deklaration würde der Zugriff auf eine nicht übergebene Variable eine PHP-Warnung auslösen.

Im Template erzeugt die Funktion `{url_for('result')}` den Pfad zur Route mit dem Namen `result`. Diese Funktion wurde – wie in Abschnitt 12.2.3 gezeigt – in der Latte-Engine registriert und greift intern auf den `RouteParser` zu. Der Vorteil: Ändert

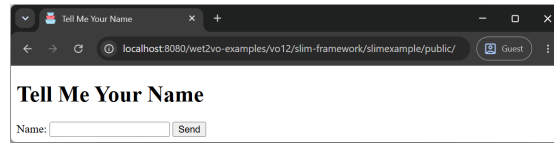


Abbildung 12.4: Die Slim-Applikation zeigt beim Aufruf des Root-Verzeichnisses das Formular an.

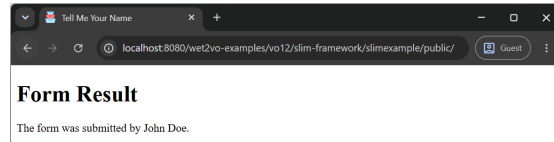


Abbildung 12.5: Das Ergebnis der Formularverarbeitung zeigt den eingegebenen Namen (John Doe) an.

sich der konkrete Pfad der Route, muss weder das Template noch der Handler angepasst werden. Die benannte Route `result` ist die POST-Route, die das Ergebnis verarbeitet:

```
1 $app->post("/", function (Request $request, Response $response, array $args) {
2     $data = $request->getParsedBody();
3     $name = $data["name"];
4
5     return $this->get(TemplateRenderer::class)->template($response, "result.latte",
6         ["name" => $name]);
7 }->setName("result");
```

Die Route zeigt auch auf das Root-Verzeichnis, wird jedoch nur aufgerufen, wenn mittels POST übertragen wird (was im `method`-Attribut des Formulars festgelegt ist). Mit Hilfe von `getParsedBody()` wird zunächst der gesamte Body als Array in `$data` gespeichert. Anschließend wird Parameter `name` über `$data["name"]` ausgelesen, dies entspricht einem Zugriff auf `$_POST["name"]`. Dieser Wert wird als Parameter ans Template `result.latte` übergeben, wo er angezeigt wird. Um die Route wie oben gezeigt über `url_for` referenzieren zu können, wird ihr mit `setName()` ein Name zugewiesen. Dies hat den Vorteil, dass die Route geändert werden kann (sie könnte z. B. auch `/result` lauten) und das Template enthält automatisch den korrekten Pfad.

Das Resultat-Template ist wie folgt aufgebaut und rendert wie in Abbildung 12.5 dargestellt.

```
1 <!DOCTYPE html>
2 <html lang="en">
3   <head>
4     <meta charset="UTF-8">
5     <title>Tell Me Your Name</title>
6   </head>
7   <body>
8     <h1>Form Result</h1>
9     <p>The form was submitted by {$name}.</p>
10  </body>
11 </html>
```




Abbildung 12.6: Wird die Seite mit „John%20Doe“ nach dem Root-Slash aufgerufen, wird dieser Name ausgelesen und an das Template übergeben. Dort wird es als `value` des Eingabefeldes gesetzt, sodass der Name dort aufscheint.

Platzhalter in der Route

Routen müssen nicht auf fixe URL-Pfade lauten, stattdessen können auch Platzhalter angegeben werden, die beliebige Werte zulassen. Diese Werte werden im Callback im dritten Parameter `$args` mitübergeben. Um einen Platzhalter zu definieren, wird dieser unter Angabe eines Namens in geschwungene Klammern gesetzt. Im Array `$args` ist er dann unter diesem Namen verfügbar.

Im folgenden Beispiel wird für das Formular noch eine zweite GET-Route definiert, die es erlaubt, die URL mit einem beliebigen Platzhalter aufzurufen. Dieser wird ausgelesen und ins Template übergeben, wo er ins `value`-Attribut für das Formularfeld gesetzt wird. Somit ist es möglich, durch Aufrufen eines eigenen URLs das Formularfeld quasi vor auszufüllen. Die Route dazu sieht wie folgt aus:

```
1 $app->get(
2    ("/{placeholder}[/]",
3     function (Request $request, Response $response, array $args) {
4         return $this->get(TemplateRenderer::class)->template($response, "form.latte",
5             ["placeholder" => $args["placeholder"]]);
6     });
```

Als Schlüssel dient der Name `placeholder`. Sein Wert kann beliebig sein und wird dem Template als Parameter `placeholder` übergeben. Der abschließende Schrägstrich ist durch Angabe in eckigen Klammern optional, d. h. die Route funktioniert sowohl mit als auch ohne. Abbildung 12.6 stellt das Ergebnis des Formulars dar, wenn die Seite mit `https://localhost:8080/wet2vo-examples/vo12/slim-framework/slimexample/public/John%20Doe` aufgerufen wurde.

Die als Platzhalter erlaubten Zeichen können hierbei etwa auch mit Regular Expressions eingeschränkt werden. Details finden sich in der Dokumentation¹⁶.

Auftrennen der einzelnen Schritte im Slim-Skeleton

Vorerst wurden die Einstellungen (`settings`), das Konfigurieren des Containers (`dependencies injection`) und das Erstellen der Routen (`routing`) in der Datei `index.php` durchgeführt, das als Startpunkt für die Webseite fungiert.

Bei der Verwendung des Slim-Skeletons sind diese Schritte in einzelne PHP-Dateien in einem Unterverzeichnis `app` aufgeteilt und nur noch über `require` in die `index.php` eingebunden.

Die Datei `index.php` im Verzeichnis `public` ändert sich dadurch für alle auf dem Slim-Skeleton basierenden Projekte auf wie folgt (Anmerkung: der Slim-Skeleton enthält

¹⁶<https://www.slimframework.com/docs/v4/objects/routing.html#route-placeholders>

noch mehr Beispielcode, daher sind noch weitere Dinge in der Hauptdatei eingebunden, die aber vorerst nicht weiter wichtig sind):

```
1 <?php
2 declare(strict_types=1);
3
4 // use-Statements
5
6 require __DIR__ . '/../vendor/autoload.php';
7
8 // Instantiate PHP-DI ContainerBuilder
9 $containerBuilder = new ContainerBuilder();
10
11 if (false) { // Should be set to true in production
12     $containerBuilder->enableCompilation(__DIR__ . '/../var/cache');
13 }
14
15 // Set up settings
16 $settings = require __DIR__ . '/../app/settings.php';
17 $settings($containerBuilder);
18
19 // Set up dependencies
20 $dependencies = require __DIR__ . '/../app/dependencies.php';
21 $dependencies($containerBuilder);
22
23 // Set up repositories
24 $repositories = require __DIR__ . '/../app/repositories.php';
25 $repositories($containerBuilder);
26
27 // Build PHP-DI Container instance
28 $container = $containerBuilder->build();
29
30 // Instantiate the app
31 AppFactory::setContainer($container);
32 $app = AppFactory::create();
33 $callableResolver = $app->getCallableResolver();
34
35 // Register middleware
36 $middleware = require __DIR__ . '/../app/middleware.php';
37 $middleware($app);
38
39 // Register routes
40 $routes = require __DIR__ . '/../app/routes.php';
41 $routes($app);
42
43 /** @var SettingsInterface $settings */
44 $settings = $container->get(SettingsInterface::class);
45
46 $displayErrorDetails = $settings->get('displayErrorDetails');
47 $logError = $settings->get('logError');
48 $logErrorDetails = $settings->get('logErrorDetails');
49
50 // Create Request object from globals
51 $serverRequestCreator = ServerRequestCreatorFactory::create();
52 $request = $serverRequestCreator->createServerRequestFromGlobals();
53
54 // Create Error Handler
```

```
55 $responseFactory = $app->getResponseFactory();
56 $errorHandler = new ErrorHandler($callableResolver, $responseFactory);
57
58 // Create Shutdown Handler
59 $shutdownHandler = new ShutdownHandler($request, $errorHandler, $displayErrorDetails
    );
60 register_shutdown_function($shutdownHandler);
61
62 // Add Routing Middleware
63 $app->addRoutingMiddleware();
64
65 // Add Body Parsing Middleware
66 $app->addBodyParsingMiddleware();
67
68 // Add Error Middleware
69 $errorMiddleware = $app->addErrorMiddleware($displayErrorDetails, $logError,
    $logErrorDetails);
70 $errorMiddleware->setDefaultErrorHandler($errorHandler);
71
72 // Run App & Emit Response
73 $response = $app->handle($request);
74 $responseEmitter = new ResponseEmitter();
75 $responseEmitter->emit($response);
```

12.3 Einführung in Testing

Das Testen von Applikationen gehört zu einer der wichtigsten Aufgaben im Softwareentwicklungsprozess. Ziel ist es, sicherzustellen, dass Webanwendungen funktionieren und dieser Zustand auch dann erhalten bleibt, wenn sie kontinuierlich weiterentwickelt werden. Vor allem, wenn Software veröffentlicht wird, sodass sie von anderen Personen verwendet wird, sollte mehr getan werden als nur zu hoffen, dass alles so funktioniert, wie es zum Zeitpunkt der Entwicklung angedacht war. Software Tests schaffen genau in diesem Punkt Sicherheit.

Tests werden jedoch allzu oft aufgrund von mangelnder Zeit außen vor gelassen. Dieses Argument ist nicht völlig von der Hand zu weisen, denn das Aufsetzen einer Testing-Infrastruktur sowie das Schreiben von sinnvollen Test-Szenarien benötigt in der Tat Zeit. Dies ist in der Regel jedoch gut investiert, da sie später durch das Vermeiden von Fehlern wieder gewonnen wird. Meist wird auch mehr Zeit eingespart, als ohne Tests.

12.3.1 Zeitpunkt für Testing

Das Schreiben von Tests wird oft ans Ende des Entwicklungsprozesses verlagert. Nachdem eine Webapplikation läuft, werden Testszenarien erstellt, um diese zu überprüfen. Eigentlich sollte Testen aber eine Aktivität sein, die bereits vor der eigentlichen Entwicklung begonnen wird. Zunächst werden die Testing Tools (siehe Abschnitt 12.4) installiert und konfiguriert, sodass sie zu Beginn der Entwicklung bereit sind. Während des Entwicklungsprozesses werden dann zeitgleich mit neuen erstellten Klassen auch sofort Test-Klassen geschrieben, die die Funktionalität der neuen Klasse überprüfen.

Selbst wenn die Applikation abgeschlossen ist, werden oft neue Testszenarien noch hinzugefügt, da zum Entwicklungszeitpunkt kaum alle Fälle abgeschätzt werden können.

12.3.2 Was wird wie getestet?

Eine moderne PHP-Applikation besteht aus Klassen, Methoden und auch Funktionen. Getestet werden in der Regel alle öffentlichen Klassen, Methoden und Funktionen, d. h. solche, die mit dem Schlüsselwort `public` versehen sind und somit für den Aufruf durch Dritte erdacht wurden. Dieses Testen von einzelnen Teilen einer Applikation wird *Unit Testing* genannt, als Unit wird ein Teilstück der Applikation (eben eine Klasse, Methode oder Funktion) angesehen. Funktioniert dieses Teilstück einzeln für sich korrekt, kann auch davon ausgegangen werden, dass es im Zusammenspiel mit anderen Teilen in der Applikation funktioniert.

Da dies jedoch nicht immer der Fall ist (eine Applikation ist mehr als die Summe ihrer Teile), werden auch sogenannte *funktionale Tests* durchgeführt, die das Verhalten der Applikation im Gesamten überprüfen.

Moderne Ansätze der Softwareentwicklung rücken Testing stark in den Fokus. Bei *Test Driven Development* (TDD) etwa werden Tests vor dem eigentlichen Applikations-Code erstellt. Die Tests schlagen dabei zunächst fehl und geben somit Auskunft, was von der Applikation erwartet wird. Im Laufe der Implementierung ist es das Ziel, dass die Tests positiv durchgeführt werden und die Applikation die Testszenarien somit erfüllt. Ein weiterer Ansatz ist *Behavior-Driven Development* (BDD). In diesem Ansatz wird das Verhalten der Applikation in sogenannten Stories beschrieben, was einen mehr menschenfreundlicheren Testansatz darstellt. Mehr Informationen dazu finden sich etwa in [38].

12.4 PHPUnit

PHPUnit ist das gängigste Tool, um sogenannte Unit Tests, also das Testen einzelner Teile einer Webapplikation, durchzuführen. Tests, die mit PHPUnit geschrieben werden, werden *Test Cases* genannt, welche wiederum in *Test Suites* zusammengefasst werden. Der *Test Runner* führt eine Test Suite dann aus.

Ein Test Case ist eine Klasse (vom Typ `\PHPUnit\Framework\TestCase`), welche Methoden enthält, die alle mit `test` beginnen und einzelne Aspekte einer Webapplikation testen. Jede dieser Methoden überprüft bestimmte Annahmen, diese können `true` oder `false` sein, wobei natürlich `true` angestrebt wird, da in diesem Fall alle Tests positiv abgeschlossen werden.

Eine Test Suite fasst mehrere Test Cases zusammen. Kleine Applikationen oder Komponenten kommen in der Regel mit einer Test Suite aus, große Applikationen können die Tests mit mehreren Suites aufteilen.

Der Test Runner ist die Art und Weise, wie Tests gestartet werden. Dies geschieht üblicherweise über die Commandline.

12.4.1 PHPUnit vorbereiten

Tests werden üblicherweise in einem Unterverzeichnis `tests` abgelegt. Dieses liegt auf der obersten Ebene der Applikation, genau wie `src`, `vendor` oder `public`.

PHPUnit selbst wird mit Composer installiert. Da es sich jedoch nicht um eine Komponente handelt, die zum Betrieb der eigenen Applikation nötig ist, sondern nur während der Entwicklung gebraucht wird, wird PHPUnit als sogenannte Development-Dependency mit dem `-dev` Flag installiert:

```
1 $ composer require --dev phpunit/phpunit
```

Composer installiert dabei die aktuelle Version (PHPUnit 13¹⁷), die PHP 8.4 oder neuer voraussetzt.

Um die Test Suites zu konfigurieren, wird die Datei `phpunit.xml` im Root des Projektes angelegt. Sie sieht wie folgt aus:

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <phpunit xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3     xsi:noNamespaceSchemaLocation="https://schema.phpunit.de/13.2/phpunit.xsd"
4     bootstrap="tests/bootstrap.php">
5     <testsuites>
6         <testsuite name="favoriteSubject">
7             <directory>tests</directory>
8         </testsuite>
9     </testsuites>
10    <source>
11        <include>
12            <directory>src</directory>
13        </include>
14    </source>
15 </phpunit>
```

Darin wird festgelegt, dass es im Verzeichnis `tests` eine Datei `bootstrap.php` liegt, die bei den Tests ausgeführt werden soll. Darin wird der Composer Autoloader aufgerufen, damit auch für die Tests alle Komponenten vorhanden sind.

Im Anschluss wird eine Test Suite (mit `<testsuite>`) definiert, welche die auszuführenden Tests festlegt. Diese werden im Verzeichnis `tests` angelegt. PHPUnit berücksichtigt dabei standardmäßig alle Dateien, die mit `Test.php` enden.

Um festzulegen, welche Quellcode-Dateien berücksichtigt werden sollen (etwa für eine Code-Coverage), wird dies mit dem `<source>`-Element angegeben, um alle Dateien mit der Endung `.php` im Ordner `src` zu berücksichtigen.

Die Datei `tests/bootstrap.php` besteht nur aus einer Codezeile und bindet den Composer Autoloader ein:

```
1 require dirname(__DIR__) . "/vendor/autoload.php";
```

12.4.2 Die Code-Klasse FavoriteMTDSubject

Getestet werden soll eine Klasse, die das Lieblingsfach in MTD speichert und ausgeben kann und mit einer weiteren Methode auf eine ähnliche Aussage reagieren kann. Sie sieht wie folgt aus:

¹⁷<https://phpunit.de/>

```
1 <?php
2
3 namespace PHPUnitExample;
4
5 use Exception;
6 use InvalidArgumentException;
7
8 class FavoriteMTDSubject
9 {
10     private(set) string $favoriteSubject;
11
12     public function __construct(string $favoriteSubject)
13     {
14         if (mb_trim($favoriteSubject) === "") {
15             throw new InvalidArgumentException("Favorite subject cannot be empty. You must
16             choose!");
17         }
18         $this->favoriteSubject = $favoriteSubject;
19     }
20     public function say(): string
21     {
22         return "The best subject in MTD is $this->favoriteSubject!";
23     }
24
25     public function respondTo(string $input): string
26     {
27         $input = mb_strtolower($input);
28         $myFavoriteSubject = mb_strtolower($this->favoriteSubject);
29
30         if (mb_strpos($input, $myFavoriteSubject) === false) {
31             throw new Exception("Never! $this->favoriteSubject is the best subject in MTD!
32             ");
33         }
34         return "Absolutely true!";
35     }
36 }
```

Im Konstruktor kann als Argument das Lieblingsfach (in der Regel Web Technologies) gesetzt werden. Wird als Argument eine leere Zeichenkette übergeben, wird eine `InvalidArgumentException` geworfen. Die Methode `say()` gibt einen String mit dem Lieblingsfach retour und mittels `respondTo()` kann auf eine andere Ansage zum Thema Lieblingsfach reagiert werden. Stimmt das Fach überein, wird zugestimmt, stimmt es nicht überein, wird eine Exception geworfen.

12.4.3 Die Test-Klasse `FavoriteMTDSubjectTest`

In der Testklasse werden nun vier Tests Cases erstellt, die das Verhalten der öffentlichen Methoden der Code-Klasse überprüfen sollen. Getestet werden das Verhalten des Konstruktors und der `say()`-Methode sowie die zwei möglichen Ergebnisse der `respondTo()`-Methode.

Test 1: Konstruktor

Für den Konstruktor muss getestet werden, dass das übergebene Argument kein leerer String ist. In diesem Fall wird eine `InvalidArgumentException` geworfen, also sieht auch der Unit-Test so aus, dass er dies als erwartetes Verhalten ansieht. Mittels der `expectException()`-Methode von PHPUnit wird signalisiert, dass in den darauffolgenden Zeilen eine `InvalidArgumentException` erwartet wird. Danach wird ein neues Objekt der `FavoriteMTDSubject`-Klasse mit einem leeren String als Parameter angelegt. Wird dabei eine Exception ausgelöst, so gilt der Test als bestanden.

```
1 public function testConstructorThrowsExceptionOnEmptyFavoriteSubject()
2 {
3     $this->expectException(InvalidArgumentException::class);
4     new FavoriteMTDSubject("");
5 }
```

Test 2: say()

Für die Methode `say()` muss getestet werden, ob der String, den diese zurückgibt, den Namen des Lieblingsfachs enthält. Dies kann mit `assertEquals()` erfolgen, das zwei Werte vergleicht:

```
1 public function testSayFavoriteSubject()
2 {
3     $subject = new FavoriteMTDSubject("Web Technologies");
4     $this->assertEquals("The best subject in MTD is Web Technologies!", $subject->say
5         ());
6 }
```

Der erste Wert ist der erwartete String, der zweite ist der tatsächliche Aufruf der Methode am zuvor angelegten Objekt.

Test 3: respondTo() bei Übereinstimmung

Um zu testen, ob `respondTo()` korrekt funktioniert, wird getestet, ob der String „Absolutely true!“ zurückgegeben wird, wenn der Name des hinterlegten Lieblingsfachs mit dem aus einer anderen Aussage übereinstimmt. Wiederum kommt `assertEquals()` zum Einsatz.

```
1 public function testRespondToInAgreement()
2 {
3     $subject = new FavoriteMTDSubject("Web Technologies");
4     $opinion = "Web Technologies is the best!";
5
6     $this->assertEquals("Absolutely true!", $subject->respondTo($opinion));
7 }
```

Zunächst wird ein Objekt angelegt und dann eine weitere Aussage. Diese muss nicht unbedingt komplett identisch mit der, die von `say()` generiert wird sein, da `respondTo()` nur überprüft, ob das Lieblingsfach enthalten ist (nicht, ob der Satz identisch ist). Dann wird `respondTo()` aufgerufen und mit dem erwarteten Rückgabestring verglichen.

Test 4: `respondTo()` bei Uneinigkeit

Schließlich wird noch überprüft, wenn keine Einigkeit über das Lieblingsfach in MTD herrscht. In diesem Fall wirft `respondTo()` eine Exception. Dies wird mit der Methode `expectException()` überprüft. Übergeben wird der Name der erwarteten Exception. Wird eine Exception geworfen, gilt der Test als bestanden.

```
1 public function testRespondToInDisagreement()
2 {
3     $subject = new FavoriteMTDSUBJECT("Web Technologies");
4     $opinion = "I love Algorithmic Thinking!";
5
6     $this->expectException(Exception::class);
7
8     $subject->respondTo($opinion);
9 }
```

12.4.4 Tests aufrufen

Um die Tests aufzurufen, wird die PHPUnit Binary im `vendor`-Verzeichnis in der Kommandozeile mit dem Parameter `-c` und der zuvor erstellten XML-Konfigurationsdatei aufgerufen:

```
1 $ vendor/bin/phpunit -c phpunit.xml
```

Dies startet die vier Tests und zeigt folgende Ausgabe an:

PHPUnit 13.2.1 by Sebastian Bergmann and contributors.

Runtime: PHP 8.5.7

Configuration: /var/www/html/wet2vo-examples/vo12/phpunit/phpunit.xml

.... 4 / 4 (100%)

Time: 00:00.025, Memory: 18.00 MB

OK (4 tests, 4 assertions)

In diesem Fall wurden alle Tests in 25 Millisekunden durchgeführt und bestanden.

Schließlich kann auch noch ein sogenannter Code-Coverage Report erstellt werden, der überprüft, wie viel vom Quellcode mit Unit Tests abgedeckt wurde. Dieser lässt sich wie folgt erzeugen:

```
1 $ vendor/bin/phpunit -c phpunit.xml --coverage-html coverage
```

Abbildung 12.7 zeigt das Ergebnis, das als HTML-Bericht im Ordner `coverage` vorliegt.

Da hierfür im Hintergrund Xdebug verwendet wird, muss dieses als Modul geladen und den Code-Coverage-Modus aktiviert haben.

12.4.5 Weitere Unit Tests

Um weitere konkrete Unit Tests zu sehen und sich daran Beispiele zu nehmen, reicht es, das `tests`-Verzeichnis der meisten Composer-Komponenten zu durchforsten. Diese enthalten in der Regel Unit Tests, die auch selbst ausgeführt werden können.

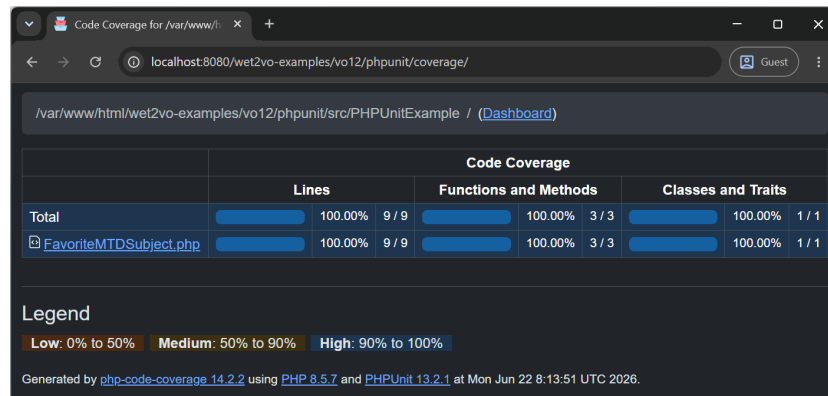


Abbildung 12.7: 100 % des Quellcodes sind durch Unit Tests abgedeckt.

12.4.6 Funktionale Tests

Unit Tests prüfen einzelne Bausteine isoliert. Um sicherzustellen, dass die Applikation auch im Zusammenspiel korrekt arbeitet, kommen *funktionale Tests* (auch Integrations- oder HTTP-Tests genannt) zum Einsatz. Bei einer Slim-Applikation lässt sich dies elegant umsetzen, da das App-Objekt selbst einen Request entgegennimmt und eine Response zurückliefert, ohne dass ein echter Webserver laufen muss.

Das Prinzip ist einfach: Die Applikation wird wie im Produktivbetrieb aufgebaut (Container, Routen, Middleware), anschließend wird ein `ServerRequest`-Objekt erzeugt und über `$app->handle($request)` verarbeitet. Das Ergebnis ist ein Response-Objekt, dessen Status-Code, Header oder Inhalt überprüft werden können.

```

1 <?php
2
3 namespace SlimExampleTest;
4
5 use PHPUnit\Framework\TestCase;
6 use Slim\Factory\AppFactory;
7 use Slim\Psr7\Factory\ServerRequestFactory;
8
9 final class HomePageTest extends TestCase
10 {
11     public function testHomePageShowsForm(): void
12     {
13         $app = AppFactory::create();
14         // Routen und Middleware wie in der Applikation registrieren ...
15
16         $request = (new ServerRequestFactory())->createServerRequest("GET", "/");
17         $response = $app->handle($request);
18
19         $this->assertSame(200, $response->getStatusCode());
20         $this->assertStringContainsString("Tell Me Your Name", (string) $response->
            getBody());
21     }
22 }
```

Damit Test und Produktivcode dieselbe Konfiguration verwenden, wird das Aufset-

zen der Applikation in der Praxis in eine gemeinsame Datei (etwa `app▸bootstrap.php`) ausgelagert, die sowohl von `public▸index.php` als auch von den Tests eingebunden wird. So wird die Applikation nur an einer Stelle definiert und im Test exakt so geprüft, wie sie später ausgeführt wird.

12.4.7 Test-Alternative: Pest

Mit *Pest*¹⁸ hat sich in den letzten Jahren eine populäre Alternative zu PHPUnit etabliert. Das Ziel von Pest ist es, Tests leserlicher und einfacher zu schreiben. Das Framework baut intern auf PHPUnit auf (die aktuelle Version Pest 4.7.3 nutzt PHPUnit 12) und benötigt mindestens PHP 8.3. Bestehende PHPUnit-Tests können weiterhin parallel ausgeführt werden. Pest lässt sich mit wenigen Schritten installieren und starten:

```
1 $ composer require pestphp/pest --dev --with-all-dependencies
2 $ ./vendor/bin/pest --init
3 $ ./vendor/bin/pest
```

Diese drei Zeilen installieren, initialisieren und starten Pest mit einem vorkonfigurierten Beispielttest.

Funktions-basierte Syntax

Anstelle von Testklassen mit Methoden setzt Pest auf einzelne, mit `test()` (oder dem Alias `it()`) definierte Funktionen. Der folgende Test verdeutlicht die Grundstruktur:

```
1 test("example", function () {
2     expect(true)->toBeTrue();
3 });
```

Mit der `expect()`-Anweisung wird ein Wert übergeben, der untersucht werden soll. Daran werden *Expectations* wie `toBeTrue()` oder `toBe()` angehängt, die überprüfen, ob der Wert dem erwarteten entspricht. Diese flüssig lesbare Schreibweise (*fluent API*) ist das Kernmerkmal von Pest.

Die Unit Tests in Pest

Die zuvor mit PHPUnit erstellten Tests der Klasse `FavoriteMTDSubject` lassen sich in Pest deutlich kompakter formulieren. Die `say()`-Methode etwa:

```
1 test("say returns the favorite subject", function () {
2     subject = new FavoriteMTDSubject("Web Technologies");
3     expect($subject->say())->toBe("The best subject in MTD is Web Technologies!");
4 });
```

Für das Testen von Exceptions bietet Pest die angehängte Methode `throws()`, was den Test gegenüber PHPUnit weiter verkürzt:

```
1 test("constructor rejects an empty subject", function () {
2     new FavoriteMTDSubject("");
3 })->throws(InvalidArgumentException::class);
```

¹⁸<https://pestphp.com/>

Darüber hinaus bietet Pest fortgeschrittene Funktionen wie *Datasets* (das Ausführen desselben Tests mit verschiedenen Eingabewerten), *Architecture Testing* (das Überprüfen von Architekturregeln) sowie seit Pest 4 auch *Browser Testing* für End-to-End-Tests im Browser. Weitere Beispiele liefert die Pest-Dokumentation¹⁹.

12.5 Weiterführende Literatur

Literatur zu Microframeworks ist eher spärlich vorhanden. In der Regel sind die offiziellen Dokumentationen der beste Ausgangspunkt, es gibt aber mit [31–33] drei eBooks zum Slim Framework 4, die von Daniel Opitz, einem der Core-Entwickler des Frameworks stammen. Wer eine Slim-Anwendung entwickelt, findet hier praktische Snippets und Artikel für die verschiedensten Erweiterungen und Szenarien.

Das Thema Unit Tests bzw. Testing ist sehr gut in [38] beschrieben. Die ausführlichste Ressource ist jedoch mit Sicherheit [4], wenngleich sie bereits in die Jahre gekommen ist und nicht mehr die neuesten Versionen abdeckt. Da das Buch jedoch vom Autor von PHPUnit verfasst ist, ist es eine gute Ressource im Umgang mit Testing und Test-Driven Development. Für aktuelle Versionen empfiehlt sich ergänzend das stets gepflegte Online-Handbuch von PHPUnit²⁰.

Das modernste Buch, das Testen mit PHP sehr holistisch abdeckt ist jedoch sicherlich [15]. Es ist sehr leserlich geschrieben, der Autor ist sehr aktiv in der PHP-Community und beantwortet auf Social Media auch gerne Fragen. Mit [14] existiert auch noch eine fortgeschrittene Version, die spezifisch auf Test-Tools eingeht.

¹⁹ <https://pestphp.com/docs/>

²⁰ <https://docs.phpunit.de/>

Quellenverzeichnis

- [1] Article 19. *How The Internet Really Works. An Illustrated Guide to Protocols, Privacy, Censorship, and Governance*. San Francisco, CA: No Starch Press, 11. Dez. 2020 (siehe S. 1-24).
- [2] Philip Ackermann. *JavaScript. Das umfassende Handbuch*. 4. Aufl. Bonn, Deutschland: Rheinwerk Computing, 6. Nov. 2025 (siehe S. 11-31).
- [3] Nils Adermann und Jordi Boggiano. *The composer.json schema - Composer*. 30. Okt. 2025. URL: <https://getcomposer.org/doc/04-schema.md> (siehe S. 5-11).
- [4] Sebastian Bergmann. *PHPUnit Pocket Guide. Test-Driven Development in PHP*. Sebastopol, USA: O'Reilly, 8. Nov. 2005 (siehe S. 12-28).
- [5] Tim Berners-Lee, Roy Fielding und Larry Masinter. *Uniform Resource Identifier (URI): Generic Syntax*. RFC 3986. IETF, Jan. 2005, S. 1–61. DOI: 10.17487/RFC3986 (siehe S. 1-3).
- [6] Tim Berners-Lee, Larry Masinter und Mark McCahill. *Uniform Resource Locators (URL)*. RFC 1738. IETF, Dez. 1994, S. 1–25. DOI: 10.17487/RFC1738 (siehe S. 1-3).
- [7] Roy T. Fielding, Mark Nottingham und Julian Reschke. *HTTP Semantics*. RFC 9110. IETF, Juni 2022, S. 1–194. DOI: 10.17487/RFC9110 (siehe S. 1-24, 11-3, 11-4).
- [8] Roy T. Fielding, Mark Nottingham und Julian Reschke. *HTTP/1.1*. RFC 9112. IETF, Juni 2022, S. 1–46. DOI: 10.17487/RFC9112 (siehe S. 1-24).
- [9] Martin Fowler. *Patterns of Enterprise Application Architecture*. Boston, USA: Addison Wesley, 1. Nov. 2002 (siehe S. 6-9).
- [10] Jeffrey E. F. Friedl. *Mastering Regular Expressions. Understand Your Data and Be More Productive*. 3. Aufl. Sebastopol, USA: O'Reilly, 1. Sep. 2006 (siehe S. 3-30).
- [11] Nicholas J. Giannini. „Vulnerable Web Application Framework“. Masterarbeit. Kingston, RI, USA: University of Rhode Island, Jan. 2015. URL: <https://digitalcommons.uri.edu/theses/629/> (siehe S. 12-2).
- [12] Jan Goyvaerts und Steven Levithan. *Regular Expressions Cookbook. Detailed Solutions in Eight Programming Languages*. 2. Aufl. Sebastopol, USA: O'Reilly, 27. Aug. 2012 (siehe S. 3-30).

- [13] David Grudl. *Latte Documentation*. Version 3.0. 27. Apr. 2026. URL: <https://latte.nette.org/en/guide> (siehe S. 6-8, 6-21).
- [14] Chris Hartjes. *The Complementary PHP Testing Tools Cookbook*. php[architect], 9. Nov. 2021 (siehe S. 12-28).
- [15] Chris Hartjes. *The Grumpy Programmer's Guide To Testing PHP Applications*. php[architect], 26. Mai 2020 (siehe S. 12-28).
- [16] Anthony T. Holdener III. *Ajax. The Definitive Guide*. Sebastopol, USA: O'Reilly, 14. Feb. 2008 (siehe S. 11-31).
- [17] *imagefilledarc*. 8. Juni 2026. URL: <https://www.php.net/manual/de/function.imagefilledarc.php> (besucht am 08.06.2026) (siehe S. 10-22).
- [18] International Organization for Standardization. *Information technology – Open Systems Interconnection – Basic Reference Model: The Basic Model*. ISO/IEC 7498-1:1994. 15. Nov. 1994. URL: <https://www.iso.org/standard/20269.html> (siehe S. 1-7).
- [19] Richard Ishida und Susan K. Miller. *Lokalisierung vs. Internationalisierung*. Übers. von Jens Oliver Meiert. 2. Nov. 2018. URL: <https://www.w3.org/International/questions/qa-i18n.de.html> (besucht am 31.05.2026) (siehe S. 9-7).
- [20] Kai Jäger. *Ajax in der Praxis. Grundlagen, Konzepte, Lösungen*. Heidelberg, Deutschland: Springer, 1. Dez. 2007 (siehe S. 11-31).
- [21] Paul M. Jones. *A History Of PHP Frameworks/Library Collections*. 15. Dez. 2022. URL: <https://github.com/pmjones/php-history> (besucht am 03.05.2026) (siehe S. 5-1).
- [22] Günter Knauf. *PHP GD Clock*. 20. Sep. 2004. URL: <https://www.codelibrary.com/snippet/655/gd-clock> (besucht am 08.06.2026) (siehe S. 10-22).
- [23] James F. Kurose und Keith W. Ross. *Computer Networking. A Top-Down Approach*. 8. Aufl. Harlow, Großbritannien: Pearson, 10. Juni 2021 (siehe S. 1-24).
- [24] Arnaud Lauret. *The Design of Web APIs*. 2. Aufl. Shelter Island, USA: Manning Publications, 22. Juli 2025 (siehe S. 11-31).
- [25] Josh Lockhart und Phil Sturgeon. *PHP: The Right Way*. 22. Jan. 2026. URL: <https://phprightway.com/> (besucht am 14.04.2026) (siehe S. 2-30, 5-21).
- [26] Josh Lockhart u. a. *Slim Middleware*. 9. Juli 2024. URL: <https://www.slimframework.com/docs/v4/concepts/middleware.html> (siehe S. 12-14).
- [27] Eric A. Mann. *PHP Cookbook. Modern Code Solutions for Professional Developers*. Sebastopol, USA: O'Reilly, 20. Juni 2023 (siehe S. 2-30, 3-30).
- [28] Mozilla Developer Network. *HTTP request methods*. 4. Juli 2025. URL: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Methods> (besucht am 10.05.2026) (siehe S. 6-10).
- [29] Mozilla Developer Network. *HTTP: Hypertext Transfer Protocol*. 22. Dez. 2025. URL: <https://developer.mozilla.org/en-US/docs/Web/HTTP> (besucht am 07.04.2026) (siehe S. 1-24).

- [30] Robin Nixon. *Learning PHP, MySQL & JavaScript. A Step-by-Step Guide to Creating Dynamic Websites*. 7. Aufl. Sebastopol, USA: O'Reilly, 18. Feb. 2025 (siehe S. 2-30).
- [31] Daniel Opitz. *Slim 4 – eBook*. Bd. 1. Eigenveröffentlichung, 12. Aug. 2023. URL: <https://ko-fi.com/dopitz/shop> (siehe S. 12-28).
- [32] Daniel Opitz. *Slim 4 – eBook*. Bd. 2. Eigenveröffentlichung, 12. Aug. 2023. URL: <https://ko-fi.com/dopitz/shop> (siehe S. 12-28).
- [33] Daniel Opitz. *Slim 4 – eBook*. Bd. 3. Eigenveröffentlichung, 12. Aug. 2023. URL: <https://ko-fi.com/dopitz/shop> (siehe S. 12-28).
- [34] Packagist. *Install Statistics*. 3. Mai 2026. URL: <https://packagist.org/statistics> (besucht am 03.05.2026) (siehe S. 5-5).
- [35] Addison Phillips und Mark Davis. *Tags for Identifying Languages*. RFC 5646. IETF, Sep. 2009, S. 1–83. DOI: 10.17487/RFC5646 (siehe S. 9-7).
- [36] PHP Framework Interoperability Group. *PSR-18: HTTP Client*. Hrsg. von Tobias Nyholm. Version 1.0.3. 23. Sep. 2023. URL: <https://www.php-fig.org/psr/psr-18/> (siehe S. 11-28).
- [37] Christopher Pitt. *Pro PHP 8 MVC. Model View Controller Architecture-Driven Application Development*. 2. Aufl. New York, USA: Apress, 27. Mai 2021 (siehe S. 6-21).
- [38] Rainier Sarabia. *Test-Driven Development with PHP 8. Build extensible, reliable, and maintainable enterprise-level applications using TDD and BDD with PHP*. Birmingham, Großbritannien: Packt Publishing, 13. Jan. 2023 (siehe S. 12-21, 12-28).
- [39] Tony Stubblebine. *Regular Expression. Pocket Reference*. 2. Aufl. Sebastopol, USA: O'Reilly, 1. Aug. 2007 (siehe S. 3-30).
- [40] Andrew S. Tanenbaum, Nick Feamster und David J. Wetherall. *Computernetzwerke*. 6. Aufl. Hallbergmoos, Deutschland: Pearson Studium, 24. Juni 2024 (siehe S. 1-24).
- [41] Kevin Tatroe und Peter MacIntyre. *Programming PHP. Creating Dynamic Web Pages*. 4. Aufl. Sebastopol, USA: O'Reilly, 31. März 2020 (siehe S. 2-30, 3-29, 4-28, 7-19, 8-19, 10-37, 11-31).
- [42] The PHP Documentation Group. *PHP-Handbuch*. 8. Juni 2026. URL: <https://www.php.net/manual/de/> (siehe S. 2-13, 2-30, 7-19, 9-26, 10-37).
- [43] Thomas Theis. *Einstieg in JavaScript*. 5. Aufl. Bonn, Deutschland: Rheinwerk Computing, 4. Juli 2024 (siehe S. 11-31).
- [44] Thomas Theis. *Einstieg in PHP 8 und MySQL*. 15. Aufl. Bonn, Deutschland: Rheinwerk Computing, 5. Apr. 2023 (siehe S. 2-30, 3-29, 4-28, 7-19, 8-19, 9-26, 10-37).
- [45] W3Techs. *Usage statistics and market shares of web servers*. 6. Apr. 2026. URL: https://w3techs.com/technologies/overview/web_server (besucht am 06.04.2026) (siehe S. 1-6).

- [46] W3Techs. *Usage statistics of server-side programming languages for websites*. 13. Apr. 2026. URL: https://w3techs.com/technologies/overview/programming_language (besucht am 14.04.2026) (siehe S. 2-5).
- [47] Web Hypertext Application Technology Working Group. *URL*. Living Standard. 31. März 2026. URL: <https://url.spec.whatwg.org/> (siehe S. 1-3).
- [48] Web Hypertext Application Technology Working Group. *XMLHttpRequest*. Living Standard. 11. März 2025. URL: <https://xhr.spec.whatwg.org/> (siehe S. 11-28).
- [49] Christian Wenz und Tobias Hauser. *PHP 8 und MySQL. Das umfassende Handbuch*. 5. Aufl. Bonn, Deutschland: Rheinwerk Computing, 3. Mai 2024 (siehe S. 2-30, 3-29, 4-28, 5-21, 7-19, 8-19, 9-26, 10-37, 11-31).
- [50] Jamie York. *Awesome PHP. A curated list of amazingly awesome PHP libraries, resources and shiny things*. 23. Apr. 2026. URL: <https://github.com/ziadoz/awesome-php> (besucht am 02.05.2026) (siehe S. 5-5).
- [51] Matt Zandstra. *PHP 8 Objects, Patterns, and Practice: Volume 1. Mastering OO Enhancements and Design Patterns*. 7. Aufl. New York, USA: Apress, 2. Dez. 2024 (siehe S. 4-29, 6-21).
- [52] Matt Zandstra. *PHP 8 Objects, Patterns, and Practice: Volume 2. Mastering Essential Development Tools*. 7. Aufl. New York, USA: Apress, 22. Juli 2025 (siehe S. 5-21).
- [53] Harald Zisler. *Computer-Netzwerke. Grundlagen, Funktionsweise, Anwendung*. 8. Aufl. Bonn, Deutschland: Rheinwerk Computing, 6. März 2025 (siehe S. 1-24).